MobilityDB 1.4 User’s Manual

Esteban Zimányi

COLLABORATORS

	TITLE : MobilityDB 1.4 User's Manual		
ACTION	*NAME*	*DATE*	*SIGNATURE*
WRITTEN BY	Esteban Zimányi	November 9, 2026	

REVISION HISTORY

NUMBER	DATE	DESCRIPTION	NAME

Contents

1 Introduction 1
1.1 Project Steering Committee 1
1.2 Other Code Contributors 2
1.3 Sponsors 2
1.4 Licenses 2
1.5 Installation from Sources 2
1.5.1 Short Version 2
1.5.2 Get the Sources 3
1.5.3 Enabling the Database 4
1.5.4 Dependencies 4
1.5.5 Configuring 5
1.5.6 Build and Install 5
1.5.7 Testing 5
1.5.8 Documentation 6
1.6 Installation from Binaries 7
1.6.1 Debian-based Linux Distributions 7
1.6.2 Windows 7
1.7 Support 7
1.7.1 Reporting Problems 7
1.7.2 Mailing Lists 7
1.8 Migrating from Version 1.2 to Version 1.3 8
1.9 Migrating from Version 1.3 to Version 1.4 8

2 Set and Span Types 10
2.1 Input and Output 11
2.2 Constructors 15
2.3 Conversions 16
2.4 Accessors 18
2.5 Transformations 22
2.6 Spatial Reference System 25

2.7 Set Operations . . . 26
2.8 Bounding Box Operations . . . 27
    2.8.1 Topological Operations . . . 27
    2.8.2 Position Operations . . . 28
    2.8.3 Splitting Operations . . . 29
2.9 Distance Operations . . . 30
2.10 Comparisons . . . 31
2.11 Aggregations . . . 32
2.12 Indexing . . . 33

3 Bounding Box Types **34**
3.1 Input and Output . . . 34
3.2 Constructors . . . 37
3.3 Conversions . . . 38
3.4 Accessors . . . 39
3.5 Transformations . . . 41
3.6 Spatial Reference System . . . 43
3.7 Splitting Operations . . . 44
3.8 Set Operations . . . 44
3.9 Bounding Box Operations . . . 45
    3.9.1 Topological Operations . . . 45
    3.9.2 Position Operations . . . 46
3.10 Comparisons . . . 48
3.11 Aggregations . . . 49
3.12 Indexing . . . 49

4 Temporal Types (Part 1) **50**
4.1 Introduction . . . 50
4.2 Examples of Temporal Types . . . 53
4.3 Validity of Temporal Types . . . 54
4.4 Temporalizing Operations . . . 55
4.5 Notation . . . 55
4.6 Input and Output . . . 56
4.7 Constructors . . . 61
4.8 Conversions . . . 63
4.9 Accessors . . . 63
4.10 Transformations . . . 67

5 Temporal Types (Part 2) 70
5.1 Modifications . . . 70
5.2 Restrictions . . . 75
5.3 Bounding Box Operators . . . 77
5.4 Comparisons . . . 78
5.4.1 Traditional Comparisons . . . 78
5.4.2 Ever and Always Comparisons . . . 79
5.4.3 Temporal Comparisons . . . 80
5.5 Miscellaneous . . . 81

6 Temporal Alphanumeric Types 82
6.1 Notation . . . 82
6.2 Conversions . . . 82
6.3 Accessors . . . 83
6.4 Transformations . . . 84
6.5 Restrictions . . . 85
6.6 Boolean Operations . . . 86
6.7 Mathematical Operations . . . 86
6.8 Text Operations . . . 90

7 Temporal Geometry Types (Part 1) 91
7.1 Spatiotemporal Types . . . 91
7.2 Functions on Geometries . . . 93
7.3 Notation . . . 99
7.4 Input and Output . . . 100
7.5 Conversions . . . 102
7.6 Accessors . . . 103
7.7 Transformations . . . 107

8 Temporal Geometry Types (Part 2) 113
8.1 Restrictions . . . 113
8.2 Spatial Reference System . . . 115
8.3 Operands of Different Dimensions . . . 116
8.4 Empty Geometries . . . 117
8.5 Bounding Box Operations . . . 117
8.6 Distance Operations . . . 117
8.7 Planar and Geodetic Computations . . . 121
8.8 Ever and Always Relationships . . . 122
8.9 Spatiotemporal Relationships . . . 124

9 Temporal Types: Analytics Operations 126
9.1 Simplification . . . 126
9.2 Reduction . . . 127
9.3 Similarity . . . 129
9.4 Extended Kalman Filter . . . 132
9.5 Reachability Filter . . . 132
9.6 Splitting Operations . . . 133
9.7 Multidimensional Tiling . . . 136
9.7.1 Bin Operations . . . 137
9.7.2 Tile Operations . . . 138
9.7.3 Bounding Box Operations . . . 141
9.7.4 Splitting Operations . . . 143

10 Temporal Types: Aggregation and Indexing 146
10.1 Aggregation . . . 146
10.2 Indexing . . . 148
10.3 Statistics and Selectivity . . . 149
10.3.1 Statistics Collection . . . 149
10.3.2 Selectivity Estimation . . . 151

11 Temporal Network Points 152
11.1 Static Network Types . . . 152
11.1.1 Input and Output . . . 153
11.1.2 Constructors . . . 155
11.1.3 Conversions . . . 155
11.1.4 Accessors . . . 156
11.1.5 Transformations . . . 157
11.1.6 Spatial Operations . . . 157
11.1.7 Comparisons . . . 158
11.2 Temporal Network Points . . . 158
11.3 Validity of Temporal Network Points . . . 159
11.4 Constructors . . . 159
11.5 Conversions . . . 160
11.6 MF-JSON Input and Output . . . 160
11.7 Accessors . . . 161
11.8 Transformations . . . 162
11.9 Modifications . . . 162
11.10 Restrictions . . . 163
11.11 Spatial Operations . . . 163

11.12 Bounding Box Operations . . . 164
11.13 Distance Operations . . . 165
11.14 Ever and Always Relationships . . . 166
11.15 Spatiotemporal Relationships . . . 166
11.16 Comparisons . . . 167
11.17 Aggregations . . . 168
11.18 Indexing . . . 168
11.19 Aggregation . . . 169

12 Temporal Circular Buffers 170

12.1 Static Circular Buffers . . . 171
    12.1.1 Input and Output . . . 171
    12.1.2 Constructors . . . 172
    12.1.3 Conversions . . . 172
    12.1.4 Accessors . . . 173
    12.1.5 Transformations . . . 173
    12.1.6 Spatial Operations . . . 173
    12.1.7 Distance Operations . . . 174
    12.1.8 Comparisons . . . 175
12.2 Temporal Circular Buffers . . . 175
12.3 Validity of Temporal Circular Buffers . . . 176
12.4 Input and Output . . . 176
12.5 Constructors . . . 178
12.6 Conversions . . . 179
12.7 Accessors . . . 179
12.8 Transformations . . . 180
12.9 Modifications . . . 182
12.10 Restrictions . . . 182
12.11 Spatial Operations . . . 183
12.12 Bounding Box Operations . . . 184
12.13 Distance Operations . . . 185
12.14 Ever and Always Relationships . . . 186
12.15 Spatiotemporal Relationships . . . 187
12.16 Comparisons . . . 187
12.17 Aggregations . . . 188
12.18 Simplification . . . 189
12.19 Indexing . . . 190

13 Temporal Poses and Pose Chains 191
13.1 Notation . . . 192
13.2 Static Poses and Pose Chains . . . 192
13.2.1 Input and Output . . . 193
13.2.2 Constructors . . . 194
13.2.3 Conversions . . . 195
13.2.4 Accessors . . . 195
13.2.5 Transformations . . . 196
13.2.6 Spatial Reference System . . . 196
13.2.7 Distance Operations . . . 197
13.2.8 Comparisons . . . 198
13.3 Pose Chain Composition . . . 198
13.4 Temporal Poses and Pose Chains . . . 200
13.5 Validity of Temporal Poses and Pose Chains . . . 201
13.6 Input and Output . . . 201
13.7 Constructors . . . 203
13.8 Conversions . . . 205
13.9 Accessors . . . 205
13.10 Transformations . . . 207
13.11 Modifications . . . 209
13.12 Restrictions . . . 210
13.13 Spatial Reference System . . . 211
13.14 Bounding Box Operations . . . 211
13.15 Distance Operations . . . 212
13.16 Ever and Always Relationships . . . 214
13.17 Spatiotemporal Relationships . . . 215
13.18 Comparisons . . . 215
13.19 Aggregations . . . 216
13.20 Spatial Operations . . . 217
13.21 Simplification . . . 218
13.22 Indexing . . . 218
13.23 OGC GeoPose v1.0 Support . . . 219
13.23.1 JSON Input and Output . . . 219
13.23.2 Quaternion Renormalization . . . 219
13.23.3 Euler-Angle Accessors . . . 220
13.23.4 Frame Metadata Registry . . . 221
13.23.5 Body and World Rigid Transform . . . 222
13.23.6 Temporal JSON I/O . . . 223
13.23.7 Composite Chain and Graph Documents . . . 225

14 Temporal Rigid Geometries 227
14.1 Input and Output . . . 227
14.2 Constructors . . . 229
14.3 Conversions . . . 229
14.4 Accessors . . . 229
14.5 Traversed Area . . . 231
14.6 Spatial Functions . . . 231
14.7 Motion Metrics . . . 232
14.8 Transformations . . . 233
14.9 Modifications . . . 235
14.10 Restrictions . . . 236
14.11 Distance Operations . . . 237
14.12 Similarity . . . 239
14.13 Bounding Box Operations . . . 239
14.14 Ever and Always Relationships . . . 239
14.15 Spatiotemporal Relationships . . . 240
14.16 Comparisons . . . 241
14.17 Tiles . . . 242
14.18 Bounding Boxes . . . 242
14.19 Aggregations . . . 243
14.20 Indexing . . . 243

15 JSON Types in MobilityDB 244
15.1 JSONB Operations . . . 244
15.2 JSONB Set Operations . . . 248
15.3 Temporal JSONB values . . . 253
15.4 Validity of temporal JSONB values . . . 254
15.5 Input and Output . . . 255
15.6 Constructors . . . 256
15.7 Conversions . . . 257
15.8 Accessors . . . 258
15.9 Transformations . . . 258
15.10 Temporal JSON Operations . . . 259
15.11 Restrictions . . . 266
15.12 Bounding Box Operations . . . 266
15.13 Comparisons . . . 267
15.14 Aggregations . . . 267
15.15 Indexing . . . 268

16 Temporal Tessellations 269
16.1 Notation . . . 271
16.2 Input and Output . . . 271
16.3 Constructors . . . 275
16.4 Conversions . . . 277
16.5 Accessors . . . 279
16.6 Transformations . . . 283
16.7 Modifications . . . 284
16.8 Restrictions . . . 285
16.9 Static Geometry Coverage . . . 286
16.10 Inspection . . . 287
16.11 Hierarchy . . . 288
16.12 Latitude and Longitude Conversions . . . 289
16.13 Bounding Box Operations . . . 292
16.14 Metric Functions . . . 294
16.15 Set-Returning Functions . . . 294
16.16 Ever and Always Relationships . . . 296
16.17 Spatiotemporal Relationships . . . 297
16.18 Comparisons . . . 297
16.19 Aggregations . . . 299
16.20 Indexing . . . 300
16.21 H3-Specific Operations . . . 300
16.22 QUADBIN-Specific Operations . . . 304
16.23 S2-Specific Operations . . . 306
16.24 Raster-Specific Operations . . . 308

17 Temporal Point Clouds 321
17.1 Point Cloud Schemas . . . 321
17.2 Static Point Clouds . . . 322
    17.2.1 Input and Output . . . 322
    17.2.2 Constructors . . . 323
    17.2.3 Accessors . . . 324
17.3 Point Cloud Sets . . . 325
    17.3.1 Constructors . . . 325
17.4 Point Cloud Bounding Boxes . . . 325
    17.4.1 Input and Output . . . 326
    17.4.2 Constructors . . . 327
    17.4.3 Conversions . . . 328
    17.4.4 Accessors . . . 328

- 17.4.5 Transformations . . . 329
- 17.4.6 Set Operations . . . 329
- 17.4.7 Topological Operations . . . 329
- 17.4.8 Position Operations . . . 330
- 17.4.9 Comparisons . . . 330

17.5 Temporal Point Cloud Points . . . 331
- 17.5.1 Input and Output . . . 331
- 17.5.2 Constructors . . . 332
- 17.5.3 Conversions . . . 333
- 17.5.4 Accessors . . . 333
- 17.5.5 Transformations . . . 334
- 17.5.6 Modifications . . . 335
- 17.5.7 Restrictions . . . 335
- 17.5.8 Splitting Operations . . . 337
- 17.5.9 Bounding Box Operations . . . 337
- 17.5.10 Distance Operations . . . 338
- 17.5.11 Ever and Always Relationships . . . 339
- 17.5.12 Spatiotemporal Relationships . . . 340
- 17.5.13 Comparisons . . . 341
- 17.5.14 Simplification . . . 341

17.6 Temporal Point Cloud Patches . . . 342
- 17.6.1 Input and Output . . . 342
- 17.6.2 Constructors . . . 343
- 17.6.3 Conversions . . . 344
- 17.6.4 Accessors . . . 345
- 17.6.5 Transformations . . . 345
- 17.6.6 Modifications . . . 346
- 17.6.7 Restrictions . . . 347
- 17.6.8 Splitting Operations . . . 348
- 17.6.9 Spatial Operations . . . 349
- 17.6.10 Bounding Box Operations . . . 350
- 17.6.11 Distance Operations . . . 350
- 17.6.12 Ever and Always Relationships . . . 350
- 17.6.13 Spatiotemporal Relationships . . . 351
- 17.6.14 Comparisons . . . 351

17.7 Indexing . . . 352

17.8 Aggregations . . . 352

18 Writing Portable Queries 354
18.1 Set Operations . . . 354
18.2 Bounding Box Operations . . . 354
18.2.1 Topological Operations . . . 355
18.2.2 Position Operations . . . 355
18.3 Distance Operations . . . 356
18.4 Comparisons . . . 356
18.4.1 Traditional Comparisons . . . 356
18.4.2 Ever and Always Comparisons . . . 356
18.4.3 Temporal Comparisons . . . 357
18.5 Boolean Operations . . . 357
18.6 Mathematical Operations . . . 357
18.7 Temporal JSON Operations . . . 357
18.8 Base Values Provided by the Host . . . 358
18.9 Aggregations . . . 359
18.10 Running a Query on Apache Spark and Apache Flink . . . 360
18.10.1 Names the Engines Own . . . 360
18.10.2 Reading and Writing Values . . . 362
18.10.3 Functions Returning Rows . . . 362
18.10.4 Example . . . 362
18.11 The Rasters of the Engines . . . 363
18.11.1 DuckDB and Its Raster Extension . . . 363
18.11.2 Spark and Apache Sedona . . . 364

A MobilityDB Reference 366
A.1 MobilityDB Types . . . 366
A.2 Set and Span Types . . . 366
A.2.1 Input and Output . . . 366
A.2.2 Constructors . . . 367
A.2.3 Conversions . . . 367
A.2.4 Accessors . . . 367
A.2.5 Transformations . . . 368
A.2.6 Spatial Reference System . . . 368
A.2.7 Set Operations . . . 368
A.2.8 Bounding Box Operations . . . 368
A.2.8.1 Topological Operations . . . 368
A.2.8.2 Position Operations . . . 369
A.2.8.3 Splitting Operations . . . 369
A.2.9 Distance Operations . . . 369

A.2.10 Comparisons . . . 369
A.2.11 Aggregations . . . 369
A.3 Box Types . . . 369
A.3.1 Input and Output . . . 369
A.3.2 Constructors . . . 370
A.3.3 Conversions . . . 370
A.3.4 Accessors . . . 370
A.3.5 Transformations . . . 370
A.3.6 Spatial Reference System . . . 370
A.3.7 Splitting Operations . . . 370
A.3.8 Set Operations . . . 370
A.3.9 Bounding Box Operations . . . 371
A.3.9.1 Topological Operations . . . 371
A.3.9.2 Position Operations . . . 371
A.3.10 Comparisons . . . 371
A.3.11 Aggregations . . . 371
A.4 Temporal Types . . . 371
A.4.1 Input and Output . . . 371
A.4.2 Constructors . . . 371
A.4.3 Conversions . . . 372
A.4.4 Accessors . . . 372
A.4.5 Transformations . . . 372
A.4.6 Modifications . . . 373
A.4.7 Restrictions . . . 373
A.4.8 Comparisons . . . 373
A.4.8.1 Traditional Comparisons . . . 373
A.4.8.2 Ever and Always Comparisons . . . 373
A.4.8.3 Temporal Comparisons . . . 373
A.4.9 Miscellaneous . . . 373
A.5 Temporal Alphanumeric Types . . . 373
A.5.1 Conversions . . . 373
A.5.2 Accessors . . . 374
A.5.3 Transformations . . . 374
A.5.4 Restrictions . . . 374
A.5.5 Boolean Operations . . . 374
A.5.6 Mathematical Operations . . . 374
A.5.7 Text Operations . . . 375
A.6 Temporal Geometry Types . . . 375
A.6.1 Functions on Geometries . . . 375

A.6.2 Input and Output . . . 376
A.6.3 Conversions . . . 376
A.6.4 Accessors . . . 376
A.6.5 Transformations . . . 377
A.6.6 Restrictions . . . 377
A.6.7 Spatial Reference System . . . 377
A.6.8 Bounding Box Operations . . . 377
A.6.9 Distance Operations . . . 378
A.6.10 Ever and Always Relationships . . . 378
A.6.11 Spatiotemporal Relationships . . . 378
A.7 Operations for Temporal Types: Analytics Operations . . . 378
A.7.1 Simplification . . . 378
A.7.2 Reduction . . . 379
A.7.3 Similarity . . . 379
A.7.4 Extended Kalman Filter . . . 379
A.7.5 Reachability Filter . . . 379
A.7.6 Splitting Operations . . . 379
A.7.7 Multidimensional Tiling . . . 379
A.7.7.1 Bin Operations . . . 379
A.7.7.2 Tile Operations . . . 380
A.7.7.3 Bounding Box Operations . . . 380
A.7.7.4 Splitting Operations . . . 380
A.7.8 Aggregation . . . 380
A.8 Temporal Poses and Pose Chains . . . 380
A.8.1 Static Poses and Pose Chains . . . 380
A.8.1.1 Input and Output . . . 380
A.8.1.2 Constructors . . . 381
A.8.1.3 Conversions . . . 381
A.8.1.4 Accessors . . . 381
A.8.1.5 Transformations . . . 381
A.8.1.6 Spatial Reference System . . . 381
A.8.1.7 Distance Operations . . . 381
A.8.1.8 Comparisons . . . 381
A.8.2 Pose Chain Composition . . . 382
A.8.3 Temporal Poses and Pose Chains . . . 382
A.8.3.1 Input and Output . . . 382
A.8.3.2 Constructors . . . 382
A.8.3.3 Conversions . . . 382
A.8.3.4 Accessors . . . 383

A.8.3.5 Transformations . . . 383
A.8.3.6 Modifications . . . 383
A.8.3.7 Restrictions . . . 383
A.8.3.8 Spatial Reference System . . . 383
A.8.3.9 Bounding Box Operations . . . 384
A.8.3.10 Distance Operations . . . 384
A.8.3.11 Ever and Always Relationships . . . 384
A.8.3.12 Spatiotemporal Relationships . . . 384
A.8.3.13 Comparisons . . . 384
A.8.3.14 Aggregations . . . 384
A.8.3.15 Spatial Operations . . . 384
A.8.3.16 Simplification . . . 385
A.8.4 OGC GeoPose v1.0 Support . . . 385
A.8.4.1 JSON Input and Output . . . 385
A.8.4.2 Quaternion Renormalization . . . 385
A.8.4.3 Euler-Angle Accessors . . . 385
A.8.4.4 Frame Metadata Registry . . . 385
A.8.4.5 Body and World Rigid Transform . . . 385
A.8.4.6 Temporal JSON I/O . . . 385
A.8.4.7 Composite Chain and Graph Documents . . . 386
A.9 Temporal Network Points . . . 386
A.9.1 Static Network Types . . . 386
A.9.1.1 Input and Output . . . 386
A.9.1.2 Constructors . . . 386
A.9.1.3 Conversions . . . 386
A.9.1.4 Accessors . . . 386
A.9.1.5 Transformations . . . 386
A.9.1.6 Spatial Operations . . . 387
A.9.1.7 Comparisons . . . 387
A.9.2 Temporal Network Points . . . 387
A.9.2.1 Constructors . . . 387
A.9.2.2 Conversions . . . 387
A.9.2.3 MF-JSON Input and Output . . . 387
A.9.2.4 Accessors . . . 387
A.9.2.5 Transformations . . . 387
A.9.2.6 Modifications . . . 387
A.9.2.7 Restrictions . . . 388
A.9.2.8 Spatial Operations . . . 388
A.9.2.9 Bounding Box Operations . . . 388

A.9.2.10 Distance Operations . . . 388

A.9.2.11 Ever and Always Relationships . . . 388

A.9.2.12 Spatiotemporal Relationships . . . 388

A.9.2.13 Comparisons . . . 388

A.9.2.14 Aggregations . . . 388

A.9.2.15 Aggregation . . . 389

A.10 Temporal Circular Buffers . . . 389

A.10.1 Static Circular Buffers . . . 389

A.10.1.1 Input and Output . . . 389

A.10.1.2 Constructors . . . 389

A.10.1.3 Conversions . . . 389

A.10.1.4 Accessors . . . 389

A.10.1.5 Transformations . . . 389

A.10.1.6 Spatial Operations . . . 389

A.10.1.7 Distance Operations . . . 389

A.10.1.8 Comparisons . . . 390

A.10.2 Temporal Circular Buffers . . . 390

A.10.2.1 Input and Output . . . 390

A.10.2.2 Constructors . . . 390

A.10.2.3 Conversions . . . 390

A.10.2.4 Accessors . . . 390

A.10.2.5 Transformations . . . 391

A.10.2.6 Modifications . . . 391

A.10.2.7 Restrictions . . . 391

A.10.2.8 Spatial Operations . . . 391

A.10.2.9 Bounding Box Operations . . . 391

A.10.2.10 Distance Operations . . . 391

A.10.2.11 Ever and Always Relationships . . . 392

A.10.2.12 Spatiotemporal Relationships . . . 392

A.10.2.13 Comparisons . . . 392

A.10.2.14 Aggregations . . . 392

A.10.2.15 Simplification . . . 392

A.11 Temporal Rigid Geometries . . . 392

A.11.1 Input and Output . . . 392

A.11.2 Constructors . . . 392

A.11.3 Conversions . . . 393

A.11.4 Accessors . . . 393

A.11.5 Traversed Area . . . 393

A.11.6 Spatial Functions . . . 393

A.11.7 Motion Metrics . . . 393
A.11.8 Transformations . . . 393
A.11.9 Modifications . . . 394
A.11.10 Restrictions . . . 394
A.11.11 Distance Operations . . . 394
A.11.12 Bounding Box Operations . . . 394
A.11.13 Ever and Always Relationships . . . 394
A.11.14 Spatiotemporal Relationships . . . 394
A.11.15 Comparisons . . . 394
A.11.16 Tiles . . . 395
A.11.17 Bounding Boxes . . . 395
A.11.18 Aggregations . . . 395
A.12 Temporal JSON . . . 395
A.12.1 JSONB Operations . . . 395
A.12.2 JSONB Set Operations . . . 396
A.12.3 Input and Output . . . 396
A.12.4 Constructors . . . 396
A.12.5 Conversions . . . 396
A.12.6 Accessors . . . 397
A.12.7 Transformations . . . 397
A.12.8 Temporal JSON Operations . . . 397
A.12.9 Restrictions . . . 397
A.12.10 Bounding Box Operations . . . 397
A.12.11 Comparisons . . . 398
A.12.12 Aggregations . . . 398
A.13 Temporal Tessellations . . . 398
A.13.1 Input and Output . . . 398
A.13.2 Constructors . . . 398
A.13.3 Conversions . . . 399
A.13.4 Accessors . . . 399
A.13.5 Transformations . . . 399
A.13.6 Modifications . . . 399
A.13.7 Restrictions . . . 400
A.13.8 Static Geometry Coverage . . . 400
A.13.9 Inspection . . . 400
A.13.10 Hierarchy . . . 400
A.13.11 Latitude and Longitude Conversions . . . 400
A.13.12 Bounding Box Operations . . . 400
A.13.13 Metric Functions . . . 400

A.13.14 Set-Returning Functions . . . 401
A.13.15 Ever and Always Relationships . . . 401
A.13.16 Spatiotemporal Relationships . . . 401
A.13.17 Comparisons . . . 401
A.13.18 Aggregations . . . 401
A.13.19 H3-Specific Operations . . . 401
A.13.20 QUADBIN-Specific Operations . . . 402
A.13.21 S2-Specific Operations . . . 402
A.13.22 Raster-Specific Operations . . . 403
A.14 Temporal Point Clouds . . . 404
A.14.1 Point Cloud Schemas . . . 404
A.14.2 Static Point Clouds . . . 404
A.14.2.1 Input and Output . . . 404
A.14.2.2 Constructors . . . 404
A.14.2.3 Accessors . . . 404
A.14.3 Point Cloud Sets . . . 404
A.14.3.1 Constructors . . . 404
A.14.4 Point Cloud Bounding Boxes . . . 405
A.14.4.1 Input and Output . . . 405
A.14.4.2 Constructors . . . 405
A.14.4.3 Conversions . . . 405
A.14.4.4 Accessors . . . 405
A.14.4.5 Transformations . . . 405
A.14.4.6 Set Operations . . . 405
A.14.4.7 Topological Operations . . . 405
A.14.4.8 Position Operations . . . 406
A.14.4.9 Comparisons . . . 406
A.14.5 Temporal Point Cloud Points . . . 406
A.14.5.1 Input and Output . . . 406
A.14.5.2 Constructors . . . 406
A.14.5.3 Conversions . . . 406
A.14.5.4 Accessors . . . 406
A.14.5.5 Transformations . . . 406
A.14.5.6 Modifications . . . 407
A.14.5.7 Restrictions . . . 407
A.14.5.8 Splitting Operations . . . 407
A.14.5.9 Bounding Box Operations . . . 407
A.14.5.10 Distance Operations . . . 407
A.14.5.11 Ever and Always Relationships . . . 407

A.14.5.12 Spatiotemporal Relationships . . . 407
A.14.5.13 Comparisons . . . 408
A.14.5.14 Simplification . . . 408
A.14.6 Temporal Point Cloud Patches . . . 408
A.14.6.1 Input and Output . . . 408
A.14.6.2 Constructors . . . 408
A.14.6.3 Conversions . . . 408
A.14.6.4 Accessors . . . 408
A.14.6.5 Transformations . . . 408
A.14.6.6 Modifications . . . 409
A.14.6.7 Restrictions . . . 409
A.14.6.8 Splitting Operations . . . 409
A.14.6.9 Spatial Operations . . . 409
A.14.6.10 Bounding Box Operations . . . 409
A.14.6.11 Distance Operations . . . 409
A.14.6.12 Ever and Always Relationships . . . 409
A.14.6.13 Spatiotemporal Relationships . . . 409
A.14.6.14 Comparisons . . . 410
A.14.7 Aggregations . . . 410
A.14.8 Indexes . . . 410

B Synthetic Data Generator **411**
B.1 Generator for PostgreSQL Types . . . 411
B.2 Generator for PostGIS Types . . . 412
B.3 Generator for MobilityDB Span, Time, and Box Types . . . 413
B.4 Generator for MobilityDB Temporal Types . . . 413
B.5 Generation of Tables with Random Values . . . 414
B.6 Generator for Temporal Network Point Types . . . 419
B.7 Generator for Temporal JSONB Types . . . 419

List of Figures

3.1 Multiresolution grid on Brussels data obtained using the BerlinMOD generator. Each cell contains at most 10,000 (left) and 1,000 (right) instants across the entire simulation period (four days in this case). On the left, we can see the high density of the traffic in the ring around Brussels, while on the right we can see other main axes in the city. 44

4.1 Hierarchy of temporal types in MobilityDB. 50

5.1 Insert operation for temporal values. 70
5.2 Update and delete operation for temporal values. 71
5.3 Modification operations for temporal tables in SQL 71

7.1 Hierarchy of spatiotemporal types in MobilityDB. 91
7.2 Illustration of the use of the `tgeompoint` (top) and the `tgeometry` (bottom) types for modelling the trajectory and the wind swath of tropical storms in 2024. 92
7.3 Visualizing the speed of a moving object using a color ramp in QGIS. 108

9.1 Difference between the spatial and the synchronous distance. 127
9.2 Sampling of temporal floats with discrete, step, and linear interpolation. 128
9.3 Changing the precision of temporal floats with discrete, step, and linear interpolation. 130
9.4 Multidimensional tiling for temporal floats. 137

12.1 Illustration of the use of the `tcbuffer` type for modelling the circular buffer around the wind swath of tropical storms in 2024. 170

16.1 An AIS vessel track leaving Frederikshavn on the three grids, with the cells the track passes through shaded and a marine protected area in green. The upper row states the whole track at the resolutions whose cells are closest in area and outlines in black the excerpt the lower row states, one resolution finer in each grid. Vessel track and protected areas from the Danish AIS data; basemap © OpenStreetMap contributors. 270

List of Tables

1.2 Variables for the user's and the developer's documentation 6

A.1 MobilityDB current instantiations of template types 366

Abstract

MobilityDB is an extension to the PostgreSQL database system and its spatial extension PostGIS. It allows temporal and spatio-temporal objects to be stored in the database, that is, objects whose attribute values and/or location evolves in time. MobilityDB includes functions for analysis and processing of temporal and spatio-temporal objects and provides support for GiST and SP-GiST indexes. MobilityDB is open source and its code is available on Github. A binding for the Python programming language is also available on Github.

MobilityDB is developed by the Computer & Decision Engineering Department of the Université Libre de Bruxelles (ULB) under the direction of Prof. Esteban Zimányi. ULB is an OGC Associate Member and member of the OGC Moving Feature Standard Working Group (MF-SWG).

The MobilityDB Manual is licensed under a Creative Commons Attribution-Share Alike 3.0 License 3. Feel free to use this material any way you like, but we ask that you attribute credit to the MobilityDB Project and wherever possible, a link back to MobilityDB.

Chapter 1

Introduction

MobilityDB is an extension of PostgreSQL and PostGIS that provides *temporal types*. Such data types represent the evolution on time of values of some element type, called the *base type* of the temporal type. For instance, temporal integers may be used to represent the evolution on time of the the gear used by a moving car. In this case, the data type is *temporal integer* and the base type is *integer*. Similarly, a temporal float may be used to represent the evolution on time of the speed of a car. As another example, a temporal point may be used to represent the evolution on time of the location of a car, as reported by GPS devices. Temporal types are useful because representing values that evolve in time is essential in many applications, for example in mobility applications. Furthermore, the operators on the base types (such as arithmetic operators and aggregation for integers and floats, topological and distance relationships for geometries) can be intuitively generalized when the values evolve in time.

MobilityDB provides the temporal types `tbool`, `tint`, `tfloat`, `ttext`, `tgeometry`, `tgeography`, `tgeompoint`, and `tgeogpoint`. These temporal types are based, respectively, on the `boolean`, `integer`, `float`, and `text` base types provided by PostgreSQL, and on the `geometry` and `geography` base types provided by PostGIS, where `tgeometry` and `tgeography` accept arbitrary geometries/geographies, while `tgeompoint` and `tgeogpoint` only accept 2D or 3D points.[1] Furthermore, MobilityDB provides *set*, *span*, and *span set* template types for representing, respectively, sets of values, ranges of values, and sets of ranges of values of base types or time types. Examples of values of set types are `intset`, `floatset`, and `tstzset`, where the latter represents set of `timestamptz` values. Examples of values of span types are `intspan`, `floatspan`, and `tstzspan`. Examples of values of span set types are `intspanset`, `floatspanset`, and `tstzspanset`.

1.1 Project Steering Committee

The MobilityDB Project Steering Committee (PSC) coordinates the general direction, release cycles, documentation, and outreach efforts for the MobilityDB project. In addition, the PSC provides general user support, accepts and approves patches from the general MobilityDB community and votes on miscellaneous issues involving MobilityDB such as developer commit access, new PSC members or significant API changes.

The current members in alphabetical order and their main responsibilities are given next:

- Mohamed Bakli: MobilityDB-docker, cloud and distributed versions, integration with Citus
- Krishna Chaitanya Bommakanti: MEOS (Mobility Engine Open Source), pyMEOS
- Anita Graser: integration with Moving Pandas and the Python ecosystem, integration with QGIS
- Darafei Praliaskouski: integration with PostGIS
- Mahmoud Sakr: co-founder of the MobilityDB project, MobilityDB workshop, co-chair of the OGC Moving Feature Standard Working Group (MF-SWG)
- Vicky Vergara: integration with pgRouting, liason with OSGeo

---

[1] Although 4D temporal points can be represented, the M dimension is currently not taken into account.

- Esteban Zimányi (chair): co-founder of the MobilityDB project, overall project coordination, main contributor of the backend code, BerlinMOD generator

1.2 Other Code Contributors

- Arthur Lesuisse
- Xinyiang Li
- Maxime Schoemans

1.3 Sponsors

These are research funding organizations (in alphabetical order) that have contributed with monetary funding to the MobilityDB project.

- European Commission
- Fonds de la Recherche Scientifique (FNRS), Belgium
- Innoviris, Belgium

These are corporate entities (in alphabetical order) that have contributed developer time or monetary funding to the MobilityDB project.

- Adonmo, India
- Georepublic, Germany
- Université libre de Bruxelles, Belgium

1.4 Licenses

The following licenses can be found in MobilityDB:

Resource	**Licence**
MobilityDB code	PostgreSQL Licence
MobilityDB documentation	Creative Commons Attribution-Share Alike 3.0 License

1.5 Installation from Sources

1.5.1 Short Version

To compile assuming you have all the dependencies in your search path

```
git clone https://github.com/MobilityDB/MobilityDB
mkdir MobilityDB/build
cd MobilityDB/build
cmake  ..
make
sudo make install
```

The above commands install the `master` branch. If you want to install another branch, for example, `develop`, you can replace the first command above as follows

```
git clone --branch develop https://github.com/MobilityDB/MobilityDB
```

You should also set the following in the `postgresql.conf` file depending on the version of PostGIS you have installed (below we use PostGIS 3):

```
shared_preload_libraries = 'postgis-3'
max_locks_per_transaction = 128
```

If you do not preload the PostGIS library with the above configuration you will not be able to load the MobilityDB library and will get an error message such as the following one:

```
ERROR:  could not load library "/usr/local/pgsql/lib/libMobilityDB-1.1.so":
  undefined symbol: ST_Distance
```

You can find the location of the `postgresql.conf` file as given next.

```
$ which postgres
/usr/local/pgsql/bin/postgres
$ ls /usr/local/pgsql/data/postgresql.conf
/usr/local/pgsql/data/postgresql.conf
```

As can be seen, the PostgreSQL binaries are in the `bin` subdirectory while the `postgresql.conf` file is in the `data` subdirectory.

Once MobilityDB is installed, it needs to be enabled in each database you want to use it in. In the example below we use a database named `mobility`.

```
createdb mobility
psql mobility -c "CREATE EXTENSION PostGIS"
psql mobility -c "CREATE EXTENSION MobilityDB"
```

The two extensions PostGIS and MobilityDB can also be created using a single command.

```
psql mobility -c "CREATE EXTENSION MobilityDB cascade"
```

1.5.2 Get the Sources

The MobilityDB latest release can be found in `https://github.com/MobilityDB/MobilityDB/releases/latest`

wget

To download this release:

```
wget -O mobilitydb-1.3.tar.gz https://github.com/MobilityDB/MobilityDB/archive/v1.3.tar.gz
```

Go to Section 1.5.1 to the extract and compile instructions.

git

To download the repository

```
git clone https://github.com/MobilityDB/MobilityDB.git
cd MobilityDB
git checkout v1.3
```

Go to Section 1.5.1 to the compile instructions (there is no tar ball).

1.5.3 Enabling the Database

MobilityDB is an extension that depends on PostGIS. Enabling PostGIS before enabling MobilityDB in the database can be done as follows

```
CREATE EXTENSION postgis;
CREATE EXTENSION mobilitydb;
```

Alternatively, this can be done in a single command by using `CASCADE`, which installs the required PostGIS extension before installing the MobilityDB extension

```
CREATE EXTENSION mobilitydb CASCADE;
```

1.5.4 Dependencies

Compilation Dependencies

To be able to compile MobilityDB, make sure that the following dependencies are met:

- `CMake` cross-platform build system.
- C compiler `gcc` or `clang`. Other ANSI C compilers can be used but may cause problems compiling some dependencies.
- GNU Make (`gmake` or `make`) version 3.1 or higher. For many systems, GNU make is the default version of make. Check the version by invoking `make -v`.
- PostgreSQL version 16 to 19. Outside that range the configuration stops with an explicit error. PostgreSQL is available from http://www.postgresql.org.
- PostGIS version 3 or higher. PostGIS is available from https://postgis.net/.

Notice that PostGIS has its own dependencies such as Proj, GEOS, LibXML2, or JSON-C. MobilityDB also uses Proj and JSON-C, while it computes the geometry operations itself and does not use GEOS. Refer to http://trac.osgeo.org/postgis/wiki/UsersWikiPostgreSQLPostGIS for a support matrix of PostGIS with PostgreSQL, GEOS, and Proj.

Optional Dependencies

For the user's documentation

- The DocBook DTD and XSL files are required for building the documentation. For Ubuntu, they are provided by the packages `docbook` and `docbook-xsl`.
- The XML validator `xmllint` is required for validating the XML files of the documentation. For Ubuntu, it is provided by the package `libxml2`.
- The XSLT processor `xsltproc` is required for building the documentation in HTML format. For Ubuntu, it is provided by the package `libxslt`.
- The program `dblatex` is required for building the documentation in PDF format. For Ubuntu, it is provided by the package `dblatex`.
- The program `dbtoepub` is required for building the documentation in EPUB format. For Ubuntu, it is provided by the package `dbtoepub`.

For the developers's documentation

- The program `doxygen` is required for building the documentation. For Ubuntu, it is provided by the package `doxygen`.

Example: Installing dependencies on Debian-based distributions

Database dependencies

```
sudo apt-get install postgresql-16 postgresql-server-dev-16 postgresql-16-postgis
```

Build dependencies

```
sudo apt-get install cmake gcc
```

Example: Installing dependencies on RPM-based distributions

On Fedora, Red Hat Enterprise Linux, and their derivatives the distribution packages carry a supported PostgreSQL and PostGIS, so a single command covers both the database and the build dependencies.

```
sudo dnf install gcc gcc-c++ cmake make postgresql-server-devel postgis \
  proj-devel json-c-devel
```

The optional families enabled by `-DALL=ON` need two further packages, `gdal-devel` for the raster operators and `h3-devel` for the H3 index type.

1.5.5 Configuring

MobilityDB uses the `cmake` system to do the configuration. The build directory must be different from the source directory.

To create the build directory

```
mkdir build
```

To see the variables that can be configured

```
cd build
cmake -L ..
```

1.5.6 Build and Install

Notice that the current version of MobilityDB has been tested on Linux, MacOS, and Windows systems. It may work on other Unix-like systems, but remain untested. We are looking for volunteers to help us to test MobilityDB on multiple platforms.

The following instructions start from `path/to/MobilityDB` on a Linux system

```
mkdir build
cd build
cmake ..
make
sudo make install
```

When the configuration changes

```
rm -rf build
```

and start the build process as mentioned above.

1.5.7 Testing

MobilityDB uses `ctest`, the CMake test driver program, for testing. This program will run the tests and report results.

To run all the tests

```
ctest
```

To run a given test file

```
ctest -R '021_tbox'
```

To run a set of given test files you can use wildcards

```
ctest -R '022_*'
```

The tests run on a temporary PostgreSQL cluster in the directory `tmptest` of the build directory, whose server listens on a Unix-domain socket in a short directory under `/tmp`, since PostgreSQL does not start when the path of the socket exceeds 107 bytes. Another directory for the cluster can be given at configure time with the variable `MOBILITYDB_TEST_TMPDIR`. The cluster is created in the C locale and in UTF-8, unless the variable `MOBILITYDB_TEST_ENCODING` names another encoding, such as `SQL_ASCII`

```
cmake -DMOBILITYDB_TEST_TMPDIR=/tmp/mobilitydb-test ..
```

1.5.8 Documentation

MobilityDB user's documentation can be generated in HTML, PDF, and EPUB format. Furthermore, the documentation is available in English and in other languages (currently, only in Spanish). The user's documentation can be generated in all formats and in all languages, or specific formats and/or languages can be specified. MobilityDB developer's documentation can only be generated in HTML format and in English.

The variables used for generating user's and the developer's documentation are as follows:

Variable	Default value	Comment
`DOC_ALL`	`BOOL=OFF`	The user's documentation is generated in HTML, PDF, and EPUB formats.
`DOC_HTML`	`BOOL=OFF`	The user's documentation is generated in HTML format.
`DOC_PDF`	`BOOL=OFF`	The user's documentation is generated in PDF format.
`DOC_EPUB`	`BOOL=OFF`	The user's documentation is generated in EPUB format.
`LANG_ALL`	`BOOL=OFF`	The user's documentation is generated in English and in all available translations.
`ES`	`BOOL=OFF`	The user's documentation is generated in English and in Spanish.
`DOC_DEV`	`BOOL=OFF`	The English developer's documentation is generated in HTML format.

Table 1.2: Variables for the user's and the developer's documentation

Generate the user's and the developer's documentation in all formats and in all languages.

```
cmake -D DOC_ALL=ON -D LANG_ALL=ON -D DOC_DEV=ON ..
make doc
make doc_dev
```

Generate the user's documentation in HTML format and in all languages.

```
cmake -D DOC_HTML=ON -D LANG_ALL=ON ..
make doc
```

Generate the English user's documentation in all formats.

```
cmake -D DOC_ALL=ON ..
make doc
```

Generate the user's documentation in PDF format in English and in Spanish.

```
cmake -D DOC_PDF=ON -D ES=ON ..
make doc
```

1.6 Installation from Binaries

1.6.1 Debian-based Linux Distributions

Support for Debian-based Linux systems, such as Ubuntu and Arch Linux, is being developed.

1.6.2 Windows

Since PostGIS version 3.3.3, MobilityDB is distributed in the PostGIS Bundle for Windows, which is available on application stackbuilder and OSGeo website. For more information, refer to the PostGIS documentation.

1.7 Support

MobilityDB community support is available through the MobilityDB github page, documentation, tutorials, mailing lists and others.

1.7.1 Reporting Problems

Bugs are reported and managed in an issue tracker. Please follow these steps:

1. Search the tickets to see if your problem has already been reported. If so, add any extra context you might have found, or at least indicate that you too are having the problem. This will help us prioritize common issues.
2. If your problem is unreported, create a new issue for it.
3. In your report include explicit instructions to replicate your issue. The best tickets include the exact SQL necessary to replicate a problem. Please also, note the operating system and versions of MobilityDB, PostGIS, and PostgreSQL.
4. It is recommended to use the following wrapper on the problem to pin point the step that is causing the problem.

   ```
   SET client_min_messages TO debug;
   <your code>
   SET client_min_messages TO notice;
   ```

1.7.2 Mailing Lists

There are two mailing lists for MobilityDB hosted on OSGeo mailing list server:

- User mailing list: http://lists.osgeo.org/mailman/listinfo/mobilitydb-users
- Developer mailing list: http://lists.osgeo.org/mailman/listinfo/mobilitydb-dev

For general questions and topics about how to use MobilityDB, please write to the user mailing list.

1.8 Migrating from Version 1.2 to Version 1.3

MobilityDB 1.3 is a major revision with respect to version 1.2. The most important change in version 1.3 was to enable new spatiotemporal types. While version 1.2 defined the spatiotemporal types `tgeompoint` (temporal geometry point), `tgeogpoint` (temporal geography point), and `tnpoint` (temporal network point), version 1.3 added the new types `tgeometry` (temporal geometry), `tgeography` (temporal geography), `tcbuffer` (temporal circular buffer), `tpose` (temporal pose), and `trgeometry` (temporal rigid geometry). At a conceptual level, all the spatiotemporal types are organized in the hierarchy depicted in Figure 7.1.

In order to enable the above, a refactoring of the code base was necessary. All the types provided by version 1.2 in addition to the new types `tgeometry` and `tgeography` are defined as core types and thus, are always included in the building process. Each one of the other spatiotemporal types can be included in the building process by setting its corresponding compilation flag. For example, when adding `-DCBUFFER=1` the types built upon the `cbuffer` base type, that is `cbuffer`, `cbufferset`, and `tcbuffer`, will be included in the building process. The possibility to select the types to be included in the executable program is important in particular for stream processing, due to the limited capability of the edge devices. Please notice that the new types `tcbuffer`, `tpose`, and `trgeometry` are still in development and will be finalized in a forthcoming release.

In addition, the API of MobilityDB and MEOS was extended with new functionality and streamlined to improve usability. Finally, version 1.3 also enables the latest versions PostgreSQL 18 and PostGIS 3.6.0. For all these reasons, the binary format of the temporal types have changed from version 1.2 to 1.3 and thus a backup and restore is needed for migrating to the newer version 1.3 of MobilityDB.

1.9 Migrating from Version 1.3 to Version 1.4

Every type that defines the nearest approach distance against a spatiotemporal box states it in version 1.4 by the function `nearestApproachDistance` and the operator `|=|`. In version 1.3 the types `cbuffer` and `pose` stated it instead by the function `distance` and the operator `<->`, where `geometry` and `geography` already used the former. The signatures concerned are `(cbuffer, stbox)`, `(stbox, cbuffer)`, `(pose, stbox)`, and `(stbox, pose)`.

The two functions answer different questions, which is what the change states. A spatiotemporal box carries a period, so `nearestApproachDistance` answers the least distance over the time that its two arguments share, and answers the null value when they share none. The function `distance` measures its two arguments as sets of points and reads no period. The signatures of `distance` whose arguments carry no period keep the name.

Version 1.4 gives MobilityDB one SQL dialect across its batch and stream SQL bindings, PostgreSQL, DuckDB, Apache Spark, and Apache Flink, which name every function identically. A function name must therefore be free in each of these engines. The ever, always, and temporal comparison functions are named by a one-letter quantifier, `e`, `a`, or `t`, followed by the comparison spelled out: a shorter name such as `eLt` would coincide with the function `elt` of Apache Spark and Apache Flink. The operators keep their symbols. The following table gives the functions behind each operator in both versions.

Operators	**Version 1.3**	**Version 1.4**
`?=`, `%=`, `#=`	`ever_eq`, `always_eq`, `temporal_teq`, `tgeo_teq`	`eEqual`, `aEqual`, `tEqual`
`?<>`, `%<>`, `#<>`	`ever_ne`, `always_ne`, `temporal_tne`, `tgeo_tne`	`eNotEqual`, `aNotEqual`, `tNotEqual`
`?<`, `%<`, `#<`	`ever_lt`, `always_lt`, `temporal_tlt`	`eLessThan`, `aLessThan`, `tLessThan`
`?<=`, `%<=`, `#<=`	`ever_le`, `always_le`, `temporal_tle`	`eLessEqual`, `aLessEqual`, `tLessEqual`
`?>`, `%>`, `#>`	`ever_gt`, `always_gt`, `temporal_tgt`	`eGreaterThan`, `aGreaterThan`, `tGreaterTh`
`?>=`, `%>=`, `#>=`	`ever_ge`, `always_ge`, `temporal_tge`	`eGreaterEqual`, `aGreaterEqual`, `tGreater`

The position functions, among which `left` and `right` coincide with functions of Apache Spark and Apache Flink, take as a prefix the class whose values or bounding boxes they compare: `set`, `span`, or `spanset` for sets, spans, and span sets, and for a box or a temporal value the type of its bounding box, `tbox`, `stbox`, or `tpcbox`. A temporal Boolean, text, or JSONB value is bounded by its time span and takes `span`. The positions in time and along the three spatial axes take the prefix alike. The following table gives the functions in both versions.

Version 1.3	**Version 1.4**
`before`	`setBefore`, `spanBefore`, `spansetBefore`, `tboxBefore`, `stboxBefore`,
`after`	`setAfter`, `spanAfter`, `spansetAfter`, `tboxAfter`, `stboxAfter`, `tpcbo`
`overbefore`	`setOverbefore`, `spanOverbefore`, `spansetOverbefore`, `tboxOverbe`
`overafter`	`setOverafter`, `spanOverafter`, `spansetOverafter`, `tboxOverafter`
`left`	`setLeft`, `spanLeft`, `spansetLeft`, `tboxLeft`, `stboxLeft`, `tpcboxLeft`
`right`	`setRight`, `spanRight`, `spansetRight`, `tboxRight`, `stboxRight`, `tpcbo`
`overleft`	`setOverleft`, `spanOverleft`, `spansetOverleft`, `tboxOverleft`, `stb`
`overright`	`setOverright`, `spanOverright`, `spansetOverright`, `tboxOverright`
`below`, `above`, `overbelow`, `overabove`	`stboxBelow`, `stboxAbove`, `stboxOverbelow`, `stboxOverabove`, `tpcbo`
`front`, `back`, `overfront`, `overback`	`stboxFront`, `stboxBack`, `stboxOverfront`, `stboxOverback`, `tpcboxF`

The topological functions, among which `contains` and `overlaps` coincide with a function or a reserved word of Apache Spark and Apache Flink, take as a prefix the class whose values or bounding boxes they compare, as the position functions do. The function `same` of a circular buffer, a network point, a pose, and a pose chain, which the manual documents as a function of these types, keeps its name. The functions comparing the route identifiers of temporal network points are written without an underscore. The following table gives the functions in both versions.

Version 1.3	**Version 1.4**
`overlaps`	`setOverlaps`, `spanOverlaps`, `spansetOverlaps`, `tboxOverlaps`, `stboxOverlaps`, `tpcboxO`
`contains`	`setContains`, `spanContains`, `spansetContains`, `tboxContains`, `stboxContains`, `tpcboxC`
`contained`	`setContained`, `spanContained`, `spansetContained`, `tboxContained`, `stboxContained`, `tp`
`adjacent`	`spanAdjacent`, `spansetAdjacent`, `tboxAdjacent`, `stboxAdjacent`, `tpcboxAdjacent`
`same`	`spanSame`, `tboxSame`, `stboxSame`, `tpcboxSame`
`same_rid`	`sameRid`
`contains_rid`	`containsRid`
`contained_rid`	`containedRid`
`overlaps_rid`	`overlapsRid`

Chapter 2

Set and Span Types

MobilityDB provides *set*, *span*, and *span set* types for representing set of values another type, which is called the *base type*. Set types are akin to *array types* in PostgreSQL restricted to one dimension, but enforce the constraint that sets do not have duplicates. Span and span set types in MobilityDB correspond to the *range and multirange types* in PostgreSQL but have additional constraints. In particular, span types in MobilityDB are of fixed length and do not allow empty spans and infinite bounds. While span types provide similar functionality to range types, they enable increasing performance. In particular, the overhead of processing variable-length types is removed and, in addition, pointer arithmetics and binary search can be used.

The base types used for constructing set, span, and span set types are the types `integer`, `bigint`, `float`, `text`, `date`, and `timestamptz` (timestamp with time zone) provided by PostgreSQL, the types `geometry` and `geography` provided by PostGIS, and the types `cbuffer` (circular buffer, see Chapter 12) and `npoint` (network point, see Chapter 11) provided by MobilityDB. MobilityDB provides the following set and span types:

- `set`: `intset`, `bigintset`, `floatset`, `textset`, `dateset`, `tstzset`, `geomset`, `geogset`, `cbufferset`, `npointset`.
- `span`: `intspan`, `bigintspan`, `floatspan`, `datespan`, `tstzspan`.
- `spanset`: `intspanset`, `bigintspanset`, `floatspanset`, `datespanset`, `tstzspanset`.

We present next the functions and operators for set and span types. These functions and operators are polymorphic, that is, their arguments may be of several types, and the result type may depend on the type of the arguments. To express this in the signature of the functions and operators, we use the following notation:

- `set` represents any set type, such as `intset` or `tstzset`.
- `span` represents any span type, such as `intspan` or `tstzspan`.
- `spanset` represents any span set type, such as `intspanset` or `tstzspanset`.
- `spans` represents any span or span set type, such as `intspan` or `tstzspanset`.
- `base` represents any base type of a set or span type, such as `integer` or `timestamptz`
- `number` represents any base type of a number span type, such as `integer` or `float`,
- `numset` represents any number set type, such as `intset` or `floatset`.
- `numspans` represents any number span or span set type, such as `intspan` or `floatspanset`.
- `numspan` represents any number span type, that is, `intspan`, `bigintspan`, or `floatspan`,
- `numspanset` represents any number span set type, that is, `intspanset`, `bigintspanset`, or `floatspanset`,
- `floatspans` represents a float span or span set type, that is, `floatspan` or `floatspanset`,

- `datespans` represents a date span or span set type, that is, `datespan` or `datespanset`,
- `tstzspans` represents a timestamptz span or span set type, that is, `tstzspan` or `tstzspanset`,
- `spatialset` represents any spatial set type, such as `geomset` or `geogset`.
- `numbers` represents any number set or range type, such as `integer`, `intset`, `intspan`, or `intspanset`,
- `dates` represents any time type with `date` granularity, that is, `date`, `dateset`, `datespan`, or `datespanset`,
- `times` represents any time type with `timestamptz` granularity, that is, `timestamptz`, `tstzset`, `tstzspan`, or `tstzspanset`,
- A set of types such as `{set, spans}` represents any of the types listed,
- A set of operators such as `{=, <>}` represents any of the operators listed,
- `type[]` represents an array of `type`.

As an example, the signature of the contains operator (`@>`) is as follows:

```
{set,spans} @> {set,spans,base} → boolean
```

Notice that the signature above is an abridged version of the more precise signature below

```
set @> {set,base} → boolean
spans @> {spans,base} → boolean
```

since sets and spans cannot be mixed in operations and thus, for instance, we cannot ask wether a span contains a set. In the following, for conciseness, we use the abridged style of signatures above. Furthermore, the time part of the timestamps is omitted in most examples. Recall that in that case PostgreSQL assumes the time `00:00:00`.

In what follows, since span and span set types have similar functions and operators, when we speak about span types we mean both span and span set types, unless we explicitly refer to *unit* span types and span *set* types to distinguish them.

2.1 Input and Output

MobilityDB generalizes Open Geospatial Consortium's (OGC) Well-Known Text (WKT) and Well-Known Binary (WKB) input and output format for all its types. In this way, applications can exchange data between them using a standardized exchange format. The WKT format is human-readable while the WKB format is more compact and more efficient than the WKT format. The WKB format can be output either as a binary string or as a character string encoded in hexadecimal ASCII.

The set types represent an *ordered* set of *distinct* values. A set must contain at least one element. Examples of set values are as follows:

```
SELECT tstzset '{2001-01-01 08:00:00, 2001-01-03 09:30:00}';
-- Singleton set
SELECT textset '{"highway"}';
-- Erroneous set: unordered elements
SELECT floatset '{3.5, 1.2}';
-- Erroneous set: duplicate elements
SELECT geomset '{"Point(1 1)", "Point(1 1)"}';
-- Circular-buffer set: each element is a cbuffer WKT
SELECT cbufferset '{"Cbuffer(Point(1 1),0.5)", "Cbuffer(Point(2 2),1.0)"}';
```

Notice that the elements of the sets `textset`, `geomset`, `geogset`, `cbufferset`, and `npointset` must be enclosed between double quotes. Notice also that geometries and geographies follow the order defined in PostGIS.

A value of a unit span type has two bounds, the *lower bound* and the *upper bound*, which are values of the underlying *base type*. For example, a value of the `tstzspan` type has two bounds, which are `timestamptz` values. The bounds can be inclusive or exclusive. An inclusive bound means that the boundary instant is included in the span, while an exclusive bound means that the boundary instant is not included in the span. In the text form of a span value, inclusive and exclusive lower bounds are represented, respectively, by "`[`" and "`(`". Likewise, inclusive and exclusive upper bounds are represented, respectively, by "`]`" and "`)`". In a span value, the lower bound must be less than or equal to the upper bound. A span value with equal and inclusive bounds is called an *instantaneous span* and corresponds to a base type value. Examples of span values are as follows:

```
SELECT intspan '[1, 3)';
SELECT floatspan '[1.5, 3.5]';
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-03 09:30:00)';
-- Instant spans
SELECT intspan '[1, 1]';
SELECT floatspan '[1.5, 1.5]';
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-01 08:00:00]';
-- Erroneous span: invalid bounds
SELECT tstzspan '[2001-01-01 08:10:00, 2001-01-01 08:00:00]';
-- Erroneous span: empty span
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-01 08:00:00)';
```

Values of `intspan`, `bigintspan`, and `datespan` are converted into *normal form* so that equivalent values have identical representations. In the canonical representation of these types, the lower bound is inclusive and the upper bound is exclusive as shown in the following examples:

```
SELECT intspan '[1, 1]';
-- [1, 2)
SELECT bigintspan '(1, 3]';
--[2, 4)
SELECT datespan '[2001-01-01, 2001-01-03]';
-- [2001-01-01, 2001-01-04)
```

The normal form replaces an exclusive lower bound or an inclusive upper bound by the next value of the base type, so a span whose bound is the largest value of the base type and needs that replacement is refused, as for the corresponding PostgreSQL ranges:

```
SELECT span(1, 2147483647, true, false);
-- [1, 2147483647)
SELECT span(1, 2147483647, true, true);
-- ERROR:  integer out of range
```

A value of a span set type represents an *ordered* set of *disjoint* span values. A span set value must contain at least one element, in which case it corresponds to a single span value. Examples of span set values are as follows:

```
SELECT floatspanset '{[8.1, 8.5],[9.2, 9.4]}';
-- Singleton spanset
SELECT tstzspanset '{[2001-01-01 08:00:00, 2001-01-01 08:10:00]}';
-- Erroneous spanset: unordered elements
SELECT intspanset '{[3,4],[1,2]}';
-- Erroneous spanset: overlapping elements
SELECT tstzspanset '{[2001-01-01 08:00:00, 2001-01-01 08:10:00],
  [2001-01-01 08:05:00, 2001-01-01 08:15:00]}';
```

Values of the span set types are converted into *normal form* so that equivalent values have identical representations. For this, consecutive adjacent span values are merged when possible. Examples of transformation into normal form are as follows:

```
SELECT intspanset '{[1,2],[3,4]}';
-- {[1, 5)}
SELECT floatspanset '{[1.5,2.5],(2.5,4.5]}';
-- {[1.5, 4.5]}
SELECT tstzspanset '{[2001-01-01 08:00:00, 2001-01-01 08:10:00),
  [2001-01-01 08:10:00, 2001-01-01 08:10:00],
  (2001-01-01 08:10:00, 2001-01-01 08:20:00]}';
-- {[2001-01-01 08:00:00,2001-01-01 08:20:00]}
```

We give next the functions for input and output of set and span types in Well-Known Text (WKT) and Well-Known Binary (WKB) format. The default output format of all set and span types is the Well-Known Text format. The function `asText` given next enables to determine the output of floating point values.

- Return the Well-Known Text (WKT) representation

  ```
  asText({floatset,floatspans},maxdecimaldigits integer=15) → text
  ```

  The `maxdecimaldigits` argument can be used to set the maximum number of decimal places in the output of floating point values (default 15).

  ```
  SELECT asText(floatset '{1.123456789,2.123456789}', 3);
  -- {1.123, 2.123}
  SELECT asText(floatspanset '{[1.55,2.55],[4,5]}',0);
  -- {[2, 3], [4, 5]}
  ```

- Return the Extended Well-Known Text (EWKT) representation of a spatial set

  ```
  asEWKT(spatialset,maxdecimaldigits integer=15) → text
  ```

  The SRID of the set prefixes its elements. A cell index set states SRID 4326 and a point cloud set the SRID its schema declares, and neither takes the `maxdecimaldigits` argument.

  ```
  SELECT asEWKT(geomset 'SRID=5676;{"Point(1 1)", "Point(2 2)"}');
  -- SRID=5676;{"POINT(1 1)", "POINT(2 2)"}
  SELECT asEWKT(h3indexset '{8928308280fffff, 8928308280bffff}');
  -- SRID=4326;{"8928308280bffff", "8928308280fffff"}
  ```

- Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation

  ```
  asBinary({set,spans},endian text='') → bytea
  asHexWKB({set,spans},endian text='') → text
  ```

  The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used.

  ```
  SELECT asBinary(dateset '{2001-01-01, 2001-01-03}');
  -- \x01050001020000006e01000070010000
  SELECT asBinary(intspan '[1, 3)');
  -- \x011300010100000003000000
  SELECT asBinary(floatspanset '{[1, 2], [4, 5]}', 'XDR');
  -- \x00000e00000002033ff00000000000004000000000000000034010000000000000004014000000000000
  SELECT asHexWKB(dateset '{2001-01-01, 2001-01-03}');
  -- 01050001020000006E01000070010000
  SELECT asHexWKB(intspan '[1, 3)');
  -- 011300010100000003000000
  SELECT asHexWKB(floatspanset '{[1, 2], [4, 5]}', 'XDR');
  -- 00000E00000002033FF00000000000004000000000000000034010000000000000004014000000000000
  ```

- Return the Extended Well-Known Binary (EWKB) or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation of a spatial set

  ```
  asEWKB(spatialset,endian text='') → bytea
  asHexEWKB(spatialset,endian text='') → text
  ```

  The SRID of the set, when known, is embedded once in the set header: `asEWKB` and `asHexEWKB` include it, while `asBinary` and `asHexWKB` emit the plain Well-Known Binary representation without it, matching the OGC convention and the `cbuffer`, `npoint`, and `pose` base-type `asBinary`/`asHexWKB`. A cell index set states SRID 4326, and a point cloud set the SRID its schema declares in the `pointcloud_formats` catalog, so the extended and plain forms of a point cloud set coincide when its schema declares no SRID.

```
SELECT asBinary(geomset 'SRID=4326;{Point(1 1)}');
-- \x012b0001010000000101000000000000000000f03f000000000000f03f
SELECT asHexWKB(geomset 'SRID=4326;{Point(1 1)}');
-- 012B0001010000000101000000000000000000F03F000000000000F03F
SELECT asEWKB(geomset 'SRID=4326;{Point(1 1)}');
-- \x012b0041e6100000010000000101000000000000000000f03f000000000000f03f
SELECT asHexEWKB(geomset 'SRID=4326;{Point(1 1)}');
-- 012B0041E6100000010000000101000000000000000000F03F000000000000F03F
SELECT asEWKB(cbufferset 'SRID=4326;{"Cbuffer(Point(1 1),0.5)"}');
-- \x013a0041e610000001000000000000000000f03f000000000000f03f000000000000e03f
SELECT asHexEWKB(npointset '{"NPoint(1,0.5)"}');
-- 01310041E610000001000000010100000000000000000000000000E03F
SELECT asHexEWKB(h3indexset '{8a2a1072b59ffff}', 'XDR');
-- 00004561000010E60000000108A2A1072B59FFFF
```

- Input from the Well-Known Text (WKT) representation

```
settypeFromText(text) → set
spantypeFromText(text) → span
spansettypeFromText(text) → spanset
```

There is one function per set or span (set) type, the name of the function has as prefix the name of the type. The function is the inverse of `asText`: it reads the text a literal of the type is written in, from a constant or from a `text` column alike. A spatial set reads the Well-Known Text of its elements; its Extended Well-Known Text, which states the SRID, is read by `FromEWKT` below.

```
SELECT intsetFromText('{1, 2, 3}');
-- {1, 2, 3}
SELECT textsetFromText('{"highway", "car"}');
-- {"car", "highway"}
SELECT floatspanFromText('[1.5, 2.5)');
-- [1.5, 2.5)
SELECT tstzspanFromText('[2001-01-01, 2001-01-03)');
-- [2001-01-01, 2001-01-03)
SELECT intspansetFromText('{[1, 3), [4, 5]}');
-- {[1, 3), [4, 6)}
SELECT asText(geomsetFromText('{"Point(1 1)", "Point(2 2)"}'));
-- {"POINT(1 1)", "POINT(2 2)"}
```

- Input from the Well-Known Binary (WKB) representation

```
settypeFromBinary(bytea) → set
spantypeFromBinary(bytea) → span
spansettypeFromBinary(bytea) → spanset
```

There is one function per set or span (set) type, the name of the function has as prefix the name of the type

```
SELECT datesetFromBinary('\x01050001020000006e01000070010000');
-- {2001-01-01, 2001-01-03}
SELECT intspanFromBinary('\x011300010100000003000000');
-- [1, 3)
SELECT floatspansetFromBinary(
  '\x00000e00000002033ff00000000000004000000000000000034010000000000000401400000000000000');
-- {[1, 2], [4, 5]}
```

- Input from the Hexadecimal Well-Known Binary (HexWKB) representation

```
settypeFromHexWKB(text) → set
spantypeFromHexWKB(text) → span
spansettypeFromHexWKB(text) → spanset
```

There is one function per set or span (set) type, the name of the function has as prefix the name of the type.

```
SELECT datesetFromHexWKB('01050001020000006E01000070010000');
-- {2001-01-01, 2001-01-03}
SELECT intspanFromHexWKB('011300010100000003000000');
-- [1, 3)
SELECT floatspanFromHexWKB('01060001000000000000F83F0000000000000440');
-- [1.5, 2.5)
SELECT floatspansetFromHexWKB(
  '00000E00000002033FF0000000000000400000000000000003401000000000000040140000000000000');
-- {[1, 2], [4, 5]}
```

- Input from the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation of a spatial set

```
spatialsettypeFromText(text) → spatialset
spatialsettypeFromEWKT(text) → spatialset
```

There is one function per spatial set type, the name of the function has as prefix the name of the type. A stated SRID must be the one a cell index or point cloud set carries, and an error is raised otherwise.

```
SELECT asText(geomsetFromText('{"Point(1 1)", "Point(2 2)"}'));
-- {"POINT(1 1)", "POINT(2 2)"}
SELECT asEWKT(geomsetFromEWKT('SRID=5676;{"Point(1 1)", "Point(2 2)"}'));
-- SRID=5676;{"POINT(1 1)", "POINT(2 2)"}
SELECT asEWKT(h3indexsetFromEWKT('SRID=4326;{8928308280fffff}'));
-- SRID=4326;{"8928308280fffff"}
SELECT h3indexsetFromEWKT('SRID=3857;{8928308280fffff}');
-- ERROR:  The SRID 3857 cannot be stated for the h3index
```

- Input from the Extended Well-Known Binary (EWKB) or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation of a spatial set

```
spatialsettypeFromEWKB(bytea) → spatialset
spatialsettypeFromHexEWKB(text) → spatialset
```

There is one function per spatial set type, the name of the function has as prefix the name of the type. The SRID the header states must be the one a cell index or point cloud set carries, and an error is raised otherwise.

```
SELECT asEWKT(geomsetFromEWKB(asEWKB(geomset 'SRID=5676;{"Point(1 1)"}')));
-- SRID=5676;{"POINT(1 1)"}
SELECT asEWKT(quadbinsetFromHexEWKB('01480041E610000001000000FFFFFFFFFFFF0F48'));
-- SRID=4326;{"480fffffffffff"}
```

2.2 Constructors

The constructor function for the set types has a single argument that is an array of values of the corresponding base type. The values must be ordered and cannot have nulls or duplicates.

- Constructor for set types

```
set(base[]) → set
```

```
SELECT set(ARRAY['highway', 'primary', 'secondary']);
-- {"highway", "primary", "secondary"}
SELECT set(ARRAY[timestamptz '2001-01-01 08:00:00', '2001-01-03 09:30:00']);
-- {2001-01-01 08:00:00, 2001-01-03 09:30:00}
```

The unit span types have a constructor function that accepts four arguments. The first two arguments specify, respectively, the lower and upper bound, and the last two arguments are Boolean values stating, respectively, whether the lower and upper bounds are inclusive or not. The last two arguments are assumed to be, respectively, true and false if not specified. Notice that integer spans are transformed into *normal form*, that is, with inclusive lower bound and exclusive upper bound.

- Constructor for span types

```
span(lower base,upper base,lower_inc boolean=true,upper_inc boolean=false) → span
```

```
SELECT span(20.5, 25);
-- [20.5, 25)
SELECT span(20, 25, false, true);
-- [21, 26)
SELECT span(timestamptz '2001-01-01 08:00:00', '2001-01-03 09:30:00', false, true);
-- (2001-01-01 08:00:00, 2001-01-03 09:30:00]
```

The constructor function for span set types have a single argument that is an array of span values of the same subtype.

- Constructor for span set types

```
spanset(span[]) → spanset
```

```
SELECT spanset(ARRAY[intspan '[10,12]', '[13,15]']);
-- {[10, 16)}
SELECT spanset(ARRAY[floatspan '[10.5,12.5]', '[13.5,15.5]']);
-- {[10.5, 12.5], [13.5, 15.5]}
SELECT spanset(ARRAY[tstzspan '[2001-01-01 08:00, 2001-01-01 08:10]',
  '[2001-01-01 08:20, 2001-01-01 08:40]']);
-- {[2001-01-01 08:00, 2001-01-01 08:10], [2001-01-01 08:20, 2001-01-01 08:40]};
```

2.3 Conversions

Values of set and span types can be converted to one another or converted to and from PostgreSQL range types using the function `CAST` or using the `::` notation.

- Convert a base value to a set, span, or span set value

```
base::{set,span,spanset}
set(base) → set
span(base) → span
spanset(base) → spanset
```

```
SELECT CAST(timestamptz '2001-01-01 08:00:00' AS tstzset);
-- {2001-01-01 08:00:00}
SELECT timestamptz '2001-01-01 08:00:00'::tstzspan;
-- [2001-01-01 08:00:00, 2001-01-01 08:00:00]
SELECT spanset(timestamptz '2001-01-01 08:00:00');
-- {[2001-01-01 08:00:00, 2001-01-01 08:00:00]}
```

- Convert a set value to a span set value

```
set::spanset
spanset(set) → spanset
```

```
SELECT spanset(tstzset '{2001-01-01 08:00:00, 2001-01-01 08:15:00, 2001-01-01 08:25:00}');
/* {[2001-01-01 08:00:00, 2001-01-01 08:00:00], [2001-01-01 08:15:00,
   2001-01-01 08:15:00],
   [2001-01-01 08:25:00, 2001-01-01 08:25:00]} */
```

- Convert a geo set into the collection of its values

```
geomset::geometry
geogset::geography
geometry(geomset) → geometry
geography(geogset) → geography
```

The collection takes its type from the types of the values alone, as the PostGIS function `ST_Collect` builds it from them, so a set of points, a set of one point included, answers a multipoint.

```
SELECT ST_AsText(geomset '{"Point(1 1)", "Point(2 2)"}'::geometry);
-- MULTIPOINT((1 1),(2 2))
SELECT ST_AsText(geomset '{"Point(1 1)"}'::geometry);
-- MULTIPOINT((1 1))
SELECT ST_AsText(geometry(geomset '{"Point(1 1)", "Linestring(0 0,1 1)"}'));
-- GEOMETRYCOLLECTION(POINT(1 1),LINESTRING(0 0,1 1))
```

- Convert between a geometry and a geography

```
geomset::geogset
geogset::geomset
geogset(geomset) → geogset
geomset(geogset) → geomset
```

A spatial value of any type converts to a geometry and to a geography, whether it is planar or geodetic. The conversion changes the type and never the coordinates, so a measure then follows the target: on the plane for a geometry, on the surface of the earth for a geography. As a geography, a value with no SRID has the SRID 4326 and a value in a projected reference system raises an error, as in PostGIS. The rule holds for every spatial type and every temporal one, each of which reaches the same conversion of its geometry or geography values.

```
SELECT ST_AsEWKT(geometry 'Point(1 2)'::geography);
-- SRID=4326;POINT(1 2)
SELECT asEWKT(geomset '{"Point(1 2)"}'::geogset);
-- SRID=4326;{"POINT(1 2)"}
SELECT asEWKT(geogset '{"Point(1 2)"}'::geomset);
-- SRID=4326;{"POINT(1 2)"}
SELECT geomset 'SRID=3857;{"Point(1 2)"}'::geogset;
-- ERROR:  Only lon/lat coordinate systems are supported
```

- Convert a span value to a span set value

```
span::spanset
spanset(span) → spanset
```

```
SELECT floatspan '[1.5,2.5]'::floatspanset;
-- {[1.5, 2.5]}
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-01 08:30:00)'::tstzspanset;
-- {[2001-01-01 08:00:00, 2001-01-01 08:30:00)}
```

- Convert a set or a span set into a span, ignoring the potential time gaps

```
{set,spanset}::span
span({set,spanset}) → span
```

```
SELECT span(dateset '{2001-01-01, 2001-01-03, 2001-01-05}');
-- [2001-01-01, 2001-01-06)
SELECT span(tstzspanset '{[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04)}');
-- [2001-01-01, 2001-01-04)
```

- Convert a span value to and from a PostgreSQL range value

```
{span,range}::{range,span}
range(span) → range
span(range) → span
```

Notice that PostgreSQL range values accept empty ranges and ranges with infinite values, which are not allowed as span values in MobilityDB

```
SELECT intspan '[10, 20)'::int4range;
-- [10,20)
SELECT tstzspan '[2001-01-01 08:00:00, 2001-01-01 08:30:00)'::tstzrange;
-- ["2001-01-01 08:00:00","2001-01-01 08:30:00")
SELECT int4range '[10, 20)'::intspan;
-- [10,20)
SELECT int4range 'empty'::intspan;
-- ERROR:  Range cannot be empty
SELECT int4range '[10,)'::intspan;
-- ERROR:  Range bounds cannot be infinite
SELECT tstzrange '[2001-01-01 08:00:00, 2001-01-01 08:30:00)'::tstzspan;
-- [2001-01-01 08:00:00, 2001-01-01 08:30:00)
```

- Convert a span set value to and from a PostgreSQL multirange value

```
{spanset,multirange}::{multirange,spanset}
multirange(spanset) → multirange
spanset(multirange) → spanset
```

```
SELECT intspanset '{[1,2],[4,5]}'::int4multirange;
-- {[1,3),[4,6)}
SELECT tstzspanset '{[2001-01-01,2001-01-02],[2001-01-04,2001-01-05]}'::tstzmultirange;
-- {[2001-01-01,2001-01-02],[2001-01-04,2001-01-05]}
SELECT int4multirange '{[1,2],[4,5]}'::intspanset;
-- {[1, 3), [4, 6)}
SELECT tstzmultirange '{[2001-01-01,2001-01-02],[2001-01-04,2001-01-05]}'::tstzspanset;
-- {[2001-01-01, 2001-01-02], [2001-01-04, 2001-01-05]}
```

2.4 Accessors

- Return the memory size in bytes

```
memSize({set,spanset}) → integer
```

```
SELECT memSize(tstzset '{2001-01-01, 2001-01-02, 2001-01-03}');
-- 48
SELECT memSize(tstzspanset '{[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04],
  [2001-01-05, 2001-01-06]}');
-- 112
```

- Return the lower or upper bound

```
lower(spans) → base
upper(spans) → base
```

```
SELECT lower(tstzspan '[2001-01-01, 2001-01-05)');
-- 2001-01-01
SELECT lower(intspanset '{[1,2],[4,5]}');
--  1
```

```
SELECT lower(tstzspan '[2001-01-01, 2001-01-05)');
-- 2001-01-01
SELECT upper(intspanset '{[1,2],[4,5]}');
--  6
SELECT lower(tstzspan '[2001-01-01, 2001-01-05)');
-- 2001-01-01
SELECT lower(intspanset '{[1,2],[4,5]}');
--  1
```

```
SELECT upper(floatspan '[20.5, 25.3)');
-- 25.3
SELECT upper(tstzspan '[2001-01-01, 2001-01-05)');
-- 2001-01-05
```

- Is the lower or upper bound inclusive?

```
lowerInc(spans) → boolean
upperInc(spans) → boolean
```

```
SELECT lowerInc(datespan '[2001-01-01, 2001-01-05)');
-- true
SELECT lowerInc(intspanset '{[1,2],[4,5]}');
-- true
```

```
SELECT upper(floatspan '[20.5, 25.3]');
-- true
SELECT upperInc(tstzspan '[2001-01-01, 2001-01-05)');
-- false
```

- Return the width of the span as a float

```
width(numspan) → float
width(numspanset,boundspan boolean=false) → float
```

An additional parameter can be set to true to compute the width of the bounding span, thus ignoring the potential value gaps

```
SELECT width(floatspan '[1, 3)');
-- 2
SELECT width(intspanset '{[1,3),[5,7)}');
-- 4
SELECT width(intspanset '{[1,3),[5,7)}', true);
-- 6
```

- Return the duration

```
duration({datespan,tstzspan}) → interval
duration({datespanset,tstzspanset},boundspan boolean=false) → interval
```

An additional parameter can be set to true to compute the duration of the bounding time span, thus ignoring the potential time gaps

```
SELECT duration(datespan '[2001-01-01, 2001-01-03)');
-- 2 days
SELECT duration(tstzspanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05)}');
-- 3 days
SELECT duration(tstzspanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05)}', true);
-- 4 days
```

- Return the (number of) values

```
numValues(set) → integer
getValues(set) → base[]
```

```
SELECT numValues(intset '{1,3,5,7}');
-- 4
SELECT getValues(tstzset '{2001-01-01, 2001-01-03, 2001-01-05, 2001-01-07}');
-- {"2001-01-01","2001-01-03","2001-01-05","2001-01-07"}
```

- Return the start, end, or n-th value

```
startValue(set) → base
endValue(set) → base
valueN(set,integer) → base
```

```
SELECT startValue(intset '{1,3,5,7}');
-- 1
SELECT endValue(dateset '{2001-01-01, 2001-01-03, 2001-01-05, 2001-01-07}');
-- 2001-01-07
SELECT valueN(floatset '{1,3,5,7}',2);
-- 3
```

- Transform a set into a set of rows, one per value of the set, in the order of the set {}

```
unnest(set) → {base}
```

```
SELECT unnest(intset '{1,3,5}');
-- 1
-- 3
-- 5
SELECT unnest(textset '{"highway", "car"}');
-- car
-- highway
SELECT ST_AsText(unnest(geomset '{"Point(1 1)", "Point(2 2)"}'));
-- POINT(1 1)
-- POINT(2 2)
```

- Return the (number of) spans

```
numSpans(spanset) → integer
spans(spanset) → span[]
```

```
SELECT numSpans(intspanset '{[1,3),[4,5),[6,7)}');
-- 3
SELECT numSpans(datespanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05),
  [2001-01-06, 2001-01-07)}');
-- 3
```

```
SELECT spans(floatspanset '{[1,3),[4,4],[6,7)}');
-- {"[1,3)","[4,4]","[6,7)"}
SELECT spans(tstzspanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-04],
  [2001-01-05, 2001-01-06)}');
-- {"[2001-01-01,2001-01-03)", "[2001-01-04,2001-01-04]", "[2001-01-05,2001-01-06)"}
```

- Return the start, end, or n-th span

```
startSpan(spanset) → span
endSpan(spanset) → span
spanN(spanset,integer) → span
```

```
SELECT startSpan(intspanset '{[1,3),[4,5),[6,7)}');
-- [1,3)
SELECT startSpan(datespanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05),
  [2001-01-06, 2001-01-07)}');
-- [2001-01-01,2001-01-03)
```

```
SELECT endSpan(floatspanset '{[1,3),[4,4],[6,7)}');
-- [6,7)
SELECT endSpan(tstzspanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-04],
  [2001-01-05, 2001-01-06)}');
-- [2001-01-05,2001-01-06)
```

```
SELECT spanN(floatspanset '{[1,3),[4,4],[6,7)}',2);
-- [4,4]
SELECT spanN(tstzspanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-04],
  [2001-01-05, 2001-01-06)}', 2);
-- [2001-01-04,2001-01-04]
```

- Return the (number of) different dates

```
numDates(datespanset) → integer
dates(datespanset) → dateset
```

```
SELECT numDates(datespanset '{[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04)}');
-- 4
SELECT dates(datespanset '{[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04)}');
-- {2001-01-01, 2001-01-02, 2001-01-03, 2001-01-04}
```

- Return the start, end, or n-th date

```
startDate(datespanset) → date
endDate(datespanset) → date
dateN(datespanset,integer) → date
```

The functions do not take into account whether the bounds are inclusive or not.

```
SELECT startDate(datespanset '{[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04)}');
-- 2001-01-01
SELECT endDate(datespanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}');
-- 2001-01-05
SELECT dateN(datespanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}', 3);
-- 2001-01-05
```

- Return the (number of) different timestamps

```
numTimestamps(tstzspanset) → integer
timestamps(tstzspanset) → tstzset
```

```
SELECT numTimestamps(tstzspanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}');
-- 3
SELECT timestamps(tstzspanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}');
-- {"2001-01-01 00:00:00", "2001-01-03 00:00:00", "2001-01-05 00:00:00"}
```

- Return the start, end, or n-th timestamp

  ```
  startTimestamp(tstzspanset) → timestamptz
  endTimestamp(tstzspanset) → timestamptz
  timestampN(tstzspanset,integer) → timestamptz
  ```

  The functions do not take into account whether the bounds are inclusive or not.

  ```
  SELECT startTimestamp(tstzspanset '{[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04)}');
  -- 2001-01-01
  SELECT endTimestamp(tstzspanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}');
  -- 2001-01-05
  SELECT timestampN(tstzspanset '{[2001-01-01, 2001-01-03), (2001-01-03, 2001-01-05)}', 3);
  -- 2001-01-05
  ```

2.5 Transformations

- Expand or shrink the bounds by a value or an interval

  ```
  expand(numspan,base) → numspan
  expand(tstzspan,interval) → tstzspan
  ```

  The function returns NULL if the value or interval given as second argument is negative and the span resulting from shifting the bounds with the argument is empty.

  ```
  SELECT expand(floatspan '[1, 3]', 1);
  -- [0, 4]
  SELECT expand(floatspan '[1, 3]', -1);
  -- [2, 2]
  SELECT expand(floatspan '[1, 3)', -1);
  -- NULL
  SELECT expand(tstzspan '[2001-01-01, 2001-01-03]', interval '1 day');
  -- [2000-12-31, 2001-01-04]
  SELECT expand(tstzspan '[2001-01-01, 2001-01-03]', interval '-1 day');
  -- [2001-01-02, 2001-01-02]
  SELECT expand(tstzspan '[2001-01-01, 2001-01-03]', interval '-2 day');
  -- NULL
  ```

- Shift by a value or interval

  ```
  shift(numbers,base) → numbers
  shift(dates,integer) → dates
  shift(times,interval) → times
  ```

  ```
  SELECT shift(dateset '{2001-01-01, 2001-01-03, 2001-01-05}', 1);
  -- {2001-01-02, 2001-01-04, 2001-01-06}
  SELECT shift(intspan '[1, 4)', -1);
  -- [0, 3)
  SELECT shift(tstzspan '[2001-01-01, 2001-01-03]', interval '1 day');
  -- [2001-01-02, 2001-01-04]
  SELECT shift(floatspanset '{[1, 2], [3, 4]}', -1);
  -- {[0, 1], [2, 3]}
  SELECT shift(tstzspanset '{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-05]}',
    interval '1 day');
  -- {[2001-01-02, 2001-01-04], [2001-01-05, 2001-01-06]}
  ```

- Scale by a value or interval

```
scale(numbers,base) → numbers
scale(dates,integer) → dates
scale(times,interval) → times
```

If the width or time span of the input value is zero (for example, for a singleton timestamp set), the result is the input value. The given value or interval must be strictly greater than zero.

```
SELECT scale(tstzset '{2001-01-01}', '1 day');
-- {2001-01-01}
SELECT scale(tstzset '{2001-01-01, 2001-01-03, 2001-01-05}', '2 days');
-- {2001-01-01, 2001-01-02, 2001-01-03}
SELECT scale(intspan '[1, 4)', 4);
-- [1, 6)
SELECT scale(datespan '[2001-01-01, 2001-01-04)', 4);
-- [2001-01-01, 2001-01-06)
SELECT scale(tstzspan '[2001-01-01, 2001-01-03]', '1 day');
-- [2001-01-01, 2001-01-02]
SELECT scale(floatspanset '{[1, 2], [3, 4]}', 6);
-- {[1, 3], [5, 7]}
SELECT scale(tstzspanset '{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-05]}', '1 day');
/* {[2001-01-01 00:00:00, 2001-01-01 12:00:00],
   [2001-01-01 18:00:00, 2001-01-02 00:00:00]} */
SELECT scale(tstzset '{2001-01-01}', '-1 day');
-- ERROR:  The duration must be a positive interval: -1 days
```

- Shift and scale by the values or intervals

```
shiftScale(numbers,base,base) → numbers
shiftScale(dates,integer,integer) → dates
shiftScale(times,interval,interval) → times
```

This function combines the functions shift and scale.

```
SELECT shiftScale(tstzset '{2001-01-01}', '1 day', '1 day');
-- {2001-01-02}
SELECT shiftScale(tstzset '{2001-01-01, 2001-01-03, 2001-01-05}', '1 day','2 days');
-- {2001-01-02, 2001-01-03, 2001-01-04}
SELECT shiftScale(intspan '[1, 4)', -1, 4);
-- [0, 5)
SELECT shiftScale(datespan '[2001-01-01, 2001-01-04)', -1, 4);
-- [2000-12-31, 2001-01-05)
SELECT shiftScale(tstzspan '[2001-01-01, 2001-01-03]', '1 day', '1 day');
-- [2001-01-02, 2001-01-03]
SELECT shiftScale(floatspanset '{[1, 2], [3, 4]}', -1, 6);
-- {[0, 2], [4, 6]}
SELECT shiftScale(tstzspanset '{[2001-01-01, 2001-01-03], [2001-01-04, 2001-01-05]}',
  '1 day', '1 day');
/* {[2001-01-02 00:00:00, 2001-01-02 12:00:00],
   [2001-01-02 18:00:00, 2001-01-03 00:00:00]} */
```

- Round down or up to the nearest integer

```
floor({floatset,floatspans}) → {floatset,floatspans}
ceil({floatset,floatspans}) → {floatset,floatspans}
```

```
SELECT floor(floatset '{1.5,2.5}');
-- {1, 2}
SELECT ceil(floatspan '[1.5,2.5)');
-- [2, 3)
SELECT floor(floatspan '(1.5, 1.6)');
```

```
-- [1, 1]
SELECT ceil(floatspanset '{[1.5, 2.5],[3.5,4.5]}');
-- {[2, 3], [4, 5]}
```

- Round to a number of decimal places

```
round({floatset,floatspans},integer=0) → {floatset,floatspans}
```

```
SELECT round(floatset '{1.123456789,2.123456789}', 3);
-- {1.123, 2.123}
SELECT round(floatspan '[1.123456789,2.123456789)', 3);
-- [1.123,2.123)
SELECT round(floatspan '[1.123456789, 1000000)', 3);
-- [1.123, 1000000)
SELECT round(floatspanset '{[1.123456789, 2.123456789],[3.123456789,4.123456789]}', 3);
-- {[1.123, 2.123], [3.123, 4.123]}
```

- Convert to degrees or radians

```
degrees({floatset,floatspans},normalize boolean=false) → {floatset,floatspans}
radians({floatset,floatspans}) → {floatset,floatspans}
```

The additional parameter in the `degrees` function can be used to normalize the values between 0 and 360 degrees.

```
SELECT round(degrees(floatset '{0, 0.5, 0.7, 1.0}', true), 3);
-- {0, 28.648, 40.107, 57.296}
SELECT round(radians(floatspanset '{[0, 45], [90, 135]}'), 3);
-- {[0, 0.785], [1.571, 2.356]}
```

- Transform to lowercase, uppercase, or initcap

```
lower(textset) → textset
upper(textset) → textset
initcap(textset) → textset
```

```
SELECT lower(textset '{"AAA", "BBB", "CCC"}');
-- {"aaa", "bbb", "ccc"}
SELECT upper(textset '{"aaa", "bbb", "ccc"}');
-- {"AAA", "BBB", "CCC"}
SELECT initcap(textset '{"aaa", "bbb", "ccc"}');
-- {"Aaa", "Bbb", "Ccc"}
```

- Text concatenation

```
{text,textset} || {text,textset} → textset
```

```
SELECT textset '{aaa, bbb}' || text 'XX';
-- {"aaaXX", "bbbXX"}
SELECT text 'XX' || textset '{aaa, bbb}';
-- {"XXaaa", "XXbbb"}
```

- Set the temporal precision of the time value to the interval with respect to the origin

```
tprecision(times,duration interval,torigin timestamptz='2000-01-03') → times
```

If the origin is not specified, it is set by default to Monday, January 3, 2000

```
SELECT tprecision(timestamptz '2001-12-03', '30 days');
-- 2001-11-23
SELECT tprecision(timestamptz '2001-12-03', '30 days', '2001-12-01');
-- 2001-12-01
SELECT tprecision(tstzset '{2001-01-01 08:00, 2001-01-01 08:10, 2001-01-01 09:00,
  2001-01-01 09:10}', '1 hour');
-- {"2001-01-01 08:00:00", "2001-01-01 09:00:00"}
SELECT tprecision(tstzspan '[2001-12-01 08:00, 2001-12-01 09:00]', '1 day');
-- [2001-12-01, 2001-12-02)
SELECT tprecision(tstzspan '[2001-12-01 08:00, 2001-12-15 09:00]', '1 day');
-- [2001-12-01, 2001-12-16)
SELECT tprecision(tstzspanset '{[2001-12-01 08:00, 2001-12-01 09:00],
  [2001-12-01 10:00, 2001-12-01 11:00]}', '1 day');
-- {[2001-12-01, 2001-12-02)}
SELECT tprecision(tstzspanset '{[2001-12-01 08:00, 2001-12-01 09:00],
  [2001-12-01 10:00, 2001-12-01 11:00]}', '1 day');
-- {[2001-12-01, 2001-12-02)}
```

2.6 Spatial Reference System

- Return or set the spatial reference identifier

```
SRID(spatialset) → integer
setSRID(spatialset,integer) → spatialset
```

```
SELECT SRID(geomset '{"Point(1 1)", "Point(2 2)"}');
-- 0
SELECT SRID(geogset '{"Linestring(1 1,2 2)","Polygon((1 1,1 2,2 2,2 1,1 1))"}');
-- 4326
SELECT SRID(geomset 'SRID=5676;{"Linestring(1 1,2 2)","Polygon((1 1,1 2,2 2,2 1,1 1))"}');
-- 5676
SELECT asEWKT(setSRID(geomset '{Point(1 1), Point(2 2)}', 5676));
-- SRID=5676;{"POINT(1 1)", "POINT(2 2)"}
SELECT asEWKT(setSRID(poseset '{"Pose(Point(1 1),1)", "Pose(Point(2 2),3)"}', 5676));
-- SRID=5676;{"Pose(POINT(2 2),3)", "Pose(POINT(1 1),1)"}
SELECT SRID(cbufferset 'SRID=4326;
  {"Cbuffer(Point(1 1),0.5)", "Cbuffer(Point(2 2),1.0)"}');
-- 4326
SELECT asEWKT(setSRID(cbufferset '{"Cbuffer(Point(1 1),0.5)", "Cbuffer(Point(2 2),1.0)"}',
  3812));
-- SRID=3812;{"Cbuffer(POINT(1 1),0.5)", "Cbuffer(POINT(2 2),1)"}
```

- Transform to a spatial reference identifier

```
transform(spatialset,srid integer) → spatialset
transformPipeline(spatialset,pipeline text,srid integer,
  is_forward boolean=true) → spatialset
```

The `transform` function specifies the transformation with a target SRID. An error is raised when the input set has an unknown SRID (represented by 0). The `transformPipeline` function specifies the transformation with a coordinate transformation pipeline in the following format:

`urn:ogc:def:coordinateOperation:AUTHORITY::CODE`

The SRID of the input set is ignored, and the SRID of the output set will be set to zero unless a value is provided via the optional `srid` parameter. As stated by the last parameter, the pipeline is executed by default in a forward direction; by setting the parameter to false, the pipeline is executed in the inverse direction.

```
SELECT asEWKT(transform(geomset 'SRID=4326;
  {Point(2.340088 49.400250), Point(6.575317 51.553167)}', 3812), 6);
-- SRID=3812;{"POINT(502773.429981 511805.120402)", "POINT(803028.908265 751590.742629)"}
WITH test(geoset, pipeline) AS (
  SELECT geogset 'SRID=4326;
    {"Point(4.3525 50.846667 100.0)", "Point(-0.1275 51.507222 100.0)"}',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(geoset, pipeline, 4326), pipeline, 4326,
  false), 6) FROM test;
-- SRID=4326;{"POINT Z (4.3525 50.846667 100)", "POINT Z (-0.1275 51.507222 100)"}
```

2.7 Set Operations

The set and span types have associated set operators, namely union, difference, and intersection, which are represented, respectively by +, −, and ∗. The set operators for the set and span types are given next.

- Union, difference, or intersection of sets or spans

```
set {+, -, *} set → set
spans {+, -, *} spans → spans
```

```
SELECT dateset '{2001-01-01, 2001-01-03, 2001-01-05}' + dateset '{2001-01-03,
  2001-01-06}';
-- {2001-01-01, 2001-01-03, 2001-01-05, 2001-01-06}
SELECT intspan '[1, 3)' + intspan '[3, 5)';
-- [1, 5)
SELECT floatspan '[1, 3)' + floatspan '[4, 5)';
-- {[1, 3), [4, 5)}
SELECT tstzspanset '{[2001-01-01, 2001-01-03),
  [2001-01-04, 2001-01-05)}' + tstzspan '[2001-01-03, 2001-01-04)';
-- {[2001-01-01, 2001-01-05)}
```

```
SELECT intset '{1, 3, 5}' - intset '{3, 6}';
-- {1, 5}
SELECT datespan '[2001-01-01, 2001-01-05)' - datespan '[2001-01-03, 2001-01-07)';
-- {[2001-01-01, 2001-01-03)}
SELECT floatspan '[1, 5]' - floatspan '[3, 4]';
-- {[1, 3), (4, 5]}
SELECT tstzspanset '{[2001-01-01, 2001-01-06],
  [2001-01-07, 2001-01-10]}' - tstzspanset '{[2001-01-02, 2001-01-03],
  [2001-01-04, 2001-01-05], [2001-01-08, 2001-01-09]}';
/* {[2001-01-01,2001-01-02), (2001-01-03,2001-01-04), (2001-01-05,2001-01-06],
   [2001-01-07,2001-01-08), (2001-01-09,2001-01-10]} */
```

```
SELECT tstzset '{2001-01-01, 2001-01-03}' * tstzset '{2001-01-03, 2001-01-05}';
-- {2001-01-03}
SELECT intspan '[1, 5)' * intspan '[3, 6)';
-- [3, 5)
SELECT floatspanset '{[1, 5),[6, 8)}' * floatspan '[1, 6)';
-- {[1, 5)}
SELECT tstzspan '[2001-01-01, 2001-01-05)' * tstzspan '[2001-01-03, 2001-01-07)';
-- [2001-01-03, 2001-01-05)
```

2.8 Bounding Box Operations

2.8.1 Topological Operations

The topological operations available for the set and span types are given next.

A value, the span containing only that value, and the span set containing only that span denote the same set, and so do a span and the span set containing only it. Every operation gives the same answer for all of these spellings.

A span set is a union of disjoint spans. The operations `&&`, `@>`, `<@` and `~=` answer on the value itself, so a value that falls in a hole of a span set is not contained in it. The operation `-|-` and the position operations of the next section answer on the *bounding span* of the value, the smallest span containing it: the holes of a span set are not boundaries of it.

- Do the values overlap (have values in common)?

```
{set,spans} && {set,spans} → boolean
```

```
SELECT intset '{1, 3}' && intset '{2, 3, 4}';
-- true
SELECT floatspan '[1, 3)' && floatspan '[3, 4)';
-- false
SELECT tstzspan '[2001-01-01, 2001-01-05)' && tstzspan '[2001-01-02, 2001-01-07)';
-- true
SELECT floatspanset '{[1, 5),[6, 8)}' && floatspan '[1, 6)';
-- true
```

- Does the first value contain the second one?

```
{set,spans} @> {base,set,spans} → boolean
```

```
SELECT floatset '{1.5, 2.5}' @> 2.5;
-- true
SELECT tstzspan '[2001-01-01, 2001-05-01)' @> timestamptz '2001-02-01';
-- true
SELECT floatspanset '{[1, 2),(2, 3)}' @> 2.0;
-- false
```

- Is the first value contained by the second one?

```
{base,set,spans} <@ {set,spans} → boolean
```

```
SELECT timestamptz '2001-01-10' <@ tstzspan '[2001-01-01, 2001-05-01)';
-- true
SELECT floatspan '[2, 5]' <@ floatspan '[1, 5)';
-- false
SELECT tstzspan '[2001-02-01, 2001-03-01)' <@ tstzspan '[2001-01-01, 2001-05-01)';
-- true
SELECT floatspanset '{[1,2],[3,4]}' <@ floatspan '[1, 6]';
-- true
```

- Is the first value adjacent to the second one?

```
spans -|- spans → boolean
```

  Two values are adjacent when they touch without overlapping, that is, when they share a boundary and have no value in common inside it. For the continuous base types `float` and `timestamptz` a shared bound is such a touch. For the discrete base types `int`, `bigint` and `date`, whose spans are stored in normal form, it means consecutive values.

```
SELECT intspan '[2, 6)' -|- intspan '[6, 7)';
-- true
SELECT intspan '[1, 5]' -|- intspan '[6, 10]';
-- true
SELECT intspan '[1, 5]' -|- intspan '[5, 10]';
-- false
SELECT floatspan '[2, 5)' -|- floatspan '(5, 6)';
-- true
SELECT floatspan '[1, 5]' -|- floatspan '[5, 9]';
-- true
SELECT floatspanset '{[2, 3],[4, 5)}' -|- floatspan '(5, 6)';
-- true
SELECT tstzspanset '{[2001-01-01, 2001-01-02]}' -|- tstzspan '[2001-01-02, 2001-01-03)';
-- true
```

A span set answers as its bounding span, and the three spellings of a value agree:

```
SELECT intspanset '{[1,3), [5,8), [10,12)}' -|- 3;
-- false
SELECT intspanset '{[1,3), [5,8), [10,12)}' -|- intspan '[3,3]';
-- false
SELECT intspanset '{[1,3), [5,8), [10,12)}' -|- intspanset '{[3,3]}';
-- false
SELECT intspanset '{[1,3), [5,8), [10,12)}' -|- 12;
-- true
```

2.8.2 Position Operations

The position operations available for set and span types are given next. Notice that the operators for time types have an additional # to distinguish them from the operators for number types.

- Is the first value strictly left of the second one?

```
numbers << numbers → boolean
times <<# times → boolean
```

```
SELECT intspan '[15, 20)' << 20;
-- true
SELECT intspanset '{[15, 17],[18, 20)}' << 20;
-- true
SELECT floatspan '[15, 20)' << floatspan '(15, 20)';
-- false
SELECT dateset '{2001-01-01, 2001-01-02}' <<# dateset '{2001-01-03, 2001-01-05}';
-- true
```

- Is the first value strictly to the right of the second one?

```
numbers >> numbers → boolean
times #>> times → boolean
```

```
SELECT intspan '[15, 20)' >> 10;
-- true
SELECT floatspan '[15, 20)' >> floatspan '[5, 10]';
-- true
SELECT floatspanset '{[15, 17], [18, 20)}' >> floatspan '[5, 10]';
-- true
SELECT tstzspan '[2001-01-04, 2001-01-05)' #>>
  tstzspanset '{[2001-01-01, 2001-01-04), [2001-01-05, 2001-01-06)}';
-- true
```

- Is the first value not to the right of the second one?

```
numbers &< numbers → boolean
times &<# times → boolean
```

```
SELECT intspan '[15, 20)' &< 18;
-- false
SELECT intspanset '{[15, 16],[17, 18)}' &< 18;
-- true
SELECT floatspan '[15, 20)' &< floatspan '[10, 20]';
-- true
SELECT dateset '{2001-01-02, 2001-01-05}' &<# dateset '{2001-01-01, 2001-01-04}';
-- false
```

- Is the first value not to the left of the second one?

```
numbers &> numbers → boolean
times #&> times → boolean
```

```
SELECT intspan '[15, 20)' &> 30;
-- true
SELECT floatspan '[1, 6]' &> floatspan '(1, 3)';
-- false
SELECT floatspanset '{[1, 2],[3, 4]}' &> floatspan '(1, 3)';
-- false
SELECT timestamp '2001-01-01' #&> tstzspan '[2001-01-01, 2001-01-05)';
-- true
```

2.8.3 Splitting Operations

When creating indexes for set or span set types, what is stored in the index is not the actual value but instead, a bounding box that *represents* the value. In this case, the index will provide a list of candidate values that *may* satisfy the query predicate, and a second step is needed to filter out candidate values by computing the query predicate on the actual values.

However, when the bounding boxes have a large empty space not covered by the actual values, the index will generate many candidate values that do not satisfy the query predicate, which reduces the efficiency of the index. In these situations, it may be better to represent a value not with a *single* bounding box, but instead with *multiple* bounding boxes. This increases considerably the efficiency of the index, provided that the index is able to manage multiple bounding boxes per value. The following functions are used for generating multiple spans from a single set or span set value.

- Return an array of N spans obtained by merging the elements of a set or the spans of a spanset

```
splitNSpans({set,spanset},integer) → span[]
```

The last argument specifies the number of output spans. If the number of input elements or spans is less than the given number, the resulting array will have one span per input element or span. Otherwise, the given number of output spans will be obtained by merging several consecutive input elements or spans.

```
SELECT splitNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 1);
-- {"[1, 11)"}
SELECT splitNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 3);
-- {"[1, 5)","[5, 8)","[8, 11)"}
SELECT splitNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 6);
-- {"[1, 3)","[3, 5)","[5, 7)","[7, 9)","[9, 10)","[10, 11)"}
SELECT splitNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 12);
/* {"[1, 2)","[2, 3)","[3, 4)","[4, 5)","[5, 6)","[6, 7)","[7, 8)","[8, 9)",
   "[9, 10)","[10, 11)"} */
```

```
SELECT splitNSpans(intspanset '{[1, 2), [3, 4), [5, 6), [7, 8), [9, 10)}');
-- {"[1, 2)","[3, 4)","[5, 6)","[7, 8)","[9, 10)"}
SELECT splitNSpans(floatspanset '{[1, 2), [3, 4), [5, 6), [7, 8), [9, 10)}', 3);
-- {"[1, 4)","[5, 8)","[9, 10)"}
SELECT splitNSpans(datespanset '{[2001-01-01, 2001-01-04), [2001-01-05, 2001-01-10)}', 3);
-- {"[2001-01-01, 2001-01-04)","[2001-01-05, 2001-01-10)"}
```

- Return an array of spans obtained by merging N consecutive elements of a set or N consecutive spans of a spanset

```
splitEachNSpans({set,spanset},integer) → span[]
```

The last argument specifies the number of input elements that are merged to produce an output span. If the number of input elements is less than the given number, the resulting array will have one output span per element. Otherwise, the given number of consecutive input elements will be merged into a single output span in the answer. Notice that, contrary to the splitNSpans function, the number of spans in the result depends on the number of input elements or spans.

```
SELECT splitEachNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 1);
/* {"[1, 2)","[2, 3)","[3, 4)","[4, 5)","[5, 6)","[6, 7)","[7, 8)","[8, 9)",
   "[9, 10)","[10, 11)"} */
SELECT splitEachNSpans(intspanset '{[1, 2), [3, 4), [5, 6), [7, 8), [9, 10)}', 3);
-- {"[1, 6)","[7, 10)"}
SELECT splitEachNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 6);
-- {"[1, 7)","[7, 11)"}
SELECT splitEachNSpans(intset '{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}', 12);
-- {"[1, 11)"}
```

2.9 Distance Operations

The distance operator `<->` for set and span types consider the bounding span and returns the smallest distance between the two values. The distance has the type of the difference of two values, as the PostgreSQL minus operator answers it: the base type for numbers, the number of days as an `integer` for dates, and an `interval` for timestamps. The distance operator can also be used for nearest neighbor searches using a GiST or an SP-GiST index (see Section 2.12).

- Return the smallest distance ever

```
numbers <-> numbers → base
dates <-> dates → integer
times <-> times → interval
{base,spatialset} <-> {base,spatialset} → float
setDistance({geography,geogset},{geography,geogset},spheroid boolean=true) → float
```

For geographies, the operator computes on the WGS 84 spheroid, and the function `setDistance` computes on the sphere, faster, if its last argument is false.

```
SELECT 3 <-> intspan '[6, 8)';
-- 3
SELECT floatspan '[1, 3]' <-> floatspan '(5.5, 7]';
-- 2.5
SELECT floatspan '[1, 3]' <-> floatspanset '{(5.5, 7],[8, 9]}';
-- 2.5
SELECT tstzspan '[2001-01-02, 2001-01-06)' <-> timestamptz '2001-01-07';
-- 1 day
SELECT dateset '{2001-01-01, 2001-01-03, 2001-01-05}' <->
  dateset '{2001-01-02, 2001-01-04}';
--  0
SELECT round((geography 'Point(0 0)' <->
  geogset '{"Point(0 1)", "Point(0 2)"}')::numeric, 3);
-- 110574.389
```

```
SELECT round(setDistance(geography 'Point(0 0)', geogset '{"Point(0 1)", "Point(0 2)"}',
  false)::numeric, 3);
-- 111195.080
```

2.10 Comparisons

The comparison operators (=, <, and so on) require that the left and right arguments be of the same type. Excepted equality and inequality, the other comparison operators are not useful in the real world but allow B-tree indexes to be constructed on set and span types. For span values, the operators compare first the lower bound, then the upper bound. For set and span set values, the operators compare first the bounding spans, and if those are equal, they compare the first N values or spans, where N is the minimum of the number of composing values or spans of both values.

The comparison operators available for the set and span types are given next. Recall that integer spans are always represented by their canonical form.

- Traditional comparisons

```
set {=, <>, <, >, <=, >=} set → boolean
spans {=, <>, <, >, <=, >=} spans → boolean
cmp({set,span,spanset},{set,span,spanset}) → integer
```

  Function `cmp` returns -1, 0, or 1 depending on whether the first argument is, respectively, less than, equal to, or greater than the second one.

```
SELECT intspan '[1,3]' = intspan '[1,4)';
-- true
SELECT floatspanset '{[1, 2),[2,3)}' = floatspanset '{[1,3)}';
-- true
SELECT tstzset '{2001-01-01, 2001-01-04}' <> tstzset '{2001-01-01, 2001-01-05}';
-- false
SELECT tstzspan '[2001-01-01, 2001-01-04)' <> tstzspan '[2001-01-03, 2001-01-05)';
-- true
SELECT floatspan '[3, 4]' < floatspan '(3, 4]';
-- true
SELECT intspanset '{[1,2],[3,4]}' < intspanset '{[3, 4]}';
-- true
SELECT floatspan '[3, 4]' > floatspan '[3, 4)';
-- true
SELECT tstzspan '[2001-01-03, 2001-01-04)' > tstzspan '[2001-01-02, 2001-01-05)';
-- true
SELECT floatspanset '{[1, 4)}' <= floatspanset '{[1, 5), [6, 7)}';
-- true
SELECT tstzspanset '{[2001-01-01, 2001-01-04)}' <= tstzspanset '{[2001-01-01,
  2001-01-05), [2001-01-06, 2001-01-07)}';
-- true
SELECT tstzspan '[2001-01-03, 2001-01-05)' >= tstzspan '[2001-01-03, 2001-01-04)';
-- true
SELECT intspanset '{[1, 4)}' >= intspanset '{[1, 5), [6, 7)}';
-- false
SELECT cmp(intspanset '{[1, 4)}', intspanset '{[1, 5), [6, 7)}');
-- -1
```

- Hash values

```
hash({set,span,spanset}) → integer
hashExtended({set,span,spanset},seed bigint) → bigint
```

  Function `hash` returns a 32-bit hash value and function `hashExtended` a 64-bit hash value computed with the given seed. Equal values have equal hash values, which is what hash indexes, hash joins, and hash aggregation rely on.

```
SELECT hash(intspan '[1,3]') = hash(intspan '[1,4)');
-- true
SELECT hash(intspanset '{[1,2],[3,4]}') = hash(intspanset '{[1,3),[3,5)}');
-- true
SELECT hash(tstzset '{2001-01-01, 2001-01-04}') =
  hash(tstzset '{2001-01-04, 2001-01-01}');
-- true
SELECT hashExtended(tstzspan '[2001-01-01, 2001-01-04)', 0) <>
  hashExtended(tstzspan '[2001-01-01, 2001-01-04)', 1);
-- true
```

2.11 Aggregations

There are several aggregate functions defined for set and span types. They are described next.

- Function `extent` returns a bounding span that encloses a set of set or span values.
- Union is a very useful operation for set and span types. As we have seen in Section 2.7, we can compute the union of two set or span values using the + operator. However, it is also very useful to have an aggregate version of the union operator for combining an arbitrary number of values. Functions `setUnion`, `spanUnion`, and `spansetUnion` can be used for this purpose.
- Function `tCount` generalizes the traditional aggregate function `count`. The temporal count can be used to compute at each point in time the number of available objects (for example, number of spans). Function `tCount` returns a temporal integer (see Chapter 4). The function has two optional parameters that specify the granularity (an `interval`) and the origin of time (a `timestamptz`). When these parameters are given, the temporal count is computed at time bins of the given granularity (see Section 9.7).

- Bounding span

```
extent({set,spans}) → span
```

```
WITH spans(r) AS (
  SELECT floatspan '[1, 4)' UNION SELECT floatspan '(5, 8)' UNION
  SELECT floatspan '(7, 9)' )
SELECT extent(r) FROM spans;
-- [1,9)
WITH times(ts) AS (
  SELECT tstzset '{2001-01-01, 2001-01-03, 2001-01-05}' UNION
  SELECT tstzset '{2001-01-02, 2001-01-04, 2001-01-06}' UNION
  SELECT tstzset '{2001-01-01, 2001-01-02}' )
SELECT extent(ts) FROM times;
-- [2001-01-01, 2001-01-06]
WITH periods(ps) AS (
  SELECT tstzspanset '{[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04]}' UNION
  SELECT tstzspanset '{[2001-01-01, 2001-01-04], [2001-01-05, 2001-01-06]}' UNION
  SELECT tstzspanset '{[2001-01-02, 2001-01-06]}' )
SELECT extent(ps) FROM periods;
-- [2001-01-01, 2001-01-06]
```

- Aggregate union

```
setUnion({base,set}) → set
spanUnion(spans) → spanset
spansetUnion(spanset) → spanset
```

```
WITH times(ts) AS (
  SELECT tstzset '{2001-01-01, 2001-01-03, 2001-01-05}' UNION
  SELECT tstzset '{2001-01-02, 2001-01-04, 2001-01-06}' UNION
  SELECT tstzset '{2001-01-01, 2001-01-02}' )
SELECT setUnion(ts) FROM times;
-- {2001-01-01, 2001-01-02, 2001-01-03, 2001-01-04, 2001-01-05, 2001-01-06}
WITH periods(ps) AS (
  SELECT tstzspanset '{[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04]}' UNION
  SELECT tstzspanset '{[2001-01-02, 2001-01-03], [2001-01-05, 2001-01-06]}' UNION
  SELECT tstzspanset '{[2001-01-07, 2001-01-08]}' )
SELECT spansetUnion(ps) FROM periods;
-- {[2001-01-01, 2001-01-04], [2001-01-05, 2001-01-06], [2001-01-07, 2001-01-08]}
```

- Temporal count

```
tCount(times) → {tintSeq,tintSeqSet}
```

```
WITH times(ts) AS (
  SELECT tstzset '{2001-01-01, 2001-01-03, 2001-01-05}' UNION
  SELECT tstzset '{2001-01-02, 2001-01-04, 2001-01-06}' UNION
  SELECT tstzset '{2001-01-01, 2001-01-02}' )
SELECT tCount(ts) FROM times;
-- {2@2001-01-01, 2@2001-01-02, 1@2001-01-03, 1@2001-01-04, 1@2001-01-05, 1@2001-01-06}
WITH periods(ps) AS (
  SELECT tstzspanset '{[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-04)}' UNION
  SELECT tstzspanset '{[2001-01-01, 2001-01-04), [2001-01-05, 2001-01-06)}' UNION
  SELECT tstzspanset '{[2001-01-02, 2001-01-06)}' )
SELECT tCount(ps) FROM periods;
-- {[2@2001-01-01, 3@2001-01-03, 1@2001-01-04, 2@2001-01-05, 2@2001-01-06)}
```

2.12 Indexing

GiST and SP-GiST indexes can be created for table columns of the set and span types. The GiST index implements an R-tree while the SP-GiST index implements a quad-tree. An example of creation of a GiST index in a column `During` of type `tstzspan` in a table `Reservation` is as follows:

```
CREATE TABLE Reservation (ReservationID integer PRIMARY KEY, RoomID integer,
  During tstzspan);
CREATE INDEX Reservation_During_Idx ON Reservation USING GIST(During);
```

A GiST or an SP-GiST index can accelerate queries involving the following operators: `=`, `&&`, `<@`, `@>`, `-|-`, `<<`, `>>`, `&<`, `&>`, `<<#`, `#>>`, `&<#`, `#&>`, and `<->`.

A spatial set, that is, a set of geometries, geographies, network points, poses, pose chains, circular buffers, H3 cells, quadbin cells, or S2 cells, is indexed by the spatiotemporal box it stores rather than by a span. Its GiST index implements an R-tree and its SP-GiST index a quad-tree, or a k-d tree when the operator class is named, as in `USING SPGIST(Stops geomset_kdtree_ops)`. Such an index can accelerate queries involving the operators `=`, `&&`, `<@`, and `@>`, and for the sets whose elements have a distance, `<->`. Since the box bounds the elements of a set without stating them, the index returns every set whose box satisfies the operator, and the operator is then checked on the set.

In addition, B-tree indexes can be created for table columns of a set or span types. For these index types, basically the only useful operation is equality. There is a B-tree sort ordering defined for values of span time types with corresponding `<`, `<=`, `>`, and `>=` operators, but the ordering is rather arbitrary and not usually useful in the real world. The B-tree support is primarily meant to allow sorting internally in queries, rather than creation of actual indexes.

Finally, hash indexes can be created for table columns of a set or span types. For these types of indexes, the only operation defined is equality.

Chapter 3

Bounding Box Types

We present next the functions and operators for bounding box types. These functions and operators are polimorhic, that is, their argumentos can be of various types and their result type may depend on the type of the arguments. To express this in the signature of the operators, we use the following notation:

- `box` represents any bounding box type, that is, `tbox` or `stbox`,
- `number` represents any number base type, that is, `integer`, `bigint`, or `float`,
- `numspan` represents any number span type, that is, `intspan`, `bigintspan`, or `floatspan`,
- `numbers` represents any number set or range type, such as `integer`, `intset`, `intspan`, or `intspanset`,
- `times` represents any time type with `timestamptz` granularity, that is, `timestamptz`, `tstzset`, `tstzspan`, or `tstzspanset`,
- `tnumber` represents any temporal number type, that is, `tint`, `tbigint`, or `tfloat`,
- `geo` represents the types `geometry` or `geography`,
- `tgeo` represents a temporal geometry/geography type, that is, `tgeometry`, `tgeography`, `tgeompoint`, or `tgeogpoint`.

3.1 Input and Output

MobilityDB generalizes Open Geospatial Consortium's Well-Known Text (WKT) and Well-Known Binary (WKB) input and output format for all temporal types. We present next the functions for input and output box types.

A `tbox` is composed of a numeric and/or time dimensions. For each dimension, a span given, that is, one of an `intspan`, `bigintspan`, or a `floatspan` for the value dimension and a `tstzspan` for the time dimension. Examples of input of `tbox` values are as follows:

```
-- Both value and time dimensions
SELECT tbox 'TBOXINT XT([1,3),[2001-01-01,2001-01-02])';
SELECT tbox 'TBOXBIGINT XT([1,3),[2001-01-01,2001-01-02])';
SELECT tbox 'TBOXFLOAT XT([1.5,2.5],[2001-01-01,2001-01-02])';
-- Only value dimension
SELECT tbox 'TBOXINT X([1,3))';
SELECT tbox 'TBOXBIGINT X([1,3))';
SELECT tbox 'TBOXFLOAT X((1.5,2.5))';
-- Only time dimension
SELECT tbox 'TBOX T((2001-01-01,2001-01-02))';
```

An `stbox` is composed of a spatial and/or time dimensions, where the coordinates of the spatial dimension may be 2D or 3D. For the time dimension a `tstzspan` is given and for the spatial dimension minimum and maximum coordinate values are given, where the latter may be Cartesian (planar) or geodetic (spherical). The SRID of the coordinates may be specified; if it is not the case, a value of 0 (unknown) and 4326 (corresponding to WGS84) is assumed, respectively, for planar and geodetic boxes. Geodetic boxes always have a Z dimension to account for the curvature of the underlying sphere or spheroid. Examples of input of `stbox` values are as follows:

```
-- Only value dimension with X and Y coordinates
SELECT stbox 'STBOX X((1.0,2.0),(1.0,2.0))';
-- Only value dimension with X, Y, and Z coordinates
SELECT stbox 'STBOX Z((1.0,2.0,3.0),(1.0,2.0,3.0))';
-- Both value (with X and Y coordinates) and time dimensions
SELECT stbox 'STBOX XT(((1.0,2.0),(1.0,2.0)),[2001-01-03,2001-01-03])';
-- Both value (with X, Y, and Z coordinates) and time dimensions
SELECT stbox 'STBOX ZT(((1.0,2.0,3.0),(1.0,2.0,3.0)),[2001-01-01,2001-01-03])';
-- Only time dimension
SELECT stbox 'STBOX T([2001-01-03,2001-01-03])';
-- Only value dimension with X, Y, and Z geodetic coordinates
SELECT stbox 'GEODSTBOX Z((1.0,2.0,3.0),(1.0,2.0,3.0))';
-- Both value (with X, Y and Z geodetic coordinates) and time dimension
SELECT stbox 'GEODSTBOX ZT(((1.0,2.0,3.0),(1.0,2.0,3.0)),[2001-01-04,2001-01-04])';
-- Only time dimension for geodetic box
SELECT stbox 'GEODSTBOX T([2001-01-03,2001-01-03])';
-- SRID is given
SELECT stbox 'SRID=5676;STBOX XT(((1.0,2.0),(1.0,2.0)),[2001-01-04,2001-01-04])';
SELECT stbox 'SRID=4326;GEODSTBOX Z((1.0,2.0,3.0),(1.0,2.0,3.0))';
```

We give next the functions for input and output of box types in Well-Known Text (WKT) and Well-Known Binary (WKB) format.

- Return the Well-Known Text (WKT) representation

  ```
  asText(box,maxdecimaldigits integer=15) → text
  ```

  The `maxdecimaldigits` argument can be used to set the maximum number of decimal places in the output of floating point values (default 15). As for every spatial type, the text of an `stbox` does not write its SRID, which `asEWKT` writes.

  ```
  SELECT asText(tbox 'TBOXFLOAT XT([1.123456789,2.123456789),[2001-01-01,2001-01-02))', 3);
  -- TBOXFLOAT XT([1.123, 2.123),[2001-01-01, 2001-01-02))
  SELECT asText(stbox 'STBOX Z((1.55,1.55,1.55),(2.55,2.55,2.55))', 0);
  -- STBOX Z((2,2,2),(3,3,3))
  SELECT asText(stbox 'SRID=3812;STBOX X((1,1),(2,2))');
  -- STBOX X((1,1),(2,2))
  ```

- Return the Extended Well-Known Text (EWKT) representation of a spatiotemporal box

  ```
  asEWKT(stbox,maxdecimaldigits integer=15) → text
  ```

  The result is the text `asText` writes prefixed with the SRID of the box, as every spatial type writes its SRID in its extended representation.

  ```
  SELECT asEWKT(stbox 'SRID=3812;STBOX X((1,1),(2,2))');
  -- SRID=3812;STBOX X((1,1),(2,2))
  SELECT asEWKT(stbox 'SRID=4326;GEODSTBOX Z((1.55,1.55,1.55),(2.55,2.55,2.55))', 0);
  -- SRID=4326;GEODSTBOX Z((2,2,2),(3,3,3))
  ```

- Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation

  ```
  asBinary(box,endian text='') → bytea
  asHexWKB(box,endian text='') → text
  ```

The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used. As for every spatial type, the representation of an `stbox` does not write its SRID, which `asEWKB` and `asHexEWKB` write.

```
SELECT asBinary(tbox 'TBOXFLOAT XT([1,2),[2001-01-01,2001-01-02))');
-- \x0103270001009c57d3c11c000000fc2ef1d51c00000d0001000000000000f03f0000000000000040
SELECT asBinary(tbox 'TBOXFLOAT XT([1,2),[2001-01-01,2001-01-02))', 'XDR');
-- \x000300270100001cc1d3579c0000001cd5f12efc00000d013ff0000000000000400000000000000000
SELECT asBinary(stbox 'STBOX X((1,1),(2,2))');
-- \x0101000000000000f03f0000000000000040000000000000f03f0000000000000040
SELECT asHexWKB(tbox 'TBOXFLOAT XT([1,2),[2001-01-01,2001-01-02))');
-- 0103270001009C57D3C11C000000FC2EF1D51C00000D0001000000000000F03F0000000000000040
SELECT asHexWKB(tbox 'TBOXFLOAT XT([1,2)[2001-01-01,2001-01-02))', 'XDR');
-- 000300270100001CC1D3579C0000001CD5F12EFC00000D013FF0000000000000400000000000000000
SELECT asHexWKB(stbox 'STBOX X((1,1),(2,2))');
-- 0101000000000000F03F0000000000000040000000000000F03F0000000000000040
```

- Return the Extended Well-Known Binary (EWKB) or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation of a spatiotemporal box

```
asEWKB(stbox,endian text='') → bytea
asHexEWKB(stbox,endian text='') → text
```

The result is the representation `asBinary` and `asHexWKB` write with the SRID of the box, as every spatial type writes its SRID in its extended representation.

```
SELECT asEWKB(stbox 'SRID=3812;STBOX X((1,1),(2,2))');
-- \x0141e40e0000000000000000f03f0000000000000040000000000000f03f0000000000000040
SELECT asHexEWKB(stbox 'SRID=3812;STBOX X((1,1),(2,2))', 'XDR');
-- 004100000EE43FF000000000000040000000000000003FF00000000000004000000000000000
```

- Input from the Well-Known Text (WKT) representation

```
boxFromText(text) → box
```

In the signature above, `box` replaces any box type, that is, `tbox` or `stbox`. The function is the inverse of `asText`: it reads the text a literal of the type is written in, from a constant or from a `text` column alike, including the SRID of a spatiotemporal box.

```
SELECT tboxFromText('TBOXFLOAT XT([1,2],[2001-01-01,2001-01-02])');
-- TBOXFLOAT XT([1, 2],[2001-01-01, 2001-01-02])
SELECT stboxFromText('SRID=3812;STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])');
-- SRID=3812;STBOX XT(((1,1),(2,2)),[2001-01-01, 2001-01-02])
```

- Input of a spatiotemporal box from the Extended Well-Known Text (EWKT) representation

```
stboxFromEWKT(text) → stbox
```

The function is the inverse of `asEWKT`, reading the SRID the text states.

```
SELECT stboxFromEWKT('SRID=3812;STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])');
-- SRID=3812;STBOX XT(((1,1),(2,2)),[2001-01-01, 2001-01-02])
```

- Input from the Well-Known Binary (WKB) or from the Hexadecimal Well-Known Binary (HexWKB) representation

```
boxFromBinary(bytea) → box
boxFromHexWKB(text) → box
```

In the signatures above, `box` replaces any box type, that is, `tbox` or `stbox`.

```
SELECT tboxFromBinary(
  '\x0103270001009c57d3c11c000000fc2ef1d51c00000d0001000000000000f03f0000000000000040');
-- TBOXFLOAT XT([1,2),[2001-01-01,2001-01-02))
SELECT stboxFromBinary(
  '\x0101000000000000f03f0000000000000040000000000000f03f0000000000000040');
--  STBOX X((1,1),(2,2))
SELECT tboxFromHexWKB(
  '0103270001009C57D3C11C000000FC2EF1D51C00000D0001000000000000F03F0000000000000040');
-- TBOXFLOAT XT([1,2),[2001-01-01,2001-01-02)))
SELECT stboxFromHexWKB(
  '0101000000000000F03F0000000000000040000000000000F03F0000000000000040');
-- STBOX X((1,1),(2,2))
```

- Input of a spatiotemporal box from the Extended Well-Known Binary (EWKB) or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

```
stboxFromEWKB(bytea) → stbox
stboxFromHexEWKB(text) → stbox
```

```
SELECT stboxFromEWKB(asEWKB(stbox 'SRID=3812;STBOX X((1,1),(2,2))'));
-- SRID=3812;STBOX X((1,1),(2,2))
SELECT stboxFromHexEWKB(
  '0141E40E0000000000000000F03F0000000000000040000000000000F03F0000000000000040');
-- SRID=3812;STBOX X((1,1),(2,2))
```

3.2 Constructors

Type `tbox` has several constructor functions depending on whether the value and/or the time extent are given. The value extent can be specified by a number or a span, while the time extent can be specified by a time type.

- Constructor for `tbox`

```
tbox({number,numspan}) → tbox
tbox({timestamptz,tstzspan}) → tbox
tbox({number,numspan},{timestamptz,tstzspan}) → tbox
```

```
-- Both value and time dimensions
SELECT tbox(1.0, timestamptz '2001-01-01');
SELECT tbox(floatspan '[1.0, 2.0)', tstzspan '[2001-01-01,2001-01-02)');
-- Only value dimension
SELECT tbox(floatspan '[1.0,2.0)');
-- Only time dimension
SELECT tbox(tstzspan '[2001-01-01,2001-01-02)');
```

Type `stbox` has several constructor functions depending on whether the space and/or the time extent are given. The coordinates for the spatial extent can be 2D or 3D and can be either Cartesian or geodetic. The spatial extent can be specified by the minimum and maximum coordinate values. The SRID can be specified in an optional last argument. If not given, a value 0 (respectively 4326) is assumed by default for planar (respectively geodetic) boxes. The spatial extent can also be specified by a geometry or a geography. The temporal extent can be specified by a time type.

- Constructor for `stbox`

```
stboxX(float,float,float,float,srid integer=0) → stbox
stboxZ(float,float,float,float,float,float,srid integer=0) → stbox
stboxT({timestamptz,tstzspan}) → stbox
```

```
stboxXT(float,float,float,float,{timestamptz,tstzspan},srid integer=0) → stbox
stboxZT(float,float,float,float,float,float,{timestamptz,tstzspan},
  srid integer=0) → stbox
geodstboxZ(float,float,float,float,float,float,srid integer=4326) → stbox
geodstboxZT(float,float,float,float,float,float,{timestamptz,tstzspan},
  srid integer=4326) → stbox
stbox(geo) → stbox
stbox(geo,{timestamptz,tstzspan}) → stbox
```

```
-- Only value dimension with X and Y coordinates
SELECT stboxX(1.0,2.0,1.0,2.0);
-- Only value dimension with X, Y, and Z coordinates
SELECT stboxZ(1.0,2.0,3.0,1.0,2.0,3.0);
-- Only value dimension with X, Y, and Z coordinates and SRID
SELECT stboxZ(1.0,2.0,3.0,1.0,2.0,3.0,5676);
-- Only time dimension
SELECT stboxT(tstzspan '[2001-01-03,2001-01-03]');
-- Both value (with X and Y coordinates) and time dimensions
SELECT stboxXT(1.0,2.0, 1.0,2.0, tstzspan '[2001-01-03,2001-01-03]');
-- Both value (with X, Y, and Z coordinates) and time dimensions
SELECT stboxZT(1.0,2.0,3.0, 1.0,2.0,3.0, tstzspan '[2001-01-03,2001-01-03]');
-- Only value dimension with X, Y, and Z geodetic coordinates
SELECT geodstboxZ(1.0,2.0,3.0,1.0,2.0,3.0);
-- Both value (with X, Y, and Z geodetic coordinates) and time dimensions
SELECT geodstboxZT(1.0,2.0,3.0, 1.0,2.0,3.0, tstzspan '[2001-01-03,2001-01-04]');
-- Geometry and time dimension
SELECT stbox(geometry 'Linestring(1 1 1,2 2 2)', tstzspan '[2001-01-03,2001-01-05]');
-- Geography and time dimension
SELECT stbox(geography 'Linestring(1 1 1,2 2 2)', tstzspan '[2001-01-03,2001-01-05]');
```

3.3 Conversions

- Convert a `tbox` to another type

```
tbox::{intspan,floatspan,tstzspan}
intspan(tbox) → tbox
floatspan(tbox) → tbox
tstzspan(tbox) → tbox
```

```
SELECT tbox 'TBOXINT XT([1,4),[2001-01-01,2001-01-02))'::intspan;
-- [1,4)
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02))'::floatspan;
-- (1, 2)
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02))'::tstzspan;
-- [2001-01-01, 2001-01-02)
```

- Convert another type to a `tbox`

```
{numbers,times,tnumber}::tbox
tbox({numbers,times,tnumber}) → tbox
```

```
SELECT intset '{1,2}'::tbox;
-- TBOXINT X([1, 3))
SELECT intspan '[1,3)'::tbox;
-- TBOXINT X([1, 3))
SELECT floatspan '(1.0,2.0)'::tbox;
-- TBOXFLOAT X((1, 2))
SELECT tstzspanset '{(2001-01-01,2001-01-02),(2001-01-03,2001-01-04)}'::tbox;
-- TBOX T((2001-01-01,2001-01-04))
```

- Convert an `stbox` to a another type

```
stbox::{box2d,box3d,geo,tstzspan}
box2d(stbox) → box2d
box3d(stbox) → box3d
geometry(stbox) → geometry
geography(stbox) → geography
tstzspan(stbox) → tstzspan
```

The conversions follow the rule of [?listitem].

```
SELECT stbox 'STBOX XT(((1.0,2.0),(3.0,4.0)),[2001-01-01,2001-01-03])'::box2d;
-- BOX(1 2,3 4)
SELECT ST_AsEWKT(stbox 'SRID=4326;
  STBOX XT(((1,1),(5,5)),[2001-01-01,2001-01-05])':: geometry);
-- SRID=4326;POLYGON((1 1,1 5,5 5,5 1,1 1))
SELECT ST_AsEWKT(stbox 'STBOX XT(((1,1),(1,5)),[2001-01-01,2001-01-05])'::geometry);
-- LINESTRING(1 1,1 5)
SELECT ST_AsEWKT(stbox 'GEODSTBOX XT(((1,1),(1,1)),[2001-01-01,2001-01-05])'::geography);
-- SRID=4326;POINT(1 1)
SELECT ST_AsEWKT(stbox 'STBOX ZT(((1,1,1),(5,5,5)),[2001-01-01,2001-01-05])':: geometry);
/* POLYHEDRALSURFACE(((1 1 1,1 5 1,5 5 1,5 1 1,1 1 1)),
    ((1 1 5,5 1 5,5 5 5,1 5 5,1 1 5)),((1 1 1,1 1 5,1 5 5,1 5 1,1 1 1)),
    ((5 1 1,5 5 1,5 5 5,5 1 5,5 1 1)),((1 1 1,5 1 1,5 1 5,1 1 5,1 1 1)),
    ((1 5 1,1 5 5,5 5 5,5 5 1,1 5 1))) */
SELECT stbox 'STBOX XT(((1.0,2.0),(3.0,4.0)),[2001-01-01,2001-01-03])'::tstzspan;
-- [2001-01-01, 2001-01-03]
```

- Convert another type to an `stbox`

```
{box2d,box3d,geo,times,tgeo}::stbox
stbox({box2d,box3d,geo,times,tgeo}) → stbox
```

```
SELECT geometry 'Linestring(1 1 1,2 2 2)'::box3d::stbox;
-- STBOX Z((1,1,1),(2,2,2))
SELECT geography 'Linestring(1 1,2 2)'::stbox;
-- SRID=4326;GEODSTBOX X((1,1),(2,2))
SELECT tstzspanset '{(2001-01-01,2001-01-02),(2001-01-03,2001-01-04)}'::stbox;
-- STBOX T((2001-01-01,2001-01-04))
```

The box of a geodetic temporal point holds the great circles it travels between its positions, which rise above them or fall below them: a trip from `Point(10 60)` to `Point(50 60)` reaches latitude 61.518762, and one whose positions lie more than half the globe apart in longitude crosses the antimeridian and spans every longitude.

```
SELECT stbox(tgeogpoint '[Point(10 60)@2001-01-01, Point(50 60)@2001-01-02]');
-- SRID=4326;GEODSTBOX XT(((10,60),(50,61.518762)),[2001-01-01, 2001-01-02])
SELECT stbox(tgeogpoint '[Point(-170 60)@2001-01-01, Point(170 60)@2001-01-02]');
-- SRID=4326;GEODSTBOX XT(((-180,60),(180,60.378349)),[2001-01-01, 2001-01-02])
```

3.4 Accessors

- Has X/Z/T dimension?

```
hasX(box) → boolean
hasZ(stbox) → boolean
hasT(box) → boolean
```

```
SELECT hasX(tbox 'TBOX T([2001-01-01,2001-01-03))');
-- false
SELECT hasX(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- true
SELECT hasZ(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- false
SELECT hasT(tbox 'TBOXFLOAT XT((1.0,3.0),[2001-01-01,2001-01-03])');
-- true
SELECT hasT(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- false
```

- Is geodetic?

```
isGeodetic(stbox) → boolean
```

```
SELECT isGeodetic(stbox 'GEODSTBOX Z((1.0,1.0,0.0),(3.0,3.0,1.0))');
-- true
SELECT isGeodetic(stbox 'STBOX XT(((1.0,2.0),(3.0,4.0)),[2001-01-01,2001-01-02])');
-- false
```

- Return the minimum X/Y/Z/T value

```
xMin(box) → float
yMin(stbox) → float
zMin(stbox) → float
tMin(box) → timestamptz
```

```
SELECT xMin(tbox 'TBOXFLOAT XT((1.0,3.0),[2001-01-01,2001-01-03))');
-- 1
SELECT yMin(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- 2
SELECT zMin(stbox 'STBOX Z((1.0,2.0,3.0),(4.0,5.0,6.0))');
-- 3
SELECT tMin(stbox 'GEODSTBOX T([2001-01-01,2001-01-03))');
-- 2001-01-01
```

Notice that for `tbox` the result value for `xMin` and `xMax` is converted to a `float` for the temporal boxes with an integer span. A big integer bound that no `float` represents exactly is refused rather than rounded; read such a bound from `bigintspan(box)`.

- Return the maximum X/Y/Z/T value

```
xMax(box) → float
yMax(stbox) → float
zMax(stbox) → float
tMax(box) → timestamptz
```

```
SELECT xMax(tbox 'TBOXINT X([1,4))');
-- 3
SELECT yMax(stbox 'STBOX X((1.0,2.0),(3.0,4.0))');
-- 4
SELECT zMax(stbox 'STBOX Z((1.0,2.0,3.0),(4.0,5.0,6.0))');
-- 6
SELECT tMax(stbox 'GEODSTBOX T([2001-01-01,2001-01-03))');
-- 2001-01-03
```

Notice that for `tbox` the result value for `xMin` and `xMax` is converted to a `float` for the temporal boxes with an integer span. A big integer bound that no `float` represents exactly is refused rather than rounded; read such a bound from `bigintspan(box)`.

- Is the minimum X/T value inclusive?

```
xMinInc(tbox) → boolean
tMinInc(box) → boolean
```

```
SELECT xMinInc(tbox 'TBOXFLOAT XT((1.0,3.0),[2001-01-01,2001-01-03))');
-- false
SELECT tMinInc(stbox 'GEODSTBOX T([2001-01-01,2001-01-03))');
-- true
```

- Is the maximum X/T value inclusive?

```
xMaxInc(tbox) → boolean
tMaxInc(box) → boolean
```

```
SELECT xMaxInc(tbox 'TBOXFLOAT XT((1.0,3.0),[2001-01-01,2001-01-03))');
-- false
SELECT tMaxInc(stbox 'GEODSTBOX T([2001-01-01,2001-01-03))');
-- true
```

- Area, volume, perimeter

```
area(stbox,spheroid boolean=true) → float
volume(stbox) → float
perimeter(stbox,spheroid boolean=true) → float
```

For geodetic boxes, the computation is done on the WGS 84 spheroid by default. If the last argument is false, a faster spherical calculation is used. The `volume` function do not accept geodetic boxes.

```
SELECT area(stbox 'STBOX XT(((1,1),(3,3)),[2001-01-01,2001-01-03))');
-- 4
SELECT volume(stbox 'STBOX ZT(((1,1,1),(3,3,3)),[2001-01-01,2001-01-03))');
-- 8
SELECT perimeter(stbox 'STBOX XT(((1,1),(3,3)),[2001-01-01,2001-01-03))');
-- 8
SELECT area(stbox 'GEODSTBOX XT(((1,1),(3,3)),[2001-01-01,2001-01-03))');
-- 49209676328.36632
SELECT perimeter(stbox 'GEODSTBOX XT(((1,1),(3,3)),[2001-01-01,2001-01-03))', false);
-- 889221.9544681838
```

3.5 Transformations

- Shift and/or scale the span of a bounding box by one or two values

```
shiftValue(tbox,number) → tbox
scaleValue(tbox,number) → tbox
shiftScaleValue(tbox,number,number) → tbox
```

```
SELECT shiftValue(tbox 'TBOXFLOAT XT([1.5, 2.5],[2001-01-01,2001-01-02])', 1.0);
-- TBOXFLOAT XT([2.5, 3.5],[2001-01-01, 2001-01-02])
SELECT scaleValue(tbox 'TBOXFLOAT XT([1.5, 2.5],[2001-01-01,2001-01-02])', 2.0);
-- TBOXFLOAT XT([1.5, 3.5],[2001-01-01, 2001-01-02])
SELECT shiftScaleValue(tbox 'TBOXFLOAT XT([1.5, 2.5],[2001-01-01,2001-01-02])', 2.0, 3.0);
-- TBOXFLOAT XT([3.5, 6.5],[2001-01-01, 2001-01-02])
```

- Shift and/or scale the span or the period of the bounding box to a value or interval

```
shiftTime(box,interval) → box
scaleTime(box,interval) → box
shiftScaleTime(box,interval,interval) → box
```

For scaling, if the width or the time span of the period is zero (that is, the lower and upper bound are equal), the result is the box. The given value or interval must be strictly greater than zero.

```
SELECT shiftTime(tbox 'TBOXFLOAT XT([1.5, 2.5],[2001-01-01,2001-01-02])',
  interval '1 day');
-- TBOXFLOAT XT([1.5, 2.5],[2001-01-02, 2001-01-03])
SELECT shiftTime(stbox 'STBOX T([2001-01-01,2001-01-02])', interval '-1 day');
-- STBOX T([2000-12-31, 2001-01-01])
SELECT shiftTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-02])',
  interval '1 day');
-- STBOX ZT(((1,1,1),(2,2,2)),[2001-01-02, 2001-01-03])
SELECT scaleTime(tbox 'TBOXFLOAT XT([1.5, 2.5],[2001-01-01,2001-01-02])',
  interval '2 days');
-- TBOXFLOAT XT([1.5, 2.5],[2001-01-01, 2001-01-03])
SELECT scaleTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-02])',
  interval '1 hour');
-- STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01 00:00:00, 2001-01-01 01:00:00])
SELECT scaleTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-02])',
  interval '-1 day');
-- ERROR:  The interval must be positive: -1 days
SELECT shiftScaleTime(tbox 'TBOXFLOAT XT([1.5, 2.5],[2001-01-01,2001-01-02])',
  interval '1 day', interval '3 days');
-- TBOXFLOAT XT([1.5, 2.5],[2001-01-02, 2001-01-05])
SELECT shiftScaleTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-02])',
  interval '1 hour', interval '3 hours');
-- STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01 01:00:00, 2001-01-01 04:00:00])
```

- Return the spatial dimension of the bounding box, removing the temporal dimension if any

```
getSpace(stbox) → stbox
```

```
SELECT getSpace(stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-03])');
-- STBOX Z((1,1,1),(2,2,2))
```

The functions given next expand the bounding boxes on the value and the time dimension or set the precision of the value dimension. These functions raise an error if the corresponding dimension is not present.

- Expand the numeric, spatial, or temporal dimension of a bounding box by a value or an interval

```
expandValue(tbox,number) → tbox
expandSpace(stbox,float) → stbox
expandTime(box,interval) → box
```

The function returns NULL if the value or interval given as second argument is negative and the span resulting from shifting the bounds with the argument is empty.

```
SELECT expandValue(tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-03])', 1.0);
-- TBOXFLOAT XT((0,3),[2001-01-01,2001-01-03])
SELECT expandValue(tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-03])', -1.0);
-- NULL
SELECT expandValue(tbox 'TBOX T([2001-01-01,2001-01-03))', 1);
-- The box must have value dimension
```

```
SELECT expandSpace(stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-03])', 1);
-- STBOX ZT(((0,0,0),(3,3,3)),[2001-01-01,2001-01-03])
SELECT expandSpace(stbox 'STBOX T([2001-01-01,2001-01-03))', 1);
-- The box must have space dimension
```

```
SELECT expandTime(tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-03])', interval '1 day');
-- TBOXFLOAT XT((1,2),[2000-12-31,2001-01-04])
SELECT expandTime(stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-03])',
  interval '-1 day');
-- STBOX ZT(((1,1,1),(2,2,2)),[2001-01-02,2001-01-02])
SELECT expandTime(tbox 'TBOX XT((1,2),[2001-01-01,2001-01-03])', interval '-2 days');
-- NULL
```

- Round the value or the coordinates of the bounding box to a number of decimal places

```
round(box,integer=0) → box
```

```
SELECT round(tbox 'TBOXFLOAT XT((1.12345,2.12345),[2001-01-01,2001-01-02])', 2);
-- TBOXFLOAT XT((1.12,2.12),[2001-01-01, 2001-01-02])
SELECT round(stbox 'STBOX XT(((1.12345, 1.12345),(2.12345, 2.12345)),
  [2001-01-01,2001-01-02])', 2);
-- STBOX XT(((1.12,1.12),(2.12,2.12)),[2001-01-01, 2001-01-02])
SELECT round(tstzspan '[2001-01-01, 2001-01-02]'::tbox);
-- The tbox must have X dimension
```

3.6 Spatial Reference System

- Return or set the spatial reference identifier

```
SRID(stbox) → integer
setSRID(stbox,integer) → stbox
```

```
SELECT SRID(stbox 'STBOX ZT(((1.0,2.0,3.0),(4.0,5.0,6.0)),[2001-01-01,2001-01-02])');
-- 0
SELECT SRID(stbox 'SRID=5676;STBOX XT(((1.0,2.0),(4.0,5.0)),[2001-01-01,2001-01-02])');
-- 5676
SELECT SRID(stbox 'GEODSTBOX T([2001-01-01,2001-01-02))');
-- ERROR:  The box must have space dimension
```

```
SELECT setSRID(stbox 'STBOX ZT(((1.0,2.0,3.0),(4.0,5.0,6.0)), [2001-01-01,2001-01-02])',
  5676);
-- SRID=5676;STBOX ZT(((1,2,3),(4,5,6)),[2001-01-01,2001-01-02])
```

- Transform to a spatial reference identifier

```
transform(stbox,srid integer) → stbox
transformPipeline(stbox,pipeline text,srid integer,is_forward boolean=true) → stbox
```

The `transform` function specifies the transformation with a target SRID. An error is raised when the input box has an unknown SRID (represented by 0). The `transformPipeline` function specifies the transformation with a defined coordinate transformation pipeline represented with the following string format:

`urn:ogc:def:coordinateOperation:AUTHORITY::CODE`

The SRID of the input box is ignored, and the SRID of the output box will be set to zero unless a value is provided via the optional `srid` parameter. As stated by the last parameter, the pipeline is executed by default in a forward direction; by setting the parameter to false, the pipeline is executed in the inverse direction.

```
SELECT round(transform(stbox 'SRID=4326;STBOX XT(((2.340088, 49.400250),
  (6.575317, 51.553167)),[2001-01-01,2001-01-02])', 3812), 6);
/* SRID=3812;STBOX XT(((502773.429981,511805.120402),(803028.908265,751590.742629)),
   [2001-01-01, 2001-01-02]) */
WITH test(box, pipeline) AS (
  SELECT stbox 'SRID=4326;GEODSTBOX Z((-0.1275,50.846667,100),(4.3525,51.507222,100))',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(box, pipeline, 4326), pipeline, 4326,
  false), 6) FROM test;
-- SRID=4326;GEODSTBOX Z((-0.1275,50.846667,100),(4.3525,51.507222,100))
```

3.7 Splitting Operations

- Split the bounding box in quadrants or octants {}

```
quadSplit(stbox) → {stbox}
```

As indicated by the {} symbol, this function is a *set-returning function* (also known as a *table function*) since it typically return more than one value.

```
SELECT quadSplit(stbox 'STBOX XT(((0,0),(4,4)),[2001-01-01,2001-01-05])');
/* {"STBOX XT(((0,0),(2,2)),[2001-01-01, 2001-01-05])",
   "STBOX XT(((2,0),(4,2)),[2001-01-01, 2001-01-05])",
   "STBOX XT(((0,2),(2,4)),[2001-01-01, 2001-01-05])",
   "STBOX XT(((2,2),(4,4)),[2001-01-01, 2001-01-05])"} */
SELECT quadSplit(stbox 'STBOX Z((0,0,0),(4,4,4))');
/* {"STBOX Z((0,0,0),(2,2,2))","STBOX Z((2,0,0),(4,2,2))","STBOX Z((0,2,0),(2,4,2))",
   "STBOX Z((2,2,0),(4,4,2))","STBOX Z((0,0,2),(2,2,4))","STBOX Z((2,0,2),(4,2,4))",
   "STBOX Z((0,2,2),(2,4,4))","STBOX Z((2,2,2),(4,4,4))"} */
```

This function is typically used for multiresolution grids, where the space is split in cells such that the cells have a maximum number of elements. Figure 3.1 shows an example of the result of using this function using synthetic trajectories in Brussels.

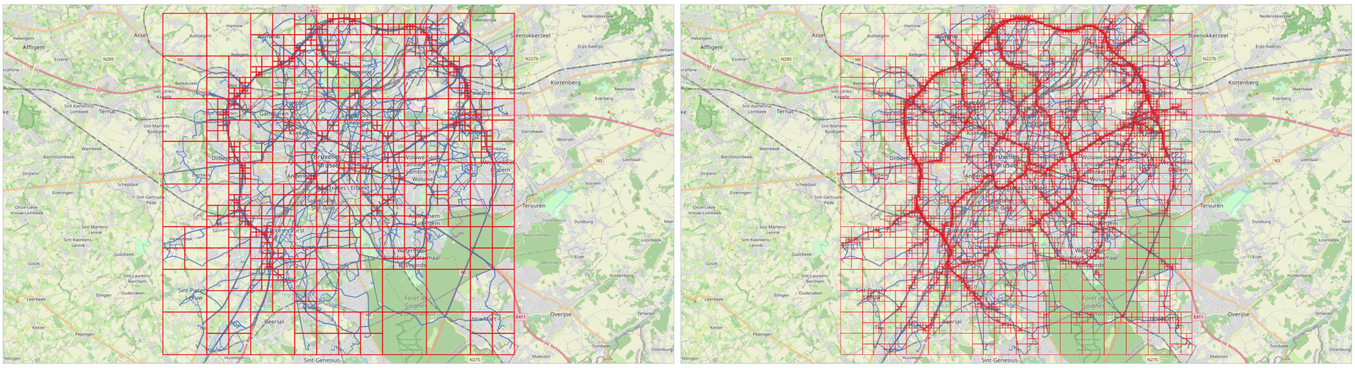

Figure 3.1: Multiresolution grid on Brussels data obtained using the BerlinMOD generator. Each cell contains at most 10,000 (left) and 1,000 (right) instants across the entire simulation period (four days in this case). On the left, we can see the high density of the traffic in the ring around Brussels, while on the right we can see other main axes in the city.

3.8 Set Operations

The set operators for box types are union (+) and intersection (*). In the case of union, the operands must have exactly the same dimensions, otherwise an error is raised. Furthermore, if the operands do not overlap on all the dimensions and error is raised, since in this would result in a box with disjoint values, which cannot be represented. The operator computes the union on all dimensions that are present in both arguments. In the case of intersection, the operands must have at least one common dimension, otherwise an error is raised. The operator computes the intersection on all dimensions that are present in both arguments.

- Union and intersection of two bounding boxes

```
box {+, *} box → box
```

```
SELECT tbox 'TBOXINT XT([1,3),[2001-01-01,2001-01-03])' + tbox 'TBOXINT XT([2,4),
  [2001-01-02,2001-01-04])';
-- TBOXINT XT([1,4),[2001-01-01,2001-01-04])
SELECT stbox 'STBOX ZT(((1,1,1),(2,2,2)),
  [2001-01-01,2001-01-02])' + stbox 'STBOX XT(((2,2),(3,3))),[2001-01-01,2001-01-03]';
-- ERROR:  The arguments must be of the same dimensionality
SELECT tbox 'TBOXFLOAT XT((1,3),[2001-01-01,2001-01-02])' + tbox 'TBOXFLOAT XT((3,4),
  [2001-01-03,2001-01-04])';
-- ERROR:  Result of box union would not be contiguous
```

```
SELECT tbox 'TBOXINT XT([1,3),
  [2001-01-01,2001-01-03])' * tbox 'TBOX T([2001-01-02,2001-01-04))';
-- TBOX T([2001-01-02,2001-01-03))
SELECT stbox 'STBOX ZT(((1,1,1),(3,3,3)),
  [2001-01-01,2001-01-02])' * stbox 'STBOX X((2,2),(4,4))';
-- STBOX X((2,2),(3,3))
```

3.9 Bounding Box Operations

3.9.1 Topological Operations

There are five topological operators: overlaps (`&&`), contains (`@>`), contained (`<@`), same (`~=`), and adjacent (`-|-`). The operators verify the topological relationship between the bounding boxes taking into account the value and/or the time dimension for as many dimensions that are present on both arguments.

A `numspan` or a `tstzspan` is an operand of these operators against a `tbox`, in either order: the operator answers on the box of the span, which has the value dimension alone or the time dimension alone.

```
SELECT intspan '[1,5)' && tbox 'TBOXINT XT([3,8),[2001-01-01,2001-01-02])';
-- true
SELECT tstzspan '[2001-01-01,2001-01-03]' @> tbox 'TBOXINT XT([3,8) ↩
    ,[2001-01-01,2001-01-02])';
-- true
```

- Do the bounding boxes overlap?

```
box && box → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((1,3),[2001-01-01,2001-01-03])' &&
  tbox 'TBOXFLOAT XT((2,4),[2001-01-02,2001-01-04])';
-- true
SELECT stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])' &&
  stbox 'STBOX T([2001-01-02,2001-01-02])';
-- true
```

- Does the first bounding box contain the second one?

```
box @> box → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((1,4),[2001-01-01,2001-01-04])' @>
  tbox 'TBOXFLOAT XT((2,3),[2001-01-01,2001-01-02])';
-- true
SELECT stbox 'STBOX Z((1,1,1),(3,3,3))' @>
  stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])';
-- true
```

- Is the first bounding box contained in the second one?

```
box <@ box → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' <@ tbox 'TBOXFLOAT XT((1,2),
  [2001-01-01,2001-01-02])';
-- true
SELECT stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])' <@
  stbox 'STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01,2001-01-02])';
-- true
```

- Are the bounding boxes equal in their common dimensions?

```
box ~= box → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((1,2),
  [2001-01-01,2001-01-02])' ~= tbox 'TBOXFLOAT T([2001-01-01,2001-01-02])';
-- true
SELECT stbox 'STBOX XT(((1,1),(3,3)),
  [2001-01-01,2001-01-03])' ~= stbox 'STBOX Z((1,1,1),(3,3,3))';
-- true
```

- Are the bounding boxes adjacent?

```
box -|- box → boolean
```

Two boxes are adjacent if their extents meet in every dimension the boxes share and touch at a single value in at least one of them. A dimension in which the boxes are apart separates them whatever the remaining dimensions do.

```
SELECT tbox 'TBOXINT XT([1,2),[2001-01-01,2001-01-02])' -|- tbox 'TBOXINT XT([2,3),
  [2001-01-02,2001-01-03])';
-- true
SELECT tbox 'TBOXFLOAT XT((1,2),
  [2001-01-01,2001-01-02])' -|- tbox 'TBOX T([2001-01-02,2001-01-03])';
-- true
SELECT stbox 'STBOX XT(((1,1),(3,3)),
  [2001-01-01,2001-01-03])' -|- stbox 'STBOX XT(((2,2),(4,4)),[2001-01-03,2001-01-04])';
-- true
```

3.9.2 Position Operations

The position operators consider the relative position of the bounding boxes. The operators `<<`, `>>`, `&<`, and `&>` consider the X value for the `tbox` type and the X coordinates for the `stbox` type, the operators `<<|`, `|>>`, `&<|`, and `|&>` consider the Y coordinates for the `stbox` type, the operators `<</`, `/>>`, `&</`, and `/&>` consider the Z coordinates for the `stbox` type, and the operators `<<#`, `#>>`, `#&<`, and `#&>` consider the time dimension for the `tbox` and `stbox` types. The operators raise an error if both boxes do not have the required dimension.

A `numspan` is an operand of the operators on the value dimension against a `tbox`, and a `tstzspan` of those on the time dimension, in either order, answered on the box of the span.

```
SELECT intspan '[1,5)' &< tbox 'TBOXINT XT([3,8),[2001-01-01,2001-01-02])';
-- true
SELECT tstzspan '[2001-01-05,2001-01-06]' #>> tbox 'TBOXINT XT([3,8) ↩
    ,[2001-01-01,2001-01-02])';
-- true
```

The operators for the numeric dimension of the `tbox` type are given next.

- Are the X/Y/Z/T values of the first bounding box strictly less than those of the second one?

```
tbox {<<, <<#} tbox → boolean
stbox {<<, <<|, <</, <<#} stbox → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' <<
  tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])';
-- true
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' <<
  tbox 'TBOXFLOAT T([2001-01-03,2001-01-04])';
-- ERROR:  The box must have value dimension
SELECT stbox 'STBOX Z((1,1,1),(2,2,2))' <<| stbox 'STBOX Z((3,3,3),(4,4,4))';
-- true
SELECT stbox 'STBOX Z((1,1,1),(2,2,2))' <</ stbox 'STBOX Z((3,3,3),(4,4,4))';
-- true
SELECT tbox 'TBOXFLOAT XT((1,2),
  [2001-01-01,2001-01-02])' <<# tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])';
-- true
```

- Are the X/Y/Z/T values of the first bounding box strictly greater than those of the second one?

```
tbox {>>, #>>} tbox → boolean
stbox {>>, |>>, />>, #>>} stbox → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])' >>
  tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])';
-- true
SELECT stbox 'STBOX Z((3,3,3),(4,4,4))' |>> stbox 'STBOX Z((1,1,1),(2,2,2))';
-- true
SELECT stbox 'STBOX Z((3,3,3),(4,4,4))' />> stbox 'STBOX Z((1,1,1),(2,2,2))';
-- true
SELECT stbox 'STBOX XT(((3,3),(4,4)),[2001-01-03,2001-01-04])' #>>
  stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])';
-- true
```

- Are the X/Y/Z/T values of the first bounding box not greater than those of the second one?

```
tbox {&<, &<#} box → boolean
stbox {&<, &<|, &</, &<#} stbox → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((1,4),[2001-01-01,2001-01-04])' &<
  tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])';
-- true
SELECT stbox 'STBOX Z((1,1,1),(4,4,4))' &<| stbox 'STBOX Z((3,3,3),(4,4,4))';
-- true
SELECT stbox 'STBOX Z((1,1,1),(4,4,4))' &</ stbox 'STBOX Z((3,3,3),(4,4,4))';
-- true
SELECT tbox 'TBOXFLOAT XT((1,4),
  [2001-01-01,2001-01-04])' &<# tbox 'TBOXFLOAT XT((3,4),[2001-01-03,2001-01-04])';
-- true
```

- Are the X/Y/Z/T values of the first bounding box not less than those of the second one?

```
tbox {&>, #&>} tbox → boolean
stbox {&>, |&>, /&>, #&>} stbox} → boolean
```

```
SELECT tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-02])' &>
  tbox 'TBOXFLOAT XT((1,4),[2001-01-01,2001-01-04])';
-- true
SELECT stbox 'STBOX Z((3,3,3),(4,4,4))' |&> stbox 'STBOX Z((1,1,1),(2,2,2))';
-- false
SELECT stbox 'STBOX Z((3,3,3),(4,4,4))' /&> stbox 'STBOX Z((1,1,1),(2,2,2))';
-- true
SELECT stbox 'STBOX XT(((1,1),(2,2)),[2001-01-01,2001-01-02])' #&>
  stbox 'STBOX XT(((1,1),(4,4)),[2001-01-01,2001-01-04])';
-- true
```

3.10 Comparisons

The traditional comparison operators (=, <, and so on) can be applied to box types. Excepted equality and inequality, the other comparison operators are not useful in the real world but allow B-tree indexes to be constructed on box types. These operators define a total order. They compare first the SRID of spatiotemporal boxes, then the dimensions the boxes have, whether spatiotemporal boxes are geodetic, and the value type of temporal boxes. Boxes agreeing on all of these are compared on their time dimension and, if those are equal, on their value or space dimension.

- Traditional comparisons

```
box {=, <>, <, >, <=, >=} box → boolean
cmp(box,box) → integer
```

Function `cmp` returns -1, 0, or 1 depending on whether the first argument is, respectively, less than, equal to, or greater than the second one.

```
SELECT tbox 'TBOXINT XT([1,1],[2001-01-01,2001-01-04])' = tbox 'TBOXINT XT([2,2],
  [2001-01-03,2001-01-05])';
-- false
SELECT tbox 'TBOXFLOAT XT([1,1],[2001-01-01,2001-01-04])' <>
  tbox 'TBOXFLOAT XT([2,2],[2001-01-03,2001-01-05])';
-- true
SELECT tbox 'TBOXINT XT([1,1],[2001-01-01,2001-01-04])' <
  tbox 'TBOXINT XT([1,2],[2001-01-03,2001-01-05])';
-- true
SELECT tbox 'TBOXFLOAT XT([1,1],[2001-01-03,2001-01-04])' >
  tbox 'TBOXFLOAT XT((1,2),[2001-01-01,2001-01-05])';
-- true
SELECT tbox 'TBOXINT XT([1,1],[2001-01-01,2001-01-04])' <= tbox 'TBOXINT XT([2,2],
  [2001-01-03,2001-01-05])';
-- true
SELECT tbox 'TBOXFLOAT XT([1,1],[2001-01-01,2001-01-04])' >= tbox 'TBOXFLOAT XT([2,2],
  [2001-01-03,2001-01-05])';
-- false
SELECT cmp(tbox 'TBOXFLOAT XT([1,1],[2001-01-01,2001-01-04])',
  tbox 'TBOXFLOAT XT([2,2],[2001-01-03,2001-01-05])');
-- -1
```

- Hash values

```
hash(box) → integer
hashExtended(box,seed bigint) → bigint
```

Function `hash` returns a 32-bit hash value and function `hashExtended` a 64-bit hash value computed with the given seed. Equal values have equal hash values, which is what hash indexes, hash joins, and hash aggregation rely on.

```
SELECT hash(tbox 'TBOXINT XT([1,1],[2001-01-01,2001-01-04])') =
  hash(tbox 'TBOXINT XT([1,2),[2001-01-01,2001-01-04])');
-- true
SELECT hashExtended(stbox 'STBOX X((1,1),(2,2))', 0) <>
  hashExtended(stbox 'STBOX X((1,1),(2,2))', 1);
-- true
```

3.11 Aggregations

- Bounding box extent

```
extent(box) → box
```

```
WITH boxes(b) AS (
  SELECT tbox 'TBOXFLOAT XT((1,3),[2001-01-01,2001-01-03])' UNION
  SELECT tbox 'TBOXFLOAT XT((5,7),[2001-01-05,2001-01-07])' UNION
  SELECT tbox 'TBOXFLOAT XT((6,8),[2001-01-06,2001-01-08])' )
SELECT extent(b) FROM boxes;
-- TBOXFLOAT XT((1,8),[2001-01-01,2001-01-08])
WITH boxes(b) AS (
  SELECT stbox 'STBOX Z((1,1,1),(3,3,3))' UNION
  SELECT stbox 'STBOX Z((5,5,5),(7,7,7))' UNION
  SELECT stbox 'STBOX Z((6,6,6),(8,8,8))' )
SELECT extent(b) FROM boxes;
-- STBOX Z((1,1,1),(8,8,8))
```

3.12 Indexing

GiST and SP-GiST indexes can be created for table columns of the `tbox` and `stbox` types. The GiST index implements an R-tree and the SP-GiST index implements an n-dimensional quad-tree. An example of creation of a GiST index in a column `Box` of type `stbox` in a table `Trips` is as follows:

```
CREATE TABLE Trips(TripID integer PRIMARY KEY, Trip tgeompoint, Box stbox);
CREATE INDEX Trips_Box_Idx ON Trips USING GIST(bbox);
```

A GiST or SP-GiST index can accelerate queries involving the following operators: `&&`, `<@`, `@>`, `~=`, `-|-`, `<<`, `>>`, `&<`, `&>`, `<<|`, `|>>`, `&<|`, `|&>`, `<</`, `/>>`, `&</`, `/&>`, `<<#`, `#>>`, `&<#`, and `#&>`.

In addition, B-tree indexes can be created for table columns of a bounding box type. For these index types, basically the only useful operation is equality. There is a B-tree sort ordering defined for values of bounding box types, with corresponding < and > operators, but the ordering is rather arbitrary and not usually useful in the real world. The B-tree support is primarily meant to allow sorting internally in queries, rather than creation of actual indexes.

Chapter 4

Temporal Types (Part 1)

4.1 Introduction

MobilityDB provides several temporal types based on PostgreSQL or PostGIS types, namely, `tbool`, `tint`, `tbigint`, `tfloat`, `ttext`, `tgeometry`, `tgeography`, `tgeompoint`, and `tgeogpoint`, which are, respectively, based on the base types `boolean`, `integer`, `bigint`, `float`, `text`, `geometry`, and `geography`, where `tgeometry` and `tgeography` accept arbitrary geometries/geographies, while `tgeompoint` and `tgeogpoint` only accept 2D or 3D points with Z dimension. In addition, MobilityDB provides other specific spatial types, namely, `cbuffer` (circular buffer), `npoint` (network point), and `pose`, and its corresponding spatiotemporal types, namely, `tcbuffer`, `tnpoint`, `tpose` and `trgeometry` (temporal rigid geometry). In this chapter and the next one we present the functions and operators available for all temporal types, while the specific functions for the spatiotemporal types will be given in subsequenct chapters. At a conceptual level, the temporal types are organized in the hierarchy depicted in Figure 4.1.

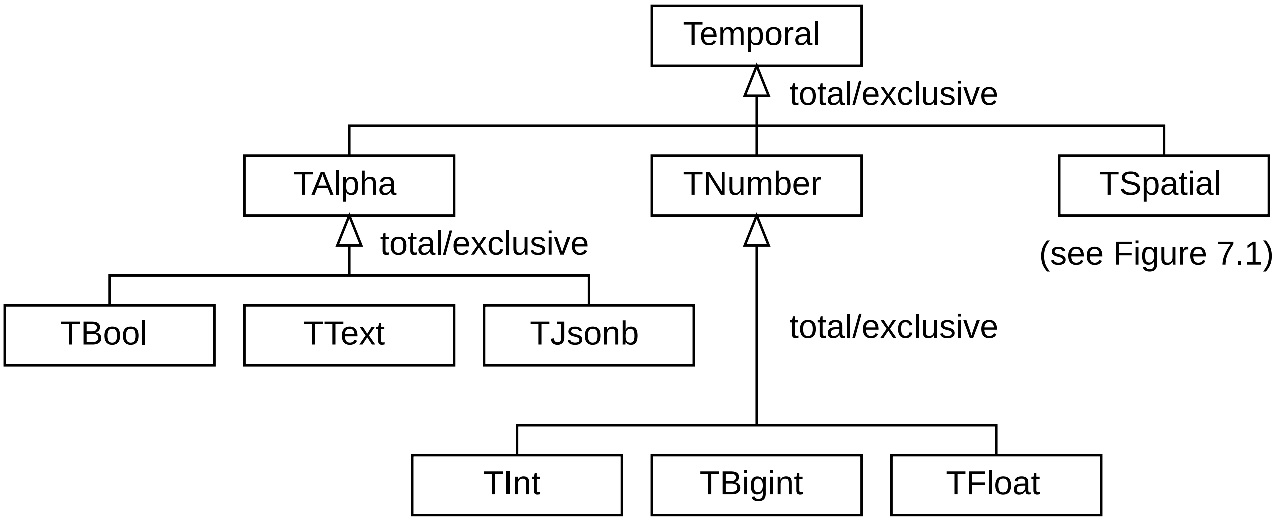

Figure 4.1: Hierarchy of temporal types in MobilityDB.

The *interpolation* of a temporal value states how the value evolves between successive instants. The interpolation is *discrete* when the value is unknown between two successive instants. They can represent, for example, checkins/checkouts when using an RFID card reader to enter or exit a building. The interpolation is *step* when the value remains constant between two successive instants. For example, the gear used by a moving car may be represented with a temporal integer, which indicates that its value is constant between two time instants. On the other hand, the interpolation is *linear* when the value evolves linearly between two successive instants. For example, the speed of a car may be represented with a temporal float, which indicates that the values are known at the time instants but continuously evolve between them. Similarly, the location of a vehicule may be represented by a temporal point where the location between two consecutive GPS readings is obtained by linear interpolation. Temporal types based on discrete base types, that is the `tbool`, `tint`, or `ttext` evolve necesssarily in a step manner. On the other hand, temporal types based on continuous base types, that is `tfloat`, `tgeompoint`, or `tgeogpoint` may evolve in a step or linear manner. Note that the types `tgeometry` and `tgeography` only support discrete or step interpolation, since it is not possible to linearly interpolate two arbitrary geometries/geographies.

The *subtype* of a temporal value states the temporal extent at which the evolution of values is recorded. Temporal values come in three subtypes, explained next.

A temporal value of *instant* subtype (briefly, *an instant value*) represents the value at a time instant, for example

```
SELECT tfloat '17@2018-01-01 08:00:00';
```

A temporal value of *sequence* subtype (briefly, a *sequence value*) represents the evolution of the value during a sequence of time instants, where the values between these instants are interpolated using a discrete, step, or a linear function (see above). An example is as follows:

```
-- Discrete interpolation
SELECT tfloat '{17@2018-01-01 08:00:00, 17.5@2018-01-01 08:05:00,
  18@2018-01-01 08:10:00}';
-- Step interpolation
SELECT tfloat 'Interp=Step;
  (10@2018-01-01 08:00:00, 20@2018-01-01 08:05:00, 15@2018-01-01 08:10:00]';
-- Linear interpolation
SELECT tfloat '(10@2018-01-01 08:00:00, 20@2018-01-01 08:05:00, 15@2018-01-01 08:10:00]';
```

As can be seen, a sequence value has a lower and an upper bound that can be inclusive (represented by '`[`' and '`]`') or exclusive (represented by '`(`' and '`)`'). By definition, both bounds must be inclusive when the interpolation is discrete or when the sequence has a single instant (called an *instantaneous sequence*), as the next example

```
SELECT tint '[10@2018-01-01 08:00:00]';
```

Sequence values must be *uniform*, that is, they must be composed of instant values of the same base type. Sequence values with step or linear interpolation are referred to as *continuous sequences*.

The value of a temporal sequence is interpreted by assuming that the period of time defined by every pair of consecutive values `v1@t1` and `v2@t2` is lower inclusive and upper exclusive, unless they are the first or the last instants of the sequence and in that case the bounds of the whole sequence apply. Furthermore, the value taken by the temporal sequence between two consecutive instants depends on whether the interpolation is step or linear. For example, the temporal sequence above represents that the value is `10` during `(2018-01-01 08:00:00, 2018-01-01 08:05:00)`, `20` during `[2018-01-01 08:05:00, 2018-01-01 08:10:00)`, and `15` at the end instant `2018-01-01 08:10:00`. On the other hand, the following temporal sequence

```
SELECT tfloat '(10@2018-01-01 08:00:00, 20@2018-01-01 08:05:00, 15@2018-01-01 08:10:00]';
```

represents that the value evolves linearly from `10` to `20` during `(2018-01-01 08:00:00, 2018-01-01 08:05:00)` and evolves from `20` to `15` during `[2018-01-01 08:05:00, 2018-01-01 08:10:00]`.

Finally, a temporal value of *sequence set* subtype (briefly, a *sequence set value*) represents the evolution of the value at a set of sequences, where the values between these sequences are unknown. An example is as follows:

```
SELECT tfloat '{[17@2018-01-01 08:00:00, 17.5@2018-01-01 08:05:00],
  [18@2018-01-01 08:10:00, 18@2018-01-01 08:15:00]}';
```

As shown in the above examples, sequence set values can only be of step or linear interpolation. Furtheremore, all composing sequences of a sequence set value must be of the same base type and the same interpolation.

Continuous sequence values are converted into *normal form* so that equivalent values have identical representations. For this, consecutive instant values are merged when possible. For step interpolation, three consecutive instant values can be merged into two if they have the same value. For linear interpolation, three consecutive instant values can be merged into two if the linear functions defining the evolution of values are the same. This is decided exactly on the stored values, so that a value in normal form is its own normal form and a value read back from its binary representation is the value written. The interpolation of geodetic points follows the great circle, and a geodetic point is merged when it lies on that path within the rounding of the interpolated point. Examples of transformation into normal form are as follows.

```
SELECT tint '[1@2001-01-01, 2@2001-01-03, 2@2001-01-04, 2@2001-01-05)';
-- [1@2001-01-01, 2@2001-01-03, 2@2001-01-05)
SELECT asText(tgeompoint '[Point(1 1)@2001-01-01 08:00:00, Point(1 1)@2001-01-01 08:05:00,
  Point(1 1)@2001-01-01 08:10:00)');
-- [POINT(1 1)@2001-01-01 08:00:00, POINT(1 1)@2001-01-01 08:10:00)
SELECT tfloat '[1@2001-01-01, 2@2001-01-03, 3@2001-01-05]';
-- [1@2001-01-01, 3@2001-01-05]
SELECT asText(tgeompoint '[Point(1 1)@2001-01-01 08:00:00, Point(2 2)@2001-01-01 08:05:00,
  Point(3 3)@2001-01-01 08:10:00]');
-- [POINT(1 1)@2001-01-01 08:00:00, POINT(3 3)@2001-01-01 08:10:00]
```

Similary, temporal sequence set values are converted into normal form. For this, consecutive sequence values are merged when possible. Examples of transformation into a normal form are as follows.

```
SELECT tint '{[1@2001-01-01, 1@2001-01-03), [2@2001-01-03, 2@2001-01-05)}';
-- '{[1@2001-01-01, 2@2001-01-03, 2@2001-01-05)}'
SELECT tfloat '{[1@2001-01-01, 2@2001-01-03), [2@2001-01-03, 3@2001-01-05]}';
-- '{[1@2001-01-01, 3@2001-01-05]}'
SELECT tfloat '{[1@2001-01-01, 3@2001-01-05), [3@2001-01-05]}';
-- '{[1@2001-01-01, 3@2001-01-05]}'
SELECT asText(tgeompoint '{[Point(0 0)@2001-01-01 08:00:00,
  Point(1 1)@2001-01-01 08:05:00, Point(1 1)@2001-01-01 08:10:00),
  [Point(1 1)@2001-01-01 08:10:00, Point(1 1)@2001-01-01 08:15:00)}');
/* {[[Point(0 0)@2001-01-01 08:00:00, Point(1 1)@2001-01-01 08:05:00,
   Point(1 1)@2001-01-01 08:15:00)} */
SELECT asText(tgeompoint '{[Point(1 1)@2001-01-01 08:00:00,
  Point(2 2)@2001-01-01 08:05:00),
  [Point(2 2)@2001-01-01 08:05:00, Point(3 3)@2001-01-01 08:10:00]}');
-- {[POINT(1 1)@2001-01-01 08:00:00, POINT(3 3)@2001-01-01 08:10:00]}
SELECT asText(tgeompoint '{[Point(1 1)@2001-01-01 08:00:00,
  Point(3 3)@2001-01-01 08:10:00), [Point(3 3)@2001-01-01 08:10:00]}');
-- {[POINT(1 1)@2001-01-01 08:00:00, POINT(3 3)@2001-01-01 08:10:00]}
```

Temporal types support *type modifiers* (or `typmod` in PostgreSQL terminology), which specify additional information for a column definition. For example, in the following table definition:

```
CREATE TABLE Department(DeptNo integer, DeptName varchar(25), NoEmps tint(Sequence));
```

the type modifier for the type `varchar` is the value 25, which indicates the maximum length of the values of the column, while the type modifier for the type `tint` is the string `Sequence`, which restricts the subtype of the values of the column to be sequences. In the case of temporal alphanumeric types (that is, `tbool`, `tint`, `tbigint`, `tfloat`, and `ttext`), the possible values for the type modifier are `Instant`, `Sequence`, and `SequenceSet`. If no type modifier is specified for a column, values of any subtype are allowed.

On the other hand, in the case of temporal geometry types (that is, `tgeompoint` `tgeogpoint`, `tgeometry`, or `tgeography`) the type modifier may be used to specify the subtype, the geometry type, and/or the spatial reference identifier (SRID). For example, in the following table definition:

```
CREATE TABLE Flight(FlightNo integer, Route tgeogpoint(Sequence, PointZ, 4326));
CREATE TABLE Storm(StormId integer, Route tgeography(Sequence, Linestring, 4326));
```

the type modifiers for the `tgeogpoint` and `tgeography` types are composed of three values, the first one indicating the subtype as above, the second one the spatial type of the geographies composing the temporal point and geographies, and the last one the SRID of the composing geographies. For temporal points, the possible values for the first argument of the type modifier are as above, those for the second argument are either `Point` or `PointZ`, and those for the third argument are valid SRIDs. All the three arguments are optional and if any of them is not specified for a column, values of any subtype, dimensionality, and/or SRID are allowed.

Each temporal type is associated to another type, referred to as its *bounding box*, which represent its extent in the value and/or the time dimension. The bounding box of the various temporal types are as follows:

- The `tstzspan` type for the `tbool` and `ttext` types, where only the temporal extent is considered.
- The `tbox` (temporal box) type for the `tint`, `tbigint`, and `tfloat` types, where the value and the time extents are defined, respectively, by a number span and a time span.
- The `stbox` (spatiotemporal box) type for the `tgeompoint` and `tgeogpoint` types, where the spatial extent is defined in the X, Y, and Z dimensions, and the time extent by a time span.

A rich set of functions and operators is available to perform various operations on temporal types. They are explained in this chapter and the following ones. Some of these operations, in particular those related to indexes, manipulate bounding boxes for efficiency reasons.

4.2 Examples of Temporal Types

Examples of usage of temporal alphanumeric types are given next.

```
CREATE TABLE Department(DeptNo integer, DeptName varchar(25), NoEmps tint);
INSERT INTO Department VALUES
  (10, 'Research', tint '[10@2001-01-01, 12@2001-04-01, 12@2001-08-01)'),
  (20, 'Human Resources', tint '[4@2001-02-01, 6@2001-06-01, 6@2001-10-01)');
CREATE TABLE Temperature(RoomNo integer, Temp tfloat);
INSERT INTO Temperature VALUES
  (1001, tfloat '{18.5@2001-01-01 08:00:00, 20.0@2001-01-01 08:10:00}'),
  (2001, tfloat '{19.0@2001-01-01 08:00:00, 22.5@2001-01-01 08:10:00}');
-- Value at a timestamp
SELECT RoomNo, valueAtTimestamp(Temp, '2001-01-01 08:10:00') FROM temperature;
-- 1001 | 20
-- 2001 | 22.5
-- Restriction to a value
SELECT DeptNo, atValue(NoEmps, 10) FROM Department;
-- 10 | [10@2001-01-01, 10@2001-04-01)
-- 20 |
-- Restriction to a period
SELECT DeptNo, atTime(NoEmps, tstzspan '[2001-01-01, 2001-04-01]') FROM Department;
-- 10 | [10@2001-01-01, 12@2001-04-01]
-- 20 | [4@2001-02-01, 4@2001-04-01]
-- Temporal comparison
SELECT DeptNo, NoEmps #<= 10 FROM Department;
-- 10 | [t@2001-01-01, f@2001-04-01, f@2001-08-01)
-- 20 | [t@2001-02-01, t@2001-10-01)
-- Temporal aggregation
SELECT tsum(NoEmps) FROM Department;
/* {[10@2001-01-01, 14@2001-02-01, 16@2001-04-01,
   18@2001-06-01, 6@2001-08-01, 6@2001-10-01)} */
```

Examples of usage of temporal point types are given next.

```
CREATE TABLE Trips(CarId integer, TripId integer, Trip tgeompoint);
INSERT INTO Trips VALUES
  (10, 1, tgeompoint '{[Point(0 0)@2001-01-01 08:00:00, Point(2 0)@2001-01-01 08:10:00,
Point(2 1)@2001-01-01 08:15:00)}'),
  (20, 1, tgeompoint '{[Point(0 0)@2001-01-01 08:05:00, Point(1 1)@2001-01-01 08:10:00,
  Point(3 3)@2001-01-01 08:20:00)}');
-- Value at a given timestamp
SELECT CarId,
  ST_AsText(valueAtTimestamp(Trip, timestamptz '2001-01-01 08:10:00')) FROM Trips;
-- 10 | POINT(2 0)
-- 20 | POINT(1 1)
-- Restriction to a value
SELECT CarId, asText(atValue(Trip, geometry 'Point(2 0)')) FROM Trips;
```

```
-- 10 | {"[POINT(2 0)@2001-01-01 08:10:00]"}
-- 20 |
-- Restriction to a period
SELECT CarId,
  asText(atTime(Trip, tstzspan '[2001-01-01 08:05:00,2001-01-01 08:10:00]')) FROM Trips;
-- 10 | {[POINT(1 0)@2001-01-01 08:05:00, POINT(2 0)@2001-01-01 08:10:00]}
-- 20 | {[POINT(0 0)@2001-01-01 08:05:00, POINT(1 1)@2001-01-01 08:10:00]}
-- Temporal distance
SELECT T1.CarId, T2.CarId, T1.Trip <-> T2.Trip FROM Trips T1,
  Trips T2 WHERE T1.CarId < T2.CarId;
/* 10 | 20 | {[1@2001-01-01 08:05:00, 1.4142135623731@2001-01-01 08:10:00,
   1@2001-01-01 08:15:00)} */
```

4.3 Validity of Temporal Types

Values of temporal types must satisfy several constraints so that they are well defined. These constraints are given next.

- The constraints on the base type and the `timestamptz` type must be satisfied.
- A value of a `tint` or a `tbigint` must be less than the largest value of its base type, since the value span of the bounding box, whose upper bound is the next value, has no representation for it, as for an `intspan` or a `bigintspan`.
- A sequence value must be composed of at least one instant value.
- An instantaneous sequence value or a sequence value with discrete interpolation must have inclusive lower and upper bounds.
- In a sequence value, the timestamps of the composing instants must be different and ordered.
- In a sequence value with step interpolation, the last two values must be equal if upper bound is exclusive.
- A sequence set value must be composed of at least one sequence value.
- In a sequence set value, the composing sequence values must be non overlapping and ordered.

An error is raised whenever one of these constraints are not satisfied. Examples of incorrect temporal values are as follows.

```
-- Incorrect value for base type
SELECT tbool '1.5@2001-01-01 08:00:00';
-- Base type value is not a point
SELECT tgeompoint 'Linestring(0 0,1 1)@2001-01-01 08:05:00';
-- Value at the largest value of the base type
SELECT tint '2147483647@2001-01-01 08:00:00';
-- Incorrect timestamp
SELECT tint '2@2001-02-31 08:00:00';
-- Empty sequence
SELECT tint '';
-- Incorrect bounds for instantaneous sequence
SELECT tint '[1@2001-01-01 09:00:00)';
-- Duplicate timestamps
SELECT tint '[1@2001-01-01 08:00:00, 2@2001-01-01 08:00:00]';
-- Unordered timestamps
SELECT tint '[1@2001-01-01 08:10:00, 2@2001-01-01 08:00:00]';
-- Incorrect end value for step interpolation
SELECT tint '[1@2001-01-01 08:00:00, 2@2001-01-01 08:10:00)';
-- Empty sequence set
SELECT tint '{[]}';
-- Duplicate timestamps
SELECT tint '{1@2001-01-01 08:00:00, 2@2001-01-01 08:00:00}';
-- Overlapping periods
SELECT tint '{[1@2001-01-01 08:00:00, 1@2001-01-01 10:00:00),
  [2@2001-01-01 09:00:00, 2@2001-01-01 11:00:00)}';
```

4.4 Temporalizing Operations

A common way to generalize the traditional operations to the temporal types is to apply the operation *at each instant*, which yields a temporal value as result. In that case, the operation is only defined on the intersection of the temporal extents of the operands; if the temporal extents are disjoint, then the result is null. For example, the temporal comparison operators, such as `#<`, test whether the values taken by their operands at each instant satisfy the condition and return a temporal Boolean. Examples of the various generalizations of the operators are given next.

```
-- Temporal comparison
SELECT tfloat '[2@2001-01-01, 2@2001-01-03)' #< tfloat '[1@2001-01-01, 3@2001-01-03)';
-- {[f@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}
SELECT tfloat '[1@2001-01-01, 3@2001-01-03)' #< tfloat '[3@2001-01-03, 1@2001-01-05)';
-- NULL
-- Temporal addition
SELECT tint '[1@2001-01-01, 1@2001-01-03)' + tint '[2@2001-01-02, 2@2001-01-05)';
-- [3@2001-01-02, 3@2001-01-03)
-- Temporal intersects
SELECT tIntersects(tgeompoint '[Point(0 1)@2001-01-01, Point(3 1)@2001-01-04)',
  geometry 'Polygon((1 0,1 2,2 2,2 0,1 0))');
-- {[f@2001-01-01, t@2001-01-02, t@2001-01-03], (f@2001-01-03, f@2001-01-04]}
-- Temporal distance
SELECT tgeompoint '[Point(0 0)@2001-01-01 08:00:00, Point(0 1)@2001-01-03 08:10:00)' <->
  tgeompoint '[Point(0 0)@2001-01-02 08:05:00, Point(1 1)@2001-01-05 08:15:00)';
-- [0.5@2001-01-02 08:05:00, 0.745184033794557@2001-01-03 08:10:00)
```

Another common requirement is to determine whether the operands *ever* or *always* satisfy a condition with respect to an operation. These can be obtained by applying the ever or always comparison operators. These operators are denoted by prefixing the traditional comparison operators with, respectively, `?` (ever) and `%` (always). Examples of ever and always comparison operators are given next.

```
-- Does the operands ever intersect?
SELECT eIntersects(tgeompoint '[Point(0 1)@2001-01-01, Point(3 1)@2001-01-04)',
  geometry 'Polygon((1 0,1 2,2 2,2 0,1 0))') ?= true;
-- true
-- Does the operands always intersect?
SELECT aIntersects(tgeompoint '[Point(0 1)@2001-01-01, Point(3 1)@2001-01-04)',
  geometry 'Polygon((0 0,0 2,4 2,4 0,0 0))') %= true;
-- true
-- Is the left operand ever less than the right one ?
SELECT (tfloat '[1@2001-01-01, 3@2001-01-03)' #<
  tfloat '[3@2001-01-01, 1@2001-01-03)') ?= true;
-- true
-- Is the left operand always less than the right one ?
SELECT (tfloat '[1@2001-01-01, 3@2001-01-03)' #<
  tfloat '[2@2001-01-01, 4@2001-01-03)') %= true;
-- true
```

For example, the `eIntersects` function determines whether there is an instant at which the two arguments spatially intersect.

We describe next the functions and operators for temporal types. For conciseness, in the examples we mostly use sequences composed of two instants.

4.5 Notation

We present next the functions and operators for temporal types. These functions and operators are polymorphic, that is, their arguments may be of several types, and the result type may depend on the type of the arguments. To express this, we use the following notation:

- `times` represents any time type with `timestamptz` granularity, that is, `timestamptz`, `tstzset`, `tstzspan`, or `tstzspanset`,
- `ttype` represents any temporal type,
- `ttypeInst`, `ttypeSeq`, and `ttypeSeqSet` represent any temporal type with, respectively, instant, sequence, and sequence set subtype,
- `tdisc` represents any temporal type with a discrete base type, that is, `tbool`, `tint`, `tbigint`, or `ttext`,
- `tcont` represents any temporal type with a continuous base type, that is, `tfloat`, `tgeompoint`, or `tgeogpoint`,
- `ttypeDiscSeq` and `ttypeContSeq` represent any temporal type with sequence subtype and, respectively, discrete and continuous interpolation,
- `base` represents any base type of a temporal type, that is, `boolean`, `integer`, `float`, `text`, `geometry`, or `geography`,
- `values` represents any set of values of a base type of a temporal type, for example, `integer`, `intset`, `intspan`, and `intspanset` for the base type `integer`
- `type[]` represents an array of `type`.
- `<type>` in the name of a function represents the functions obtained by replacing `<type>` by a specific `type`. For example, `tintSeq` or `tfloatSeq` are represented by `ttypeSeq`.

4.6 Input and Output

MobilityDB generalizes Open Geospatial Consortium's Well-Known Text (WKT), Well-Known Binary (WKB), and Moving Features JSON (MF-JSON) input and output format for all temporal types. We start by describing the WKT format.

An *instant value* is a couple of the form `v@t`, where `v` is a value of the base type and `t` is a `timestamptz` value. The temporal extent of an instant value is a single timestamp. Examples of instant values are as follows:

```sql
SELECT tbool 'true@2001-01-01 08:00:00';
SELECT tint '1@2001-01-01 08:00:00';
SELECT tbigint '5@2001-01-01 08:00:00';
SELECT tfloat '1.5@2001-01-01 08:00:00';
SELECT ttext 'AAA@2001-01-01 08:00:00';
SELECT tgeompoint 'Point(0 0)@2017-01-01 08:00:05';
SELECT tgeogpoint 'Point(0 0)@2017-01-01 08:00:05';
SELECT tgeometry 'Linestring(0 0,1 1)@2017-01-01 08:00:05';
SELECT tgeography 'Polygon((0 0,1 1,2 0,0 0))@2017-01-01 08:00:05';
```

A *sequence value* is a set of values `v1@t1,...,vn@tn` delimited by lower and upper bounds, which can be inclusive (represented by '`[`' and '`]`') or exclusive (represented by '`(`' and '`)`'). A sequence value composed of a single couple `v@t` is called an *instantaneous sequence*. Sequence values have an associated *interpolation function* which may be discrete, linear, or step. By definition, the lower and upper bounds of an instantaneous sequence or of a sequence value with discrete interpolation are inclusive. The temporal extent of a sequence value with discrete interpolation is a timestamp set. Examples of sequence values with discrete interpolation are as follows.

```sql
SELECT tbool '{true@2001-01-01 08:00:00, false@2001-01-03 08:00:00}';
SELECT tint '{1@2001-01-01 08:00:00, 2@2001-01-03 08:00:00}';
SELECT tbigint '{1@2001-01-01 08:00:00}'; -- Instantaneous sequence
SELECT tfloat '{1.0@2001-01-01 08:00:00, 2.0@2001-01-03 08:00:00}';
SELECT ttext '{AAA@2001-01-01 08:00:00, BBB@2001-01-03 08:00:00}';
SELECT tgeompoint '{Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-02 08:05:00}';
SELECT tgeogpoint '{Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-02 08:05:00}';
SELECT tgeometry '{Point(0 0)@2017-01-01 08:00:00,
  Linestring(0 0,0 1)@2017-01-02 08:05:00}';
SELECT tgeography '{Point(0 0)@2017-01-01 08:00:00,
  Polygon((0 0,1 1,2 0,0 0))@2017-01-02 08:05:00}';
```

The temporal extent of a sequence value with linear or step interpolation is a period defined by the first and last instants as well as the lower and upper bounds. Examples of sequence values with linear interpolation are as follows:

```
SELECT tbool '[true@2001-01-01 08:00:00, true@2001-01-03 08:00:00]';
SELECT tint '[1@2001-01-01 08:00:00, 1@2001-01-03 08:00:00]';
SELECT tfloat '[2.5@2001-01-01 08:00:00, 3@2001-01-03 08:00:00, 1@2001-01-04 08:00:00]';
SELECT tfloat '[1.5@2001-01-01 08:00:00]'; -- Instantaneous sequence
SELECT ttext '[BBB@2001-01-01 08:00:00, BBB@2001-01-03 08:00:00]';
SELECT tgeompoint '[Point(0 0)@2017-01-01 08:00:00, Point(0 0)@2017-01-01 08:05:00)';
SELECT tgeogpoint '[Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-01 08:05:00,
  Point(0 0)@2017-01-01 08:10:00)';
SELECT tgeometry '[Point(0 0)@2017-01-01 08:00:00,
  Linestring(0 0,0 1)@2017-01-02 08:05:00]';
SELECT tgeography '[Point(0 0)@2017-01-01 08:00:00,
  Polygon((0 0,1 1,2 0,0 0))@2017-01-02 08:05:00]';
```

Sequence values whose base type is continuous may specify that the interpolation is step with the prefix `Interp=Step`. If this is not specified, it is supposed that the interpolation is linear by default. Example of sequence values with step interpolation are given next:

```
SELECT tfloat 'Interp=Step;[2.5@2001-01-01 08:00:00, 3@2001-01-01 08:10:00]';
SELECT tgeompoint 'Interp=Step;[Point(0 0)@2017-01-01 08:00:00,
  Point(1 1)@2017-01-01 08:05:00, Point(1 1)@2017-01-01 08:10:00)';
SELECT tgeompoint 'Interp=Step;[Point(0 0)@2017-01-01 08:00:00,
 Point(1 1)@2017-01-01 08:05:00, Point(0 0)@2017-01-01 08:10:00)';
ERROR:  Invalid end value for temporal sequence with step interpolation
SELECT tgeogpoint 'Interp=Step;
  [Point(0 0)@2017-01-01 08:00:00, Point(1 1)@2017-01-01 08:10:00]';
```

The last two instants of a sequence value with discrete interpolation and exclusive upper bound must have the same base value, as shown in the second and third examples above.

The prefix `Interp=Linear` states the default linear interpolation explicitly. A temporal type whose values cannot be interpolated linearly, such as `tint` or `tgeometry`, refuses this prefix, as shown next:

```
SELECT tfloat 'Interp=Linear;[2.5@2001-01-01 08:00:00, 3@2001-01-01 08:10:00]';
-- [2.5@2001-01-01 08:00:00+01, 3@2001-01-01 08:10:00+01]
SELECT tgeometry 'Interp=Linear;[Point(0 0)@2017-01-01 08:00:00,
  Linestring(0 0,0 1)@2017-01-02 08:05:00]';
ERROR:  The temporal type cannot have linear interpolation
```

A *sequence set value* is a set `{v1,...,vn}` where every `vi` is a sequence value. The interpolation of sequence set values can only be linear or step, not discrete. All sequences composing a sequence set value must have the same interpolation. The temporal extent of a sequence set value is a set of periods. Examples of sequence set values with linear interpolation are as follows:

```
SELECT tbool '{[false@2001-01-01 08:00:00, false@2001-01-03 08:00:00),
  [true@2001-01-03 08:00:00], (false@2001-01-04 08:00:00, false@2001-01-06 08:00:00]}';
SELECT tint '{[1@2001-01-01 08:00:00, 1@2001-01-03 08:00:00),
  [2@2001-01-04 08:00:00, 3@2001-01-05 08:00:00, 3@2001-01-06 08:00:00]}';
SELECT tfloat '{[1@2001-01-01 08:00:00, 2@2001-01-03 08:00:00, 2@2001-01-04 08:00:00,
  3@2001-01-06 08:00:00]}';
SELECT ttext '{[AAA@2001-01-01 08:00:00, BBB@2001-01-03 08:00:00,
  BBB@2001-01-04 08:00:00), [CCC@2001-01-05 08:00:00, CCC@2001-01-06 08:00:00]}';
SELECT tgeompoint '{[Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-01 08:05:00),
  [Point(0 1)@2017-01-01 08:10:00, Point(1 1)@2017-01-01 08:15:00)}';
SELECT tgeogpoint '{[Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-01 08:05:00),
  [Point(0 1)@2017-01-01 08:10:00, Point(1 1)@2017-01-01 08:15:00)}';
SELECT tgeometry '{[Point(0 0)@2017-01-01 08:00:00,
  Linestring(0 0,1 1)@2017-01-01 08:05:00),
  [Point(0 1)@2017-01-01 08:10:00, Point(1 1)@2017-01-01 08:15:00)}';
SELECT tgeography '{[Point(0 0)@2017-01-01 08:00:00,
  Polygon((0 0,1 1,2 0,0 0))@2017-01-01 08:05:00),
  [Point(0 1)@2017-01-01 08:10:00, Linestring(0 0,1 1)@2017-01-01 08:15:00)}';
```

Sequence set values whose base type is continuous may specify that the interpolation is step with the prefix `Interp=Step`. If this is not specified, it is supposed that the interpolation is linear by default. Example of sequence set values with step interpolation are given next:

```
SELECT tfloat 'Interp=Step;{[1@2001-01-01 08:00:00, 2@2001-01-03 08:00:00,
  2@2001-01-04 08:00:00, 3@2001-01-06 08:00:00]}';
SELECT tgeompoint 'Interp=Step;
  {[Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-01 08:05:00],
  [Point(0 1)@2017-01-01 08:10:00, Point(0 1)@2017-01-01 08:15:00)}';
SELECT tgeogpoint 'Interp=Step;
  {[Point(0 0)@2017-01-01 08:00:00, Point(0 1)@2017-01-01 08:05:00],
  [Point(0 1)@2017-01-01 08:10:00, Point(0 1)@2017-01-01 08:15:00)}';
```

For temporal geometries, it is possible to specify the spatial reference identifier (SRID) using the Extended Well-Known Text (EWKT) representation as follows:

```
SELECT tgeompoint 'SRID=5435;[Point(0 0)@2001-01-01,Point(0 1)@2001-01-02]'
SELECT tgeography 'SRID=7844;[Point(0 0)@2001-01-01,Linestring(1 0,0 1)@2001-01-02]';
```

All component geometries of a temporal geometry will then be of the given SRID. Furthermore, each component geometry can specify its SRID with the EWKT format as in the following example

```
SELECT tgeompoint '[SRID=5435;Point(0 0)@2001-01-01,SRID=5435;Point(0 1)@2001-01-02]';
```

An error is raised if the component geometries are not all in the same SRID or if the SRID of a component geometry is different from the one of the temporal point, as shown below.

```
SELECT tgeompoint '[SRID=5435;Point(0 0)@2001-01-01,SRID=4326;Point(0 1)@2001-01-02]';
-- ERROR: Geometry SRID (4326) does not match temporal type SRID (5435)
SELECT tgeography 'SRID=7844;
  [SRID=4326;Point(0 0)@2001-01-01, SRID=4326;Linestring(1 0,0 1)@2001-01-02]';
-- ERROR: Geometry SRID (4326) does not match temporal type SRID (7844)
```

If not specified, the default SRID for temporal geometric points is 0 (unknown) and for temporal geographies is 4326 (WGS 84). Temporal geometries with step interpolation may also specify the SRID, as shown next.

```
SELECT tgeompoint 'SRID=5435,Interp=Step;[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02]';
SELECT tgeogpoint 'Interp=Step;
  [SRID=4326;Point(0 0)@2001-01-01, SRID=4326;Point(0 1)@2001-01-02]';
```

We give below the input and output functions in Well-Known Text (WKT), Well-Known Binary (WKB), and Moving Features JSON (MF-JSON) format for temporal alphanumeric types. The corresponding functions for temporal geometries are detailed in Section 7.4. The default output format of all temporal alphanumeric types is the Well-Known Text format. The function `asText` given next enables to determine the output of temporal float values.

- Return the Well-Known Text (WKT) representation

  ```
  asText(ttype,maxdecimaldigits integer=15) → text
  ```

  The `maxdecimaldigits` argument can be used to set the maximum number of decimal places in the output of floating point values (default 15).

  ```
  SELECT asText(tfloat '[10.55@2001-01-01, 25.55@2001-01-02]', 0);
  -- [11@2001-01-01, 26@2001-01-02]
  SELECT asText(tgeometry '[Point(1.55 1.55)@2001-01-01,
    Linestring(1.55 1.55,3.55 3.55)@2001-01-02]', 0);
  -- [POINT(1 1)@2001-01-01, LINESTRING(1 1,3 3)@2001-01-02]
  ```

- Return the Moving Features JSON (MF-JSON) representation

  ```
  asMFJSON(ttype,options integer=0,flags integer=0,maxdecimaldigits integer=15) → bytea
  ```

  The `options` argument can be used to add a bounding box in the MFJSON output:

  - 0: means no option (default value)
  - 1: MFJSON BBOX

  The `flags` argument can be used to customize the JSON output, for example, to produce an easy-to-read (for human readers) JSON output. Refer to the documentation of the `json-c` library for the possible values. Typical values are as follows:

  - 0: means no option (default value)
  - 1: JSON_C_TO_STRING_SPACED
  - 2: JSON_C_TO_STRING_PRETTY

  The `maxdecimaldigits` argument can be used to set the maximum number of decimal places in the output of floating point values (default 15).

  ```
  SELECT asMFJSON(tbool 't@2001-01-01 18:00:00', 1);
  /* {"type":"MovingBoolean","period":{"begin":"2001-01-01T18:00:00",
     "end":"2001-01-01T18:00:00","lowerInc":true,"upperInc":true},
     "values":[true],"datetimes":["2001-01-01T18:00:00"],"interpolation":"None"} */
  SELECT asMFJSON(tint '{10@2001-01-01 18:00:00, 25@2001-01-01 18:10:00}', 1);
  /* {"type":"MovingInteger","bbox":[10,25],"period":{"begin":"2001-01-01T18:00:00",
     "end":"2001-01-01T18:10:00"},"values":[10,25],"datetimes":["2001-01-01T18:00:00",
     "2001-01-01T18:10:00"],"lowerInc":true,"upperInc":true, "interpolation":"Discrete"} */
  SELECT asMFJSON(tfloat '[10.5@2001-01-01 18:00:00+02, 25.5@2001-01-01 18:10:00+02]');
  /* {"type":"MovingFloat","values":[10.5,25.5],"datetimes":["2001-01-01T17:00:00",
     "2001-01-01T17:10:00"],"lowerInc":true,"upperInc":true,"interpolation":"Linear"} */
  SELECT asMFJSON(ttext '{[walking@2001-01-01 18:00:00+02,
    driving@2001-01-01 18:10:00+02]}');
  /* {"type":"MovingText","sequences":[{"values":["walking","driving"],
     "datetimes":["2001-01-01T17:00:00","2001-01-01T17:10:00"],
     "lowerInc":true,"upperInc":true}],"interpolation":"Step"} */
  SELECT asMFJSON(tgeometry '{[Point(1 1)@2001-01-01 18:00:00+02,
    Linestring(1 1,2 2)@2001-01-01 18:10:00+02]}');
  /* {"type":"MovingGeometry","sequences":[{"values":[{"type":"Point",
     "coordinates":[1,1]},{"type":"LineString","coordinates":[[1,1],[2,2]]}],
     "datetimes":["2001-01-01T17:00:00","2001-01-01T17:10:00"],
     "lower_inc":true,"upper_inc":true}],"interpolation":"Step"} */
  ```

- Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation

  ```
  asBinary(ttype,endian text='') → bytea
  asHexWKB(ttype,endian text='') → text
  ```

  The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used.

  ```
  SELECT asBinary(tbool 'true@2001-01-01');
  -- \x011a000101009c57d3c11c0000
  SELECT asBinary(tfloat '1.5@2001-01-01');
  -- \x01210001000000000000f83f009c57d3c11c0000
  SELECT asHexWKB(tint '1@2001-01-01', 'XDR');
  -- 000023010000000100001CC1D3579C00
  SELECT asHexWKB(ttext 'AAA@2001-01-01');
  -- 01290001040000000000000041414100009C57D3C11C0000
  ```

- Input from the Well-Known Text (WKT) representation

```
ttypeFromText(text) → ttype
```

There is one function per temporal type, the name of the function has as prefix the name of the type, which is `tbool`, `tint`, `tfloat` or `ttext` in the examples below. The function is the inverse of `asText`: it reads the text a literal of the type is written in, from a constant or from a `text` column alike, so that `tintFromText(col)` and `col::tint` give the same value. The function of a spatiotemporal type is given in tspatialFromText and that of `tjsonb` in tjsonbFromText.

```
SELECT tboolFromText('[true@2001-01-01, false@2001-01-03]');
-- [t@2001-01-01, f@2001-01-03]
SELECT tintFromText('[1@2001-01-01, 2@2001-01-02]');
-- [1@2001-01-01, 2@2001-01-02]
SELECT tfloatFromText('Interp=Step;[1.5@2001-01-01, 2.5@2001-01-02]');
-- Interp=Step;[1.5@2001-01-01, 2.5@2001-01-02]
SELECT ttextFromText('{"walk"@2001-01-01, "bus"@2001-01-02}');
-- {"walk"@2001-01-01, "bus"@2001-01-02}
SELECT tintFromText(t) FROM (VALUES (text '[1@2001-01-01, 2@2001-01-02]')) v(t);
-- [1@2001-01-01, 2@2001-01-02]
```

- Input from the Moving Features JSON (MF-JSON) representation

```
ttypeFromMFJSON(bytea) → ttype
```

There is one function per temporal type, the name of the function has as prefix the name of the type, which is `tbool` or `tint` in the examples below.

```
SELECT tboolFromMFJSON(text '{"type":"MovingBoolean",
  "period":{"begin":"2001-01-01T18:00:00+01", "end":"2001-01-01T18:00:00+01",
  "lowerInc":true,"upperInc":true}, "values":[true],
  "datetimes":["2001-01-01T18:00:00+01"],"interpolation":"None"}');
-- t@2001-01-01 18:00:00
SELECT tintFromMFJSON(text '{"type":"MovingInteger","bbox":[10,25],
  "period":{"begin":"2001-01-01T18:00:00+01", "end":"2001-01-01T18:10:00+01"},
  "values":[10,25],"datetimes":["2001-01-01T18:00:00+01", "2001-01-01T18:10:00+01"],
  "lowerInc":true,"upperInc":true, "interpolation":"Discrete"}');
-- {10@2001-01-01 18:00:00, 25@2001-01-01 18:10:00}
SELECT tfloatFromMFJSON(text '{"type":"MovingFloat","values":[10.5,25.5],
  "datetimes":["2001-01-01T17:00:00+01", "2001-01-01T17:10:00+01"],"lowerInc":true,
  "upperInc":true, "interpolation":"Linear"}');
-- [10.5@2001-01-01 18:00:00, 25.5@2001-01-01 18:10:00]'
SELECT ttextFromMFJSON(text '{"type":"MovingText",
  "sequences":[{"values":["walking","driving"],
  "datetimes":["2001-01-01T17:00:00+01","2001-01-01T17:10:00+01"], "lowerInc":true,
  "upperInc":true}],"interpolation":"Step"}');
-- {["walking"@2001-01-01 18:00:00, "driving"@2001-01-01 18:10:00]}');
SELECT asText(tgeometryFromMFJSON(text '{"type":"MovingGeometry",
  "sequences":[{"values":[{"type":"Point", "coordinates":[1,1]},
  {"type":"LineString","coordinates":[[1,1],[2,2]]}],
  "datetimes":["2001-01-01T17:00:00+01","2001-01-01T17:10:00+01"], "lower_inc":true,
  "upper_inc":true}],"interpolation":"Step"}'));
-- {[POINT(1 1)@2001-01-01 17:00:00, LINESTRING(1 1,2 2)@2001-01-01 17:10:00]}
```

- Input from the Well-Known Binary (WKB) or from the Hexadecimal Well-Known Binary (HexWKB) representation

```
ttypeFromBinary(bytea) → ttype
ttypeFromHexWKB(text) → ttype
```

There is one function per temporal type, the name of the function has as prefix the name of the type, which is `tbool` or `tint` in the examples below.

```
SELECT tboolFromBinary('\x011a000101009c57d3c11c0000');
-- t@2001-01-01
```

```
SELECT tintFromBinary('\x000023010000000100001cc1d3579c00');
-- 1@2001-01-01
SELECT tfloatFromBinary('\x0121000100000000000000f83f009c57d3c11c0000');
-- 1.5@2001-01-01
SELECT ttextFromBinary('\x012900010400000000000000004141410000 9c57d3c11c0000');
-- "AAA"@2001-01-01
SELECT tboolFromHexWKB('011A000101009C57D3C11C0000');
-- t@2001-01-01
SELECT tintFromHexWKB('000023010000000100001CC1D3579C00');
-- 1@2001-01-01
SELECT tfloatFromHexWKB('0121000100000000000000F83F009C57D3C11C0000');
-- 1.5@2001-01-01
SELECT ttextFromHexWKB('0129000104000000000000000041414100009C57D3C11C0000');
-- "AAA"@2001-01-01
```

4.7 Constructors

We give next the constructor functions for the various subtypes. Using the constructor function is frequently more convenient than writing a literal constant.

- Constructors for temporal types having a constant value

  ```
  ttype(base,timestamptz) → ttypeInst
  ttype(base,tstzset) → ttypeDiscSeq
  ttype(base,tstzspan,interp text='linear') → ttypeContSeq
  ttype(base,tstzspanset,interp text='linear') → ttypeSeqSet
  ```

  These contructors have two arguments, a base type and a time value, where the latter is a `timestamptz`, a `tstzset`, a `tstzspan`, or a `tstzspanset` value for constructing, respectively, an instant, a sequence with discrete interpolation, a sequence with linear or step interpolation, or a sequence set value. The functions for sequence or sequence set values with continuous base type have an optional third argument stating whether the resulting temporal value has linear or step interpolation. Linear interpolation is assumed by default if the argument is not specified.

  ```
  SELECT tbool(true, timestamptz '2001-01-01');
  SELECT tint(1, timestamptz '2001-01-01');
  SELECT tfloat(1.5, tstzset '{2001-01-01, 2001-01-02}');
  SELECT ttext('AAA', tstzset '{2001-01-01, 2001-01-02}');
  SELECT tfloat(1.5, tstzspan '[2001-01-01, 2001-01-02]');
  SELECT tfloat(1.5, tstzspan '[2001-01-01, 2001-01-02]', 'step');
  SELECT tgeompoint('Point(0 0)', tstzspan '[2001-01-01, 2001-01-02]');
  SELECT tgeography('SRID=7844;Point(0 0)',
    tstzspanset '{[2001-01-01, 2001-01-02], [2001-01-03, 2001-01-04]}', 'step');
  ```

- Constructors for temporal types of sequence subtype

  ```
  ttypeSeq(ttypeInst[],interp text={'step','linear'},lowerInc boolean=true,
    upperInc boolean=true) → ttypeSeq
  ```

  These contructors have a first mandatory argument, which is an array of values of the corresponding instant values and additional optional arguments. The second argument states the interpolation. If the argument is not given, it is by default step for discrete base types such as integer, and linear for continuous base types such as float. An error is raised when linear interpolation is stated for temporal values with discrete base types. For continuous sequences, the third and fourth arguments are Boolean values stating, respectively, whether the left and right bounds are inclusive or exclusive. These arguments are assumed to be true by default if they are not specified.

  ```
  SELECT tboolSeq(ARRAY[tbool 'true@2001-01-01 08:00:00','false@2001-01-01 08:05:00'],
    'discrete');
  SELECT tintSeq(ARRAY[tint '1@2001-01-01 08:00:00', '2@2001-01-01 08:05:00']);
  ```

```
SELECT tintSeq(ARRAY[tint '1@2001-01-01 08:00:00', '2@2001-01-01 08:05:00'], 'linear');
-- ERROR:  The temporal type cannot have linear interpolation
SELECT tfloatSeq(ARRAY[tfloat '1.0@2001-01-01 08:00:00', '2.0@2001-01-01 08:05:00'],
  'step', false, true);
SELECT ttextSeq(ARRAY[ttext 'AAA@2001-01-01 08:00:00', 'BBB@2001-01-01 08:05:00']);
SELECT tgeompointSeq(ARRAY[tgeompoint 'Point(0 0)@2001-01-01 08:00:00',
  'Point(0 1)@2001-01-01 08:05:00', 'Point(1 1)@2001-01-01 08:10:00'], 'discrete');
SELECT tgeographySeq(ARRAY[tgeography 'Point(1 1)@2001-01-01 08:00:00',
  'Point(2 2)@2001-01-01 08:05:00']);
```

- Constructors for temporal types of sequence set subtype

```
ttypeSeqSet(ttypeContSeq[]) → ttypeSeqSet
ttypeSeqSetGaps(ttypeInst[],maxt interval=NULL,maxdist float=NULL,
  interp text='linear') → ttypeSeqSet
```

A first set of contructors have a single argument, which is an array of values of the corresponding *sequence* values. The interpolation of the resulting temporal value depends on the interpolation of the composing sequences. An error is raised if the sequences composing the array have different interpolation.

```
SELECT tboolSeqSet(ARRAY[tbool '[false@2001-01-01 08:00:00, false@2001-01-01 08:05:00)',
  '[true@2001-01-01 08:05:00]','(false@2001-01-01 08:05:00, false@2001-01-01 08:10:00)']);
SELECT tintSeqSet(ARRAY[tint '[1@2001-01-01 08:00:00, 2@2001-01-01 08:05:00,
  2@2001-01-01 08:10:00, 2@2001-01-01 08:15:00)']);
SELECT tfloatSeqSet(ARRAY[tfloat '[1.0@2001-01-01 08:00:00, 2.0@2001-01-01 08:05:00,
  2.0@2001-01-01 08:10:00]', '[2.0@2001-01-01 08:15:00, 3.0@2001-01-01 08:20:00)']);
SELECT tfloatSeqSet(ARRAY[tfloat 'Interp=Step;
  [1.0@2001-01-01 08:00:00, 2.0@2001-01-01 08:05:00, 2.0@2001-01-01 08:10:00]',
  'Interp=Step;[3.0@2001-01-01 08:15:00, 3.0@2001-01-01 08:20:00)']);
SELECT ttextSeqSet(ARRAY[ttext '[AAA@2001-01-01 08:00:00, AAA@2001-01-01 08:05:00)',
  '[BBB@2001-01-01 08:10:00, BBB@2001-01-01 08:15:00)']);
SELECT tgeompointSeqSet(ARRAY[tgeompoint '[Point(0 0)@2001-01-01 08:00:00,
  Point(0 1)@2001-01-01 08:05:00, Point(0 1)@2001-01-01 08:10:00)',
  '[Point(0 1)@2001-01-01 08:15:00, Point(0 0)@2001-01-01 08:20:00)']);
SELECT tgeographySeqSet(ARRAY[tgeography '[Point(0 0)@2001-01-01 08:00:00,
  Point(0 0)@2001-01-01 08:05:00)',
  '[Point(1 1)@2001-01-01 08:10:00, Point(1 1)@2001-01-01 08:15:00)']);
SELECT tfloatSeqSet(ARRAY[tfloat 'Interp=Step;
  [1.0@2001-01-01 08:00:00, 2.0@2001-01-01 08:05:00, 2.0@2001-01-01 08:10:00]',
  '[3.0@2001-01-01 08:15:00, 3.0@2001-01-01 08:20:00)']);
-- ERROR:  The temporal values must have the same interpolation
```

Another set of constructors for sequence set values have as first argument an array of the corresponding *instant* values, and two optional arguments stating a maximum time interval and a maximum distance such that a gap is introduced between sequences of the result whenever two consecutive input instants have a time gap or a distance greater than these values. For temporal geometries, the distance is specified in units of the coordinate system. When none of the gap arguments are given, the resulting value will be a singleton sequence set. In addition, when the base type is continuous, an additional last argument states whether the interpolation is step or linear. It this argument is not specified it is assumed to be linear by default.

The parameters of the function depend on the temporal type. For example, the interpolation parameter is not allowed for temporal types with discrete subtype such as `tint`. Similarly, the parameter `maxdist` is not allowed for scalar types such as `ttext` that do not have a standard distance function.

```
SELECT tintSeqSetGaps(ARRAY[tint '1@2001-01-01', '3@2001-01-02', '4@2001-01-03',
  '5@2001-01-05']);
-- {[1@2001-01-01, 3@2001-01-02, 4@2001-01-03, 5@2001-01-05]}
  SELECT tbigintSeqSetGaps(ARRAY[tbigint '1@2001-01-01', '3@2001-01-02', '4@2001-01-03',
    '5@2001-01-05'], '1 day', 1);
-- {[1@2001-01-01], [3@2001-01-02, 4@2001-01-03], [5@2001-01-05]}
SELECT ttextSeqSetGaps(ARRAY[ttext 'AA@2001-01-01', 'BB@2001-01-02', 'AA@2001-01-03',
  'CC@2001-01-05'], '1 day');
-- {["AA"@2001-01-01, "BB"@2001-01-02, "AA"@2001-01-03], ["CC"@2001-01-05]}
```

```
SELECT asText(tgeometrySeqSetGaps(ARRAY[tgeometry 'Point(1 1)@2001-01-01',
  'Linestring(2 2,3 3)@2001-01-02', 'Point(4 2)@2001-01-03',
  'Polygon((5 0,6 1,7 0,5 0))@2001-01-05'], '1 day', 1));
/* {[POINT(1 1)@2001-01-01], [LINESTRING(2 2,3 3)@2001-01-02],
   [POINT(4 2)@2001-01-03], [POLYGON((5 0,6 1,7 0,5 0))@2001-01-05]} */
SELECT asText(tgeompointSeqSetGaps(ARRAY[tgeompoint 'Point(1 1)@2001-01-01',
  'Point(2 2)@2001-01-02', 'Point(3 2)@2001-01-03', 'Point(3 2)@2001-01-05'], '1 day', 1,
  'step'));
/* Interp=Step;{[POINT(1 1)@2001-01-01], [POINT(2 2)@2001-01-02, POINT(3 2)@2001-01-03],
   [POINT(3 2)@2001-01-05]} */
```

4.8 Conversions

A temporal value can be converted into a compatible type using the notation `CAST(ttype1 AS ttype2)` or `ttype1::ttype2`.

- Convert a temporal value to a timestamptz span

```
ttype::tstzspan
```

```
SELECT tint '[1@2001-01-01, 2@2001-01-03]'::tstzspan;
-- [2001-01-01, 2001-01-03]
SELECT ttext '(A@2001-01-01, B@2001-01-03, C@2001-01-05]'::tstzspan;
-- (2001-01-01, 2001-01-05]
SELECT tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03]'::tstzspan;
-- [2001-01-01, 2001-01-03]
```

4.9 Accessors

- Return the memory size in bytes

```
memSize(ttype) → integer
```

```
SELECT memSize(tint '{1@2001-01-01, 2@2001-01-02, 3@2001-01-03}');
-- 176
```

- Return the temporal type

```
tempSubtype(ttype) → {'Instant','Sequence','SequenceSet'}
```

```
SELECT tempSubtype(tint '[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- Sequence
```

- Return the base type

```
tempBasetype(ttype) → text
```

```
SELECT tempBasetype(tint '[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- integer
SELECT tempBasetype(tgeompoint 'Point(1 1)@2001-01-01');
-- geometry
```

- Return the interpolation

```
interp(ttype) → {'None','Discrete','Step','Linear'}
```

```
SELECT interp(tbool 'true@2001-01-01');
-- None
SELECT interp(tfloat '{1@2001-01-01, 2@2001-01-02, 3@2001-01-03}');
-- Discrete
SELECT interp(tint '[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- Step
SELECT interp(tfloat '[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- Linear
SELECT interp(tfloat 'Interp=Step;[1@2001-01-01, 2@2001-01-02, 3@2001-01-03]');
-- Step
SELECT interp(tgeompoint 'Interp=Step;
  [Point(1 1)@2001-01-01, Point(2 2)@2001-01-02, Point(3 3)@2001-01-03]');
-- Step
SELECT interp(tgeometry '[Point(1 1)@2001-01-01, Linestring(1 1,2 2)@2001-01-02,
  Polygon((3 3,4 4,5 3,3 3))@2001-01-03]');
-- Step
```

- Return the value or the timestamp of an instant

```
getValue(ttypeInst) → base
getTimestamp(ttypeInst) → timestamptz
```

```
SELECT getValue(tint '1@2001-01-01');
-- 1
SELECT getTimestamp(tfloat '1@2001-01-01');
-- 2001-01-01
```

- Return the values or the time on which a temporal value is defined

```
getValues(ttype) → {boolean[],numspanset,set}
getTime(ttype) → tstzspanset
```

```
SELECT getValues(tbool '[false@2001-01-01, true@2001-01-02, false@2001-01-03]');
-- {f,t}
SELECT getValues(tint '[1@2001-01-01, 3@2001-01-02, 1@2001-01-03]');
-- {[1, 2), [3, 4)}
SELECT getValues(tint '{[1@2001-01-01, 2@2001-01-02, 1@2001-01-03],
  [4@2001-01-04, 4@2001-01-05]}');
-- {[1, 3), [4, 5)}
SELECT getValues(tfloat '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03}');
-- {[1, 1], [2, 2]}
SELECT getValues(tfloat 'Interp=Step;
  {[1@2001-01-01, 2@2001-01-02, 1@2001-01-03], [3@2001-01-04, 3@2001-01-05]}');
-- {[1, 1], [2, 2], [3, 3]}
SELECT getValues(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]');
-- {[1, 2]}
SELECT getValues(tfloat '{[1@2001-01-01, 2@2001-01-02, 1@2001-01-03],
  [3@2001-01-04, 3@2001-01-05]}');
-- {[1, 2], [3, 3]}
```

```
SELECT getTime(ttext 'walking@2001-01-01');
-- {[2001-01-01, 2001-01-01]}
SELECT getTime(tfloat '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03}');
-- {[2001-01-01, 2001-01-01], [2001-01-02, 2001-01-02], [2001-01-03, 2001-01-03]}
SELECT getTime(tint '[1@2001-01-01, 1@2001-01-15)');
-- {[2001-01-01, 2001-01-15)}
SELECT getTime(tfloat '{[1@2001-01-01, 1@2001-01-10), [12@2001-01-12, 12@2001-01-15]}');
-- {[2001-01-01, 2001-01-10), [2001-01-12, 2001-01-15]}
```

- Return the bounding period on which a temporal value is defined

```
timeSpan(ttype) → tstzspan
```

```
SELECT timeSpan(tfloat '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03}');
-- [2001-01-01, 2001-01-03]
SELECT timeSpan(tint '[1@2001-01-01, 1@2001-01-15)');
-- [2001-01-01, 2001-01-15)
```

- Return the start, end, or n-th value

```
startValue(ttype) → base
endValue(ttype) → base
valueN(ttype,integer) → base
```

The functions do not take into account whether the bounds are inclusive or not.

```
SELECT startValue(tfloat '(1@2001-01-01, 2@2001-01-03)');
-- 1
SELECT endValue(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- 5
SELECT valueN(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}', 3);
-- 3
```

- Return the value at a timestamp

```
valueAtTimestamp(ttype,timestamptz) → base
```

```
SELECT valueAtTimestamp(tfloat '[1@2001-01-01, 4@2001-01-04)', '2001-01-02');
-- 2
```

- Return the time interval

```
duration(ttype,boundspan boolean=false) → interval
```

An additional parameter can be set to true to compute the duration of the bounding time span, thus ignoring the potential time gaps

```
SELECT duration(tfloat '{1@2001-01-01, 2@2001-01-03, 2@2001-01-05}');
-- 00:00:00
SELECT duration(tfloat '{1@2001-01-01, 2@2001-01-03, 2@2001-01-05}', true);
-- 4 days
SELECT duration(tfloat '[1@2001-01-01, 2@2001-01-03, 2@2001-01-05)');
-- 4 days
SELECT duration(tfloat '{[1@2001-01-01, 2@2001-01-03), [2@2001-01-04, 2@2001-01-05)}');
-- 3 days
SELECT duration(tfloat '{[1@2001-01-01, 2@2001-01-03), [2@2001-01-04, 2@2001-01-05)}',
  true);
-- 4 days
```

- Is the start/end instant inclusive?

```
lowerInc(ttype) → boolean
upperInc(ttype) → boolean
```

```
SELECT lowerInc(tint '[1@2001-01-01, 2@2001-01-02)');
-- true
SELECT upperInc(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03)}');
-- false
```

- Return the (number of) different instants

```
numInstants(ttype) → integer
instants(ttype) → ttypeInst[]
```

```
SELECT numInstants(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03)}');
-- 3
SELECT instants(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03)}');
-- {"1@2001-01-01","2@2001-01-02","3@2001-01-03"}
```

- Return the start, end, or n-th instant

```
startInstant(ttype) → ttypeInst
endInstant(ttype) → ttypeInst
instantN(ttype,integer) → ttypeInst
```

The functions do not take into account whether the bounds are inclusive or not.

```
SELECT startInstant(tfloat '{[1@2001-01-01, 2@2001-01-02),
  (2@2001-01-02, 3@2001-01-03)}');
-- 1@2001-01-01
SELECT endInstant(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03)}');
-- 3@2001-01-03
SELECT instantN(tfloat '{[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03)}', 3);
-- 3@2001-01-03
```

- Return the (number of) different timestamps

```
numTimestamps(ttype) → integer
timestamps(ttype) → timestamptz[]
```

```
SELECT numTimestamps(tfloat '{[1@2001-01-01, 2@2001-01-03),
  [3@2001-01-03, 5@2001-01-05)}');
-- 3
SELECT timestamps(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- {"2001-01-01", "2001-01-03", "2001-01-05"}
```

- Return the start, end, or n-th timestamp

```
startTimestamp(ttype) → timestamptz
endTimestamp(ttype) → timestamptz
timestampN(ttype,integer) → timestamptz
```

The functions do not take into account whether the bounds are inclusive or not.

```
SELECT startTimestamp(tfloat '[1@2001-01-01, 2@2001-01-03)');
-- 2001-01-01
SELECT endTimestamp(tfloat '{[1@2001-01-01, 2@2001-01-03),
  [3@2001-01-03, 5@2001-01-05)}');
-- 2001-01-05
SELECT timestampN(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}',
  3);
-- 2001-01-05
```

- Return the (number of) sequences

```
numSequences({ttypeContSeq,ttypeSeqSet}) → integer
sequences({ttypeContSeq,ttypeSeqSet}) → ttypeContSeq[]
```

```
SELECT numSequences(tfloat '{[1@2001-01-01, 2@2001-01-03),
  [3@2001-01-03, 5@2001-01-05)}');
-- 2
SELECT sequences(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- {"[1@2001-01-01, 2@2001-01-03)", "[3@2001-01-03, 5@2001-01-05)"}
```

- Return the start, end, or n-th sequence

```
startSequence({ttypeContSeq,ttypeSeqSet}) → ttypeContSeq
endSequence({ttypeContSeq,ttypeSeqSet}) → ttypeContSeq
sequenceN({ttypeContSeq,ttypeSeqSet},integer) → ttypeContSeq
```

```
SELECT startSequence(tfloat '{[1@2001-01-01, 2@2001-01-03),
  [3@2001-01-03, 5@2001-01-05)}');
-- [1@2001-01-01, 2@2001-01-03)
SELECT endSequence(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- [3@2001-01-03, 5@2001-01-05)
SELECT sequenceN(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}',
  2);
-- [3@2001-01-03, 5@2001-01-05)
```

- Return the segments

```
segments({ttypeContSeq,ttypeSeqSet}) → ttypeContSeq[]
```

```
SELECT segments(tint '{[1@2001-01-01, 3@2001-01-02, 2@2001-01-03],
  (3@2001-01-03, 5@2001-01-05]}');
/* {"[1@2001-01-01, 1@2001-01-02)","[3@2001-01-02, 3@2001-01-03)","[2@2001-01-03]",
   "(3@2001-01-03, 3@2001-01-05)","[5@2001-01-05]"} */
SELECT segments(tfloat '{[1@2001-01-01, 3@2001-01-02, 2@2001-01-03],
  (3@2001-01-03, 5@2001-01-05]}');
/* {"[1@2001-01-01, 3@2001-01-02)","[3@2001-01-02, 2@2001-01-03]",
   "(3@2001-01-03, 5@2001-01-05]"} */
```

4.10 Transformations

A temporal value can be transformed to another subtype. An error is raised if the subtypes are incompatible.

- Transform a temporal value to another subtype

```
ttypeInst(ttype) → ttypeInst
ttypeSeq(ttype) → ttypeSeq
ttypeSeqSet(ttype) → ttypeSeqSet
```

```
SELECT tboolInst(tbool '{[true@2001-01-01]}');
-- t@2001-01-01
SELECT tboolInst(tbool '{[true@2001-01-01, true@2001-01-02]}');
-- ERROR: Cannot transform input value to a temporal instant
SELECT tintSeq(tint '1@2001-01-01');
-- [1@2001-01-01]
SELECT tfloatSeqSet(tfloat '2.5@2001-01-01');
-- {[2.5@2001-01-01]}
SELECT tfloatSeqSet(tfloat '{2.5@2001-01-01, 1.5@2001-01-02, 3.5@2001-01-03}');
-- {[2.5@2001-01-01],[1.5@2001-01-02],[3.5@2001-01-03]}
```

- Transform a temporal value to another interpolation

```
setInterp(ttype,interp text) → ttype
```

```
SELECT setInterp(tbool 'true@2001-01-01','discrete');
-- {t@2001-01-01}
SELECT setInterp(tfloat '{[1@2001-01-01], [2@2001-01-02], [1@2001-01-03]}', 'discrete');
-- {1@2001-01-01, 2@2001-01-02, 1@2001-01-03}
SELECT setInterp(tfloat 'Interp=Step;
  [1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 2@2001-01-04]', 'linear');
/* {[1@2001-01-01, 1@2001-01-02), [2@2001-01-02, 2@2001-01-03),
   [1@2001-01-03, 1@2001-01-04), [2@2001-01-04]} */
SELECT asText(setInterp(tgeompoint 'Interp=Step;{[Point(1 1)@2001-01-01,
  Point(2 2)@2001-01-02], [Point(3 3)@2001-01-05, Point(4 4)@2001-01-06]}', 'linear'));
/* {[POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02), [POINT(2 2)@2001-01-02],
   [POINT(3 3)@2001-01-05, POINT(3 3)@2001-01-06), [POINT(4 4)@2001-01-06]} */
SELECT setInterp(tgeometry '[Point(1 1)@2001-01-01, Linestring(1 1,2 2)@2001-01-02]',
  'linear');
-- ERROR:  The temporal type cannot have linear interpolation
```

- Return a temporal boolean stating whether each segment of a continuous temporal sequence (set) has, respectively, at least or at most a given duration

```
segmentMinDuration(ttype,interval,strict boolean=true) → tbool
segmentMaxDuration(ttype,interval,strict boolean=true) → tbool
```

If `strict` is not specified, the value `true` is assumed by default, and therefore the function checks whether the segment duration is strictly less than or greater than the interval. If the argument is set to `false`, the function checks whether the segment duration is less/greater than or *equal to* the interval.

```
SELECT segmentMinDuration(tfloat '[1@2001-01-01, 0@2001-01-03, -1@2001-01-04,
  -1@2001-01-06]', '1 day');
-- {[t@2001-01-01, f@2001-01-03, t@2001-01-04, t@2001-01-06]}
SELECT segmentMinDuration(tfloat '[1@2001-01-01, 0@2001-01-03, -1@2001-01-04,
  -1@2001-01-06]', '1 day', false);
-- {[t@2001-01-01, t@2001-01-06]}
SELECT segmentMaxDuration(tfloat '[1@2001-01-01, 0@2001-01-03, -1@2001-01-04,
  -1@2001-01-06]', '1 day');
-- {[f@2001-01-01, f@2001-01-06]}
SELECT segmentMaxDuration(tfloat '[1@2001-01-01, 0@2001-01-03, -1@2001-01-04,
  -1@2001-01-06]', '1 day', false);
-- {[f@2001-01-01, t@2001-01-03, f@2001-01-04, f@2001-01-06]}
```

- Shift and/or scale the time span of a temporal value by one or two intervals

```
shiftTime(ttype,interval) → ttype
scaleTime(ttype,interval) → ttype
shiftScaleTime(ttype,interval,interval) → ttype
```

The given intervals must be strictly greater than zero. If the time span of the temporal value is zero (for example, for a temporal instant), the result of a scale operation is the temporal value.

```
SELECT shiftTime(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}', '1 day');
-- {1@2001-01-02, 2@2001-01-04, 1@2001-01-06}
SELECT shiftTime(tfloat '[1@2001-01-01, 2@2001-01-03]', '1 day');
-- [1@2001-01-02, 2@2001-01-04]
SELECT asText(shiftTime(tgeompoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-03],
  [Point(2 2)@2001-01-04, Point(1 1)@2001-01-05]}', '1 day'));
/* {[POINT(1 1)@2001-01-02, POINT(2 2)@2001-01-04],
   [POINT(2 2)@2001-01-05, POINT(1 1)@2001-01-06]} */
SELECT scaleTime(tint '1@2001-01-01', '1 day');
-- 1@2001-01-01
```

```
SELECT scaleTime(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}', '1 day');
-- {1@2001-01-01, 2@2001-01-01 12:00:00, 1@2001-01-02}
SELECT scaleTime(tfloat '[1@2001-01-01, 2@2001-01-03]', '1 day');
-- [1@2001-01-01, 2@2001-01-02]
SELECT asText(scaleTime(tgeompoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03], [Point(2 2)@2001-01-04, Point(1 1)@2001-01-05]}', '1 day'));
/* {[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-01 06:00:00,
   POINT(1 1)@2001-01-01 12:00:00], [POINT(2 2) @2001-01-01 18:00:00,
   POINT(1 1)@2001-01-02]} */
SELECT scaleTime(tint '1@2001-01-01', '-1 day');
-- ERROR:  The interval must be positive: -1 days
SELECT shiftScaleTime(tint '1@2001-01-01', '1 day', '1 day');
-- 1@2001-01-02
SELECT shiftScaleTime(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}', '1 day','1 day');
-- {1@2001-01-02, 2@2001-01-02 12:00:00, 1@2001-01-03}
SELECT shiftScaleTime(tfloat '[1@2001-01-01, 2@2001-01-03]', '1 day', '1 day');
-- [1@2001-01-02, 2@2001-01-03]
SELECT asText(shiftScaleTime(tgeometry '{[Point(1 1)@2001-01-01,
  Linestring(1 1,2 2)@2001-01-02, Point(1 1)@2001-01-03],
  [Point(2 2)@2001-01-04, Polygon((1 1,2 2,3 1,1 1))@2001-01-05]}', '1 day', '1 day'));
/* {[POINT(1 1)@2001-01-02 00:00:00, LINESTRING(1 1,2 2)@2001-01-02 06:00:00,
   POINT(1 1)@2001-01-02 12:00:00], [POINT(2 2)@2001-01-02 18:00:00,
   POLYGON((1 1,2 2,3 1,1 1))@2001-01-03 00:00:00]} */
```

- Transform a nonlinear temporal value into a set of rows, each one is a pair composed of a base value and a period set during which the temporal value has the base value {}

```
unnest(ttype) → {(value,time)}
```

```
SELECT (un).value, (un).time
FROM (SELECT unnest(tfloat '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03}') AS un) t;
-- 1 | {[2001-01-01, 2001-01-01], [2001-01-03, 2001-01-03]}
-- 2 | {[2001-01-02, 2001-01-02]}
SELECT (un).value, (un).time
FROM (SELECT unnest(tint '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]') AS un) t;
-- 1 | {[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-03]}
-- 2 | {[2001-01-02, 2001-01-03)}
SELECT (un).value, (un).time
FROM (SELECT unnest(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]') AS un) t;
-- ERROR:  Operation not supported for temporal values with linear interpolation
SELECT ST_AsText((un).value),
  (un).time FROM (SELECT unnest(tgeompoint 'Interp=Step;
  [Point(1 1)@2001-01-01, Point(2 2)@2001-01-02, Point(1 1)@2001-01-03]') AS un) t;
--  POINT(1 1) | {[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-03]}
--  POINT(2 2) | {[2001-01-02, 2001-01-03)}
SELECT ST_AsText((un).value),
  (un).time FROM (SELECT unnest(tgeometry '[Point(1 1)@2001-01-01,
  Linestring(1 1,2 2)@2001-01-02, Point(1 1)@2001-01-03]') AS un) t;
-- POINT(1 1)           | {[2001-01-01, 2001-01-02), [2001-01-03, 2001-01-03]}
-- LINESTRING(1 1,2 2) | {[2001-01-02, 2001-01-03)}
```

Chapter 5

Temporal Types (Part 2)

5.1 Modifications

We explain next the semantics of the modification operations (that is, `insert`, `update`, and `delete`) for temporal types. These operations have similar semantics as the corresponding operations for application-time temporal tables introduced in the SQL:2011 standard. The main difference is that SQL uses *tuple timestamping* (where timestamps are attached to tuples), while temporal values in MobilityDB use *attribute timestamping* (where timestamps are attached to attribute values). Please refer to this article for a detailed discussion about tuple versus attribute timestamping in temporal databases.

The `insert` operation adds to a temporal value the instants of another one without modifying the existing instants, as illustrated in Figure 5.1.

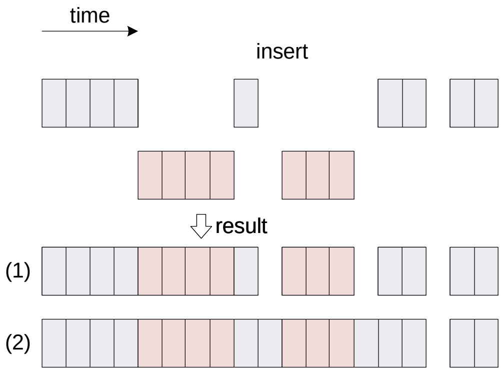

Figure 5.1: Insert operation for temporal values.

As shown in the figure, the temporal values may only intersect at their boundary, and in that case, they must have the same base value at their common timestamps, otherwise an error is raised. The result of the operation is the union of the instants for both temporal values, as shown in the first result of the figure. This is equivalent to a `merge` operation explained below. Alternatively, as shown in the second result of the figure, the inserted fragments that are disjoint with the original value are connected to the last instant before and the first instant after the fragment. A Boolean parameter `connect` is used to choose between the two results, and the parameter is set to true by default. Notice that this only applies to continuous temporal values.

The `update` operation replaces the instants in the first temporal value with those of the second one as illustrated in Figure 5.2.

As in the case of an `insert` operation, an additional Boolean parameter determines whether the replaced disconnected fragments are connected in the resulting value, as shown in the two possible results in the figure. When the two temporal values are either disjoint or only overlap at their boundary, this corresponds to an `insert` operation as explained above. In this case, the `update` operation behaves as an `upsert` operation in SQL.

The `deleteTime` operation removes the instants of a temporal value that intersect a time value. This operation can be used in two different situations, illustrated in Figure 5.2.

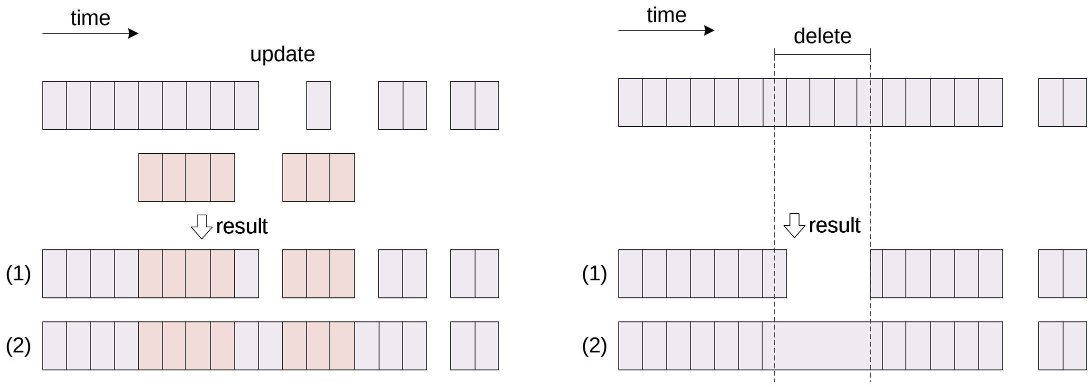

Figure 5.2: Update and delete operation for temporal values.

1. In the first case, shown as the top result in the figure, the meaning of operation is to introduce time gaps after removing the instants of the temporal value intersecting the time value. This is equivalent to the restriction operations (Section 5.2), which restrict a temporal value to the complement of the time value.
2. The second case, shown as the bottom result in the figure, is used for removing erroneous values (e.g., detected as outliers) without introducing a time gap, or for removing time gaps. In this case, the instants of the temporal value are deleted and the last instant before and the first instant after a removed fragment are connected. This behaviour is specified by setting an additional Boolean parameter of the operation. Notice that this only applies to continuous temporal values.

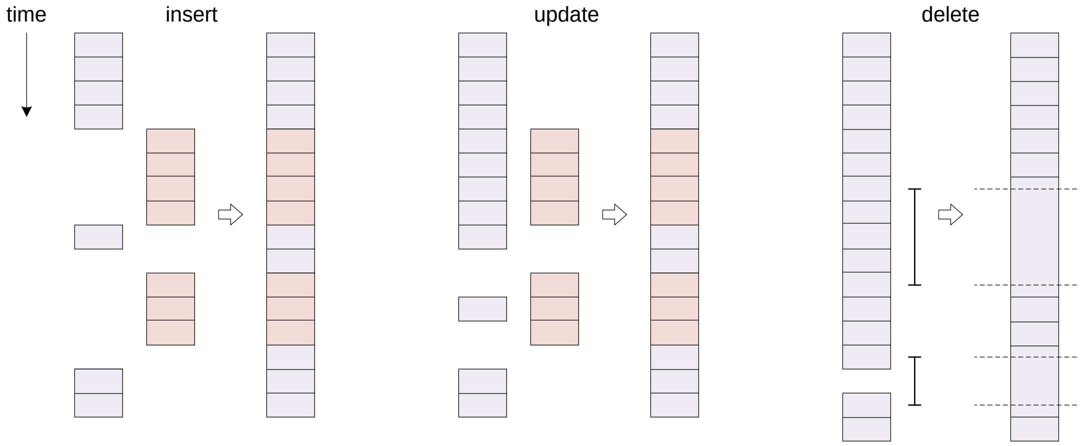

Figure 5.3: Modification operations for temporal tables in SQL

Figure 5.3 shows the equivalent modification operations for temporal tables in the SQL standard. Intuitively, these figures are obtained by rotating 90 degrees clockwise the corresponding figures for temporal values (Figure 5.1 and Figure 5.2). This follows from the fact that in SQL consecutive tuples ordered by time are typically connected through the `LEAD` and `LAG` window functions.

- Insert a temporal value into another one

```
insert(ttype,ttype,connect boolean=true) → ttype
```

```
SELECT insert(tint '{1@2001-01-01, 3@2001-01-03, 5@2001-01-05}',
  tint '{3@2001-01-03, 7@2001-01-07}');
-- {1@2001-01-01, 3@2001-01-03, 5@2001-01-05, 7@2001-01-07}
SELECT insert(tbigint '{1@2001-01-01, 3@2001-01-03, 5@2001-01-05}',
  tbigint '{5@2001-01-03, 7@2001-01-07}');
-- ERROR: The temporal values have different value at their overlapping instant 2001-01-03
SELECT insert(tfloat '[1@2001-01-01, 2@2001-01-02]',
  tfloat '[1@2001-01-03, 1@2001-01-05]');
-- [1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 1@2001-01-05]
SELECT insert(tfloat '[1@2001-01-01, 2@2001-01-02]',
  tfloat '[1@2001-01-03, 1@2001-01-05]', false);
-- {[1@2001-01-01, 2@2001-01-02], [1@2001-01-03, 1@2001-01-05]}
SELECT asText(insert(tgeompoint '{[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02],
  [Point(3 3 3)@2001-01-04],[Point(1 1 1)@2001-01-05]}',
  tgeompoint 'Point(1 1 1)@2001-01-03'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02, POINT Z (1 1 1)@2001-01-03,
   POINT Z (3 3 3)@2001-01-04], [POINT Z (1 1 1)@2001-01-05]} */
```

- Update a temporal value with another one

```
update(ttype,ttype,connect boolean=true) → ttype
```

```
SELECT update(tint '{1@2001-01-01, 3@2001-01-03, 5@2001-01-05}',
  tint '{5@2001-01-03, 7@2001-01-07}');
-- {1@2001-01-01, 5@2001-01-03, 5@2001-01-05, 7@2001-01-07}
SELECT update(tfloat '[1@2001-01-01, 1@2001-01-05]',
  tfloat '[1@2001-01-02, 3@2001-01-03, 1@2001-01-04]');
-- {[1@2001-01-01, 1@2001-01-02, 3@2001-01-03, 1@2001-01-04, 1@2001-01-05]}
SELECT asText(update(tgeompoint '{[Point(1 1 1)@2001-01-01, Point(3 3 3)@2001-01-03,
  Point(1 1 1)@2001-01-05], [Point(1 1 1)@2001-01-07]}',
  tgeompoint '[Point(2 2 2)@2001-01-02, Point(2 2 2)@2001-01-04]'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02, POINT Z (2 2 2)@2001-01-04,
   POINT Z (1 1 1)@2001-01-05], [POINT Z (1 1 1)@2001-01-07]} */
SELECT asText(update( tgeometry '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]',
  tgeometry '[Linestring(2 2,3 3)@2001-01-02, Linestring(3 3,2 2)@2001-01-04]'));
/* [POINT(1 1)@2001-01-01, LINESTRING(2 2,3 3)@2001-01-02,
   LINESTRING(3 3,2 2)@2001-01-04], (POINT(3 3)@2001-01-04, POINT(1 1)@2001-01-05] */
```

- Delete the instants of a temporal value that intersect a time value

```
deleteTime(ttype,times,connect boolean=true) → ttype
```

```
SELECT deleteTime(tint '[1@2001-01-01, 1@2001-01-03]', timestamptz '2001-01-02', false);
-- {[1@2001-01-01, 1@2001-01-02), (1@2001-01-02, 1@2001-01-03]}
SELECT deleteTime(tbigint '[1@2001-01-01, 1@2001-01-03]', timestamptz '2001-01-04');
-- [1@2001-01-01, 1@2001-01-03]
SELECT deleteTime(tfloat '[1@2001-01-01, 4@2001-01-02, 2@2001-01-04, 5@2001-01-05]',
  tstzspan '[2001-01-02, 2001-01-04]');
-- [1@2001-01-01, 5@2001-01-05]
SELECT asText(deleteTime(tgeompoint '{[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02],
  [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
  tstzspan '[2001-01-02, 2001-01-04]'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02, POINT Z (3 3 3)@2001-01-04,
   POINT Z (3 3 3)@2001-01-05]} */
```

- Append a temporal instant to a temporal value

```
appendInstant(ttype,ttypeInst,interp text=NULL) → ttype
appendInstant(ttypeInst,interp text=NULL,maxdist float=NULL,
  maxt interval=NULL) → ttypeSeq
```

The first version of the function returns the result of appending the second argument to the first one. If either input is NULL, then NULL is returned. The second version, the *aggregate* `appendInstant`, returns the result of successively appending a set of rows of temporal values. This means that it operates in the same way the `SUM()` and `AVG()` functions do and like most aggregates, it also ignores NULL values. Two optional arguments state a maximum distance and a maximum time interval such that a gap is introduced whenever two consecutive instants have a distance or a time gap greater than these values. For temporal points the distance is specified in units of the coordinate system. If one of the arguments are not given, it is not taken into account for determining the gaps. The maximum distance is an argument of the types whose values have a distance: the temporal numbers, points, geometries and geographies, circular buffers, network points, poses and rigid geometries; the other temporal types take the interpolation and the maximum time interval alone, as in `appendInstant(ttext, interp text, maxt interval)`. Over a single row the aggregate returns a sequence of one instant. DuckDB, Apache Spark and Apache Flink call this aggregate `appendInstantAgg` (Section 18.10.1).

```
SELECT appendInstant(tint '1@2001-01-01', tint '1@2001-01-02');
-- [1@2001-01-01, 1@2001-01-02]
SELECT appendInstant(tbigint '1@2001-01-01', tbigint '1@2001-01-02', 'discrete');
-- {1@2001-01-01, 1@2001-01-02}
SELECT appendInstant(tint '[1@2001-01-01]', tint '1@2001-01-02');
-- [1@2001-01-01, 1@2001-01-02]
SELECT asText(appendInstant(tgeompoint '{[Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02], [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
  tgeompoint 'Point(1 1 1)@2001-01-06'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02],
   [POINT Z (3 3 3)@2001-01-04, POINT Z (3 3 3)@2001-01-05,
   POINT Z (1 1 1)@2001-01-06]} */
SELECT asText(appendInstant(tgeometry '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
  [Linestring(2 2,3 3)@2001-01-04, Point(3 3)@2001-01-05]}',
  tgeometry 'Linestring(1 1,2 2)@2001-01-06'));
/* {[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02], [LINESTRING(2 2,3 3)@2001-01-04,
   POINT(3 3)@2001-01-05, LINESTRING(1 1,2 2)@2001-01-06]} */
```

```
WITH temp(inst) AS (
  SELECT tfloat '1@2001-01-01' UNION SELECT tfloat '2@2001-01-02' UNION
  SELECT tfloat '3@2001-01-03' UNION SELECT tfloat '4@2001-01-04' UNION
  SELECT tfloat '5@2001-01-05' )
SELECT appendInstant(inst ORDER BY inst) FROM temp;
-- [1@2001-01-01, 5@2001-01-05]
WITH temp(inst) AS (
  SELECT tfloat '1@2001-01-01' UNION SELECT tfloat '2@2001-01-02' UNION
  SELECT tfloat '3@2001-01-03' UNION SELECT tfloat '4@2001-01-04' UNION
  SELECT tfloat '5@2001-01-05' )
SELECT appendInstant(inst, 'discrete' ORDER BY inst) FROM temp;
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 4@2001-01-04, 5@2001-01-05}
WITH temp(inst) AS (
  SELECT tgeogpoint 'Point(1 1)@2001-01-01' UNION
  SELECT tgeogpoint 'Point(2 2)@2001-01-02' UNION
  SELECT tgeogpoint 'Point(3 3)@2001-01-03' UNION
  SELECT tgeogpoint 'Point(4 4)@2001-01-04' UNION
  SELECT tgeogpoint 'Point(5 5)@2001-01-05' )
SELECT asText(appendInstant(inst ORDER BY inst)) FROM temp;
/* [POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02, POINT(3 3)@2001-01-03,
   POINT(4 4)@2001-01-04, POINT(5 5)@2001-01-05] */
```

Notice that in the first query above with `tfloat`, the intermediate observations were removed by the normalization process since they were redundant due to linear interpolation. This is not the case for the second query with discrete interpolation or the third query with `tgeogpoint`, where geodetic coordinates are used.

```
WITH temp(inst) AS (
  SELECT tfloat '1@2001-01-01' UNION SELECT tfloat '2@2001-01-02' UNION
  SELECT tfloat '4@2001-01-04' UNION SELECT tfloat '5@2001-01-05' UNION
  SELECT tfloat '7@2001-01-07' )
SELECT appendInstant(inst, NULL, 0.0, '1 day' ORDER BY inst) FROM temp;
-- {[1@2001-01-01, 2@2001-01-02], [4@2001-01-04, 5@2001-01-05], [7@2001-01-07]}
WITH temp(inst) AS (
  SELECT tgeompoint 'Point(1 1)@2001-01-01' UNION
  SELECT tgeompoint 'Point(2 2)@2001-01-02' UNION
  SELECT tgeompoint 'Point(4 4)@2001-01-04' UNION
  SELECT tgeompoint 'Point(5 5)@2001-01-05' UNION
  SELECT tgeompoint 'Point(7 7)@2001-01-07' )
SELECT asText(appendInstant(inst, NULL, sqrt(2), '1 day' ORDER BY inst)) FROM temp;
/* {[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02],
   [POINT(4 4)@2001-01-04, POINT(5 5)@2001-01-05], [POINT(7 7)@2001-01-07]} */
```

- Append a temporal sequence to a temporal value

```
appendSequence(ttype,ttypeSeq) → {ttypeSeq,ttypeSeqSet}
appendSequence(ttypeSeq) → {ttypeSeq,ttypeSeqSet}
```

The first version of the function returns the result of appending the second argument to the first one. If either input is NULL, then NULL is returned. The second version, the *aggregate* `appendSequence`, returns the result of successively appending a set of rows of temporal values. This means that it operates in the same way the `SUM()` and `AVG()` functions do and like most aggregates, it also ignores NULL values. The bare name `appendSequence` names the same aggregate and is kept for backward compatibility.

```
SELECT appendSequence(tint '1@2001-01-01', tint '{2@2001-01-02, 3@2001-01-03}');
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03}
SELECT appendSequence(tbigint '[1@2001-01-01, 2@2001-01-02]',
  tbigint '[2@2001-01-02, 3@2001-01-03]');
-- [1@2001-01-01, 2@2001-01-02, 3@2001-01-03]
SELECT asText(appendSequence(tgeompoint '{[Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02], [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
  tgeompoint '[Point(3 3 3)@2001-01-05, Point(1 1 1)@2001-01-06]'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02],
   [POINT Z (3 3 3)@2001-01-04, POINT Z (3 3 3)@2001-01-05,
   POINT Z (1 1 1)@2001-01-06]} */
SELECT asText(appendSequence(tgeometry '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
  [Linestring(3 3,2 2)@2001-01-04, Point(3 3)@2001-01-05]}',
  tgeometry '[Point(3 3)@2001-01-05, Linestring(3 3,1 1)@2001-01-06]'));
/* {[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02], [LINESTRING(3 3,2 2)@2001-01-04,
   POINT(3 3)@2001-01-05, LINESTRING(3 3,1 1)@2001-01-06]} */
```

- Merge the temporal values

```
merge(ttype,ttype) → ttype
merge(ttype[]) → ttype
merge(ttype) → ttype
```

The temporal values may only intersect at their boundary. In that case, for all temporal types excepted `tgeometry` and `tgeography`, their base values at the common timestamps must be the same, otherwise an error is raised. For the `tgeometry` and `tgeography` types, if the value at their common timestamps is different, the resulting value after the `merge` will be the spatial union of the values. The reason for a different behaviour for these types is that they are the only temporal types that are not *scalar* types, that is, their value at a given timestamp is not a single element of the domain of the base type, but rather a subset of their domain. A NULL operand of the function over two values is ignored, so that `merge(v, NULL)` returns `v`. The function `merge` is an *aggregate* function that returns the result of successively appending a set of rows of temporal values. This means that it operates in the same way the `SUM()` and `AVG()` functions do and like most aggregates, it also ignores NULL values. The bare name `merge` names the same aggregate and is kept for backward compatibility.

```
SELECT merge(tint '1@2001-01-01', tint '1@2001-01-02');
-- {1@2001-01-01, 1@2001-01-02}
SELECT merge(tbigint '[1@2001-01-01, 2@2001-01-02]',
  tbigint '[2@2001-01-02, 1@2001-01-03]');
-- [1@2001-01-01, 2@2001-01-02, 1@2001-01-03]
SELECT merge(tint '[1@2001-01-01, 2@2001-01-02]', tint '[3@2001-01-03, 1@2001-01-04]');
-- {[1@2001-01-01, 2@2001-01-02], [3@2001-01-03, 1@2001-01-04]}
SELECT merge(tint '[1@2001-01-01, 2@2001-01-02]', tint '[1@2001-01-02, 2@2001-01-03]');
```

```
-- ERROR:  The temporal values have different value at their common timestamp 2001-01-02
SELECT asText(merge(tgeompoint '{[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02],
  [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
  tgeompoint '{[Point(3 3 3)@2001-01-05, Point(1 1 1)@2001-01-06]}'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02],
   [POINT Z (3 3 3)@2001-01-04, POINT Z (3 3 3)@2001-01-05,
   POINT Z (1 1 1)@2001-01-06]} */
SELECT asText(merge(tgeometry 'Linestring(1 1,5 1)@2001-01-01',
  tgeometry '{Linestring(5 1,10 1)@2001-01-01, Point(3 3)@2001-01-02}'));
-- {LINESTRING(1 1,5 1,10 1)@2001-01-01, POINT(3 3)@2001-01-02}
```

```
SELECT merge(ARRAY[tint '1@2001-01-01', '1@2001-01-02']);
-- {1@2001-01-01, 1@2001-01-02}
SELECT merge(ARRAY[tint '{1@2001-01-01, 2@2001-01-02}', '{2@2001-01-02, 3@2001-01-03}']);
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03}
SELECT merge(ARRAY[tint '{1@2001-01-01, 2@2001-01-02}', '{3@2001-01-03, 4@2001-01-04}']);
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 4@2001-01-04}
SELECT merge(ARRAY[tbigint '[1@2001-01-01, 2@2001-01-02]',
  '[2@2001-01-02, 1@2001-01-03]']);
-- [1@2001-01-01, 2@2001-01-02, 1@2001-01-03]
SELECT merge(ARRAY[tbigint '[1@2001-01-01, 2@2001-01-02]',
  '[3@2001-01-03, 4@2001-01-04]']);
-- {[1@2001-01-01, 2@2001-01-02], [3@2001-01-03, 4@2001-01-04]}
SELECT asText(merge(ARRAY[tgeompoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
  [Point(3 3)@2001-01-03, Point(4 4)@2001-01-04]}', '{[Point(4 4)@2001-01-04,
  Point(3 3)@2001-01-05], [Point(6 6)@2001-01-06, Point(7 7)@2001-01-07]}']));
/* {[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02], [POINT(3 3)@2001-01-03,
   POINT(4 4)@2001-01-04, POINT(3 3)@2001-01-05],
   [POINT(6 6)@2001-01-06, POINT(7 7)@2001-01-07]} */
SELECT asText(merge(ARRAY[tgeompoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02]}',
  '{[Point(2 2)@2001-01-02, Point(1 1)@2001-01-03]}']));
-- [POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02, POINT(1 1)@2001-01-03]
SELECT asText(merge(ARRAY[tgeometry '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02]}',
  '{[Linestring(2 2,1 1)@2001-01-02, Point(1 1)@2001-01-03]}']));
-- {[POINT(1 1)@2001-01-01, LINESTRING(2 2,1 1)@2001-01-02, POINT(1 1)@2001-01-03]}
```

```
WITH temp(inst) AS (
SELECT tfloat '1@2001-01-01' UNION SELECT tfloat '2@2001-01-02' UNION
SELECT tfloat '3@2001-01-03' UNION SELECT tfloat '4@2001-01-04' UNION
SELECT tfloat '5@2001-01-05' ) SELECT merge(inst) FROM temp;
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 4@2001-01-04, 5@2001-01-05}
```

5.2 Restrictions

There are three complementary sets of restriction functions. The first set functions restricts the temporal value with respect to a value or a time extent. Examples are `atValues` or `atTime`. The second set functions restricts the temporal value with respect to the *complement* of a value or a time extent. Examples are `minusValues` or `minusTime`. Finally, the third set functions restricts the temporal value before or after a timestamptz. Examples are `beforeTimestamp` or `afterTimestamp`.

- Restrict to (the complement of) a value or a set of values, or, for a temporal number, a span or a span set of values

```
atValue(ttype,base) → ttype
minusValue(ttype,base) → ttype
atValues(ttype,values) → ttype
minusValues(ttype,values) → ttype
atSpan(tnumber,span) → tnumber
minusSpan(tnumber,span) → tnumber
atSpanset(tnumber,spanset) → tnumber
minusSpanset(tnumber,spanset) → tnumber
```

```
SELECT atValue(tint '[1@2001-01-01, 1@2001-01-15)', 1);
-- [1@2001-01-01, 1@2001-01-15)
SELECT atValues(tfloat '[1@2001-01-01, 4@2001-01-4)', floatset '{1, 3, 5}');
-- {[1@2001-01-01], [3@2001-01-03]}
SELECT atSpan(tfloat '[1@2001-01-01, 4@2001-01-4)', floatspan '[1,3]');
-- [1@2001-01-01, 3@2001-01-03]
SELECT atSpanset(tfloat '[1@2001-01-01, 5@2001-01-05)', floatspanset '{[1,2], [3,4]}');
-- {[1@2001-01-01, 2@2001-01-02],[3@2001-01-03, 4@2001-01-04]}
SELECT asText(atValue(tgeompoint '[Point(0 0 0)@2001-01-01, Point(2 2 2)@2001-01-03)',
  geometry 'Point(1 1 1)'));
-- {[POINT Z (1 1 1)@2001-01-02]}
SELECT asText(atValues(tgeompoint '[Point(0 0)@2001-01-01, Point(2 2)@2001-01-03)',
  geomset '{"Point(0 0)", "Point(1 1)"}'));
-- {[POINT(0 0)@2001-01-01], [POINT(1 1)@2001-01-02]}
SELECT asText(atValue(tgeometry '[Point(0 0)@2001-01-01, Linestring(0 0,2 2)@2001-01-02,
  Linestring(0 0,2 2)@2001-01-03]', geometry 'Linestring(0 0,2 2)'));
-- {[LINESTRING(0 0,2 2)@2001-01-02, LINESTRING(0 0,2 2)@2001-01-03]}
```

```
SELECT minusValue(tint '[1@2001-01-01, 2@2001-01-02, 2@2001-01-03)', 1);
-- {[2@2001-01-02, 2@2001-01-03)}
SELECT minusValues(tfloat '[1@2001-01-01, 4@2001-01-4)', floatset '{2, 3}');
/* {[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 3@2001-01-03),
   (3@2001-01-03, 4@2001-01-04)} */
SELECT minusSpan(tfloat '[1@2001-01-01, 4@2001-01-4)', floatspan '[2,3]');
-- {[1@2001-01-01, 2@2001-01-02), (3@2001-01-03, 4@2001-01-04)}
SELECT minusSpanset(tfloat '[1@2001-01-01, 5@2001-01-05)', floatspanset '{[1,2], [3,4]}');
-- {(2@2001-01-02, 3@2001-01-03), (4@2001-01-04, 5@2001-01-05)}
SELECT asText(minusValue(tgeompoint '[Point(0 0 0)@2001-01-01, Point(2 2 2)@2001-01-03)',
  geometry 'Point(1 1 1)'));
/* {[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02),
   (POINT Z (1 1 1)@2001-01-02, POINT Z (2 2 2)@2001-01-03)} */
SELECT asText(minusValues(tgeompoint '[Point(0 0 0)@2001-01-01, Point(3 3 3)@2001-01-04)',
  geomset '{"Point(1 1 1)', 'Point(2 2 2)"}'));
/* {[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02),
   (POINT Z (1 1 1)@2001-01-02, POINT Z (2 2 2)@2001-01-03),
   (POINT Z (2 2 2)@2001-01-03, POINT Z (3 3 3)@2001-01-04)} */
SELECT asText(minusValue(tgeometry '[Point(0 0)@2001-01-01,
  Linestring(0 0,2 2)@2001-01-02, Linestring(0 0,2 2)@2001-01-03]',
  geometry 'Linestring(0 0,2 2)'));
--  {[POINT(0 0)@2001-01-01, POINT(0 0)@2001-01-02)}
```

- Restrict to (the complement of) a time value

```
atTime(ttype,times) → ttype
minusTime(ttype,times) → ttype
```

```
SELECT atTime(tfloat '[1@2001-01-01, 5@2001-01-05)', timestamptz '2001-01-02');
-- 2@2001-01-02
SELECT atTime(tbigint '[1@2001-01-01, 1@2001-01-15)', tstzset '{2001-01-01, 2001-01-03}');
-- {1@2001-01-01, 1@2001-01-03}
SELECT atTime(tfloat '{[1@2001-01-01, 3@2001-01-03), [3@2001-01-04, 1@2001-01-06)}',
  tstzspan '[2001-01-02,2001-01-05)');
-- {[2@2001-01-02, 3@2001-01-03), [3@2001-01-04, 2@2001-01-05)}
SELECT atTime(tint '[1@2001-01-01, 1@2001-01-15)',
  tstzspanset '{[2001-01-01, 2001-01-03), [2001-01-04, 2001-01-05)}');
-- {[1@2001-01-01, 1@2001-01-03),[1@2001-01-04, 1@2001-01-05)}
```

```
SELECT minusTime(tfloat '[1@2001-01-01, 5@2001-01-05)', timestamptz '2001-01-02');
-- {[1@2001-01-01, 2@2001-01-02), (2@2001-01-02, 5@2001-01-05)}
SELECT minusTime(tint '[1@2001-01-01, 1@2001-01-15)', tstzset '{2001-01-02, 2001-01-03}');
```

```
/* {[1@2001-01-01, 1@2001-01-02), (1@2001-01-02, 1@2001-01-03),
   (1@2001-01-03, 1@2001-01-15)} */
SELECT minusTime(tfloat '{[1@2001-01-01, 3@2001-01-03), [3@2001-01-04, 1@2001-01-06)}',
  tstzspan '[2001-01-02,2001-01-05)');
-- {[1@2001-01-01, 2@2001-01-02), [2@2001-01-05, 1@2001-01-06)}
SELECT minusTime(tint '[1@2001-01-01, 1@2001-01-15)',
  tstzspanset '{[2001-01-02, 2001-01-03), [2001-01-04, 2001-01-05)}');
/* {[1@2001-01-01, 1@2001-01-02), [1@2001-01-03, 1@2001-01-04),
   [1@2001-01-05, 1@2001-01-15)} */
```

- Keep the instants of a temporal value that are before/after or equal a timestamp

```
beforeTimestamp(ttype,timestamptz,strict boolean=true) → ttype
afterTimestamp(ttype,timestamptz,strict boolean=true) → ttype
```

```
SELECT beforeTimestamp(tint '[1@2001-01-01, 1@2001-01-03]', timestamptz '2001-01-02');
-- [1@2001-01-01, 1@2001-01-02)
SELECT beforeTimestamp(tint '[1@2001-01-01, 1@2001-01-03]', timestamptz '2001-01-01');
-- NULL
SELECT beforeTimestamp(tbigint '[1@2001-01-01, 1@2001-01-03]', timestamptz '2001-01-01',
  false);
-- [1@2001-01-01]
SELECT afterTimestamp(tfloat '[1@2001-01-01, 3@2001-01-03]', timestamptz '2001-01-02',
  false);
-- [2@2001-01-02, 3@2001-01-03]
SELECT asText(afterTimestamp(tgeompoint '{[Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02], [Point(3 3 3)@2001-01-04, Point(3 3 3)@2001-01-05]}',
  timestamptz '2001-01-03'));
-- {[POINT Z (3 3 3)@2001-01-04, POINT Z (3 3 3)@2001-01-05]}
```

5.3 Bounding Box Operators

These operators test whether the bounding boxes of their arguments satisfy the predicate and result in a Boolean value. As stated in Chapter 4, the bounding box associated to a temporal type depends on the base type: It is the `tstzspan` type for the `tbool` and `ttext` types, the `tbox` type for the `tint`, `tbigint`, and `tfloat` types, and the `stbox` type for the `tgeompoint` and `tgeogpoint` types. Furthermore, as seen in Section 3.3, many PostgreSQL, PostGIS, or MobilityDB types can be converted to the `tbox` and `stbox` types. For example, numeric and span types can be converted to type `tbox`, types `geometry` and `geography` can be converted to type `stbox`, and time types and temporal types can be converted to types `tbox` and `stbox`.

A first set of operators consider the topological relationships between the bounding boxes. There are five topological operators: overlaps (`&&`), contains (`@>`), contained (`<@`), same (`~=`), and adjacent (`-|-`). The arguments of these operators can be a base type, a box, or a temporal type and the operators verify the topological relationship taking into account the value and/or the time dimension depending on the type of the arguments.

Another set of operators consider the relative position of the bounding boxes. The operators `<<`, `>>`, `&<`, and `&>` consider the value dimension for the `tint`, `tbigint`, and `tfloat` types and the X coordinates for the `tgeompoint` and `tgeogpoint` types, the operators `<<|`, `|>>`, `&<|`, and `|&>` consider the Y coordinates for the `tgeompoint` and `tgeogpoint` types, the operators `<</`, `/>>`, `&</`, and `/&>` consider the Z coordinates for the `tgeompoint` and `tgeogpoint` types, and the operators `<<#`, `#>>`, `#&<`, and `#&>` consider the time dimension for all temporal types.

Finally, it is worth noting that the bounding box operators read the dimensions both operands have, see Section 8.3.

We refer to Section 3.9 for the bounding box operators.

5.4 Comparisons

5.4.1 Traditional Comparisons

The traditional comparison operators (=, <, and so on) require that the left and right operands be of the same base type. Excepted equality and inequality, the other comparison operators are not useful in the real world but allow B-tree indexes to be constructed on temporal types. These operators compare the bounding periods (see Section 2.10), then the bounding boxes (see Section 3.10) and if those are equal, then the comparison depends on the subtype. For instant values, they compare first the timestamps and if those are equal, compare the values. For sequence values, they compare the first N instants, where N is the minimum of the number of composing instants of both values. Finally, for sequence set values, they compare the first N sequence values, where N is the minimum of the number of composing sequences of both values.

The equality and inequality operators consider the equivalent representation for different subtypes as shown next.

```
SELECT tint '1@2001-01-01' = tint '{1@2001-01-01}';
-- true
SELECT tfloat '1.5@2001-01-01' = tfloat '[1.5@2001-01-01]';
-- true
SELECT ttext 'AAA@2001-01-01' = ttext '{[AAA@2001-01-01]}';
-- true
SELECT tgeompoint '{Point(1 1)@2001-01-01, Point(2 2)@2001-01-02}' =
  tgeompoint '{[Point(1 1)@2001-01-01], [Point(2 2)@2001-01-02]}';
-- true
SELECT tgeography '[Point(1 1)@2001-01-01, Linestring(1 1,2 2)@2001-01-02]' =
  tgeography '{[Point(1 1)@2001-01-01, Linestring(1 1,2 2)@2001-01-02]}';
-- true
```

- Traditional comparisons

  ```
  ttype {=, <>, <, >, <=, >=} ttype → boolean
  cmp(ttype,ttype) → integer
  ```

  Function `cmp` returns -1, 0, or 1 depending on whether the first argument is, respectively, less than, equal to, or greater than the second one.

  ```
  SELECT tint '[1@2001-01-01, 1@2001-01-04)' = tint '[2@2001-01-03, 2@2001-01-05)';
  -- false
  SELECT tint '[1@2001-01-01, 1@2001-01-04)' <> tint '[2@2001-01-03, 2@2001-01-05)';
  -- true
  SELECT tint '[1@2001-01-01, 1@2001-01-04)' < tint '[2@2001-01-03, 2@2001-01-05)';
  -- true
  SELECT ttfloat '[1@2001-01-01, 1@2001-01-04)' > ttfloat '[2@2001-01-03, 2@2001-01-05)';
  -- false
  SELECT tbigint '[1@2001-01-01, 1@2001-01-04)' <= tbigint '[2@2001-01-03, 2@2001-01-05)';
  -- true
  SELECT tbigint '[1@2001-01-01, 1@2001-01-04)' >= tbigint '[2@2001-01-03, 2@2001-01-05)';
  -- false
  SELECT cmp(tfloat '[1@2001-01-01, 1@2001-01-04)', '[2@2001-01-03, 2@2001-01-05)');
  -- -1
  ```

- Hash values

  ```
  hash(ttype) → integer
  hashExtended(ttype,seed bigint) → bigint
  ```

  Function `hash` returns a 32-bit hash value and function `hashExtended` a 64-bit hash value computed with the given seed. Equal values have equal hash values, which is what hash indexes, hash joins, and hash aggregation rely on; this holds across subtypes, so that an instant has the hash value of a sequence or sequence set holding only that instant.

```
SELECT hash(tint '1@2001-01-01') = hash(tint '{1@2001-01-01}');
-- true
SELECT hash(tfloat '1.5@2001-01-01') = hash(tfloat '{[1.5@2001-01-01]}');
-- true
SELECT hash(tgeompoint '{Point(1 1)@2001-01-01, Point(2 2)@2001-01-02}') =
  hash(tgeompoint '{[Point(1 1)@2001-01-01], [Point(2 2)@2001-01-02]}');
-- true
SELECT hashExtended(tint '[1@2001-01-01, 2@2001-01-02]', 0) <>
  hashExtended(tint '[1@2001-01-01, 2@2001-01-02]', 1);
-- true
```

5.4.2 Ever and Always Comparisons

A possible generalization of the traditional comparison operators (=, <>, <, <=, etc.) to temporal types consists in determining whether the comparison is ever or always true. In this case, the result is a Boolean value. MobilityDB provides operators to test whether the comparison of a temporal value and a value of the base type or two temporal values is ever or always true. These operators are denoted by prefixing the traditional comparison operators with, respectively, `?` (ever) and `%` (always). Some examples are `?=`, `%<>`, or `?<=`. Ever and always equality and non-equality are available for all temporal types, while ever and always inequalities are only available for temporal types whose base type has a total order defined, that is, `tint`, `tfloat`, or `ttext`. The ever and always comparisons are inverse operators: for example, `?=` is the inverse of `%<>`, and `?>` is the inverse of `%<=`.

- Ever and always comparisons

```
{base,ttype} {?=, ?<>, ?<, ?>, ?<=, ?>=} {base,ttype} → boolean
{base,ttype} {%=, %<>, %<, %>, %<=, %>=} {base,ttype} → boolean
```

The operators do not take into account whether the bounds are inclusive or not.

```
SELECT tint '[1@2001-01-01, 3@2001-01-04]' ?= 2;
-- false
SELECT tgeometry '[Point(0 0)@2001-01-01,
  Linestring(0 0,1 1)@2001-01-04]' ?= geometry 'Linestring(1 1,0 0)';
-- true
SELECT tfloat '[1@2001-01-01, 1@2001-01-04)' %= 1;
-- true
SELECT tgeometry '[Linestring(0 0,1 1)@2001-01-01,
  Linestring(1 1,0 0)@2001-01-04)' %= geometry 'Linestring(0 0,1 1)';
-- true
```

```
SELECT tfloat '[1@2001-01-01, 3@2001-01-04)' ?<> 2;
-- true
SELECT tgeompoint '[Point(1 1)@2001-01-01, Point(1 1)@2001-01-04)' ?<>
  geometry 'Point(1 1)';
-- false
SELECT tfloat '[1@2001-01-01, 3@2001-01-04)' %<> 2;
-- false
SELECT tgeogpoint '[Point(1 1)@2001-01-01, Point(1 1)@2001-01-04)' %<>
  geography 'Point(2 2)';
-- true
```

```
SELECT tint '[1@2001-01-01, 4@2001-01-04]' ?< 2;
-- true
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' %< 2;
-- false
```

```
SELECT tint '[1@2001-01-03, 1@2001-01-05]' ?> 0;
-- true
SELECT tfloat '[1@2001-01-03, 1@2001-01-05)' %> 1;
-- false
```

```
SELECT tint '[1@2001-01-01, 1@2001-01-05]' ?<= 2;
-- true
SELECT tfloat '[1@2001-01-01, 1@2001-01-05)' %<= 4;
-- true
```

```
SELECT ttext '{[AAA@2001-01-01, AAA@2001-01-03),
  [BBB@2001-01-04, BBB@2001-01-05)}' ?> 'AAA'::text;
-- true
SELECT ttext '{[AAA@2001-01-01, AAA@2001-01-03),
  [BBB@2001-01-04, BBB@2001-01-05)}' %> 'AAA'::text;
-- false
```

5.4.3 Temporal Comparisons

Another possible generalization of the traditional comparison operators (=, <>, <, <=, etc.) to temporal types consists in determining whether the comparison is true or false at each instant. In this case, the result is a temporal Boolean. The temporal comparison operators are denoted by prefixing the traditional comparison operators with `#`. Some examples are `#=` or `#<=`. Temporal equality and non-equality are available for all temporal types, while temporal inequalities are only available for temporal types whose base type has a total order defined, that is, `tint`, `tfloat`, or `ttext`.

- Temporal comparisons

```
{base,ttype} {#=, #<>, #<, #>, #<=, #>=} {base,ttype} → boolean
```

```
SELECT tfloat '[1@2001-01-01, 2@2001-01-04)' #= 3;
-- {[f@2001-01-01, f@2001-01-04)}
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' #= tfloat '[4@2001-01-02, 1@2001-01-05)';
-- {[f@2001-01-02, t@2001-01-03], (f@2001-01-03, f@2001-01-04)}
SELECT tgeompoint '[Point(0 0)@2001-01-01,
  Point(2 2)@2001-01-03)' #= geometry 'Point(1 1)';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tgeometry '[Point(0 0)@2001-01-01,
  Linestring(1 1,2 2)@2001-01-03]' #= geometry 'Linestring(2 2,1 1)';
-- {[f@2001-01-01, t@2001-01-03]}
```

```
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' #<> 2;
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, 2001-01-04)}
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' #<> tfloat '[2@2001-01-02, 2@2001-01-05)';
-- {[f@2001-01-02], (t@2001-01-02, t@2001-01-04)}
SELECT tgeometry '[Point(0 0)@2001-01-01, Linestring(1 1,2 2)@2001-01-03]' #<>
  geometry 'Linestring(2 2,1 1)';
-- {[t@2001-01-01, f@2001-01-03]}
```

```
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' #< 2;
-- {[t@2001-01-01, f@2001-01-02, f@2001-01-04)}
SELECT 1 #> tint '[1@2001-01-03, 1@2001-01-05)';
-- [f@2001-01-03, f@2001-01-05)
SELECT tfloat '[1@2001-01-01, 1@2001-01-05)' #<= tfloat '{2@2001-01-03, 3@2001-01-04}';
-- {t@2001-01-03, t@2001-01-04}
SELECT 'AAA'::text #> ttext '{[AAA@2001-01-01, AAA@2001-01-03),
  [BBB@2001-01-04, BBB@2001-01-05)}';
-- {[f@2001-01-01, f@2001-01-03), [t@2001-01-04, t@2001-01-05)}
```

5.5 Miscellaneous

- Version of the MobilityDB extension

```
mobilitydbVersion() → text
```

```
SELECT mobilitydbVersion();
-- MobilityDB 1.3.0
```

- Versions of the MobilityDB extension and its dependencies

```
mobilitydbFullVersion() → text
```

```
SELECT mobilitydbFullVersion();
/* MobilityDB 1.3.0, PostgreSQL 17.2, PostGIS 3.5.2, PROJ 9.2.0, JSON-C 0.13.1 */
```

Chapter 6

Temporal Alphanumeric Types

6.1 Notation

We presented in Section 4.5 the notation used for defining the signature of the functions and operators for temporal alphanumeric types. We extend next this notations for temporal alphanumeric types.

- `torder` represents any temporal type whose base type has a total order defined, that is, `tint`, `tbigint`, `tfloat`, or `ttext`,
- `talpha` represents any temporal alphanumeric type, such as, `tint` or `ttext`,
- `tnumber` represents any temporal number type, that is, `tint`, `tbigint`, or `tfloat`,
- `number` represents any number base type, that is, `integer`, `bigint`, or `float`,
- `numspan` represents any number span type, that is, either `intspan`, `bigintspan`, or `floatspan`,

6.2 Conversions

A temporal value can be converted into a compatible type using the notation `CAST(ttype1 AS ttype2)` or `ttype1::ttype2`.

- Convert a temporal number to a span or to a temporal bounding box

```
tnumber::{span,tbox}
valueSpan(tnumber) → numspan
timeSpan(tnumber) → tstzspan
tbox(tnumber) → tbox
```

```
SELECT tint '[1@2001-01-01, 2@2001-01-03]'::intspan;
-- [1, 3)
SELECT tfloat '(1@2001-01-01, 3@2001-01-03, 2@2001-01-05]'::floatspan;
-- (1, 3]
SELECT valueSpan(tfloat '(1@2001-01-01, 3@2001-01-03, 2@2001-01-05]');
-- (1, 3]
SELECT timeSpan(tfloat 'Interp=Step;(1@2001-01-01, 3@2001-01-03, 2@2001-01-05]');
-- (2001-01-01, 2001-01-05]
```

```
SELECT tint '[1@2001-01-01, 2@2001-01-03]'::tbox;
-- TBOXINT XT((1,2),[2001-01-01,2001-01-03])
SELECT tfloat '(1@2001-01-01, 3@2001-01-03, 2@2001-01-05]'::tbox;
-- TBOXFLOAT XT((1,3),[2001-01-01,2001-01-05])
```

- Convert between a temporal Boolean and a temporal integer

```
tbool::tint
tint(tbool) → tint
```

```
SELECT tbool '[true@2001-01-01, false@2001-01-03]'::tint;
-- [1@2001-01-01, 0@2001-01-03]
```

- Convert between temporal integers, temporal big integers, and temporal floats

```
tnumber::tnumber
tnumber(tnumber) → tnumber
```

```
SELECT tint '[1@2001-01-01, 2@2001-01-03]'::tfloat;
-- Interp=Step;[1@2001-01-01, 2@2001-01-03]
SELECT tbigint '[1@2001-01-01, 2@2001-01-03, 3@2001-01-05]'::tfloat;
-- Interp=Step;[1@2001-01-01, 2@2001-01-03, 3@2001-01-05]
SELECT tfloat 'Interp=Step;[1.5@2001-01-01, 2.5@2001-01-03]'::tint;
-- [1@2001-01-01, 2@2001-01-03]
SELECT tint(tfloat '[1.5@2001-01-01, 2.5@2001-01-03]');
-- ERROR:  Cannot cast temporal float with linear interpolation to temporal integer
```

6.3 Accessors

- Return the span of values, ignoring any gaps

```
valueSpan(tnumber) → numspan
```

```
SELECT valueSpan(tint '{1@2001-01-01, 5@2001-01-02, 3@2001-01-03}');
-- [1, 6)
SELECT valueSpan(tfloat '[1.5@2001-01-01, 4.5@2001-01-03]');
-- [1.5, 4.5]
```

- Return the values of a temporal number as a set

```
valueSet(tnumber) → numset
```

```
SELECT valueSet(tint '[1@2001-01-01, 2@2001-01-03]');
-- {1, 2}
SELECT valueSet(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 4@2001-01-05)}');
-- {1, 2, 3, 4}
```

- Return the minimum, maximum, or average value

```
minValue(torder) → base
maxValue(torder) → base
avgValue(tnumber) → base
```

The functions do not take into account whether the bounds are inclusive or not.

```
SELECT minValue(tfloat '{1@2001-01-01, 2@2001-01-03, 3@2001-01-05}');
-- 1
SELECT maxValue(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- 5
SELECT avgValue(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- 2.75
```

```
SELECT minValue(ttext '{[A@2001-01-01, B@2001-01-02], [C@2001-01-03, E@2001-01-05]}');
-- A
SELECT maxValue(ttext '{[A@2001-01-01, B@2001-01-02], [C@2001-01-03, E@2001-01-05]}');
-- E
```

- Return the instant with the minimum or maximum value

```
minInstant(torder) → base
maxInstant(torder) → base
```

The function does not take into account whether the bounds are inclusive or not. If several instants have the minimum value, the first one is returned.

```
SELECT minInstant(tfloat '{1@2001-01-01, 2@2001-01-03, 3@2001-01-05}');
-- 1@2001-01-01
SELECT maxInstant(tfloat '{[1@2001-01-01, 2@2001-01-03), [3@2001-01-03, 5@2001-01-05)}');
-- 5@2001-01-05
```

6.4 Transformations

- Shift and/or the value span of a temporal number by one or two numbers

```
shiftValue(tnumber,base) → tnumber
scaleValue(tnumber,base) → tnumber
shiftScaleValue(tnumber,base,base) → tnumber
```

For scaling, if the value span of the temporal value is a single value (for example, for a temporal instant), the result is the temporal value. Furthermore, the given width must be strictly greater than zero.

```
SELECT shiftValue(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}', 1);
-- {2@2001-01-02, 3@2001-01-04, 2@2001-01-06}
SELECT shiftValue(tfloat '{[1@2001-01-01,2@2001-01-02],[3@2001-01-03,4@2001-01-04]}', -1);
--  {[0@2001-01-01, 1@2001-01-02], [2@2001-01-03, 3@2001-01-04]}
SELECT scaleValue(tint '1@2001-01-01', 1);
-- 1@2001-01-01
SELECT scaleValue(tfloat '{[1@2001-01-01,2@2001-01-02], [3@2001-01-03,4@2001-01-04]}', 6);
-- {[1@2001-01-01, 3@2001-01-03], [5@2001-01-05, 7@2001-01-07]}
SELECT scaleValue(tint '1@2001-01-01', -1);
-- ERROR:  The value must be strictly positive: -1
SELECT shiftScaleValue(tint '1@2001-01-01', 1, 1);
-- 2@2001-01-01
SELECT shiftScaleValue(tfloat '{[1@2001-01-01,2@2001-01-02],[3@2001-01-03,4@2001-01-04]}',
  -1, 6);
--  {[0@2001-01-01, 2@2001-01-02], [4@2001-01-03, 6@2001-01-04]}
```

- Extract from a temporal float with linear interpolation the subsequences where the values stay within a span of a given width for at least a given duration

```
stops(tfloat,maxdist float=0.0,minduration interval='0 minutes') → tfloat
```

If `maxDist` is not given, a value 0.0 is assumed by default and thus, the function extracts the constant segments of the temporal float. A stop that begins at the first instant of a sequence includes that instant only if the sequence does, and the same holds for a stop that ends at its last instant. Stops are found in time order: each begins at the earliest instant from which the value stays within the maximum distance for the minimum duration, and ends at the last instant before it leaves.

```
SELECT stops(tfloat '[1@2001-01-01, 1@2001-01-02, 2@2001-01-03]');
-- {[1@2001-01-01, 1@2001-01-02]}
SELECT stops(tfloat '[1@2001-01-01, 1@2001-01-02, 2@2001-01-03]', 0.0, '2 days');
-- NULL
```

6.5 Restrictions

- Restrict to (the complement of) the minimum or maximum value

```
atMin(torder) → torder
atMax(torder) → torder
minusMin(torder) → torder
minusMax(torder) → torder
```

The functions returns a NULL value if the minimum value only happens at exclusive bounds.

```
SELECT atMin(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}');
-- {1@2001-01-01, 1@2001-01-05}
SELECT atMin(tint '(1@2001-01-01, 3@2001-01-03]');
-- {(1@2001-01-01, 1@2001-01-03)}
SELECT atMin(tfloat '(1@2001-01-01, 3@2001-01-03]');
-- NULL
SELECT atMin(ttext '{(AA@2001-01-01, AA@2001-01-03), (BB@2001-01-03, AA@2001-01-05]}');
-- {(AA@2001-01-01, AA@2001-01-03), [AA@2001-01-05]}
SELECT atMax(tint '{1@2001-01-01, 2@2001-01-03, 3@2001-01-05}');
-- {3@2001-01-05}
SELECT atMax(tfloat '(1@2001-01-01, 3@2001-01-03)');
-- NULL
SELECT atMax(tfloat '{(2@2001-01-01, 1@2001-01-03), [2@2001-01-03, 2@2001-01-05)}');
-- {[2@2001-01-03, 2@2001-01-05]}
SELECT atMax(ttext '{(AA@2001-01-01, AA@2001-01-03), (BB@2001-01-03, AA@2001-01-05]}');
-- {("BB"@2001-01-03, "BB"@2001-01-05)}
```

```
SELECT minusMin(tint '{1@2001-01-01, 2@2001-01-03, 1@2001-01-05}');
-- {2@2001-01-03}
SELECT minusMin(tfloat '[1@2001-01-01, 3@2001-01-03]');
-- {(1@2001-01-01, 3@2001-01-03]}
SELECT minusMin(tfloat '(1@2001-01-01, 3@2001-01-03)');
-- {(1@2001-01-01, 3@2001-01-03)}
SELECT minusMin(tint '{[1@2001-01-01, 1@2001-01-03), (1@2001-01-03, 1@2001-01-05)}');
-- NULL
SELECT minusMax(tint '{1@2001-01-01, 2@2001-01-03, 3@2001-01-05}');
-- {1@2001-01-01, 2@2001-01-03}
SELECT minusMax(tfloat '[1@2001-01-01, 3@2001-01-03]');
-- {[1@2001-01-01, 3@2001-01-03)}
SELECT minusMax(tfloat '(1@2001-01-01, 3@2001-01-03)');
-- {(1@2001-01-01, 3@2001-01-03)}
SELECT minusMax(tfloat '{[2@2001-01-01, 1@2001-01-03), [2@2001-01-03, 2@2001-01-05)}');
-- {(2@2001-01-01, 1@2001-01-03)}
SELECT minusMax(tfloat '{[1@2001-01-01, 3@2001-01-03), (3@2001-01-03, 1@2001-01-05)}');
-- {[1@2001-01-01, 3@2001-01-03), (3@2001-01-03, 1@2001-01-05)}
```

- Restrict to (the complement of) a `tbox`

```
atTbox(tnumber,tbox) → tnumber
minusTbox(tnumber,tbox) → tnumber
```

Cuando el cuadro delimitador tiene dimensiones tanto de valores como temporales, las funciones restringen el número temporal con respecto al valor y las extensiones temporales del cuadro.

```
SELECT atTbox(tfloat '[0@2001-01-01, 3@2001-01-04)',
  tbox 'TBOXFLOAT XT((0,2),[2001-01-02, 2001-01-04])');
-- {[1@2001-01-02, 2@2001-01-03]}
```

```
SELECT minusTbox(tfloat '[1@2001-01-01, 4@2001-01-04)',
  'TBOXFLOAT XT((1,4),[2001-01-03, 2001-01-04])');
-- {[1@2001-01-01, 3@2001-01-03)}
WITH temp(temp, box) AS (
  SELECT tfloat '[1@2001-01-01, 4@2001-01-04)',
    tbox 'TBOXFLOAT XT((1,2),[2001-01-03, 2001-01-04])' )
SELECT minusSpan(minusTime(temp, box::tstzspan), box::floatspan) FROM temp;
-- {[1@2001-01-01], [2@2001-01-02, 3@2001-01-03)}
```

6.6 Boolean Operations

- Temporal and, temporal or

```
{boolean,tbool} {&, |} {boolean,tbool} → tbool
```

```
SELECT tbool '[true@2001-01-03, true@2001-01-05)' &
  tbool '[false@2001-01-03, false@2001-01-05)';
-- [f@2001-01-03, f@2001-01-05)
SELECT tbool '[true@2001-01-03, true@2001-01-05)' &
  tbool '{[false@2001-01-03, false@2001-01-04), [true@2001-01-04, true@2001-01-05)}';
-- {[f@2001-01-03, t@2001-01-04, t@2001-01-05)}
SELECT tbool '[true@2001-01-03, true@2001-01-05)' | tbool '[false@2001-01-03,
  false@2001-01-05)';
-- [t@2001-01-03, t@2001-01-05)
```

- Temporal not

```
~tbool → tbool
```

```
SELECT ~tbool '[true@2001-01-03, true@2001-01-05)';
-- [f@2001-01-03, f@2001-01-05)
```

- Return the time when a temporal Boolean takes the value true

```
whenTrue(tbool) → tstzspanset
```

```
SELECT whenTrue(tfloat '[1@2001-01-01, 4@2001-01-04, 1@2001-01-07]' #> 2);
-- {(2001-01-02, 2001-01-06)}
SELECT whenTrue(tdwithin(tgeompoint '[Point(1 1)@2001-01-01, Point(4 4)@2001-01-04,
  Point(1 1)@2001-01-07]', geometry 'Point(1 1)', sqrt(2)));
-- {[2001-01-01, 2001-01-02], [2001-01-06, 2001-01-07]}
```

6.7 Mathematical Operations

- Temporal addition, subtraction, multiplication, and division

```
{number,tnumber} {+, -, *, /} {number,tnumber} → tnumber
```

The temporal division will raise an error if the denominator is ever equal to zero during the common timespan of the arguments.

The product of two temporal floats with linear interpolation is, between two consecutive instants, a polynomial of degree two in time, and the quotient by a temporal float with linear interpolation is a ratio of two linear functions, neither of which linear interpolation can state. The result then holds the value at the instants of the arguments and, within a segment, at the extremum of the product and at one further instant of a quotient of two temporal floats, and it is linear between the instants it holds. Every value the result holds is exact, and so are its minimum and its maximum, since a quotient varies monotonically between two instants, but between the instants it holds the result is the chord of the product or of the quotient. In the examples below, the product is 3.75 at noon on January 1, where the result interpolates 3.5, and 2 divided by the temporal float is 1 at noon on January 2, where the result interpolates 1.333. The product of a temporal float by a number or by a temporal float with step interpolation, and its quotient by a number or by a temporal float with step interpolation, are linear between two instants and are exact.

```
SELECT tint '[2@2001-01-01, 2@2001-01-04)' + 1;
-- [3@2001-01-01, 3@2001-01-04)
SELECT tfloat '[2@2001-01-01, 2@2001-01-04)' + tfloat '[1@2001-01-01, 4@2001-01-04)';
-- [3@2001-01-01, 6@2001-01-04)
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' + tfloat '{[1@2001-01-01, 2@2001-01-02),
  [1@2001-01-02, 2@2001-01-04)}';
-- {[2@2001-01-01, 4@2001-01-04), [3@2001-01-02, 6@2001-01-04)}
```

```
SELECT tint '[1@2001-01-01, 1@2001-01-04)' - tint '[2@2001-01-03, 2@2001-01-05)';
-- [-1@2001-01-03, -1@2001-01-04)
SELECT tfloat '[3@2001-01-01, 6@2001-01-04)' - tfloat '[2@2001-01-01, 2@2001-01-04)';
-- [1@2001-01-01, 4@2001-01-04)
```

```
SELECT tint '[1@2001-01-01, 4@2001-01-04]' * 2;
-- [2@2001-01-01, 8@2001-01-04]
SELECT tfloat '[1@2001-01-01, 4@2001-01-04)' * tfloat '[2@2001-01-01, 2@2001-01-04)';
-- [2@2001-01-01, 8@2001-01-04)
SELECT tfloat '[1@2001-01-01, 3@2001-01-03)' * '[3@2001-01-01, 1@2001-01-03)';
-- [3@2001-01-01, 4@2001-01-02, 3@2001-01-03)
SELECT valueAtTimestamp(tfloat '[1@2001-01-01, 3@2001-01-03)' *
  '[3@2001-01-01, 1@2001-01-03)', '2001-01-01 12:00');
-- 3.5
```

```
SELECT 2 / tfloat '[1@2001-01-01, 3@2001-01-04)';
-- [2@2001-01-01, 0.666666666666667@2001-01-04)
SELECT valueAtTimestamp(2 / tfloat '[1@2001-01-01, 3@2001-01-04)', '2001-01-02 12:00');
-- 1.3333333333333333
SELECT tfloat '[1@2001-01-01, 5@2001-01-05)' / tfloat '[5@2001-01-01, 1@2001-01-05)';
-- [0.2@2001-01-01, 1@2001-01-03, 5@2001-01-05)
SELECT 2 / tfloat '[-1@2001-01-01, 1@2001-01-02]';
-- ERROR:  Division by zero
SELECT tfloat '[-1@2001-01-04, 1@2001-01-05]' / tfloat '[-1@2001-01-01, 1@2001-01-05]';
-- [-2@2001-01-04, 1@2001-01-05]
```

- Return the absolute value of the temporal number

```
abs(tnumber) → tnumber
```

```
SELECT abs(tfloat '[1@2001-01-01, -1@2001-01-03, 1@2001-01-05]');
-- [1@2001-01-01, 0@2001-01-02, 1@2001-01-03, 0@2001-01-04, 1@2001-01-05]
SELECT abs(tint '[1@2001-01-01, -1@2001-01-03, 1@2001-01-05]');
-- [1@2001-01-01, 1@2001-01-05]
```

- Round up or down to the neareast integer

```
floor(tfloat) → tfloat
ceil(tfloat) → tfloat
```

```
SELECT floor(tfloat '[0.5@2001-01-01, 1.5@2001-01-02]');
-- [0@2001-01-01, 1@2001-01-02]
SELECT ceil(tfloat '[0.5@2001-01-01, 0.6@2001-01-02, 0.7@2001-01-03]');
-- [1@2001-01-01, 1@2001-01-03]
```

- Round to a number of decimal places

```
round(tfloat,integer=0) → tfloat
```

```
SELECT round(tfloat '[0.785398163397448@2001-01-01, 2.356194490192345@2001-01-02]', 2);
-- [0.79@2001-01-01, 2.36@2001-01-02]
```

- Convert to degrees or radians

```
degrees({float,tfloat},normalize boolean=false) → tfloat
radians(tfloat) → tfloat
```

The additional parameter in the `degrees` function can be used to normalize the values between 0 and 360 degrees.

```
SELECT degrees(pi() * 5);
-- 900
SELECT degrees(pi() * 5, true);
-- 180
SELECT round(degrees(tfloat '[0.785398163397448@2001-01-01,
  2.35619449019234 5@2001-01-02]'));
-- [45@2001-01-01, 135@2001-01-02]
SELECT radians(tfloat '[45@2001-01-01, 135@2001-01-02]');
-- [0.785398163397448@2001-01-01, 2.35619449019234 5@2001-01-02]
```

- Return the value difference between consecutive instants of the temporal number

```
deltaValue(tnumber) → tnumber
```

```
SELECT deltaValue(tint '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03]');
-- [1@2001-01-01, -1@2001-01-02, -1@2001-01-03)
SELECT deltaValue(tfloat '{[1.5@2001-01-01, 2@2001-01-02, 1@2001-01-03],
  [2@2001-01-04, 2@2001-01-05]}');
/* Interp=Step;{[0.5@2001-01-01, -1@2001-01-02, -1@2001-01-03),
   [0@2001-01-04, 0@2001-01-05)} */
```

- Return the trend of a temporal float with linear interpolation, which states whether its value is increasing, constant, or decreasing, represented, respectively, by 1, 0, and -1

```
trend(tfloat) → tint
```

Note that the trend is NULL for instantaneous sequences.

```
SELECT trend(tfloat '[1@2001-01-01, 2@2001-01-02, 4@2001-01-03, 4@2001-01-04,
  3@2001-01-05, 2@2001-01-06]');
-- [1@2001-01-01, 0@2001-01-03, -1@2001-01-04, -1@2001-01-06]
SELECT trend(tfloat '[1@2001-01-01]');
-- NULL
```

- Return the derivative over time of a temporal float in units per second

```
derivative(tfloat) → tfloat
```

The temporal float must have linear interpolation. Note that this function corresponds to the speed function for temporal points.

```
SELECT derivative(tfloat '{[0@2001-01-01, 10@2001-01-02, 5@2001-01-03],
  [1@2001-01-04, 0@2001-01-05]}') * 3600 * 24;
/* Interp=Step;{[-10@2001-01-01, 5@2001-01-02, 5@2001-01-03],
   [1@2001-01-04, 1@2001-01-05]} */
SELECT derivative(tfloat 'Interp=Step;[0@2001-01-01, 10@2001-01-02, 5@2001-01-03]');
-- ERROR:  The temporal value must have linear interpolation
```

- Return the area under the curve

```
integral(tnumber) → float
```

```
SELECT integral(tint '[1@2001-01-01,2@2001-01-02]') / (24 * 3600 * 1e6);
-- 1
SELECT integral(tfloat '[1@2001-01-01,2@2001-01-02]') / (24 * 3600 * 1e6);
-- 1.5
```

- Return the time-weighted average

```
twAvg(tnumber) → float
```

```
SELECT twAvg(tfloat '{[1@2001-01-01, 2@2001-01-03), [2@2001-01-04, 2@2001-01-06)}');
-- 1.75
```

- Return the natural logarithm and the base 10 logarithm of a temporal float

```
ln(tfloat) → tfloat
log10(tfloat) → tfloat
```

  The temporal float cannot be zero or negative. With linear interpolation, the result has, in each segment, the instants of the segment and the instant where the linear interpolation departs most from the logarithm, and is interpolated linearly between them

```
SELECT ln(tfloat '{[1@2001-01-01, 10@2001-01-02, 5@2001-01-03],
  [1@2001-01-04, 1@2001-01-05]}');
/* {[0@2001-01-01, 1.363192132076512@2001-01-01 07:45:23.043236,
   2.302585092994046@2001-01-02, 1.975950833017318@2001-01-02 13:22:31.148467,
   1.6094379124341@2001-01-03], [0@2001-01-04, 0@2001-01-05]} */
SELECT log10(tfloat 'Interp=Step;[-10@2001-01-01, 10@2001-01-02]');
-- ERROR:  Cannot take logarithm of zero or a negative number
```

- Return the exponential (e raised to the given power) of a temporal float

```
exp(tfloat) → tfloat
```

  With linear interpolation, the result has, in each segment, the instants of the segment and the instant where the linear interpolation departs most from the exponential, and is interpolated linearly between them

```
SELECT exp(tfloat '{[1@2001-01-01, 10@2001-01-02], [1@2001-01-04, 1@2001-01-05]}');
/* {[2.718281828459045@2001-01-01, 2447.0830569101154@2001-01-01 18:08:25.45925,
   22026.465794806718@2001-01-02],
   [2.718281828459045@2001-01-04, 2.718281828459045@2001-01-05]} */
SELECT exp(tfloat '{-10@2001-01-01, 0@2001-01-02, 10@2001-01-03}');
-- {0.000045399929762@2001-01-01, 1@2001-01-02, 22026.465794806718@2001-01-03}
```

- Return the sine, cosine, or tangent of a temporal float (argument in radians)

```
sin(tfloat) → tfloat
cos(tfloat) → tfloat
tan(tfloat) → tfloat
```

  With linear interpolation, the result of `sin` and `cos` has, in each segment, the instants of the segment and those of the extrema of the function, the result of `tan` is refined by bisection where the function is steep, and the result is interpolated linearly between its instants. The temporal float of `tan` cannot reach $\pi/2$ plus a multiple of π, at an instant or along a segment with linear interpolation

```
SELECT round(sin(tfloat '[0@2001-01-01, 6.283185307@2001-01-02]'), 6);
-- [0@2001-01-01, 1@2001-01-01 06:00:00.000001, -1@2001-01-01 18:00:00.000002, 0@2001 ↩
   -01-02]
SELECT round(cos(tfloat '[0@2001-01-01, 3.141592654@2001-01-02]'), 6);
-- [1@2001-01-01, -1@2001-01-02]
SELECT round(tan(tfloat '{0@2001-01-01, 0.785398163@2001-01-02}'), 6);
-- {0@2001-01-01, 1@2001-01-02}
SELECT tan(tfloat '[0@2001-01-01, 3@2001-01-02]');
-- ERROR:  Cannot take the tangent of pi/2 plus a multiple of pi
```

6.8 Text Operations

- Text concatenation

```
{text,ttext} || {text,ttext} → ttext
```

```
SELECT ttext '[AA@2001-01-01, AA@2001-01-04)' || text 'B';
-- ["AAB"@2001-01-01, "AAB"@2001-01-04)
SELECT ttext '[AA@2001-01-01, AA@2001-01-04)' || ttext '[BB@2001-01-02, BB@2001-01-05)';
-- ["AABB"@2001-01-02, "AABB"@2001-01-04)
SELECT ttext '[A@2001-01-01, B@2001-01-03,
  C@2001-01-04]' || ttext '{[D@2001-01-01, D@2001-01-02), [E@2001-01-02, E@2001-01-04)}';
-- {["AD"@2001-01-01, "AE"@2001-01-02, "BE"@2001-01-03, "BE"@2001-01-04)}
```

- Transform in lowercase, uppercase, or initcap

```
upper(ttext) → ttext
lower(ttext) → ttext
initcap(ttext) → ttext
```

```
SELECT lower(ttext '[AA@2001-01-01, bb@2001-01-02]');
-- ["aa"@2001-01-01, "bb"@2001-01-02]
SELECT upper(ttext '[AA@2001-01-01, bb@2001-01-02]');
-- ["AA"@2001-01-01, "BB"@2001-01-02]
SELECT initcap(ttext '[AA@2001-01-01, bb@2001-01-02]');
-- ["Aa"@2001-01-01, "Bb"@2001-01-02]
```

Chapter 7

Temporal Geometry Types (Part 1)

7.1 Spatiotemporal Types

MobilityDB provides a set of spatiotemporal types, namely, `tgeometry` (temporal geometry), `tgeography` (temporal geography), `tgeompoint` (temporal geometry point), `tgeogpoint` (temporal geography point), `tcbuffer` (temporal circular buffer), `tnpoint` (temporal network point), `tpose` (temporal pose), `tposechain` (temporal pose chain), `trgeometry` (temporal rigid geometry), `th3index`, `tquadbin`, and `ts2cell` (the temporal DGGS cell indexes), and `tpcpoint` (temporal point cloud point) and `tpcpatch` (temporal point cloud patch). At a conceptual level, these spatiotemporal types are organized in the hierarchy depicted in Figure 7.1.

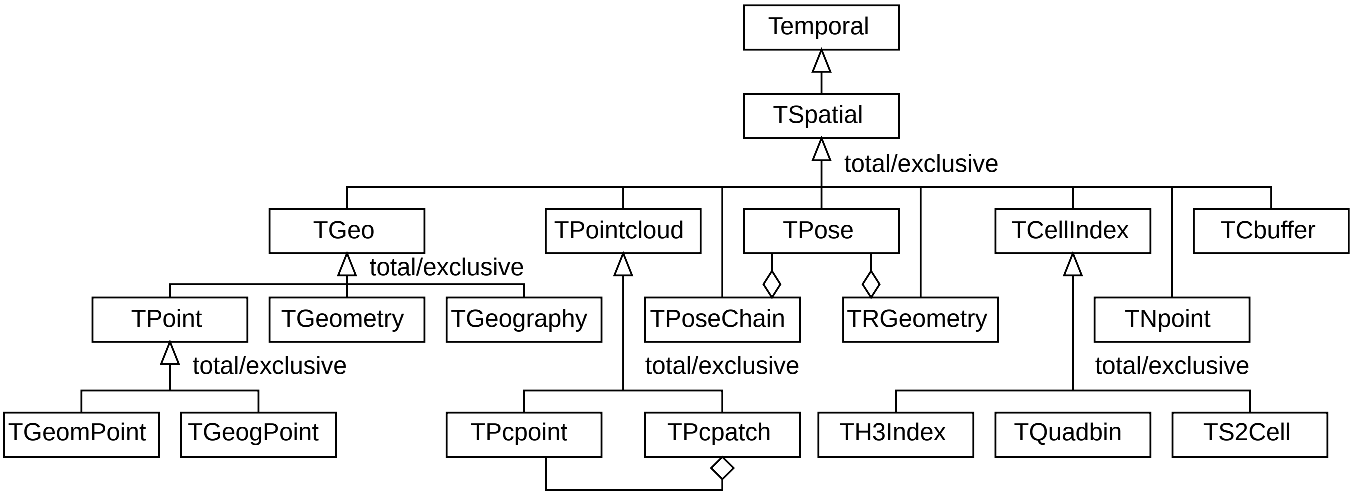
Temporal
TSpatial
total/exclusive
TGeo
TPointcloud
TPose
TCellIndex
TCbuffer
total/exclusive
TPoint
TGeometry
TGeography
TPoseChain
TRGeometry
TNpoint
total/exclusive
total/exclusive
total/exclusive
TGeomPoint
TGeogPoint
TPcpoint
TPcpatch
TH3Index
TQuadbin
TS2Cell

Figure 7.1: Hierarchy of spatiotemporal types in MobilityDB.

In this chapter and the following one we cover the type `TGeo` and its subtypes, that is, the spatiotemporal types derived from the PostGIS types `geometry` and `geography`. In the subsequent chapters we continue describing the remaining spatiotemporal types.

Figure 7.2 illustrates the use of the `tgeompoint` (top) and the `tgeometry` (bottom) types for modelling the trajectory and the wind swath of tropical storms in 2024. The data is obtained from the National Oceanic and Atmospherique Administration (NOAA). As illustrated in the figure, the type `tgeompoint` allows *linear* interpolation, while the type `tgeometry` only allows *step* interpolation.

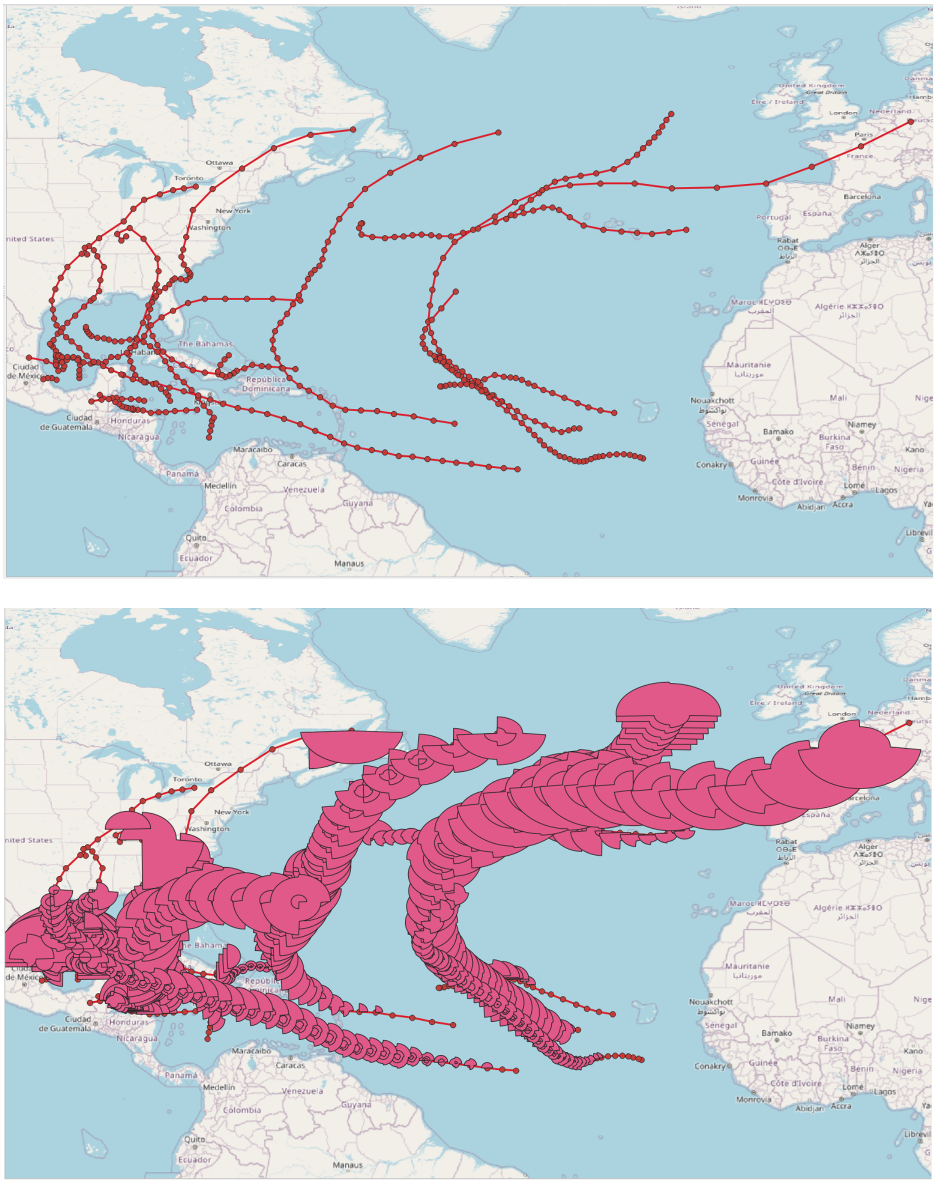
Ottawa
Toronto
New York
Washington
nited States
Ciudad de México
Ciudad de Guatemala
Honduras
Nicaragua
Panamá
The Bahamas
Habana
República Dominicana
Maracaibo
Caracas
Medellín
Venezuela
Colombia
Guyana
Quito
Ecuador
Manaus
United Kingdom
Éire / Ireland
London
Nederland
Paris
France
Barcelona
Portugal
España
Rabat
Alger
Maroc
Algérie
Mauritanie
Nouakchott
Sénégal
Mali
Bamako
Niamey
Burkina Faso
Guinée
Conakry
Côte d'Ivoire
Monrovia
Abidjan
Accra
Lomé
Lagos
Bénin
Nigeria
Kano
Libreville

Figure 7.2: Illustration of the use of the `tgeompoint` (top) and the `tgeometry` (bottom) types for modelling the trajectory and the wind swath of tropical storms in 2024.

7.2 Functions on Geometries

The functions in this section answer a question about a geometry rather than about a temporal type. Each states how it reads a geometry bounded by circular arcs, since an answer built from the arcs themselves and an answer built from a polygonal approximation of them are not the same answer.

- Return the rectangle of minimum area enclosing a geometry

  ```
  orientedEnvelope(geometry) → geometry
  ```

  The rectangle is at the angle that makes it smallest, which is not in general the angle of the axes. It is answered as a line when the geometry is contained in one and as a point when the geometry is one. A geometry carrying a circular arc is answered on the arc itself, the rectangle being placed against the circle the arc lies on rather than against the chain of segments approximating it.

  ```
  SELECT ST_AsText(round(orientedEnvelope(geometry 'Polygon((0 0,3 4,-1 7,-4 3,0 0))'), 6));
  -- POLYGON((0 0,3 4,-1 7,-4 3,0 0))
  SELECT ST_AsText(round(orientedEnvelope(geometry 'Linestring(0 0,10 0)'), 6));
  -- LINESTRING(0 0,10 0)
  SELECT ST_AsText(round(orientedEnvelope(geometry 'Circularstring(0 0,0.5 0.1,1 0)'), 6));
  -- POLYGON((1 0.1,0 0.1,0 0,1 0,1 0.1))
  ```

- Return the smallest convex geometry enclosing a geometry

  ```
  convexHull(geometry) → geometry
  ```

  The result is a polygon whose every corner is a point of the geometry, a line when the geometry is contained in one, and a point when the geometry is one. A geometry carrying a circular arc is answered on the arc itself, so the hull is bounded by the arc and is returned as a curved geometry rather than as a polygon approximating it.

  ```
  SELECT ST_AsText(round(convexHull(geometry 'Multipoint(0 0,10 0,10 10,0 10,5 5)'), 6));
  -- POLYGON((0 0,0 10,10 10,10 0,0 0))
  SELECT ST_AsText(round(convexHull(geometry 'Polygon((0 0,10 0,10 10,5 5,0 10,0 0))'), 6));
  -- POLYGON((0 0,0 10,10 10,10 0,0 0))
  SELECT ST_AsText(round(convexHull(geometry 'Circularstring(5 0,0 5,-5 0)'), 6));
  -- CURVEPOLYGON(COMPOUNDCURVE((5 0,-5 0),CIRCULARSTRING(-5 0,0 5,5 0)))
  ```

- Return true if a geometry has no point at which it crosses or touches itself

  ```
  isSimple(geometry) → boolean
  ```

  A point is always simple, a multipoint is simple when it repeats no point, a line is simple when it meets itself only where two of its segments follow one another and, when it closes, at the point where it closes, and an areal geometry is simple when each of its rings is. Two lines of a multiline may additionally meet at a point that ends both. A geometry carrying a circular arc meets itself along an arc rather than at a point, which the segment intersection this rests on does not read, so such a geometry is answered on the chain of segments approximating its arcs.

  ```
  SELECT isSimple(geometry 'Linestring(0 0,10 10,10 0,0 10)');
  -- false
  SELECT isSimple(geometry 'Linestring(0 0,10 0,10 10,0 0)');
  -- true
  SELECT isSimple(geometry 'Multilinestring((0 0,10 0),(10 0,10 10))');
  -- true
  ```

- Return the geometry holding every point within a distance of a geometry

  ```
  buffer(geometry,float,options text='') → geometry
  ```

The boundary of the result is made of the two offsets of the geometry at the given distance, of the join filling the outer side of each turn, and of the cap closing each end of an open line. A negative distance shrinks an areal geometry and answers an empty geometry for one of any other dimension. The `options` string carries the styles as space-separated `key=value` pairs: `endcap` is one of `round`, `flat` or `square`, `join` one of `round`, `mitre` or `bevel`, and `mitre_limit` the ratio at which a mitre join falls back to a bevel.

```
SELECT ST_AsText(round(buffer(geometry 'Point(0 0)', 1), 6));
-- CURVEPOLYGON(CIRCULARSTRING(-1 0,1 0,-1 0))
SELECT ST_AsText(round(buffer(geometry 'Polygon((0 0,10 0,10 10,0 10,0 0))', -1,
  'join=mitre'), 6));
-- CURVEPOLYGON(COMPOUNDCURVE((1 1,9 1),(9 1,9 9),(9 9,1 9),(1 9,1 1)))
SELECT ST_AsText(round(buffer(geometry 'Curvepolygon(Circularstring(0 0,2 2,4 0,2 -2,
  0 0))', 1), 6));
-- CURVEPOLYGON(COMPOUNDCURVE(CIRCULARSTRING(-1 0,2 3,5 0),CIRCULARSTRING(5 0,2 -3,-1 0)))
```

- Return the DE-9IM intersection matrix of two geometries or whether it matches a pattern

```
relate(geometry,geometry) → text
relate(geometry,geometry,text) → boolean
```

The matrix states, for each pair drawn from the interior, the boundary and the exterior of the first geometry and of the second, the dimension of what they share: `F` where they share nothing, `0`, `1` or `2` where they share points, a curve or an area. A circular arc is read on its own circle rather than on a chain of segments approximating it, and the interior and the boundary of a geometry collection are those of the union of its members. With a pattern, the function answers whether the matrix matches it, as `ST_Relate` with three arguments does: a character `T` of the pattern matches any of `0`, `1` and `2`, and a character `*` matches anything.

```
SELECT relate(geometry 'Polygon((0 0,4 0,4 4,0 4,0 0))', geometry 'Linestring(2 2,6 2)');
-- 1020F1102
SELECT relate(geometry 'Polygon((0 0,2 0,2 2,0 2,0 0))',
  geometry 'Polygon((2 0,4 0,4 2,2 2,2 0))');
-- FF2F11212
SELECT relate(geometry 'Curvepolygon(Circularstring(0 0,2 2,4 0,2 -2,0 0))',
  geometry 'Point(2 1.9)');
-- 0F2FF1FF2
SELECT relate(geometry 'Polygon((0 0,4 0,4 4,0 4,0 0))', geometry 'Point(2 2)',
  'T*****FF*');
-- true
```

- Return true if two geometries satisfy a spatial relationship

```
contains(geometry,geometry) → boolean
covers(geometry,geometry) → boolean
touches(geometry,geometry) → boolean
geoEquals(geometry,geometry) → boolean
disjoint({geometry,geography},{geometry,geography}) → boolean
intersects({geometry,geography},{geometry,geography}) → boolean
```

Each answers as the PostGIS function of the same name with the prefix `ST_` on the same arguments, without calling GEOS. Over two geometries with Z, `disjoint` and `intersects` are tested in 3D, as `ST_3DIntersects`, and in 2D when one of them has no Z. The containment relationships `contains`, `covers` and `touches` have no 3D form and refuse a geometry with Z. Two geographies intersect on the WGS 84 spheroid, as `ST_Intersects` decides it. The equality of two sets of points is named `geoEquals` in every engine, since PostGIS declares `equals(geometry, geometry)` as a deprecated alias of `ST_Equals`; it reads the points, where the operator = compares the vertices.

```
SELECT contains(geometry 'Polygon((0 0,4 0,4 4,0 4,0 0))', geometry 'Point(2 2)');
-- true
SELECT touches(geometry 'Polygon((0 0,4 0,4 4,0 4,0 0))', geometry 'Point(4 2)');
-- true
SELECT intersects(geometry 'Point(1 1 1)', geometry 'Point(1 1 2)');
-- false
```

```
SELECT geoEquals(geometry 'Linestring(0 0,2 0)', geometry 'Linestring(0 0,1 0,2 0)');
-- true
```

- Return true if two geometries are within a distance

  ```
  dwithin(geometry,geometry,float) → boolean
  dwithin(geography,geography,float,spheroid boolean=true) → boolean
  ```

  Two geometries with Z are measured in 3D, as `ST_3DDWithin`, and in 2D when one of them has no Z, see Section 8.3. For geographies, the computation is done on the WGS 84 spheroid by default. If the last argument is false, a faster spherical calculation is used.

  ```
  SELECT dwithin(geometry 'Point(0 0 0)', geometry 'Point(0 0 2)', 1.5);
  -- false
  SELECT dwithin(geometry 'Point(0 0 0)', geometry 'Point(0 0)', 1.5);
  -- true
  SELECT dwithin(geography 'Point(0 0)', geography 'Point(0 1)', 110800);
  -- true
  SELECT dwithin(geography 'Point(0 0)', geography 'Point(0 1)', 110800, false);
  -- false
  ```

- Return the area, the perimeter or the centroid of a geometry or a geography

  ```
  area(geometry) → float
  area(geography,spheroid boolean=true) → float
  perimeter(geometry) → float
  perimeter(geography,spheroid boolean=true) → float
  centroid(geometry) → geometry
  centroid(geography,spheroid boolean=true) → geography
  ```

  Each answers as the PostGIS function of the same name with the prefix `ST_`, without calling GEOS, except that the area of a surface bounded by circular arcs is measured on its arcs, by the closed form of the circular segment each arc adds to its chord: for the disc of radius 1 below the area is π, which `ST_Area`, replacing each quarter circle by 32 segments, reads as 3.1403. A geography is measured in square meters or in meters on the WGS 84 spheroid by default, and on the sphere, a faster calculation, if the last argument is false. On the sphere, the area is the one the great circles joining the vertices bound, which `ST_Area` over a geography on the sphere approximates: for the square of one degree at the equator below, the approximation is 0.67 percent smaller.

  ```
  SELECT area(geometry 'Polygon((0 0,4 0,4 4,0 4,0 0))');
  -- 16
  SELECT area(geometry 'CurvePolygon(CircularString(0 0,1 1,2 0,1 -1,0 0))');
  -- 3.141592653589793
  SELECT ST_AsText(centroid(geometry 'Polygon((0 0,4 0,4 4,0 4,0 0))'));
  -- POINT(2 2)
  SELECT round(area(geography 'Polygon((0 0,1 0,1 1,0 1,0 0))')::numeric);
  -- 12308778361
  SELECT round(area(geography 'Polygon((0 0,1 0,1 1,0 1,0 0))', false)::numeric);
  -- 12364031799
  ```

- Return the distance, the line joining the closest points, or the maximum distance of two geometries or geographies

  ```
  distance(geometry,geometry) → float
  distance(geography,geography,spheroid boolean=true) → float
  shortestLine(geometry,geometry) → geometry
  shortestLine(geography,geography,spheroid boolean=true) → geography
  maxDistance(geometry,geometry) → float
  ```

  Two geometries with Z are measured in 3D, as the PostGIS functions with the prefix `ST_3D`, and a geometry with Z and one without are refused. For geographies, the computation is done on the WGS 84 spheroid by default. If the last argument is false, a faster spherical calculation is used. The shortest line of two geographies joins their closest points on the sphere, as `ST_ShortestLine` over geographies joins them.

```
SELECT distance(geometry 'Point(0 0 0)', geometry 'Point(3 4 12)');
-- 13
SELECT ST_AsText(shortestLine(geometry 'Point(0 0 0)',
  geometry 'Linestring(1 -1 1,1 1 1)'));
-- LINESTRING Z (0 0 0,1 0 1)
SELECT round(maxDistance(geometry 'Linestring(0 0 0,1 0 0)',
  geometry 'Point(0 0 2)')::numeric, 6);
-- 2.236068
SELECT round(distance(geography 'Point(0 0)', geography 'Point(0 1)', false)::numeric, 3);
-- 111195.080
```

- Return the boundary, the reversal or the parts of a geometry

```
boundary(geometry) → geometry
reverse(geometry) → geometry
numGeometries(geometry) → integer
geometryN(geometry,integer) → geometry
numPoints(geometry) → integer
```

Each answers as the PostGIS function of the same name with the prefix `ST_`, `numPoints` counting every vertex as `ST_NPoints` does and `geometryN` counting from 1. The boundary of an empty geometry is the empty geometry of the dimension of a boundary. The boundary of a compound curve is its two end points, or empty when it closes, and the boundary of a multisurface is the boundaries of its members; `ST_Boundary` refuses both.

```
SELECT ST_AsText(boundary(geometry 'Polygon((0 0,4 0,4 4,0 4,0 0))'));
-- LINESTRING(0 0,4 0,4 4,0 4,0 0)
SELECT ST_AsText(reverse(geometry 'Linestring(0 0,4 0,4 3)'));
-- LINESTRING(4 3,4 0,0 0)
SELECT numGeometries(geometry 'MultiPoint(1 1,5 5)');
-- 2
SELECT ST_AsText(geometryN(geometry 'MultiPoint(1 1,5 5)', 2));
-- POINT(5 5)
SELECT numPoints(geometry 'Polygon((0 0,4 0,4 4,0 4,0 0))');
-- 5
```

- Return the point at a fraction of a line, the part between two fractions, or the fraction closest to a point

```
lineInterpolatePoint(geometry,float) → geometry
lineSubstring(geometry,float,float) → geometry
lineLocatePoint(geometry,geometry) → float
```

Each answers as the PostGIS function of the same name with the prefix `ST_`, a fraction being a share of the length of the line between 0 and 1.

```
SELECT ST_AsText(lineInterpolatePoint(geometry 'Linestring(0 0,4 0,4 3)', 0.25));
-- POINT(1.75 0)
SELECT ST_AsText(lineSubstring(geometry 'Linestring(0 0,4 0,4 3)', 0.25, 0.8));
-- LINESTRING(1.75 0,4 0,4 1.6)
SELECT lineLocatePoint(geometry 'Linestring(0 0,4 0,4 3)', geometry 'Point(4 4)');
-- 1
```

- Return the collection of an array of geometries or the line joining them

```
collect(geometry[]) → geometry
makeLine(geometry[]) → geometry
```

A collection takes its type from the types of the geometries alone, as the PostGIS function `ST_Collect` gives it, so an array of one point answers a multipoint, and a line joins the points and lines of the array in their order, as `ST_MakeLine` joins them. An empty array answers `NULL`.

```
SELECT ST_AsText(collect(ARRAY[geometry 'Point(1 1)']));
-- MULTIPOINT((1 1))
SELECT ST_AsText(collect(ARRAY[geometry 'Point(1 1)', geometry 'Linestring(0 0,1 1)']));
-- GEOMETRYCOLLECTION(POINT(1 1),LINESTRING(0 0,1 1))
SELECT ST_AsText(makeLine(ARRAY[geometry 'Point(1 1)', geometry 'Point(2 2)']));
-- LINESTRING(1 1,2 2)
```

- Return the Well-Known Text, Extended Well-Known Text, Extended Well-Known Binary or GeoJSON representation of a geometry or a geography

```
asText({geometry,geography},maxdecimaldigits integer=15) → text
asEWKT({geometry,geography},maxdecimaldigits integer=15) → text
asEWKB({geometry,geography},endian text='') → bytea
asGeoJSON(geometry,maxdecimaldigits integer=9,options integer=8) → text
asGeoJSON(geography,maxdecimaldigits integer=9,options integer=0) → text
```

Each answers as the PostGIS function of the same name with the prefix `ST_`. The GeoJSON states the coordinate reference system its options ask for, the sum of 1 for the bounding box, 2 for the short name of the system, as in `EPSG:3857`, 4 for its long name, and 8 for the short name of any system but WGS 84, which is the default for a geometry; a geography takes no option by default.

```
SELECT asText(geometry 'SRID=3857;Point(1.123456789 2)', 3);
-- POINT(1.123 2)
SELECT asEWKT(geometry 'SRID=3857;Point(1 2)');
-- SRID=3857;POINT(1 2)
SELECT asEWKB(geometry 'Point(1 1)', 'XDR');
-- \x00000000013ff00000000000003ff0000000000000
SELECT asGeoJSON(geometry 'SRID=3857;Point(1 2)');
-- {"type":"Point","crs":{"type":"name","properties":{"name":"EPSG:3857"}},"coordinates ↩
   ":[1,2]}
SELECT asGeoJSON(geometry 'SRID=4326;Point(1 2)');
-- {"type":"Point","coordinates":[1,2]}
```

- Transform a geometry or a geography to a spatial reference identifier

```
transform(geometry,integer) → geometry
transform(geography,integer) → geography
```

A geometry is transformed as the PostGIS function `ST_Transform` transforms it. A geography is transformed only into a coordinate system of longitude and latitude, as the cast of a geometry into a geography requires.

```
SELECT asEWKT(transform(geometry 'SRID=4326;Point(4.35 50.85)', 3857), 2);
-- SRID=3857;POINT(484239.78 6594803.23)
SELECT asEWKT(transform(geography 'Point(4.35 50.85)', 3857));
-- ERROR:  Only lon/lat coordinate systems are supported
```

- Densify a geometry or a geography so that no segment is longer than a distance

```
segmentize(geometry,float) → geometry
segmentize(geography,float) → geography
```

A geometry is densified in the plane, the distance in the units of its spatial reference system, and a geography on the sphere, the distance in meters, as the PostGIS function `ST_Segmentize` densifies them. A point, a multipoint and an empty value are returned unchanged, and so are a triangle, a TIN and a polyhedral surface. A distance that is not positive is refused.

```
SELECT ST_AsText(segmentize(geometry 'Linestring(0 0,3 0)', 1));
-- LINESTRING(0 0,1 0,2 0,3 0)
SELECT asText(segmentize(geography 'Linestring(4.35 50.85,2.35 48.85)', 100000), 4);
-- LINESTRING(4.35 50.85,3.8343 50.3533,3.3293 49.8543,2.8347 49.3532,2.35 48.85)
```

- Return the clusters of an array of geometries

```
clusterKMeans(geometry[],k integer) → integer[]
clusterDBSCAN(geometry[],eps float,minpoints integer) → integer[]
clusterIntersecting(geometry[]) → geometry[]
clusterWithin(geometry[],distance float) → geometry[]
```

  `clusterKMeans` and `clusterDBSCAN` return the number of the cluster of each geometry, in the order of the array, as the PostGIS window functions `ST_ClusterKMeans` and `ST_ClusterDBSCAN` number them over the rows in that order: an empty geometry is in the cluster -1 of k-means, and a geometry DBSCAN places in no cluster has the number `NULL`. `clusterIntersecting` and `clusterWithin` return a geometry collection per cluster, as the PostGIS aggregates `ST_ClusterIntersecting` and `ST_ClusterWithin` return them. An empty array answers `NULL`.

```
SELECT clusterKMeans(ARRAY[geometry 'Point(0 0)', geometry 'Point(1 0)', geometry 'Point ↩
    (10 10)',
  geometry 'Point(11 10)', geometry 'Point(30 30)'], 2);
-- {1,1,1,1,0}
SELECT clusterDBSCAN(ARRAY[geometry 'Point(0 0)', geometry 'Point(1 0)', geometry 'Point ↩
    (10 10)',
  geometry 'Point(11 10)', geometry 'Point(30 30)'], 1.5, 2);
-- {0,0,1,1,NULL}
SELECT array_length(clusterWithin(ARRAY[geometry 'Point(0 0)', geometry 'Point(1 0)',  ↩
    geometry 'Point(10 10)',
  geometry 'Point(11 10)', geometry 'Point(30 30)'], 2), 1);
-- 3
```

- Return the region two geometries share

```
intersection(geometry,geometry) → geometry
```

  The four Boolean operations of this section answer over the (multi)polygons the temporal types whose values are regions carry, and each takes the name of the PostGIS function it answers for without the `ST_` prefix, so a query reaches this engine by dropping the prefix and PostGIS by keeping it. They are planar and two-dimensional: a geography, a Z dimension and a geometry that is not a (multi)polygon are refused. An empty result is answered as `NULL`.

```
SELECT ST_AsText(intersection(geometry 'Polygon((1 1,1 5,5 5,5 1,1 1))',
  geometry 'Polygon((3 3,3 7,7 7,7 3,3 3))'));
-- POLYGON((5 3,3 3,3 5,5 5,5 3))
SELECT ST_AsText(intersection(geometry 'Polygon((1 1,1 5,5 5,5 1,1 1))',
  geometry 'MultiPolygon(((2 2,2 4,3 4,3 2,2 2)),((4 2,4 4,6 4,6 2,4 2)))'));
-- MULTIPOLYGON(((3 2,2 2,2 4,3 4,3 2)),((5 2,4 2,4 4,5 4,5 2)))
SELECT intersection(geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))',
  geometry 'Polygon((10 10,10 11,11 11,11 10,10 10))') IS NULL;
-- true
```

- Return the region two geometries cover together

```
geoUnion(geometry,geometry) → geometry
```

  `UNION` is a reserved word of SQL, so this one carries the name of the type it answers over, as `setUnion` and `spanUnion` do. Two geometries that do not meet are answered as a multipolygon.

```
SELECT ST_AsText(geoUnion(geometry 'Polygon((1 1,1 5,5 5,5 1,1 1))',
  geometry 'Polygon((3 3,3 7,7 7,7 3,3 3))'));
-- POLYGON((5 1,1 1,1 5,3 5,3 7,7 7,7 3,5 3,5 1))
```

- Return the region of the first geometry that the second does not cover

```
difference(geometry,geometry) → geometry
```

  A second geometry lying inside the first leaves a hole rather than dividing it.

```
SELECT ST_AsText(difference(geometry 'Polygon((1 1,1 5,5 5,5 1,1 1))',
  geometry 'Polygon((3 3,3 7,7 7,7 3,3 3))'));
-- POLYGON((5 1,1 1,1 5,3 5,3 3,5 3,5 1))
SELECT ST_AsText(difference(geometry 'Polygon((1 1,1 5,5 5,5 1,1 1))',
  geometry 'Polygon((2 2,2 4,4 4,4 2,2 2))'));
-- POLYGON((5 1,1 1,1 5,5 5,5 1),(2 4,2 2,4 2,4 4,2 4))
```

- Return the region exactly one of two geometries covers

  ```
  symDifference(geometry,geometry) → geometry
  ```

  It is the union of the two geometries less the region they share, so a geometry against itself is answered as `NULL`.

  ```
  SELECT ST_AsText(symDifference(geometry 'Polygon((1 1,1 5,5 5,5 1,1 1))',
    geometry 'Polygon((3 3,3 7,7 7,7 3,3 3))'));
  -- MULTIPOLYGON(((7 3,5 3,5 5,3 5,3 7,7 7,7 3)),((5 1,1 1,1 5,3 5,3 3,5 3,5 1)))
  SELECT symDifference(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
    geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))') IS NULL;
  -- true
  ```

7.3 Notation

We presented in Section 4.5 and in Section 6.1 the notation used for defining the signature of the functions and operators for temporal types. We extend next this notations for spatiotemporal types.

- `tspatial` represents a spatiotemporal type, such as, `tgeometry`, `tgeompoint`, `tpose`, or `tnpoint`,
- `tgeo` represents a temporal geometry/geography type, that is, `tgeometry`, `tgeography`, `tgeompoint`, or `tgeogpoint`,
- `tgeom` represents a temporal geometry type that is, `tgeometry` or `tgeompoint`,
- `tgeog` represents a temporal geography type, that is, `tgeography` or `tgeogpoint`,
- `tpoint` represents a temporal point type, that is, `tgeompoint` or `tgeogpoint`,
- `spatial` represents a spatial base type, such as, `geometry`, `geography`, `pose`, or `npoint`,
- `geo` represents the types `geometry` or `geography`,
- `geompoint` represents the type `geometry` restricted to a point.
- `geogpoint` represents the type `geography` restricted to a point.
- `point` represents the types `geometry` or `geography` restricted to a point.
- `lines` represents the types `geometry` or `geography` restricted to a (multi)line.

In the following, we specify with the symbol ⬡ that the function supports 3D geometries and with the symbol 🌐 that the function supports geographies.

7.4 Input and Output

- Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation

```
asText({tspatial,tspatial[],spatial[]}) → {text,text[]}
asEWKT({tspatial,tspatial[],spatial[]}) → {text,text[]}
```

```
SELECT asText(tgeompoint 'SRID=4326;[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02)');
-- [POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02)
SELECT asText(ARRAY[tgeometry 'SRID=4326;[Point(0 0)@2001-01-01,
  Linestring(1 1,2 1)@2001-01-02]', 'Polygon((1 1,2 2,3 1,1 1))@2001-01-01']);
/* {"[POINT(0 0)@2001-01-01, LINESTRING(1 1,2 1)@2001-01-02]",
   "POLYGON((1 1,2 2,3 1,1 1))@2001-01-01"} */
SELECT asText(ARRAY[geometry 'Point(0 0)', 'Point(1 1)']);
-- {"POINT(0 0)","POINT(1 1)"}
```

```
SELECT asEWKT(tgeompoint 'SRID=4326;[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02)');
-- SRID=4326;[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02)
SELECT asEWKT(ARRAY[tgeometry 'SRID=4326;[Point(0 0)@2001-01-01,
  Linestring(1 1,2 1)@2001-01-02]', 'Polygon((1 1,2 2,3 1,1 1))@2001-01-01']);
-- {"SRID=4326;[POINT(0 0)@2001-01-01, LINESTRING(1 1,2 1)@2001-01-02]",
    "POLYGON((1 1,2 2,3 1,1 1))@2001-01-01"}
SELECT asEWKT(ARRAY[geometry 'SRID=5676;Point(0 0)', 'SRID=5676;Point(1 1)']);
-- {"SRID=5676;POINT(0 0)","SRID=5676;POINT(1 1)"}
```

- Return the Moving Features JSON representation

```
asMFJSON(tspatial,options integer=0,flags integer=0,maxdecimaldigits integer=15) → text
```

The `options` argument can be used to add BBOX and/or CRS in MFJSON output:

- 0: means no option (default value)
- 1: MFJSON BBOX
- 2: MFJSON Short CRS (e.g., EPSG:4326)
- 4: MFJSON Long CRS (e.g., urn:ogc:def:crs:EPSG::4326)

The `flags` argument can be used to customize the JSON output, for example, to produce an easy-to-read (for human readers) JSON output. Refer to the documentation of the `json-c` library for the possible values. Typical values are as follows:

- 0: means no option (default value)
- 1: JSON_C_TO_STRING_SPACED
- 2: JSON_C_TO_STRING_PRETTY

The `maxdecimaldigits` argument can be used to set the maximum number of decimal places in the output of floating point values (default 15).

```
SELECT asMFJSON(tgeompoint 'Point(1 2)@2019-01-01 18:00:00.15+02');
/* {"type":"MovingPoint","coordinates":[[1,2]],"datetimes":["2019-01-01T17:00:00.15"],
   "interpolation":"None"} */
SELECT asMFJSON(tgeometry 'SRID=3812;Linestring(1 1,1 2)@2019-01-01 18:00:00.15+02');
/* {"type":"MovingGeometry", "crs":{"type":"Name","properties":{"name":"EPSG:3812"}},
   "values":[{"type":"LineString","coordinates":[[1,1],[1,2]]}],
   "datetimes":["2019-01-01T17:00:00.15"],"interpolation":"None"} */
SELECT asMFJSON(tgeompoint 'SRID=4326;
  Point(50.813810 4.384260)@2019-01-01 18:00:00.15+02', 3, 0, 2);
/* {"type":"MovingPoint","crs":{"type":"name","properties":{"name":"EPSG:4326"}},
   "stBoundedBy":{"bbox":[50.81,4.38,50.81,4.38],
   "period":{"begin":"2019-01-01 17:00:00.15","end":"2019-01-01 17:00:00.15"}},
   "coordinates":[[50.81,4.38]],"datetimes":["2019-01-01T17:00:00.15"],
   "interpolations":"None"} */
```

- Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

  ```
  asBinary(tspatial,endian text='') → bytea
  asEWKB(tspatial,endian text='') → bytea
  asHexEWKB({tspatial,geo},endian text='') → text
  ```

  The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used.

  ```
  SELECT asBinary(tgeompoint 'Point(1 2 3)@2001-01-01');
  -- \x012e001101e9030000000000000000f03f00000000000000400000000000000840009c57d3c11c0000
  SELECT asEWKB(tgeogpoint 'SRID=7844;Point(1 2 3)@2001-01-01');
  -- \x012f0071a41e0000000000000000f03f00000000000000400000000000000840009c57d3c11c0000
  SELECT asHexEWKB(tgeompoint 'SRID=3812;Point(1 2 3)@2001-01-01');
  -- 012E0051E40E0000000000000000F03F00000000000000400000000000000840009C57D3C11C0000
  SELECT asHexEWKB(geometry 'SRID=3812;Point(1 2)');
  -- 0101000020E40E0000000000000000F03F0000000000000040
  ```

- Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation

  ```
  tspatialFromText(text) → tspatial
  tspatialFromEWKT(text) → tspatial
  ```

  In the above functions, `tspatial` replaces any spatial type, such as `tgeompoint`, `tgeometry`, or `tpose`.

  ```
  SELECT asEWKT(tgeompointFromText(text '[POINT(1 2)@2001-01-01, POINT(3 4)@2001-01-02]'));
  -- [POINT(1 2)@2001-01-01, POINT(3 4)@2001-01-02]
  SELECT asEWKT(tgeographyFromText(text '[Point(1 2)@2001-01-01,
    Linestring(1 2,3 4)@2001-01-02]'));
  -- SRID=4326;[POINT(1 2)@2001-01-01, LINESTRING(1 2,3 4)@2001-01-02]
  ```

  ```
  SELECT asEWKT(tgeompointFromEWKT(text 'SRID=3812;
    [Point(1 2)@2001-01-01, Point(3 4)@2001-01-02]'));
  -- SRID=3812;[POINT(1 2)@2001-01-01, POINT(3 4)@2001-01-02]
  SELECT asEWKT(tgeographyFromEWKT(text 'SRID=7844;
    [Point(1 2)@2001-01-01, Linestring(1 2,3 4)@2001-01-02]'));
  -- SRID=7844;[POINT(1 2)@2001-01-01, LINESTRING(1 2,3 4)@2001-01-02]
  ```

- Input from the Moving Features JSON representation

  ```
  tspatialFromMFJSON(text) → tspatial
  ```

  In the above function, `tspatial` replaces any spatiotemporal type, for example, `tgeompoint`, `tgeometry`, or `tpose`

  ```
  SELECT asEWKT(tgeompointFromMFJSON(text '{"type":"MovingPoint",
    "crs":{"type":"name", "properties":{"name":"EPSG:4326"}},"coordinates":[[50.81,4.38]],
    "datetimes":["2019-01-01T17:00:00.15+01"],"interpolation":"None"}'));
  -- SRID=4326;POINT(50.81 4.38)@2019-01-01 17:00:00.15
  SELECT asEWKT(tgeogpointFromMFJSON(text '{"type":"MovingPoint",
    "crs":{"type":"name", "properties":{"name":"EPSG:4326"}},"coordinates":[[50.81,4.38]],
    "datetimes":["2019-01-01T17:00:00.15+01"],"interpolation":"None"}'));
  -- SRID=4326;POINT(50.81 4.38)@2019-01-01 17:00:00.15
  SELECT asEWKT(tgeographyFromMFJSON(text '{"type":"MovingGeometry",
    "crs":{"type":"Name","properties":{"name":"EPSG:7844"}},
    "values":[{"type":"LineString","coordinates":[[1,1],[1,2]]}],
    "datetimes":["2019-01-01T17:00:00.15+01"],"interpolation":"None"}'));
  -- SRID=7844;LINESTRING(1 1,1 2)@2019-01-01 17:00:00.15
  ```

- Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), from the Hexadecimal Well-Known Binary (HexWKB), or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

```
tspatialFromBinary(bytea) → tspatial
tspatialFromEWKB(bytea) → tspatial
tspatialFromHexWKB(text) → tspatial
tspatialFromHexEWKB(text) → tspatial
geometryFromHexEWKB(text) → geometry
geographyFromHexEWKB(text) → geography
```

In the previous functions, `tspatial` replaces any spatiotemporal type, for example, `tgeompoint`, `tgeometry`, or `tpose`

```
SELECT asEWKT(tgeompointFromBinary(
  '\x012e0011000000000000f03f00000000000000400000000000000840009c57d3c11c0000'));
-- POINT Z (1 2 3)@2001-01-01
SELECT asEWKT(tgeogpointFromEWKB(
  '\x012f0071a41e0000000000000000f03f00000000000000400000000000000840009c57d3c11c0000'));
-- SRID=7844;POINT Z (1 1 1)@2001-01-01
SELECT asEWKT(tgeompointFromHexEWKB(
  '012E0051E40E0000000000000000F03F00000000000000400000000000000840009C57D3C11C0000'));
-- SRID=3812;POINT(1 2 3)@2001-01-01
SELECT ST_AsEWKT(geographyFromHexEWKB(
  '0101000020E61000006666666666661140CDCCCCCCCC6C4940'));
-- SRID=4326;POINT(4.35 50.85)
```

7.5 Conversions

- Convert a spatiotemporal value to a spatiotemporal box

```
tspatial::stbox
stbox(tspatial)
```

```
SELECT tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03]'::stbox;
-- STBOX XT(((1,1),(3,3)),[2001-01-01, 2001-01-03])
SELECT tgeography '[Point(1 1 1)@2001-01-01, Point(3 3 3)@2001-01-03]'::stbox;
-- SRID=4326;GEODSTBOX ZT(((1,1,1),(3,3,3)),[2001-01-01, 2001-01-03])
```

- Convert between a temporal geometry and a temporal geography

```
{tgeometry,tgeography}::{tgeography,tgeometry}
{tgeompoint,tgeogpoint}::{tgeogpoint,tgeompoint}
```

The conversions follow the rule of [?listitem].

```
SELECT asText((tgeompoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02)')::tgeogpoint);
-- [POINT(0 0)@2001-01-01, POINT(0 1)@2001-01-02)
SELECT asText((tgeography 'Linestring(0 0,1 1)@2001-01-01')::tgeometry);
-- LINESTRING(0 0,1 1)@2001-01-01
```

A common way to store temporal points in PostGIS is to represent them as geometries of type `LINESTRING M` and use the M dimension to encode timestamps as seconds since 1970-01-01 00:00:00. These time-enhanced geometries, called trajectories, can be validated with the function `ST_IsValidTrajectory` to verify that the M value is growing from each vertex to the next. Trajectories can be manipulated with the functions `ST_ClosestPointOfApproach`, `ST_DistanceCPA`, and `ST_CPAWithin`. Temporal point values can be converted to/from PostGIS trajectories.

- Convert between a temporal point and a PostGIS trajectory

```
{tpoint,geo}::{geo,tpoint}
```

```
SELECT ST_AsText((tgeompoint 'Point(0 0)@2001-01-01')::geometry);
-- POINT M (0 0 978307200)
SELECT ST_AsText((tgeompoint '{Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(1 1)@2001-01-03}')::geometry);
-- MULTIPOINT M (0 0 978307200,1 1 978393600,1 1 978480000)"
SELECT ST_AsText((tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02)')::geometry);
-- LINESTRING M (0 0 978307200,1 1 978393600)
SELECT ST_AsText((tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02),
  [Point(1 1)@2001-01-03, Point(1 1)@2001-01-04),
  [Point(1 1)@2001-01-05, Point(0 0)@2001-01-06)}')::geometry);
/* MULTILINESTRING M ((0 0 978307200,1 1 978393600),(1 1 978480000,1 1 978566400),
   (1 1 978652800,0 0 978739200)) */
SELECT ST_AsText((tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02),
  [Point(1 1)@2001-01-03], [Point(1 1)@2001-01-05, Point(0 0)@2001-01-06)}')::geometry);
/* GEOMETRYCOLLECTION M (LINESTRING M (0 0 978307200,1 1 978393600),
   POINT M (1 1 978480000),LINESTRING M (1 1 978652800,0 0 978739200)) */
```

```
SELECT asText(geometry 'LINESTRING M (0 0 978307200,0 1 978393600,
  1 1 978480000)'::tgeompoint);
-- [POINT(0 0)@2001-01-01, POINT(0 1)@2001-01-02, POINT(1 1)@2001-01-03];
SELECT asText(geometry 'GEOMETRYCOLLECTION M (LINESTRING M (0 0 978307200,1 1 978393600),
  POINT M (1 1 978480000),LINESTRING M (1 1 978652800,0 0 978739200))'::tgeompoint);
/* {[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02], [POINT(1 1)@2001-01-03],
   [POINT(1 1)@2001-01-05, POINT(0 0)@2001-01-06]} */
```

7.6 Accessors

- Return the trajectory or the traversed area

```
trajectory(tpoint,unary_union boolean=false) → geo
traversedArea(tgeo,unary_union boolean=false) → geo
```

This function is equivalent to getValues for temporal alphanumeric values. The last argument states whether the PostGIS function ST_UnaryUnion is applied to remove redundant geometries in the result. Notice that setting this argument to true is computationally expensive.

```
SELECT ST_AsText(trajectory(tgeompoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02,
  Point(0 0)@2001-01-03]'));
-- LINESTRING(0 0,0 1,0 0)
SELECT ST_AsText(trajectory(tgeompoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02,
  Point(0 0)@2001-01-03]', true));
-- LINESTRING(0 0,0 1)
SELECT ST_AsText(trajectory(tgeompoint '{[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02),
  [Point(0 1)@2001-01-03, Point(0 0)@2001-01-04)}'));
-- LINESTRING(0 0,0 1)
SELECT ST_AsText(trajectory(tgeompoint 'Interp=Step;{[Point(0 0)@2001-01-01,
  Point(0 1)@2001-01-02], [Point(0 1)@2001-01-03, Point(1 1)@2001-01-04]}'));
-- MULTIPOINT((0 0),(1 1),(0 1))
```

```
SELECT ST_AsText(traversedArea(tgeometry '[Point(1 1)@2001-01-01,
  Linestring(1 1,2 2)@2001-01-02, Point(1 1)@2001-01-03]'));
-- GEOMETRYCOLLECTION(POINT(1 1),LINESTRING(1 1,2 2))
SELECT ST_AsText(traversedArea(tgeometry '[Point(1 1)@2001-01-01,
  Linestring(1 1,2 2)@2001-01-02, Point(1 1)@2001-01-03]', true));
-- LINESTRING(1 1,2 2)
```

- Return the centroid as a temporal point

```
centroid(tgeo) → tpoint
centroid(tgeography,spheroid boolean=true) → tgeogpoint
```

For geographies, the computation is done on the WGS 84 spheroid by default. If the last argument is false, a faster spherical calculation is used.

```
SELECT asText(centroid(tgeometry '[Point(1 1)@2001-01-01, Linestring(1 1,3 3)@2001-01-02,
  Polygon((1 1,4 4,7 1,1 1))@2001-01-03]'));
-- Interp=Step;[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02, POINT(4 2)@2001-01-03]
SELECT asText(centroid(tgeography '[MultiPoint(1 1,4 4,7 1)@2001-01-01,
  Polygon((1 1,4 4,7 1,1 1))@2001-01-02]'),6);
-- Interp=Step;[POINT(4 2.001727)@2001-01-01, POINT(4 2.001727)@2001-01-02]
SELECT asText(centroid(tgeography '{Linestring(0 0,0 1,1 1)@2001-01-01}', false), 6);
-- {POINT(0.249967 0.750007)@2001-01-01}
```

- Return the X/Y/Z coordinate values as a temporal float

```
getX(tpoint) → tfloat
getY(tpoint) → tfloat
getZ(tpoint) → tfloat
```

```
SELECT getX(tgeompoint '{Point(1 2)@2001-01-01, Point(3 4)@2001-01-02,
  Point(5 6)@2001-01-03}');
-- {1@2001-01-01, 3@2001-01-02, 5@2001-01-03}
SELECT getX(tgeogpoint 'Interp=Step;
  [Point(1 2 3)@2001-01-01, Point(4 5 6)@2001-01-02, Point(7 8 9)@2001-01-03]');
-- Interp=Step;[1@2001-01-01, 4@2001-01-02, 7@2001-01-03]
SELECT getY(tgeompoint '{Point(1 2)@2001-01-01, Point(3 4)@2001-01-02,
  Point(5 6)@2001-01-03}');
-- {2@2001-01-01, 4@2001-01-02, 6@2001-01-03}
SELECT getY(tgeogpoint 'Interp=Step;
  [Point(1 2 3)@2001-01-01, Point(4 5 6)@2001-01-02, Point(7 8 9)@2001-01-03]');
-- Interp=Step;[2@2001-01-01, 5@2001-01-02, 8@2001-01-03]
SELECT getZ(tgeompoint '{Point(1 2)@2001-01-01, Point(3 4)@2001-01-02,
  Point(5 6)@2001-01-03}');
-- The temporal point must have Z dimension
SELECT getZ(tgeogpoint 'Interp=Step;
  [Point(1 2 3)@2001-01-01, Point(4 5 6)@2001-01-02, Point(7 8 9)@2001-01-03]');
-- Interp=Step;[3@2001-01-01, 6@2001-01-02, 9@2001-01-03]
```

- Return true if the temporal point does not spatially self-intersect

```
isSimple(tpoint) → boolean
```

Notice that a temporal sequence set point is simple if every composing sequence is simple.

```
SELECT isSimple(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(0 0)@2001-01-03]');
-- false
SELECT isSimple(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02,
  Point(2 0 2)@2001-01-03, Point(0 0 0)@2001-01-04]');
-- false
SELECT isSimple(tgeompoint '{[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02],
  [Point(1 1 1)@2001-01-03, Point(0 0 0)@2001-01-04]}');
-- true
```

- Return the length traversed by the temporal point

```
length(tpoint) → float
length(geo) → float
length({geography,tgeogpoint},spheroid boolean=true) → float
```

The function `length` returns the length of a geometry or a geography, the one of a geography in meters on the spheroid, as the PostGIS function `ST_Length` does. If the last argument is false, a faster spherical calculation is used.

```
SELECT length(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02]');
-- 1.73205080756888
SELECT length(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02,
  Point(0 0 0)@2001-01-03]');
-- 3.46410161513775
SELECT length(tgeompoint 'Interp=Step;
  [Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02, Point(0 0 0)@2001-01-03]');
-- 0
SELECT length(geometry 'Linestring(0 0,3 4)');
-- 5
SELECT round(length(geography 'Linestring(4.35 50.85,4.40 50.85)'), 6);
-- 3521.188745
SELECT round(length(tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]',
  false), 3);
-- 111195.08
```

- Return the cumulative length traversed by the temporal point

```
cumulativeLength(tpoint) → tfloatSeq
cumulativeLength(tgeogpoint,spheroid boolean=true) → tfloatSeq
```

For geographies, the computation is done on the WGS 84 spheroid by default. If the last argument is false, a faster spherical calculation is used.

```
SELECT round(cumulativeLength(tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(1 0)@2001-01-03], [Point(1 0)@2001-01-04, Point(0 0)@2001-01-05]}'), 6);
-- {[0@2001-01-01, 1.414214@2001-01-02, 2.414214@2001-01-03],
  [2.414214@2001-01-04, 3.414214@2001-01-05]}
SELECT cumulativeLength(tgeompoint 'Interp=Step;
  [Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02, Point(0 0 0)@2001-01-03]');
-- Interp=Step;[0@2001-01-01, 0@2001-01-03]
SELECT round(cumulativeLength(tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]',
  false), 3);
-- [0@2001-01-01, 111195.08@2001-01-02]
```

- Return the speed of the temporal point in units per second

```
speed(tpoint) → tfloatSeqSet
speed(tgeogpoint,spheroid boolean=true) → tfloatSeqSet
```

The temporal point must have linear interpolation. For geographies, the computation is done on the WGS 84 spheroid by default. If the last argument is false, a faster spherical calculation is used.

```
SELECT speed(tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(1 0)@2001-01-03], [Point(1 0)@2001-01-04, Point(0 0)@2001-01-05]}') * 3600 * 24;
/* Interp=Step;{[1.4142135623731@2001-01-01, 1@2001-01-02, 1@2001-01-03],
   [1@2001-01-04, 1@2001-01-05]} */
SELECT speed(tgeompoint 'Interp=Step;
  [Point(0 0)@2001-01-01, Point(1 1)@2001-01-02, Point(1 0)@2001-01-03]');
-- ERROR:  The temporal value must have linear interpolation
SELECT round(speed(tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]',
  false), 6);
-- Interp=Step;[1.28698@2001-01-01, 1.28698@2001-01-02]
```

- Return the time-weighted centroid

```
twCentroid(tgeompoint) → point
```

```
SELECT ST_AsText(twCentroid(tgeompoint '{[Point(0 0 0)@2001-01-01,
  Point(0 1 1)@2001-01-02, Point(0 1 1)@2001-01-03, Point(0 0 0)@2001-01-04)}'));
-- POINT Z (0 0.666666666666667 0.666666666666667)
```

- Return the direction, that is, the azimuth between the start and end locations

```
direction(tpoint) → float
```

The result is expressed in radians. It is NULL if there is only one location or if the start and end locations are equal.

```
SELECT round(degrees(direction(tgeompoint '[Point(0 0)@2001-01-01,
  Point(-1 -1)@2001-01-02, Point(1 1)@2001-01-03]'))::numeric, 6);
-- 45.000000
SELECT direction(tgeompoint '{[Point(0 0 0)@2001-01-01, Point(0 1 1)@2001-01-02,
  Point(0 1 1)@2001-01-03, Point(0 0 0)@2001-01-04)}');
-- NULL
```

- Return the temporal azimuth

```
azimuth(tpoint) → tfloat
```

The result is expressed in radians. The azimut is undefined when two succesive locations are equal and in this case a temporal gap is added.

```
SELECT round(degrees(azimuth(tgeompoint '[Point(0 0 0)@2001-01-01,
  Point(1 1 1)@2001-01-02, Point(1 1 1)@2001-01-03, Point(0 0 0)@2001-01-04)')));
-- Interp=Step;{[45@2001-01-01, 45@2001-01-02], [225@2001-01-03, 225@2001-01-04)}
```

- Return the temporal angular difference

```
angularDifference(tpoint) → tfloat
```

The result is expressed in degrees.

```
SELECT round(angularDifference(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03]'), 3);
-- {0@2001-01-01, 180@2001-01-02, 0@2001-01-03}
SELECT round(degrees(angularDifference(tgeompoint '{[Point(1 1)@2001-01-01,
  Point(2 2)@2001-01-02], [Point(2 2)@2001-01-03, Point(1 1)@2001-01-04]}')), 3);
-- {0@2001-01-01, 0@2001-01-02, 0@2001-01-03, 0@2001-01-04}
```

- Return the temporal bearing

```
bearing({tpoint,point},{tpoint,point}) → tfloat
```

Notice that this function does not accept two temporal geographic points.

```
SELECT degrees(bearing(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03]',
  geometry 'Point(2 2)'));
-- [45@2001-01-01, 0@2001-01-02, 225@2001-01-03]
SELECT round(degrees(bearing(tgeompoint '[Point(0 0)@2001-01-01, Point(2 0)@2001-01-03]',
  tgeompoint '[Point(2 1)@2001-01-01, Point(0 1)@2001-01-03]')), 3);
--  [63.435@2001-01-01, 0@2001-01-02, 296.565@2001-01-03]
SELECT round(degrees(bearing(tgeompoint '[Point(2 1)@2001-01-01, Point(0 1)@2001-01-03]',
  tgeompoint '[Point(0 0)@2001-01-01, Point(2 0)@2001-01-03]')), 3);
-- [243.435@2001-01-01, 116.565@2001-01-03]
```

7.7 Transformations

- Round the coordinate values to a number of decimal places ⬡🌐

```
round(tspatial,integer=0) → tspatial
```

```
SELECT asText(round(tgeompoint '{Point(1.12345 1.12345 1.12345)@2001-01-01,
  Point(2 2 2)@2001-01-02, Point(1.12345 1.12345 1.12345)@2001-01-03}', 2));
/* {POINT Z (1.12 1.12 1.12)@2001-01-01, POINT Z (2 2 2)@2001-01-02,
   POINT Z (1.12 1.12 1.12)@2001-01-03} */
SELECT asText(round(tgeography 'Linestring(1.12345 1.12345,2.12345 2.12345)@2001-01-01',
  2));
-- LINESTRING(1.12 1.12,2.12 2.12)@2001-01-01
```

- Return an array of fragments of the temporal point which are simple ⬡

```
makeSimple(tpoint) → tgeompoint[]
```

```
SELECT asText(makeSimple(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(0 0)@2001-01-03]'));
/* {"[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02)",
   "[POINT(1 1)@2001-01-02, POINT(0 0)@2001-01-03]"} */
SELECT asText(makeSimple(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02,
  Point(2 0 2)@2001-01-03, Point(0 0 0)@2001-01-04]'));
/* {"[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02, POINT Z (2 0 2)@2001-01-03,
   POINT Z (0 0 0)@2001-01-04]"} */
SELECT asText(makeSimple(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02,
  Point(0 1)@2001-01-03, Point(1 0)@2001-01-04]'));
/* {"[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02, POINT(0 1)@2001-01-03)",
   "[POINT(0 1)@2001-01-03, POINT(1 0)@2001-01-04]"} */
SELECT asText(makeSimple(tgeompoint '{[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-02],
  [Point(1 1 1)@2001-01-03, Point(0 0 0)@2001-01-04]}'));
/* {"{[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02],
   [POINT Z (1 1 1)@2001-01-03, POINT Z (0 0 0)@2001-01-04]}"} */
```

- Construct a geometry/geography with M measure from a temporal point and a temporal float ⬡🌐

```
geoMeasure(tpoint,tfloat,segmentize boolean=false) → geo
```

The last argument `segmentize` states whether the resulting value is a either `Linestring M` or a `MultiLinestring M` where each component is a segment of two points.

```
SELECT ST_AsText(geoMeasure(tgeompoint '{Point(1 1 1)@2001-01-01,
  Point(2 2 2)@2001-01-02}', '{5@2001-01-01, 5@2001-01-02}'));
-- MULTIPOINT ZM (1 1 1 5,2 2 2 5)
SELECT ST_AsText(geoMeasure(tgeogpoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
  [Point(1 1)@2001-01-03, Point(1 1)@2001-01-04]}',
  '{[5@2001-01-01, 5@2001-01-02],[7@2001-01-03, 7@2001-01-04]}'));
-- GEOMETRYCOLLECTION M (POINT M (1 1 7),LINESTRING M (1 1 5,2 2 5))
SELECT ST_AsText(geoMeasure(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03]', '[5@2001-01-01, 7@2001-01-02, 5@2001-01-03]', true));
-- MULTILINESTRING M ((1 1 5,2 2 5),(2 2 7,1 1 7))
```

A typical visualization for mobility data is to show on a map the trajectory of the moving object using different colors according to the speed. Figure 7.3 shows the result of the query below using a color ramp in QGIS.

```
WITH Temp(t) AS (
  SELECT tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-05, Point(2 0)@2001-01-08,
    Point(3 1)@2001-01-10, Point(4 0)@2001-01-11]' )
```

```
SELECT ST_AsText(geoMeasure(t, round(speed(t) * 3600 * 24, 2), true)) FROM Temp;
/* MULTILINESTRING M ((0 0 0.35,1 1 0.35),(1 1 0.47,2 0 0.47),(2 0 0.71,3 1 0.71),
   (3 1 1.41,4 0 1.41)) */
```

The following expression is used in QGIS to achieve this. The `scale_linear` function transforms the M value of each composing segment to the range [0, 1]. This value is then passed to the `ramp_color` function.

```
ramp_color('RdYlBu', scale_linear(
    m(start_point(geometry_n($geometry,@geometry_part_num))), 0, 2, 0, 1) )
```

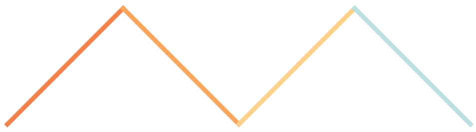

Figure 7.3: Visualizing the speed of a moving object using a color ramp in QGIS.

- Return a temporal geometry with no Z or with a Z, as the PostGIS functions `ST_Force2D` and `ST_Force3D` do for a geometry

  ```
  force2D(tgeo) → tgeo
  force3D(tgeo,zvalue float=0.0) → tgeo
  ```

  The type and the interpolation of the value stay. `force3D` gives the Z of a value with none, and a Z the value has stays. `force2D` may leave instants that a linear motion in two dimensions does not need, and the result is normalized, so it can hold fewer instants. These functions choose the dimensions that a relationship between two values reads, see Section 8.3. Since a geography is measured on the surface of the earth, its Z does not move its distances.

  ```
  SELECT asEWKT(force2D(tgeompoint '[Point(0 0 7)@2001-01-01, Point(2 0 7)@2001-01-03]'));
  -- [POINT(0 0)@2001-01-01, POINT(2 0)@2001-01-03]
  SELECT asEWKT(force3D(tgeompoint '[Point(0 0)@2001-01-01, Point(2 0)@2001-01-03]', 50));
  -- [POINT Z (0 0 50)@2001-01-01, POINT Z (2 0 50)@2001-01-03]
  SELECT force3D(tgeompoint '[Point(0 0)@2001-01-01, Point(10 0)@2001-01-02]') |=|
    tgeompoint '[Point(0 0 10000)@2001-01-01, Point(10 0 10000)@2001-01-02]';
  -- 10000
  ```

- Return the 3D affine transform of a temporal geometry to translate, rotate, and/or scale it in a single step

  ```
  affine(tgeo,a float,b float,c float,d float,e float,f float,g float,
    h float,i float,xoff float,yoff float,zoff float) → tgeo
  affine(tgeo,a float,b float,d float,e float,xoff float,yoff float) → tgeo
  affine(geometry,a float,b float,c float,d float,e float,f float,g float,
    h float,i float,xoff float,yoff float,zoff float) → geometry
  affine(geometry,a float,b float,d float,e float,xoff float,yoff float) → geometry
  ```

  Over a geometry, the function answers as its PostGIS sibling for a geometry the transformation keeps in its plane. A two-dimensional value the transformation moves out of its plane becomes three-dimensional, as a pose does.

  ```
  -- Rotate a 3D temporal point 180 degrees about the z axis
  SELECT asEWKT(affine(temp, cos(pi()), -sin(pi()), 0, sin(pi()), cos(pi()), 0, 0, 0, 1,
    0, 0, 0))
  FROM (SELECT tgeompoint '[POINT(1 2 3)@2001-01-01, POINT(1 4 3)@2001-01-02]' AS temp) t;
  -- [POINT Z (-1 -2 3)@2001-01-01, POINT Z (-1 -4 3)@2001-01-02]
  SELECT asEWKT(rotate(temp, pi())) FROM (SELECT tgeompoint '[POINT(1 2 3)@2001-01-01,
    POINT(1 4 3)@2001-01-02]' AS temp) t;
  -- [POINT Z (-1 -2 3)@2001-01-01, POINT Z (-1 -4 3)@2001-01-02]
  -- Rotate a 3D temporal point 180 degrees in both the x and z axis
  SELECT asEWKT(round(affine(temp, cos(pi()), -sin(pi()), 0, sin(pi()), cos(pi()), -sin(pi() ↩
     ),
  ```

```
  0, sin(pi()), cos(pi()), 0, 0, 0), 6)) FROM (SELECT tgeometry '[Point(1 1)@2001-01-01,
  Linestring(1 1,2 2)@2001-01-02]' AS temp) t;
-- [POINT Z (-1 -1 0)@2001-01-01, LINESTRING Z (-1 -1 0,-2 -2 0)@2001-01-02]
SELECT ST_AsEWKT(affine(geometry 'Point(1 2 3)', cos(pi()), -sin(pi()), 0,
  sin(pi()), cos(pi()), 0, 0, 0, 1, 0, 0, 0));
-- POINT(-1 -2 3)
```

- Return the temporal point rotated counter-clockwise about the origin point

```
rotate(tgeo,radians float) → tgeo
rotate(tgeo,radians float,x0 float,y0 float) → tgeo
rotate(tgeo,radians float,origin geometry) → tgeo
rotate(geometry,radians float) → geometry
rotate(geometry,radians float,x0 float,y0 float) → geometry
rotate(geometry,radians float,origin geometry) → geometry
```

Over a geometry, the function answers as its PostGIS sibling for a geometry the transformation keeps in its plane.

An origin given as a geometry is a point that is not empty and states the SRID of the temporal value or none, otherwise an error is raised.

```
-- Rotate a temporal point 180 degrees
SELECT asEWKT(rotate(tgeompoint '[Point(5 10)@2001-01-01, Point(5 5)@2001-01-02,
  Point(10 5)@2001-01-03]', pi()), 6);
-- [POINT(-5 -10)@2001-01-01, POINT(-5 -5)@2001-01-02, POINT(-10 -5)@2001-01-03]
-- Rotate 30 degrees counter-clockwise at x=5, y=10
SELECT asEWKT(rotate(tgeompoint '[Point(5 10)@2001-01-01, Point(5 5)@2001-01-02,
  Point(10 5)@2001-01-03]', pi()/6, 5, 10), 6);
-- [POINT(5 10)@2001-01-01, POINT(7.5 5.67)@2001-01-02, POINT(11.83 8.17)@2001-01-03]
-- Rotate 60 degrees clockwise from centroid
SELECT asEWKT(rotate(temp, -pi()/3, ST_Centroid(traversedArea(temp))),
  2) FROM (SELECT tgeometry '[Point(5 10)@2001-01-01, Point(5 5)@2001-01-02,
  Linestring(5 5,10 5)@2001-01-03]' AS temp) AS t;
/* [POINT(10.58 9.67)@2001-01-01, POINT(6.25 7.17)@2001-01-02,
   LINESTRING(6.25 7.17,8.75 2.83)@2001-01-03] */
SELECT ST_AsEWKT(rotate(geometry 'Linestring(5 10,5 5)', pi()/6, 5, 10), 2);
-- LINESTRING(5 10,7.5 5.67)
```

- Return a temporal geometry rotated counter-clockwise about the x axis

```
rotateX(tgeo,radians float) → tgeo
rotateX(geometry,radians float) → geometry
```

Over a geometry, the function answers as its PostGIS sibling for a geometry the transformation keeps in its plane. A two-dimensional value the transformation moves out of its plane becomes three-dimensional, as a pose does.

```
SELECT asEWKT(rotateX(tgeompoint '[Point(1 2 3)@2001-01-01, Point(1 4 3)@2001-01-02]',
  pi()), 6);
-- [POINT Z (1 -2 -3)@2001-01-01, POINT Z (1 -4 -3)@2001-01-02]
SELECT ST_AsEWKT(round(rotateX(geometry 'Point(1 1)', pi()/2), 6));
-- POINT(1 0 1)
SELECT asEWKT(round(rotateX(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02]',
  pi()/2), 6));
-- [POINT Z (1 0 1)@2001-01-01, POINT Z (2 0 2)@2001-01-02]
```

- Return a temporal geometry rotated counter-clockwise about the y axis

```
rotateY(tgeo,radians float) → tgeo
rotateY(geometry,radians float) → geometry
```

Over a geometry, the function answers as its PostGIS sibling for a geometry the transformation keeps in its plane. A two-dimensional value the transformation moves out of its plane becomes three-dimensional, as a pose does.

```
SELECT asEWKT(rotateY(tgeompoint '[Point(1 2 3)@2001-01-01, Point(1 4 3)@2001-01-02]',
  pi()), 6);
-- [POINT Z (-1 2 -3)@2001-01-01, POINT Z (-1 4 -3)@2001-01-02]
SELECT ST_AsEWKT(round(rotateY(geometry 'Linestring(1 1,2 2)', pi()/2), 6));
-- LINESTRING(0 1 -1,0 2 -2)
```

- Return a temporal geometry rotated counter-clockwise about the z axis

```
rotateZ(tgeo,radians float) → tgeo
rotateZ(geometry,radians float) → geometry
```

Over a geometry, the function answers as its PostGIS sibling for a geometry the transformation keeps in its plane.

```
SELECT asEWKT(rotateZ(tgeometry '[Point(1 1)@2001-01-01, Linestring(1 1,2 2)@2001-01-02]',
  pi()/2), 6);
-- [POINT(-1 1)@2001-01-01, LINESTRING(-1 1,-2 2)@2001-01-02]
SELECT ST_AsEWKT(rotateZ(geometry 'Linestring(1 1,2 2)', pi()/2), 6);
-- LINESTRING(-1 1,-2 2)
```

- Return a temporal point scaled by given factors

```
scale(tgeo,xfactor float,yfactor float,zfactor float) → tgeo
scale(tgeo,xfactor float,yfactor float) → tgeo
scale(tgeo,factor geometry) → tgeo
scale(tgeo,factor geometry,origin geometry) → tgeo
scale(geometry,xfactor float,yfactor float,zfactor float) → geometry
scale(geometry,xfactor float,yfactor float) → geometry
scale(geometry,factor geometry) → geometry
scale(geometry,factor geometry,origin geometry) → geometry
```

Over a geometry, the function answers as its PostGIS sibling for a geometry the transformation keeps in its plane. A two-dimensional value the transformation moves out of its plane becomes three-dimensional, as a pose does.

```
SELECT asEWKT(scale(tgeompoint '[Point(1 2 3)@2001-01-01, Point(1 1 1)@2001-01-02]', 0.5,
  0.75, 0.8));
--  [POINT Z (0.5 1.5 2.4)@2001-01-01, POINT Z (0.5 0.75 0.8)@2001-01-02]
SELECT asEWKT(scale(tgeompoint '[Point(1 2 3)@2001-01-01, Point(1 1 1)@2001-01-02]', 0.5,
  0.75));
-- [POINT Z (0.5 1.5 3)@2001-01-01, POINT Z (0.5 0.75 1)@2001-01-02]
SELECT asEWKT(scale(tgeompoint '[Point(1 2 3)@2001-01-01, Point(1 1 1)@2001-01-02]',
  geometry 'Point(0.5 0.75 0.8)'));
-- [POINT Z (0.5 1.5 2.4)@2001-01-01, POINT Z (0.5 0.75 0.8)@2001-01-02]
SELECT asEWKT(scale(tgeometry '[Point(1 1)@2001-01-01, Linestring(1 1,2 2)@2001-01-02]',
  geometry 'Point(2 2)', geometry 'Point(1 1)'));
-- [POINT(1 1)@2001-01-01, LINESTRING(1 1,3 3)@2001-01-02]
SELECT ST_AsEWKT(scale(geometry 'Linestring(1 1,2 2)', geometry 'Point(2 2)',
  geometry 'Point(1 1)'));
-- LINESTRING(1 1,3 3)
```

- Return a temporal geometry translated by given offsets and then scaled by given factors

```
transscale(tgeo,deltax float,deltay float,xfactor float,yfactor float) → tgeo
transscale(geometry,deltax float,deltay float,xfactor float,yfactor float) → geometry
```

Over a geometry, the function answers as its PostGIS sibling for a geometry the transformation keeps in its plane.

```
SELECT asEWKT(transscale(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02]',
  1, 1, 2, 3));
-- [POINT(4 6)@2001-01-01, POINT(6 9)@2001-01-02]
SELECT ST_AsEWKT(transscale(geometry 'Point(1 1)', 1, 1, 2, 3));
-- POINT(4 6)
```

- Return a temporal geometry translated by given offsets

```
translate(tgeo,deltax float,deltay float) → tgeo
translate(tgeo,deltax float,deltay float,deltaz float) → tgeo
translate(geometry,deltax float,deltay float) → geometry
translate(geometry,deltax float,deltay float,deltaz float) → geometry
```

Over a geometry, the function answers as its PostGIS sibling for a geometry the transformation keeps in its plane. A two-dimensional value the transformation moves out of its plane becomes three-dimensional, as a pose does.

```
SELECT asEWKT(translate(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02]',
  10, 20));
-- [POINT(11 21)@2001-01-01, POINT(12 22)@2001-01-02]
SELECT asEWKT(translate(tgeompoint '[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02]',
  10, 20, 30));
-- [POINT Z (11 21 31)@2001-01-01, POINT Z (12 22 32)@2001-01-02]
SELECT ST_AsEWKT(translate(geometry 'Point(1 1)', 10, 20));
-- POINT(11 21)
SELECT ST_AsEWKT(translate(geometry 'Point(1 1)', 0, 0, 5));
-- POINT(1 1 5)
```

- Transform a temporal geometric point into the coordinate space of a Mapbox Vector Tile

```
asMVTGeom(tpoint,bounds stbox,extent integer=4096,buffer integer=256,
  clip boolean=true) → (geom,times)
```

The result is a couple composed of a `geometry` value and an array of associated timestamp values encoded as Unix epoch. The parameters are as follows:

  - `tpoint` is the temporal point to transform
  - `bounds` is an `stbox` defining the geometric bounds of the tile contents without buffer
  - `extent` is the tile extent in tile coordinate space
  - `buffer` is the buffer distance in tile coordinate space
  - `clip` is a Boolean that determines if the resulting geometries and timestamps should be clipped or not

```
SELECT ST_AsText((mvt).geom),
  (mvt).times FROM (SELECT asMVTGeom(tgeompoint '[Point(0 0)@2001-01-01,
  Point(100 100)@2001-01-02]', stbox 'STBOX X((40,40),(60,60))') AS mvt ) AS t;
-- LINESTRING(-256 4352,4352 -256) | {946714680,946734120}
SELECT ST_AsText((mvt).geom), (mvt).times
FROM (SELECT asMVTGeom(tgeompoint '[Point(0 0)@2001-01-01, Point(100 100)@2001-01-02]',
  stbox 'STBOX X((40,40),(60,60))', clip:=false) AS mvt ) AS t;
-- LINESTRING(-8192 12288,12288 -8192) | {946681200,946767600}
```

- Extract from a temporal point with linear interpolation the subsequences where the point stays within an area with a specified maximum size for at least the given duration

```
stops(tpoint,maxdist float=0.0,minduration interval='0 minutes') → tpoint
```

The size of the area is the greatest distance between two points of the subsequence, which is the quantity the temporal float form of the function reads as the difference between the greatest and the least value it takes. If `maxDist` is not given it is assumed 0.0 and thus, the function extracts the constant segments of the given temporal point. The distance is computed in the units of the coordinate system for a `tgeompoint` and in meters on the WGS84 spheroid for a `tgeogpoint`. Note that even though the function accepts 3D geometries, the computation is always performed in 2D. A stop that begins at the first instant of a sequence includes that instant only if the sequence does, and the same holds for a stop that ends at its last instant. Stops are found in time order: each begins at the earliest instant from which the value stays within an area of the maximum size for the minimum duration, and ends at the last instant before it leaves the area.

```
SELECT asText(stops(tgeompoint '[Point(1 1)@2001-01-01, Point(1 1)@2001-01-02,
  Point(2 2)@2001-01-03, Point(2 2)@2001-01-04]'));
/* {[POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02], [POINT(2 2)@2001-01-03,
   POINT(2 2)@2001-01-04]} */
SELECT asText(stops(tgeompoint '[Point(1 1 1)@2001-01-01, Point(1 1 1)@2001-01-02,
  Point(2 2 2)@2001-01-03, Point(2 2 2)@2001-01-04]', 1.75));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (1 1 1)@2001-01-02, POINT Z (2 2 2)@2001-01-03,
   POINT Z (2 2 2)@2001-01-04]} */
```

Chapter 8

Temporal Geometry Types (Part 2)

8.1 Restrictions

- Restrict to (the complement of) a geometry

```
atGeometry(tgeom,geometry) → tgeom
minusGeometry(tgeom,geometry) → tgeom
```

The geometry must be in 2D and the computation with respect to it is done in 2D. The result preserves the Z dimension of the temporal point, if it exists.

A geodetic point is restricted to a `geography`, which must be a polygon or a multipolygon, and the computation is done on the sphere: the point travels the great circle between two positions and the region is bounded by the great circles through its own vertices, so the trip enters and leaves where those paths cross. The difference grows with latitude, a trip along a parallel standing well to the north of it.

```
SELECT asText(atGeometry(tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04)',
  geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))'));
-- {"[POINT(1 1)@2001-01-02, POINT(2 2)@2001-01-03]"}
SELECT astext(atGeometry(tgeompoint '[Point(0 0 0)@2001-01-01, Point(4 4 4)@2001-01-05]',
  geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))'));
-- {[POINT Z (1 1 1)@2001-01-02, POINT Z (2 2 2)@2001-01-03]}
SELECT asText(atGeometry(tgeompoint '[Point(1 1 1)@2001-01-01, Point(3 1 1)@2001-01-03,
  Point(3 1 3)@2001-01-05]', 'Polygon((2 0,2 2,2 4,4 0,2 0))'));
-- {[POINT Z (2 1 1)@2001-01-02, POINT Z (3 1 1)@2001-01-03, POINT Z (3 1 3)@2001-01-05]}
SELECT asText(atGeometry(tgeometry 'Linestring(1 1,10 1)@2001-01-01',
  'Polygon((0 0,0 5,5 5,5 0,0 0))'));
-- LINESTRING(1 1,5 1)@2001-01-01
```

```
SELECT asText(minusGeometry(tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04)',
  geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))'));
/* {[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02), (POINT(2 2)@2001-01-03,
   POINT(3 3)@2001-01-04)} */
SELECT astext(minusGeometry(tgeompoint '[Point(0 0 0)@2001-01-01,
  Point(4 4 4)@2001-01-05]', geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))'));
/* {[POINT Z (0 0 0)@2001-01-01, POINT Z (1 1 1)@2001-01-02),
   (POINT Z (2 2 2)@2001-01-03, POINT Z (4 4 4)@2001-01-05]} */
SELECT asText(minusGeometry(tgeompoint '[Point(1 1 1)@2001-01-01, Point(3 1 1)@2001-01-03,
  Point(3 1 3)@2001-01-05]', 'Polygon((2 0,2 2,2 4,4 0,2 0))'));
-- {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 1 1)@2001-01-02)}
SELECT asText(minusGeometry(tgeometry 'Linestring(1 1,10 1)@2001-01-01',
  'Polygon((0 0,0 5,5 5,5 0,0 0))'));
-- LINESTRING(5 1,10 1)@2001-01-01
```

```
SELECT asText(atGeometry(tgeogpoint 'SRID=4326;[Point(-170 65)@2001-01-01,
  Point(-100 65)@2001-01-03]',
  geography 'SRID=4326;Polygon((-150 60,-150 70,-120 70,-120 60,-150 60))'), 6);
/* {[POINT(-150 68.423603)@2001-01-01 14:39:22.550164,
   POINT(-120 68.423603)@2001-01-02 09:20:37.449835]} */
```

- Restrict to (the complement of) an `stbox`

```
atStbox(tgeom,stbox,borderInc boolean=true) → tgeompoint
minusStbox(tgeom,stbox,borderInc boolean=true) → tgeompoint
```

The third optional argument is used for multidimensional tiling (see Section 9.7) to exclude the upper border of the tiles when a temporal value is split in multiple tiles, so that all fragments of the temporal geometry are exclusive.

A temporal point is restricted in 3D. The Z dimension is taken into account only when both the temporal point and the box have it, see Section 8.3, and the result preserves the Z dimension of the temporal point. The body of a temporal geometry is instead clipped in 2D: a box holding coordinates refuses a temporal geometry having Z dimension, while a box holding only a period restricts it in time as for any other temporal type. The restriction of a `tgeography` to a spatiotemporal box is not implemented.

A geodetic temporal point travels the great circle between two positions, and the box holds it where that circle enters and leaves the box: a trip from `Point(10 60)` to `Point(50 60)` reaches latitude 61.518762, and a trip across the antimeridian sweeps the longitudes the short way round. Two boxes meeting at a crossing state the same instant for it, so the pieces of a trip hold the period of the trip and its positions.

```
SELECT asText(atStbox(tgeogpoint '[Point(10 60)@2001-01-01, Point(50 60)@2001-01-02]',
  stbox 'SRID=4326;GEODSTBOX X((0 55),(60 61))', false), 6);
/* {[POINT(10 60)@2001-01-01, POINT(18.168064 61)@2001-01-01 05:02:50.433397),
   [POINT(41.831936 61)@2001-01-01 18:57:09.566602, POINT(50 60)@2001-01-02]} */
```

```
SELECT asText(atStbox(tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04)',
  stbox 'STBOX XT(((0,0),(2,2)),[2001-01-02, 2001-01-04])'));
-- {[POINT(1 1)@2001-01-02, POINT(2 2)@2001-01-03]}
SELECT asText(atStbox(tgeompoint '[Point(1 1 1)@2001-01-01, Point(3 3 3)@2001-01-03,
  Point(3 3 2)@2001-01-04, Point(3 3 7)@2001-01-09]', stbox 'STBOX Z((2,2,2),(3,3,3))'));
/* {[POINT Z (2 2 2)@2001-01-02, POINT Z (3 3 3)@2001-01-03, POINT Z (3 3 2)@2001-01-04,
   POINT Z (3 3 3)@2001-01-05]} */
SELECT asText(atStbox(tgeometry '[Point(1 1)@2001-01-01, Linestring(1 1,3 3)@2001-01-03,
  Point(2 2)@2001-01-04, Linestring(3 3,4 4)@2001-01-09]', stbox 'STBOX X((2,2),(3,3))'));
-- {[LINESTRING(2 2,3 3)@2001-01-03, POINT(2 2)@2001-01-04, POINT(3 3)@2001-01-09]}
```

```
SELECT asText(minusStbox(tgeompoint '[Point(1 1)@2001-01-01, Point(4 4)@2001-01-04)',
  stbox 'STBOX XT(((1,1),(2,2)),[2001-01-03,2001-01-04])'));
-- {(POINT(2 2)@2001-01-02, POINT(3 3)@2001-01-03)}
SELECT asText(minusStbox(tgeompoint '[Point(1 1 1)@2001-01-01, Point(3 3 3)@2001-01-03,
  Point(3 3 2)@2001-01-04, Point(3 3 7)@2001-01-09]', stbox 'STBOX Z((2,2,2),(3,3,3))'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02),
   (POINT Z (3 3 3)@2001-01-05, POINT Z (3 3 7)@2001-01-09]} */
SELECT asText(minusStbox(tgeometry '[Point(1 1)@2001-01-01,
  Linestring(1 1,3 3)@2001-01-03, Point(2 2)@2001-01-04, Linestring(1 1,4 4)@2001-01-09]',
  stbox 'STBOX X((2,2),(3,3))'));
/* {[POINT(1 1)@2001-01-01, LINESTRING(1 1,2 2)@2001-01-03,
   LINESTRING(1 1,2 2)@2001-01-04),[MULTILINESTRING((1 1,2 2),(3 3,4 4))@2001-01-09]} */
```

- Restrict to (the complement of) an elevation span

```
atElevation(tpoint,floatspan) → tpoint
minusElevation(tpoint,floatspan) → tpoint
```

The elevation of a temporal point is its Z coordinate, which runs linearly in time whether the point is planar or geodetic, so the periods the span admits are the same for both. The positions at the bounds of those periods are not: a geodetic point travels the great circle between two positions and reaches the elevation of a bound there rather than on the straight line in longitude and latitude.

```
SELECT astext(atElevation(tgeompoint '[Point(1 1 1)@2001-01-01, Point(4 4 4)@2001-01-04,
  Point(1 1 1)@2001-01-07]', floatspan '[2,3]'));
/* {[POINT Z (2 2 2)@2001-01-02, POINT Z (3 3 3)@2001-01-03],
   [POINT Z (3 3 3)@2001-01-05, POINT Z (2 2 2)@2001-01-06]} */
SELECT astext(minusElevation(tgeompoint '[Point(1 1 1)@2001-01-01,
  Point(4 4 4)@2001-01-04, Point(1 1 1)@2001-01-07]', floatspan '[2,3]'));
/* {[POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02),
   (POINT Z (3 3 3)@2001-01-03, POINT Z (4 4 4)@2001-01-04,
   POINT Z (3 3 3)@2001-01-05), (POINT Z (2 2 2)@2001-01-06,
   POINT Z (1 1 1)@2001-01-07]} */
```

```
SELECT asText(atElevation(tgeogpoint 'SRID=4326;[Point(1 1 1)@2001-01-01,
  Point(4 4 4)@2001-01-04, Point(1 1 1)@2001-01-07]', floatspan '[2,3]'), 6);
/* {[POINT Z (1.998577 2.00071 2)@2001-01-02, POINT Z (2.998373 3.000812 3)@2001-01-03],
   [POINT Z (2.998373 3.000812 3)@2001-01-05, POINT Z (1.998577 2.00071 2)@2001-01-06]} */
```

8.2 Spatial Reference System

- Return or set the spatial reference identifier

```
SRID(tspatial) → integer
setSRID(tspatial,integer) → tspatial
```

```
SELECT SRID(tgeompoint 'Point(0 0)@2001-01-01');
-- 0
SELECT asEWKT(setSRID(tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02)', 4326));
-- SRID=4326;[POINT(0 0)@2001-01-01, POINT(1 1)@2001-01-02)
```

- Transform to a spatial reference identifier

```
transform(tspatial,integer) → tspatial
transformPipeline(tspatial,pipeline text,srid integer,
  is_forward boolean=true) → tspatial
transformPipeline(geo,pipeline text,srid integer,
  is_forward boolean=true) → geo
```

The function `transformPipeline` transforms a geometry or a geography with a pipeline in the same way.

The `transform` function specifies the transformation with a target SRID. An error is raised when the input temporal point has an unknown SRID (represented by 0).

The `transformPipeline` function specifies the transformation with a defined coordinate transformation pipeline represented with the following string format:

`urn:ogc:def:coordinateOperation:AUTHORITY::CODE`

The SRID of the input temporal point is ignored, and the SRID of the output temporal point will be set to zero unless a value is provided via the optional `srid` parameter. As stated by the last parameter, the pipeline is executed by default in a forward direction; by setting the parameter to false, the pipeline is executed in the inverse direction.

```
SELECT asEWKT(transform(tgeompoint 'SRID=4326;Point(4.35 50.85)@2001-01-01', 3812));
-- SRID=3812;POINT(648679.018035303 671067.055638114)@2001-01-01
SELECT ST_AsText(transformPipeline(geometry 'SRID=4326;Point(2 49)',
  'urn:ogc:def:coordinateOperation:EPSG::16031'), 3);
-- POINT(426857.988 5427937.523)
```

```
WITH test(tgeo, pipeline) AS (
  SELECT tgeogpoint 'SRID=4326;{Point(4.3525 50.846667 100.0)@2001-01-01,
    Point(-0.1275 51.507222 100.0)@2001-01-02}',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(tgeo, pipeline, 4326), pipeline, 4326,
  false), 6) FROM test;
/* SRID=4326;{POINT Z (4.3525 50.846667 100)@2001-01-01,
   POINT Z (-0.1275 51.507222 100)@2001-01-02} */
```

8.3 Operands of Different Dimensions

An operation relating two spatial values reads only the dimensions both of them have. Two values with Z are read in 3D. If one of them has no Z, both are read in 2D, the Z of the other one is ignored, and neither an error nor a notice is raised. The order of the operands does not change the answer.

```
-- A car with no Z and a plane above it are at the distance of the map
SELECT tgeompoint '[Point(0 0)@2001-01-01, Point(10 0)@2001-01-02]' |=|
  tgeompoint '[Point(0 0 10000)@2001-01-01, Point(10 0 10000)@2001-01-02]';
-- 0
-- Two values with Z are read in 3D
SELECT tgeompoint '[Point(0 0 0)@2001-01-01, Point(10 0 0)@2001-01-02]' |=|
  tgeompoint '[Point(0 0 10000)@2001-01-01, Point(10 0 10000)@2001-01-02]';
-- 10000
-- An operation building a value requires the same dimensions
SELECT merge(tgeompoint 'Point(0 0)@2001-01-01', tgeompoint 'Point(0 0 0)@2001-01-02');
-- ERROR:  The arguments must be of the same dimensionality
-- Two reference systems are not comparable
SELECT tgeompoint 'SRID=3857;Point(0 0)@2001-01-01' |=|
  tgeompoint 'SRID=4326;Point(0 0)@2001-01-01';
-- ERROR:  Operation on mixed SRID
```

- The rule covers the distances (`|=|`, `<->`, `nearestApproachInstant`, `shortestLine`, `distance`, `maxDistance`, and the similarity distances), the distance within (`eDwithin`, `aDwithin`, `tDwithin`), the relationships `intersects` and `disjoint` in their ever, always, and temporal forms, and the bounding box operators. The nearest approach instant keeps the Z of the value it is taken from, and the shortest line of operands of different dimensions has no Z.
- An operation building or comparing values requires the same dimensions and raises an error otherwise. This is the case of equality and comparison, `merge` and the constructors, the restriction to a value, the aggregates, the union and the intersection of boxes, and the tiles.
- An operation with no definition in 3D, such as `contains`, `covers`, or `touches`, raises an error on any operand with Z.
- A type with two dimensions, such as a temporal circular buffer or a temporal rigid geometry, reads the X and Y of a 3D operand of its distances. Its other relationships raise an error on an operand with Z.
- A geography is measured on the surface of the earth: its Z is not read, whether or not the other operand has one.
- Values in two spatial reference systems always raise an error, since their units are not comparable.

Since the rule raises no error, a value that lacks its Z by mistake answers in 2D with no warning. The functions [?listitem] state the dimensions to read. A column declared with a Z dimension, such as `tgeompoint(Sequence, PointZ)`, accepts no value with no Z, so that a table cannot hold one.

8.4 Empty Geometries

An empty geometry is the empty set of points: it is nowhere. As in PostGIS, an operation gives it the answer the empty set gives, and neither an error nor a notice is raised, except where the geometry must be the position of a value.

```
-- It meets nothing
SELECT eIntersects(geometry 'Point empty',
  tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02, Point(1 1)@2001-01-03]');
-- f
SELECT tDisjoint(geometry 'Point empty',
  tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02, Point(1 1)@2001-01-03]');
-- [t@2001-01-01, t@2001-01-03]
-- Nothing can be measured to it
SELECT geometry 'Point empty' <->
  tgeompoint '[Point(2 2)@2001-01-01, Point(1 1)@2001-01-02, Point(2 2)@2001-01-03]';
-- NULL
-- It removes nothing and keeps nothing
SELECT asText(minusGeometry(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03]', geometry 'Linestring empty'));
-- [POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02, POINT(1 1)@2001-01-03]
SELECT asText(atGeometry(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03]', geometry 'Linestring empty'));
-- NULL
-- It cannot be the position of a value
SELECT tgeompoint(geometry 'Point empty', timestamptz '2001-01-01');
-- ERROR:  Only non-empty geometries accepted
```

- **It meets nothing.** A relationship asking for a common point, such as `intersects`, `contains`, `covers`, `touches`, the distance within, and the equality, is false, while `disjoint` and the inequality are true. A temporal relationship or comparison takes this answer over the whole time of the temporal value, and the ever and always forms answer the same.
- **Nothing can be measured to it.** The distances (`|=|`, `<->`, `nearestApproachInstant`, `shortestLine`) and `bearing` are `NULL`. It has no bounding box, so that `stbox`, `expandSpace`, `stboxes`, `splitNStboxes`, and `splitEachNStboxes` are `NULL`.
- **It removes nothing and keeps nothing.** The restrictions `atGeometry` and `atValue` are `NULL`, since no instant is left, and `minusGeometry` and `minusValue` return the temporal value unchanged.
- **It cannot be the position of a value.** Building a temporal value or a set from it, or giving it as the origin of `rotate` and `scale`, raises an error.

8.5 Bounding Box Operations

- Return the spatiotemporal bounding box expanded in the spatial dimension by a float value

  `expandSpace({spatial,tspatial},float) → stbox`

  ```
  SELECT expandSpace(geography 'Linestring(0 0,1 1)', 2);
  -- SRID=4326;GEODSTBOX X((-2,-2),(3,3))
  SELECT expandSpace(tgeompoint 'Point(0 0)@2001-01-01', 2);
  -- STBOX XT(((-2,-2),(2,2)),[2001-01-01,2001-01-01])
  ```

8.6 Distance Operations

Operands of different dimensions read the dimensions both of them have, see Section 8.3.

- Return the smallest distance ever

```
{geo,tgeo,stbox} |=| {geo,tgeo,stbox} → float
nearestApproachDistance({geography,tgeog,stbox},{geography,tgeog,stbox},
  spheroid boolean=true) → float
```

For geographies, the operator computes on the WGS 84 spheroid, and the function `nearestApproachDistance` computes on the sphere, faster, if its last argument is false.

```
SELECT tgeompoint '[Point(0 0)@2001-01-02, Point(1 1)@2001-01-04,
  Point(0 0)@2001-01-06)' |=| geometry 'Linestring(2 2,2 1,3 1)';
-- 1
SELECT tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03, Point(0 0)@2001-01-05)'
  |=| tgeompoint '[Point(2 0)@2001-01-02, Point(1 1)@2001-01-04, Point(2 2)@2001-01-06)';
-- 0.5
SELECT tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-03,
  Point(0 0 0)@2001-01-05)' |=| tgeompoint '[Point(2 0 0)@2001-01-02,
  Point(1 1 1)@2001-01-04, Point(2 2 2)@2001-01-06)';
-- 0.5
SELECT tgeometry '(Point(1 1)@2001-01-01,
  Linestring(3 1,1 1)@2001-01-03]' |=| geometry 'Linestring(1 3,2 2,3 3)';
-- 1
SELECT round((tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]' |=|
  geography 'Point(0 0)')::numeric, 3);
-- 110574.389
SELECT round(nearestApproachDistance(
  tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]',
  geography 'Point(0 0)', false)::numeric, 3);
-- 111195.080
```

The operator `|=|` can be used for doing nearest neighbor searches using a GiST or an SP-GiST index (see Section 10.2). This operator corresponds to the PostGIS function `ST_DistanceCPA`, altough the latter requires both arguments to be a trajectory.

When one argument is a spatiotemporal box carrying a period, the distance is measured from the part of the temporal value inside that period, and the result is `NULL` when no part of it is. A box carrying no period measures the whole temporal value.

```
SELECT ST_DistanceCPA(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-03,
  Point(0 0 0)@2001-01-05)'::geometry,
  tgeompoint '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04,
  Point(2 2 2)@2001-01-06)'::geometry);
-- 0.5
```

- Return the instant of the first temporal point at which the two arguments are at the nearest distance

```
nearestApproachInstant({geo,tgeo},{geo,tgeo}) → tgeo
nearestApproachInstant({geography,tgeog},{geography,tgeog},spheroid boolean=true) → tgeog
```

The function will only return the first instant that it finds if there are more than one. The resulting instant may be at an exclusive bound. For geographies, the closest points of each segment are located on the sphere and their distances are compared on the WGS 84 spheroid by default, as the PostGIS function `ST_Distance` measures a geography distance. If the last argument is false, they are compared on the sphere.

```
SELECT asText(nearestApproachInstant(tgeompoint '(Point(1 1)@2001-01-01,
  Point(3 1)@2001-01-03]', geometry 'Linestring(1 3,2 2,3 3)'));
-- POINT(2 1)@2001-01-02
SELECT asText(nearestApproachInstant(tgeompoint 'Interp=Step;
  (Point(1 1)@2001-01-01, Point(3 1)@2001-01-03]', geometry 'Linestring(1 3,2 2,3 3)'));
-- POINT(1 1)@2001-01-01
SELECT asText(nearestApproachInstant(tgeompoint '(Point(1 1)@2001-01-01,
  Point(2 2)@2001-01-03]', tgeompoint '(Point(1 1)@2001-01-01, Point(4 1)@2001-01-03]'));
-- POINT(1 1)@2001-01-01
```

```
SELECT asText(nearestApproachInstant(tgeometry
  '[Linestring(0 0 0,1 1 1)@2001-01-01, Point(0 0 0)@2001-01-03]', tgeometry
  '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04, Point(2 2 2)@2001-01-06]'));
-- LINESTRING Z (0 0 0,1 1 1)@2001-01-02
SELECT asText(nearestApproachInstant(tgeogpoint '{Point(0 1)@2001-01-01,
  Point(0.998 0)@2001-01-02}', geography 'Linestring(0 0,0 -0.5)'));
-- POINT(0 1)@2001-01-01
SELECT asText(nearestApproachInstant(tgeogpoint '{Point(0 1)@2001-01-01,
  Point(0.998 0)@2001-01-02}', geography 'Linestring(0 0,0 -0.5)', false));
-- POINT(0.998 0)@2001-01-02
```

Function `nearestApproachInstant` generalizes the PostGIS function `ST_ClosestPointOfApproach`. First, the latter function requires both arguments to be trajectories. Second, function `nearestApproachInstant` returns both the point and the timestamp of the nearest point of approach while the PostGIS function only provides the timestamp as shown next.

```
SELECT to_timestamp(ST_ClosestPointOfApproach( tgeompoint '[Point(0 0 0)@2001-01-01,
  Point(1 1 1)@2001-01-03, Point(0 0 0)@2001-01-05)'::geometry,
  tgeompoint '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04,
  Point(2 2 2)@2001-01-06)'::geometry));
-- 2001-01-03 12:00:00
```

- Return the minimum spatial distance between two sets of temporal geometries, ignoring time

```
minDistance(tgeo[],tgeo[]) → float
minDistance(tgeog[],tgeog[],spheroid boolean=true) → float
```

The distance is measured between the trajectories with time discarded, equal to `ST_Distance` of the trajectory of each argument, and the result is the minimum over every pair drawn from the two sets. Two values that follow the same path at different times are therefore at distance zero. The time-synchronous quantity, which compares two arguments only at the instants they share, is nearestApproachDistance. A single pair is the one-element case. For geographies, the computation is done on the WGS 84 spheroid by default. If the last argument is false, a faster spherical calculation is used.

```
SELECT minDistance(ARRAY[tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03)',
  tgeompoint '[Point(5 5)@2001-01-01, Point(6 6)@2001-01-03)'],
  ARRAY[tgeompoint '[Point(0 1)@2001-01-01, Point(1 0)@2001-01-03)']);
-- 0
SELECT round(minDistance(
  ARRAY[tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]'],
  ARRAY[tgeogpoint 'Point(0 0)@2001-01-01'], false)::numeric, 3);
-- 111195.080
```

- Return the qualifying index pairs of two arrays of temporal geometries, generalising the scalar spatial relationships to a set-set spatial join

```
eDwithinPairs(tgeo[],tgeo[],float) → setof(i integer,j integer)
aDwithinPairs(tgeo[],tgeo[],float) → setof(i integer,j integer)
eIntersectsPairs(tgeo[],tgeo[]) → setof(i integer,j integer)
aIntersectsPairs(tgeo[],tgeo[]) → setof(i integer,j integer)
eDisjointPairs(tgeo[],tgeo[]) → setof(i integer,j integer)
aDisjointPairs(tgeo[],tgeo[]) → setof(i integer,j integer)
eTouchesPairs(tgeometry[],tgeometry[]) → setof(i integer,j integer)
aTouchesPairs(tgeometry[],tgeometry[]) → setof(i integer,j integer)
tDwithinPairs(tgeo[],tgeo[],float) → setof(i integer,j integer,periods tstzspanset)
tIntersectsPairs(tgeo[],tgeo[]) → setof(i integer,j integer,periods tstzspanset)
tDisjointPairs(tgeo[],tgeo[]) → setof(i integer,j integer,periods tstzspanset)
tTouchesPairs(tgeometry[],tgeometry[]) → setof(i integer,j integer,periods tstzspanset)
eDwithinPairs(tgeog[],tgeog[],float,spheroid boolean=true) → setof(i integer,j integer)
aDwithinPairs(tgeog[],tgeog[],float,spheroid boolean=true) → setof(i integer,j integer)
```

Each function resolves the pruned cross product of the two input arrays inside a single call and returns the one-based indexes `i` and `j` of the qualifying pairs. The `e`-prefixed functions return the pairs for which the relationship ever holds, the `a`-prefixed functions the pairs for which it always holds, and the `t`-prefixed functions additionally return the periods during which it holds. Each input's spatiotemporal box is used as a bounding-box prefilter so that the exact relationship is computed only on the surviving candidate pairs, and pairs whose temporal extents do not overlap are excluded, matching the scalar relationships. The functions are defined for temporal points and temporal geometries in both the planar and geodetic cases, except the touches relationship, which is defined for planar geometries only. The indexes map back to the original identities through a parallel `array_agg(identity ORDER BY ...)`. For geographies, the computation is done on the WGS 84 spheroid by default. If the last argument is false, a faster spherical calculation is used.

```
SELECT i, j FROM eDwithinPairs(
  ARRAY[tgeompoint '[Point(0 0)@2001-01-01, Point(0 10)@2001-01-02]',
        tgeompoint '[Point(50 50)@2001-01-01, Point(50 60)@2001-01-02]'],
  ARRAY[tgeompoint '[Point(1 0)@2001-01-01, Point(1 10)@2001-01-02]'], 2.0);
-- 1 | 1
SELECT i, j, periods FROM tDwithinPairs(
  ARRAY[tgeompoint '[Point(0 0)@2001-01-01, Point(0 10)@2001-01-02]'],
  ARRAY[tgeompoint '[Point(1 0)@2001-01-01, Point(1 10)@2001-01-02]'], 2.0);
-- one pair, with the period during which it is within the distance
SELECT i, j FROM aDisjointPairs(
  ARRAY[tgeompoint '[Point(0 0)@2001-01-01, Point(0 10)@2001-01-02]'],
  ARRAY[tgeompoint '[Point(5 0)@2001-01-01, Point(5 10)@2001-01-02]']);
-- 1 | 1
```

- Return the line connecting the nearest approach point

```
shortestLine({geo,tgeo},{geo,tgeo}) → geo
shortestLine({geography,tgeog},{geography,tgeog},spheroid boolean=true) → geography
```

The function will only return the first line that it finds if there are more than one. For geographies, the line joins the points at the nearest approach instant, which the WGS 84 spheroid decides by default and the sphere if the last argument is false.

```
SELECT ST_AsText(shortestLine(tgeompoint '(Point(1 1)@2001-01-01, Point(3 1)@2001-01-03]',
  geometry 'Linestring(1 3,2 2,3 3)'));
-- LINESTRING(2 1,2 2)
SELECT ST_AsText(shortestLine(tgeompoint 'Interp=Step;
  (Point(1 1)@2001-01-01, Point(3 1)@2001-01-03]', geometry 'Linestring(1 3,2 2,3 3)'));
-- LINESTRING(1 1,2 2)
SELECT ST_AsText(shortestLine(tgeometry
  '[Linestring(0 0 0,1 1 1)@2001-01-01, Point(0 0 0)@2001-01-03]', tgeometry
  '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04, Point(2 2 2)@2001-01-06]'));
-- LINESTRING Z (0 0 0,2 0 0)
SELECT ST_AsText(shortestLine(tgeogpoint '{Point(0 1)@2001-01-01,
  Point(0.998 0)@2001-01-02}', geography 'Linestring(0 0,0 -0.5)'));
-- LINESTRING(0 1,0 0)
SELECT ST_AsText(shortestLine(tgeogpoint '{Point(0 1)@2001-01-01,
  Point(0.998 0)@2001-01-02}', geography 'Linestring(0 0,0 -0.5)', false));
-- LINESTRING(0.998 0,0 0)
```

Function `shortestLine` can be used to obtain the result provided by the PostGIS function `ST_CPAWithin` when both arguments are trajectories as shown next.

```
SELECT ST_Length(shortestLine( tgeompoint '[Point(0 0 0)@2001-01-01,
  Point(1 1 1)@2001-01-03, Point(0 0 0)@2001-01-05)',
  tgeompoint '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04,
  Point(2 2 2)@2001-01-06)')) <= 0.5;
-- true
SELECT ST_CPAWithin(tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 1)@2001-01-03,
  Point(0 0 0)@2001-01-05)'::geometry,
  tgeompoint '[Point(2 0 0)@2001-01-02, Point(1 1 1)@2001-01-04,
  Point(2 2 2)@2001-01-06)'::geometry, 0.5);
-- true
```

The temporal distance operator, denoted `<->`, computes the distance at each instant of the intersection of the temporal extents of their arguments and results in a temporal float. Computing temporal distance is useful in many mobility applications. For example, a moving cluster (also known as convoy or flock) is defined as a set of objects that move close to each other for a long time interval. This requires to compute temporal distance between two moving objects.

A temporal point with linear interpolation takes as its other argument a temporal point, a point, or, when it is planar and 2D, a geometry of any other type. A temporal point of any other interpolation, an instant included, has a value at its instants only, and takes any geometry or geography, its distance at each instant being the distance of the point to it. Notice that the temporal types only consider linear interpolation between values, while the distance is a root of a quadratic function. Therefore, the temporal distance operator gives a linear approximation of the actual distance value for temporal sequence points. In this case, the arguments are synchronized in the time dimension, and for each of the composing line segments of the arguments, the spatial distance between the start point, the end point, and the nearest point of approach is computed, as shown in the examples below.

- Return the temporal distance

  ```
  {geo,tgeo} <-> {geo,tgeo} → tfloat
  tDistance({geography,tgeog},{geography,tgeog},spheroid boolean=true) → tfloat
  ```

  For geographies, the operator computes on the WGS 84 spheroid, and the function `tDistance` computes on the sphere, faster, if its last argument is false.

  ```
  SELECT tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03)' <->
    geometry 'Point(0 1)';
  -- [1@2001-01-01, 0.707106781186548@2001-01-02, 1@2001-01-03)
  SELECT tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03)' <->
    tgeompoint '[Point(0 1)@2001-01-01, Point(1 0)@2001-01-03)';
  -- [1@2001-01-01, 0@2001-01-02, 1@2001-01-03)
  SELECT tgeompoint '[Point(0 1)@2001-01-01, Point(0 0)@2001-01-03)' <->
    tgeompoint '[Point(0 0)@2001-01-01, Point(1 0)@2001-01-03)';
  -- [1@2001-01-01, 0.707106781186548@2001-01-02, 1@2001-01-03)
  SELECT tgeometry '[Point(0 0)@2001-01-01, Linestring(0 0,1 1)@2001-01-02]' <->
    tgeometry '[Point(0 1)@2001-01-01, Point(1 0)@2001-01-02)';
  -- Interp=Step;[1@2001-01-01, 1@2001-01-02]
  SELECT round(tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]' <->
    geography 'Point(0 0)', 3);
  -- [110574.389@2001-01-01, 221149.453@2001-01-02]
  SELECT round(tDistance(tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]',
    geography 'Point(0 0)', false), 3);
  -- [111195.08@2001-01-01, 222390.159@2001-01-02]
  ```

8.7 Planar and Geodetic Computations

The coordinates of a temporal geometry (`tgeometry` or `tgeompoint`) are interpreted in the plane of their spatial reference system, and those of a temporal geography (`tgeography` or `tgeogpoint`) as longitudes and latitudes on the WGS 84 ellipsoid. Which operations a temporal geography supports, and how exactly, follows from three rules.

1. An operation is available for geographies when it is defined on the ellipsoid. This is the case of the metric operations, such as the distance, the distance within, the length, the area, the azimuth, or the centroid, which are computed on the WGS 84 spheroid, or on the sphere, faster, when their argument `spheroid` is false. It is also the case of the intersection and the disjointness, since two geographies intersect when their distance is zero. It is not the case of `contains`, `covers`, and `touches`, which are defined by the dimensionally extended nine-intersection model of planar topology, nor of the planar constructions, such as the union or the intersection of two geometries, which PostGIS does not define for geographies either. These functions raise an error for geographies.

2. The ever, always, and temporal versions of an operation available for geographies are computed exactly. The value of a temporal geography, and of a temporal geography point with discrete or step interpolation, is constant between two instants, so the operation is evaluated at each instant. A temporal geography point with linear interpolation moves along

a great circle between two instants, and the result of a temporal relationship changes at the instants at which its geodetic distance to the other argument crosses the given distance. These instants are computed when the other argument is a geography point or a temporal geography point. Against other geographies, such as lines and polygons, they are computed from the crossings of the great circle of each segment with the vertices and the edges of the geography on the sphere. On the spheroid, each instant at which the distance on the sphere stops increasing or decreasing near the given distance is moved to the extremum of the distance on the spheroid, and every crossing is computed on the distance on the spheroid.

3. A temporal relationship available for geographies accepts a geography or a temporal geography on either side, as its planar version accepts a geometry or a temporal geometry.

A temporal geometry in SRID 4326 is converted into a temporal geography with the same values by the cast `::tgeography`, and back by the cast `::tgeometry`. The function `transform` reprojects it instead into a planar spatial reference system whose unit is the meter, in which distances and areas are planar.

```
SELECT round(nearestApproachDistance(tgeometry 'SRID=4326;[Polygon((4.35 50.84,
  4.40 50.84,4.40 50.88,4.35 50.84))@2001-01-01, Point(4.37 50.85)@2001-01-02]',
  geometry 'SRID=4326;Point(4.45 50.84)')::numeric, 4);
-- 0.0500
SELECT round(nearestApproachDistance(tgeometry 'SRID=4326;[Polygon((4.35 50.84,
  4.40 50.84,4.40 50.88,4.35 50.84))@2001-01-01, Point(4.37 50.85)@2001-01-02]'
  ::tgeography, geography 'Point(4.45 50.84)')::numeric, 1);
-- 3521.9
```

8.8 Ever and Always Relationships

Operands of different dimensions read the dimensions both of them have, see Section 8.3.

- Ever or always contains

```
eContains({geometry,tgeom},{geometry,tgeom}) → boolean
aContains({geometry,tgeom},{geometry,tgeom}) → boolean
```

This function returns true if the temporal geometry and the geometry ever or always intersect at their interior. Recall that a geometry does not contain things in its boundary and thus, polygons and lines do not contain lines and points lying in their boundary. Please refer to the documentation of the ST_Contains function in PostGIS.

```
SELECT eContains(geometry 'Linestring(1 1,3 3)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-02]');
-- false
SELECT eContains(geometry 'Linestring(1 1,3 3,1 1)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-03]');
-- true
SELECT eContains(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(0 1)@2001-01-01, Point(4 1)@2001-01-02]');
-- false
SELECT eContains(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeometry '[Linestring(1 1,4 4)@2001-01-01, Point(3 3)@2001-01-04]');
-- true
```

- Ever or always covers

```
eCovers({geometry,tgeom},{geometry,tgeom}) → boolean
aCovers({geometry,tgeom},{geometry,tgeom}) → boolean
```

Please refer to the documentation of the ST_Contains and the ST_Covers function in PostGIS for detailed explanations about the difference between the two functions.

```
SELECT eCovers(geometry 'Linestring(1 1,3 3)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-02]');
-- false
SELECT eCovers(geometry 'Linestring(1 1,3 3,1 1)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-03]');
-- true
SELECT eCovers(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(0 1)@2001-01-01, Point(4 1)@2001-01-02]');
-- false
SELECT eCovers(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeometry '[Linestring(1 1,4 4)@2001-01-01, Point(3 3)@2001-01-04]');
-- true
```

- Is ever or always disjoint

```
eDisjoint({geo,tgeo},{geo,tgeo}) → boolean
aDisjoint({geo,tgeo},{geo,tgeo}) → boolean
```

```
SELECT eDisjoint(geometry 'Polygon((0 0,0 1,1 1,1 0,0 0))',
  tgeompoint '[Point(0 0)@2001-01-01, Point(1 1)@2001-01-03)');
-- false
SELECT eDisjoint(geometry 'Polygon((0 0 0,0 1 1,1 1 1,1 0 0,0 0 0))',
  tgeometry '[Linestring(1 1 1,2 2 2)@2001-01-01, Point(2 2 2)@2001-01-03]');
-- true
```

- Is ever or always at distance within

```
eDwithin({geo,tgeo},{geo,tgeo},float) → boolean
aDwithin({geometry,tgeom},{geometry,tgeom},float) → boolean
eDwithin({geography,tgeog},{geography,tgeog},float,spheroid boolean=true) → boolean
aDwithin({geography,tgeog},{geography,tgeog},float,spheroid boolean=true) → boolean
```

For geographies, the computation is done on the WGS 84 spheroid by default. If the last argument is false, a faster spherical calculation is used. Geographies are supported as stated in Section 8.7.

```
SELECT eDwithin(geometry 'Point(1 1 1)',
  tgeompoint '[Point(0 0 0)@2001-01-01, Point(1 1 0)@2001-01-02]', 1);
-- true
SELECT eDwithin(geometry 'Polygon((0 0 0,0 1 1,1 1 1,1 0 0,0 0 0))',
  tgeompoint '[Point(0 2 2)@2001-01-01,Point(2 2 2)@2001-01-02]', 1);
-- false
SELECT eDwithin(tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]',
  geography 'Point(0 0)', 110800);
-- true
SELECT eDwithin(tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]',
  geography 'Point(0 0)', 110800, false);
-- false
SELECT aDwithin(tgeogpoint 'Interp=Step;[Point(0.5 2.5)@2001-01-01, Point(2.5 2.5)@2001 ↩
    -01-02]',
  geography 'Polygon((0 0,3 0,3 3,2 3,2 1,1 1,1 3,0 3,0 0))', 50000);
-- true
```

- Ever or always intersects

```
eIntersects({geo,tgeo},{geo,tgeo}) → boolean
aIntersects({geometry,tgeom},{geometry,tgeom}) → boolean
```

```
SELECT eIntersects(geometry 'Polygon((0 0 0,0 1 0,1 1 0,1 0 0,0 0 0))',
  tgeompoint '[Point(0 0 1)@2001-01-01, Point(1 1 1)@2001-01-03)');
-- false
SELECT eIntersects(geometry 'Polygon((0 0 0,0 1 1,1 1 1,1 0 0,0 0 0))',
  tgeompoint '[Point(0 0 1)@2001-01-01, Point(1 1 1)@2001-01-03)');
-- true
```

- Ever or always touches

```
eTouches({geometry,tgeom},{geometry,tgeom}) → boolean
aTouches({geometry,tgeom},{geometry,tgeom}) → boolean
```

```
SELECT eTouches(geometry 'Polygon((0 0,0 1,1 1,1 0,0 0))',
  tgeompoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-03)');
-- true
```

8.9 Spatiotemporal Relationships

- Temporal contains

```
tContains(geometry,tgeom) → tbool
```

```
SELECT tContains(geometry 'Linestring(1 1,3 3)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-02]');
-- {[f@2001-01-01, f@2001-01-02]}
SELECT tContains(geometry 'Linestring(1 1,3 3,1 1)',
  tgeompoint '[Point(4 2)@2001-01-01, Point(2 4)@2001-01-03]');
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03]}
SELECT tContains(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(0 1)@2001-01-01, Point(4 1)@2001-01-02]');
-- {[f@2001-01-01, f@2001-01-02]}
SELECT tContains(geometry 'Polygon((1 1,1 3,3 3,3 1,1 1))',
  tgeompoint '[Point(1 4)@2001-01-01, Point(4 1)@2001-01-04]');
-- {[f@2001-01-01, f@2001-01-02], (t@2001-01-02, f@2001-01-03, f@2001-01-04]}
```

- Temporal disjoint

```
tDisjoint({geo,tgeo},{geo,tgeo}) → tbool
```

The function is the negation of `tIntersects`. Geographies are supported as stated in Section 8.7.

```
SELECT tDisjoint(geometry 'Polygon((1 1,1 2,2 2,2 1,1 1))',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04)');
-- {[t@2001-01-01, f@2001-01-02, f@2001-01-03], (t@2001-01-03, t@2001-01-04]}
SELECT tDisjoint(tgeompoint '[Point(0 3)@2001-01-01, Point(3 0)@2001-01-05)',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-05)');
-- {[t@2001-01-01, f@2001-01-03], (t@2001-01-03, t@2001-01-05)}
```

- Temporal distance within

```
tDwithin({geo,tgeo},{geo,tgeo},float) → tbool
tDwithin({geography,tgeog},{geography,tgeog},float,spheroid boolean=true) → tbool
```

Geographies are supported as stated in Section 8.7. The distance is measured on the WGS 84 spheroid by default. If the last argument is false, it is measured on the sphere. A segment of a temporal geography point moves along a great circle, and the result changes its value at every instant at which the geodetic distance crosses the given distance, which a segment may cross several times.

```
SELECT tDwithin(geometry 'Point(1 1)',
  tgeompoint '[Point(0 0)@2001-01-01, Point(2 2)@2001-01-03)', sqrt(2));
--  {[t@2001-01-01, t@2001-01-03)}
SELECT tDwithin(tgeompoint '[Point(1 0)@2001-01-01, Point(1 4)@2001-01-05]',
  tgeompoint 'Interp=Step;[Point(1 2)@2001-01-01, Point(1 3)@2001-01-05]', 1);
-- {[f@2001-01-01, t@2001-01-02, t@2001-01-04], (f@2001-01-04, t@2001-01-05]}
SELECT tDwithin(geography 'Point(1 0.1)',
  tgeogpoint '[Point(0 0)@2001-01-01, Point(2 0)@2001-01-03]', 15000);
/* {[f@2001-01-01, t@2001-01-01 21:48:53.195554, t@2001-01-02 02:11:06.804445],
   (f@2001-01-02 02:11:06.804445, f@2001-01-03]} */
SELECT tDwithin(tgeogpoint '[Point(0 0)@2001-01-01, Point(2 0)@2001-01-03]',
  tgeogpoint '[Point(1 1)@2001-01-01, Point(1 -1)@2001-01-03]', 20000, false);
/* {[f@2001-01-01, t@2001-01-01 20:56:51.37512, t@2001-01-02 03:03:08.624879],
   (f@2001-01-02 03:03:08.624879, f@2001-01-03]} */
SELECT tDwithin(tgeography '[Polygon((0 0,2 0,2 2,0 2,0 0))@2001-01-01,
  Point(3 3)@2001-01-02, Polygon((2 2,4 2,4 4,2 4,2 2))@2001-01-03]',
  geography 'Point(1 1)', 200000);
-- [t@2001-01-01, f@2001-01-02, t@2001-01-03]
```

- Temporal intersects

```
tIntersects({geo,tgeo},{geo,tgeo}) → tbool
```

Two temporal points intersect at the instants at which they are equal. Two temporal geometries or geographies, whose values are constant between their instants, intersect at the instants at which their values intersect, and two geographies intersect where their distance is zero. Geographies are supported as stated in Section 8.7.

```
SELECT tIntersects(geometry 'MultiPoint(1 1,2 2)',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-04)');
/* {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, t@2001-01-03],
   (f@2001-01-03, f@2001-01-04]} */
SELECT tIntersects(tgeompoint '[Point(0 3)@2001-01-01, Point(3 0)@2001-01-05)',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 3)@2001-01-05)');
-- {[f@2001-01-01, t@2001-01-03], (f@2001-01-03, f@2001-01-05)}
SELECT tIntersects(tgeometry '[Polygon((0 0,2 0,2 2,0 2,0 0))@2001-01-01,
  Point(3 3)@2001-01-02]', tgeometry '[Point(1 1)@2001-01-01, Point(1 1)@2001-01-02]');
-- [t@2001-01-01, f@2001-01-02]
SELECT tIntersects(tgeography '[Polygon((0 0,2 0,2 2,0 2,0 0))@2001-01-01,
  Point(3 3)@2001-01-02, Polygon((2 2,4 2,4 4,2 4,2 2))@2001-01-03]',
  geography 'Point(1 1)');
-- [t@2001-01-01, f@2001-01-02, f@2001-01-03]
```

- Temporal touches

```
tTouches({geometry,tgeom},{geometry,tgeom}) → tbool
```

```
SELECT tTouches(geometry 'Polygon((1 0,1 2,2 2,2 0,1 0))',
  tgeompoint '[Point(0 0)@2001-01-01, Point(3 0)@2001-01-04)');
-- {[f@2001-01-01, t@2001-01-02, t@2001-01-03], (f@2001-01-03, f@2001-01-04]}
```

Chapter 9

Temporal Types: Analytics Operations

9.1 Simplification

- Return a temporal float or a temporal geometry/geography simplified ensuring that consecutive values are at least a certain distance or time interval apart

  ```
  minDistSimplify({tfloat,tgeo},mindist float) → {tfloat,tgeo}
  minTimeDeltaSimplify({tfloat,tgeo},mint interval) → {tfloat,tgeo}
  ```

  In the case of temporal points, the distance is specified in the units of the coordinate system. Notice that simplification applies only to temporal sequences or sequence sets with linear interpolation. In all other cases, a copy of the given temporal value is returned.

  ```
  SELECT minDistSimplify(tfloat '[1@2001-01-01,2@2001-01-02,3@2001-01-04,4@2001-01-05]', 1);
  -- [1@2001-01-01, 3@2001-01-04, 4@2001-01-05]
  SELECT asText(minDistSimplify(tgeompoint '[Point(1 1 1)@2001-01-01,
    Point(2 2 2)@2001-01-02, Point(3 3 3)@2001-01-04, Point(5 5 5)@2001-01-05)', sqrt(3)));
  -- [POINT Z (1 1 1)@2001-01-01, POINT Z (3 3 3)@2001-01-04, POINT Z (5 5 5)@2001-01-05)
  SELECT asText(minDistSimplify(tgeompoint '[Point(1 1 1)@2001-01-01,
    Point(2 2 2)@2001-01-02, Point(3 3 3)@2001-01-04, Point(4 4 4)@2001-01-05)', sqrt(3)));
  -- [POINT Z (1 1 1)@2001-01-01, POINT Z (3 3 3)@2001-01-04, POINT Z (4 4 4)@2001-01-05)
  ```

  ```
  SELECT minTimeDeltaSimplify(tfloat '[1@2001-01-01, 2@2001-01-02, 3@2001-01-04,
    4@2001-01-05]', '1 day');
  -- [1@2001-01-01, 3@2001-01-04, 4@2001-01-05]
  SELECT asText(minTimeDeltaSimplify(tgeogpoint '[Point(1 1 1)@2001-01-01,
    Point(2 2 2)@2001-01-02, Point(3 3 3)@2001-01-04, Point(5 5 5)@2001-01-05)', '1 day'));
  -- [POINT Z (1 1 1)@2001-01-01, POINT Z (3 3 3)@2001-01-04, POINT Z (5 5 5)@2001-01-05)
  ```

- Return a temporal float or a temporal point simplified using the Douglas-Peucker algorithm

  ```
  maxDistSimplify({tfloat,tgeompoint,tgeometry},maxdist float,
    syncdist boolean=true) → {tfloat,tgeompoint,tgeometry}
  douglasPeuckerSimplify({tfloat,tgeompoint,tgeometry},maxdist float,
    syncdist boolean=true) → {tfloat,tgeompoint,tgeometry}
  ```

  The difference between the two functions is that `maxDistSimplify` uses a single-pass version of the algorithm whereas `douglasPeuckerSimplify` uses the standard recursive algorithm. The function removes values or points that are less than or equal to the distance passed as second argument. In the case of temporal points, the distance is specified in the units of the coordinate system. The third argument applies only for temporal points and specifies whether the spatial or the synchronized distance is used. Notice that simplification applies only to temporal sequences or sequence sets with linear interpolation. In all other cases, a copy of the given temporal value is returned.

```
-- Only synchronous distance for temporal floats
SELECT maxDistSimplify(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 3@2001-01-04,
  1@2001-01-05]', 1, false);
-- [1@2001-01-01, 1@2001-01-03, 3@2001-01-04, 1@2001-01-05]
SELECT asText(maxDistSimplify(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(3 1)@2001-01-03, Point(3 3)@2001-01-05, Point(5 1)@2001-01-06]', 2));
-- [POINT(1 1)@2001-01-01, POINT(3 3)@2001-01-05, POINT(5 1)@2001-01-06]
SELECT asText(maxDistSimplify(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(3 1)@2001-01-03, Point(3 3)@2001-01-05, Point(5 1)@2001-01-06]', 2, false));
-- [POINT(1 1)@2001-01-01, POINT(5 1)@2001-01-06]
```

```
-- Spatial vs synchronized Euclidean distance
SELECT asText(douglasPeuckerSimplify(tgeompoint '[Point(1 1)@2001-01-01,
  Point(6 1)@2001-01-06, Point(7 4)@2001-01-07]', 2.3, false));
-- [POINT(1 1)@2001-01-01, POINT(7 4)@2001-01-07]
SELECT asText(douglasPeuckerSimplify(tgeompoint '[Point(1 1)@2001-01-01,
  Point(6 1)@2001-01-06, Point(7 4)@2001-01-07]', 2.3, true));
-- [POINT(1 1)@2001-01-01, POINT(6 1)@2001-01-06, POINT(7 4)@2001-01-07]
```

The difference between the spatial and the synchronized distance is illustrated in the last two examples above and in Figure 9.1. In the first example, which uses the spatial distance, the second instant is removed since the perpendicular distance between `POINT(6 1)` and the line defined by `POINT(1 1)` and `POINT(7 4)` is equal to 2.23. On the contrary, in the second example the second instant is kept since the projection of `Point(6 2)` at timestamp `2001-01-06` over the temporal line segment results in `Point(6 3.5)` and the distance between the original point and its projection is 2.5.

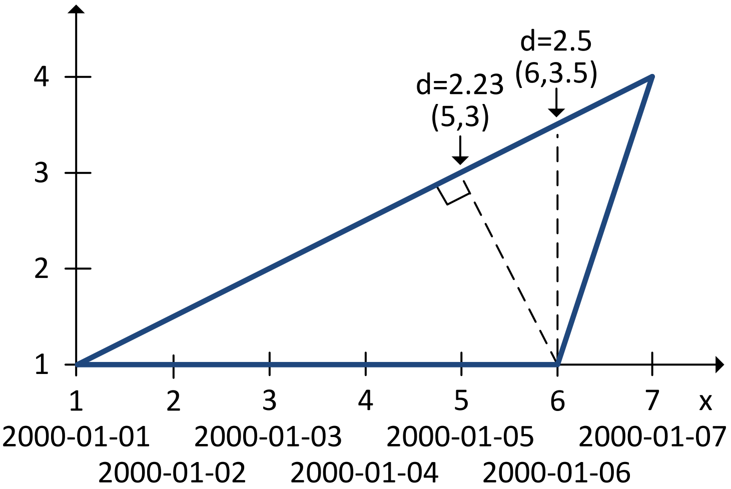

Figure 9.1: Difference between the spatial and the synchronous distance.

A typical use for the `douglasPeuckerSimplify` function is to reduce the size of a dataset, in particular for visualization purposes. If the visualization is static, then the spatial distance should be preferred, if the visualization is dynamic or animated, the synchronized distance should be preferred.

9.2 Reduction

- Sample a temporal value with respect to an interval

```
tsample(ttype,duration interval,torigin timestamptz='2000-01-03',
  interp text='discrete') → ttype
```

If the origin is not specified, it is set by default to Monday, January 3, 2000. The given interval must be strictly greater than zero.

```
SELECT tsample(tbool '[true@2001-01-01,false@2001-01-02]', '12 hours', '2001-01-01',
  'step');
-- [t@2001-01-01, f@2001-01-02]
SELECT tsample(tint '{1@2001-01-01,5@2001-01-05}', '3 days', '2001-01-01');
-- {1@2001-01-01}
SELECT tsample(tfloat '[1@2001-01-01,5@2001-01-05]', '1 day', '2001-01-01');
-- {1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 4@2001-01-04, 5@2001-01-05}
SELECT tsample(tfloat '[1@2001-01-01,5@2001-01-05]', '3 days', '2001-01-01');
-- {1@2001-01-01, 4@2001-01-04}
SELECT tsample(tfloat '[1@2001-01-01,5@2001-01-05]', '3 days', '2001-01-01', 'linear');
-- [1@2001-01-01, 4@2001-01-04]
```

```
SELECT asText(tsample(tgeompoint '[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05]',
  '2 days', '2001-01-01'));
-- {POINT(1 1)@2001-01-01, POINT(3 3)@2001-01-03, POINT(5 5)@2001-01-05}
SELECT asText(tsample(tgeompoint '{[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05],
  [Point(1 1)@2001-01-06, Point(5 5)@2001-01-08]}', '3 days', '2001-01-01'));
-- {POINT(1 1)@2001-01-01, POINT(4 4)@2001-01-04, POINT(3 3)@2001-01-07}
SELECT asText(tsample(tgeompoint '{[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05],
  [Point(1 1)@2001-01-06, Point(5 5)@2001-01-08]}', '3 days', '2001-01-01', 'step'));
-- Interp=Step;{[POINT(1 1)@2001-01-01, POINT(4 4)@2001-01-04], [POINT(3 3)@2001-01-07]}
```

Figure 9.2 illustrates the sampling of temporal floats with various interpolations. As shown in the figure, the sampling operation is best suited for temporal values with continuous interpolation.

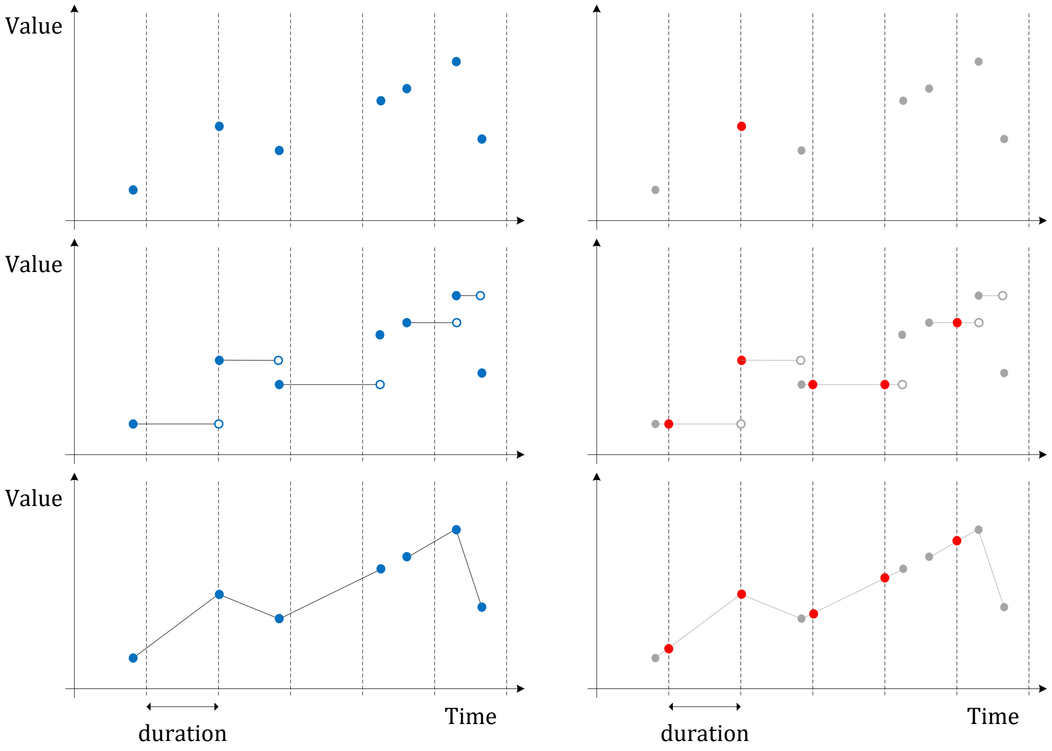

Figure 9.2: Sampling of temporal floats with discrete, step, and linear interpolation.

- Reduce the temporal precision of a temporal value with respect to an interval computing the time-weighted average/centroid in each time bin

```
tprecision({tnumber,tgeompoint,tcbuffer,tpose,tgeo,trgeometry},duration interval,
  torigin timestamptz='2000-01-03') → {tfloat,tpoint,tcbuffer,tpose,tgeo,trgeometry}
```

If the origin is not specified, it is set by default to Monday, January 3, 2000. The given interval must be strictly greater than zero. A temporal number yields a temporal float, since the time-weighted average of the values of a bin need not be an integer, as `twAvg` yields a float.

```
SELECT tprecision(tint '[1@2001-01-01,5@2001-01-05,1@2001-01-09]','1 day', '2001-01-01');
-- Interp=Step;[1@2001-01-01, 5@2001-01-05, 1@2001-01-09]
SELECT tprecision(tint '[1@2001-01-01 00:00, 3@2001-01-01 00:05, 3@2001-01-01 00:10]',
  '10 minutes');
-- Interp=Step;[2@2001-01-01 00:00:00, 3@2001-01-01 00:10:00]
SELECT tprecision(tfloat '[1@2001-01-01,5@2001-01-05,1@2001-01-09)','1 day',
  '2001-01-01');
-- [1.5@2001-01-01, 4.5@2001-01-04, 4.5@2001-01-05, 1.5@2001-01-08]
SELECT tprecision(tfloat '[1@2001-01-01,5@2001-01-05,1@2001-01-09]','1 day',
  '2001-01-01');
-- [1.5@2001-01-01, 4.5@2001-01-04, 4.5@2001-01-05, 1.5@2001-01-08, 1@2001-01-09]
SELECT tprecision(tfloat '[1@2001-01-01,5@2001-01-05,1@2001-01-09)','2 days',
  '2001-01-01');
-- [2@2001-01-01, 4@2001-01-03, 4@2001-01-05, 2@2001-01-07]
```

```
SELECT asText(tprecision(tgeompoint '[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05,
  Point(1 1)@2001-01-09)', '1 day', '2001-01-01'));
/* [POINT(1.5 1.5)@2001-01-01, POINT(4.5 4.5)@2001-01-04, POINT(4.5 4.5)@2001-01-05,
   POINT(1.5 1.5)@2001-01-08] */
SELECT asText(tprecision(tgeompoint '[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05,
  Point(1 1)@2001-01-09)', '2 days', '2001-01-01'));
/* [POINT(2 2)@2001-01-01, POINT(4 4)@2001-01-03, POINT(4 4)@2001-01-05, POINT(2
   2)@2001-01-07] */
SELECT asText(tprecision(tgeompoint '[Point(1 1)@2001-01-01, Point(5 5)@2001-01-05,
  Point(1 1)@2001-01-09)', '4 days', '2001-01-01'));
-- [POINT(3 3)@2001-01-01, POINT(3 3)@2001-01-05]
```

Changing the precision of a temporal value is akin to changing its *temporal granularity*, for example, from timestamps to hours or days, although the precision can be set to an arbitrary interval, such as 2 hours and 15 minutes. Figure 9.3 illustrates a change of temporal precision for temporal floats with various interpolations.

9.3 Similarity

- Return the discrete Hausdorff distance, the discrete Fréchet distance, the Dynamic Time Warp (DTW) distance, the average Hausdorff distance, or the Longest Common SubSequence (LCSS) distance between two temporal values

```
hausdorffDistance({tnumber,tgeo},{tnumber,tgeo}) → float
frechetDistance({tnumber,tgeo},{tnumber,tgeo}) → float
dynTimeWarpDistance({tnumber,tgeo},{tnumber,tgeo}) → float
averageHausdorffDistance({tnumber,tgeo},{tnumber,tgeo}) → float
lcssDistance({tnumber,tgeo},{tnumber,tgeo},float) → float
hausdorffDistance(tgeogpoint,tgeogpoint,spheroid boolean=true) → float
frechetDistance(tgeogpoint,tgeogpoint,spheroid boolean=true) → float
dynTimeWarpDistance(tgeogpoint,tgeogpoint,spheroid boolean=true) → float
averageHausdorffDistance(tgeogpoint,tgeogpoint,spheroid boolean=true) → float
lcssDistance(tgeogpoint,tgeogpoint,float,spheroid boolean=true) → float
```

The `lcssDistance` function requires an additional parameter that specifies the distance threshold under which two values are considered to match. These functions have a linear space complexity since only two rows of the distance matrix are allocated in memory. Nevertheless, their time complexity is quadratic in the number of instants of the temporal values. Therefore, the functions require considerable time for temporal values with large number of instants. For geographies, the computation is done on the WGS 84 spheroid by default. If the last argument is false, a faster spherical calculation is used.

```
SELECT hausdorffDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
 tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
```

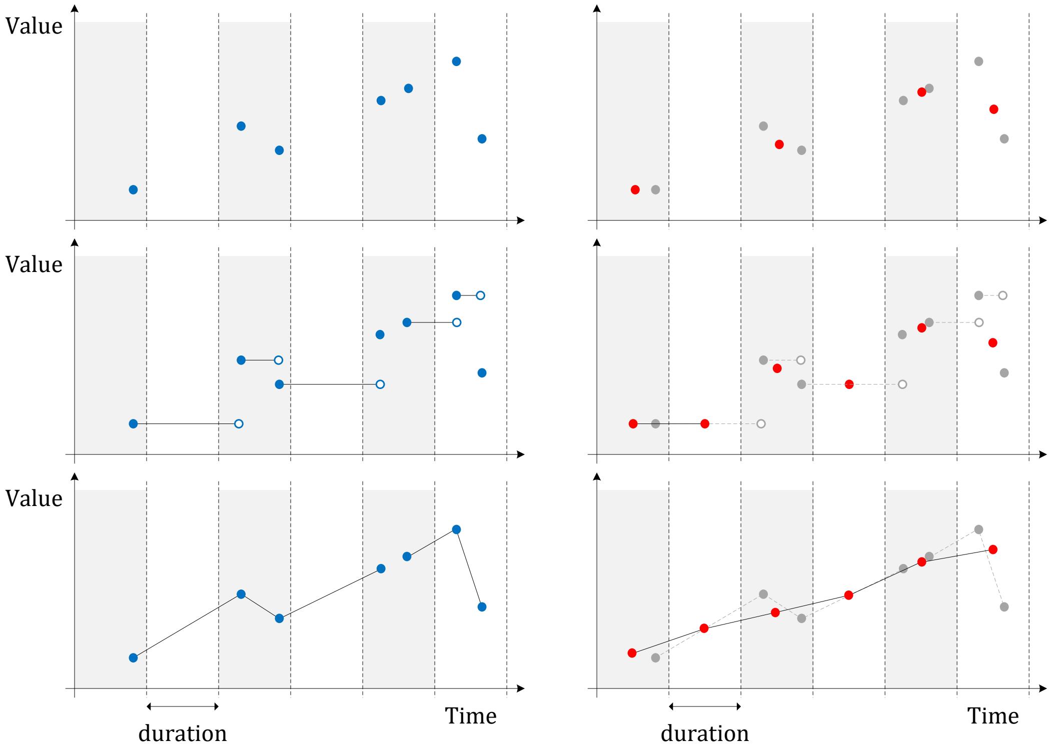

Figure 9.3: Changing the precision of temporal floats with discrete, step, and linear interpolation.

```
-- 0.5
SELECT round(hausdorffDistance(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]',
  tgeompoint '[Point(1.1 1.1)@2001-01-01, Point(2.5 2.5)@2001-01-02,
  Point(4 4)@2001-01-03, Point(3 3)@2001-01-04, Point(1.5 2)@2001-01-05]')::numeric, 6);
-- 1.414214
SELECT round(frechetDistance(tgeogpoint '[Point(0 0)@2001-01-01, Point(0 1)@2001-01-02]',
  tgeogpoint '[Point(0 1)@2001-01-01, Point(0 2)@2001-01-02]', false), 3);
-- 111195.08
```

```
SELECT frechetDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
 tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- 0.5
SELECT round(frechetDistance(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]',
  tgeompoint '[Point(1.1 1.1)@2001-01-01, Point(2.5 2.5)@2001-01-02,
  Point(4 4)@2001-01-03, Point(3 3)@2001-01-04, Point(1.5 2)@2001-01-05]')::numeric, 6);
-- 1.414214
```

```
SELECT dynTimeWarpDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
 tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- 2
SELECT round(dynTimeWarpDistance(tgeompoint '[Point(1 1)@2001-01-01,
  Point(3 3)@2001-01-03, Point(1 1)@2001-01-05]',
  tgeompoint '[Point(1.1 1.1)@2001-01-01, Point(2.5 2.5)@2001-01-02,
  Point(4 4)@2001-01-03, Point(3 3)@2001-01-04, Point(1.5 2)@2001-01-05]')::numeric, 6);
-- 3.380776
```

```
SELECT round(averageHausdorffDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
  tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04,
  1.5@2001-01-05]')::numeric, 6);
-- 0.283333
SELECT round(averageHausdorffDistance(tgeompoint '[Point(1 1)@2001-01-01,
  Point(3 3)@2001-01-03, Point(1 1)@2001-01-05]',
  tgeompoint '[Point(1.1 1.1)@2001-01-01, Point(2.5 2.5)@2001-01-02,
  Point(4 4)@2001-01-03, Point(3 3)@2001-01-04, Point(1.5 2)@2001-01-05]')::numeric, 6);
-- 0.385218
```

```
SELECT lcssDistance(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
  tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]',
  1.0);
-- 3
SELECT round(lcssDistance(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]', tgeompoint '[Point(1.1 1.1)@2001-01-01,
  Point(2.5 2.5)@2001-01-02, Point(4 4)@2001-01-03, Point(3 3)@2001-01-04,
  Point(1.5 2)@2001-01-05]', 1.0)::numeric, 6);
-- 2.000000
```

- Return the correspondence pairs between two temporal values with respect to the discrete Fréchet distance or the Dynamic Time Warp Distance {}

```
frechetDistancePath({tnumber,tgeo},{tnumber,tgeo}) → {(i,j)}
dynTimeWarpPath({tnumber,tgeo},{tnumber,tgeo}) → {(i,j)}
frechetDistancePath(tgeogpoint,tgeogpoint,spheroid boolean=true) → {(i,j)}
dynTimeWarpPath(tgeogpoint,tgeogpoint,spheroid boolean=true) → {(i,j)}
```

The result is a set of pairs `(i,j)`. This function requires to allocate in memory a distance matrix whose size is quadratic in the number of instants of the temporal values. Therefore, the function will fail for temporal values with large number of instants depending on the available memory. For geographies, the computation is done on the WGS 84 spheroid by default. If the last argument is false, a faster spherical calculation is used.

```
SELECT frechetDistancePath(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
 tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- (0,0)
-- (1,0)
-- (2,1)
-- (3,2)
-- (4,2)
SELECT frechetDistancePath(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]',
  tgeompoint '[Point(1.1 1.1)@2001-01-01, Point(2.5 2.5)@2001-01-02,
  Point(4 4)@2001-01-03, Point(3 3)@2001-01-04, Point(1.5 2)@2001-01-05]');
-- (0,0)
-- (1,1)
-- (2,1)
-- (3,1)
-- (4,2)
```

```
SELECT dynTimeWarpPath(tfloat '[1@2001-01-01, 3@2001-01-03, 1@2001-01-06]',
 tfloat '[1@2001-01-01, 1.5@2001-01-02, 2.5@2001-01-03, 1.5@2001-01-04, 1.5@2001-01-05]');
-- (0,0)
-- (1,0)
-- (2,1)
-- (3,2)
-- (4,2)
SELECT dynTimeWarpPath(tgeompoint '[Point(1 1)@2001-01-01, Point(3 3)@2001-01-03,
  Point(1 1)@2001-01-05]',
  tgeompoint '[Point(1.1 1.1)@2001-01-01, Point(2.5 2.5)@2001-01-02,
  Point(4 4)@2001-01-03, Point(3 3)@2001-01-04, Point(1.5 2)@2001-01-05]');
-- (0,0)
-- (1,1)
-- (2,1)
-- (3,1)
-- (4,2)
```

9.4 Extended Kalman Filter

- Return a smoothed temporal float or temporal point obtained by filtering a noisy trajectory with an Extended Kalman Filter

  ```
  extendedKalmanFilter(tfloat,gate float,q float,variance float,
    to_drop boolean=true) → tfloat
  extendedKalmanFilter(tgeompoint,gate float,q float,variance float,
    to_drop boolean=true) → tgeompoint
  ```

  The filter models the underlying motion of a moving object and separates it from measurement noise and occasional outliers. The `gate` parameter controls how strict the outlier detection is, `q` is the process-noise scale, `variance` is the measurement-noise variance, and `to_drop` chooses whether outliers are removed (`true`) or replaced by the predicted trajectory (`false`). These functions are intended for trajectories with at least three instants and are typically applied to temporal values with linear interpolation.

  ```
  SELECT asEWKT(extendedKalmanFilter(tgeompoint '[Point(1 1)@2001-01-01,
    Point(50 50)@2001-01-02, Point(2 2)@2001-01-03]', 1e-5, 5e-10, 5e-10, true));
  -- [POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-03]
  SELECT asEWKT(extendedKalmanFilter(tgeompoint '[Point(1 1)@2001-01-01,
    Point(50 50)@2001-01-02, Point(2 2)@2001-01-03]', 1e-5, 5e-10, 5e-10, false));
  -- [POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02, POINT(2 2)@2001-01-03]
  ```

9.5 Reachability Filter

The reachability filter removes the reports of a moving object that it could not have reached: a report is kept when the last kept report could reach it at a maximum speed, given in units of the values per second, which are meters per second for a geography. The distance is the difference of two floats, the planar distance of two geometry points, or the distance of two geography points on the spheroid. The filter trusts its first report, so the walk starts from a seed, the earliest of the first eleven reports that a majority of them can reach, and walks the reports before the seed backwards from it. A value whose share of reports unreachable from the report before exceeds one half is dropped whole.

- Return a temporal float or temporal point with the instants the reachability filter keeps

  ```
  reachabilityFilter({tfloat,tgeompoint,tgeogpoint},maxspeed float)
    → {tfloat,tgeompoint,tgeogpoint}
  ```

  The kept instants keep their sequences and the interpolation of the value. The function returns NULL when the value is dropped whole.

  ```
  SELECT asText(reachabilityFilter(tgeompoint '[Point(0 0)@2001-01-01 00:00:00,
    Point(10 0)@2001-01-01 00:00:01, Point(5000 0)@2001-01-01 00:00:02,
    Point(30 0)@2001-01-01 00:00:03]', 100));
  -- [POINT(0 0)@2001-01-01 00:00:00, POINT(10 0)@2001-01-01 00:00:01,
     POINT(30 0)@2001-01-01 00:00:03]
  SELECT reachabilityFilter(tgeompoint '[Point(0 0)@2001-01-01 00:00:00,
    Point(5000 0)@2001-01-01 00:00:01, Point(0 0)@2001-01-01 00:00:02,
    Point(5000 0)@2001-01-01 00:00:03]', 100) IS NULL;
  -- true
  ```

- Return a temporal float or temporal point with the instants the reachability filter keeps among the instants of a group

```
reachabilityFilterAgg({tfloat,tgeompoint,tgeogpoint},maxspeed float)
  → {tfloat,tgeompoint,tgeogpoint}
```

The aggregate takes the instants in the order of their times, which the query states with `ORDER BY`, and decides each one as it arrives, as a stream does. It returns a sequence with the default interpolation of the type, and NULL when the instants are dropped whole. Its name differs from that of the function, since PostgreSQL does not hold a function and an aggregate of the same name and arguments.

```
SELECT asText(reachabilityFilterAgg(inst, 5 ORDER BY getTimestamp(inst)))
FROM unnest(instants(tfloat '{1@2001-01-01 00:00:00, 2@2001-01-01 00:00:01,
  1000@2001-01-01 00:00:02, 3@2001-01-01 00:00:03}')) AS inst;
-- [1@2001-01-01 00:00:00, 2@2001-01-01 00:00:01, 3@2001-01-01 00:00:03]
```

9.6 Splitting Operations

When creating indexes for temporal types, what is stored in the index is not the actual value but instead, a bounding box that *represents* the value. In this case, the index will provide a list of candidate values that *may* satisfy the query predicate, and a second step is needed to filter out candidate values by computing the query predicate on the actual values.

However, when the bounding boxes have a large empty space not covered by the actual values, the index will generate many candidate values that do not satisfy the query predicate, which reduces the efficiency of the index. In these situations, it may be better to represent a value not with a *single* bounding box, but instead with *multiple* bounding boxes. This increases considerably the efficiency of the index, provided that the index is able to manage multiple bounding boxes per value. The following functions are used for generating multiple bounding boxes for a single temporal value.

- Return an array of N time spans obtained by merging the instants or segments of a temporal value

```
splitNSpans(ttype,integer) → tstzspan[]
```

The choice between instants or segments depends on whether the interpolation is discrete or continuous. The last argument specifies the number of output spans. If the number of instants or segments is less than or equal to the given number, the resulting array will have one span per instant or segment of the temporal value. Otherwise, the given number of spans will be obtained by merging consecutive instants or segments.

```
SELECT splitNSpans(ttext '{A@2001-01-01, B@2001-01-02, A@2001-01-03, B@2001-01-04,
  A@2001-01-05}', 1);
-- {"[2001-01-01, 2001-01-05]"}
SELECT splitNSpans(tfloat '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 2@2001-01-04,
  1@2001-01-05}', 2);
-- {"[2001-01-01, 2001-01-03]","[2001-01-04, 2001-01-05]"}
SELECT splitNSpans(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03, Point(2 2)@2001-01-04, Point(1 1)@2001-01-05]', 2);
-- {"[2001-01-01, 2001-01-03]","[2001-01-03, 2001-01-05]"}
SELECT splitNSpans(tgeogpoint '{[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02],
  [Point(1 1 1)@2001-01-03, Point(2 2 2)@2001-01-04], [Point(1 1 1)@2001-01-05]}', 2);
-- {"[2001-01-01, 2001-01-04]","[2001-01-05, 2001-01-05]"}
SELECT splitNSpans(tint '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 2@2001-01-04,
  2@2001-01-05}', 6);
/* {"[2001-01-01, 2001-01-01]","[2001-01-02, 2001-01-02]","[2001-01-03,
   2001-01-03]","[2001-01-04, 2001-01-04]","[2001-01-05, 2001-01-05]"} */
SELECT splitNSpans(ttext '[A@2001-01-01, B@2001-01-02, A@2001-01-03, B@2001-01-04,
  A@2001-01-05]', 6);
/* {"[2001-01-01, 2001-01-02]","[2001-01-02, 2001-01-03]","[2001-01-03,
   2001-01-04]","[2001-01-04, 2001-01-05]"} */
```

- Return an array of time spans obtained by merging N consecutive instants or segments of a temporal value

```
splitEachNSpans(ttype,integer) → tstzspan[]
```

The choice between instants or segments depends on whether the interpolation is discrete or continuous. The last argument specifies the number of input instants or segments that are merged to produce an output span. If the number of input elements is less than or equal to the given number, the resulting array will have a single span per sequence. Otherwise, the given number of consecutive instants or segments will be merged into each output span. Notice that, contrary to the splitNSpans function, the number of spans in the result depends on the number of input instants or segments.

```
SELECT splitEachNSpans(ttext '{A@2001-01-01, B@2001-01-02, A@2001-01-03, B@2001-01-04,
  A@2001-01-05}', 1);
/* {"[2001-01-01, 2001-01-01]","[2001-01-02, 2001-01-02]","[2001-01-03,
   2001-01-03]","[2001-01-04, 2001-01-04]","[2001-01-05, 2001-01-05]"} */
SELECT splitEachNSpans(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 2@2001-01-04,
  1@2001-01-05]', 2);
-- {"[2001-01-01, 2001-01-03]","[2001-01-03, 2001-01-05]"}
SELECT splitEachNSpans(tgeompoint '{[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02],
  [Point(1 1 1)@2001-01-03, Point(2 2 2)@2001-01-04], [Point(1 1 1)@2001-01-05]}', 2);
-- {"[2001-01-01, 2001-01-02]","[2001-01-03, 2001-01-04]","[2001-01-05, 2001-01-05]"}
SELECT splitEachNSpans(tgeogpoint '[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02,
  Point(1 1 1)@2001-01-03, Point(2 2 2)@2001-01-04, Point(1 1 1)@2001-01-05]', 6);
-- {"[2001-01-01, 2001-01-05]"}
SELECT splitEachNSpans(tgeogpoint '{[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02],
  [Point(1 1 1)@2001-01-03, Point(2 2 2)@2001-01-04], [Point(1 1 1)@2001-01-05]}', 6);
-- {"[2001-01-01, 2001-01-02]","[2001-01-03, 2001-01-04]","[2001-01-05, 2001-01-05]"}
```

- Return an array of N temporal boxes obtained by merging the instants or segments of a temporal number

```
splitNTboxes(tnumber,integer) → tbox[]
```

The choice between instants or segments depends on whether the interpolation is discrete or continuous. The last argument specifies the number of output boxes. If the number of input instants or segments is less than or equal to the given number, the resulting array will have one box per instant or segment of the temporal number. Otherwise, the specified number of boxes will be obtained by merging consecutive instants or segments.

```
SELECT splitNTboxes(tint '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05}', 1);
-- {"TBOXINT XT([1, 5),[2001-01-01, 2001-01-05])"}
SELECT splitNTboxes(tfloat '{[1@2001-01-01, 2@2001-01-02], [1@2001-01-03, 4@2001-01-04],
  [1@2001-01-05]}', 2);
/* {"TBOXFLOAT XT([1, 4],[2001-01-01, 2001-01-04])","TBOXFLOAT XT([1, 1],[2001-01-05,
   2001-01-05])"} */
SELECT splitNTboxes(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05]', 3);
/* {"TBOXFLOAT XT([1, 2],[2001-01-01, 2001-01-03])","TBOXFLOAT XT([1, 4],[2001-01-03,
   2001-01-04])","TBOXFLOAT XT([1, 4],[2001-01-04, 2001-01-05])"} */
SELECT splitNTboxes(tint '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05}', 6);
/* {"TBOXINT XT([1, 2),[2001-01-01, 2001-01-01])","TBOXINT XT([2, 3),[2001-01-02,
   2001-01-02])","TBOXINT XT([1, 2),[2001-01-03, 2001-01-03])","TBOXINT XT([4,
   5),[2001-01-04, 2001-01-04])","TBOXINT XT([1, 2),[2001-01-05, 2001-01-05])"} */
SELECT splitNTboxes(tint '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05]', 6);
/* {"TBOXINT XT([1, 3),[2001-01-01, 2001-01-02])","TBOXINT XT([1, 3),[2001-01-02,
   2001-01-03])","TBOXINT XT([1, 5),[2001-01-03, 2001-01-04])","TBOXINT XT([1,
   5),[2001-01-04, 2001-01-05])"} */
```

- Return an array of temporal boxes obtained by merging N consecutive instants or segments of a temporal number

```
splitEachNTboxes(tnumber,integer) → tbox[]
```

The choice between instants or segments depends on whether the interpolation is discrete or continuous. The last argument specifies the number of input instants or segments that are merged to produce an output box. If the number of input instants or segments is less than or equal to the given number, the resulting array will have a single box per sequence. Otherwise, the specified number of consecutive instants or segments will be merged into each output box. Notice that, contrary to the splitNTboxes function, the number of boxes in the result depends on the number of input instants or segments.

```
SELECT splitEachNTboxes(tint '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05}', 1);
/* {"TBOXINT XT([1, 2),[2001-01-01, 2001-01-01])","TBOXINT XT([2, 3),[2001-01-02,
   2001-01-02])","TBOXINT XT([1, 2),[2001-01-03, 2001-01-03])","TBOXINT XT([4,
   5),[2001-01-04, 2001-01-04])","TBOXINT XT([1, 2),[2001-01-05, 2001-01-05])"} */
SELECT splitEachNTboxes(tint '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05]', 1);
/* {"TBOXINT XT([1, 3),[2001-01-01, 2001-01-02])","TBOXINT XT([1, 3),[2001-01-02,
   2001-01-03])","TBOXINT XT([1, 5),[2001-01-03, 2001-01-04])","TBOXINT XT([1,
   5),[2001-01-04, 2001-01-05])"} */
SELECT splitEachNTboxes(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05]', 3);
/* {"TBOXFLOAT XT([1, 4],[2001-01-01, 2001-01-04])","TBOXFLOAT XT([1, 4],[2001-01-04,
   2001-01-05])"} */
SELECT splitEachNTboxes(tfloat '{[1@2001-01-01, 2@2001-01-02],
  [1@2001-01-03, 4@2001-01-04], [1@2001-01-05]}', 6);
/* {"TBOXFLOAT XT([1, 2],[2001-01-01, 2001-01-02])","TBOXFLOAT XT([1, 4],[2001-01-03,
   2001-01-04])","TBOXFLOAT XT([1, 1],[2001-01-05, 2001-01-05])"} */
```

- Return either an array of N spatial boxes obtained by merging the segments of a (multi)line or an array of N spatiotemporal boxes obtained by merging the instants or segments of a temporal point

```
splitNStboxes({lines,tpoint},integer) → stbox[]
```

For temporal points, the choice between instants or segments depends on whether the interpolation is discrete or continuous. The last argument specifies the number of output boxes. If the number of instants or segments is less than or equal to the given number, the resulting array will have either one box per segment of the (multi)line or one box per instant or segment of the temporal point. Otherwise, the specified number of boxes will be obtained by merging consecutive instants or segments.

```
SELECT splitNStboxes(geometry 'Linestring(1 1,2 2,3 1,4 2,5 1)', 1);
-- {"STBOX X((1,1),(5,2))"}
SELECT splitNStboxes(geometry 'Linestring(1 1,2 2,3 1,4 2,5 1)', 2);
-- {"STBOX X((1,1),(3,2))","STBOX X((3,1),(5,2))"}
SELECT splitNStboxes(geography 'Linestring(1 1 1,2 2 1,3 1 1,4 2 1,5 1 1)', 6);
/* {"SRID=4326;GEODSTBOX Z((1,1,1),(2,2,1))","SRID=4326;GEODSTBOX
   Z((2,1,1),(3,2,1))","SRID=4326;GEODSTBOX Z((3,1,1),(4,2,1))","SRID=4326;GEODSTBOX
   Z((4,1,1),(5,2,1))"} */
SELECT splitNStboxes(geometry 'MultiLinestring((1 1,2 2),(3 1,4 2),(5 1,6 2))', 2);
-- {"STBOX X((1,1),(4,2))","STBOX X((5,1),(6,2))"}
```

```
SELECT splitNStboxes(tgeompoint '{Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(3 1)@2001-01-03, Point(4 2)@2001-01-04, Point(5 1)@2001-01-05}', 1);
-- {"STBOX XT(((1,1),(5,2)),[2001-01-01, 2001-01-05])"}
SELECT splitNStboxes(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(3 1)@2001-01-03, Point(4 2)@2001-01-04, Point(5 1)@2001-01-05]', 4);
/* {"STBOX XT(((1,1),(2,2)),[2001-01-01, 2001-01-02])",
   "STBOX XT(((2,1),(3,2)),[2001-01-02, 2001-01-03])",
   "STBOX XT(((3,1),(4,2)),[2001-01-03, 2001-01-04])",
   "STBOX XT(((4,1),(5,2)),[2001-01-04, 2001-01-05])"} */
SELECT splitNStboxes(tgeompoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
  [Point(3 1)@2001-01-03, Point(4 2)@2001-01-04], [Point(5 1)@2001-01-05]}', 2);
/* {"STBOX XT(((1,1),(4,2)),[2001-01-01, 2001-01-04])","STBOX
   XT(((5,1),(5,1)),[2001-01-05, 2001-01-05])"} */
SELECT splitNStboxes(tgeogpoint '{Point(1 1 1)@2001-01-01, Point(2 2 1)@2001-01-02,
  Point(3 1 1)@2001-01-03, Point(4 2 1)@2001-01-04, Point(5 1 1)@2001-01-05}', 6);
```

```
/* {"SRID=4326;GEODSTBOX ZT(((1,1,1),(1,1,1)),[2001-01-01,
   2001-01-01])","SRID=4326;GEODSTBOX ZT(((2,2,1),(2,2,1)),[2001-01-02,
   2001-01-02])","SRID=4326;GEODSTBOX ZT(((3,1,1),(3,1,1)),[2001-01-03,
   2001-01-03])","SRID=4326;GEODSTBOX ZT(((4,2,1),(4,2,1)),[2001-01-04,
   2001-01-04])","SRID=4326;GEODSTBOX ZT(((5,1,1),(5,1,1)),[2001-01-05, 2001-01-05])"} */
SELECT splitNStboxes(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(3 1)@2001-01-03, Point(4 2)@2001-01-04, Point(5 1)@2001-01-05]', 6);
/* {"STBOX XT(((1,1),(2,2)),[2001-01-01, 2001-01-02])","STBOX
   XT(((2,1),(3,2)),[2001-01-02, 2001-01-03])","STBOX XT(((3,1),(4,2)),[2001-01-03,
   2001-01-04])","STBOX XT(((4,1),(5,2)),[2001-01-04, 2001-01-05])"} */
```

- Return either an array of spatial boxes obtained by merging N consecutive segments of a (multi)line or an array of spatiotemporal boxes obtained by merging N consecutive instants or segments of a temporal point

```
splitEachNStboxes({lines,tpoint},integer) → stbox[]
```

For temporal points, the choice between instants or segments depends on whether the interpolation is discrete or continuous. The last argument specifies the number of input instants or segments that are merged to produce an output box. If the number of instants or segments is less than or equal to the given number, the resulting array will have a single box per sequence. Otherwise, the specified number of consecutive instants or segments will be merged into each output box. Notice that, contrary to the splitNStboxes function, the number of boxes in the result depends on the number of input instants or segments.

```
SELECT splitEachNStboxes(geometry 'Linestring(1 1,2 2,3 1,4 2,5 1)', 1);
/* {"STBOX X((1,1),(2,2))","STBOX X((2,1),(3,2))","STBOX X((3,1),(4,2))","STBOX
   X((4,1),(5,2))"} */
SELECT splitEachNStboxes(geometry 'Linestring(1 1,2 2,3 1,4 2,5 1)', 2);
-- {"STBOX X((1,1),(3,2))","STBOX X((3,1),(5,2))"}
SELECT splitEachNStboxes(geography 'Linestring(1 1 1,2 2 1,3 1 1,4 2 1,5 1 1)', 6);
-- {"SRID=4326;GEODSTBOX Z((1,1,1),(5,2,1))"}
SELECT splitEachNStboxes(geometry 'MultiLinestring((1 1,2 2),(3 1,4 2),(5 1,6 2))', 2);
-- {"STBOX X((1,1),(2,2))","STBOX X((3,1),(4,2))","STBOX X((5,1),(6,2))"}
```

```
SELECT splitEachNStboxes(tgeompoint '{Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(3 1)@2001-01-03, Point(4 2)@2001-01-04, Point(5 1)@2001-01-05}', 1);
/* {"STBOX XT(((1,1),(1,1)),[2001-01-01, 2001-01-01])","STBOX
   XT(((2,2),(2,2)),[2001-01-02, 2001-01-02])","STBOX XT(((3,1),(3,1)),[2001-01-03,
   2001-01-03])","STBOX XT(((4,2),(4,2)),[2001-01-04, 2001-01-04])","STBOX
   XT(((5,1),(5,1)),[2001-01-05, 2001-01-05])"} */
SELECT splitEachNStboxes(tgeompoint '[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(3 1)@2001-01-03, Point(4 2)@2001-01-04, Point(5 1)@2001-01-05]', 1);
/* {"STBOX XT(((1,1),(2,2)),[2001-01-01, 2001-01-02])","STBOX
   XT(((2,1),(3,2)),[2001-01-02, 2001-01-03])","STBOX XT(((3,1),(4,2)),[2001-01-03,
   2001-01-04])","STBOX XT(((4,1),(5,2)),[2001-01-04, 2001-01-05])"} */
SELECT splitEachNStboxes(tgeogpoint '{[Point(1 1)@2001-01-01, Point(2 2)@2001-01-02],
  [Point(3 1)@2001-01-03, Point(4 2)@2001-01-04], [Point(5 1)@2001-01-05]}', 2);
/* {"SRID=4326;GEODSTBOX XT(((1,1),(2,2)),[2001-01-01,
   2001-01-02])","SRID=4326;GEODSTBOX XT(((3,1),(4,2)),[2001-01-03,
   2001-01-04])","SRID=4326;GEODSTBOX XT(((5,1),(5,1)),[2001-01-05, 2001-01-05])"} */
```

9.7 Multidimensional Tiling

Multidimensional tiling is the mechanism used to partition the domain of temporal values in bins or tiles of varying number of dimensions. In the case of a single dimension, the domain can be partitioned by value or by time using bins of the same size or duration, respectively. For temporal numbers, the domain can be partitioned in two-dimensional tiles of the same size for the value dimension and the same duration for the time dimension. For temporal points, the domain can be partitioned in space in two- or three-dimensional tiles, depending on the number of dimensions of the spatial coordinates. Finally, for temporal points, the domain can be partitioned in space and time using three- or four-dimensional tiles.

Multidimensional tiling can be used for various purposes. It can be used for computing multidimensional histograms, where the temporal values are aggregated according to the underlying partition of the domain. On the other hand, multidimensional tiling can also be used for indexing purposes, where the bounding box of a temporal value can be fragmented into multiple boxes in order to improve the efficiency of the index. Finally, multidimensional tiling can be used for fragmenting temporal values according to a multidimensional grid defined over the underlying domain. This enables the distribution of a dataset across a cluster of servers, where each server contains a partition of the dataset. The advantage of this partition mechanism is that it preserves proximitity in value/space and time, unlike the traditional hash-based partition mechanisms.

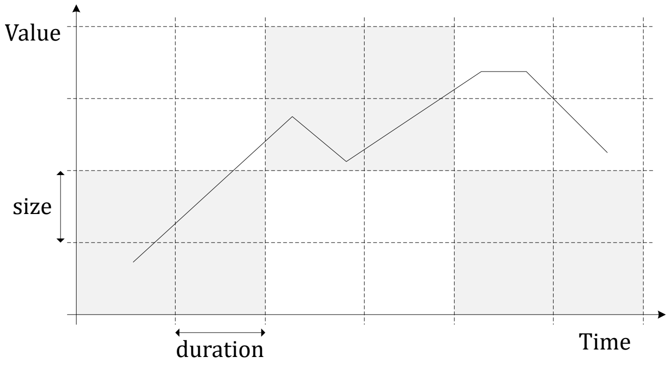

Figure 9.4: Multidimensional tiling for temporal floats.

Figure 9.4 illustrates multidimensional tiling for temporal floats. The two-dimensional domain is split into tiles having the same size for the value dimension and the same duration for the time dimension. Suppose that this tiling scheme is used for distribute a dataset across a cluster of six servers, as suggested by the gray pattern in the figure. In this case, the values are fragmented so each server will receive the data of contiguous tiles. This implies in particular that four nodes will receive one fragment of the temporal float depicted in the figure. One advantage of this distribution of data based on multidimensional tiling is that it reduces the data that needs to be exchanged between nodes when processing queries, a process typically referred to as *reshuffling*.

Many of the functions in this section are *set-returning functions* (also known as a *table functions*) since they typically return more than one value. In this case, the functions are marked with the {} symbol.

Every bin, tile and box contains its lower bound and excludes its upper one, so a position on the boundary of two of them belongs to the one starting there. The extent of a value or of a box contains its upper bound, so when that bound lies on a boundary of the grid, the bin or tile starting there holds nothing of the extent but its upper border. The optional argument `borderInc`, true by default, states whether that bin or tile is part of the grid. When it is true, the border has its own bin, tile, box or fragment, so the fragments of a split cover the whole value; when it is false, the border is left out, as for an extent read without its upper bound. A value whose extent excludes its upper bound has no border to leave out and answers alike. The grids of H3 and S2 cells, which have no axis to align with the extent, assign a position on a cell boundary to one of the cells meeting there, and the cell a trajectory enters at its last instant is the one cell holding only the upper border of its extent: `borderInc` states whether that cell is part of the result.

9.7.1 Bin Operations

- Return an array of bins that cover a value or time span with bins of the same size or duration

```
bins(numspans,vsize number,vorigin number=0,borderInc boolean=true) → span[]
bins(datespans,duration interval,torigin date='2000-01-01',
  borderInc boolean=true) → span[]
bins(tstzspans,duration interval,torigin timestamptz='2000-01-03',
  borderInc boolean=true) → span[]
```

If the origin is not specified, it is set by default to 0 for value spans, to Saturday, January 1, 2000 for date spans and to Monday, January 3, 2000 for timestamp spans. The optional argument `borderInc` states whether the upper border of the span is included and thus, the bin starting at that border and holding only it is generated.

```
SELECT bins(floatspan '[-10, -1]', 2.5, -7);
```

```
-- {"[-12, -9.5)","[-9.5, -7)","[-7, -4.5)","[-4.5, -2)","[-2, 0.5)"}
SELECT bins(intspan '[15, 25]', 2);
-- {"[14, 16)","[16, 18)","[18, 20)","[20, 22)","[22, 24)","[24, 26)"}
SELECT index, span FROM unnest(bins(datespan '[2001-01-15, 2001-01-25]', '2 days'))
  WITH ORDINALITY t(span, index);
-- 1 | [2001-01-15, 2001-01-17)
-- 2 | [2001-01-17, 2001-01-19)
-- 3 | [2001-01-19, 2001-01-21)
-- ...
SELECT index, span
FROM unnest(bins(tstzspanset '{[2001-01-15, 2001-01-17],[2001-01-22, 2001-01-25]}',
  '2 days', '2001-01-02')) WITH ORDINALITY t(span, index);
-- 1 | [2001-01-15, 2001-01-16)
-- 2 | [2001-01-16, 2001-01-17]
-- 3 | [2001-01-22, 2001-01-24)
-- 4 | [2001-01-24, 2001-01-25]
```

- Return the bin that contains a number or a timestamp

```
getBin(number,vsize number,vorigin number=0) → span
getBin(date,duration interval,torigin date='2000-01-03') → datespan
getBin(timestamptz,duration interval,torigin timestamptz='2000-01-03') → tstzspan
```

If the origin is not specified, it is set by default to 0 for value bins and to Monday, January 3, 2000 for time bins.

```
SELECT getBin(2, 2);
-- [2, 4)
SELECT getBin(2, 2.5, 1.5);
-- [1.5, 4)
SELECT getBin('2001-01-04', interval '1 week');
-- [2001-01-01 08:00:00, 2001-01-08 08:00:00)
SELECT getBin('2001-01-04', interval '1 week', '2001-01-07');
-- [2000-12-31, 2001-01-07)
```

9.7.2 Tile Operations

- Return the set of tiles that covers a temporal box with tiles of the same size and/or duration {}

```
valueTiles(tbox,vsize number,vorigin number=0,borderInc boolean=true) → {(index,tile)}
timeTiles(tbox,duration interval,torigin timestamptz='2000-01-03',
  borderInc boolean=true) → {(index,tile)}
valueTimeTiles(tbox,vsize number,duration interval,vorigin number=0,
  torigin timestamptz='2000-01-03',borderInc boolean=true) → {(index,tile)}
```

The result is a set of pairs `(index,tile)`. If the origin of the value and/or time dimension is not specified, it is set by default to 0 and to Monday, January 3, 2000, respectively. The value size and the duration of a value and time grid are strictly positive, otherwise an error is raised. The value size and origin have the base type of the values of the box, and a box of floats also takes integer ones. The time tiles of a box with a value dimension keep the value span of the box, with its bounds. The optional argument `borderInc` states whether the upper border of the box is included and thus, extra tiles containing the border are generated.

```
SELECT (gr).index, (gr).tile
FROM (SELECT valueTiles(tfloat '[15@2001-01-15, 25@2001-01-25]'::tbox, 2.0) AS gr) t;
-- 1 | TBOXFLOAT X([14, 16))
-- 2 | TBOXFLOAT X([16, 18))
-- 3 | TBOXFLOAT X([18, 20))
-- ...
SELECT (gr).index, (gr).tile
FROM (SELECT timeTiles(tfloat '[15@2001-01-15, 25@2001-01-25]'::tbox, '2 days') AS gr) t;
-- 1 | TBOXFLOAT XT([15, 25],[2001-01-15, 2001-01-17))
-- 2 | TBOXFLOAT XT([15, 25],[2001-01-17, 2001-01-19))
-- 3 | TBOXFLOAT XT([15, 25],[2001-01-19, 2001-01-21))
-- ...
SELECT (gr).index,
  (gr).tile FROM (SELECT valueTimeTiles(tfloat '[15@2001-01-15, 25@2001-01-25]'::tbox,
  2.0, '2 days') AS gr) t;
-- 1 | TBOX XT([14,16),[2001-01-15,2001-01-17))
-- 2 | TBOX XT([16,18),[2001-01-15,2001-01-17))
-- 3 | TBOX XT([18,20),[2001-01-15,2001-01-17))
-- ...
SELECT valueTiles(tbox 'TBOXINT XT([15, 25],[2001-01-15, 2001-01-25])', 2, 15);
-- (1,"TBOXINT X([15, 17))")
-- (2,"TBOXINT X([17, 19))")
-- (3,"TBOXINT X([19, 21))")
-- ...
SELECT valueTimeTiles(tfloat '[15@2001-01-15,25@2001-01-25]'::tbox, 2.0, '2 days', 11.5);
-- (1,"TBOX XT([13.5,15.5),[2001-01-15,2001-01-17))")
-- (2,"TBOX XT([15.5,17.5),[2001-01-15,2001-01-17))")
-- (3,"TBOX XT([17.5,19.5),[2001-01-15,2001-01-17))")
-- ...
```

- Return the set of tiles that covers a spatiotemporal box with tiles of the same size and/or duration {}

```
spaceTiles({stbox,tgeompoint},xsize float,[ysize float,zsize float,]
  sorigin geompoint='Point(0 0 0)',borderInc boolean=true) → {(index,tile)}
timeTiles({stbox,tgeompoint},duration interval,torigin timestamptz='2000-01-03',
  borderInc boolean=true) → {(index,tile)}
spaceTimeTiles({stbox,tgeompoint},xsize float,[ysize float,zsize float,]duration interval,
  sorigin geompoint='Point(0 0 0)',torigin timestamptz='2000-01-03',
  borderInc boolean=true) → {(index,tile)}
```

The result is a set of pairs `(index,tile)`. If the origin of the space and/or time dimension is not specified, it is set by default to `'Point(0 0 0)'` and to Monday, January 3, 2000, respectively. The optional argument `borderInc` states whether the upper border of the extent is included and thus, extra tiles containing the border are generated. When the first argument is a `tgeompoint`, the bounding spatiotemporal box is derived implicitly via `stbox(temp)`. In the case of a spatiotemporal grid, `ysize` and `zsize` are optional. A size left out, or given as 0, takes the value of `xsize`, and `zsize` is read only for a value with Z dimension. Every size is a finite number that is not negative, otherwise an error is raised. The size `xsize` and the duration of a spatiotemporal grid are strictly positive, otherwise an error is raised. The SRID of the tile coordinates is determined by the input box and the sizes are given in the units of the SRID. If the origin for the spatial coordinates is given, which must be a point, its dimensionality and SRID should be equal to the one of box, otherwise an error is raised.

```
SELECT spaceTiles(tgeompoint '[Point(3 3)@2001-01-15, Point(15 15)@2001-01-25]', 2.0);
-- (1,"STBOX X((2,2),(4,4))")
-- (2,"STBOX X((4,2),(6,4))")
-- (3,"STBOX X((6,2),(8,4))")
-- ...
SELECT timeTiles(tgeompoint '[Point(3 3)@2001-01-15, Point(15 15)@2001-01-25]', '2 days');
-- (1,"STBOX T([2001-01-15, 2001-01-17))")
-- (2,"STBOX T([2001-01-17, 2001-01-19))")
-- (3,"STBOX T([2001-01-19, 2001-01-21))")
-- ...
SELECT spaceTiles(tgeompoint 'SRID=3812;
  [Point(3 3)@2001-01-15, Point(15 15)@2001-01-25]'::stbox, 2.0, geometry 'Point(3 3)');
-- (1,"SRID=3812;STBOX X((3,3),(5,5))")
-- (2,"SRID=3812;STBOX X((5,3),(7,5))")
-- (3,"SRID=3812;STBOX X((7,3),(9,5))")
-- ...
SELECT spaceTiles(tgeompoint '[Point(3 3 3)@2001-01-15,
  Point(15 15 15)@2001-01-25]'::stbox, 2.0, geometry 'Point(3 3 3)');
-- (1,"STBOX Z((3,3,3),(5,5,5))")
-- (2,"STBOX Z((5,3,3),(7,5,5))")
-- (3,"STBOX Z((7,3,3),(9,5,5))")
-- ...
SELECT spaceTimeTiles(tgeompoint '[Point(3 3)@2001-01-15, Point(15 15)@2001-01-25]', 2.0,
  interval '2 days');
-- (1,"STBOX XT(((2,2),(4,4)),[2001-01-15,2001-01-17))")
-- (2,"STBOX XT(((4,2),(6,4)),[2001-01-15,2001-01-17))")
-- (3,"STBOX XT(((6,2),(8,4)),[2001-01-15,2001-01-17))")
-- ...
SELECT spaceTimeTiles(tgeompoint '[Point(3 3 3)@2001-01-15,
  Point(15 15 15)@2001-01-25]'::stbox, 2.0, interval '2 days', 'Point(3 3 3)',
  '2001-01-15');
-- (1,"STBOX ZT(((3,3,3),(5,5,5)),[2001-01-15,2001-01-17))")
-- (2,"STBOX ZT(((5,3,3),(7,5,5))),[2001-01-15,2001-01-17)")
-- (3,"STBOX ZT(((7,3,3),(9,5,5))),[2001-01-15,2001-01-17)")
-- ...
```

- Return the temporal tile that covers a value and/or a timestamp

```
getValueTile(v float,vsize float,vorigin float=0.0) → tbox
getTboxTimeTile(time timestamptz,duration interval,
  torigin timestamptz='2000-01-03') → tbox
getValueTimeTile(v float,t timestamptz,vsize float,duration interval,
  vorigin float=0.0,torigin timestamptz='2000-01-03') → tbox
```

If the origin of the value and/or time dimension is not specified, it is set by default to 0 and to Monday, January 3, 2000, respectively.

```
SELECT getValueTile(15, 2);
-- TBOXFLOAT X([14, 16))
SELECT getTboxTimeTile('2001-01-15', interval '2 days');
-- TBOX T([2001-01-13 08:00:00, 2001-01-15 08:00:00))
SELECT getValueTimeTile(15, '2001-01-15', 2, interval '2 days');
-- TBOXFLOAT XT([14, 16),[2001-01-13 08:00:00, 2001-01-15 08:00:00))
SELECT getValueTimeTile(15, '2001-01-15', 2, interval '2 days', 1, '2001-01-02');
-- TBOXFLOAT XT([15, 17),[2001-01-14, 2001-01-16))
```

- Return the spatiotemporal tile that covers a point and/or a timestamp

```
getSpaceTile(point geometry,xsize float,[ysize float,zsize float],
  sorigin geompoint='Point(0 0 0)') → stbox
getStboxTimeTile(time timestamptz,duration interval,
  torigin timestamptz='2000-01-03') → stbox
getSpaceTimeTile(point geometry,time timestamptz,xsize float,
  [ysize float,zsize float,]duration interval,sorigin geompoint='Point(0 0 0)',
  torigin timestamptz='2000-01-03') → stbox
```

If the origin of the space and/or time dimension is not specified, it is set by default to `Point(0 0 0)` and to Monday, January 3, 2000, respectively. In the case of a spatiotemporal grid, `ysize` and `zsize` are optional. A size left out, or given as 0, takes the value of `xsize`, and `zsize` is read only for a value with Z dimension. Every size is a finite number that is not negative, otherwise an error is raised. The size `xsize` and the duration of a spatiotemporal grid are strictly positive, otherwise an error is raised. The SRID of the tile coordinates is determined by the input point and the sizes are given in the units of the SRID. If the origin for the spatial coordinates is given, which must be a point, its dimensionality and SRID should be equal to the one of box, otherwise an error is raised.

```
SELECT getSpaceTile(geometry 'Point(1 1 1)', 2.0);
-- STBOX Z((0,0,0),(2,2,2))
SELECT getStboxTimeTile(timestamptz '2001-01-01', interval '2 days');
-- STBOX T([2000-12-30 08:00:00, 2001-01-01 08:00:00))
SELECT getSpaceTimeTile(geometry 'Point(1 1)', '2001-01-01', 2.0, interval '2 days');
-- STBOX XT(((0,0),(2,2)),[2000-12-30 08:00:00, 2001-01-01 08:00:00))
SELECT getSpaceTimeTile(geometry 'Point(1 1)', '2001-01-01', 2.0, interval '2 days',
  'Point(1 1)', '2001-01-02');
-- STBOX XT(((1,1),(3,3)),[2000-12-31, 2001-01-02))
```

9.7.3 Bounding Box Operations

These functions fragment the bounding box of a temporal value with respect to a multidimensional grid. They provide an alternative to the functions in Section 9.6 to split the bounding boxes by specifying the maximum size of the boxes in the various dimensions.

- Return an array of value or time spans obtained from the instants or segments of a temporal number with respect to a value or time grid

```
valueBins(tnumber,vsize number,vorigin number=0,borderInc boolean=true) → numspan[]
timeBins(ttype,duration interval,torigin timestamptz='2000-01-03',
  borderInc boolean=true) → tstzspan[]
```

  The choice between instants or segments depends on whether the interpolation is discrete or continuous. If the origin of values or time is not specified, it is set by default to 0 for value spans or to Monday, January 3, 2000 for time spans. The optional argument `borderInc` states whether the upper border of the extent is included and thus, the span starting at that border and holding only it is generated.

```
SELECT valueBins(tint '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05}', 3);
-- {"[0, 3)","[3, 6)"}
SELECT valueBins(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05]', 3, 1);
-- {"[1, 4)","[4, 7)"}
```

```
SELECT timeBins(ttext '{AAA@2001-01-01, BBB@2001-01-02, AAA@2001-01-03, CCC@2001-01-04,
  AAA@2001-01-05}', '3 days');
/* {"[1999-12-31 08:00:00, 2001-01-03 08:00:00)","[2001-01-03 08:00:00, 2001-01-06
   08:00:00)"} */
SELECT timeBins(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05]', '3 days', '2001-01-01');
-- {"[2001-01-01, 2001-01-04)","[2001-01-04, 2001-01-07)"}
```

- Return an array of temporal boxes obtained from the instants or segments of a temporal number with respect to a value and/or time grid

```
valueBoxes(tnumber,vsize number,vorigin number=0,borderInc boolean=true) → tbox[]
timeBoxes(tnumber,duration interval,torigin timestamptz='2000-01-03',
  borderInc boolean=true) → tbox[]
valueTimeBoxes(tnumber,vsize number,duration interval,vorigin number=0,
  torigin timestamptz='2000-01-03',borderInc boolean=true) → tbox[]
```

  The choice between instants or segments depends on whether the interpolation is discrete or continuous. If the origin of value and/or time dimension is not specified, it is set by default to 0 and to Monday, January 3, 2000, respectively. The value size and the duration of a value and time grid are strictly positive, otherwise an error is raised. The optional argument `borderInc` states whether the upper border of the extent is included and thus, the boxes starting at that border and holding only it are generated.

```
SELECT valueBoxes(tint '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05}', 3);
/* {"TBOXINT XT([1, 3),[2001-01-01, 2001-01-05])","TBOXINT XT([4, 5),[2001-01-04,
   2001-01-04])"} */
SELECT timeBoxes(tint '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05}', '3 days');
/* {"TBOXINT XT([1, 3),[2001-01-01, 2001-01-03])","TBOXINT XT([1, 5),[2001-01-04,
   2001-01-05])"} */
```

```
SELECT valueTimeBoxes(tint '{1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05}', 3, '3 days');
/* {"TBOXINT XT([1, 3),[2001-01-01, 2001-01-03])","TBOXINT XT([1, 2),[2001-01-05,
   2001-01-05])","TBOXINT XT([4, 5),[2001-01-04, 2001-01-04])"} */
SELECT valueTimeBoxes(tfloat '[1@2001-01-01, 2@2001-01-02, 1@2001-01-03, 4@2001-01-04,
  1@2001-01-05]', 3, '3 days', 1, '2001-01-01');
/* {"TBOXFLOAT XT([1, 4),[2001-01-01, 2001-01-04))","TBOXFLOAT XT([1, 4),(2001-01-04,
   2001-01-05])","TBOXFLOAT XT([4, 4],[2001-01-04, 2001-01-04])"} */
```

- Return an array of spatiotemporal boxes obtained from the instants or segments of a temporal point with respect to a space and/or time grid

```
spaceBoxes({tgeompoint,tgeometry,tpose,tpcpoint,trgeometry},xsize float,
  [ysize float,zsize float,]
  sorigin geompoint='Point(0 0 0)',borderInc boolean=true) → stbox[]
timeBoxes({tgeompoint,tgeometry,tpose,tpcpoint,trgeometry},duration interval,
  torigin timestamptz='2000-01-03',
  borderInc boolean=true) → stbox[]
spaceTimeBoxes({tgeompoint,tgeometry,tpose,tpcpoint,trgeometry},xsize float,
  [ysize float,zsize float,]duration interval,
  sorigin geompoint='Point(0 0 0)',torigin timestamptz='2000-01-03',
  borderInc boolean=true) → stbox[]
```

The choice between instants or segments depends on whether the interpolation is discrete or continuous. The arguments `ysize` and `zsize` are optional. A size left out, or given as 0, takes the value of `xsize`, and `zsize` is read only for a value with Z dimension. Every size is a finite number that is not negative, otherwise an error is raised. The size `xsize` and the duration of a spatiotemporal grid are strictly positive, otherwise an error is raised. The SRID of the tile coordinates is determined by the temporal point and the sizes are given in the units of the SRID. If the origin for the spatial coordinates is given, which must be a point, its dimensionality and SRID should be equal to the one of temporal point, otherwise an error is raised. If the origin of space and/or time dimension is not specified, it is set by default to `'Point(0 0 0)'` and to Monday, January 3, 2000, respectively. The optional argument `borderInc` states whether the upper border of the extent is included and thus, extra tiles containing the border are generated. A temporal pose and a temporal point cloud point answer these at the temporal geometry point their position resolves to, so a value carrying linear interpolation is tiled along its trajectory rather than at its instants alone.

```
SELECT spaceBoxes(tgeompoint '{Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03, Point(4 4)@2001-01-04, Point(1 1)@2001-01-05}', 3);
/* {"STBOX XT(((1,1),(2,2)),[2001-01-01, 2001-01-05])","STBOX
   XT(((4,4),(4,4)),[2001-01-04, 2001-01-04])"} */
SELECT timeBoxes(tgeompoint '{Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02,
  Point(1 1 1)@2001-01-03, Point(4 4 4)@2001-01-04, Point(1 1 1)@2001-01-05}',
  interval '2 days', '2001-01-01');
/* {"STBOX ZT(((1,1,1),(2,2,2)),[2001-01-01, 2001-01-02])","STBOX
   ZT(((1,1,1),(4,4,4)),[2001-01-03, 2001-01-04])","STBOX
   ZT(((1,1,1),(1,1,1)),[2001-01-05, 2001-01-05])"} */
SELECT spaceTimeBoxes(tgeompoint '{Point(1 1)@2001-01-01, Point(2 2)@2001-01-02,
  Point(1 1)@2001-01-03, Point(4 4)@2001-01-04, Point(1 1)@2001-01-05}', 3, '3 days');
/* {"STBOX XT(((1,1),(2,2)),[2001-01-01, 2001-01-03])","STBOX
   XT(((1,1),(1,1)),[2001-01-05, 2001-01-05])","STBOX XT(((4,4),(4,4)),[2001-01-04,
   2001-01-04])"} */
SELECT spaceTimeBoxes(tgeompoint '[Point(1 1 1)@2001-01-01, Point(2 2 2)@2001-01-02,
  Point(1 1 1)@2001-01-03, Point(4 4 4)@2001-01-04, Point(1 1 1)@2001-01-05]', 3,
  interval '3 days', 'Point(1 1 1)', '2001-01-01');
/* {"STBOX ZT(((1,1,1),(4,4,4)),[2001-01-01, 2001-01-04))","STBOX
   ZT(((1,1,1),(4,4,4)),(2001-01-04, 2001-01-05])","STBOX
   ZT(((4,4,4),(4,4,4)),[2001-01-04, 2001-01-04])"} */
SELECT spaceBoxes(tpose '[Pose(Point(1 1),0.1)@2001-01-01,
  Pose(Point(3 3),0.1)@2001-01-03]', 2.0);
/* {"STBOX XT(((1,1),(2,2)),[2001-01-01, 2001-01-02))","STBOX
   XT(((2,2),(3,3)),[2001-01-02, 2001-01-03])"} */
```

9.7.4 Splitting Operations

These functions split a temporal value with respect to a sequence of bins (see Section 9.7.1) or tiles (see Section 9.7.2).

- Split a temporal number with respect to value and/or time bins {}

```
valueSplit(tnumber,vsize number,vorigin number=0,
  borderInc boolean=true) → {(number,tnumber)}
timeSplit(ttype,duration interval,torigin timestamptz='2000-01-03',
  borderInc boolean=true) → {(time,temp)}
valueTimeSplit(tnumber,vsize number,duration interval,vorigin number=0,
  torigin timestamptz='2000-01-03',borderInc boolean=true) → {(number,time,tnumber)}
```

The result is a set of pairs or a set of triples. If the origin of the value and/or the time dimension is not specified, they are set by default to 0 and to Monday, January 3, 2000, respectively. The value size and the duration of a value and time grid are strictly positive, otherwise an error is raised. The optional argument `borderInc` states whether the upper border of the extent is included and thus, the fragments starting at that border and holding only it are generated.

```
SELECT (sp).number,
  (sp).tnumber FROM (SELECT valueSplit(tint '[1@2001-01-01, 2@2001-01-02, 5@2001-01-05,
  10@2001-01-10]', 2) AS sp) t;
--  0 | {[1@2001-01-01, 1@2001-01-02)}
--  2 | {[2@2001-01-02, 2@2001-01-05)}
--  4 | {[5@2001-01-05, 5@2001-01-10)}
-- 10 | {[10@2001-01-10]}
SELECT valueSplit(tfloat '[1@2001-01-01, 10@2001-01-10)', 2.0, 1.0);
-- (1,"{[1@2001-01-01, 3@2001-01-03)}")
-- (3,"{[3@2001-01-03, 5@2001-01-05)}")
-- (5,"{[5@2001-01-05, 7@2001-01-07)}")
-- (7,"{[7@2001-01-07, 9@2001-01-09)}")
-- (9,"{[9@2001-01-09, 10@2001-01-10)}")
```

```
SELECT (ts).time, (ts).temp
FROM (SELECT timeSplit(tfloat '[1@2001-02-01, 10@2001-02-10)', '2 days') AS ts) t;
-- 2001-01-31 | [1@2001-02-01, 2@2001-02-02)
-- 2001-02-02 | [2@2001-02-02, 4@2001-02-04)
-- 2001-02-04 | [4@2001-02-04, 6@2001-02-06)
-- ...
SELECT (ts).time,
  astext((ts).temp) AS temp FROM (SELECT timeSplit(tgeompoint '[Point(1 1)@2001-02-01,
  Point(10 10)@2001-02-10]', '2 days', '2001-02-01') AS ts) AS t;
-- 2001-02-01 | [POINT(1 1)@2001-02-01, POINT(3 3)@2001-02-03)
-- 2001-02-03 | [POINT(3 3)@2001-02-03, POINT(5 5)@2001-02-05)
-- 2001-02-05 | [POINT(5 5)@2001-02-05, POINT(7 7)@2001-02-07)
-- ...
```

Notice that we can split a temporal value in cyclic (instead of linear) time bins. The following two examples show how to split a temporal value by hour and by day of the week.

```
SELECT (ts).time::time AS hour,
  merge((ts).temp) AS temp FROM (SELECT timeSplit(tfloat '[1@2001-01-01, 10@2001-01-03]',
  '1 hour') AS ts) t GROUP BY hour ORDER BY hour;
/* 00:00:00 | {[1@2001-01-01, 1.1875@2001-01-01 01:00:00),
   [5.5@2001-01-02, 5.6875@2001-01-02 01:00:00)} */
/* 01:00:00 | {[1.1875@2001-01-01 01:00:00, 1.375@2001-01-01 02:00:00),
   [5.6875@2001-01-02 01:00:00, 5.875@2001-01-02 02:00:00)} */
/* 02:00:00 | {[1.375@2001-01-01 02:00:00, 1.5625@2001-01-01 03:00:00),
   [5.875@2001-01-02 02:00:00, 6.0625@2001-01-02 03:00:00)} */
/* 03:00:00 | {[1.5625@2001-01-01 03:00:00, 1.75@2001-01-01 04:00:00),
   [6.0625@2001-01-02 03:00:00, 6.25@2001-01-02 04:00:00)} */
-- ...
```

```
SELECT EXTRACT(DOW FROM (ts).time) AS dow_no, TO_CHAR((ts).time, 'Dy') AS dow,
  asText(round(merge((ts).temp),
  2)) AS temp FROM (SELECT timeSplit(tgeompoint '[Point(1 1)@2001-01-01,
  Point(10 10)@2001-01-14)', '1 hour') AS ts) t GROUP BY dow, dow_no ORDER BY dow_no;
/* 0 | Sun | {[POINT(1 1)@2001-01-01, POINT(1.69 1.69)@2001-01-02),
   [POINT(5.85 5.85)@2001-01-08, POINT(6.54 6.54)@2001-01-09)} */
/* 1 | Mon | {[POINT(1.69 1.69)@2001-01-02, POINT(2.38 2.38)@2001-01-03),
   [POINT(6.54 6.54)@2001-01-09, POINT(7.23 7.23)@2001-01-10)} */
/* 2 | Tue | {[POINT(2.38 2.38)@2001-01-03, POINT(3.08 3.08)@2001-01-04),
   [POINT(7.23 7.23)@2001-01-10, POINT(7.92 7.92)@2001-01-11)} */
-- ...
```

```
SELECT (sp).number, (sp).time,
  (sp).tnumber FROM (SELECT valueTimeSplit(tint '[1@2001-02-01, 2@2001-02-02,
  5@2001-02-05, 10@2001-02-10]', 5, '5 days') AS sp) t;
--  0 | 2001-02-01 | {[1@2001-02-01, 2@2001-02-02, 2@2001-02-05)}
--  5 | 2001-02-01 | {[5@2001-02-05, 5@2001-02-06)}
--  5 | 2001-02-06 | {[5@2001-02-06, 5@2001-02-10)}
-- 10 | 2001-02-06 | {[10@2001-02-10]}
SELECT (sp).number, (sp).time,
  (sp).tnumber FROM (SELECT valueTimeSplit(tfloat '[1@2001-02-01, 10@2001-02-10)', 5.0,
  '5 days', 1.0, '2001-02-01') AS sp) t;
-- 1 | 2001-01-01 | [1@2001-01-01, 6@2001-01-06)
-- 6 | 2001-01-06 | [6@2001-01-06, 10@2001-01-10)
```

- Split a temporal point with respect to a spatial grid {}

```
spaceSplit({tgeompoint,tgeogpoint,tgeometry,tpose,tpcpoint},xsize float,
  [ysize float,zsize float,]
  origin point='Point(0 0 0)',bitmatrix boolean=true,
  borderInc boolean=true) → {(point,tpoint)}
timeSplit(tpoint,duration interval,torigin timestamptz='2000-01-03',
  borderInc boolean=true) → {(time,tpoint)}
spaceTimeSplit({tgeompoint,tgeogpoint,tgeometry,tpose,tpcpoint},xsize float,
  [ysize float,zsize float,]
  duration interval,sorigin point='Point(0 0 0)',
  torigin timestamptz='2000-01-03',bitmatrix boolean=true,
  borderInc boolean=true) → {(point,time,tpoint)}
```

A temporal pose and a temporal point cloud point answer these at the temporal geometry point their position resolves to, so a value carrying linear interpolation is tiled along its trajectory rather than at its instants alone. The second column of a split carries the type of the first argument, so splitting a temporal pose returns temporal poses. The result is a set of pairs or a set of triples. If the origin of the space and/or time dimension is not specified, it is set by default to `'Point(0 0 0)'` and to Monday, January 3, 2000, respectively. The arguments `ysize` and `zsize` are optional. A size left out, or given as 0, takes the value of `xsize`, and `zsize` is read only for a value with Z dimension. Every size is a finite number that is not negative, otherwise an error is raised. The size `xsize` and the duration of a spatiotemporal grid are strictly positive, otherwise an error is raised. If the argument `bitmatrix` is not specified, then the computation will use a bit matrix to speed up the process. The optional argument `borderInc` states whether the upper border of the extent is included and thus extra tiles containing the border are generated.

A geodetic temporal point travels the great circle between two positions, so it is split at the tiles that circle crosses rather than at those of the straight line in longitude and latitude: a trip whose positions both lie at latitude 61 reaches latitude 62.485889 and enters the row of tiles north of the one that line stays in. The grid laid on a geodetic value takes a geography origin and its tiles are geodetic, so a tile and the value restricted to it state coordinates of one kind. The bit matrix is filled from the straight line between two positions, which a great circle leaves, so a geodetic value is split without it and every tile of its bounding box is read instead.

```
SELECT ST_AsText((sp).point) AS point, astext((sp).tpoint) AS tpoint
FROM (SELECT spaceSplit(tgeompoint '[Point(1 1)@2001-03-01, Point(10 10)@2001-03-10]',
  2.0) AS sp) t;
-- POINT(0 0) | {[POINT(1 1)@2001-03-01, POINT(2 2)@2001-03-02)}
-- POINT(2 2) | {[POINT(2 2)@2001-03-02, POINT(4 4)@2001-03-04)}
-- POINT(4 4) | {[POINT(4 4)@2001-03-04, POINT(6 6)@2001-03-06)}
-- ...
SELECT ST_AsText((sp).point) AS point, astext((sp).tpoint) AS tpoint
FROM (SELECT spaceSplit(tgeompoint '[Point(1 1 1)@2001-03-01,
  Point(10 10 10)@2001-03-10]', 2.0, geometry 'Point(1 1 1)') AS sp) t;
-- POINT Z(1 1 1) | {[POINT Z (1 1 1)@2001-03-01, POINT Z (3 3 3)@2001-03-03)}
-- POINT Z(3 3 3) | {[POINT Z (3 3 3)@2001-03-03, POINT Z (5 5 5)@2001-03-05)}
-- POINT Z(5 5 5) | {[POINT Z (5 5 5)@2001-03-05, POINT Z (7 7 7)@2001-03-07)}
-- ...
SELECT ST_AsText((sp).point) AS point, asText((sp).tpose) AS tpose
FROM (SELECT spaceSplit(tpose '[Pose(Point(1 1),0.1)@2001-01-01,
  Pose(Point(3 3),0.1)@2001-01-03]', 2.0) AS sp) t;
-- POINT(0 0) | {[Pose(POINT(1 1),0.1)@2001-01-01, Pose(POINT(2 2),0.1)@2001-01-02)}
-- POINT(2 2) | {[Pose(POINT(2 2),0.1)@2001-01-02, Pose(POINT(3 3),0.1)@2001-01-03]}
SELECT ST_AsText((sp).point) AS point, asText((sp).tpoint, 6) AS tpoint
FROM (SELECT spaceSplit(tgeogpoint '[Point(10 61)@2001-03-01,
  Point(50 61)@2001-03-03]', 20.0) AS sp) t;
-- POINT(0 60)  | {[POINT(10 61)@2001-03-01, POINT(20 62.124941)@2001-03-01  ↩
   12:17:24.687202)}
-- POINT(20 60) | {[POINT(20 62.124941)@2001-03-01 12:17:24.687202,
--                 POINT(40 62.124941)@2001-03-02 11:42:35.312797)}
-- POINT(40 60) | {[POINT(40 62.124941)@2001-03-02 11:42:35.312797, POINT(50 61)@2001 ↩
   -03-03]}
```

```
SELECT (sp).time,
  astext((sp).temp) AS tpoint FROM (SELECT timeSplit(tgeompoint '[Point(1 1)@2001-02-01,
  Point(10 10)@2001-02-10]', interval '2 days') AS sp) t;
-- 2001-01-31 | [POINT(1 1)@2001-02-01, POINT(2 2)@2001-02-02)
-- 2001-02-02 | [POINT(2 2)@2001-02-02, POINT(4 4)@2001-02-04)
-- 2001-02-04 | [POINT(4 4)@2001-02-04, POINT(6 6)@2001-02-06)
-- ...
```

```
SELECT ST_AsText((sp).point) AS point, (sp).time, astext((sp).tpoint) AS tpoint
FROM (SELECT spaceTimeSplit(tgeompoint '[Point(1 1)@2001-02-01, Point(10 10)@2001-02-10]',
  2.0, interval '2 days') AS sp) t;
-- POINT(0 0) | 2001-01-31 | {[POINT(1 1)@2001-02-01, POINT(2 2)@2001-02-02)}
-- POINT(2 2) | 2001-01-31 | {[POINT(2 2)@2001-02-02]}
-- POINT(2 2) | 2001-02-02 | {[POINT(2 2)@2001-02-02, POINT(4 4)@2001-02-04)}
-- ...
SELECT ST_AsText((sp).point) AS point, (sp).time, astext((sp).tpoint) AS tpoint
FROM (SELECT spaceTimeSplit(tgeompoint '[Point(1 1 1)@2001-02-01,
  Point(10 10 10)@2001-02-10]', 2.0, interval '2 days', 'Point(1 1 1)',
  '2001-03-01') AS sp) t;
-- POINT Z(1 1 1) | 2001-02-01 | {[POINT Z(1 1 1)@2001-02-01, POINT Z(3 3 3)@2001-02-03)}
-- POINT Z(3 3 3) | 2001-02-01 | {[POINT Z(3 3 3)@2001-02-03]}
-- POINT Z(3 3 3) | 2001-02-03 | {[POINT Z(3 3 3)@2001-02-03, POINT Z (5 5 5)@2001-02-05)}
-- ...
```

Chapter 10

Temporal Types: Aggregation and Indexing

10.1 Aggregation

The temporal aggregate functions generalize the traditional aggregate functions. Their semantics is that they compute the value of the function at every instant in the *union* of the temporal extents of the values to aggregate. In contrast, recall that all other functions manipulating temporal types compute the value of the function at every instant in the *intersection* of the temporal extents of the arguments.

The temporal aggregate functions are the following ones:

- For all temporal types, the function `tCount` generalize the traditional function `count`. The temporal count can be used to compute at each point in time the number of available objects (for example, number of cars in an area).
- For all temporal types, function `extent` returns a bounding box that encloses a set of temporal values. Depending on the base type, the result of this function can be a `tstzspan`, a `tbox` or an `stbox`.
- For the temporal Boolean type, the functions `tAnd` and `tOr` generalize the traditional functions `and` and `or`.
- For temporal numeric types, there are two types of temporal aggregate functions. The functions `tMin`, `tMax`, `tSum`, and `tAvg` generalize the traditional functions `min`, `max`, `sum`, and `avg`. Furthermore, the functions `wMin`, `wMax`, `wCount`, `wSum`, and `wAvg` are window (or cumulative) versions of the traditional functions that, given a time interval w, compute the value of the function at an instant t by considering the values during the interval [t-w, t]. All window aggregate functions are available for temporal integers, while for temporal floats only window minimum and maximum are meaningful.
- For the temporal text type, the functions `tMin` y `tMax` generalize the traditional functions `min` and `max`.
- Finally, for temporal point types, the function `tCentroid` generalizes the function `ST_Centroid` provided by PostGIS. For example, given set of objects that move together (that is, a convoy or a flock) the temporal centroid will produce a temporal point that represents at each instant the geometric center (or the center of mass) of all the moving objects.

Note

The names `tMin`, `tMax`, `merge`, `appendInstant` and `appendSequence` each denote both a scalar function and an aggregate, which PostgreSQL tells apart by their arguments: the scalar `tMin(tbox)` is the minimum of a box, the aggregate `tMin(tint)` the temporal minimum of a set of rows. DuckDB, Apache Spark and Apache Flink hold one function per name, and call these aggregates `tMinAgg`, `tMaxAgg`, `mergeAgg`, `appendInstantAgg` and `appendSequenceAgg` (Section 18.10.1). DuckDB names its weighted average `wavg` and compares names without case, so the three engines call the window average `wAvgAgg`.

In the examples that follow, we suppose the tables `Department` and `Trip` contain the two tuples introduced in Section 4.2.

- Temporal count

```
tCount(ttype) → {tintSeq,tintSeqSet}
```

```
SELECT tCount(NoEmps) FROM Department;
-- {[1@2001-01-01, 2@2001-02-01, 1@2001-08-01, 1@2001-10-01)}
```

- Bounding box extent

```
extent(ttype) → {tstzspan,tbox,stbox}
```

```
SELECT extent(noEmps) FROM Department;
-- TBOX XT((4,12),[2001-01-01,2001-10-01])
SELECT extent(Trip) FROM Trips;
-- STBOX XT(((0,0),(3,3)),[2001-01-01 08:00:00, 2001-01-01 08:20:00))
```

- Temporal and, temporal or

```
tOr(tbool) → tbool
tAnd(tbool) → tbool
```

```
SELECT tAnd(NoEmps #> 6) FROM Department;
-- {[t@2001-01-01, f@2001-04-01, f@2001-10-01)}
SELECT tOr(NoEmps #> 6) FROM Department;
-- {[t@2001-01-01, f@2001-08-01, f@2001-10-01)}
```

- Temporal minimum, maximum, sum, and average

```
tMin(ttype) → ttype
tMax(ttype) → ttype
tSum(tnumber) → {tnumberSeq,tnumberSeqSet}
tAvg(tnumber) → {tfloatSeq,tfloatSeqSet}
```

```
SELECT tMin(NoEmps) FROM Department;
-- {[10@2001-01-01, 4@2001-02-01, 6@2001-06-01, 6@2001-10-01)}
SELECT tMax(NoEmps) FROM Department;
-- {[10@2001-01-01, 12@2001-04-01, 6@2001-08-01, 6@2001-10-01)}
SELECT tSum(NoEmps) FROM Department;
/* {[10@2001-01-01, 14@2001-02-01, 16@2001-04-01, 18@2001-06-01, 6@2001-08-01,
   6@2001-10-01)} */
SELECT tAvg(NoEmps) FROM Department;
/* {[10@2001-01-01, 10@2001-02-01), [7@2001-02-01, 7@2001-04-01),
   [8@2001-04-01, 8@2001-06-01), [9@2001-06-01, 9@2001-08-01),
   [6@2001-08-01, 6@2001-10-01) */
```

- Window count

```
wCount(tnumber,interval) → {tintSeq,tintSeqSet}
```

```
SELECT wCount(NoEmps, interval '2 days') FROM Department;
/* {[1@2001-01-01, 2@2001-02-01, 3@2001-04-01, 2@2001-04-03, 3@2001-06-01, 2@2001-06-03,
   1@2001-08-03, 1@2001-10-03)} */
```

- Window minimum, maximum, sum, and average

```
wMin(tnumber,interval) → {tnumberSeq,tnumberSeqSet}
wMax(tnumber,interval) → {tnumberDiscSeq,tnumberSeqSet}
wSum(tint,interval) → {tintSeq,tintSeqSet}
wAvg(tint,interval) → {tfloatSeq,tfloatSeqSet}
```

```
SELECT wMin(NoEmps, interval '2 days') FROM Department;
-- {[10@2001-01-01, 4@2001-04-01, 6@2001-06-03, 6@2001-10-03)}
SELECT wMax(NoEmps, interval '2 days') FROM Department;
-- {[10@2001-01-01, 12@2001-04-01, 6@2001-08-03, 6@2001-10-03)}
SELECT wSum(NoEmps, interval '2 days') FROM Department;
/* {[10@2001-01-01, 14@2001-02-01, 26@2001-04-01, 16@2001-04-03, 22@2001-06-01,
   18@2001-06-03, 6@2001-08-03, 6@2001-10-03)} */
SELECT round(wAvg(NoEmps, interval '2 days'), 3) FROM Department;
/*  Interp=Step;{[10@2001-01-01, 7@2001-02-01, 8.667@2001-04-01, 8@2001-04-03,
   7.333@2001-06-01, 9@2001-06-03, 6@2001-08-03, 6@2001-10-03)} */
```

- Temporal centroid

```
tCentroid(tgeompoint) → tgeompoint
```

```
SELECT tCentroid(Trip) FROM Trips;
/* {[POINT(0 0)@2001-01-01 08:00:00, POINT(1 0)@2001-01-01 08:05:00),
   [POINT(0.5 0)@2001-01-01 08:05:00, POINT(1.5 0.5)@2001-01-01 08:10:00,
   POINT(2 1.5)@2001-01-01 08:15:00),
   [POINT(2 2)@2001-01-01 08:15:00, POINT(3 3)@2001-01-01 08:20:00)} */
```

10.2 Indexing

GiST and SP-GiST indexes can be created for table columns of temporal types. The GiST index implements an R-tree and the SP-GiST index implements an n-dimensional quad-tree. Examples of index creation are as follows:

```
CREATE INDEX Department_NoEmps_Gist_Idx ON Department USING Gist(NoEmps);
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist(Trip);
```

The GiST and SP-GiST indexes store the bounding box for the temporal types. As explained in Chapter 4, these are

- the `tstzspan` type for the `tbool` and `ttext` types,
- the `tbox` type for the `tint` and `tfloat` types,
- the `stbox` type for the `tgeompoint`, `tgeogpoint`, `tgeometry` and `tgeography` types.

A GiST or SP-GiST index can accelerate queries involving the following operators (see Section 5.3 for more information):

- `<<`, `&<`, `&>`, `>>`, which only consider the value dimension in temporal alphanumeric types,
- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, `&</`, `<</`, `/>>`, and `/&>`, which only consider the spatial dimension in temporal point types,
- `&<#`, `<<#`, `#>>`, `#&>`, which only consider the time dimension for all temporal types,
- `&&`, `@>`, `<@`, `~=`, and `|=|`, which consider as many dimensions as they are shared by the indexed column and the query argument. These operators work on bounding boxes (that is, `tstzspan`, `tbox`, or `stbox`), not the entire values.

For example, given the index defined above on the `Department` table and a query that involves a condition with the `&&` (overlaps) operator, if the right argument is a temporal float then both the value and the time dimensions are considered for filtering the tuples of the relation, while if the right argument is a float value, a float span, or a time type, then either the value or the time dimension will be used for filtering the tuples of the relation. Furthermore, a bounding box can be constructed from a value/span and/or a timestamp/period, which can be used for filtering the tuples of the relation. Examples of queries using the index on the `Department` table defined above are given next.

```
SELECT * FROM Department WHERE NoEmps && intspan '[1, 5)';
SELECT * FROM Department WHERE NoEmps && tstzspan '[2001-04-01, 2001-05-01)';
SELECT * FROM Department WHERE NoEmps &&
  tbox(intspan '[1, 5)', tstzspan '[2001-04-01, 2001-05-01)');
SELECT * FROM Department WHERE NoEmps &&
  tfloat '{[1@2001-01-01, 1@2001-02-01), [5@2001-04-01, 5@2001-05-01)}';
```

Similarly, examples of queries using the index on the `Trips` table defined above are given next.

```
SELECT * FROM Trips WHERE Trip && geometry 'Polygon((0 0,0 1,1 1,1 0,0 0))';
SELECT * FROM Trips WHERE Trip && timestamptz '2001-01-01';
SELECT * FROM Trips WHERE Trip && tstzspan '[2001-01-01, 2001-01-05)';
SELECT * FROM Trips WHERE Trip &&
  stbox(geometry 'Polygon((0 0,0 1,1 1,1 0,0 0))', tstzspan '[2001-01-01, 2001-01-05]');
SELECT * FROM Trips WHERE Trip &&
  tgeompoint '{[Point(0 0)@2001-01-01, Point(1 1)@2001-01-02, Point(1 1)@2001-01-05)}';
```

Finally, B-tree indexes can be created for table columns of all temporal types. For this index type, the only useful operation is equality. There is a B-tree sort ordering defined for values of temporal types, with corresponding <, <=, >, >= and operators, but the ordering is rather arbitrary and not usually useful in the real world. B-tree support for temporal types is primarily meant to allow sorting internally in queries, rather than creation of actual indexes.

In order to speed up several of the functions for temporal types, we can add in the `WHERE` clause of queries a bounding box comparison that make uses of the available indexes. For example, this would be typically the case for the functions that project the temporal types to the value/spatial and/or time dimensions. This will filter out the tuples with an index as shown in the following query.

```
SELECT atTime(T.Trip, tstzspan '[2001-01-01, 2001-01-02)') FROM Trips T
-- Bounding box index filtering
WHERE T.Trip && tstzspan '[2001-01-01, 2001-01-02)';
```

In the case of temporal points, all spatial relationships with the ever and always semantics (see Section 8.9) automatically include a bounding box comparison that will make use of any indexes that are available on the temporal points. For this reason, the first version of the relationships is typically used for filtering the tuples with the help of an index when computing the temporal relationships as shown in the following query.

```
SELECT tIntersects(T.Trip, R.Geom) FROM Trips T, Regions R
-- Bounding box index filtering
WHERE eIntersects(T.Trip, R.Geom);
```

10.3 Statistics and Selectivity

10.3.1 Statistics Collection

The PostgreSQL planner relies on statistical information about the contents of tables in order to generate the most efficient execution plan for queries. These statistics include a list of some of the most common values in each column and a histogram showing the approximate data distribution in each column. For large tables, a random sample of the table contents is taken, rather than examining every row. This enables large tables to be analyzed in a small amount of time. The statistical information is gathered by the `ANALYZE` command and stored in the `pg_statistic` catalog table. Since different kinds of statistics may be appropriate for different kinds of data, the table only stores very general statistics (such as number of null values) in dedicated columns. Everything else is stored in five “slots”, which are couples of array columns that store the statistics for a column of an arbitrary type.

The statistics collected for time types and temporal types are based on those collected by PostgreSQL for scalar types and span types. For scalar types, such as `float`, the following statistics are collected:

1. fraction of null values,

2. average width, in bytes, of non-null values,
3. number of different non-null values,
4. array of most common values and array of their frequencies,
5. histogram of values, where the most common values are excluded,
6. correlation between physical and logical row ordering.

For span types, like `tstzspan`, three additional histograms are collected:

7. length histogram of non-empty spans,
8. histograms of lower and upper bounds.

For geometries, in addition to (1)–(3), the following statistics are collected:

9. number of dimensions of the values, N-dimensional bounding box, number of rows in the table, number of rows in the sample, number of non-null values,
10. N-dimensional histogram that divides the bounding box into a number of cells and keeps the proportion of values that intersects with each cell.

The statistics collected for columns of the time and span types `tstzset`, `tstzspan`, `tstzspanset`, `intspan`, and `floatspan` replicate those collected by PostgreSQL for the `tstzrange`. This is clear for the span types in MobilityDB, which are more efficient versions of the range types in PostgreSQL. For the `tstzset` and the `tstzspanset` types, a value is converted into its bounding period, then the statistics for the `tstzspan` type are collected.

The statistics collected for columns of temporal types depend on their subtype and their base type. In addition to statistics (1)–(3) that are collected for all temporal types, statistics are collected for the time and the value dimensions independently. More precisely, the following statistics are collected for the time dimension:

- For columns of instant subtype, the statistics (4)–(6) are collected for the timestamps.
- For columns of other subtype, the statistics (7)–(8) are collected for the (bounding box) periods.

The following statistics are collected for the value dimension:

- For columns of temporal types with step interpolation (that is, `tbool`, `ttext`, or `tint`):
  - For the instant subtype, the statistics (4)–(6) are collected for the values.
  - For all other subtypes, the statistics (7)–(8) are collected for the values.
- For columns of the temporal float type (that is, `tfloat`):
  - For the instant subtype, the statistics (4)–(6) are collected for the values.
  - For all other subtype, the statistics (7)–(8) are collected for the (bounding) value spans.
- For columns of temporal point types (that is, `tgeompoint` and `tgeogpoint`) the statistics (9)–(10) are collected for the points.

10.3.2 Selectivity Estimation

Boolean operators in PostgreSQL can be associated with two selectivity functions, which compute how likely a value of a given type will match a given criterion. These selectivity functions rely on the statistics collected. There are two types of selectivity functions. The *restriction* selectivity functions try to estimate the percentage of the rows in a table that satisfy a `WHERE`-clause condition of the form `column OP constant`. On the other hand, the *join* selectivity functions try to estimate the percentage of the rows in a table that satisfy a `WHERE`-clause condition of the form `table1.column1 OP table2.column2`.

MobilityDB defines 23 classes of Boolean operators (such as =, <, `&&`, <<, etc.), each of which can have as left or right arguments a PostgreSQL type (such as `integer`, `timestamptz`, etc.) or a MobilityDB type (such as `tstzspan`, `tint`, etc.). As a consequence, there is a very high number of operators with different arguments to be considered for the selectivity functions. The approach taken was to group these combinations into classes corresponding to the value and time dimensions. The classes correspond to the type of statistics collected as explained in the previous section.

MobilityDB estimates both restriction and join selectivity for time, span, and temporal types.

Chapter 11

Temporal Network Points

The temporal points that we have considered so far represent the movement of objects that can move freely on space since it is assumed that they can change their position from one location to the next one without any motion restriction. This is the case for animals and for flying objects such as planes or drones. However, in many cases, objects do not move freely in space but rather within spatially embedded networks such as routes or railways. In this case, it is necessary to take the embedded networks into account while describing the movements of these moving objects. Temporal network points account for these requirements.

Compared with the free-space temporal points, network-based points have the following advantages:

- Network points provide road constraints that reflect the real movements of moving objects.
- The geometric information is not stored with the moving point, but once and for all in the fixed networks. In this way, the location representations and interpolations are more precise.
- Network points are more efficient in terms of data storage, location update, formulation of query, as well as indexing. These are discussed later in this document.

Temporal network points are based on pgRouting, a PostgreSQL extension for developing network routing applications and doing graph analysis. Therefore, temporal network points asume that the underlying network is defined in a table named `ways`, which has at least three columns: `gid` containing the unique route identifier, `length` containing the route length, and `the_geom` containing the route geometry.

There are two static network types, `npoint` (short for network point) and `nsegment` (short for network segment), which represent, respectively, a point and a segment of a route. An `npoint` value is composed of a route identifier and a float number in the range [0,1] determining a relative position of the route, where 0 corresponds to the begining of the route and 1 to the end of the route. An `nsegment` value is composed of a route identifier and two float numbers in the range [0,1] determining the start and end relative positions. A `nsegment` value whose start and end positions are equal corresponds to an `npoint` value.

The `npoint` type serves as base type for defining the temporal network point type `tnpoint`. The `tnpoint` type has similar functionality as the temporal point type `tgeompoint` with the exception that it only considers two dimensions. Thus, all functions and operators described before for the `tgeompoint` type are also applicable for the `tnpoint` type. In addition, there are specific functions defined for the `tnpoint` type.

11.1 Static Network Types

An `npoint` value is a couple of the form `(rid,position)` where `rid` is a `bigint` value representing a route identifier and `position` is a `float` value in the range [0,1] indicating its relative position. The values 0 and 1 of `position` denote, respectively, the starting and the ending position of the route. The road distance between an `npoint` value and the starting position of route with identifier `rid` is computed by multiplying `position` by length, where `length` is the route length. Examples of input of network point values are as follows:

```
SELECT npoint 'Npoint(76, 0.3)';
SELECT npoint 'Npoint(64, 1.0)';
```

The constructor function for network points has one argument for the route identifier and one argument for the relative position. An example of a network point value defined with the constructor function is as follows:

```
SELECT npoint(76, 0.3);
```

An `nsegment` value is a triple of the form `(rid,startPosition,endPosition)` where `rid` is a `bigint` value representing a route identifier and `startPosition` and `endPosition` are `float` values in the range [0,1] such that `startPosition ≤ endPosition`. Semantically, a network segment represents a set of network points `(rid,position)` with `startPosition ≤ position ≤ endPosition`. If `startPosition=0` and `endPosition=1`, the network segment is equivalent to the entire route. If `startPosition=endPosition`, the network segment represents into a single network point. Examples of input of network point values are as follows:

```
SELECT nsegment 'Nsegment(76, 0.3, 0.5)';
SELECT nsegment 'Nsegment(64, 0.5, 0.5)';
SELECT nsegment 'Nsegment(64, 0.0, 1.0)';
SELECT nsegment 'Nsegment(64, 1.0, 0.0)';
-- converted to nsegment 'Nsegment(64, 0.0, 1.0)';
```

As can be seen in the last example, the `startPosition` and `endPosition` values will be inverted to ensure that the condition `startPosition ≤ endPosition` is always satisfied. The constructor function for network segments has one argument for the route identifier and two optional arguments for the start and end positions. Examples of network segment values defined with the constructor function are as follows:

```
SELECT nsegment(76, 0.3, 0.3);
SELECT nsegment(76); -- start and end position assumed to be 0 and 1 respectively
SELECT nsegment(76, 0.5); -- end position assumed to be 1
```

Values of the `npoint` type can be converted to the `nsegment` type using an explicit `CAST` or using the `::` notation as shown next.

```
SELECT npoint(76, 0.33)::nsegment;
```

Values of static network types must satisfy several constraints so that they are well defined. These constraints are given next.

- The route identifier `rid` must be found in column `gid` of table `ways`.
- The `position`, `startPosition`, and `endPosition` values must be in the range [0,1]. An error is raised whenever one of these constraints are not satisfied.

Examples of incorrect static network type values are as follows.

```
-- Incorrect rid value
SELECT npoint 'Npoint(87.5, 1.0)';
-- Incorrect position value
SELECT npoint 'Npoint(87, 2.0)';
-- rid value not found in the ways table
SELECT npoint 'Npoint(99999999, 1.0)';
```

We give next the functions and operators for the static network types.

11.1.1 Input and Output

- Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation

```
asText({npoint,npoint[]}) → {text,text[]}
asText(nsegment) → text
asEWKT({npoint,npoint[]}) → {text,text[]}
asEWKT(nsegment) → text
```

```
SELECT asText(npoint 'SRID=5676;Npoint(1,0.5)');
-- NPoint(1,0.5)
SELECT asText(ARRAY[npoint 'Npoint(1,0.2)', 'Npoint(1,0.5)']);
-- {"NPoint(1,0.2)","NPoint(1,0.5)"}
SELECT asText(nsegment 'Nsegment(1,0.3,0.5)');
-- NSegment(1,0.3,0.5)
SELECT asEWKT(npoint 'SRID=5676;Npoint(1,0.5)');
-- SRID=5676;NPoint(1,0.5)
SELECT asEWKT(ARRAY[npoint 'SRID=5676;Npoint(1,0.2)', 'SRID=5676;Npoint(1,0.5)']);
-- {"SRID=5676;NPoint(1,0.2)","SRID=5676;NPoint(1,0.5)"}
SELECT asEWKT(nsegment 'Nsegment(1,0.3,0.5)');
-- SRID=5676;NSegment(1,0.3,0.5)
```

- Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), the Hexadecimal Well-Known Binary (HexWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

```
asBinary(npoint,endian text='') → bytea
asEWKB(npoint,endian text='') → bytea
asHexWKB(npoint,endian text='') → text
asHexEWKB(npoint,endian text='') → text
```

The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used. A network point takes its SRID from the underlying route network: `asEWKB` and `asHexEWKB` embed it, while the plain `asBinary` and `asHexWKB` forms omit it.

```
SELECT asBinary(npoint 'SRID=5676;Npoint(1,0.5)');
-- \x01010100000000000000000000000000e03f
SELECT asEWKB(npoint 'SRID=5676;Npoint(1,0.5)');
-- \x01412c1600000100000000000000000000000000e03f
SELECT asHexWKB(npoint 'SRID=5676;Npoint(1,0.5)');
-- 01010100000000000000000000000000E03F
SELECT asHexEWKB(npoint 'SRID=5676;Npoint(1,0.5)');
-- 01412C1600000100000000000000000000000000E03F
```

- Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation

```
npointFromText(text) → npoint
npointFromEWKT(text) → npoint
nsegmentFromText(text) → nsegment
nsegmentFromEWKT(text) → nsegment
```

An SRID stated by the extended input must be the SRID of the route network. When no route network is loaded, a network point has no SRID and the stated one is not kept.

```
SELECT asEWKT(npointFromText(text 'Npoint(1,0.5)'));
-- SRID=5676;NPoint(1,0.5)
SELECT asEWKT(npointFromEWKT(text 'SRID=5676;Npoint(1,0.5)'));
-- SRID=5676;NPoint(1,0.5)
SELECT npointFromEWKT(text 'SRID=4326;Npoint(1,0.5)');
-- ERROR:  The SRID of the npoint (4326) does not match the SRID of the network (5676)
SELECT asText(nsegmentFromText(text 'Nsegment(1,0.3,0.5)'));
-- NSegment(1,0.3,0.5)
SELECT asEWKT(nsegmentFromEWKT(text 'SRID=5676;Nsegment(1,0.3,0.5)'));
-- SRID=5676;NSegment(1,0.3,0.5)
```

- Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), from the Hexadecimal Well-Known Binary (HexWKB), or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

```
npointFromBinary(bytea) → npoint
npointFromEWKB(bytea) → npoint
npointFromHexWKB(text) → npoint
npointFromHexEWKB(text) → npoint
```

```
SELECT asEWKT(npointFromEWKB('\x01412c1600000100000000000000000000000000e03f'));
-- SRID=5676;NPoint(1,0.5)
SELECT asEWKT(npointFromHexEWKB('01412C1600000100000000000000000000000000E03F'));
-- SRID=5676;NPoint(1,0.5)
```

- Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), the Hexadecimal Well-Known Binary (HexWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation of a network segment

  ```
  asBinary(nsegment,endian text='') → bytea
  asEWKB(nsegment,endian text='') → bytea
  asHexWKB(nsegment,endian text='') → text
  asHexEWKB(nsegment,endian text='') → text
  ```

  The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used. A network segment takes its SRID from the underlying route network, as a network point does: `asEWKB` and `asHexEWKB` embed it, while the plain `asBinary` and `asHexWKB` forms omit it.

  ```
  SELECT asBinary(nsegment 'NSegment(1,0.5,0.7)');
  -- \x01010100000000000000000000000000e03f666666666666e63f
  SELECT asEWKB(nsegment 'NSegment(1,0.5,0.7)');
  -- \x01412c1600000100000000000000000000000000e03f666666666666e63f
  SELECT asHexWKB(nsegment 'NSegment(1,0.5,0.7)');
  -- 01010100000000000000000000000000E03F666666666666E63F
  SELECT asHexEWKB(nsegment 'NSegment(1,0.5,0.7)', 'XDR');
  -- 00410000162C00000000000000013FE00000000000003FE6666666666666
  ```

- Input of a network segment from the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), the Hexadecimal Well-Known Binary (HexWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

  ```
  nsegmentFromBinary(bytea) → nsegment
  nsegmentFromEWKB(bytea) → nsegment
  nsegmentFromHexWKB(text) → nsegment
  nsegmentFromHexEWKB(text) → nsegment
  ```

  An SRID stated by an extended input must be the SRID of the route network, and the route must be one of the network.

  ```
  SELECT nsegmentFromBinary(asBinary(nsegment 'NSegment(1,0.5,0.7)'));
  -- NSegment(1,0.5,0.7)
  SELECT nsegmentFromHexEWKB(asHexEWKB(nsegment 'NSegment(1,0.5,0.7)'));
  -- NSegment(1,0.5,0.7)
  ```

11.1.2 Constructors

- Constructors for network points and network segments

  ```
  npoint(bigint,float) → npoint
  nsegment(bigint,float,float) → nsegment
  ```

  ```
  SELECT npoint(76, 0.3);
  SELECT nsegment(76, 0.3, 0.5);
  ```

11.1.3 Conversions

Values of the `npoint` and `nsegment` types can be converted to the `geometry` type using an explicit `CAST` or using the `::` notation as shown below. Similarly, `geometry` values of subtype `point` or `linestring` (restricted to two points) can be converted, respectively, to `npoint` and `nsegment` values. For this, the route on which the given points lie must be found, up to the rounding of a point computed on a route, about 1e-13 for coordinates of 100, so that a point taken from a network point maps back to it while a point lying off the network does not. If no such route is found, a null value is returned. A point measured near the network, such as a GPS position, is mapped with the function `npoint(geometry,float)`, whose second argument is the maximum distance between the point and the route: the result is located on the nearest route within that distance, the one with the least identifier among equally near routes, and it is null when no route is within that distance. A spatial index on the `the_geom` column of the `ways` table serves this search.

- Convert between a network point or segment and a geometry

  ```
  {npoint,nsegment}::{geometry,geography}
  geometry::{npoint,nsegment}
  npoint(geometry,float) → npoint
  ```

  The conversions follow the rule of [?listitem].

  ```
  SELECT ST_AsText(npoint(76, 0.33)::geometry, 6);
  -- POINT(27.842471 43.855103)
  SELECT ST_AsText(nsegment(76, 0.33, 0.66)::geometry, 6);
  -- LINESTRING(27.842471 43.855103,45.316951 45.399688)
  SELECT ST_AsText(nsegment(76, 0.33, 0.33)::geometry, 6);
  -- POINT(27.842471 43.855103)
  ```

  ```
  SELECT round(geometry 'SRID=5676;Point(27.842471 43.855103)'::npoint, 6);
  -- NPoint(76,0.33)
  SELECT round(geometry 'SRID=5676;LINESTRING(27.842471 43.855103,
    45.316951 45.399688)'::nsegment, 6);
  -- NSegment(76,0.33,0.66)
  SELECT geometry 'SRID=5676;Point(28 44)'::npoint;
  -- NULL
  SELECT npoint(geometry 'SRID=5676;Point(28 44)', 0.1);
  -- NULL
  SELECT round(npoint(geometry 'SRID=5676;Point(28 44)', 0.5), 6);
  -- NPoint(76,0.333192)
  ```

- Construct a spatiotemporal box from a network point and, optionally, a timestamp or a period

  ```
  stbox(npoint) → stbox
  stbox(npoint,{timestamptz,tstzspan}) → stbox
  ```

  ```
  SELECT stbox(npoint 'NPoint(1,0.3)');
  -- SRID=5676;STBOX X((62.786633,80.143556),(62.786633,80.143556))
  SELECT stbox(npoint 'NPoint(1,0.3)', timestamptz '2001-01-01');
  /* SRID=5676;STBOX XT(((62.786633,80.143556),(62.786633,80.143556)),[2001-01-01,
     2001-01-01]) */
  SELECT stbox(npoint 'NPoint(1,0.3)', tstzspan '[2001-01-01,2001-01-02]');
  /* SRID=5676;STBOX XT(((62.786633,80.143556),(62.786633,80.143556)),[2001-01-01,
     2001-01-02]) */
  ```

11.1.4 Accessors

- Return the route identifier

  ```
  route({npoint,nsegment}) → bigint
  ```

  ```
  SELECT route(npoint 'Npoint(63, 0.3)');
  -- 63
  SELECT route(nsegment 'Nsegment(76, 0.3, 0.3)');
  -- 76
  ```

- Return the position

```
getPosition(npoint) → float
```

```
SELECT getPosition(npoint 'Npoint(63, 0.3)');
-- 0.3
```

- Return the start/end position

```
startPosition(nsegment) → float
endPosition(nsegment) → float
```

```
SELECT startPosition(nsegment 'Nsegment(76, 0.3, 0.5)');
-- 0.3
SELECT endPosition(nsegment 'Nsegment(76, 0.3, 0.5)');
-- 0.5
```

11.1.5 Transformations

- Round the position(s) of the network point or the network segment to the number of decimal places

```
round({npoint,nsegment},integer=0) → {npoint,nsegment}
```

```
SELECT round(npoint(76, 0.123456789), 6);
--  NPoint(76,0.123457)
SELECT round(nsegment(76, 0.123456789, 0.223456789), 6);
--  NSegment(76,0.123457,0.223457)
```

11.1.6 Spatial Operations

- Return the spatial reference identifier

```
SRID({npoint,nsegment}) → integer
```

```
SELECT SRID(npoint 'Npoint(76, 0.3)');
-- 5676
SELECT SRID(nsegment 'Nsegment(76, 0.3, 0.5)');
-- 5676
```

Two `npoint` values with different route identifiers may denote the same location, at the intersection of their routes. The comparison operators compare the route identifier and the position, so they tell these two values apart. Whether they denote the same location is answered on their geometries. Since the point of a network point is interpolated along its route, two such points computed along different routes may differ by the rounding of that interpolation, so their geometries are compared within a distance, for example with `ST_DWithin`, rather than with =:

```
WITH inter(geom) AS (
  SELECT st_intersection(t1.the_geom, t2.the_geom)
  FROM ways t1, ways t2 WHERE t1.gid = 1 AND t2.gid = 2), fractions(f1, f2) AS (
  SELECT ST_LineLocatePoint(t1.the_geom, i.geom), ST_LineLocatePoint(t2.the_geom, i.geom)
  FROM ways t1, ways t2, inter i WHERE t1.gid = 1 AND t2.gid = 2)
SELECT npoint(1, f1) = npoint(2, f2) FROM fractions;
-- false
SELECT ST_DWithin(npoint(1, f1)::geometry, npoint(2, f2)::geometry, 1e-9) FROM fractions;
-- true
```

11.1.7 Comparisons

The comparison operators (=, <, and so on) for static network types require that the left and right arguments be of the same type. Excepted the equality and inequality, the other comparison operators are not useful in the real world but allow B-tree indexes to be constructed on static network types.

- Traditional comparisons

```
npoint {=, <>, <, >, <=, >=} npoint
nsegment {=, <>, <, >, <=, >=} nsegment
```

```
SELECT npoint 'Npoint(3, 0.5)' = npoint 'Npoint(3, 0.5)';
-- true
SELECT nsegment 'Nsegment(3, 0.5, 0.5)' <> nsegment 'Nsegment(3, 0.5, 0.5)';
-- false
SELECT nsegment 'Nsegment(3, 0.5, 0.5)' < nsegment 'Nsegment(3, 0.5, 0.6)';
-- true
SELECT nsegment 'Nsegment(3, 0.5, 0.5)' > nsegment 'Nsegment(2, 0.5, 0.5)';
-- true
SELECT npoint 'Npoint(1, 0.5)' <= npoint 'Npoint(2, 0.5)';
-- true
SELECT npoint 'Npoint(1, 0.6)' >= npoint 'Npoint(1, 0.5)';
-- true
```

11.2 Temporal Network Points

The temporal network point type `tnpoint` allows to represent the movement of objects over a network. It corresponds to the temporal point type `tgeompoint` restricted to two-dimensional coordinates. As all the other temporal types it comes in three subtypes, namely, instant, sequence, and sequence set. Examples of `tnpoint` values in these subtypes are given next.

```
SELECT tnpoint 'Npoint(1, 0.5)@2001-01-01';
SELECT tnpoint '{Npoint(1, 0.3)@2001-01-01, Npoint(1, 0.5)@2001-01-02,
  Npoint(1, 0.5)@2001-01-03}';
SELECT tnpoint '[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.4)@2001-01-02,
  Npoint(1, 0.5)@2001-01-03]';
SELECT tnpoint '{[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.4)@2001-01-02,
  Npoint(1, 0.5)@2001-01-03], [Npoint(2, 0.6)@2001-01-04, Npoint(2, 0.6)@2001-01-05]}';
```

The temporal network point type accepts type modifiers (or `typmod` in PostgreSQL terminology). The possible values for the type modifier are `Instant`, `Sequence`, and `SequenceSet`. If no type modifier is specified for a column, values of any subtype are allowed.

```
SELECT tnpoint(Sequence) '[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.4)@2001-01-02,
  Npoint(1, 0.5)@2001-01-03]';
SELECT tnpoint(Sequence) 'Npoint(1, 0.2)@2001-01-01';
-- ERROR: Temporal type (Instant) does not match column type (Sequence)
```

Temporal network point values of sequence subtype and linear or step interpolation must be defined on a single route. Therefore, a value of sequence set subtype is needed for representing the movement of an object that traverses several routes, even if there is no temporal gap. For example, in the following value

```
SELECT tnpoint '{[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.5)@2001-01-03),
  [NPoint(2, 0.4)@2001-01-03, NPoint(2, 0.6)@2001-01-04)}';
```

the network point changes its route at 2001-01-03.

Temporal network point values of sequence or sequence set subtype are converted into a normal form so that equivalent values have identical representations. For this, consecutive instant values are merged when possible. Three consecutive instant values can be merged into two if the linear functions defining the evolution of values are the same. Examples of transformation into a normal form are as follows.

```
SELECT tnpoint '[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.4)@2001-01-02,
  NPoint(1, 0.6)@2001-01-03)';
-- [NPoint(1,0.2)@2001-01-01, NPoint(1,0.6)@2001-01-03)
SELECT tnpoint '{[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.3)@2001-01-02,
  NPoint(1, 0.5)@2001-01-03), [NPoint(1, 0.5)@2001-01-03, NPoint(1, 0.7)@2001-01-04)}';
-- {[NPoint(1,0.2)@2001-01-01, NPoint(1,0.3)@2001-01-02, NPoint(1,0.7)@2001-01-04)}
```

11.3 Validity of Temporal Network Points

Temporal network point values must satisfy the constraints specified in Section 4.3 so that they are well defined. An error is raised whenever one of these constraints are not satisfied. Examples of incorrect values are as follows.

```
-- Null values are not allowed
SELECT tnpoint 'NULL@2001-01-01 08:05:00';
SELECT tnpoint 'Point(0 0)@NULL';
-- Base type is not a network point
SELECT tnpoint 'Point(0 0)@2001-01-01 08:05:00';
-- Multiple routes in a continuous sequence
SELECT tnpoint '[Npoint(1, 0.2)@2001-01-01 09:00:00, Npoint(2, 0.2)@2001-01-01 09:05:00)';
```

We present next the operation for temporal network point types. Most functions for temporal types described in the previous chapters can be applied for temporal network point types. Therefore, in the signatures of the functions, the notation `base` also represents an `npoint` and the notations `ttype`, `tpoint`, and `tgeompoint` also represent a `tnpoint`. Furthermore, the functions that have an argument of type `geometry` accept in addition an argument of type `npoint`. To avoid redundancy, we only present next some examples of these functions and operators for temporal network points.

11.4 Constructors

- Constructor for temporal network points having a constant value

  ```
  tnpoint(npoint,timestamptz) → tnpointInst
  tnpoint(npoint,tstzset) → tnpointDiscSeq
  tnpoint(npoint,tstzspan,interp text='linear') → tnpointContSeq
  tnpoint(npoint,tstzspanset,interp text='linear') → tnpointSeqSet
  ```

  ```
  SELECT tnpoint('Npoint(1, 0.5)', timestamptz '2001-01-01');
  -- NPoint(1,0.5)@2001-01-01
  SELECT tnpointSeq('Npoint(1, 0.3)', tstzset '{2001-01-01, 2001-01-03, 2001-01-05}');
  -- {NPoint(1,0.3)@2001-01-01, NPoint(1,0.3)@2001-01-03, NPoint(1,0.3)@2001-01-05}
  SELECT tnpointSeq('Npoint(1, 0.5)', tstzspan '[2001-01-01, 2001-01-02]');
  -- [NPoint(1,0.5)@2001-01-01, NPoint(1,0.5)@2001-01-02]
  SELECT tnpointSeqSet('Npoint(1, 0.2)', tstzspanset '{[2001-01-01, 2001-01-03]}', 'step');
  -- Interp=Step;{[NPoint(1,0.2)@2001-01-01, NPoint(1,0.2)@2001-01-03]}
  ```

- Constructor for temporal network points of sequence subtype

  ```
  tnpointSeq(tnpointInst[],interp text={'step','linear'},lower_inc boolean=true,
    upper_inc boolean=true) → tnpointSeq
  ```

  ```
  SELECT tnpointSeq(ARRAY[tnpoint 'Npoint(1, 0.3)@2001-01-01', 'Npoint(1, 0.5)@2001-01-02',
    'Npoint(1, 0.5)@2001-01-03']);
  -- {NPoint(1,0.3)@2001-01-01, NPoint(1,0.5)@2001-01-02, NPoint(1,0.5)@2001-01-03}
  SELECT tnpointSeq(ARRAY[tnpoint 'Npoint(1, 0.2)@2001-01-01', 'Npoint(1, 0.4)@2001-01-02',
    'Npoint(1, 0.5)@2001-01-03']);
  -- [NPoint(1,0.2)@2001-01-01, NPoint(1,0.4)@2001-01-02, NPoint(1,0.5)@2001-01-03]
  ```

- Constructor for temporal network points of sequence set subtype

```
tnpointSeqSet(tnpoint[]) → tnpointSeqSet
tnpointSeqSetGaps(tnpointInst[],maxt interval=NULL,maxdist float=NULL,
  interp text='linear') → tnpointSeqSet
```

```
SELECT tnpointSeqSet(ARRAY[tnpoint '[Npoint(1,0.2)@2001-01-01, Npoint(1,0.4)@2001-01-02,
  Npoint(1,0.5)@2001-01-03]', '[Npoint(2,0.6)@2001-01-04, Npoint(2,0.6)@2001-01-05]']);
/* {[NPoint(1,0.2)@2001-01-01, NPoint(1,0.4)@2001-01-02, NPoint(1,0.5)@2001-01-03],
   [NPoint(2,0.6)@2001-01-04, NPoint(2,0.6)@2001-01-05]} */
SELECT tnpointSeqSetGaps(ARRAY[tnpoint 'NPoint(1,0.1)@2001-01-01',
  'NPoint(1,0.3)@2001-01-03', 'NPoint(1,0.5)@2001-01-05'], '1 day');
-- {[NPoint(1,0.1)@2001-01-01], [NPoint(1,0.3)@2001-01-03], [NPoint(1,0.5)@2001-01-05]}
```

11.5 Conversions

A temporal network point value can be converted to and from a temporal geometry point. This can be done using an explicit `CAST` or using the `::` notation. A null value is returned if any of the composing geometry point values cannot be converted into a `npoint` value.

- Convert between a temporal network point and a temporal geometry point

```
{tnpoint,tgeompoint}::{tgeompoint,tnpoint}
tnpoint::tgeogpoint
```

  The conversions follow the rule of [?listitem].

```
SELECT asText((tnpoint '[NPoint(1, 0.2)@2001-01-01,
  NPoint(1, 0.3)@2001-01-02)')::tgeompoint);
/* [POINT(23.057077727326 28.7666335767956)@2001-01-01,
   POINT(48.7117553116406 20.9256801894708)@2001-01-02) */
SELECT tgeompoint '[POINT(23.057077727326 28.7666335767956)@2001-01-01,
  POINT(48.7117553116406 20.9256801894708)@2001-01-02)'::tnpoint;
-- [NPoint(1,0.2)@2001-01-01, NPoint(1,0.3)@2001-01-02)
SELECT tgeompoint '[POINT(23.057077727326 28.7666335767956)@2001-01-01,
  POINT(48.7117553116406 20.9)@2001-01-02)'::tnpoint;
-- NULL
```

11.6 MF-JSON Input and Output

A temporal network point is written to and read from Moving-Features JSON. Each instant renders its network point as `{"route":<route>,"position":<position>}`, the route id as a JSON number and the position as a float in `[0, 1]`, matching the SQL accessor surface (`route`, `getPosition`).

- Output a temporal network point as MF-JSON and read it back; the round trip is lossless

```
asMFJSON(tnpoint,options integer,flags integer,maxdecimaldigits integer) → text
tnpointFromMFJSON(text) → tnpoint
```

  The output takes the options, flags, and decimal-digits arguments shared with the other temporal types.

```
SELECT tnpointFromMFJSON(asMFJSON(tnpoint 'NPoint(1, 0.5)@2001-01-01'))
  = tnpoint 'NPoint(1, 0.5)@2001-01-01';
-- true
```

11.7 Accessors

- Return the values

```
getValues(tnpoint) → npointset
```

```
SELECT getValues(tnpoint '{[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02)}');
-- {"NPoint(1,0.3)","NPoint(1,0.5)"}
SELECT getValues(tnpoint '{[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.3)@2001-01-02)}');
-- {"NPoint(1,0.3)"}
```

- Return the road identifiers

```
routes(tnpoint) → bigintset
```

```
SELECT routes(tnpoint '{NPoint(3, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02}');
-- {1, 3}
```

- Return the value at a timestamp

```
valueAtTimestamp(tnpoint,timestamptz) → npoint
```

```
SELECT valueAtTimestamp(tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03)',
  '2001-01-02');
-- NPoint(1,0.4)
```

- Return the length traversed by the temporal network point

```
length(tnpoint) → float
```

```
SELECT length(tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]');
-- 54.3757408468784
```

- Return the cumulative length traversed by the temporal network point

```
cumulativeLength(tnpoint) → tfloat
```

```
SELECT cumulativeLength(tnpoint '{[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02,
  NPoint(1, 0.5)@2001-01-03], [NPoint(1, 0.6)@2001-01-04, NPoint(1, 0.7)@2001-01-05]}');
/* {[0@2001-01-01, 54.3757408468784@2001-01-02, 54.3757408468784@2001-01-03],
   [54.3757408468784@2001-01-04, 81.5636112703177@2001-01-05]} */
```

- Return the speed of the temporal network point in units per second

```
speed({tnpointSeq,tnpointSeqSet}) → tfloatSeqSet
```

```
SELECT speed(tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.4)@2001-01-02,
  NPoint(1, 0.6)@2001-01-03]') * 3600 * 24;
/* Interp=Step;[21.4016800272077@2001-01-01, 14.2677866848051@2001-01-02,
   14.2677866848051@2001-01-03] */
```

11.8 Transformations

- Transform a temporal network point to another subtype

```
tnpointInst(tnpoint) → tnpointInst
tnpointSeq(tnpoint) → tnpointSeq
tnpointSeqSet(tnpoint) → tnpointSeqSet
```

```
SELECT tnpointSeq(tnpoint 'NPoint(1, 0.5)@2001-01-01', 'discrete');
-- {NPoint(1,0.5)@2001-01-01}
SELECT tnpointSeq(tnpoint 'NPoint(1, 0.5)@2001-01-01');
-- [NPoint(1,0.5)@2001-01-01]
SELECT tnpointSeqSet(tnpoint 'NPoint(1, 0.5)@2001-01-01');
-- {[NPoint(1,0.5)@2001-01-01]}
```

- Transform a temporal network point to another interpolation

```
setInterp(tnpoint,interp text) → tnpoint
```

```
SELECT setInterp(tnpoint 'NPoint(1,0.2)@2001-01-01','linear');
-- [NPoint(1,0.2)@2001-01-01]
SELECT setInterp(tnpoint '{[NPoint(1,0.1)@2001-01-01], [NPoint(1,0.2)@2001-01-02]}',
  'discrete');
-- {NPoint(1,0.1)@2001-01-01, NPoint(1,0.2)@2001-01-02}
```

- Round the fraction of the temporal network point to the number of decimal places

```
round(tnpoint,integer=0) → tnpoint
```

```
SELECT round(tnpoint '{[NPoint(1,0.123456789)@2001-01-01, NPoint(1,0.5)@2001-01-02)}', 6);
-- {[NPoint(1,0.123457)@2001-01-01, NPoint(1,0.5)@2001-01-02)}
```

11.9 Modifications

- Append a temporal instant or a temporal sequence

```
appendInstant(tnpoint,tnpointInst) → tnpoint
appendSequence(tnpoint,tnpointSeq) → tnpoint
```

```
SELECT asText(appendInstant(tnpoint 'Npoint(1, 0.1)@2001-01-01',
  'Npoint(1, 0.2)@2001-01-02'));
-- [NPoint(1,0.1)@2001-01-01, NPoint(1,0.2)@2001-01-02]
SELECT asText(appendSequence(tnpoint '[Npoint(1, 0.1)@2001-01-01]',
  '[Npoint(1, 0.2)@2001-01-02]'));
-- {[NPoint(1,0.1)@2001-01-01], [NPoint(1,0.2)@2001-01-02]}
```

- Merge the temporal network points

```
merge(tnpoint,tnpoint) → tnpoint
merge(tnpoint[]) → tnpoint
merge(tnpoint) → tnpoint
```

```
SELECT asText(merge(tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]',
  '[Npoint(1, 0.3)@2001-01-03, Npoint(1, 0.5)@2001-01-05]'));
-- [NPoint(1,0.1)@2001-01-01, NPoint(1,0.5)@2001-01-05]
SELECT asText(merge(ARRAY[tnpoint '[Npoint(1, 0.1)@2001-01-01,
  Npoint(1, 0.2)@2001-01-02]',
  '[Npoint(1, 0.2)@2001-01-02, Npoint(1, 0.3)@2001-01-03]']));
-- [NPoint(1,0.1)@2001-01-01, NPoint(1,0.3)@2001-01-03]
WITH temp(inst) AS (
  SELECT tnpoint 'Npoint(1, 0.1)@2001-01-01' UNION
  SELECT tnpoint 'Npoint(1, 0.2)@2001-01-02' UNION
  SELECT tnpoint 'Npoint(1, 0.3)@2001-01-03' )
SELECT asText(merge(inst)) FROM temp;
-- {NPoint(1,0.1)@2001-01-01, NPoint(1,0.2)@2001-01-02, NPoint(1,0.3)@2001-01-03}
```

11.10 Restrictions

- Restrict to (the complement of) a value or a set of values

```
atValue(tnpoint,base) → tnpoint
minusValue(tnpoint,base) → tnpoint
atValues(tnpoint,values) → tnpoint
minusValues(tnpoint,values) → tnpoint
```

```
SELECT atValue(tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-03]',
  'NPoint(2, 0.5)');
-- {[NPoint(2,0.5)@2001-01-02]}
SELECT minusValue(tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-03]',
  'NPoint(2, 0.5)');
/* {[NPoint(2,0.3)@2001-01-01, NPoint(2,0.5)@2001-01-02),
   (NPoint(2,0.5)@2001-01-02, NPoint(2,0.7)@2001-01-03]} */
```

- Restrict to (the complement of) a geometry

```
atGeometry(tnpoint,geometry) → tnpoint
minusGeometry(tnpoint,geometry) → tnpoint
```

```
SELECT atGeometry(tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-03]',
  'Polygon((40 40,40 50,50 50,50 40,40 40))');
SELECT minusGeometry(tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-03]',
  'Polygon((40 40,40 50,50 50,50 40,40 40))');
-- {(NPoint(2,0.342593)@2001-01-01 05:06:40.364673, NPoint(2,0.7)@2001-01-03]}
```

11.11 Spatial Operations

- Return the time-weighted centroid

```
twCentroid(tnpoint) → geometry(Point)
```

```
SELECT ST_AsText(twCentroid(tnpoint '{[NPoint(1, 0.3)@2001-01-01,
  NPoint(1, 0.5)@2001-01-02, NPoint(1, 0.5)@2001-01-03, NPoint(1, 0.7)@2001-01-04)}'));
-- POINT(79.9787466444847 46.2385558051041)
```

11.12 Bounding Box Operations

- Route operators, which compare the route identifiers a temporal network point visits: contained in, contains, overlaps, and equal

```
{bigint,bigintset,npoint,tnpoint} {?@, @?, @@, @=} {bigint,bigintset,npoint,
  tnpoint} → boolean
```

```
SELECT bigint '1' ?@ tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]';
-- true
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]' @? bigint '1';
-- true
SELECT tnpoint '{NPoint(1, 0.3)@2001-01-01, NPoint(3, 0.5)@2001-01-02}' @@ bigintset '{3,
  7}';
-- true
SELECT tnpoint '{NPoint(1, 0.3)@2001-01-01, NPoint(3, 0.5)@2001-01-02}' @= bigintset '{1,
  3}';
-- true
SELECT tnpoint '{NPoint(1, 0.3)@2001-01-01,
  NPoint(3, 0.5)@2001-01-02}' @= bigintset '{1}';
-- false
```

- Topological operators

```
{tstzspan,stbox,tnpoint} {&&, <@, @>, ~=, -|-} {tstzspan,stbox,tnpoint} → boolean
```

```
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]' &&
  tstzspan '[2001-01-02,2001-01-03]';
-- true
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]' @>
  stbox(npoint 'NPoint(1, 0.5)');
-- true
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01,
  NPoint(1, 0.5)@2001-01-03]' ~= tnpoint '[NPoint(1, 0.3)@2001-01-01,
  NPoint(1, 0.35)@2001-01-02, NPoint(1, 0.5)@2001-01-03]';
-- true
```

- Position operators

```
{stbox,tnpoint} {<<, &<, >>, &>} {stbox,tnpoint} → boolean
{stbox,tnpoint} {<<|, &<|, |>>, |&>} {stbox,tnpoint} → boolean
{tstzspan,stbox,tnpoint} {<<#, &<#, #>>, #&>} {tstzspan,stbox,tnpoint} → boolean
```

```
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-02]' <<
  stbox(npoint 'NPoint(1, 0.2)');
-- true
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01,
  NPoint(1, 0.5)@2001-01-02]' <<| stbox(npoint 'NPoint(1, 0.5)');
-- false
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-03, NPoint(1, 0.5)@2001-01-05]' #&>
  tstzspan '[2001-01-01,2001-01-03]';
-- true
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-03, NPoint(1, 0.3)@2001-01-05]' #>>
  tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.3)@2001-01-02]';
-- true
```

11.13 Distance Operations

Operands of different dimensions read the dimensions both of them have, see Section 8.3.

- Return the smallest distance ever

  ```
  {geo,npoint,tnpoint,stbox} |=| {geo,npoint,tnpoint,stbox} → float
  ```

  The operator `|=|` can be used for doing nearest neighbor searches using a GiST or an SP-GiST index (see Section 10.2).

  ```
  SELECT tnpoint '[NPoint(2, 0.3)@2001-01-01,
    NPoint(2, 0.7)@2001-01-02]' |=| geometry 'SRID=5676;Linestring(50 50,55 55)';
  -- 31.69220882252415
  SELECT tnpoint '[NPoint(2, 0.3)@2001-01-01,
    NPoint(2, 0.7)@2001-01-02]' |=| npoint 'NPoint(1, 0.5)';
  -- 19.49691305292373
  SELECT tnpoint '[NPoint(2, 0.3)@2001-01-01, NPoint(2, 0.7)@2001-01-02]' |=|
    tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.7)@2001-01-02]';
  -- 5.231180723735304
  ```

- Return the instant of the first temporal network point at which the two arguments are at the nearest distance

  ```
  nearestApproachInstant({geo,npoint,tnpoint},{geo,npoint,tnpoint}) → tnpoint
  ```

  ```
  SELECT nearestApproachInstant(tnpoint '[NPoint(2, 0.3)@2001-01-01,
    NPoint(2, 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)');
  -- NPoint(2,0.349928)@2001-01-01 02:59:44.402905
  SELECT nearestApproachInstant(tnpoint '[NPoint(2, 0.3)@2001-01-01,
    NPoint(2, 0.7)@2001-01-02]', npoint 'NPoint(1, 0.5)');
  -- NPoint(2,0.592181)@2001-01-01 17:31:51.080405
  ```

- Return the line connecting the nearest approach point between the two arguments

  ```
  shortestLine({geo,npoint,tpoint},{geo,npoint,tpoint}) → geometry
  ```

  The function will only return the first line that it finds if there are more than one

  ```
  SELECT ST_AsText(shortestLine(tnpoint '[NPoint(2, 0.3)@2001-01-01,
    NPoint(2, 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
  -- LINESTRING(50.7960725266492 48.8266286733015,50 50)
  SELECT ST_AsText(shortestLine(tnpoint '[NPoint(2, 0.3)@2001-01-01,
    NPoint(2, 0.7)@2001-01-02]', npoint 'NPoint(1, 0.5)'));
  -- LINESTRING(77.0902838115125 66.6659083092593,90.8134936900394 46.4385792121146)
  ```

- Return the temporal distance

  ```
  tnpoint <-> tnpoint → tfloat
  ```

  ```
  SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' <->
    npoint 'NPoint(1, 0.2)';
  -- [2.34988300875063@2001-01-02, 2.34988300875063@2001-01-03]
  SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' <->
    geometry 'Point(50 50)';
  -- [25.0496666945044@2001-01-01, 26.4085688426232@2001-01-03]
  SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' <->
    tnpoint '[NPoint(1, 0.3)@2001-01-02, NPoint(1, 0.5)@2001-01-04]';
  -- [2.34988300875063@2001-01-02, 2.34988300875063@2001-01-03]
  ```

11.14 Ever and Always Relationships

- Ever and always relationships

```
eContains(geometry,tnpoint) → boolean
eCovers(geometry,tnpoint) → boolean
eDisjoint({geometry,tnpoint},{geometry,tnpoint}) → boolean
eDwithin({geometry,tnpoint},{geometry,tnpoint},float) → boolean
eIntersects({geometry,tnpoint},{geometry,tnpoint}) → boolean
eTouches(geometry,tnpoint) → boolean
eTouches(tnpoint,geometry) → boolean
aContains(geometry,tnpoint) → boolean
aCovers(geometry,tnpoint) → boolean
aDisjoint({geometry,tnpoint},{geometry,tnpoint}) → boolean
aDwithin({geometry,tnpoint},{geometry,tnpoint},float) → boolean
aIntersects({geometry,tnpoint},{geometry,tnpoint}) → boolean
aTouches(geometry,tnpoint) → boolean
aTouches(tnpoint,geometry) → boolean
```

The *ever* and *always* relationships determine whether the topological or distance relationship is ever or always satisfied and result in a `boolean`. A temporal network point answers them at the position its route and fraction resolve to, so a sequence with linear interpolation is answered along its route rather than at its instants alone. The temporal network point declares the relationships a moving point answers: `eContains`, `aContains`, `eCovers` and `aCovers` take the geometry first only, and `eTouches` and `aTouches` have no direction between two temporal network points, since a moving point neither contains nor covers a geometry and two moving points do not touch.

```
SELECT eContains(geometry 'SRID=5676;Polygon((0 0,0 50,50 50,50 0,0 0))',
  tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]');
-- false
SELECT eCovers(geometry 'SRID=5676;Polygon((0 0,0 50,50 50,50 0,0 0))',
  tnpoint 'Npoint(1, 0.1)@2001-01-01');
-- false
SELECT eDisjoint(geometry(npoint 'NPoint(2, 0.0)'),
  tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]');
-- true
SELECT eIntersects(tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]',
  tnpoint '[Npoint(2, 0.0)@2001-01-01, Npoint(2, 1)@2001-01-03]');
-- false
SELECT eTouches(tnpoint 'Npoint(1, 0.1)@2001-01-01', geometry 'SRID=5676;
  Polygon((0 0,0 50,50 50,50 0,0 0))');
-- false
SELECT aDwithin(tnpoint '[Npoint(1, 0.3)@2001-01-01, Npoint(1, 0.5)@2001-01-03]',
  tnpoint '[Npoint(1, 0.5)@2001-01-01, Npoint(1, 0.3)@2001-01-03]', 1);
-- false
```

11.15 Spatiotemporal Relationships

- Spatiotemporal relationships

```
tContains(geometry,tnpoint) → tbool
tCovers(geometry,tnpoint) → tbool
tDisjoint({geometry,tnpoint},{geometry,tnpoint}) → tbool
tDwithin({geometry,tnpoint},{geometry,tnpoint},float) → tbool
tIntersects({geometry,tnpoint},{geometry,tnpoint}) → tbool
tTouches(geometry,tnpoint) → tbool
tTouches(tnpoint,geometry) → tbool
```

The *temporal* relationships compute the topological or distance relationship at each instant and result in a `tbool`. A temporal network point answers them at the position its route and fraction resolve to. The temporal network point declares the relationships a moving point answers: `tContains` and `tCovers` take the geometry first only, and `tTouches` has no direction between two temporal network points, since a moving point neither contains nor covers a geometry and two moving points do not touch.

```
SELECT tContains(geometry 'SRID=5676;Polygon((0 0,0 50,50 50,50 0,0 0))',
  tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]');
-- {[f@2001-01-01, f@2001-01-03]}
SELECT tCovers(geometry 'SRID=5676;Polygon((0 0,0 50,50 50,50 0,0 0))',
  tnpoint 'Npoint(1, 0.1)@2001-01-01');
-- f@2001-01-01
SELECT tDisjoint(geometry(npoint 'NPoint(2, 0.0)'),
  tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]');
-- {[t@2001-01-01, t@2001-01-03]}
SELECT tTouches(tnpoint 'Npoint(1, 0.1)@2001-01-01', geometry 'SRID=5676;
  Polygon((0 0,0 50,50 50,50 0,0 0))');
-- f@2001-01-01
SELECT tIntersects(tnpoint '[Npoint(1, 0.1)@2001-01-01, Npoint(1, 0.3)@2001-01-03]',
  tnpoint '[Npoint(1, 0.3)@2001-01-01, Npoint(1, 0.1)@2001-01-03]');
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03]}
SELECT tDwithin(tnpoint '[Npoint(1, 0.3)@2001-01-01, Npoint(1, 0.5)@2001-01-03]',
  tnpoint '[Npoint(1, 0.5)@2001-01-01, Npoint(1, 0.3)@2001-01-03]', 1);
/* {[f@2001-01-01, t@2001-01-01 22:19:04.400634,
   t@2001-01-02 01:40:55.599365], (f@2001-01-02 01:40:55.599365, f@2001-01-03]} */
```

11.16 Comparisons

- Traditional comparisons

```
tnpoint {=, <>, <, >, <=, >=} tnpoint → boolean
```

```
SELECT tnpoint '{[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-02),
  [NPoint(1, 0.3)@2001-01-02, NPoint(1, 0.5)@2001-01-03]}' =
  tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.5)@2001-01-03]';
-- true
SELECT tnpoint '{[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.5)@2001-01-03]}' <>
  tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.5)@2001-01-03]';
-- false
SELECT tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.5)@2001-01-03]' <
  tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.6)@2001-01-03]';
-- true
```

- Ever and always comparisons

```
{npoint,tnpoint} {?=, ?<>, %=, %<>} {npoint,tnpoint} → boolean
```

```
SELECT tnpoint '[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.4)@2001-01-04)' ?= Npoint(1, 0.3);
-- true
SELECT tnpoint '[Npoint(1, 0.2)@2001-01-01, Npoint(1, 0.2)@2001-01-04)' &= Npoint(1, 0.2);
-- true
```

- Temporal comparisons

```
{npoint,tnpoint} {#=, #<>} {npoint,tnpoint} → tbool
```

```
SELECT tnpoint '[NPoint(1, 0.2)@2001-01-01,
  NPoint(1, 0.4)@2001-01-03)' #= npoint 'NPoint(1, 0.3)';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tnpoint '[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.8)@2001-01-03)' #<>
  tnpoint '[NPoint(1, 0.3)@2001-01-01, NPoint(1, 0.7)@2001-01-03)';
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}
```

11.17 Aggregations

The three aggregate functions for temporal network points are illustrated next.

- Temporal count

```
tCount(tnpoint) → {tintSeq,tintSeqSet}
```

```
WITH Temp(temp) AS (
  SELECT tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-03)' UNION
  SELECT tnpoint '[NPoint(1, 0.2)@2001-01-02, NPoint(1, 0.4)@2001-01-04)' UNION
  SELECT tnpoint '[NPoint(1, 0.3)@2001-01-03, NPoint(1, 0.5)@2001-01-05)' )
SELECT tCount(Temp) FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 1@2001-01-04, 1@2001-01-05)}
```

- Window count

```
wCount(tnpoint) → {tintSeq,tintSeqSet}
```

```
WITH Temp(temp) AS (
  SELECT tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-03)' UNION
  SELECT tnpoint '[NPoint(1, 0.2)@2001-01-02, NPoint(1, 0.4)@2001-01-04)' UNION
  SELECT tnpoint '[NPoint(1, 0.3)@2001-01-03, NPoint(1, 0.5)@2001-01-05)' )
SELECT wCount(Temp, '1 day') FROM Temp;
/* {[1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 2@2001-01-04, 1@2001-01-05,
   1@2001-01-06)} */
```

- Temporal centroid

```
tCentroid(tnpoint) → tgeompoint
```

```
WITH Temp(temp) AS (
SELECT tnpoint '[NPoint(1, 0.1)@2001-01-01, NPoint(1, 0.3)@2001-01-03)' UNION
SELECT tnpoint '[NPoint(1, 0.2)@2001-01-01, NPoint(1, 0.4)@2001-01-03)' UNION
SELECT tnpoint '[NPoint(1, 0.3)@2001-01-01,
  NPoint(1, 0.5)@2001-01-03)' ) SELECT astext(tCentroid(Temp)) FROM Temp;
/* {[POINT(72.451531682218 76.5231414472853)@2001-01-01,
   POINT(55.7001249027598 72.9552602410653)@2001-01-03)} */
```

11.18 Indexing

GiST and SP-GiST indexes can be created for table columns of temporal networks points. An example of index creation is follows:

```
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist(Trip);
```

The GiST and SP-GiST indexes store the bounding box for the temporal network points, which is an `stbox` and thus stores the absolute coordinates of the underlying space.

A GiST or SP-GiST index can accelerate queries involving the following operators:

- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, which only consider the spatial dimension in temporal network points,
- `<<#`, `&<#`, `#&>`, `#>>`, which only consider the time dimension in temporal network points,
- `&&`, `@>`, `<@`, `~=`, `-|-`, and `|=|` , which consider as many dimensions as they are shared by the indexed column and the query argument.

These operators work on bounding boxes, not the entire values.

11.19 Aggregation

- Return the spatiotemporal bounding box covering every materialised position in a set of network points

  ```
  extent(tnpoint) → stbox
  ```

  The aggregate is polymorphic with the `extent` aggregates on the other spatial temporal types, so one query can compute a single box across a heterogeneous set of `tgeompoint`, `tgeometry`, and `tnpoint` columns. Each instant's spatial extent is resolved through the ways registry, which dominates the cost over a large input.

  ```
  SELECT extent(temp) FROM tbl_tnpoint;
  ```

Chapter 12

Temporal Circular Buffers

A static circular buffer type `cbuffer` represents a 2D point and a radius greater than or equal to 0 around the point. The radius represent the "impact" of the point on its surroundings. This impact could be interpreted as noise, pollution, or similar aspects depending on application requirements.

The `cbuffer` type serves as base type for defining the temporal circular buffer type `tcbuffer`. The `tcbuffer` type has similar functionality as the temporal point type `tgeompoint` with the exception that it only considers two dimensions. Thus, most functions and operators described before for the `tgeompoint` type are also applicable for the `tcbuffer` type. In addition, there are specific functions defined for the `tcbuffer` type.

Figure 12.1 illustrates the use of the `tcbuffer` type for modelling the circular buffer around the wind swath of tropical storms in 2024. The data is obtained from the National Oceanic and Atmospherique Administration (NOAA). As illustrated in the figure, the types `tgeompoint` and `tcbuffer` allow *linear* interpolation, while as we have seen in Figure 7.2, the type `tgeometry` only allows *step* interpolation.

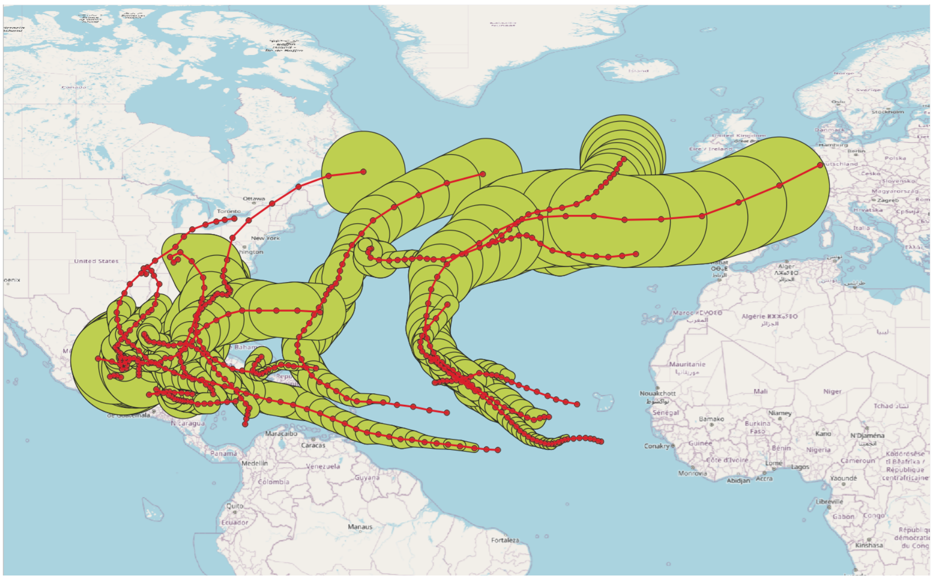

Figure 12.1: Illustration of the use of the `tcbuffer` type for modelling the circular buffer around the wind swath of tropical storms in 2024.

12.1 Static Circular Buffers

A `cbuffer` value is a couple of the form `(point,radius)` where `point` is a geometric point and `radius` is a `float` value greater than or equal to zero representing a radius around the point expressed using the units of the SRID of the point. Examples of input of circular buffer values are as follows:

```
SELECT cbuffer 'Cbuffer(Point(1 1), 0.3)';
SELECT cbuffer 'Cbuffer(Point(1 1), 10.0)';
```

The `cbuffer` type corresponds to the result of the PostGIS function ST_MinimumBoundingRadius.

Values of the circular buffer type must satisfy several constraints so that they are well defined. Examples of incorrect circular buffer type values are as follows.

```
-- Incorrect point value
SELECT cbuffer 'Cbuffer(Linestring(1 1,2 2), 1.0)';
-- Incorrect 3D point
SELECT cbuffer 'Cbuffer(Point Z(1 1 1), 1.0)';
-- Incorrect radius value
SELECT cbuffer 'Cbuffer(Point(1 1), -2.0)';
```

We give next the functions and operators for the circular buffer type.

12.1.1 Input and Output

- Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation

  ```
  asText({cbuffer,cbuffer[]}) → {text,text[]}
  asEWKT({cbuffer,cbuffer[]}) → {text,text[]}
  ```

  ```
  SELECT asText(cbuffer 'Cbuffer(Point(1 1), 0.5)');
  -- Cbuffer(POINT(1 1),0.5)
  SELECT asText(ARRAY[cbuffer 'Cbuffer(Point(1 1),0.2)', 'Cbuffer(Point(1 1),0.5)']);
  -- {"Cbuffer(POINT(1 1),0.2)","Cbuffer(POINT(1 1),0.5)"}
  SELECT asEWKT(cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.5)');
  -- SRID=5676;Cbuffer(POINT(1 1),0.5)
  SELECT asEWKT(ARRAY[cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.2)',
    'Cbuffer(SRID=5676;Point(1 1),0.5)']);
  -- {"SRID=5676;Cbuffer(POINT(1 1),0.2)","SRID=5676;Cbuffer(POINT(1 1),0.5)"}
  ```

- Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), the Hexadecimal Well-Known Binary (HexWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

  ```
  asBinary(cbuffer,endian text='') → bytea
  asEWKB(cbuffer,endian text='') → bytea
  asHexWKB(cbuffer,endian text='') → text
  asHexEWKB(cbuffer,endian text='') → text
  ```

  The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used. A circular buffer takes its SRID from its underlying point: `asEWKB` and `asHexEWKB` embed it, while the plain `asBinary` and `asHexWKB` forms omit it.

  ```
  SELECT asBinary(cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.5)');
  -- \x0100000000000000f03f000000000000f03f000000000000e03f
  SELECT asEWKB(cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.5)');
  -- \x01402c160000000000000000f03f000000000000f03f000000000000e03f
  SELECT asHexWKB(cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.5)');
  -- 0100000000000000F03F000000000000F03F000000000000E03F
  SELECT asHexEWKB(cbuffer 'Cbuffer(SRID=5676;Point(1 1),0.5)');
  -- 01402C160000000000000000F03F000000000000F03F000000000000E03F
  ```

- Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation

```
cbufferFromText(text) → cbuffer
cbufferFromEWKT(text) → cbuffer
```

```
SELECT asText(cbufferFromText(text 'Cbuffer(Point(1 1),0.5)'));
-- Cbuffer(POINT(1 1),0.5)
SELECT asEWKT(cbufferFromEWKT(text 'SRID=5676;Cbuffer(Point(1 1),0.5)'));
-- SRID=5676;Cbuffer(POINT(1 1),0.5)
```

- Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), from the Hexadecimal Well-Known Binary (HexWKB), or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

```
cbufferFromBinary(bytea) → cbuffer
cbufferFromEWKB(bytea) → cbuffer
cbufferFromHexWKB(text) → cbuffer
cbufferFromHexEWKB(text) → cbuffer
```

```
SELECT asEWKT(cbufferFromBinary(
  '\x0100000000000000f03f000000000000f03f000000000000e03f'));
-- Cbuffer(POINT(1 1),0.5)
SELECT asEWKT(cbufferFromEWKB(
  '\x01402c160000000000000000f03f000000000000f03f000000000000e03f'));
-- SRID=5676;Cbuffer(POINT(1 1),0.5)
SELECT asEWKT(cbufferFromHexEWKB(
  '01402C160000000000000000F03F000000000000F03F000000000000E03F'));
-- SRID=5676;Cbuffer(POINT(1 1),0.5)
```

12.1.2 Constructors

- Constructor for circular buffers

```
cbuffer(geompoint,float) → cbuffer
```

```
SELECT asText(cbuffer(ST_Point(1,1), 0.3));
-- Cbuffer(POINT(1 1),0.3)
```

12.1.3 Conversions

Values of the `cbuffer` type can be converted to the `geometry` type using an explicit `CAST` or using the `::` notation as shown below. Similarly, a `geometry` can be converted to a `cbuffer` value using the `ST_MinimumBoundingRadius` function in PostGIS.

- Convert between a circular buffer and a geometry

```
{geometry,cbuffer}::{cbuffer,geometry}
cbuffer::geography
```

As a geography, which has no curves, the circle is read as a polygon of 32 segments per quadrant, and it is not a circle on the surface of the earth. The conversions follow the rule of [?listitem].

```
SELECT ST_AsText(cbuffer(ST_Point(1,1), 1)::geometry);
-- CURVEPOLYGON(CIRCULARSTRING(0 1,2 1,0 1))
SELECT asText(geometry 'SRID=5676;CIRCULARSTRING(0 1,2 1,0 1)'::cbuffer);
-- Cbuffer(POINT(1 1),1)
SELECT asText(geometry 'SRID=5676;LINESTRING(0 0,2 2)'::cbuffer);
-- Cbuffer(POINT(1 1),1.414213562373095)
```

- Convert a circular buffer and, optionally, a timestamp or a period, to a spatiotemporal box

```
stbox(cbuffer) → stbox
stbox(cbuffer,{timestamptz,tstzspan}) → stbox
```

```
SELECT stbox(cbuffer 'SRID=5676;Cbuffer(Point(1 1),0.3)');
-- SRID=5676;STBOX X((0.7,0.7),(1.3,1.3))
SELECT stbox(cbuffer 'Cbuffer(Point(1 1),0.3)', timestamptz '2001-01-01');
-- STBOX XT(((0.7,0.7),(1.3,1.3)),[2001-01-01, 2001-01-01])
SELECT stbox(cbuffer 'Cbuffer(Point(1 1),0.3)', tstzspan '[2001-01-01,2001-01-02]');
-- STBOX XT(((0.7,0.7),(1.3,1.3)),[2001-01-01, 2001-01-02])
```

12.1.4 Accessors

- Return the point

```
point(cbuffer) → geompoint
```

```
SELECT ST_AsText(point(cbuffer 'Cbuffer(Point(1 1), 0.3)'));
-- POINT(1 1)
```

- Return the radius

```
radius(cbuffer) → float
```

```
SELECT radius(cbuffer 'Cbuffer(Point(1 1), 0.3)');
-- 0.3
```

12.1.5 Transformations

- Round the point and the radius of the circular buffer to the number of decimal places

```
round({cbuffer,cbuffer[]},integer=0) → {cbuffer,cbuffer[]}
```

```
SELECT asText(round(cbuffer(ST_Point(1.123456789,1.123456789), 0.123456789), 6));
-- Cbuffer(POINT(1.123457 1.123457),0.123457)
SELECT asText(round(ARRAY[cbuffer(ST_Point(1.123456789,1.123456789), 0.123456789),
  cbuffer(ST_Point(2.987654321,2.987654321), 0.987654321)], 6));
-- {"Cbuffer(POINT(1.123457 1.123457),0.123457)","Cbuffer(POINT(2.987654 2.987654) ↩
    ,0.987654)"}
```

12.1.6 Spatial Operations

- Return or set the spatial reference identifier

```
SRID(cbuffer) → integer
setSRID(cbuffer,integer) → cbuffer
```

```
SELECT SRID(cbuffer 'Cbuffer(SRID=5676;Point(1 1), 0.3)');
-- 5676
SELECT asEWKT(setSRID(cbuffer 'Cbuffer(Point(0 0),1)', 4326));
-- SRID=4326;Cbuffer(POINT(0 0),1)
```

- Transform to a spatial reference identifier

```
transform(cbuffer,integer) → cbuffer
transformPipeline(cbuffer,pipeline text,srid integer,
  is_forward boolean=true) → cbuffer
```

The `transform` function specifies the transformation with a target SRID. An error is raised when the input circular buffer has an unknown SRID (represented by 0).

The `transformPipeline` function specifies the transformation with a defined coordinate transformation pipeline represented with the following string format:

`urn:ogc:def:coordinateOperation:AUTHORITY::CODE`

The SRID of the input circular buffer is ignored, and the SRID of the output circular buffer will be set to zero unless a value is provided via the optional `srid` parameter. As stated by the last parameter, the pipeline is executed by default in a forward direction; by setting the parameter to false, the pipeline is executed in the inverse direction.

```
SELECT asEWKT(transform(cbuffer 'SRID=4326;Cbuffer(Point(4.35 50.85),1)', 3812));
-- SRID=3812;Cbuffer(POINT(648679.0180353033 671067.0556381135),1)
```

```
WITH test(cbuffer, pipeline) AS (
  SELECT cbuffer 'Cbuffer(SRID=4326;Point(4.3525 50.846667),1)',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(cbuffer, pipeline, 4326), pipeline,
  4326, false), 6) FROM test;
-- SRID=4326;Cbuffer(POINT(4.3525 50.846667),1)
```

12.1.7 Distance Operations

- Return the distance

```
distance({geo,cbuffer},cbuffer) → float
distance(cbuffer,{geo,cbuffer}) → float
{geo,cbuffer} <-> cbuffer → float
```

The distance between two circular buffers is the distance between their points reduced by both radii, and is zero when the buffers overlap. The result is `NULL` when the geometry is empty.

```
SELECT round(distance(geometry 'Point(1 0)', cbuffer 'Cbuffer(Point(4 0),0.5)'), 6);
-- 2.5
SELECT round(cbuffer 'Cbuffer(Point(0 0),0.5)' <-> cbuffer 'Cbuffer(Point(3 4),0.5)', 6);
-- 4
SELECT round(cbuffer 'Cbuffer(Point(0 0),3)' <-> cbuffer 'Cbuffer(Point(3 4),3)', 6);
-- 0
```

- Return the nearest approach distance

```
nearestApproachDistance({stbox,cbuffer},{cbuffer,stbox}) → float
{stbox,cbuffer} |=| {cbuffer,stbox} → float
```

The value is the least distance over the time the two arguments share, and is `NULL` when they share none. A circular buffer carries no period, so it is present at every time the box spans.

```
SELECT round(nearestApproachDistance(stbox 'STBOX X((1,-1),(3,1))', cbuffer 'Cbuffer(Point ↩
    (4 0),0.5)'), 6);
-- 0.5
SELECT round(cbuffer 'Cbuffer(Point(4 0),0.5)' |=| stbox 'STBOX X((1,-1),(3,1))', 6);
-- 0.5
```

12.1.8 Comparisons

The comparison operators (=, <, and so on) are available for circular buffers. They compare first the points and then the radius of the arguments. Excepted the equality and inequality, the other comparison operators are not useful in the real world but allow B-tree indexes to be constructed on circular buffers.

- Traditional comparisons

```
cbuffer {=, <>, <, >, <=, >=} cbuffer
```

```
SELECT cbuffer 'Cbuffer(Point(3 3), 0.5)' = cbuffer 'Cbuffer(Point(3 3), 0.5)';
-- true
SELECT cbuffer 'Cbuffer(Point(3 3), 0.5)' <> cbuffer 'Cbuffer(Point(3 3), 0.6)';
-- true
SELECT cbuffer 'Cbuffer(Point(3 3), 0.5)' < cbuffer 'Cbuffer(Point(3 3), 0.6)';
-- true
SELECT cbuffer 'Cbuffer(Point(3 3), 0.6)' > cbuffer 'Cbuffer(Point(2 2), 0.6)';
-- true
SELECT cbuffer 'Cbuffer(Point(1 1), 0.5)' <= cbuffer 'Cbuffer(Point(2 2), 0.5)';
-- true
SELECT cbuffer 'Cbuffer(Point(1 1), 0.6)' >= cbuffer 'Cbuffer(Point(1 1), 0.5)';
-- true
```

12.2 Temporal Circular Buffers

The temporal circular buffer type `tcbuffer` allows to represent the movement of objects together with a circular radius around them. As all temporal types, it comes in three subtypes, namely, instant, sequence, and sequence set. Examples of `tcbuffer` values in these subtypes are given next.

```
SELECT tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01';
SELECT tcbuffer '{Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-03}';
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.4)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-03]';
SELECT tcbuffer '{[Cbuffer(Point(1 1), 1)@2001-01-01, Cbuffer(Point(2 2), 1)@2001-01-02],
  [Cbuffer(Point(2 2), 2)@2001-01-04, Cbuffer(Point(2 2), 3)@2001-01-05]}';
```

The temporal circular buffer type accepts type modifiers (or `typmod` in PostgreSQL terminology) to specify the subtype and/or the spatial reference identifier (SRID). The possible values for the subtype are `Instant`, `Sequence`, and `SequenceSet`. The two arguments are optional and if any of them is not specified for a column, values of any subtype and/or SRID are allowed.

```
SELECT asEWKT(tcbuffer(Sequence,5676) 'SRID=5676;
  [Cbuffer(Point(1 1), 0.2)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-03]');
-- SRID=5676;[Cbuffer(POINT(1 1),0.2)@2001-01-01, Cbuffer(POINT(1 1),0.5)@2001-01-03]
SELECT tcbuffer(Sequence) 'Cbuffer(Point(1 1), 0.2)@2001-01-01';
-- ERROR: Temporal subtype (Instant) does not match column type (Sequence)
SELECT tcbuffer(5676) 'Cbuffer(Point(1 1), 0.2)@2001-01-01';
-- ERROR:  Temporal circular buffer SRID (0) does not match column SRID (5676)
```

Temporal circular buffer values of sequence or sequence set subtype are converted into a normal form so that equivalent values have identical representations. For this, consecutive instant values are merged when possible. Three consecutive instant values can be merged into two if the linear functions defining the evolution of values are the same. Examples of transformation into a normal form are as follows.

```
SELECT asText(tcbuffer '[Cbuffer(Point(1 1), 0.25)@2001-01-01,
  Cbuffer(Point(2 2), 0.5)@2001-01-02, Cbuffer(Point(3 3), 0.75)@2001-01-03)');
-- [Cbuffer(POINT(1 1),0.25)@2001-01-01, Cbuffer(POINT(3 3),0.75)@2001-01-03)
SELECT asText(tcbuffer '{[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(2 2), 0.3)@2001-01-02, Cbuffer(Point(2 2), 0.5)@2001-01-03),
  [Cbuffer(Point(2 2), 0.5)@2001-01-03, Cbuffer(Point(2 2), 0.7)@2001-01-04)}');
/* {[Cbuffer(POINT(1 1),0.2)@2001-01-01, Cbuffer(POINT(2 2),0.3)@2001-01-02,
   Cbuffer(POINT(2 2),0.7)@2001-01-04)} */
```

12.3 Validity of Temporal Circular Buffers

Temporal circular buffer values must satisfy the constraints specified in Section 4.3 so that they are well defined. An error is raised whenever one of these constraints are not satisfied. Examples of incorrect values are as follows.

```
-- Null values are not allowed
SELECT tcbuffer 'NULL@2001-01-01 08:05:00';
SELECT tcbuffer 'Point(0 0)@NULL';
-- Base type is not a circular buffer
SELECT tcbuffer 'Point(0 0)@2001-01-01 08:05:00';
```

We give next the functions and operators for temporal circular buffer. Most functions and operators for temporal types described in the previous chapters can be applied for temporal circular buffer types. Therefore, in the signatures of the functions, the notation `base` represents a `cbuffer` and the notations `ttype`, `tpoint`, and `tgeompoint` also represent a `tcbuffer`. Furthermore, the functions that have an argument of type `geometry` accept in addition an argument of type `cbuffer`. To avoid redundancy, we only present next some examples of these functions and operators for temporal circular buffers.

12.4 Input and Output

- Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation

  ```
  asText({tcbuffer,tcbuffer[],cbuffer[]}) → {text,text[]}
  asEWKT({tcbuffer,tcbuffer[],cbuffer[]}) → {text,text[]}
  ```

  ```
  SELECT asText(tcbuffer 'SRID=4326;
    [Cbuffer(Point(0 0),1)@2001-01-01, Cbuffer(Point(1 1),2)@2001-01-02)');
  -- [Cbuffer(POINT(0 0),1)@2001-01-01, Cbuffer(POINT(1 1),2)@2001-01-02)
  SELECT asText(ARRAY[cbuffer 'Cbuffer(Point(0 0),1)', 'Cbuffer(Point(1 1),2)']);
  -- {"Cbuffer(POINT(0 0),1)","Cbuffer(POINT(1 1),2)"}
  SELECT asEWKT(tcbuffer 'SRID=4326;
    [Cbuffer(Point(0 0),1)@2001-01-01, Cbuffer(Point(1 1),2)@2001-01-02)');
  -- SRID=4326;[Cbuffer(POINT(0 0),1)@2001-01-01, Cbuffer(POINT(1 1),2)@2001-01-02)
  SELECT asEWKT(ARRAY[cbuffer 'Cbuffer(SRID=5676;Point(0 0),1)',
    'Cbuffer(SRID=5676;Point(1 1),2)']);
  -- {"SRID=5676;Cbuffer(POINT(0 0),1)","SRID=5676;Cbuffer(POINT(1 1),2)"}
  ```

- Return the Moving Features JSON (MF-JSON) representation

  ```
  asMFJSON(tcbuffer) → text
  ```

  A circular buffer is always planar 2D. Each value is an object with a `point` member, an `[x, y]` coordinate array, and a numeric `radius` member, mirroring the `point` and `radius` accessors.

  ```
  SELECT asMFJSON(tcbuffer 'Cbuffer(Point(1 2),0.5)@2001-01-01', 3);
  /* {"type":"MovingCircularBuffer","bbox":[[0.5,1.5],[1.5,2.5]],
     "period":{"begin":"2001-01-01T00:00:00","end":"2001-01-01T00:00:00",
     "lower_inc":true,"upper_inc":true}, "values":[{"point":[1,2],"radius":0.5}],
     "datetimes":["2001-01-01T00:00:00"],"interpolation":"None"} */
  SELECT asMFJSON(tcbuffer 'SRID=4326;
    [Cbuffer(Point(1 1),0.2)@2001-01-01, Cbuffer(Point(2 2),0.4)@2001-01-02)', 3);
  /* {"type":"MovingCircularBuffer","crs":{"type":"Name",
  ```

```
   "properties":{"name":"EPSG:4326"}},"bbox":[[0.8,0.8],[2.4,2.4]],
   "period":{"begin":"2001-01-01T00:00:00","end":"2001-01-02T00:00:00",
   "lower_inc":true,"upper_inc":false},
   "values":[{"point":[1,1],"radius":0.2},{"point":[2,2],"radius":0.4}],
   "datetimes":["2001-01-01T00:00:00","2001-01-02T00:00:00"],
   "lower_inc":true,"upper_inc":false,"interpolation":"Linear"} */
```

- Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), the Hexadecimal Well-Known Binary (HexWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

  ```
  asBinary(tcbuffer,endian text='') → bytea
  asEWKB(tcbuffer,endian text='') → bytea
  asHexWKB(tcbuffer,endian text='') → text
  asHexEWKB(tcbuffer,endian text='') → text
  ```

  The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used.

  ```
  SELECT asBinary(tcbuffer 'Cbuffer(Point(1 2),1)@2001-01-01');
  -- \x013b0001000000000000f03f0000000000000040000000000000f03f009c57d3c11c0000
  SELECT asEWKB(tcbuffer 'SRID=7844;Cbuffer(Point(1 2),1)@2001-01-01');
  -- \x013b0041a41e0000000000000000f03f0000000000000040000000000000f03f009c57d3c11c0000
  SELECT asHexWKB(tcbuffer 'Cbuffer(Point(1 2),1)@2001-01-01');
  -- 013B0001000000000000F03F0000000000000040000000000000F03F009C57D3C11C0000
  SELECT asHexEWKB(tcbuffer 'SRID=3812;Cbuffer(Point(1 2),1)@2001-01-01');
  -- 013B0041E40E0000000000000000F03F0000000000000040000000000000F03F009C57D3C11C0000
  ```

- Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation

  ```
  tcbufferFromText(text) → tcbuffer
  tcbufferFromEWKT(text) → tcbuffer
  ```

  ```
  SELECT asEWKT(tcbufferFromText(text '[Cbuffer(Point(1 2),1)@2001-01-01,
    Cbuffer(Point(3 4),2)@2001-01-02]'));
  -- [Cbuffer(POINT(1 2),1)@2001-01-01, Cbuffer(POINT(3 4),2)@2001-01-02]
  SELECT asEWKT(tcbufferFromEWKT(text 'SRID=3812;
    [Cbuffer(Point(1 2),1)@2001-01-01, Cbuffer(Point(3 4),2)@2001-01-02]'));
  -- SRID=3812;[Cbuffer(POINT(1 2),1)@2001-01-01, Cbuffer(POINT(3 4),2)@2001-01-02]
  ```

- Input from the Moving Features JSON (MF-JSON) representation

  ```
  tcbufferFromMFJSON(text) → tcbuffer
  ```

  ```
  SELECT asEWKT(tcbufferFromMFJSON(asMFJSON(tcbuffer 'SRID=3812;
    Cbuffer(Point(1 2),0.5)@2001-01-01', 3)));
  -- SRID=3812;Cbuffer(POINT(1 2),0.5)@2001-01-01
  SELECT asEWKT(tcbufferFromMFJSON(asMFJSON(tcbuffer 'SRID=5676;
    [Cbuffer(Point(1 2),1)@2001-01-01, Cbuffer(Point(3 4),2)@2001-01-02]', 3)));
  -- SRID=5676;[Cbuffer(POINT(1 2),1)@2001-01-01, Cbuffer(POINT(3 4),2)@2001-01-02]
  ```

- Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), from the Hexadecimal Well-Known Binary (HexWKB), or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

  ```
  tcbufferFromBinary(bytea) → tcbuffer
  tcbufferFromEWKB(bytea) → tcbuffer
  tcbufferFromHexWKB(text) → tcbuffer
  tcbufferFromHexEWKB(text) → tcbuffer
  ```

```
SELECT asEWKT(tcbufferFromBinary(
  '\x013b0001000000000000f03f0000000000000040000000000000f03f009c57d3c11c0000'));
-- Cbuffer(POINT(1 2),1)@2001-01-01
SELECT asEWKT(tcbufferFromEWKB(
  '\x013b0041a41e0000000000000000f03f0000000000000040000000000000f03f009c57d3c11c0000'));
-- SRID=7844;Cbuffer(POINT(1 2),1)@2001-01-01
SELECT asEWKT(tcbufferFromHexEWKB(
  '013B0041E40E0000000000000000F03F0000000000000040000000000000F03F009C57D3C11C0000'));
-- SRID=3812;Cbuffer(POINT(1 2),1)@2001-01-01
```

12.5 Constructors

- Constructor for temporal circular buffers having a constant value

```
tcbuffer(cbuffer,timestamptz) → tcbufferInst
tcbuffer(cbuffer,tstzset) → tcbufferDiscSeq
tcbuffer(cbuffer,tstzspan,interp text='linear') → tcbufferContSeq
tcbuffer(cbuffer,tstzspanset,interp text='linear') → tcbufferSeqSet
```

```
SELECT asText(tcbuffer('Cbuffer(Point(1 1), 0.5)', timestamptz '2001-01-01'));
-- Cbuffer(POINT(1 1),0.5)@2001-01-01
SELECT asText(tcbuffer('Cbuffer(Point(1 1), 0.3)',
  tstzset '{2001-01-01, 2001-01-03, 2001-01-05}'));
/* {Cbuffer(POINT(1 1),0.3)@2001-01-01, Cbuffer(POINT(1 1),0.3)@2001-01-03,
   Cbuffer(POINT(1 1),0.3)@2001-01-05} */
SELECT asText(tcbuffer('Cbuffer(Point(1 1), 0.5)', tstzspan '[2001-01-01, 2001-01-02]'));
-- [Cbuffer(POINT(1 1),0.5)@2001-01-01, Cbuffer(POINT(1 1),0.5)@2001-01-02]
SELECT asText(tcbuffer('Cbuffer(Point(1 1), 0.2)',
  tstzspanset '{[2001-01-01, 2001-01-03]}', 'step'));
-- Interp=Step;{[Cbuffer(POINT(1 1),0.2)@2001-01-01, Cbuffer(POINT(1 1),0.2)@2001-01-03]}
```

- Constructor for temporal circular buffers of sequence subtype

```
tcbufferSeq(tcbufferInst[],interp text='linear',lower_inc boolean=true,
  upper_inc boolean=true) → tcbufferSeq
```

```
SELECT asText(tcbufferSeq(ARRAY[tcbuffer 'Cbuffer(Point(1 1), 0.3)@2001-01-01',
  'Cbuffer(Point(2 2), 0.5)@2001-01-02', 'Cbuffer(Point(1 1), 0.5)@2001-01-03']));
/* [Cbuffer(POINT(1 1),0.3)@2001-01-01, Cbuffer(POINT(2 2),0.5)@2001-01-02,
   Cbuffer(POINT(1 1),0.5)@2001-01-03] */
SELECT asText(tcbufferSeq(ARRAY[tcbuffer 'Cbuffer(Point(1 1), 0.3)@2001-01-01',
  'Cbuffer(Point(2 2), 0.5)@2001-01-02', 'Cbuffer(Point(1 1), 0.5)@2001-01-03'],
  'discrete'));
/* {Cbuffer(POINT(1 1),0.3)@2001-01-01, Cbuffer(POINT(2 2),0.5)@2001-01-02,
   Cbuffer(POINT(1 1),0.5)@2001-01-03} */
SELECT asText(tcbufferSeq(ARRAY[tcbuffer 'Cbuffer(Point(1 1), 0.2)@2001-01-01',
  'Cbuffer(Point(1 1), 0.4)@2001-01-02', 'Cbuffer(Point(1 1), 0.5)@2001-01-03'], 'step'));
/* Interp=Step;[Cbuffer(POINT(1 1),0.2)@2001-01-01, Cbuffer(POINT(1 1),0.4)@2001-01-02,
   Cbuffer(POINT(1 1),0.5)@2001-01-03] */
```

- Constructor for temporal circular buffers of sequence set subtype

```
tcbufferSeqSet(tcbuffer[]) → tcbufferSeqSet
tcbufferSeqSetGaps(tcbufferInst[],maxt interval=NULL,maxdist float=NULL,
  interp text='linear') → tcbufferSeqSet
```

```
SELECT asText(tcbufferSeqSet(ARRAY[tcbuffer '[Cbuffer(Point(1 1),0.2)@2001-01-01,
  Cbuffer(Point(2 2),0.4)@2001-01-02]',
  '[Cbuffer(Point(2 2),0.6)@2001-01-03, Cbuffer(Point(2 2),0.8)@2001-01-04]']));
/* {[Cbuffer(POINT(1 1),0.2)@2001-01-01, Cbuffer(POINT(2 2),0.4)@2001-01-02],
   [Cbuffer(POINT(2 2),0.6)@2001-01-03, Cbuffer(POINT(2 2),0.8)@2001-01-04]} */
SELECT asText(tcbufferSeqSetGaps(ARRAY[tcbuffer 'Cbuffer(Point(1 1),0.1)@2001-01-01',
  'Cbuffer(Point(1 1),0.3)@2001-01-03', 'Cbuffer(Point(1 1),0.5)@2001-01-05'], '1 day'));
/* {[Cbuffer(POINT(1 1),0.1)@2001-01-01], [Cbuffer(POINT(1 1),0.3)@2001-01-03],
   [Cbuffer(POINT(1 1),0.5)@2001-01-05]} */
```

- Construct a temporal circular buffer from a temporal geometry point and a temporal float

  ```
  tcbuffer(tgeompoint,tfloat) → tcbuffer
  ```

  The time frame of the result is the intersection of the time frames of the temporal point and the temporal float. If they do not intersect, a null value is returned

  ```
  SELECT asText(tcbuffer(tgeompoint '[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02)',
    tfloat '[1@2001-01-01, 2@2001-01-02)'));
  -- [Cbuffer(POINT(1 1),1)@2001-01-01, Cbuffer(POINT(2 2),2)@2001-01-02)
  SELECT asText(tcbuffer(tgeompoint '[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02)',
    tfloat '[2@2001-01-03, 3@2001-01-04)'));
  -- NULL
  ```

12.6 Conversions

A temporal circular buffer value can be converted to and from a temporal geometry point. This can be done using an explicit `CAST` or using the `::` notation. A null value is returned if any of the composing geometry point values cannot be converted into a `cbuffer` value.

- Convert a temporal circular buffer to a temporal geometry point

  ```
  tcbuffer::{tgeompoint,tgeogpoint}
  ```

  The conversions follow the rule of [?listitem].

  ```
  SELECT asText((tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
    Cbuffer(Point(1 1), 0.3)@2001-01-02)')::tgeompoint);
  -- [POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02)
  ```

- Convert a temporal circular buffer to a temporal float

  ```
  tcbuffer::tfloat
  ```

  ```
  SELECT (tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
    Cbuffer(Point(1 1), 0.3)@2001-01-02)')::tfloat;
  -- [0.2@2001-01-01, 0.3@2001-01-02)
  ```

12.7 Accessors

- Return the values

  ```
  getValues(tcbuffer) → cbufferset
  ```

```
SELECT asText(getValues(tcbuffer '{[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02)}'));
-- {"Cbuffer(POINT(1 1),0.3)", "Cbuffer(POINT(1 1),0.5)"}
SELECT asEWKT(getValues(tcbuffer 'SRID=5676;
  {[Cbuffer(Point(1 1), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.3)@2001-01-02)}'));
-- SRID=5676;{"Cbuffer(POINT(1 1),0.3)"}
```

- Return the points or the radii

```
points(tcbuffer) → geomset
radius(tcbuffer) → floatset
```

```
SELECT asEWKT(points(tcbuffer 'SRID=5676;
  {Cbuffer(Point(3 3), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-02}'));
-- SRID=5676;{"POINT(1 1)", "POINT(3 3)"}
SELECT radius(tcbuffer 'SRID=5676;
  {Cbuffer(Point(3 3), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-02}');
-- {0.3, 0.5}
```

- Return the value at a timestamp

```
valueAtTimestamp(tcbuffer,timestamptz) → cbuffer
```

```
SELECT asText(valueAtTimestamp(tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(3 3), 0.5)@2001-01-03)', '2001-01-02'));
-- Cbuffer(POINT(2 2),0.4)
```

12.8 Transformations

- Transform a temporal circular buffer to another subtype

```
tcbufferInst(tcbuffer) → tcbufferInst
tcbufferSeq(tcbuffer) → tcbufferSeq
tcbufferSeqSet(tcbuffer) → tcbufferSeqSet
```

```
SELECT asText(tcbufferSeq(tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01', 'discrete'));
-- {Cbuffer(POINT(1 1),0.5)@2001-01-01}
SELECT asText(tcbufferSeq(tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01'));
-- [Cbuffer(POINT(1 1),0.5)@2001-01-01]
SELECT asText(tcbufferSeqSet(tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01'));
-- {[Cbuffer(POINT(1 1),0.5)@2001-01-01]}
```

- Transform a temporal circular buffer to another interpolation

```
setInterp(tcbuffer,interp text) → tcbuffer
```

```
SELECT asText(setInterp(tcbuffer 'Cbuffer(Point(1 1),0.2)@2001-01-01','linear'));
-- [Cbuffer(POINT(1 1),0.2)@2001-01-01]
SELECT asText(setInterp(tcbuffer '{[Cbuffer(Point(1 1),0.1)@2001-01-01],
  [Cbuffer(Point(1 1),0.2)@2001-01-02]}', 'discrete'));
-- {Cbuffer(POINT(1 1),0.1)@2001-01-01, Cbuffer(POINT(1 1),0.2)@2001-01-02}
```

- Round the points and the radii of the temporal circular buffer to the number of decimal places

```
round(tcbuffer,integer) → tcbuffer
```

```
SELECT asText(round(tcbuffer '{[Cbuffer(Point(1 1.123456789),0.123456789)@2001-01-01,
  Cbuffer(Point(1 1),0.5)@2001-01-02)}', 3));
-- {[Cbuffer(POINT(1 1.123),0.123)@2001-01-01, Cbuffer(POINT(1 1),0.5)@2001-01-02)}
```

- Extract the subsequences where the temporal circular buffer stays within an area with a specified maximum size for at least the given duration

  ```
  stops(tcbuffer,maxdist float=0.0,minduration interval='0 minutes') → tcbuffer
  ```

  The size of the area is the greatest distance between two centres of the subsequence, and a stop that begins or ends at the first or last instant of a sequence includes that instant only if the sequence does, as stated for a temporal point in stops.

  ```
  SELECT asText(stops(tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
    Cbuffer(Point(1 1), 0.5)@2001-01-02, Cbuffer(Point(2 2), 0.5)@2001-01-03,
    Cbuffer(Point(2 2), 0.5)@2001-01-04]'));
  /* {[Cbuffer(POINT(1 1),0.5)@2001-01-01, Cbuffer(POINT(1 1),0.5)@2001-01-02],
     [Cbuffer(POINT(2 2),0.5)@2001-01-03, Cbuffer(POINT(2 2),0.5)@2001-01-04]} */
  ```

- Return a temporal value translated by given offsets

  ```
  translate(tcbuffer,deltax float,deltay float) → tcbuffer
  ```

  The motion moves the centre of every circular buffer and keeps its radius. A circular buffer is planar, so it takes no vertical offset and no rotation about the x or y axis.

  The motion is a planar map, so a geodetic value raises an error.

  ```
  SELECT asText(translate(tcbuffer '[Cbuffer(Point(1 1),0.5)@2001-01-01,
    Cbuffer(Point(3 2),1)@2001-01-02]', 1, 2), 6);
  -- [Cbuffer(POINT(2 3),0.5)@2001-01-01, Cbuffer(POINT(4 4),1)@2001-01-02]
  ```

- Return a temporal value rotated counter-clockwise about the origin or about a point

  ```
  rotate(tcbuffer,angle float) → tcbuffer
  rotate(tcbuffer,angle float,x0 float,y0 float) → tcbuffer
  ```

  The motion moves the centre of every circular buffer and keeps its radius. A circular buffer is planar, so it takes no vertical offset and no rotation about the x or y axis.

  The motion is a planar map, so a geodetic value raises an error.

  ```
  SELECT asText(rotate(tcbuffer '[Cbuffer(Point(1 1),0.5)@2001-01-01,
    Cbuffer(Point(3 2),1)@2001-01-02]', 0.7, 1, 1), 6);
  /* [Cbuffer(POINT(1 1),0.5)@2001-01-01,
     Cbuffer(POINT(1.885467 3.053278),1)@2001-01-02] */
  ```

- Return a temporal value rotated counter-clockwise about the z axis

  ```
  rotateZ(tcbuffer,angle float) → tcbuffer
  ```

  The motion moves the centre of every circular buffer and keeps its radius. A circular buffer is planar, so it takes no vertical offset and no rotation about the x or y axis.

  The motion is a planar map, so a geodetic value raises an error.

  ```
  SELECT asText(rotateZ(tcbuffer '[Cbuffer(Point(1 1),0.5)@2001-01-01,
    Cbuffer(Point(3 2),1)@2001-01-02]', 0.7), 6);
  /* [Cbuffer(POINT(0.120625 1.40906),0.5)@2001-01-01,
     Cbuffer(POINT(1.006091 3.462337),1)@2001-01-02] */
  ```

12.9 Modifications

- Append a temporal instant or a temporal sequence

```
appendInstant(tcbuffer,tcbufferInst) → tcbuffer
appendSequence(tcbuffer,tcbufferSeq) → tcbuffer
```

```
SELECT asText(appendInstant(tcbuffer 'Cbuffer(Point(1 1), 0.1)@2001-01-01',
  tcbuffer 'Cbuffer(Point(2 2), 0.2)@2001-01-02'));
-- [Cbuffer(POINT(1 1),0.1)@2001-01-01, Cbuffer(POINT(2 2),0.2)@2001-01-02]
SELECT asText(appendSequence(tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01]',
  '[Cbuffer(Point(2 2), 0.2)@2001-01-02]'));
-- {[Cbuffer(POINT(1 1),0.1)@2001-01-01], [Cbuffer(POINT(2 2),0.2)@2001-01-02]}
```

- Merge the temporal circular buffers

```
merge(tcbuffer,tcbuffer) → tcbuffer
merge(tcbuffer[]) → tcbuffer
merge(tcbuffer) → tcbuffer
```

```
SELECT asText(merge(tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-03]',
  '[Cbuffer(Point(1 1), 0.3)@2001-01-03, Cbuffer(Point(1 1), 0.5)@2001-01-05]'));
-- [Cbuffer(POINT(1 1),0.1)@2001-01-01, Cbuffer(POINT(1 1),0.5)@2001-01-05]
SELECT asText(merge(ARRAY[tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
  Cbuffer(Point(1 1), 0.2)@2001-01-02]',
  '[Cbuffer(Point(1 1), 0.2)@2001-01-02, Cbuffer(Point(1 1), 0.3)@2001-01-03]']));
-- [Cbuffer(POINT(1 1),0.1)@2001-01-01, Cbuffer(POINT(1 1),0.3)@2001-01-03]
WITH temp(inst) AS (
  SELECT tcbuffer 'Cbuffer(Point(1 1), 0.1)@2001-01-01' UNION
  SELECT tcbuffer 'Cbuffer(Point(2 2), 0.2)@2001-01-02' )
SELECT asText(merge(inst)) FROM temp;
-- {Cbuffer(POINT(1 1),0.1)@2001-01-01, Cbuffer(POINT(2 2),0.2)@2001-01-02}
```

12.10 Restrictions

- Restrict to (the complement of) a value or a set of values

```
atValue(tcbuffer,base) → tcbuffer
minusValue(tcbuffer,base) → tcbuffer
atValues(tcbuffer,values) → tcbuffer
minusValues(tcbuffer,values) → tcbuffer
```

```
SELECT asText(atValue(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-03]', cbuffer 'Cbuffer(Point(2 2), 0.5)'));
-- {[Cbuffer(POINT(2 2),0.5)@2001-01-02]}
SELECT asText(minusValue(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-03]', cbuffer 'Cbuffer(Point(2 2), 0.5)'));
/* {[Cbuffer(POINT(2 2),0.3)@2001-01-01, Cbuffer(POINT(2 2),0.5)@2001-01-02),
   (Cbuffer(POINT(2 2),0.5)@2001-01-02, Cbuffer(POINT(2 2),0.7)@2001-01-03]} */
```

- Restrict to (the complement of) a geometry

```
atGeometry(tcbuffer,geometry) → tcbuffer
minusGeometry(tcbuffer,geometry) → tcbuffer
```

```
SELECT asText(atGeometry(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-03]', 'Polygon((0 0,0 2,2 2,2 0,0 0))'));
-- {[Cbuffer(POINT(2 2),0.3)@2001-01-01, Cbuffer(POINT(2 2),0.7)@2001-01-03]}
SELECT asText(minusGeometry(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-03]', 'Polygon((0 0,0 2,2 2,2 0,0 0))'));
-- NULL
```

- Restrict a temporal circular buffer to (the complement of) a spatiotemporal box

```
atStbox(tcbuffer,stbox,borderInc boolean=true) → tcbuffer
minusStbox(tcbuffer,stbox,borderInc boolean=true) → tcbuffer
```

```
SELECT asText(atStbox(tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
  Cbuffer(Point(3 3), 0.5)@2001-01-03]', stbox 'STBOX X((0,0),(2,2))'));
/* {[Cbuffer(POINT(1 1),0.5)@2001-01-01,
   Cbuffer(POINT(2.353553390590278 2.353553390590278),0.5)@2001-01-02 08:29:07.012947]} */
```

12.11 Spatial Operations

- Return or set the spatial reference identifier

```
SRID(tcbuffer) → integer
setSRID(tcbuffer,integer) → tcbuffer
```

```
SELECT SRID(tcbuffer 'SRID=5676;
  [Cbuffer(Point(0 0),1)@2001-01-01, Cbuffer(Point(1 1),2)@2001-01-02)');
-- 5676
SELECT asEWKT(setSRID(tcbuffer '[Cbuffer(Point(0 0),1)@2001-01-01,
  Cbuffer(Point(1 1),2)@2001-01-02)', 5676));
-- SRID=5676;[Cbuffer(POINT(0 0),1)@2001-01-01, Cbuffer(POINT(1 1),2)@2001-01-02)
```

- Transform to a spatial reference identifier

```
transform(tcbuffer,integer) → tcbuffer
transformPipeline(tcbuffer,pipeline text,srid integer,
  is_forward boolean=true) → tcbuffer
```

The `transform` function specifies the transformation with a target SRID. An error is raised when the input temporal circular buffer has an unknown SRID (represented by 0). The `transformPipeline` function specifies the transformation with a defined coordinate transformation pipeline represented with the following string format: `urn:ogc:def:coordinateOperation:AUTHORITY::CODE`. The SRID of the input temporal circular buffer is ignored, and the SRID of the output temporal circular buffer will be set to zero unless a value is provided via the optional `srid` parameter. As stated by the last parameter, the pipeline is executed by default in a forward direction; by setting the parameter to false, the pipeline is executed in the inverse direction.

```
SELECT asEWKT(transform(tcbuffer 'SRID=4326;Cbuffer(Point(4.35 50.85),1)@2001-01-01',
  3812));
-- SRID=3812;Cbuffer(POINT(648679.0180353033 671067.0556381135),1)@2001-01-01
```

```
WITH test(tcbuffer, pipeline) AS (
  SELECT tcbuffer 'SRID=4326;{Cbuffer(Point(4.3525 50.846667),1)@2001-01-01,
    Cbuffer(Point(-0.1275 51.507222),2)@2001-01-02}',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(tcbuffer, pipeline, 4326), pipeline,
  4326, false), 6) FROM test;
/* SRID=4326;{Cbuffer(POINT(4.3525 50.846667),1)@2001-01-01,
   Cbuffer(POINT(-0.1275 51.507222),2)@2001-01-02} */
```

- Return the area traversed by the temporal circular buffer

```
traversedArea(tcbuffer,unary_union boolean=false) → geometry
```

```
SELECT ST_AsText(traversedArea(tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]'));
-- CURVEPOLYGON(CIRCULARSTRING(0.5 1,1.5 1,0.5 1))
SELECT ST_AsText(traversedArea(tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
  Cbuffer(Point(3 1), 0.5)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-03]', true));
/* CURVEPOLYGON(COMPOUNDCURVE(CIRCULARSTRING(3 0.5,3.5 1,3 1.5),(3 1.5,1 1.5),
   CIRCULARSTRING(1 1.5,0.646446609406726 1.353553390593274,0.5 1),
   CIRCULARSTRING(0.5 1,0.646446609406726 0.646446609406726,1 0.5),(1 0.5,3 0.5))) */
SELECT ST_AsText(traversedArea(tcbuffer '[Cbuffer(Point(1 1), 0)@2001-01-01,
  Cbuffer(Point(3 1), 0)@2001-01-02, Cbuffer(Point(4 1), 0.5)@2001-01-03]', true));
/* GEOMETRYCOLLECTION(CURVEPOLYGON(COMPOUNDCURVE(CIRCULARSTRING(3.75 0.566987298107781,
   4.5 1,3.75 1.433012701892219),(3.75 1.433012701892219,3 1,3.75 0.566987298107781))),
   LINESTRING(1 1,3 1)) */
```

The argument is false unless it is given, and the function then answers the pieces the buffer sweeps, each a disc or the region a disc sweeps between two consecutive instants, gathered into one geometry. With `unary_union = true` they are dissolved into the region they cover, computed exactly: every meeting of the circles and of the segments joining them is decided on the input coordinates, no arc is replaced by segments, and only the vertices of the result are rounded, each once. The result keeps its arcs, as a `CURVEPOLYGON` or a `MULTISURFACE`. Where the buffer keeps a zero radius, the line or the point it draws that no disc covers is returned beside the surfaces in a `GEOMETRYCOLLECTION`. Where the computation cannot decide a meeting or an order of the boundaries, the function raises an error rather than answer an approximation.

- Return the temporal geometric point given by the centers of a temporal circular buffer

```
centroid(tcbuffer) → tgeompoint
```

```
SELECT asText(centroid(tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
  Cbuffer(Point(3 3), 0.5)@2001-01-03]'));
-- [POINT(1 1)@2001-01-01, POINT(3 3)@2001-01-03]
```

- Return the convex hull of the area traversed by the temporal circular buffer

```
convexHull(tcbuffer) → geometry
```

The convex hull is computed on the exact traversed area, arcs included: an arc reaches past the points that define it, so it contributes to the hull the piece of its circle that no other feature of the area reaches past. The result is therefore a curved `CURVEPOLYGON` bounded by a `COMPOUNDCURVE` of circular arcs and straight segments, as exact as the region returned by `traversedArea`. The hull is linear only when the buffer keeps a zero radius, since the traversed area then carries no arc.

```
SELECT ST_AsText(convexHull(tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
  Cbuffer(Point(1 3), 0.5)@2001-01-02]'));
/* CURVEPOLYGON(COMPOUNDCURVE((1.5 3,1.5 1),CIRCULARSTRING(1.5 1,1 0.5,0.5 1),
   (0.5 1,0.5 3),CIRCULARSTRING(0.5 3,1 3.5,1.5 3))) */
SELECT ST_AsText(convexHull(tcbuffer '[Cbuffer(Point(1 1), 0)@2001-01-01,
  Cbuffer(Point(1 3), 0)@2001-01-02, Cbuffer(Point(3 3), 0)@2001-01-03]'));
-- POLYGON((1 1,1 3,3 3,1 1))
```

12.12 Bounding Box Operations

- Topological operators

```
{tstzspan,stbox,tcbuffer} {&&, <@, @>, ~=, -|-} {tstzspan,stbox,tcbuffer} → boolean
```

```
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' && stbox(cbuffer 'Cbuffer(Point(1 1), 0.5)');
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' @> stbox(cbuffer 'Cbuffer(Point(1 1), 0.5)');
-- true
SELECT stbox(cbuffer 'Cbuffer(Point(1 1), 0.5)') <@
  tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-02]';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]' ~= tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.35)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-03]';
-- true
```

- Position operators

```
{stbox,tcbuffer} {<<, &<, >>, &>} {stbox,tcbuffer} → boolean
{stbox,tcbuffer} {<<|, &<|, |>>, |&>} {stbox,tcbuffer} → boolean
{tstzspan,stbox,tcbuffer} {<<#, &<#, #>>, #&>} {tstzspan,stbox,tcbuffer} → boolean
```

```
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' <<
  stbox(cbuffer 'Cbuffer(Point(1 1), 0.2)');
-- false
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-02]' <<| stbox(cbuffer 'Cbuffer(Point(1 1), 0.5)');
-- false
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-03,
  Cbuffer(Point(1 1), 0.5)@2001-01-05]' #&> tstzspan '[2001-01-01,2001-01-03]';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-03,
  Cbuffer(Point(1 1), 0.5)@2001-01-05]' #>>
  tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-02]';
-- true
```

- Return the spatiotemporal bounding box expanded in the spatial dimension by a float value

```
expandSpace(tcbuffer,float) → stbox
```

```
SELECT expandSpace(tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
  Cbuffer(Point(3 3), 0.5)@2001-01-03]', 2);
-- STBOX XT(((-1.5,-1.5),(5.5,5.5)),[2001-01-01, 2001-01-03])
```

12.13 Distance Operations

Operands of different dimensions read the dimensions both of them have, see Section 8.3.

- Return the smallest distance ever

```
{geo,cbuffer,tcbuffer,stbox} |=| {geo,cbuffer,tcbuffer,stbox} → float
```

The operator `|=|` that can be used for doing nearest neighbor searches using a GiST or an SP-GiST index (see Section 10.2).

```
SELECT tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]' |=| geometry 'Linestring(50 50,55 55)';
-- 67.18225099390857
SELECT tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]' |=| cbuffer 'Cbuffer(Point(1 1), 0.5)';
-- 0.2142135623730952
```

- Return the instant of the first temporal circular buffer at which the two arguments are at the nearest distance

```
nearestApproachInstant({geo,cbuffer,tcbuffer},{geo,cbuffer,tcbuffer}) → tcbuffer
```

```
SELECT asText(nearestApproachInstant(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
-- Cbuffer(POINT(2 2),0.7)@2001-01-02
SELECT asText(nearestApproachInstant(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]', cbuffer 'Cbuffer(Point(1 1), 0.5)'));
-- Cbuffer(POINT(2 2),0.7)@2001-01-02
```

- Return the line connecting the nearest approach point between the two arguments

```
shortestLine({geo,cbuffer,tcbuffer},{geo,cbuffer,tcbuffer}) → geometry
```

The function will only return the first line that it finds if there are more than one

```
SELECT ST_AsText(shortestLine(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
-- LINESTRING(2.494974746830583 2.494974746830583,50 50)
SELECT ST_AsText(shortestLine(tcbuffer '[Cbuffer(Point(2 2), 0.3)@2001-01-01,
  Cbuffer(Point(2 2), 0.7)@2001-01-02]', cbuffer 'Cbuffer(Point(1 1), 0.5)'));
-- LINESTRING(1.505025253169417 1.505025253169417,1.353553390593274 1.353553390593274)
```

- Return the temporal distance

```
{geo,cbuffer} <-> tcbuffer → tfloat
tcbuffer <-> {geo,cbuffer,tcbuffer} → tfloat
```

```
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]' <-> cbuffer 'Cbuffer(Point(1 1), 0.2)';
-- [0@2001-01-01, 0@2001-01-03]
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]' <-> geometry 'Point(50 50)';
-- [68.99646455628165@2001-01-01, 68.79646455628166@2001-01-03]
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]' <->
  tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-04]';
-- [0@2001-01-02, 0@2001-01-03]
```

- Return the minimum spatial distance between two sets of temporal circular buffers, ignoring time

```
minDistance(tcbuffer[],tcbuffer[]) → float
```

The distance is measured between the traversed areas with time discarded, equal to `ST_Distance` of the traversed area of each argument, and the result is the minimum over every pair drawn from the two sets. Two values whose discs sweep the same region at different times are therefore at distance zero. The time-synchronous quantity, which compares two arguments only at the instants they share, is |=|. A single pair is the one-element case.

```
SELECT minDistance(ARRAY[tcbuffer '[Cbuffer(Point(0 0),1)@2001-01-01,
  Cbuffer(Point(2 0),1)@2001-01-02]'], ARRAY[tcbuffer
  '[Cbuffer(Point(0 5),0.5)@2001-01-03, Cbuffer(Point(2 5),0.5)@2001-01-04]']);
-- 3.5
```

12.14 Ever and Always Relationships

- Ever and always relationships

```
eContains({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
aContains({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
eCovers({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
aCovers({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
eDisjoint({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
aDisjoint({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
eDwithin({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer},float) → boolean
aDwithin({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer},float) → boolean
eIntersects({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
aIntersects({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
eTouches({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
aTouches({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → boolean
```

The *ever* and *always* relationships determine whether the topological or distance relationship is ever or always satisfied (see Section 5.4.2) and return a `boolean`

```
SELECT eContains(geometry 'Polygon((0 0,0 50,50 50,50 0,0 0))',
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.3)@2001-01-03)');
-- true
SELECT eDisjoint(cbuffer 'Cbuffer(Point(2 2), 0.0)',
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.3)@2001-01-03)');
-- true
SELECT eIntersects(tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-03)',
  tcbuffer '[Cbuffer(Point(2 2), 0.0)@2001-01-01, Cbuffer(Point(2 2), 1)@2001-01-03)');
-- false
```

12.15 Spatiotemporal Relationships

- Spatiotemporal relationships

```
tContains({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → tbool
tCovers({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → tbool
tDisjoint({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → tbool
tDwithin({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer},float) → tbool
tIntersects({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → tbool
tTouches({geometry,cbuffer,tcbuffer},{geometry,cbuffer,tcbuffer}) → tbool
```

The *temporal* relationships compute the topological or distance relationship at each instant and result in a `tbool`. A circular buffer covers an area, so every relationship is declared in both directions and between two temporal circular buffers, which is what distinguishes this family from the moving-point ones.

```
SELECT tDisjoint(geometry 'Polygon((0 0,0 50,50 50,50 0,0 0))',
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.3)@2001-01-03)');
-- {[f@2001-01-01, f@2001-01-03)}
SELECT tDwithin(tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03)', tcbuffer '[Cbuffer(Point(1 1), 0.5)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-03)', 1);
-- {[t@2001-01-01, t@2001-01-03)}
```

12.16 Comparisons

- Traditional comparisons

```
tcbuffer {=, <>, <, >, <=, >=} tcbuffer → boolean
```

```
SELECT tcbuffer '{[Cbuffer(Point(1 1), 0.1)@2001-01-01,
  Cbuffer(Point(1 1), 0.3)@2001-01-02),
  [Cbuffer(Point(1 1), 0.3)@2001-01-02, Cbuffer(Point(1 1), 0.5)@2001-01-03]}' =
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-03]';
-- true
SELECT tcbuffer '{[Cbuffer(Point(1 1), 0.1)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]}' <>
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.5)@2001-01-03]';
-- false
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
  Cbuffer(Point(1 1), 0.5)@2001-01-03]' <
  tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01, Cbuffer(Point(1 1), 0.6)@2001-01-03]';
-- false
```

- Ever and always comparisons

```
{tcbuffer,cbuffer} {?=, ?<>, %=, %<>} {tcbuffer,cbuffer} → boolean
```

```
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.4)@2001-01-03)' ?= cbuffer 'Cbuffer(Point(1 1), 0.3)';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.4)@2001-01-03)' ?= tcbuffer '[Cbuffer(Point(1 1), 0.4)@2001-01-01,
  Cbuffer(Point(1 1), 0.2)@2001-01-03)';
-- true
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.2)@2001-01-04)' %= cbuffer 'Cbuffer(Point(1 1), 0.2)';
-- true
```

- Temporal comparisons

```
{tcbuffer,cbuffer} {#=, #<>} {tcbuffer,cbuffer} → tbool
```

```
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.4)@2001-01-03)' #= cbuffer 'Cbuffer(Point(1 1), 0.3)';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-01,
  Cbuffer(Point(1 1), 0.8)@2001-01-03)' #<>
  tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-01, Cbuffer(Point(1 1), 0.7)@2001-01-03)';
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}
```

12.17 Aggregations

The four aggregate functions for temporal circular buffers are illustrated next.

- Bounding box extent

```
extent(tcbuffer) → stbox
```

```
SELECT extent(temp) FROM (VALUES
   (tcbuffer 'Cbuffer(Point(1 1), 0.5)@2001-01-01'),
   (tcbuffer '[Cbuffer(Point(2 2), 0.2)@2001-01-02,
     Cbuffer(Point(3 3), 0.4)@2001-01-03]')) t(temp);
-- STBOX XT(((0.5,0.5),(3.4,3.4)),[2001-01-01, 2001-01-03])
```

- Temporal count

```
tCount(tcbuffer) → {tintSeq,tintSeqSet}
```

```
WITH Temp(temp) AS (
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
    Cbuffer(Point(1 1), 0.3)@2001-01-03)' UNION
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-02,
    Cbuffer(Point(1 1), 0.4)@2001-01-04)' UNION
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-03,
    Cbuffer(Point(1 1), 0.5)@2001-01-05)' )
SELECT tCount(Temp) FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 1@2001-01-04, 1@2001-01-05)}
```

- Window count

```
wCount(tcbuffer) → {tintSeq,tintSeqSet}
```

```
WITH Temp(temp) AS (
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
    Cbuffer(Point(1 1), 0.3)@2001-01-03)' UNION
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.2)@2001-01-02,
    Cbuffer(Point(1 1), 0.4)@2001-01-04)' UNION
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.3)@2001-01-03,
    Cbuffer(Point(1 1), 0.5)@2001-01-05)' )
SELECT wCount(Temp, '1 day') FROM Temp;
/* {[1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 2@2001-01-04, 1@2001-01-05,
   1@2001-01-06)} */
```

- Aggregate the centres of a set of temporal circular buffers into their temporal centroid, by casting to `tgeompoint`

```
tCentroid(tcbuffer::tgeompoint) → tgeompoint
```

The aggregate belongs to the `tgeompoint` type: a circular buffer’s centre trajectory is a temporal point, so the cast reaches the aggregate rather than the buffer type carrying its own copy of it.

```
WITH Temp(temp) AS (
  SELECT tcbuffer '[Cbuffer(Point(1 1), 0.1)@2001-01-01,
    Cbuffer(Point(1 1), 0.3)@2001-01-03)' UNION
  SELECT tcbuffer '[Cbuffer(Point(3 3), 0.2)@2001-01-01,
    Cbuffer(Point(3 3), 0.4)@2001-01-03)' )
SELECT asText(tCentroid(temp::tgeompoint)) FROM Temp;
-- {[POINT(2 2)@2001-01-01, POINT(2 2)@2001-01-03)}
```

12.18 Simplification

- Return a temporal circular buffer simplified ensuring that consecutive centers are at least a certain distance or time interval apart

```
minDistSimplify(tcbuffer,mindist float) → tcbuffer
minTimeDeltaSimplify(tcbuffer,mint interval) → tcbuffer
```

The simplification operates on the center trajectory of the temporal circular buffer and preserves the radius at each surviving instant. Simplification applies only to temporal sequences or sequence sets with linear interpolation. In all other cases, a copy of the given value is returned.

```
SELECT asText(minDistSimplify(tcbuffer '[Cbuffer(Point(0 0), 1)@2001-01-01,
  Cbuffer(Point(1 0), 1)@2001-01-02, Cbuffer(Point(4 0), 1)@2001-01-03]', 2));
-- [Cbuffer(POINT(0 0),1)@2001-01-01, Cbuffer(POINT(4 0),1)@2001-01-03]
-- The result keeps the interpolation of the value it simplifies.
SELECT interp(minDistSimplify(tcbuffer '[Cbuffer(Point(0 0), 1)@2001-01-01,
  Cbuffer(Point(1 0), 1)@2001-01-02, Cbuffer(Point(4 0), 1)@2001-01-03]', 2));
-- Linear
SELECT asText(minTimeDeltaSimplify(tcbuffer '[Cbuffer(Point(0 0), 1)@2001-01-01,
  Cbuffer(Point(1 0), 1)@2001-01-02, Cbuffer(Point(4 0), 1)@2001-01-04]',
  interval '2 days'));
-- [Cbuffer(POINT(0 0),1)@2001-01-01, Cbuffer(POINT(4 0),1)@2001-01-04]
```

- Return a temporal circular buffer simplified using the Douglas-Peucker algorithm

```
maxDistSimplify(tcbuffer,maxdist float,syncdist boolean=true) → tcbuffer
douglasPeuckerSimplify(tcbuffer,maxdist float,syncdist boolean=true) → tcbuffer
```

  The difference between the two functions is that `maxDistSimplify` uses a single-pass version of the algorithm whereas `douglasPeuckerSimplify` uses the standard recursive algorithm. As for the functions above, the simplification operates on the center trajectory and preserves the radius at each surviving instant.

```
SELECT asText(douglasPeuckerSimplify(tcbuffer '[Cbuffer(Point(1 1),0.5)@2001-01-01,
  Cbuffer(Point(2 2),0.5)@2001-01-02, Cbuffer(Point(3 1),0.5)@2001-01-03,
  Cbuffer(Point(4 4),0.5)@2001-01-04]', 2.0));
-- [Cbuffer(POINT(1 1),0.5)@2001-01-01, Cbuffer(POINT(4 4),0.5)@2001-01-04]
```

12.19 Indexing

GiST and SP-GiST indexes can be created for table columns of temporal circular buffers. An example of index creation is follows:

```
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist(Trip);
```

The GiST and SP-GiST indexes store the bounding box for the temporal circular buffers, which is an `stbox`, as all spatiotemporal types.

A GiST or SP-GiST index can accelerate queries involving the following operators:

- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, which only consider the spatial dimension in temporal circular buffers,
- `<<#`, `&<#`, `#&>`, `#>>`, which only consider the time dimension in temporal circular buffers,
- `&&`, `@>`, `<@`, `~=`, `-|-`, and `|=|` , which consider as many dimensions as they are shared by the indexed column and the query argument.

These operators work on bounding boxes, not the entire values.

Chapter 13

Temporal Poses and Pose Chains

The `pose` type is used to represent the *location* and *orientation* of geometric objects within coordinate systems anchored to the earth's surface or within other astronomical coordinate systems. The location is represented by a 2D or 3D point. For 2D poses, the orientation is defined by a rotation angle in (-π, π] expressed in radians. For 3D poses, the orientation is defined by four float values `W`, `X`, `Y`, `Z`, representing a unit quaternion $Q = \langle W, X, Y, Z \rangle$ where $\| Q^2 \| = \sqrt{W^2+X^2+Y^2+Z^2} = 1$.

The pose constructor accepts quaternions whose norm is within `1e-3` of unit (a tolerance wide enough to absorb the integrator drift typical of sensor-fusion clients such as IMUs and AR/VR runtimes), then renormalizes to exactly unit norm before storing. Way-off norms (e.g., the obvious-bug case `(1, 1, 1, 1)` where `|q| = 2`), zero quaternions, and quaternions with `NaN`/`Inf` components are rejected up front. The on-disk representation is therefore independent of the caller's floating-point hygiene, and downstream comparison, hashing, SLERP, and Euler-decomposition code can rely on the unit-norm invariant.

The GeoPose Standards Working Group (SWG), working under the auspices of the Open Geospatial Consortium, has defined a standard for exchanging pose information across different users, devices, and platforms. More information about the standard can be found in the GitHub repository of the GeoPose SWG.

The `pose` type serves as base type for defining the temporal pose type `tpose`. The `tpose` type has similar functionality as the temporal point type `tgeompoint`. Thus, most functions and operators described before for the `tgeompoint` type are also applicable for the `tpose` type. In addition, there are specific functions defined for the `tpose` type. We cover these functions in this chapter.

The `tpose` type is used for defining the type `trgeometry` (that is, temporal rigid geometry) defined in the next chapter. The implementation of the these types in MobilityDB has been studied in the following PhD thesis.

- Maxime Schoemans, *Managing Rigid Temporal Geometries in Moving Object Databases*, PhD thesis, Université libre de Bruxelles, 2025.

The `posechain` type represents a body whose parts are placed with respect to one another: a camera on a gimbal on a vehicle, or an arm whose forearm is placed with respect to its upper arm. A pose chain is an ordered list of at least one `pose`. The first link is expressed in the chain's outer frame, and every later link is expressed in the frame the link before it defines, so a link states where a part sits relative to its parent rather than relative to the world.

The OGC GeoPose standard calls such a structure a chain of frame transforms, where each transform is a pair of reference frames and the outer frame is the domain while the inner frame is the range. The standard states that an inner frame cannot be a topocentric frame, so a chain carries exactly one frame that names a place on the earth, at its outside. Two properties follow, and both are enforced: the whole chain has one SRID, and every link has the same dimension. Only the outer link may be geodetic, and only the outer link may carry an SRID.

Where the outer frame is geographic, the outer link's position is in degrees while a later link's translation is in metres along the local East-North-Up axes at its parent. The composition takes that sum in geocentric coordinates and reads the position back as a geographic one, so a chain over a geographic frame is composed on the ellipsoid rather than in the plane, and the orientation is re-expressed against the East-North-Up basis at the position the composition reaches.

Composing every link gives the pose of the innermost frame in the outer frame of the chain, which is what the conversion to `pose` returns. That single conversion gives a chain the whole surface defined for poses, so the type itself stays small. The two types therefore answer the same questions, and this chapter states each operation once, naming the type only where the answer depends on it.

What it states	Base type	Set type	Temporal type
One placement	`pose`	`poseset`	`tpose`
A composition of placements	`posechain`	`posechainset`	`tposechain`

13.1 Notation

Most functions and operators for temporal types described in the previous chapters can be applied to poses and to pose chains. Therefore, in the signatures of the functions, the notation `base` represents a `pose` or a `posechain` and the notation `ttype` a `tpose` or a `tposechain`; the notations `geompoint2D` and `geompoint3D` represent a `geompoint` with, respectively, two and three dimensions, and the notations `geogpoint2D` and `geogpoint3D` represent a `geogpoint` with, respectively, two and three dimensions. To avoid redundancy, we present next only what is specific to the two types.

A signature both types answer names them as an alternation, `{pose,posechain}`, and a signature only one of them answers names that one. A function whose *name* differs between the two keeps a line of its own, so `tpose(pose,timestamptz)` and `tposechain(posechain,timestamptz)` are never collapsed into one callable name that does not exist.

13.2 Static Poses and Pose Chains

A 2D `pose` is a couple of the form `(point2D,radius)` where `point2D` is a 2D geometric point and `radius` is a `float` value representing a rotation angle in (-π, π] expressed in radians. A 3D `pose` is a tuple of the form `(point3D,W,Y,X,Z)` where `point3D` is a 3D geometric point, and `W`, and `X`, `Y`, and `Z` are four `floats` representing a *unit* quaternion. Examples of input of pose values are as follows:

```
SELECT pose 'Pose(Point(1 1), 0.5)';
SELECT pose 'Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)';
```

An SRID can be specified for a pose either at the begining of the pose literal or before the point literal as shown below.

```
SELECT pose 'SRID=3812;Pose(Point(1 1), 0.5)';
SELECT pose 'Pose(SRID=5676;Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)';
```

As for other spatial types, a `pose` can be either planar (geometric) or geodetic (geographic). A geodetic pose is denoted by the `GeodPose` keyword, in the same way that a geodetic `stbox` is denoted by `GEODSTBOX`. A geodetic pose that states no SRID has the SRID 4326 of a geography, while a planar pose has the SRID 0.

```
SELECT pose 'GeodPose(Point(1 1), 0.5)';
SELECT pose 'SRID=4326;GeodPose(Point Z(1 1 1), 1, 0, 0, 0)';
```

Values of the pose type must satisfy several constraints so that they are well defined. Examples of incorrect pose type values are as follows.

```
-- Empty point
select pose 'Pose(Point empty, 0.5)';
-- Incorrect point value
SELECT pose 'Pose(Linestring(1 1,2 2), 1.0)';
-- Incorrect radius value
SELECT pose 'Pose(Point(1 1), -10.0)';
-- Incorrect 3D point
SELECT pose 'Pose(Point Z(1 1), 1.0)';
-- Incomplete 3D orientation
SELECT pose 'Pose(Point Z(1 1 1), 1.0)';
```

We give next the functions and operators for the pose type.

13.2.1 Input and Output

- Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation

```
asText({pose,posechain,pose[],posechain[]}) → {text,text[]}
asEWKT({pose,posechain,pose[],posechain[]}) → {text,text[]}
```

```
SELECT asText(pose 'SRID=4326;Pose(Point(0 0),1)');
-- Pose(POINT(0 0),1)
SELECT asText(ARRAY[pose 'Pose(Point(0 0),1)', 'Pose(Point(1 1),2)']);
-- {"Pose(POINT(0 0),1)","Pose(POINT(1 1),2)"}
SELECT asEWKT(pose 'SRID=4326;Pose(Point(0 0),1)');
-- SRID=4326;Pose(POINT(0 0),1)
SELECT asEWKT(ARRAY[pose 'Pose(SRID=5676;Point(0 0),1)', 'Pose(SRID=5676;Point(1 1),2)']);
-- {"Pose(SRID=5676;POINT(0 0),1)","Pose(SRID=5676;POINT(1 1),2))"}
```

- Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), the Hexadecimal Well-Known Binary (HexWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

```
asBinary({pose,posechain},endian text='') → bytea
asEWKB({pose,posechain},endian text='') → bytea
asHexWKB({pose,posechain},endian text='') → text
asHexEWKB({pose,posechain},endian text='') → text
```

  The chain writes its flags and its one SRID once, then the number of links, then the values of every link in order. The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used.

  The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used.

```
SELECT asBinary(pose 'Pose(Point(1 2),1)');
-- \x0101000000000000f03f0000000000000040000000000000f03f
SELECT asEWKB(pose 'SRID=7844;Pose(Point(1 2),1)');
-- \x0141a41e0000000000000000f03f0000000000000040000000000000f03f
SELECT asHexWKB(pose 'Pose(Point(1 2),1)');
-- 0101000000000000F03F0000000000000040000000000000F03F
SELECT asHexEWKB(pose 'SRID=3812;Pose(Point(1 2),1)');
-- 0141E40E0000000000000000F03F0000000000000040000000000000F03F
```

- Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation

```
poseFromText(text) → pose
poseFromEWKT(text) → pose
posechainFromText(text) → posechain
posechainFromEWKT(text) → posechain
```

  These functions read the text form alone. The type input, `pose '...'`, also reads an OGC GeoPose document, as the `geometry` input reads a GeoJSON one, and poseFromGeoPose reads a GeoPose document alone.

```
SELECT asEWKT(poseFromText(text 'Pose(Point(1 2),1)'));
-- Pose(POINT(1 2),1)
SELECT asEWKT(poseFromEWKT(text 'SRID=3812;Pose(Point(1 2),1)'));
-- SRID=3812;Pose(POINT(1 2),1)
SELECT asEWKT(posechainFromText('PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),1))'));
-- PoseChain(Pose(POINT(0 0),0),Pose(POINT(1 0),1))
SELECT asEWKT(posechainFromEWKT('SRID=3812;PoseChain(Pose(Point(0 0),0))'));
-- SRID=3812;PoseChain(Pose(POINT(0 0),0))
```

- Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), from the Hexadecimal Well-Known Binary (HexWKB), or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

```
poseFromBinary(bytea) → pose
poseFromEWKB(bytea) → pose
poseFromHexWKB(text) → pose
poseFromHexEWKB(text) → pose
posechainFromBinary(bytea) → posechain
posechainFromEWKB(bytea) → posechain
posechainFromHexWKB(text) → posechain
posechainFromHexEWKB(text) → posechain
```

```
SELECT asEWKT(poseFromBinary('\x0101000000000000f03f000000000000004000000000000000f03f'));
-- Pose(POINT(1 2),1)
SELECT asEWKT(poseFromEWKB(
  '\x0141a41e0000000000000000f03f000000000000004000000000000000f03f'));
-- SRID=7844;Pose(POINT(1 2),1)
SELECT asEWKT(poseFromHexEWKB(
  '0141E40E0000000000000000F03F000000000000004000000000000000F03F'));
-- SRID=3812;Pose(POINT(1 2),1)
SELECT posechainFromBinary(asBinary(posechain 'PoseChain(Pose(Point(1 2),0.5))')) =
  posechain 'PoseChain(Pose(Point(1 2),0.5))';
-- t
SELECT posechainFromHexEWKB(asHexEWKB( posechain 'SRID=3812;
  PoseChain(Pose(Point(1 2),0.5))')) = posechain 'SRID=3812;PoseChain(Pose(Point(1 2),
  0.5))';
-- t
```

13.2.2 Constructors

- Constructor for geodetic poses from a geography point

```
geodpose(geogpoint2D,float) → pose
geodpose(geogpoint3D,float,float,float,float) → pose
geodpose(geogpoint3D,yaw float,pitch float,roll float) → pose
```

It is a function of its own, as `geodstbox` is, since a point written as an untyped literal resolves to a geometry in `pose`.

```
SELECT asText(geodpose(geography 'Point(1 2)', 0.5));
-- GeodPose(POINT(1 2),0.5)
SELECT asText(geodpose(geography 'Point(1 2 3)', 1, 0, 0, 0));
-- GeodPose(POINT Z (1 2 3),1,0,0,0)
```

- Constructor for poses

```
pose(geompoint2D,float) → pose
pose(geompoint3D,float,float,float,float) → pose
pose(geompoint3D,yaw float,pitch float,roll float) → pose
```

The three-angle form takes the orientation in the other encoding the OGC GeoPose v1.0 standard prescribes, a yaw / pitch / roll triple in radians under the ZYX intrinsic Tait-Bryan convention, and stores the quaternion it denotes. It is the inverse of ypr.

```
SELECT asText(pose(ST_Point(1,1), radians(45)), 6);
-- Pose(POINT(1 1),0.785398)
SELECT asEWKT(pose(ST_Point(1,1,3812), radians(45)), 6);
-- SRID=3812;Pose(POINT(1 1),0.785398)
SELECT asText(pose(ST_PointZ(1,1,1), 1, 0, 0, 0));
-- Pose(POINT Z (1 1 1),1,0,0,0)
SELECT asText(pose(ST_PointZ(1,1,1), radians(90), 0, 0), 6);
-- Pose(POINT Z (1 1 1),0.707107,0,0,0.707107)
```

13.2.3 Conversions

Values of the `pose` type can be converted to the `geometry` point type using an explicit `CAST` or using the `::` notation as shown below.

- Convert a pose and, optionally, a timestamp or a period, into a spatiotemporal box

  ```
  {pose,posechain}::stbox
  stbox({pose,posechain}) → stbox
  stbox({pose,posechain},{timestamptz,tstzspan}) → stbox
  ```

  The box covers the composed position of every prefix of the chain, not only of the whole of it: every joint is a place, and a query window that meets the elbow but not the hand still meets the chain.

  ```
  SELECT stbox(pose 'SRID=5676;Pose(Point(1 1),0.3)');
  -- SRID=5676;STBOX X((1,1),(1,1))
  SELECT stbox(pose 'Pose(Point(1 1),0.3)', timestamptz '2001-01-01');
  -- STBOX XT(((1,1),(1.3,1.3)),[2001-01-01, 2001-01-01])
  SELECT stbox(pose 'Pose(Point(1 1),0.3)', tstzspan '[2001-01-01,2001-01-02]');
  -- STBOX XT(((1,1),(1.3,1.3)),[2001-01-01, 2001-01-02])
  ```

- Convert a pose, planar or geodetic, into a geography point

  The conversions follow the rule of [?listitem].

  ```
  {pose,posechain}::geogpoint
  ```

  ```
  SELECT ST_AsEWKT(pose 'GeodPose(Point(1 2), 0.5)'::geography);
  -- SRID=4326;POINT(1 2)
  SELECT ST_AsEWKT(pose 'Pose(Point(1 2), 0.5)'::geography);
  -- SRID=4326;POINT(1 2)
  SELECT pose 'SRID=3857;Pose(Point(1 2), 0.5)'::geography;
  -- ERROR:  Only lon/lat coordinate systems are supported in geography.
  ```

- Convert a pose, planar or geodetic, into a geometry point

  ```
  {pose,posechain}::geompoint
  ```

  ```
  SELECT ST_AsText(pose(ST_Point(1, 1), 1)::geometry);
  -- POINT(1 1)
  SELECT ST_AsEWKT(pose(ST_PointZ(1, 1, 1, 5676), 1, 0, 0, 0)::geometry);
  -- SRID=5676;POINT(1 1 1)
  ```

13.2.4 Accessors

- Return the point, which for a pose chain is the point of its innermost frame

  ```
  point({pose,posechain}) → geometry
  ```

  ```
  SELECT ST_AsText(point(pose 'Pose(Point(1 1), 0.3)'));
  -- POINT(1 1)
  ```

- Return the orientation quaternion of a pose

  ```
  quaternion(pose) → quaternion
  ```

The components are returned in the order `W`, `X`, `Y`, `Z`, the Hamilton convention in which a 3D pose stores them. This is one of the two orientation encodings the OGC GeoPose v1.0 standard prescribes; the other, yaw / pitch / roll, is given by ypr. Both are defined for both dimensions: the orientation of a 2D pose is a turn about the local vertical by its stored angle, so it puts half that angle in `W` and `Z` and leaves `X` and `Y` at zero. This is the quaternion a GeoPose Basic-Quaternion document carries for a planar pose.

```
SELECT quaternion(pose 'Pose(Point Z(1 1 1), 0, 0, 0, 1)');
-- (0,0,0,1)
SELECT quaternion(pose 'Pose(Point(1 1), 0)');
-- (1,0,0,0)
```

13.2.5 Transformations

- Round the point and the orientation of the pose to the number of decimal places

  The rotation of a 3D pose is normalized to unit length after its components are rounded, so its components may have more decimal places than requested.

  ```
  round({pose,pose[],posechain},integer=0) → {pose,pose[],posechain}
  ```

  ```
  SELECT asText(round(pose(ST_Point(1.123456789,1.123456789), 0.123456789), 6));
  -- Pose(POINT(1.123457 1.123457),0.123457)
  SELECT asText(round(ARRAY[pose(ST_Point(1.123456789,1.123456789), 0.123456789),
    pose(ST_Point(2.987654321,2.987654321), 0.987654321)], 6));
  -- {"Pose(POINT(1.123457 1.123457),0.123457)","Pose(POINT(2.987654 2.987654),0.987654)"}
  ```

13.2.6 Spatial Reference System

- Return or set the spatial reference identifier, which for a pose chain is the identifier of its outer frame

  ```
  SRID({pose,posechain}) → integer
  setSRID({pose,posechain},integer) → {pose,posechain}
  ```

  ```
  SELECT SRID(pose 'Pose(SRID=5676;Point(1 1), 0.3)');
  -- 5676
  SELECT asEWKT(setSRID(pose 'Pose(Point(0 0),1)', 4326));
  -- SRID=4326;Pose(POINT(0 0),1)
  ```

- Transform to a spatial reference identifier

  ```
  transform({pose,posechain},integer) → {pose,posechain}
  transformPipeline({pose,posechain},pipeline text,srid integer,
    is_forward boolean=true) → {pose,posechain}
  ```

  The `transform` function specifies the transformation with a target SRID. An error is raised when the input has an unknown SRID (represented by 0). The position is transformed as a point is, and the orientation is written in the axes of the target frame at the transformed position, for any pair of reference systems. A geographic system writes an orientation in the East-North-Up frame at the position, and any other system in its coordinate axes. The orientation turns by the angle between the two frames at the position, which for a projection is its meridian convergence, so that the body faces the same direction on the ground before and after the transformation. A projection that does not preserve angles carries that direction exactly, and the other axes of a 3D pose are the nearest ones orthogonal to it. In a chain only the outer link names a frame and only it is transformed, every other link being a rigid transform read in the axes of its parent, which a change of the outer frame leaves as they were. The `transformPipeline` function specifies the transformation with a defined coordinate transformation pipeline represented with the following string format:

  `urn:ogc:def:coordinateOperation:AUTHORITY::CODE`

The pipeline transforms the position whatever the SRID of the input pose, and the SRID of the output pose will be set to zero unless a value is provided via the optional `srid` parameter. The orientation is written in the target frame as for `transform`, the frames being the ones that the SRID of the input pose and the `srid` parameter name; it is kept as it is when either of them is unknown. As stated by the last parameter, the pipeline is executed by default in a forward direction; by setting the parameter to false, the pipeline is executed in the inverse direction.

```
-- The Belgian Lambert projection turns the North by -0.007111 degrees at this position
SELECT asEWKT(transform(pose 'SRID=4326;Pose(Point(4.35 50.85),1)', 3812), 6);
-- SRID=3812;Pose(POINT(648679.018035 671067.055638),0.999876)
-- A body facing East at 12E 45N faces 2.1223 degrees North of the East of the grid of
-- UTM zone 32
SELECT round(degrees(yaw(transform(pose 'SRID=4326;Pose(Point(12 45),0)', 32632))):: ↩
    numeric, 6);
-- 2.122300
```

```
-- Round-tripping a 3D pose through ECEF and back lands at the input pose
SELECT asEWKT(round( transform(transform(pose 'SRID=4326;Pose(Point(8 47 0), 1, 0, 0, 0)',
  4978), 4326), 6));
-- SRID=4326;Pose(POINT Z (8 47 0),1,0,0,0)
-- At the equator-meridian (lat=lon=0), the body identity quaternion in the ECEF basis is
-- the canonical East-North-Up to ECEF rotation
SELECT asEWKT(round(transform(pose 'SRID=4326;Pose(Point(0 0 0), 1, 0, 0, 0)', 4978), 6));
-- SRID=4978;Pose(POINT Z (6378137 0 0),0.5,0.5,0.5,0.5)
```

```
WITH test(pose, pipeline) AS (
  SELECT pose 'Pose(SRID=4326;Point(4.3525 50.846667),1)',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(pose, pipeline, 32631), pipeline, 4326,
  false), 6) FROM test;
-- SRID=4326;Pose(POINT(4.3525 50.846667),1)
```

13.2.7 Distance Operations

- Return the distance

```
distance({geo,pose},pose) → float
distance(pose,{geo,pose}) → float
distance({geo,posechain},posechain) → float
distance(posechain,{geo,posechain}) → float
{geo,pose} <-> pose → float
{geo,posechain} <-> posechain → float
```

A pose chain is measured by the pose its composition gives. Only the point component of the pose is taken into account, the orientation is ignored. The distance is computed in two dimensions, even when the arguments are three-dimensional. The result is `NULL` when the geometry is empty.

A geodetic pose and a geography are measured on the surface of the earth, in meters, whatever their Z.

```
SELECT round(pose 'GeodPose(Point(0 0), 0)' <-> geography 'Point(1 0)', 3);
-- 111319.491
SELECT round(distance(geometry 'Point(1 0)', pose 'Pose(Point(4 0),0)'), 6);
-- 3
SELECT round(pose 'Pose(Point(0 0),0)' <-> pose 'Pose(Point(3 4),0)', 6);
-- 5
SELECT round(pose 'Pose(Point(0 0 0), 1, 0, 0, 0)' <->
  pose 'Pose(Point(3 4 12), 1, 0, 0, 0)', 6);
-- 5
SELECT round(geometry 'Point empty' <-> pose 'Pose(Point(4 0),0)', 6);
-- NULL
SELECT round(distance(posechain 'PoseChain(Pose(Point(1 0),0), Pose(Point(1 0),0))',
  geometry 'Point(5 0)'), 6);
-- 3
```

- Return the nearest approach distance

```
nearestApproachDistance({stbox,pose},{pose,stbox}) → float
nearestApproachDistance({stbox,posechain},{posechain,stbox}) → float
{stbox,pose} |=| {pose,stbox} → float
{stbox,posechain} |=| {posechain,stbox} → float
```

The value is the least distance over the time the two arguments share, and is `NULL` when they share none. A pose carries no period, so it is present at every time the box spans. Only the point component of the pose is taken into account, the orientation is ignored.

```
SELECT round(nearestApproachDistance(stbox 'STBOX X((1,-1),(3,1))', pose 'Pose(Point(4 0) ↩
    ,0)'), 6);
-- 1
SELECT round(pose 'Pose(Point(4 0),0)' |=| stbox 'STBOX X((1,-1),(3,1))', 6);
-- 1
```

13.2.8 Comparisons

The comparison operators (=, <, and so on) are available for poses. Excepted the equality and inequality, the other comparison operators are not useful in the real world but allow B-tree indexes to be constructed on poses.

- Traditional comparisons

```
{pose,posechain} {=, <>, <, >, <=, >=} {pose,posechain} → boolean
```

The ordering compares the dimension, then the SRID, then the links in order, and last the number of links, so a chain precedes every chain that extends it.

```
SELECT pose 'Pose(Point(3 3), 0.5)' = pose 'Pose(Point(3 3), 0.5)';
-- true
SELECT pose 'Pose(Point(3 3), 0.5)' <> pose 'Pose(Point(3 3), 0.6)';
-- true
SELECT pose 'Pose(Point(3 3), 0.5)' < pose 'Pose(Point(3 3), 0.6)';
-- true
SELECT pose 'Pose(Point(3 3), 0.6)' > pose 'Pose(Point(2 2), 0.6)';
-- true
SELECT pose 'Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)' <=
  pose 'Pose(Point Z(2 2 2), 0.5, 0.5, 0.5, 0.5)';
-- true
SELECT pose 'Pose(Point(1 1), 0.6)' >= pose 'Pose(Point(1 1), 0.5)';
-- true
```

13.3 Pose Chain Composition

A pose chain states its links, and this section holds what only a chain answers: how it is built from poses, how many links it holds, which pose a link is, and how the whole chain composes back into the single pose it denotes.

- Construct a pose chain from an array of poses ordered from the outermost frame inwards

```
posechain(pose[]) → posechain
```

```
SELECT asEWKT(posechain(ARRAY[pose 'Pose(Point(0 0),0)', pose 'Pose(Point(1 0),0)']));
-- PoseChain(Pose(POINT(0 0),0),Pose(POINT(1 0),0))
```

- Convert a pose into a pose chain of a single link

```
posechain(pose) → posechain
```

```
SELECT asEWKT(posechain(pose 'SRID=3812;Pose(Point(1 2),0.5)'));
-- SRID=3812;PoseChain(Pose(POINT(1 2),0.5))
```

- Return the pose of a frame the chain defines, read in the outer frame of the chain

```
pose(posechain) → pose
pose(posechain,integer) → pose
```

Every instant holds the pose the whole chain composes to, which is where its innermost frame sits in the frame of the chain. The result is a step function: composing a chain multiplies the rotations of its links, so the pose of the chain interpolated between two instants is not the pose interpolated between the two composed poses, and a linear result would answer a pose the chain never holds. A chain of one link composes to itself, and there the interpolation of the argument carries over unchanged.

With one argument the whole chain is composed, giving the pose of its innermost frame. With two the first `n` links are composed, giving the pose of the frame the n-th link defines, that is, where that joint of the chain sits.

```
SELECT asEWKT(pose(posechain 'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0),
  Pose(Point(1 0),0))'));
-- Pose(POINT(2 0),0)
SELECT asEWKT(pose(posechain 'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0),
  Pose(Point(1 0),0))', 2));
-- Pose(POINT(1 0),0)
```

- Return a pose chain with a pose appended as its innermost link

```
appendPose(posechain,pose) → posechain
```

The pose is read in the frame the last link of the chain defines, so it carries neither an SRID nor the geodetic marker.

```
SELECT asEWKT(appendPose(posechain 'PoseChain(Pose(Point(0 0),0))',
  pose 'Pose(Point(1 0),0)'));
-- PoseChain(Pose(POINT(0 0),0),Pose(POINT(1 0),0))
```

- Return the number of links of a pose chain

```
numPoses(posechain) → integer
```

```
SELECT numPoses(posechain 'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0),
  Pose(Point(2 0),0))');
-- 3
```

- Return the links of a pose chain

```
startPose(posechain) → pose
endPose(posechain) → pose
poseN(posechain,integer) → pose
poses(posechain) → pose[]
```

A link is returned as it is stored, that is, in the frame the link before it defines. Where that frame sits in the outer frame of the chain is what `pose(posechain,integer)` answers. The function `poseN` returns `NULL` if the chain has fewer links than the number given.

```
SELECT asEWKT(startPose(posechain 'SRID=3812;
  PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0))'));
-- SRID=3812;Pose(POINT(0 0),0)
SELECT asEWKT(endPose(posechain 'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0))'));
-- Pose(POINT(1 0),0)
```

```
SELECT asEWKT(poseN(posechain 'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0))', 2));
-- Pose(POINT(1 0),0)
SELECT asEWKT(poses(posechain 'PoseChain(Pose(Point(0 0),0),Pose(Point(1 0),0))'));
-- {"Pose(POINT(0 0),0)","Pose(POINT(1 0),0)"}
```

- Return the number of links every value of a temporal pose chain holds

  ```
  numPoses(tposechain) → integer
  ```

  The number is the same at every instant, so one instant answers for the whole value.

  ```
  SELECT numPoses(tposechain 'PoseChain(Pose(Point(0 0), 0),
    Pose(Point(10 0), 0))@2001-01-01');
  -- 2
  ```

13.4 Temporal Poses and Pose Chains

The temporal pose chain type `tposechain` represents the evolution in time of a chain of nested frames, that is, of an articulated object whose joints move with respect to one another. As all temporal types, it comes in three subtypes, namely, instant, sequence, and sequence set. Examples of `tposechain` values in these subtypes are given next.

Every value of a temporal pose chain holds the same number of links. A chain that gains a joint is a different structure rather than a later value of the same one, and that constant number is also what makes the interpolation below well defined. An error is raised otherwise.

A temporal pose chain interpolates link by link, each link as a pose does, that is, linearly in position and along the shortest arc in rotation. In the example below a two-link arm turns a quarter turn at its shoulder while its elbow stays where it is: halfway through, the shoulder has turned an eighth of a turn and the elbow is unchanged.

A pose chain is placed where the pose its composition gives is placed, so its distance is the one of that pose, and the distance of a temporal pose chain is the one of the temporal pose its composition gives, `tpose(tposechain)`. That composition multiplies the rotations of the links, so for a chain of more than one link it is a step function, and the distance then holds the value of each instant until the next one.

The temporal pose type `tpose` allows to represent the evolution in time of the position of objects and their orientation. As all temporal types, it comes in three subtypes, namely, instant, sequence, and sequence set. Examples of `tpose` values in these subtypes are given next.

```
SELECT tpose 'Pose(Point(1 1), 0.5)@2001-01-01';
SELECT tpose '{Pose(Point(1 1), 0.3)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-02,
  Pose(Point(1 1), 0.5)@2001-01-03}';
SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01, Pose(Point(1 1), 0.4)@2001-01-02,
  Pose(Point(1 1), 0.5)@2001-01-03]';
SELECT tpose '{[Pose(Point(1 1), 1)@2001-01-01, Pose(Point(2 2), 1)@2001-01-02],
  [Pose(Point(2 2), 2)@2001-01-04, Pose(Point(2 2), 3)@2001-01-05]}';
```

As for temporal point types, the temporal pose and the temporal pose chain types accept type modifiers (or `typmod` in PostgreSQL terminology) to specify the subtype, the dimensionality of the point, and/or the spatial reference identifier (SRID). The possible values for the subtype are `Instant`, `Sequence`, and `SequenceSet` and the possible values for the geometry type are either `Point` or `PointZ`. The three arguments are optional and if any of them is not specified for a column, values of any subtype, dimensionality, and/or SRID are allowed.

```
SELECT asEWKT(tpose(Sequence,5676) 'SRID=5676;
  [Pose(Point(1 1), 0.2)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-03]');
-- SRID=5676;[Pose(POINT(1 1),0.2)@2001-01-01, Pose(POINT(1 1),0.5)@2001-01-03]
SELECT tpose(Sequence) 'Pose(Point(1 1), 0.2)@2001-01-01';
-- ERROR: Temporal type (Instant) does not match column type (Sequence)
SELECT tpose(PointZ) 'Pose(Point(1 1), 0.2)@2001-01-01';
-- Column has Z dimension but the tpose value does not
```

```
SELECT tpose(5676) 'Pose(Point(1 1), 0.2)@2001-01-01';
-- ERROR:  Temporal pose SRID (0) does not match column SRID (5676)
SELECT asEWKT(tposechain(Sequence,5676) 'SRID=5676;
  [PoseChain(Pose(Point(1 1), 0.2))@2001-01-01,
  PoseChain(Pose(Point(1 1), 0.5))@2001-01-03]');
/* SRID=5676;[PoseChain(Pose(POINT(1 1),0.2))@2001-01-01,
   PoseChain(Pose(POINT(1 1),0.5))@2001-01-03] */
```

Temporal pose values of sequence or sequence set subtype are converted into a normal form so that equivalent values have identical representations. For this, consecutive instant values are merged when possible. Three consecutive instant values can be merged into two if the linear functions defining the evolution of values are the same. Examples of transformation into a normal form are as follows.

```
SELECT asText(tpose '[Pose(Point(1 1), 0.2)@2001-01-01, Pose(Point(2 2), 0.4)@2001-01-02,
  Pose(Point(3 3), 0.6)@2001-01-03]');
-- [Pose(POINT(1 1),0.2)@2001-01-01, Pose(POINT(3 3),0.6)@2001-01-03)
SELECT asText(tpose '{[Pose(Point(1 1), 0.2)@2001-01-01, Pose(Point(2 2), 0.3)@2001-01-02,
  Pose(Point(2 2), 0.5)@2001-01-03),
  [Pose(Point(2 2), 0.5)@2001-01-03, Pose(Point(2 2), 0.7)@2001-01-04)}');
/* {[Pose(POINT(1 1),0.2)@2001-01-01, Pose(POINT(2 2),0.3)@2001-01-02,
   Pose(POINT(2 2),0.7)@2001-01-04)} */
```

13.5 Validity of Temporal Poses and Pose Chains

Temporal pose values must satisfy the constraints specified in Section 4.3 so that they are well defined. An error is raised whenever one of these constraints are not satisfied. Examples of incorrect values are as follows.

```
-- Null values are not allowed
SELECT tpose 'NULL@2001-01-01 08:05:00';
SELECT tpose 'Point(0 0)@NULL';
-- Base type is not a pose
SELECT tpose 'Point(0 0)@2001-01-01 08:05:00';
```

We give next the functions and operators for temporal poses. Most functions and operators for temporal types and spatiotemporal types described in the previous chapters can be applied for temporal pose types. Therefore, in the signatures of the functions, the type `pose` can be used whenever the types `base` and `spatial` are specified, and the type `tpose` can be used whenever the types `ttype` and `tspatial` are specified. To avoid redundancy, we only present some examples of these functions and operators for temporal poses and we refer to previous chapters for detailed explanations about them.

13.6 Input and Output

- Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation

```
asText({tpose,tposechain,tpose[],tposechain[]}) → {text,text[]}
asEWKT({tpose,tposechain,tpose[],tposechain[]}) → {text,text[]}
```

```
SELECT asText(tpose 'SRID=4326;
  [Pose(Point(0 0),1)@2001-01-01, Pose(Point(1 1),2)@2001-01-02)');
-- [Pose(POINT(0 0),1)@2001-01-01, Pose(POINT(1 1),2)@2001-01-02)
SELECT asText(ARRAY[pose 'Pose(Point(0 0),1)', 'Pose(Point(1 1),2)']);
-- {"Pose(POINT(0 0),1)","Pose(POINT(1 1),2)"}
SELECT asEWKT(tpose 'SRID=4326;
  [Pose(Point(0 0),1)@2001-01-01, Pose(Point(1 1),2)@2001-01-02)');
-- SRID=4326;[Pose(POINT(0 0),1)@2001-01-01, Pose(POINT(1 1),2)@2001-01-02)
SELECT asEWKT(ARRAY[pose 'Pose(SRID=5676;Point(0 0),1)', 'Pose(SRID=5676;Point(1 1),2)']);
-- {"Pose(SRID=5676;POINT(0 0),1)","Pose(SRID=5676;POINT(1 1),2))"}
```

- Return the Moving Features JSON (MF-JSON) representation

  ```
  asMFJSON({tpose,tposechain}) → text
  ```

  A chain is written as the array of its links, in the order that says in whose frame each of them is read.

  ```
  SELECT asMFJSON(tpose 'Pose(Point(1 2),0.5)@2001-01-01', 3);
  /* {"type":"MovingPose","bbox":[[1,2],[1,2]],"period":{"begin":"2001-01-01T00:00:00",
     "end":"2001-01-01T00:00:00","lower_inc":true,"upper_inc":true},
     "values":[{"position":{"lat":2,"lon":1},"yaw":0.5}],
     "datetimes":["2001-01-01T00:00:00"],"interpolation":"None"} */
  SELECT asMFJSON(tpose 'Pose(Point Z(1 2 3),1,0,0,0)@2001-01-01', 3);
  /* {"type":"MovingPose","bbox":[[1,2,3],[1,2,3]],
     "period":{"begin":"2001-01-01T00:00:00","end":"2001-01-01T00:00:00",
     "lower_inc":true,"upper_inc":true},
     "values":[{"position":{"lat":2,"lon":1,"h":3},"quaternion":{"x":0,"y":0,"z":0,"w":1}}],
     "datetimes":["2001-01-01T00:00:00"],"interpolation":"None"} */
  ```

- Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), the Hexadecimal Well-Known Binary (HexWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

  ```
  asBinary({tpose,tposechain},endian text='') → bytea
  asEWKB({tpose,tposechain},endian text='') → bytea
  asHexWKB({tpose,tposechain},endian text='') → text
  asHexEWKB({tpose,tposechain},endian text='') → text
  ```

  The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used.

  ```
  SELECT asBinary(tpose 'Pose(Point(1 2),1)@2001-01-01');
  -- \x0138000101000000000000f03f0000000000000040000000000000f03f009c57d3c11c0000
  SELECT asEWKB(tpose 'SRID=7844;Pose(Point(1 2),1)@2001-01-01');
  -- \x01380041a41e000001000000000000f03f0000000000000040000000000000f03f009c57d3c11c0000
  SELECT asHexWKB(tpose 'Pose(Point(1 2),1)@2001-01-01');
  -- 0138000101000000000000F03F0000000000000040000000000000F03F009C57D3C11C0000
  SELECT asHexEWKB(tpose 'SRID=3812;Pose(Point(1 2),1)@2001-01-01');
  -- 01380041E40E000001000000000000F03F0000000000000040000000000000F03F009C57D3C11C0000
  ```

- Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation

  ```
  tposeFromText(text) → tpose
  tposeFromEWKT(text) → tpose
  tposechainFromText(text) → tposechain
  ```

  These functions read the temporal text form alone, as the type input `tpose ’...’` does. As in the text of every temporal type, a leading brace opens a discrete sequence or a sequence set, and a value inside it may be quoted. A GeoPose document is read by tposeFromGeoPose.

  ```
  SELECT asEWKT(tposeFromText(text '[Pose(Point(1 2),1)@2001-01-01,
    Pose(Point(3 4),2)@2001-01-02]'));
  -- [Pose(POINT(1 2),1)@2001-01-01, Pose(POINT(3 4),2)@2001-01-02]
  SELECT asEWKT(tposeFromEWKT(text 'SRID=3812;
    [Pose(Point(1 2),1)@2001-01-01, Pose(Point(3 4),2)@2001-01-02]'));
  -- SRID=3812;[Pose(POINT(1 2),1)@2001-01-01, Pose(POINT(3 4),2)@2001-01-02]
  SELECT asEWKT(tposechainFromText('SRID=5676;PoseChain(Pose(Point(1 1),0.5))@2001-01-01'));
  -- SRID=5676;PoseChain(Pose(POINT(1 1),0.5))@2001-01-01
  SELECT asEWKT(tposechainFromBinary(asBinary(tposechain
    'PoseChain(Pose(Point(1 1), 0.5))@2001-01-01')));
  -- PoseChain(Pose(POINT(1 1),0.5))@2001-01-01
  SELECT asEWKT(tposechainFromHexEWKB(asHexEWKB(tposechain 'SRID=5676;
    PoseChain(Pose(Point(1 1), 0.5))@2001-01-01')));
  -- SRID=5676;PoseChain(Pose(POINT(1 1),0.5))@2001-01-01
  ```

- Input from the Moving Features JSON (MF-JSON) representation

```
tposeFromMFJSON(text) → tpose
tposechainFromMFJSON(text) → tposechain
```

A pose chain is written as the array of its links, so a `MovingPoseChain` document holds an array of pose objects for every instant

```
SELECT asEWKT(tposeFromMFJSON(asMFJSON(tpose 'SRID=3812;Pose(Point(1 2),0.5)@2001-01-01',
  3)));
-- SRID=3812;Pose(POINT(1 2),0.5)@2001-01-01
SELECT asEWKT(tposeFromMFJSON(asMFJSON(tpose 'SRID=5676;
  Pose(Point Z(1 2 3),1,0,0,0)@2001-01-01', 3)));
-- SRID=5676;Pose(POINT Z(1 2 3),1,0,0,0)@2001-01-01
SELECT asEWKT(tposechainFromMFJSON(asMFJSON(tposechain 'SRID=3812;
  PoseChain(Pose(Point(1 2),0.5),Pose(Point(3 4),0.25))@2001-01-01', 3)));
-- SRID=3812;PoseChain(Pose(POINT(1 2),0.5),Pose(POINT(3 4),0.25))@2001-01-01
```

- Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), from the Hexadecimal Well-Known Binary (HexWKB), or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

```
tposeFromBinary(bytea) → tpose
tposeFromEWKB(bytea) → tpose
tposeFromHexWKB(text) → tpose
tposeFromHexEWKB(text) → tpose
tposechainFromBinary(bytea) → tposechain
tposechainFromHexWKB(text) → tposechain
tposechainFromHexEWKB(text) → tposechain
```

```
SELECT asEWKT(tposeFromBinary(
  '\x013a0001000000000000f03f0000000000000040000000000000f03f009c57d3c11c0000'));
-- Pose(POINT(1 2),1)@2001-01-01
SELECT asEWKT(tposeFromEWKB(
  '\x013a0001000000000000f03f0000000000000040000000000000f03f009c57d3c11c0000'));
-- SRID=7844;Pose(POINT(1 1),2)@2001-01-01
SELECT asEWKT(tposeFromHexEWKB(
  '013A0041E40E0000E40E0000000000000000F03F...40000000000000F03F009C57D3C11C0000'));
-- SRID=3812;Pose(POINT(1 2),1)@2001-01-01
```

13.7 Constructors

- Constructor for temporal poses having a constant value

```
tpose(pose,timestamptz) → tposeInst
tpose(pose,tstzset) → tposeDiscSeq
tpose(pose,tstzspan,interp text='linear') → tposeContSeq
tpose(pose,tstzspanset,interp text='linear') → tposeSeqSet
tposechain(posechain,{timestamptz,tstzset}) → tposechain
tposechain(posechain,{tstzspan,tstzspanset},interp text='linear') → tposechain
```

```
SELECT asText(tpose('Pose(Point(1 1), 0.5)', timestamptz '2001-01-01'));
-- Pose(POINT(1 1),0.5)@2001-01-01
SELECT asText(tpose('Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)',
  tstzset '{2001-01-01, 2001-01-05}'));
/* {Pose(POINT Z (1 1 1),0.5,0.5,0.5,0.5)@2001-01-01,
   Pose(POINT Z (1 1 1),0.5,0.5,0.5,0.5)@2001-01-05} */
SELECT asText(tpose('Pose(Point(1 1), 0.5)', tstzspan '[2001-01-01, 2001-01-02]'));
-- [Pose(POINT(1 1),0.5)@2001-01-01, Pose(POINT(1 1),0.5)@2001-01-02]
SELECT asText(tpose('Pose(Point(1 1), 0.2)', tstzspanset '{[2001-01-01, 2001-01-03]}',
  'step'));
-- Interp=Step;{[Pose(POINT(1 1),0.2)@2001-01-01, Pose(POINT(1 1),0.2)@2001-01-03]}
SELECT asEWKT(tposechain(posechain 'PoseChain(Pose(Point(1 1), 0.5))',
  timestamptz '2001-01-01'));
-- PoseChain(Pose(POINT(1 1),0.5))@2001-01-01
SELECT asEWKT(tposechain(posechain 'PoseChain(Pose(Point(1 1), 0.5))',
  tstzset '{2001-01-01, 2001-01-02}'));
/* {PoseChain(Pose(POINT(1 1),0.5))@2001-01-01,
   PoseChain(Pose(POINT(1 1),0.5))@2001-01-02} */
SELECT asEWKT(tposechain(posechain 'PoseChain(Pose(Point(1 1), 0.5))',
  tstzspan '[2001-01-01, 2001-01-02]'));
/* [PoseChain(Pose(POINT(1 1),0.5))@2001-01-01,
   PoseChain(Pose(POINT(1 1),0.5))@2001-01-02] */
```

- Constructor for temporal poses of sequence subtype

```
tposeSeq(tposeInst[],interp text={'step','linear'},lower_inc boolean=true,
  upper_inc boolean=true) → tposeSeq
```

```
SELECT asText(tposeSeq(ARRAY[tpose 'Pose(Point(1 1), 0.3)@2001-01-01',
  'Pose(Point(2 2), 0.5)@2001-01-02', 'Pose(Point(1 1), 0.5)@2001-01-03']));
/* {Pose(Point(1 1),0.3)@2001-01-01, Pose(Point(2 2),0.5)@2001-01-02,
   Pose(Point(1 1),0.5)@2001-01-03} */
SELECT asText(tposeSeq(ARRAY[tpose 'Pose(Point(1 1), 0.2)@2001-01-01',
  'Pose(Point(1 1), 0.4)@2001-01-02', 'Pose(Point(1 1), 0.5)@2001-01-03']));
/* [Pose(POINT(1 1),0.2)@2001-01-01, Pose(POINT(1 1),0.4)@2001-01-02,
   Pose(POINT(1 1),0.5)@2001-01-03] */
```

- Constructor for temporal poses of sequence set subtype

```
tposeSeqSet(tpose[]) → tposeSeqSet
tposeSeqSetGaps(tposeInst[],maxt interval=NULL,maxdist float=NULL,
  interp text='linear') → tposeSeqSet
```

```
SELECT asText(tposeSeqSet(ARRAY[tpose '[Pose(Point(1 1),0.2)@2001-01-01,
  Pose(Point(2 2),0.4)@2001-01-02]',
  '[Pose(Point(2 2),0.6)@2001-01-03, Pose(Point(2 2),0.8)@2001-01-04]']));
/* {[Pose(Point(1 1),0.2)@2001-01-01, Pose(Point(2 2),0.4)@2001-01-02],
    [Pose(Point(2 2),0.6)@2001-01-03, Pose(Point(2 2),0.6)@2001-01-04]} */
SELECT asText(tposeSeqSetGaps(ARRAY[tpose 'Pose(Point(1 1),0.1)@2001-01-01',
  'Pose(Point(1 1),0.3)@2001-01-03', 'Pose(Point(1 1),0.5)@2001-01-05'], '1 day'));
/* {[Pose(Point(1 1),0.1)@2001-01-01], [Pose(Point(1 1),0.3)@2001-01-03],
    [Pose(Point(1 1),0.5)@2001-01-05]} */
```

- Construct a 2D temporal pose from a temporal geometry point and a temporal float

```
tpose(tgeompoint,tfloat) → tpose
```

The time frame of the result is the intersection of the time frames of the temporal point and the temporal float. If they do not intersect, a null value is returned

```
SELECT asText(tpose(tgeompoint '[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02)',
  tfloat '[1@2001-01-01, 2@2001-01-02)'));
-- [Pose(POINT(1 1),1)@2001-01-01, Pose(POINT(2 2),2)@2001-01-02)
SELECT asText(tpose(tgeompoint '[POINT(1 1)@2001-01-01, POINT(2 2)@2001-01-02)',
  tfloat '[2@2001-01-03, 3@2001-01-04)'));
-- NULL
```

13.8 Conversions

A temporal pose value can be converted to and from a temporal point, which is a temporal geometry point or a temporal geography point whatever the kind of the pose, see [?listitem]. This can be done using an explicit `CAST` or using the `::` notation. Converting a temporal pose to the temporal point type of the other reference frame raises an error. A null value is returned if any of the composing geometry point values cannot be converted into a `pose` value.

- Convert a temporal pose, planar or geodetic, into a temporal geometry point

```
{tpose,tposechain}::tgeompoint
```

```
SELECT asText((tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(1 1), 0.3)@2001-01-02)')::tgeompoint);
-- [POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02)
SELECT asEWKT((tpose '[Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)@2001-01-01,
  Pose(Point Z(2 2 2), 1, 0, 0, 0)@2001-01-02)')::tgeompoint);
-- [POINT Z (1 1 1)@2001-01-01, POINT Z (2 2 2)@2001-01-02)
```

- Convert a temporal pose, planar or geodetic, into a temporal geography point

```
{tpose,tposechain}::tgeogpoint
```

```
SELECT asText((tpose '[GeodPose(Point(1 1), 0.2)@2001-01-01,
  GeodPose(Point(1 1), 0.3)@2001-01-02)')::tgeogpoint);
-- [POINT(1 1)@2001-01-01, POINT(1 1)@2001-01-02)
```

13.9 Accessors

- Return the values

```
getValues({tpose,tposechain}) → {poseset,posechainset}
```

```
SELECT asText(getValues(tpose '{[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-02)}'));
-- {"Pose(POINT(1 1),0.3)","Pose(POINT(1 1),0.5)"}
SELECT asEWKT(getValues(tpose 'SRID=5676;
  {[Pose(Point(1 1), 0.3)@2001-01-01, Pose(Point(1 1), 0.3)@2001-01-02)}'));
-- SRID=5676;{"Pose(POINT(1 1),0.3)"}
```

- Return the points

```
points(tpose) → geomset
```

```
SELECT asEWKT(points(tpose 'SRID=5676;
  {Pose(Point(3 3), 0.3)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-02}'));
-- SRID=5676;{POINT(1 1), POINT(3 3)}
```

- Return the trajectory

```
trajectory(tpose) → geometry
```

A pose has a position and an orientation but no extent, so the geometry it covers over time is the trajectory of its position, as for a temporal point. A pose that interpolates moves along the line between two consecutive positions; one that does not stands at each of them and covers nothing between them.

```
SELECT ST_AsText(trajectory(tpose '[Pose(Point(1 1),0.5)@2001-01-01,
  Pose(Point(3 3),0.5)@2001-01-02]'));
-- LINESTRING(1 1,3 3)
SELECT ST_AsText(trajectory(tpose 'Interp=Step;
  [Pose(Point(1 1),0.5)@2001-01-01, Pose(Point(3 3),0.5)@2001-01-02]'));
-- MULTIPOINT((1 1),(3 3))
```

- Return the value at a timestamp

```
valueAtTimestamp(tpose,timestamptz) → pose
```

```
SELECT asText(valueAtTimestamp(tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(3 3), 0.5)@2001-01-03)', '2001-01-02'));
-- Pose(POINT(2 2),0.4)
SELECT asText(valueAtTimestamp(tpose
  '[Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)@2001-01-01,
  Pose(Point Z(3 3 3), 1, 0, 0, 0)@2001-01-03)', timestamptz '2001-01-02'), 6);
-- Pose(POINT Z (2 2 2),0.866025,0.288675,0.288675,0.288675)
```

- Return the speed of a temporal pose as a temporal float, the magnitude of the position-component velocity (distance travelled per unit time)

```
speed(tpose) → tfloat
```

The orientation is not considered; for angular velocity see `angularSpeed`.

```
SELECT asText(speed(tpose '[Pose(Point(0 0), 0)@2001-01-01 08:00,
  Pose(Point(1 1), 0.5)@2001-01-02 08:00]') * 3600);
-- Interp=Step;[1.414213562373095@2001-01-01, 1.414213562373095@2001-01-02]
```

The answer is given in the units of the SRID's CRS, which multiplied by 3600 in this case, corresponds to meters per hour.

- Return the angular speed of a temporal pose as a step-interpolated temporal float (radians per unit time)

```
angularSpeed(tpose) → tfloat
```

For 2D poses this is the per-segment shortest-arc theta delta divided by the segment duration; for 3D poses it is the SLERP arc angle `2 · acos(|q1 · q2|)` divided by the segment duration. SLERP is by construction constant-angular-velocity along a segment, so the result is piecewise-constant.

```
-- 90-degree yaw rotation over one day -> pi/2 rad / 86400 s
SELECT asText(angularSpeed(tpose '[Pose(Point(0 0 0), 1, 0, 0, 0)@2001-01-01,
  Pose(Point(0 0 0), 0.7071067811865476, 0, 0, 0.7071067811865475)@2001-01-02]'));
-- Interp=Step;[1.8181e-5@2001-01-01, 1.8181e-5@2001-01-02]
```

- Return the value of a temporal instant, the set of values, or the time span

```
getValue({tpose,tposechain}) → {pose,posechain}
timeSpan({tpose,tposechain}) → tstzspan
```

```
SELECT asEWKT(getValue(tposechain 'PoseChain(Pose(Point(1 1), 0.5))@2001-01-01'));
-- PoseChain(Pose(POINT(1 1),0.5))
SELECT asEWKT(getValues(tposechain '{PoseChain(Pose(Point(1 1), 0.5))@2001-01-01,
  PoseChain(Pose(Point(2 2), 0.5))@2001-01-02}'));
-- {"PoseChain(Pose(POINT(1 1),0.5))", "PoseChain(Pose(POINT(2 2),0.5))"}
SELECT timeSpan(tposechain '[PoseChain(Pose(Point(1 1), 0.5))@2001-01-01,
  PoseChain(Pose(Point(2 2), 0.5))@2001-01-02]');
-- [2001-01-01, 2001-01-02]
```

13.10 Transformations

- Transform to another subtype

```
tposeInst(tpose) → tposeInst
tposeSeq(tpose) → tposeSeq
tposeSeqSet(tpose) → tposeSeqSet
```

```
SELECT asText(tposeSeq(tpose 'Pose(Point(1 1), 0.5)@2001-01-01', 'discrete'));
-- {Pose(POINT(1 1),0.5)@2001-01-01}
SELECT asText(tposeSeq(tpose 'Pose(Point(1 1), 0.5)@2001-01-01'));
-- [Pose(POINT(1 1),0.5)@2001-01-01]
SELECT asText(tposeSeqSet(tpose 'Pose(PointZ(1 1 1), 0.5, 0.5, 0.5, 0.5)@2001-01-01'));
-- {[Pose(POINT Z (1 1 1),0.5,0.5,0.5,0.5)@2001-01-01]}
```

- Transform to another interpolation

```
setInterp(tpose,interp text) → tpose
```

```
SELECT asText(setInterp(tpose 'Pose(Point(1 1),0.2)@2001-01-01','linear'));
-- [Pose(POINT(1 1),0.2)@2001-01-01]
SELECT asText(setInterp(tpose '{[Pose(Point Z(1 1 1),1,0,0,0)@2001-01-01],
  [Pose(Point Z(2 2 2),0,0,0,1)@2001-01-02]}', 'discrete'));
-- {Pose(POINT Z (1 1 1),1,0,0,0)@2001-01-01, Pose(POINT Z (2 2 2),0,0,0,1)@2001-01-02}
```

- Round the points and the orientations of the temporal pose to the number of decimal places

```
round({tpose,tposechain},integer=0) → {tpose,tposechain}
shiftTime({tpose,tposechain},interval) → {tpose,tposechain}
```

```
SELECT asText(round(tpose '{[Pose(Point(1 1.123456789),0.123456789)@2001-01-01,
  Pose(Point(1 1),0.5)@2001-01-02)}', 3));
-- {[Pose(POINT(1 1.123),0.123)@2001-01-01, Pose(POINT(1 1),0.5)@2001-01-02)}
SELECT asEWKT(round(tposechain
  'PoseChain(Pose(Point(1.123456789 2.123456789), 0.5))@2001-01-01', 3));
-- PoseChain(Pose(POINT(1.123 2.123),0.5))@2001-01-01
SELECT asEWKT(shiftTime(tposechain 'PoseChain(Pose(Point(1 1), 0.5))@2001-01-01',
  interval '1 day'));
-- PoseChain(Pose(POINT(1 1),0.5))@2001-01-02
```

- Return a temporal value translated by given offsets

```
translate({tpose,tposechain},deltax float,deltay float,deltaz float=0)
  → {tpose,tposechain}
```

A vertical offset takes a two-dimensional value out of its plane, so the result is three-dimensional.

The chain of every instant moves its first link, which is expressed in the outer frame of the chain, and keeps the others.

The motion is a planar map, so a geodetic value raises an error.

```
SELECT asText(translate(tpose '[Pose(Point(1 1),0.3)@2001-01-01,
  Pose(Point(3 2),1.1)@2001-01-02]', 1, 2), 6);
-- [Pose(POINT(2 3),0.3)@2001-01-01, Pose(POINT(4 4),1.1)@2001-01-02]
SELECT asText(translate(tpose '[Pose(Point(1 1),0.3)@2001-01-01,
  Pose(Point(3 2),1.1)@2001-01-02]', 1, 2, 3), 6);
/* [Pose(POINT Z (2 3 3),0.988771,0,0,0.149438)@2001-01-01,
   Pose(POINT Z (4 4 3),0.852525,0,0,0.522687)@2001-01-02] */
```

- Return a temporal value rotated counter-clockwise about the origin or about a point

```
rotate({tpose,tposechain},angle float) → {tpose,tposechain}
rotate({tpose,tposechain},angle float,x0 float,y0 float) → {tpose,tposechain}
```

The chain of every instant moves its first link, which is expressed in the outer frame of the chain, and keeps the others.

The motion is a planar map, so a geodetic value raises an error.

```
SELECT asText(rotate(tpose '[Pose(Point(1 1),0.3)@2001-01-01,
  Pose(Point(3 2),1.1)@2001-01-02]', 0.7, 1, 1), 6);
-- [Pose(POINT(1 1),1)@2001-01-01, Pose(POINT(1.885467 3.053278),1.8)@2001-01-02]
SELECT asText(rotate(tposechain '[PoseChain(Pose(Point(1 1),0.3),Pose(Point(2 0),0.5)) ↩
    @2001-01-01,
  PoseChain(Pose(Point(3 2),1.1),Pose(Point(2 0),0.5))@2001-01-02]', 0.7, 1, 1), 6);
/* [PoseChain(Pose(POINT(1 1),1),Pose(POINT(2 0),0.5))@2001-01-01,
   PoseChain(Pose(POINT(1.885467 3.053278),1.8),Pose(POINT(2 0),0.5))@2001-01-02] */
```

- Return a temporal value rotated counter-clockwise about the x axis

```
rotateX({tpose,tposechain},angle float) → {tpose,tposechain}
```

The rotation takes a two-dimensional value out of its plane, so the result is three-dimensional: each position gains the height the rotation gives it, and each orientation is the rotation composed with the angle about the vertical that the value states.

The chain of every instant moves its first link, which is expressed in the outer frame of the chain, and keeps the others.

The motion is a planar map, so a geodetic value raises an error.

```
SELECT asText(rotateX(tpose '[Pose(Point(1 1),0.3)@2001-01-01,
  Pose(Point(3 2),1.1)@2001-01-02]', 0.7), 6);
/* [Pose(POINT Z (1 0.764842 0.644218),0.928825,0.339047,-0.051242,0.140378)@2001-01-01,
   Pose(POINT Z (3 1.529684 1.288435),0.800838,0.292329,-0.179228,0.490998)@2001-01-02] */
```

- Return a temporal value rotated counter-clockwise about the y axis

```
rotateY({tpose,tposechain},angle float) → {tpose,tposechain}
```

The rotation takes a two-dimensional value out of its plane, so the result is three-dimensional: each position gains the height the rotation gives it, and each orientation is the rotation composed with the angle about the vertical that the value states.

The chain of every instant moves its first link, which is expressed in the outer frame of the chain, and keeps the others.

The motion is a planar map, so a geodetic value raises an error.

```
SELECT asText(rotateY(tpose '[Pose(Point(1 1),0.3)@2001-01-01,
  Pose(Point(3 2),1.1)@2001-01-02]', 0.7), 6);
/* [Pose(POINT Z (0.764842 1 -0.644218),0.928825,0.051242,0.339047,0.140378)@2001-01-01,
   Pose(POINT Z (2.294527 2 -1.932653),0.800838,0.179228,0.292329,0.490998)@2001-01-02] */
```

- Return a temporal value rotated counter-clockwise about the z axis

```
rotateZ({tpose,tposechain},angle float) → {tpose,tposechain}
```

The chain of every instant moves its first link, which is expressed in the outer frame of the chain, and keeps the others.

The motion is a planar map, so a geodetic value raises an error.

```
SELECT asText(rotateZ(tpose '[Pose(Point(1 1),0.3)@2001-01-01,
  Pose(Point(3 2),1.1)@2001-01-02]', 0.7), 6);
/* [Pose(POINT(0.120625 1.40906),1)@2001-01-01,
   Pose(POINT(1.006091 3.462337),1.8)@2001-01-02] */
```

- Return a temporal pose projected to the ground

```
force2D(tpose) → tpose
```

Each instant keeps the x and the y of its position and the yaw of its orientation. The rotation of a segment in three dimensions takes the shortest arc between two quaternions, so the result agrees with the value at the instants, and between them where the rotation is about the vertical. A value in two dimensions stays as it is.

```
SELECT asText(force2D(tpose '[Pose(Point(1 1 5),0.7071067811865476,0,0,0.7071067811865476) ↩
    @2001-01-01,
  Pose(Point(3 2 7),1,0,0,0)@2001-01-02]'), 6);
-- [Pose(POINT(1 1),1.570796)@2001-01-01, Pose(POINT(3 2),0)@2001-01-02]
```

13.11 Modifications

- Append a temporal instant or a temporal sequence

```
appendInstant({tpose,tposechain},{tposeInst,tposechainInst}) → {tpose,tposechain}
appendSequence({tpose,tposechain},{tposeSeq,tposechainSeq}) → {tpose,tposechain}
```

```
SELECT asText(appendInstant(tpose 'Pose(Point(1 1), 0.5)@2001-01-01',
  'Pose(Point(2 2), 1)@2001-01-02'));
-- [Pose(POINT(1 1),0.5)@2001-01-01, Pose(POINT(2 2),1)@2001-01-02]
SELECT asText(appendSequence(tpose '[Pose(Point(1 1), 0.5)@2001-01-01]',
  '[Pose(Point(2 2), 1)@2001-01-02]'));
-- {[Pose(POINT(1 1),0.5)@2001-01-01], [Pose(POINT(2 2),1)@2001-01-02]}
```

- Merge the temporal poses

```
merge({tpose,tposechain},{tpose,tposechain}) → {tpose,tposechain}
merge({tpose,tposechain}[]) → {tpose,tposechain}
merge({tpose,tposechain}) → {tpose,tposechain}
```

The name `merge` also denotes an aggregate, which merges the values of a set of rows.

```
SELECT asText(merge(tpose '[Pose(Point(1 1 1),1,0,0,0)@2001-01-01,
  Pose(Point(2 2 2),0,1,0,0)@2001-01-02]',
  '[Pose(Point(2 2 2),0,1,0,0)@2001-01-02, Pose(Point(3 3 3),0,0,0,1)@2001-01-03]'));
/* [Pose(POINT Z (1 1 1),1,0,0,0)@2001-01-01, Pose(POINT Z (2 2 2),0,1,0,0)@2001-01-02,
   Pose(POINT Z (3 3 3),0,0,0,1)@2001-01-03] */
SELECT asText(merge(ARRAY[tpose '{[Pose(Point(1 1),0.1)@2001-01-01,
  Pose(Point(2 2),0.2)@2001-01-02],
  [Pose(Point(3 3),0.3)@2001-01-03, Pose(Point(4 4),0.4)@2001-01-04]}',
  '[Pose(Point(4 4),0.4)@2001-01-04, Pose(Point(5 5),0.5)@2001-01-05]']));
/* {[Pose(POINT(1 1),0.1)@2001-01-01, Pose(POINT(2 2),0.2)@2001-01-02],
   [Pose(POINT(3 3),0.3)@2001-01-03, Pose(POINT(5 5),0.5)@2001-01-05]} */
WITH temp(inst) AS (
  SELECT tpose 'Pose(Point(1 1),0.1)@2001-01-01' UNION
  SELECT tpose 'Pose(Point(2 2),0.2)@2001-01-02' UNION
  SELECT tpose 'Pose(Point(3 3),0.3)@2001-01-03' )
SELECT asText(merge(inst)) FROM temp;
/* {Pose(POINT(1 1),0.1)@2001-01-01, Pose(POINT(2 2),0.2)@2001-01-02,
   Pose(POINT(3 3),0.3)@2001-01-03} */
SELECT asText(merge(temp)) FROM (VALUES
  (tposechain 'PoseChain(Pose(Point(1 1), 0.1))@2001-01-01'),
  (tposechain 'PoseChain(Pose(Point(2 2), 0.2))@2001-01-02')) t(temp);
/* {PoseChain(Pose(POINT(1 1),0.1))@2001-01-01,
   PoseChain(Pose(POINT(2 2),0.2))@2001-01-02} */
```

- Insert or update the second temporal pose chain into the first one, or delete a time value from it

```
insert({tpose,tposechain},{tpose,tposechain},connect boolean=true) → {tpose,tposechain}
update({tpose,tposechain},{tpose,tposechain},connect boolean=true) → {tpose,tposechain}
deleteTime({tpose,tposechain},times,connect boolean=true) → {tpose,tposechain}
```

```
SELECT asText(deleteTime(tposechain
  '{PoseChain(Pose(Point(1 1),0.5), Pose(Point(2 0),0))@2001-01-01,
  PoseChain(Pose(Point(3 3),0.5), Pose(Point(4 0),0))@2001-01-02}',
  timestamptz '2001-01-02'));
-- {PoseChain(Pose(POINT(1 1),0.5),Pose(POINT(2 0),0))@2001-01-01}
```

13.12 Restrictions

- Restrict to (the complement of) a value or a set of values

```
atValue({tpose,tposechain},{pose,posechain}) → {tpose,tposechain}
minusValue({tpose,tposechain},{pose,posechain}) → {tpose,tposechain}
atValues({tpose,tposechain},{poseset,posechainset}) → {tpose,tposechain}
minusValues({tpose,tposechain},{poseset,posechainset}) → {tpose,tposechain}
```

```
SELECT atValue(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-03]', 'Pose(Point(2 2), 0.5)');
-- {[Pose(POINT(2 2),0.5)@2001-01-02]}
SELECT minusValue(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-03]', 'Pose(Point(2 2), 0.5)');
/* {[Pose(Point(2 2),0.3)@2001-01-01, Pose(Point(2 2),0.5)@2001-01-02),
   (Pose(Point(2 2),0.5)@2001-01-02, Pose(Point(2 2),0.7)@2001-01-03]} */
```

- Restrict to (the complement of) a geometry

```
atGeometry(tpose,geometry) → tpose
minusGeometry(tpose,geometry) → tpose
```

```
SELECT atGeometry(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-03]', 'Polygon((40 40,40 50,50 50,50 40,40 40))');
SELECT minusGeometry(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-03]', 'Polygon((40 40,40 50,50 50,50 40,40 40))');
/* {(Pose(Point(2 2),0.342593)@2001-01-01 05:06:40.364673,
   Pose(Point(2 2),0.7)@2001-01-03]} */
```

- Restrict to (the complement of) an elevation span

```
atElevation(tpose,floatspan) → tpose
minusElevation(tpose,floatspan) → tpose
```

The elevation of a pose is the elevation of its position, so the restriction keeps the times at which that position lies within the span. The temporal pose must have Z dimension.

```
SELECT asText(atElevation(tpose '[Pose(Point(0 0 0),1,0,0,0)@2001-01-01,
  Pose(Point(0 0 4),1,0,0,0)@2001-01-05]', floatspan '[1, 2]'));
/* {[Pose(POINT Z (0 0 1),1,0,0,0)@2001-01-02,
   Pose(POINT Z (0 0 2),1,0,0,0)@2001-01-03]} */
SELECT asText(minusElevation(tpose '[Pose(Point(0 0 0),1,0,0,0)@2001-01-01,
  Pose(Point(0 0 4),1,0,0,0)@2001-01-05]', floatspan '[1, 2]'));
/* {[Pose(POINT Z (0 0 0),1,0,0,0)@2001-01-01, Pose(POINT Z (0 0 1),1,0,0,0)@2001-01-02),
   (Pose(POINT Z (0 0 2),1,0,0,0)@2001-01-03,
   Pose(POINT Z (0 0 4),1,0,0,0)@2001-01-05]} */
```

- Restrict a temporal pose chain to a time or to a value

```
atTime({tpose,tposechain},times) → {tpose,tposechain}
```

```
SELECT asEWKT(atTime(tposechain '[PoseChain(Pose(Point(1 1), 0.5))@2001-01-01,
  PoseChain(Pose(Point(2 2), 0.5))@2001-01-03]', timestamptz '2001-01-02'));
-- PoseChain(Pose(POINT(1.5 1.5),0.5))@2001-01-02
SELECT asEWKT(atValue(tposechain '{PoseChain(Pose(Point(1 1), 0.5))@2001-01-01,
  PoseChain(Pose(Point(2 2), 0.5))@2001-01-02}',
  posechain 'PoseChain(Pose(Point(2 2), 0.5))'));
-- {PoseChain(Pose(POINT(2 2),0.5))@2001-01-02}
```

13.13 Spatial Reference System

- Return or set the spatial reference identifier, which for a temporal pose chain is the identifier of its outer frame

```
SRID({tpose,tposechain}) → integer
setSRID({tpose,tposechain},integer) → {tpose,tposechain}
```

```
SELECT SRID(tpose 'SRID=5676;
  [Pose(Point(0 0),1)@2001-01-01, Pose(Point(1 1),2)@2001-01-02)'));
-- 5676
SELECT asEWKT(setSRID(tpose '[Pose(Point(0 0),1)@2001-01-01,
  Pose(Point(1 1),2)@2001-01-02)', 5676));
-- SRID=5676;[Pose(POINT(0 0),1)@2001-01-01, Pose(POINT(1 1),2)@2001-01-02)
```

- Transform to a spatial reference identifier

```
transform({tpose,tposechain},integer) → {tpose,tposechain}
transformPipeline({tpose,tposechain},pipeline text,srid integer,
  is_forward boolean=true) → {tpose,tposechain}
```

The chain of every instant is transformed as the corresponding static chain is, which is to say that only the outer link is transformed, as described for transform.

Each instant's pose is transformed as the corresponding static pose is, orientation correction included, as described for transform. The same holds for a `poseset`. A `trgeometry` pairs its poses with a reference geometry that shares their frame, and transformation to another SRID is not supported for it.

```
SELECT asEWKT(transform(tpose 'SRID=4326;Pose(Point(4.35 50.85),1)@2001-01-01', 3812));
-- SRID=3812;Pose(POINT(648679.0180353033 671067.0556381135),1)@2001-01-01
```

```
WITH test(tpose, pipeline) AS (
  SELECT tpose 'SRID=4326;{Pose(Point(4.3525 50.846667),1)@2001-01-01,
    Pose(Point(-0.1275 51.507222),2)@2001-01-02}',
    text 'urn:ogc:def:coordinateOperation:EPSG::16031' )
SELECT asEWKT(transformPipeline(transformPipeline(tpose, pipeline, 4326), pipeline, 4326,
  false), 6) FROM test;
/* SRID=4326;{Pose(POINT(4.3525 50.846667),1)@2001-01-01,
   Pose(POINT(-0.1275 51.507222),2)@2001-01-02} */
```

13.14 Bounding Box Operations

- Topological operators

```
{tstzspan,stbox,tpose,tposechain} {&&, <@, @>, ~=, -|-}
  {tstzspan,stbox,tpose,tposechain} → boolean
```

```
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' && tstzspan '[2001-01-02,2001-01-04]';
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-02]' @> stbox(pose 'Pose(Point(1 1), 0.5)');
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' ~= tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.35)@2001-01-02, Pose(Point(1 1), 0.5)@2001-01-03]';
-- true
```

- Position operators

```
{stbox,tpose} {<<, &<, >>, &>} {stbox,tpose}  → boolean
{stbox,tpose} {<<|, &<|, |>>, |&>} {stbox,tpose}  → boolean
{stbox,tpose} {<</, &</, />>, /&>} {stbox,tpose}  → boolean
{tstzspan,stbox,tpose} {<<#, &<#, #>>, #&>} {tstzspan,stbox,tpose} → boolean
```

```
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-02]' << stbox(pose 'Pose(Point(2 2), 0.5)');
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-02]' <<| stbox(pose 'Pose(Point(2 2), 0.5)');
-- true
SELECT tpose '[Pose(Point(1 1 1), 1,0,0,0)@2001-01-01,
  Pose(Point(1 1 1), 1,0,0,0)@2001-01-02]' <</ stbox(pose 'Pose(Point(2 2 2), 1,0,0,0)');
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-02]' <<# tstzspan '[2001-01-03, 2001-01-04]';
-- true
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-03, Pose(Point(1 1), 0.5)@2001-01-05]' #>>
  tpose '[Pose(Point(1 1), 0.3)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-02]';
-- true
```

- Return the spatiotemporal bounding box expanded in the spatial dimension, or the array of boxes covering the temporal pose chain

```
expandSpace({tpose,tposechain},float) → stbox
stboxes({tpose,tposechain}) → stbox[]
```

```
SELECT expandSpace(tposechain
  '[PoseChain(Pose(Point(1 1),0.5), Pose(Point(2 0),0))@2001-01-01,
  PoseChain(Pose(Point(3 3),0.5), Pose(Point(4 0),0))@2001-01-03]', 2);
/* STBOX XT(((-1,-1),(8.510330247561491,6.917702154416812)),
   [2001-01-01, 2001-01-03]) */
```

13.15 Distance Operations

Operands of different dimensions read the dimensions both of them have, see Section 8.3.

We present next the functions that compute distance operations for temporal poses and temporal pose chains. Notice that for these operations only the point component is considered, not the orientation. A temporal pose chain is measured by the temporal pose its composition gives, `tpose(tposechain)`, which is a step function for a chain of more than one link.

- Return the smallest distance ever

```
{geo,pose,tpose,stbox} |=| {geo,pose,tpose,stbox} → float
{geo,posechain,tposechain,stbox} |=| {geo,posechain,tposechain,stbox} → float
```

```
SELECT tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-02]' |=| geometry 'Linestring(50 50,55 55)';
-- 67.88225099390856
SELECT tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-02]' |=| pose 'Pose(Point(1 1), 0.5)';
-- 1.4142135623730951
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' |=| pose 'Pose(Point(2 2), 0.2)';
-- 1.4142135623730951
SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' |=| geometry 'Linestring(2 2,2 1,3 1)';
-- 1
```

The operator `|=|` can be used for doing nearest neightbor searches using GiST or SP-GiST indexes (see Section 10.2).

- Return the instant of the first temporal pose at which the two arguments are at the nearest distance

```
nearestApproachInstant({geo,pose,tpose},{geo,pose,tpose}) → tpose
nearestApproachInstant({geo,posechain,tposechain},{geo,posechain,tposechain}) →  ↩
    tposechain
```

```
SELECT asText(nearestApproachInstant(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
-- Pose(POINT(2 2),0.3)@2001-01-01
SELECT asText(nearestApproachInstant(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-02]', pose 'Pose(Point(1 1), 0.5)'));
-- Pose(POINT(2 2),0.3)@2001-01-01
```

- Return the line connecting the nearest approach point

```
shortestLine({geo,pose,tpose},{geo,pose,tpose}) → geometry
shortestLine({geo,posechain,tposechain},{geo,posechain,tposechain}) → geometry
```

The function will only return the first line that it finds if there are more than one.

```
SELECT ST_AsText(shortestLine(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-02]', geometry 'Linestring(50 50,55 55)'));
-- LINESTRING(2 2,50 50)
SELECT ST_AsText(shortestLine(tpose '[Pose(Point(2 2), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.7)@2001-01-02]', pose 'Pose(Point(1 1), 0.5)'));
-- LINESTRING(2 2,1 1)
```

- Return the temporal distance

```
tDistance({geompoint,geogpoint,pose,tpose},{geompoint,geogpoint,pose,tpose}) → tfloat
tDistance({geompoint,geogpoint,posechain,tposechain},{geompoint,geogpoint,posechain, ↩
    tposechain}) → tfloat
{geompoint,geogpoint,pose,tpose} <-> {geompoint,geogpoint,pose,tpose} → tfloat
{geompoint,geogpoint,posechain,tposechain} <-> {geompoint,geogpoint,posechain,tposechain}  ↩
    → tfloat
```

```
SELECT round(tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' <-> pose 'Pose(Point(2 2), 0.2)', 6);
-- [1.414214@2001-01-01, 1.414214@2001-01-03]
SELECT round(tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-03]' <-> geometry 'Point(50 50)', 6);
-- [69.296465@2001-01-01, 69.296465@2001-01-03]
SELECT round(tpose '[Pose(Point(1 1), 0.3)@2001-01-01,
  Pose(Point(2 2), 0.5)@2001-01-03]' <->
  tpose '[Pose(Point(1 2), 0.3)@2001-01-02, Pose(Point(2 1), 0.5)@2001-01-04]', 6);
```

```
-- [0.707107@2001-01-02, 0.5@2001-01-02 12:00:00, 0.707107@2001-01-03]
SELECT asText(round(tposechain '[PoseChain(Pose(Point(0 0),0), Pose(Point(1 0),0))@2001 ←
    -01-01,
  PoseChain(Pose(Point(4 0),0), Pose(Point(1 0),0))@2001-01-05]' <->
  geometry 'Point(1 0)', 6));
-- Interp=Step;[0@2001-01-01, 4@2001-01-05]
```

13.16 Ever and Always Relationships

- Ever and always relationships

```
eContains(geometry,{tpose,tposechain}) → boolean
eCovers(geometry,{tpose,tposechain}) → boolean
eDisjoint({geo,tpose,tposechain},{geo,tpose,tposechain}) → boolean
eDwithin({geo,tpose,tposechain},{geo,tpose,tposechain},float) → boolean
eIntersects({geo,tpose,tposechain},{geo,tpose,tposechain}) → boolean
eTouches(geometry,{tpose,tposechain}) → boolean
eTouches({tpose,tposechain},geometry) → boolean
aContains(geometry,{tpose,tposechain}) → boolean
aCovers(geometry,{tpose,tposechain}) → boolean
aDisjoint({geo,tpose,tposechain},{geo,tpose,tposechain}) → boolean
aDwithin({geometry,tpose,tposechain},{geometry,tpose,tposechain},float) → boolean
aIntersects({geo,tpose,tposechain},{geo,tpose,tposechain}) → boolean
aTouches(geometry,{tpose,tposechain}) → boolean
aTouches({tpose,tposechain},geometry) → boolean
```

The *ever* and *always* relationships determine whether the topological or distance relationship is ever or always satisfied and result in a `boolean`. A temporal pose answers them at the position it occupies, so a value with linear interpolation is answered along its trajectory rather than at its instants alone. The temporal pose declares the relationships a moving point answers: `eContains`, `aContains`, `eCovers` and `aCovers` take the geometry first only, and `eTouches` and `aTouches` have no direction between two temporal poses, since a moving point neither contains nor covers a geometry and two moving points do not touch. A temporal pose chain answers them at the position of the temporal pose its composition gives, `tpose(tposechain)`.

```
SELECT eContains(geometry 'Polygon((0 0,0 50,50 50,50 0,0 0))',
  tpose '[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(3 3),0.3)@2001-01-03]');
-- true
SELECT aCovers(geometry 'Polygon((0 0,0 50,50 50,50 0,0 0))',
  tpose '[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(3 3),0.3)@2001-01-03]');
-- true
SELECT eDisjoint(geometry(pose 'Pose(Point(2 2), 0.0)'),
  tpose '[Pose(Point(1 1),0.1)@2001-01-01, Pose(Point(1 1),0.3)@2001-01-03)');
-- true
SELECT eIntersects(tpose '[Pose(Point(1 1),0.1)@2001-01-01,
  Pose(Point(3 3),0.1)@2001-01-03]',
  tpose '[Pose(Point(3 3),0.1)@2001-01-01, Pose(Point(1 1),0.1)@2001-01-03]');
-- true
SELECT eTouches(tpose 'Pose(Point(0 0), 0.1)@2001-01-01',
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- true
SELECT aDwithin(tpose '[Pose(Point(1 1),0.1)@2001-01-01,
  Pose(Point(3 3),0.1)@2001-01-03]', geometry 'Point(9 9)', 1.0);
-- false
SELECT eIntersects(tposechain 'PoseChain(Pose(Point(1 1),0), Pose(Point(9 -1),0))@2001 ←
    -01-01',
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- false
```

13.17 Spatiotemporal Relationships

- Spatiotemporal relationships

```
tContains(geometry,{tpose,tposechain}) → tbool
tCovers(geometry,{tpose,tposechain}) → tbool
tDisjoint({geometry,tpose,tposechain},{geometry,tpose,tposechain}) → tbool
tDwithin({geo,tpose,tposechain},{geo,tpose,tposechain},float) → tbool
tIntersects({geometry,tpose,tposechain},{geometry,tpose,tposechain}) → tbool
tTouches(geometry,{tpose,tposechain}) → tbool
tTouches({tpose,tposechain},geometry) → tbool
```

The *temporal* relationships compute the topological or distance relationship at each instant and result in a `tbool`. A temporal pose answers them at the position it occupies, so a value with linear interpolation is answered along its trajectory rather than at its instants alone. The temporal pose declares the relationships a moving point answers: `tContains` and `tCovers` take the geometry first only, and `tTouches` has no direction between two temporal poses, since a moving point neither contains nor covers a geometry and two moving points do not touch.

```
SELECT tIntersects(tpose '[Pose(Point(1 1), 0.1)@2001-01-01,
  Pose(Point(3 3), 0.1)@2001-01-03]', geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- {[t@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03]}
SELECT tDisjoint(tpose '[Pose(Point(1 1), 0.1)@2001-01-01,
  Pose(Point(3 3), 0.1)@2001-01-03]', geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- {[f@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03]}
SELECT tContains(geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))',
  tpose 'Pose(Point(0 0), 0.1)@2001-01-01');
-- f@2001-01-01
SELECT tCovers(geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))',
  tpose 'Pose(Point(0 0), 0.1)@2001-01-01');
-- t@2001-01-01
SELECT tTouches(tpose 'Pose(Point(0 0), 0.1)@2001-01-01',
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- t@2001-01-01
SELECT tDwithin(tpose '[Pose(Point(1 1),0.3)@2001-01-01,
  Pose(Point(1 1),0.5)@2001-01-03)',
  tpose '[Pose(Point(1 1),0.5)@2001-01-01, Pose(Point(3 3),0.3)@2001-01-03)', 1);
/* {[t@2001-01-01, t@2001-01-01 16:58:14.025894],
   (f@2001-01-01 16:58:14.025894, f@2001-01-03)} */
```

13.18 Comparisons

- Traditional comparisons

```
tpose {=, <>, <, >, <=, >=} tpose → boolean
```

```
SELECT tpose '{[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.3)@2001-01-02),
  [Pose(Point(1 1), 0.3)@2001-01-02, Pose(Point(1 1), 0.5)@2001-01-03]}' =
  tpose '[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-03]';
-- true
SELECT tpose '{[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-03]}' <>
  tpose '[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-03]';
-- false
SELECT tpose '[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-03]' <
  tpose '[Pose(Point(1 1), 0.1)@2001-01-01, Pose(Point(1 1), 0.6)@2001-01-03]';
-- true
```

- Ever and always comparisons

```
tpose {?=, ?<>, %=, %<>} tpose → boolean
```

```
SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(1 1), 0.4)@2001-01-04)' ?= Pose(Point(1 1), 0.3);
-- true
SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(1 1), 0.2)@2001-01-04)' &= Pose(Point(1 1), 0.2);
-- true
```

- Temporal comparisons

```
tpose {#=, #<>} tpose → tbool
```

```
SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(1 1), 0.4)@2001-01-03)' #= pose 'Pose(Point(1 1), 0.3)';
-- {[f@2001-01-01, t@2001-01-02], (f@2001-01-02, f@2001-01-03)}
SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-01, Pose(Point(1 1), 0.8)@2001-01-03)' #<>
  tpose '[Pose(Point(1 1), 0.3)@2001-01-01, Pose(Point(1 1), 0.7)@2001-01-03)';
-- {[t@2001-01-01, f@2001-01-02], (t@2001-01-02, t@2001-01-03)}
```

- Compare two temporal pose chains, or a temporal pose chain and a pose chain

```
eEqual({pose,posechain,tpose,tposechain},{pose,posechain,tpose,tposechain}) → boolean
```

```
SELECT tposechain 'PoseChain(Pose(Point(1 1), 0.5))@2001-01-01' =
  tposechain 'PoseChain(Pose(Point(1 1), 0.5))@2001-01-01';
-- t
SELECT eEqual(tposechain '{PoseChain(Pose(Point(1 1), 0.5))@2001-01-01,
  PoseChain(Pose(Point(2 2), 0.5))@2001-01-02}',
  posechain 'PoseChain(Pose(Point(2 2), 0.5))');
-- t
```

13.19 Aggregations

The three aggregate functions for temporal poses are illustrated next.

- Temporal count

```
tCount({tpose,tposechain}) → {tintSeq,tintSeqSet}
```

```
WITH Temp(temp) AS (
  SELECT tpose '[Pose(Point(1 1), 0.1)@2001-01-01,
    Pose(Point(1 1), 0.3)@2001-01-03)' UNION
  SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-02,
    Pose(Point(1 1), 0.4)@2001-01-04)' UNION
  SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-03, Pose(Point(1 1), 0.5)@2001-01-05)' )
SELECT tCount(Temp) FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 1@2001-01-04, 1@2001-01-05)}
```

- Window count

```
wCount({tpose,tposechain},interval) → {tintSeq,tintSeqSet}
```

```
WITH Temp(temp) AS (
  SELECT tpose '[Pose(Point(1 1), 0.1)@2001-01-01,
    Pose(Point(1 1), 0.3)@2001-01-03)' UNION
  SELECT tpose '[Pose(Point(1 1), 0.2)@2001-01-02,
    Pose(Point(1 1), 0.4)@2001-01-04)' UNION
  SELECT tpose '[Pose(Point(1 1), 0.3)@2001-01-03, Pose(Point(1 1), 0.5)@2001-01-05)' )
SELECT wCount(Temp, '1 day') FROM Temp;
/* {[1@2001-01-01, 2@2001-01-02, 3@2001-01-03, 2@2001-01-04, 1@2001-01-05,
   1@2001-01-06)} */
```

- Bounding box extent

```
extent({tpose,tposechain}) → stbox
```

```
SELECT extent(temp) FROM (VALUES (NULL::tposechain),
  (tposechain 'PoseChain(Pose(Point(1 1), 0.1))@2001-01-01'),
  (tposechain '{PoseChain(Pose(Point(2 2), 0.2))@2001-01-01,
  PoseChain(Pose(Point(3 3), 0.3))@2001-01-02}')) t(temp);
-- STBOX XT(((1,1),(3,3)),[2001-01-01, 2001-01-02])
```

- Append the aggregated instants into a sequence

```
appendInstant({tpose,tposechain}) → {tpose,tposechain}
appendInstant({tpose,tposechain},interp text) → {tpose,tposechain}
appendInstant({tpose,tposechain},interp text,maxdist float,maxt interval) → {tpose, ↩
    tposechain}
appendInstant(tposechain,interp text,maxt interval) → tposechain
```

The gaps are stated by a maximum distance between consecutive instants, the distance of a pose chain being the one of the pose its composition gives, and by a maximum time.

```
SELECT asText(appendInstant(inst)) FROM (VALUES
  (tposechain 'PoseChain(Pose(Point(1 1), 0.1))@2001-01-01'),
  (tposechain 'PoseChain(Pose(Point(2 2), 0.2))@2001-01-02')) t(inst);
/* [PoseChain(Pose(POINT(1 1),0.1))@2001-01-01,
   PoseChain(Pose(POINT(2 2),0.2))@2001-01-02] */
```

- Append the aggregated sequences into a sequence set

```
appendSequence({tpose,tposechain}) → {tpose,tposechain}
```

```
SELECT asText(appendSequence(seq)) FROM (VALUES
  (tposechain '[PoseChain(Pose(Point(1 1), 0.1))@2001-01-01]'),
  (tposechain '[PoseChain(Pose(Point(2 2), 0.2))@2001-01-02]')) t(seq);
/* {[PoseChain(Pose(POINT(1 1),0.1))@2001-01-01],
   [PoseChain(Pose(POINT(2 2),0.2))@2001-01-02]} */
```

13.20 Spatial Operations

- Return the temporal geometry point given by the points of a temporal pose

```
centroid(tpose) → tgeompoint
```

```
SELECT asText(centroid(tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(3 3), 0.5)@2001-01-03]'));
-- [POINT(1 1)@2001-01-01, POINT(3 3)@2001-01-03]
```

- Return the convex hull of the points of a temporal pose

```
convexHull(tpose) → geometry
```

The hull is taken over the positions the pose occupies; the orientation carries no extent and takes no part in it.

```
SELECT ST_AsText(convexHull(tpose '[Pose(Point(1 1), 0.2)@2001-01-01,
  Pose(Point(3 1), 0.3)@2001-01-02, Pose(Point(2 3), 0.5)@2001-01-03]'));
-- POLYGON((1 1,2 3,3 1,1 1))
```

13.21 Simplification

- Return a temporal pose simplified ensuring that consecutive points are at least a certain distance or time interval apart

```
minDistSimplify(tpose,mindist float) → tpose
minTimeDeltaSimplify(tpose,mint interval) → tpose
```

The simplification operates on the trajectory of the points of the temporal pose and preserves the orientation at each surviving instant. The trajectory is simplified in the plane, so a geodetic pose with linear interpolation raises an error. Simplification applies only to temporal sequences or sequence sets with linear interpolation. In all other cases, a copy of the given value is returned.

```
SELECT asText(minDistSimplify(tpose '[Pose(Point(1 1), 0)@2001-01-01,
  Pose(Point(2 2), 0)@2001-01-02, Pose(Point(3 3), 0)@2001-01-03]', 2));
-- [Pose(POINT(1 1),0)@2001-01-01, Pose(POINT(3 3),0)@2001-01-03]
SELECT asText(minTimeDeltaSimplify(tpose '[Pose(Point(1 1), 0)@2001-01-01,
  Pose(Point(2 2), 0)@2001-01-02, Pose(Point(3 3), 0)@2001-01-04]', interval '2 days'));
-- [Pose(POINT(1 1),0)@2001-01-01, Pose(POINT(3 3),0)@2001-01-04]
```

- Return a temporal pose simplified using the Douglas-Peucker algorithm

```
maxDistSimplify(tpose,maxdist float,syncdist boolean=true) → tpose
douglasPeuckerSimplify(tpose,maxdist float,syncdist boolean=true) → tpose
```

The `syncdist` argument states whether the distance is measured between synchronized instants; when it is false the distance is the spatial one, ignoring time.

```
SELECT asText(maxDistSimplify(tpose '[Pose(Point(1 1), 0)@2001-01-01,
  Pose(Point(2 1), 0)@2001-01-02, Pose(Point(3 3), 0)@2001-01-03]', 1));
-- [Pose(POINT(1 1),0)@2001-01-01, Pose(POINT(3 3),0)@2001-01-03]
SELECT asText(douglasPeuckerSimplify(tpose '[Pose(Point(1 1), 0)@2001-01-01,
  Pose(Point(2 1), 0)@2001-01-02, Pose(Point(3 3), 0)@2001-01-03]', 1));
-- [Pose(POINT(1 1),0)@2001-01-01, Pose(POINT(3 3),0)@2001-01-03]
```

13.22 Indexing

GiST and SP-GiST indexes can be created for table columns of temporal poses and temporal pose chains, and of pose sets and pose chain sets. An example of index creation is follows:

```
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist(Trip);
```

The GiST and SP-GiST indexes store the bounding box of the values, which is an `stbox`, as for all spatiotemporal types; the box of a pose chain covers the composed position of every one of its joints. A set is indexed by the box it stores, as described in Section 2.12.

A GiST or SP-GiST index can accelerate queries involving the following operators:

- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, which only consider the spatial dimension in temporal poses,
- `<<#`, `&<#`, `#&>`, `#>>`, which only consider the time dimension in temporal poses,
- `&&`, `@>`, `<@`, `~=`, `-|-`, and `|=|` , which consider as many dimensions as they are shared by the indexed column and the query argument.

These operators work on bounding boxes, not the entire values.

13.23 OGC GeoPose v1.0 Support

The `pose` type implements the data model behind the OGC GeoPose v1.0 standard: a `position` plus an `orientation` in a known reference frame, where the orientation is a unit quaternion in Hamilton convention or, equivalently, a yaw / pitch / roll triple under the ZYX intrinsic Tait-Bryan convention. This section groups the SQL surface that exposes that interoperability, JSON I/O for the standard's Basic and Advanced conformance classes, an explicit renormalization helper for callers running long compositions, and the Euler-angle accessors that Basic-YPR consumers expect.

13.23.1 JSON Input and Output

- Convert to or from the OGC GeoPose v1.0 JSON encoding (Basic-Quaternion, Basic-YPR or Advanced conformance class)

```
asGeoPose(pose,conformance integer=0,maxdecimaldigits integer=-1) → text
poseFromGeoPose(text) → pose
```

The `conformance` argument selects the output class: `0` = Basic-Quaternion (default, lossless), `1` = Basic-YPR (yaw, pitch, roll in degrees, ZYX intrinsic Tait-Bryan), `2` = Advanced. The `maxdecimaldigits` argument is the number of significant digits to keep in the JSON numbers; `-1` uses json-c's lossless default. The input function auto-detects the conformance class from the JSON keys, where a `frameSpecification` member implies Advanced, a `quaternion` member implies Basic-Quaternion, and an `angles` member implies Basic-YPR. The Basic classes carry the position in a `position` member and leave the frame implicit. The Advanced class has no position member: it names its outer frame explicitly, and the pose sits at the origin of that frame, so the two encodings of one pose read back equal. Every class mandates a geographic outer frame, so the pose must be geodetic; a planar pose is rejected at the conversion boundary, as is a projected SRID.

```
SELECT asGeoPose(pose 'Geodpose(Point(8 47 1500), 0.707107, 0, 0, 0.707107)', 0, 6);
/* {"position":{"lat":47,"lon":8,"h":1500},"quaternion":{"x":0,"y":0,"z":0.707107,
   "w":0.707107}} */
SELECT asGeoPose(pose 'Geodpose(Point(8 47 1500), 0.707107, 0, 0, 0.707107)', 1, 6);
-- {"position":{"lat":47,"lon":8,"h":1500},"angles":{"yaw":90,"pitch":0,"roll":0}}
SELECT asGeoPose(pose 'Geodpose(Point(8 47 1500), 0.707107, 0, 0, 0.707107)', 2, 6);
/* {"frameSpecification":{"authority":"/geopose/1.0","id":"LTP-ENU","parameters":
   "longitude=8&latitude=47&height=1500&crs=EPSG:4979"},
   "quaternion":{"x":0,"y":0,"z":0.707107,"w":0.707107}} */
SELECT asEWKT(poseFromGeoPose( '{"position":{"lat":47,"lon":8,"h":1500},
  "angles":{"yaw":90,"pitch":0,"roll":0}}'), 6);
-- SRID=4326;GeodPose(POINT Z (8 47 1500),0.707107,0,0,0.707107)
```

13.23.2 Quaternion Renormalization

- Return a pose whose orientation quaternion has been transformed to a unit norm

```
poseNormalize(pose) → pose
```

A 2D pose is returned unchanged since its orientation is a single angle. A 3D pose has its quaternion divided by its Euclidean norm, so long compositions of SLERPs (or any other path that accumulates floating-point drift in `|q|`) can be brought back to the `|q| = 1` invariant required by the SLERP and Euler-decomposition code.

```
SELECT asText(poseNormalize(pose 'Pose(Point(1 1 1), 0.5, 0.5, 0.5, 0.5)'));
-- Pose(POINT Z (1 1 1),0.5,0.5,0.5,0.5)
```

- Return the inverse of a pose

```
poseInverse(pose) → pose
```

The inverse reverses the frame relationship: where a pose carries a value from the frame it names into the frame it is expressed in, the inverse carries it back. It is `R_BA = R_AB^T` and `t_BA = -R_AB^T t_AB`, so composing a pose with its inverse gives the identity. This is what expresses a world position in the frame of a vehicle. A pose over a geographic frame has no inverse: the frame it would name is not a frame of the ellipsoid.

```
SELECT asEWKT(round(poseInverse(pose(ST_Point(10,5), pi()/2)), 6));
-- Pose(POINT(-5 10),-1.570796)
SELECT asEWKT(round(applyPose(poseInverse(pose(ST_Point(10,5), pi()/2)),
  pose(ST_Point(10,5), pi()/2)), 6));
-- Pose(POINT(0 0),0)
SELECT asEWKT(round(poseInverse(pose 'Pose(Point(1 0 0), 0, 0, 0, 1)'), 6));
-- Pose(POINT Z (1 0 0),0,0,0,-1)
```

13.23.3 Euler-Angle Accessors

- Return the yaw, pitch, or roll angle of a (temporal) pose, in radians, under the ZYX intrinsic Tait-Bryan convention required by the OGC GeoPose Basic-YPR conformance class

```
yaw({pose,tpose}) → {float,tfloat}
pitch({pose,tpose}) → {float,tfloat}
roll({pose,tpose}) → {float,tfloat}
```

For a 2D pose `yaw` returns the stored rotation `theta`, by convention the yaw of the body frame, and `pitch` and `roll` return `0`. For a 3D pose the three values come from the ZYX intrinsic Tait-Bryan decomposition of the orientation quaternion. The `asin` pitch term is clamped to `[-1, 1]` to absorb the small numeric drift that long quaternion compositions can introduce. On a temporal pose the interpolation of the result follows the dimension. A 2D pose interpolates its stored angle linearly and that angle is its yaw, so a linear 2D `tpose` projects to a linear `tfloat`. A 3D pose interpolates by SLERP, whose Tait-Bryan decomposition is not linear in time, so a 3D `tpose` projects to a step `tfloat`: reading it between two instants answers the angle of the instant on the left rather than an angle no pose holds.

```
SELECT yaw(pose 'Pose(Point(1 1), 0.5)');
-- 0.5
SELECT pitch(pose 'Pose(Point(1 1), 0.5)');
-- 0
SELECT roll(pose 'Pose(Point(0 0 0), 0.7071067811865476, 0, 0, 0.7071067811865475)');
-- 0
SELECT yaw(pose 'Pose(Point(0 0 0), 0.7071067811865476, 0, 0, 0.7071067811865475)');
-- 1.5707963267948966
```

```
SELECT asText(yaw(tpose '[Pose(Point(0 0), 0.0)@2001-01-01,
  Pose(Point(1 1), 0.5)@2001-01-02]'));
-- [0@2001-01-01, 0.5@2001-01-02]
```

- Return the orientation of a pose as a yaw / pitch / roll triple, in radians

```
ypr(pose) → ypr
```

This is the OGC GeoPose Basic-YPR encoding of the orientation, the counterpart of quaternion, and it returns the three angles yaw, `pitch` and `roll` answer one at a time. Like `quaternion`, it is defined for both dimensions: a 2D pose yaws by its stored angle and neither pitches nor rolls. The angles are in radians, where the GeoPose JSON encoding writes them in degrees.

```
SELECT ypr(pose 'Pose(Point(1 1), 0.5)');
-- (0.5,0,0)
SELECT ypr(pose 'Pose(Point Z(1 1 1), 1, 0, 0, 0)');
-- (0,0,0)
SELECT ypr(pose 'Pose(Point Z(1 1 1), 0.5, 0.5, 0.5, 0.5)');
-- (1.5707963267948966,0,1.5707963267948966)
```

- Lift the Euler angle accessors over a temporal pose, returning a temporal float in radians

```
yaw(tpose) → tfloat
pitch(tpose) → tfloat
roll(tpose) → tfloat
```

The result carries step interpolation, since an angle read from a rotation does not vary linearly between two poses.

```
SELECT round(yaw(tpose '[Pose(Point(0 0),0.5)@2001-01-01,
  Pose(Point(1 1),1.5)@2001-01-02]'), 6);
-- Interp=Step;[0.5@2001-01-01, 1.5@2001-01-02]
SELECT round(pitch(tpose 'Pose(Point(0 0 0),1,0,0,0)@2001-01-01'), 6);
-- 0@2001-01-01
```

13.23.4 Frame Metadata Registry

The OGC GeoPose v1.0 standard distinguishes the *outer frame* (the global reference, e.g., WGS-84 geographic or ECEF) from the *inner frame* (the body frame whose orientation is the pose’s quaternion). The Basic conformance classes mandate WGS-84 geographic as the outer frame and an implicit right-handed body-axes inner frame; the Advanced class names its outer frame explicitly and places the pose at that frame’s origin.

In MobilityDB the `pose` type encodes the outer frame implicitly via its SRID and uses the conventional right-handed body-axes inner frame. The `geopose_frames` table records this mapping, and states every frame identifier a document can name: a Composite Sequence Series names `LTP-ENU` and `RotateTranslate`, and a Chain names the same two frames as `/Extrinsic/LTP-ENU` and `/Intrinsic/Translate-Rotate`, all under the `/geopose/1.0` authority. Advanced-class frame stacks are not supported.

```
SELECT frame_id, authority, code, name FROM geopose_frames ORDER BY frame_id;
-- 1 | EPSG | 4326 | WGS-84 geographic (lat/lon/h)
-- 2 | EPSG | 4978 | WGS-84 ECEF (Earth-Centred Earth-Fixed)
-- 3 | OGC | LTP | Local Tangent Plane (East-North-Up)
-- 4 | OGC | BODY | Right-handed body axes (default inner frame)
-- 5 | /geopose/1.0 | LTP-ENU | GeoPose outer frame of a Composite Sequence Series
-- 6 | /geopose/1.0 | RotateTranslate | GeoPose inner frame of a Composite Sequence Series
-- 7 | /geopose/1.0 | /Extrinsic/LTP-ENU | GeoPose outer frame of a Chain
-- 8 | /geopose/1.0 | /Intrinsic/Translate-Rotate | GeoPose inner frame of a Chain
```

The rows above come from MEOS, which builds them from the identifiers the encoders write, so a frame this build can emit is never absent from the registry. A user registers further frames by inserting into the table. The `is_geographic` column, omitted above, is true for the WGS-84 geographic frame alone.

- Return every frame the standard and the encoders name

```
geoPoseFrames() → setof record
```

```
SELECT count(*) FROM geoPoseFrames();
-- 8
```

Three functions answer for a frame MEOS registers, one of the rows `geoPoseFrames` returns, and raise an error for any other identifier; a frame a user registers is read from the table `geopose_frames` directly:

- Return the SRID of a frame MEOS registers, or `NULL` if the frame is parametric (LTP, BODY)

```
geoPoseFrameSRID(integer) → integer
```

```
SELECT geoPoseFrameSRID(1), geoPoseFrameSRID(2);
-- 4326 | 4978
SELECT geoPoseFrameSRID(3);
-- NULL
```

- Return the human-readable name of a frame MEOS registers

```
geoPoseFrameName(integer) → text
```

```
SELECT geoPoseFrameName(1);
-- WGS-84 geographic (lat/lon/h)
SELECT geoPoseFrameName(2), geoPoseFrameName(3);
-- WGS-84 ECEF (Earth-Centred Earth-Fixed) | Local Tangent Plane (East-North-Up)
```

- Return `true` for a lat/lon/h frame MEOS registers, `false` for a Cartesian or projected one

```
geoPoseFrameIsGeographic(integer) → boolean
```

```
SELECT geoPoseFrameIsGeographic(1), geoPoseFrameIsGeographic(2);
-- true | false
```

Users can register custom frames by inserting into `geopose_frames`; the catalog is marked as a configuration table so `pg_dump` preserves user rows.

13.23.5 Body and World Rigid Transform

- Applies the rigid-body transform encoded by a pose (the OGC GeoPose *body to world* mapping) to a body-frame geometry, producing the corresponding world-frame geometry

```
applyPose(geometry,pose) → geometry
applyPose(geometry,tpose) → tgeompoint
applyPose(pose,pose) → pose
```

The transform is

```
world = R(q) · body + p
```

where `(p, q)` are the pose's position and orientation. The static form takes a single pose and a static geometry; the temporal form lifts the per-instant rigid transform of a `tpose` across its instants, producing a `tgeompoint` world-frame trajectory of the body geometry. The static form accepts a body geometry of any type and carries its shape into the pose's frame; the temporal form returns a `tgeompoint`, so its body geometry must be a point. The pose and the body geometry must have the same dimensionality and must agree on their SRID, an unknown one adopting the other. Linear interpolation of the resulting trajectory is the chord on each segment, which approximates the true rigid-body trajectory (a circular arc under SLERP), the same trade-off MobilityDB already takes for spatial trajectories.

```
-- A body sensor offset 1 unit along the body X axis, traced through a tpose that ends 90
-- degrees yawed and translated to (10, 20).
SELECT asText(applyPose(ST_Point(1,0), tpose '[Pose(Point(0 0), 0)@2026-01-01,
  Pose(Point(10 20), 1.5707963267948966)@2026-01-02]'));
-- [POINT(1 0)@2026-01-01, POINT(10 21)@2026-01-02]
```

```
-- The shape of the body geometry is carried into the pose's frame: a hull 4 by 2 about
-- the body origin, placed at (10, 20)
SELECT ST_AsText(applyPose(ST_MakeEnvelope(-2,-1,2,1), pose 'Pose(Point(10 20), 0)'));
-- POLYGON((8 19,8 21,12 21,12 19,8 19))
```

Either frame may move. Applying a pose to a `tpose` carries a fixed body through the whole movement of its parent, which is what a sensor mounted on a moving vehicle asks for; applying a `tpose` to a pose carries a moving body into a frame that stands still; and applying a `tpose` to a `tpose` composes two frames that both move, over the time they share. `poseInverse` lifts the same way, changing the point of view onto a moving object for the whole of its movement.

```
applyPose(pose,tpose) → tpose
applyPose(tpose,pose) → tpose
applyPose(tpose,tpose) → tpose
poseInverse(tpose) → tpose
```

```
-- A sensor one unit ahead of a body that ends yawed a quarter turn at (10, 20) ends one
-- unit north of it
SELECT asEWKT(round(applyPose(pose 'Pose(Point(1 0), 0)',
  tpose '[Pose(Point(0 0), 0)@2026-01-01,
  Pose(Point(10 20), 1.5707963267948966)@2026-01-02]'), 6));
-- [Pose(POINT(1 0),0)@2026-01-01, Pose(POINT(10 21),1.570796)@2026-01-02]
```

A pose names a frame, so it is itself such a body-frame value: applying one pose to another composes the two frame relationships. Writing `P_WV` for the pose of a vehicle in the world and `P_VS` for the pose of a sensor in the vehicle, the pose of the sensor in the world is `P_WS = P_WV ∘ P_VS`. This is the operation a pose chain folds over its links, so a two-link chain composes to what this returns.

```
-- A sensor one unit ahead of a vehicle turned a quarter turn is one unit North of it, and
-- carries the vehicle's orientation
SELECT asEWKT(round(applyPose(pose 'Pose(Point(1 0), 0)', pose(ST_Point(0,0), pi()/2)),
  6));
-- Pose(POINT(0 1),1.570796)
SELECT asEWKT(round(applyPose(pose 'Pose(Point(1 0), 0)', pose(ST_Point(10,5), pi()/2)),
  6));
-- Pose(POINT(10 6),1.570796)
SELECT asEWKT(round(applyPose(pose 'Pose(Point(1 0 0), 1, 0, 0, 0)',
  pose 'Pose(Point(0 0 5), 1, 0, 0, 0)'), 6));
-- Pose(POINT Z (1 0 5),1,0,0,0)
```

13.23.6 Temporal JSON I/O

- Convert a temporal pose to or from its OGC GeoPose v1.0 encoding

```
asGeoPose(tpose,conformance integer=0,maxdecimaldigits integer=-1) → text
tposeFromGeoPose(text) → tpose
```

The conformance class follows from the value. A single instant is a Basic document carrying its `validTime`; a sequence or sequence set is a Composite Sequence Series, of the Regular class when the instants are equally spaced by a whole number of milliseconds and of the Irregular class otherwise. The `conformance` argument chooses the orientation encoding of a Basic document, as it does for a `pose`, and has no effect on a Series, whose inner frames carry a quaternion and offer no such choice. A Series states its outer frame once, as the local tangent plane East-North-Up frame at the position of the first pose, and gives each pose as an inner frame holding its translation from that tangent point, in metres, and its rotation relative to that frame's basis. Times are `GeoPose_Instant` values, that is Unix time in integer milliseconds, which is what the standard requires of every time position it defines. Sub-millisecond sampling is therefore not carried; the MF-JSON encoding, which the standard behind it types as a datetime string, does carry it. The transition model reports the interpolation. Linear interpolation is the standard's `interpolate` model and step and discrete interpolation are its `none` model; since those two identifiers cannot tell step from discrete apart, the model's `parameters` name the interpolation with the same words the MF-JSON encoding uses.

```
SELECT asGeoPose(tpose '[Pose(Point(0 0 0), 0.5, 0.5, 0.5, 0.5)@2026-01-01,
  Pose(Point(0 0 100), 0.5, 0.5, 0.5, 0.5)@2026-01-02,
  Pose(Point(0 0 300), 0.5, 0.5, 0.5, 0.5)@2026-01-05]', 0, 6);
-- {
--   "header": {
```

```
--   "poseCount": 3,
--   "startInstant": 1767254400000,
--   "stopInstant": 1767600000000,
--   "transitionModel": {
--    "authority": "/geopose/1.0",
--    "id": "interpolate",
--    "parameters": "interpolation=Linear"
--   }
--  },
--  "outerFrame": {
--   "authority": "/geopose/1.0",
--   "id": "LTP-ENU",
--   "parameters": "longitude=0&latitude=0&height=0&crs=EPSG:4979"
--  },
--  "innerFrameAndTimeSeries": [
--   {
--    "frame": {
--     "authority": "/geopose/1.0",
--     "id": "RotateTranslate",
--     "parameters": "translation=[0, 0, 0]&rotation=[0.5, 0.5, 0.5, 0.5]"
--    },
--    "validTime": 1767254400000
--   },
--   {
--    "frame": {
--     "authority": "/geopose/1.0",
--     "id": "RotateTranslate",
--     "parameters": "translation=[0, 0, 100]&rotation=[0.5, 0.5, 0.5, 0.5]"
--    },
--    "validTime": 1767340800000
--   },
--   {
--    "frame": {
--     "authority": "/geopose/1.0",
--     "id": "RotateTranslate",
--     "parameters": "translation=[0, 0, 300]&rotation=[0.5, 0.5, 0.5, 0.5]"
--    },
--    "validTime": 1767600000000
--   }
--  ],
--  "trailer": {
--   "poseCount": 3
--  }
-- }
```

A Series has neither gaps nor open bounds, so a sequence set is flattened into a single closed sequence and the bounds inclusion of a sequence is not preserved. Reading also accepts the `TemporalGeoPose` envelope that earlier releases wrote, an array of Basic-class objects each with an added `validTime`, so that data already stored in that form still loads.

That envelope's shape is

```
{
  "type":          "TemporalGeoPose",
  "version":       "1.0",
  "conformance":   "Basic-Quaternion" | "Basic-YPR",
  "interpolation": "None" | "Discrete" | "Step" | "Linear",
  "instants":      [...]                  // for TInstant + TSequence
  "lower_inc":     true|false,            // for TSequence only
  "upper_inc":     true|false,            // for TSequence only
  "sequences":     [{...}, ...]           // for TSequenceSet only
}
```

```
SELECT asGeoPose(tpose '[Pose(Point(8 47), 0)@2026-01-01,
  Pose(Point(9 48), 0.5)@2026-01-02]', 1, 4);
/* {"type":"TemporalGeoPose","version":"1.0","conformance":"Basic-YPR",
   "interpolation":"Linear","lower_inc":true,"upper_inc":true,
   "instants":[
   {"position":{"lat":47,"lon":8,"h":0},
   "angles":{"yaw":0,"pitch":0,"roll":0},
   "validTime":"2026-01-01"},
   {"position":{"lat":48,"lon":9,"h":0},
   "angles":{"yaw":28.65,"pitch":0,"roll":0},
   "validTime":"2026-01-02"}]} */
```

- Write a temporal pose as an OGC GeoPose Stream

```
asGeoPoseStream(tpose,maxdecimaldigits integer=-1) → text
```

A stream carries the frames of an Irregular Series while stating neither how many poses there are nor when they end, since more may arrive. The standard writes it as a `header`, holding the transition model and the outer frame, and a `streamElements` array holding one element per pose. The outer frame is the local tangent plane East-North-Up frame at the position of the first pose, so the header and every element speak of one tangent point. This function writes a value already held whole; asGeoPoseStreamHeader and `asGeoPoseStreamElement` write the same two documents a piece at a time.

```
SELECT asGeoPoseStream(tpose 'Geodpose(Point(0 0 0), 1, 0, 0, 0)@2026-01-01', 6);
/* {"header":{"transitionModel":{"authority":"/geopose/1.0","id":"none",
   "parameters":"interpolation=None"},"outerFrame":{"authority":"/geopose/1.0",
   "id":"LTP-ENU","parameters":"longitude=0&latitude=0&height=0&crs=EPSG:4979"}},
   "streamElements":[{"streamElement":{"frame":{"authority":"/geopose/1.0",
   "id":"RotateTranslate","parameters":"translation=[0, 0, 0]&rotation=[1, 0, 0, 0]"},
   "validTime":1767254400000}}]} */
```

- Write the OGC GeoPose StreamHeader of a temporal pose, or the StreamElement of one of its instants

```
asGeoPoseStreamHeader(tpose,maxdecimaldigits integer=-1) → text
asGeoPoseStreamElement(tpose,tpose,maxdecimaldigits integer=-1) → text
```

These are the two documents a stream engine writes one at a time: the header once, then an element for each instant as it arrives. The second argument of `asGeoPoseStreamElement` is an instant of the first, and the element states its frame against the outer frame the header anchors at the first pose, so the documents are the members of the stream `asGeoPoseStream` writes whole.

```
SELECT asGeoPoseStreamHeader(tpose '[Geodpose(Point(0 0 0), 1, 0, 0, 0)@2026-01-01,
  Geodpose(Point(0 0 0), 0.707107, 0, 0, 0.707107)@2026-01-02]', 6);
/* {"transitionModel":{"authority":"/geopose/1.0","id":"interpolate",
   "parameters":"interpolation=Linear"},"outerFrame":{"authority":"/geopose/1.0",
   "id":"LTP-ENU","parameters":"longitude=0&latitude=0&height=0&crs=EPSG:4979"}} */
SELECT asGeoPoseStreamElement(tpose '[Geodpose(Point(0 0 0), 1, 0, 0, 0)@2026-01-01,
  Geodpose(Point(0 0 0), 0.707107, 0, 0, 0.707107)@2026-01-02]',
  tpose 'Geodpose(Point(0 0 0), 0.707107, 0, 0, 0.707107)@2026-01-02', 6);
/* {"streamElement":{"frame":{"authority":"/geopose/1.0","id":"RotateTranslate",
   "parameters":"translation=[0, 0, 0]&rotation=[0.707107, 0, 0, 0.707107]"},
   "validTime":1767340800000}} */
```

13.23.7 Composite Chain and Graph Documents

The standard states a composition as a Chain document and a set of frames as a Graph document, which is what a pose chain writes.

- Write or read an OGC GeoPose Composite Chain document

```
asGeoPose(tposechain,maxdecimaldigits integer=-1) → text
tposechainFromGeoPose(text) → tposechain
```

A Chain document is an outer frame and a sequence of transformations reaching a final innermost frame, which is what a pose chain holds. The document names the LTP-ENU frame tangent at the outer link's position and one transformation per link: the first takes that tangent frame to the outer link's own frame, and each later one is the link as it is stored, read in the axes of its parent. The document carries one valid time, so it is written from a single instant; atTime obtains one from a longer value. Its frame chain holds at least two frames, so a chain of one link has no Chain document. The Chain class names its two frames `/Extrinsic/LTP-ENU` and `/Intrinsic/Translate-Rotate`, where the Advanced, Series and Stream classes name the same two frames `LTP-ENU` and `RotateTranslate`. A document is written with the pair its own class uses, and either pair is accepted on reading.

```
SELECT asGeoPose(tposechain 'SRID=4326;
  PoseChain( GeodPose(Point Z(-122.3 47.7 11), 1, 0, 0, 0),
  Pose(Point Z(2 0 0), 1, 0, 0, 0))@2021-04-28 05:36:10.083+00', 6);
/* {"validTime":1619588170083,"outerFrame":{"authority":"/geopose/1.0",
   "id":"/Extrinsic/LTP-ENU","parameters":"longitude=-122.3&latitude=47.7&
   height=11&crs=EPSG:4979"},"frameChain":[{"authority":"/geopose/1.0",
   "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[0, 0, 0]&
   rotation=[1, 0, -1.38778e-17, 0]"},{"authority":"/geopose/1.0",
   "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[2, 0, 0]&
   rotation=[1, 0, 0, 0]"}]} */
```

- Write an OGC GeoPose Composite Graph document

```
asGeoPose(tposechain[],maxdecimaldigits integer=-1) → text
```

A Graph document is a set of frames and the transformations between them, which several pose chains sharing their outermost frame hold. The frame list names the LTP-ENU frame tangent at that outermost link followed by the links of every chain, and the transform list names the parent and the child of each edge by their position in that list. The edges carry no transformation of their own; it lives in the frames they name. The document carries one valid time, so it is written from chains read at one and the same instant; atTime obtains them from longer values. Its frame list holds at least two frames, which one link of one chain already gives beside the tangent frame.

```
SELECT asGeoPose(ARRAY[tposechain 'SRID=4326;
  PoseChain( GeodPose(Point Z(-122.3 47.7 11), 1, 0, 0, 0),
  Pose(Point Z(2 0 0), 1, 0, 0, 0))@2021-04-28 05:36:10.083+00',
  tposechain 'SRID=4326;
  PoseChain(GeodPose(Point Z(-122.3 47.7 11), 1, 0, 0, 0),
  Pose(Point Z(0 3 0), 1, 0, 0, 0))@2021-04-28 05:36:10.083+00'], 6);
/* {"validTime":1619588170083,"frameList":[{"authority":"/geopose/1.0",
   "id":"/Extrinsic/LTP-ENU","parameters":"longitude=-122.3&latitude=47.7&
   height=11&crs=EPSG:4979"},{"authority":"/geopose/1.0",
   "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[0, 0, 0]&
   rotation=[1, 0, -1.38778e-17, 0]"},{"authority":"/geopose/1.0",
   "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[2, 0, 0]&
   rotation=[1, 0, 0, 0]"},{"authority":"/geopose/1.0",
   "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[0, 0, 0]&
   rotation=[1, 0, -1.38778e-17, 0]"},{"authority":"/geopose/1.0",
   "id":"/Intrinsic/Translate-Rotate","parameters":"translation=[0, 3, 0]&
   rotation=[1, 0, 0, 0]"}],"transformList":[{"link":[0,1]},{"link":[1,2]},
   {"link":[0,3]},{"link":[3,4]}]} */
```

Chapter 14

Temporal Rigid Geometries

A temporal rigid geometry (`trgeometry`) is a static reference geometry, in two or three dimensions, typically a polygon describing the shape of a moving body, paired with a temporal pose (`tpose`) describing how that geometry translates and rotates over time. The materialised geometry at time `t` is the reference geometry rotated and translated according to the pose interpolated at `t`.

This is the natural representation for moving objects whose *shape* matters, not just their position. The motivating example is AIS maritime traffic, where each ship's hull (a pentagon derived from the antenna offsets `A`, `B`, `C`, `D`) follows a pose path defined by lat/lon and heading reports. The materialised geometry at any instant is the actual ship outline at that moment.

The `trgeometry` type builds on Chapter 13. Most accessor, comparison, topological, position, and indexing operators have the same shape as the corresponding `tpose` functions; this chapter covers the trgeometry-specific surface, pose-to-shape materialisation, traversed area, materialised distance, and spatial restrictions on the moving body.

14.1 Input and Output

A `trgeometry` literal pairs a reference geometry with a temporal pose. The reference geometry comes first, separated from the pose by a semicolon; the pose follows the standard `tpose` syntax (instant, sequence, or sequence set).

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(0 0), 0.5)@2001-01-01';
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));{Pose(Point(0 0), 0.0)@2001-01-01,
  Pose(Point(5 0), 0.5)@2001-01-02, Pose(Point(0 0), 0.0)@2001-01-03}';
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 1.5)@2001-01-02]';
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  {[Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(5 0), 0.0)@2001-01-02],
  [Pose(Point(0 0), 0.0)@2001-01-04, Pose(Point(2 0), 0.0)@2001-01-05]}';
```

The reference geometry can be any polygon or polyhedral surface, with the dimension of the pose: a pose with a quaternion needs a reference geometry in three dimensions, and `ST_Force3D` gives a flat one, in the plane z = 0 of the body. Other values give an error. The pose interpolation is linear in translation and angular-shortest-path in rotation. SRIDs are inherited from the reference geometry and the pose; they must agree.

An SRID can be specified for a trgeometry at the beginning of the literal as shown below, or before its first pose, and it is then the SRID of the reference geometry and of every pose.

```
SELECT trgeometry 'SRID=25832;Polygon((0 0,1 0,1 1,0 1,0 0));
  Pose(Point(0 0), 0.5)@2001-01-01';
SELECT asEWKT(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  SRID=25832;Pose(Point(0 0), 0.5)@2001-01-01');
-- SRID=25832;POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(0 0),0.5)@2001-01-01
```

Step interpolation can also be specified in the same way as for other temporal types:

```
SELECT trgeometry 'Interp=Step;Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]';
```

The standard wire formats are supported.

- Return the Well-Known Text (WKT) representation

  ```
  asText({trgeometry,trgeometry[]} [, maxdecimaldigits integer=15]) → text
  ```

  ```
  SELECT asText(trgeometry 'Polygon((1 1,2 2,3 1,1 1));
    Pose(Point(1.123456789 1.123456789), 0.5)@2001-01-01', 6);
  -- POLYGON((1 1,2 2,3 1,1 1));Pose(POINT(1.123457 1.123457),0.5)@2001-01-01
  ```

- Return the Extended Well-Known Text (EWKT) representation, prefixed with the SRID

  ```
  asEWKT({trgeometry,trgeometry[]} [, maxdecimaldigits integer=15]) → text
  ```

  ```
  SELECT asEWKT(trgeometry 'SRID=25832;Polygon((1 1,2 2,3 1,1 1));
    Pose(Point(1 1),0.5)@2001-01-01');
  -- SRID=25832;POLYGON((1 1,2 2,3 1,1 1));Pose(POINT(1 1),0.5)@2001-01-01
  ```

- Return the Moving Features JSON representation

  ```
  asMFJSON(trgeometry [, options integer=0 [, flags integer=0
    [, maxdecimaldigits integer=15]]]) → text
  ```

  ```
  SELECT asMFJSON(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
    Pose(Point(0 0),0)@2001-01-01');
  ```

  `asMFJSON` does not yet support a polyhedral surface.

- Return the Extended Well-Known Binary (EWKB), the Hexadecimal Well-Known Binary (HexWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

  ```
  asEWKB(trgeometry [, endian text]) → bytea
  asHexWKB(trgeometry [, endian text]) → text
  asHexEWKB(trgeometry [, endian text]) → text
  ```

  Companion `asHexWKB` and `asHexEWKB` return the hex-encoded text of, respectively, the plain and the SRID-carrying representation; the two coincide only when the value has no SRID, as in the example below.

  ```
  SELECT length(asEWKB(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
    Pose(Point(1 1), 0.5)@2001-01-01'));
  -- 130
  -- The bytes round-trip through the matching input function.
  SELECT asText(trgeometryFromHexEWKB(asHexEWKB(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
    Pose(Point(1 1), 0.5)@2001-01-01')));
  -- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(1 1),0.5)@2001-01-01
  SELECT asText(trgeometryFromHexEWKB(asHexWKB(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
    Pose(Point(1 1), 0.5)@2001-01-01')));
  -- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(1 1),0.5)@2001-01-01
  ```

- Input from the Well-Known Text (WKT), the Extended Well-Known Text (EWKT), the Moving Features JSON (MF-JSON), the Extended Well-Known Binary (EWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

  ```
  trgeometryFromText(text) → trgeometry
  trgeometryFromEWKT(text) → trgeometry
  trgeometryFromMFJSON(text) → trgeometry
  trgeometryFromEWKB(bytea) → trgeometry
  trgeometryFromHexWKB(text) → trgeometry
  trgeometryFromHexEWKB(text) → trgeometry
  ```

```
SELECT asText(trgeometryFromEWKT('SRID=4326;Polygon((0 0,1 0,1 1,0 1,0 0));
  Pose(Point(1 1), 0.5)@2001-01-01'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(1 1),0.5)@2001-01-01
SELECT asText(trgeometryFromHexEWKB(asHexEWKB(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  Pose(Point(1 1), 0.5)@2001-01-01')));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(1 1),0.5)@2001-01-01
```

14.2 Constructors

A `trgeometry` can be built directly from its component types, a reference geometry plus a temporal pose:

- Construct a temporal rigid geometry from a reference geometry and a temporal pose

```
trgeometry(geometry,tpose) → trgeometry
```

```
SELECT asText(trgeometry(geometry 'Polygon((0 0,1 0,1 1,0 1,0 0))',
  tpose 'Pose(Point(0 0), 0.0)@2001-01-01'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(0 0),0)@2001-01-01
```

Both arguments must share the same SRID. The reference geometry may be a polygon or a polyhedral surface; the pose may be of any temporal subtype.

14.3 Conversions

The `trgeometry` can be projected to its component types or to a pure-point trajectory.

- Return the underlying temporal pose

```
tpose(trgeometry) → tpose
```

```
SELECT asText(tpose(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.5)@2001-01-02]'));
-- [Pose(POINT(0 0),0)@..., Pose(POINT(10 0),0.5)@...]
```

- Return the centroid (antenna) trajectory as a temporal point

```
tgeompoint(trgeometry) → tgeompoint
tgeogpoint(trgeometry) → tgeogpoint
tgeography(trgeometry) → tgeography
```

The conversions follow the rule of [?listitem].

```
SELECT asText(tgeompoint(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]'));
-- [POINT(0 0)@..., POINT(10 0)@...]
```

14.4 Accessors

The standard temporal-type accessors apply: each returns the materialised geometry at the requested timestamp (the reference shape rotated and translated according to the interpolated pose), or a metadata value derived from the pose path.

- Return the materialised geometry at the start, end, or a chosen timestamp

```
startValue(trgeometry) → geometry
endValue(trgeometry) → geometry
valueAtTimestamp(trgeometry,timestamptz) → geometry
```

```
SELECT asText(startValue(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0))
SELECT asText(valueAtTimestamp( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]',
  timestamptz '2001-01-01 12:00:00'));
-- POLYGON((5 0,6 0,6 1,5 1,5 0))
```

Note that the rotation interpolates linearly along the angular shortest path; a 90° rotation between two instants placed one day apart returns a 45°-rotated polygon at the midpoint.

- Return the number of distinct instants, sequences, or timestamps

```
numInstants(trgeometry) → integer
numSequences(trgeometry) → integer
numTimestamps(trgeometry) → integer
```

```
SELECT numInstants(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- 2
SELECT numSequences(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- 1
SELECT numTimestamps(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- 2
```

- Return the array of constituent instants, sequences, or inter-instant segments

```
instants(trgeometry) → trgeometry[]
sequences(trgeometry) → trgeometry[]
segments(trgeometry) → trgeometry[]
```

```
SELECT array_length( instants(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  {Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(5 0), 0.5)@2001-01-02,
  Pose(Point(0 0), 0.0)@2001-01-03}'), 1);
-- 3
```

- Return the set of distinct centroid (antenna) points seen along the trgeometry

```
points(trgeometry) → geomset
```

```
SELECT asText(points(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  {Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(5 5), 0.0)@2001-01-02,
  Pose(Point(0 0), 0.0)@2001-01-03}'));
-- {"POINT(0 0)", "POINT(5 5)"}
```

- Return the yaw, pitch, or roll angle as a temporal float, in radians, under the ZYX intrinsic Tait-Bryan convention required by the OGC GeoPose Basic-YPR conformance class

```
yaw(trgeometry) → tfloat
pitch(trgeometry) → tfloat
roll(trgeometry) → tfloat
```

A rigid geometry carries its orientation in its poses, so each of these projects onto the temporal pose and asks it, exactly as the tpose accessors of the same name do. A 2D rigid geometry yaws by the stored angle of each pose and neither pitches nor rolls.

```
SELECT asText(yaw(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(0 0), 1.5)@2001-01-02]'));
-- [0@2001-01-01, 1.5@2001-01-02]
SELECT asText(pitch(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(0 0), 1.5)@2001-01-02]'));
-- [0@2001-01-01, 0@2001-01-02]
```

- Return the static reference geometry

```
geometry(trgeometry) → geometry
```

```
SELECT ST_AsText(geometry(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  Pose(Point(5 0), 0.5)@2001-01-01'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0))
```

14.5 Traversed Area

The `traversedArea` function returns the region the moving body covers over the trgeometry's time domain: the placements it takes, or the union of them when the function is asked for it.

- Return the area traversed by the temporal rigid geometry

```
traversedArea(trgeometry,unary_union boolean=false) → geometry
```

```
SELECT ST_AsText(traversedArea( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]'));
-- MULTIPOLYGON(((0 0,1 0,1 1,0 1,0 0)),((10 0,11 0,11 1,10 1,10 0)))
SELECT ST_AsText(traversedArea( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]', true));
-- POLYGON((0 0,0 1,11 1,11 0,0 0))
```

The argument is false unless it is given, and the function then answers the placements themselves gathered into one geometry, a multipolygon for a body that is a polygon, which is what a caller that does its own post-processing wants. With `unary_union = true` they are dissolved into the region they cover.

Sampling note: the merged form of a convex body that translates is exact. Where the body turns, it is read at placements no more than 0.05 radians of turn apart, and nearer where it travels far while it turns, each vertex moving along the chord of its arc between two of them. The region then holds only ground the body covers and leaves out the bulge of the arcs past their chords: over a corpus of turning bodies, at most 5.8e-4 of the diameter of the body.

The merged form of a value in three dimensions is the region on the ground, with no Z. Where the body turns, its area is not yet the one the same value has in two dimensions.

14.6 Spatial Functions

These functions derive a spatial value from the moving body. `centroid` and `bodyPointTrajectory` return temporal points tracking a position over time; `convexHull` returns a static geometry, with no Z for a value in three dimensions.

- Return the centroid of the moving body as a temporal point

```
centroid(trgeometry) → tgeompoint
```

```
SELECT asText(centroid( trgeometry 'Polygon((0 0, 1 0, 1 1, 0 1, 0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(2 0),0)@2001-01-02]'));
-- [POINT(0.5 0.5)@2001-01-01, POINT(2.5 0.5)@2001-01-02]
```

- Return the convex hull of the area traversed by the moving body

```
convexHull(trgeometry) → geometry
```

```
SELECT ST_GeometryType(convexHull( trgeometry 'Polygon((0 0, 1 0, 1 1, 0 1, 0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(2 0),0)@2001-01-02]'));
-- ST_Polygon
```

- Return the world-frame trajectory of a body-frame point carried by the moving rigid geometry

```
bodyPointTrajectory(trgeometry,geometry) → tgeompoint
```

The point is expressed in the reference shape's local frame and the time-varying pose is applied to it at each instant. The result has the interpolation of the temporal rigid geometry. With linear interpolation, the result is exact along a segment in which the body does not rotate. Along a segment in which the body rotates, the point follows a curve, which the result replaces by the segment joining its positions at the two instants, as in the second example below, where the point moves from (1 0) to (0 1) along a quarter of a circle.

```
SELECT asText(bodyPointTrajectory( trgeometry 'Polygon((0 0, 1 0, 1 1, 0 1, 0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(2 0),0)@2001-01-02]',
  geometry 'Point(0 0)'));
-- [POINT(0 0)@2001-01-01, POINT(2 0)@2001-01-02]
SELECT asText(round(bodyPointTrajectory( trgeometry 'Polygon((0 0, 1 0, 1 1, 0 1, 0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),1.5707963267948966)@2001-01-02]',
  geometry 'Point(1 0)'), 6));
-- [POINT(1 0)@2001-01-01, POINT(0 1)@2001-01-02]
```

14.7 Motion Metrics

These functions report the motion of a temporal rigid geometry along its centroid trajectory.

- Return the total length traversed by the centroid, or its cumulative length over time

```
length(trgeometry) → float
cumulativeLength(trgeometry) → tfloat
```

```
SELECT length(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(3 4),0)@2001-01-02]');
-- 5
```

- Return the speed of the centroid as a temporal float

```
speed(trgeometry) → tfloat
```

```
SELECT speed(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- Interp=Step;[0.000115740740741@2001-01-01, 0.000115740740741@2001-01-02]
```

- Return the angular speed as a temporal float

```
angularSpeed(trgeometry) → tfloat
```

The shape of a rigid geometry never changes, so its angular speed is the angular speed of the pose that carries it, in radians per unit time.

```
SELECT asText(angularSpeed(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),1)@2001-01-02]'));
-- Interp=Step;[0.0000115740740741@2001-01-01, 0.0000115740740741@2001-01-02]
```

- Return the time-weighted centroid of the centroid trajectory as a geometry

```
twCentroid(trgeometry) → geometry
```

```
SELECT ST_AsText(twCentroid(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(3 4),0)@2001-01-02]'));
-- POINT(1.5 2)
```

14.8 Transformations

The standard sequence-modifying functions apply: `round`, `setInterp`, `shiftTime`, `scaleTime`, `shiftScaleTime`, `deleteTimestamp`, etc. `round` does not yet support a polyhedral surface, and the transformation to another SRID, `transform` and `transformPipeline`, is not yet supported for a temporal rigid geometry.

- Round all numeric components to a given number of decimal places

```
round(trgeometry,integer) → trgeometry
```

```
SELECT asText(round( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  Pose(Point(1.123456789 1.123456789), 0.123456)@2001-01-01', 3));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(1.123 1.123),0.123)@...
```

- Convert between linear and step interpolations

```
setInterp(trgeometry,text) → trgeometry
```

```
SELECT asText(setInterp(trgeometry 'Interp=Step;Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]', 'linear'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));
-- {[Pose(POINT(0 0),0)@2001-01-01, Pose(POINT(0 0),0)@2001-01-02),
--   [Pose(POINT(10 0),0)@2001-01-02]}
```

- Shift, scale, or both shift and scale the time domain of a trgeometry

```
shiftTime(trgeometry,interval) → trgeometry
scaleTime(trgeometry,interval) → trgeometry
shiftScaleTime(trgeometry,interval,interval) → trgeometry
```

```
SELECT asText(shiftTime(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0),
  0.0)@2001-01-02]', interval '1 day'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));
-- [Pose(POINT(0 0),0)@2001-01-02, Pose(POINT(10 0),0)@2001-01-03]
SELECT asText(scaleTime(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0),
  0.0)@2001-01-02]', interval '2 days'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));
-- [Pose(POINT(0 0),0)@2001-01-01, Pose(POINT(10 0),0)@2001-01-03]
SELECT asText(shiftScaleTime(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0),
  0.0)@2001-01-02]', interval '1 day', interval '2 days'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));
-- [Pose(POINT(0 0),0)@2001-01-02, Pose(POINT(10 0),0)@2001-01-04]
```

- Transform a nonlinear trgeometry into a set of rows, one per distinct pose, each one is a pair composed of the reference geometry with the pose applied and the period set during which the trgeometry takes that pose {}

```
unnest(trgeometry) → {(value,time)}
```

```
SELECT ST_AsText((un).value), (un).time
FROM (SELECT unnest(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  {Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(4 0), 0.0)@2001-01-02,
  Pose(Point(0 0), 0.0)@2001-01-03}') AS un) t;
-- POLYGON((0 0,1 0,1 1,0 1,0 0)) | {[2001-01-01, 2001-01-01], [2001-01-03, 2001-01-03]}
-- POLYGON((4 0,5 0,5 1,4 1,4 0)) | {[2001-01-02, 2001-01-02]}
SELECT ST_AsText(ST_SnapToGrid((un).value, 0.000001)), (un).time
FROM (SELECT unnest(merge(
  trgeometry(geometry 'Polygon((0 0,1 0,1 1,0 1,0 0))',
    pose(geometry 'Point(0 0)', 0), timestamptz '2001-01-01'),
  trgeometry(geometry 'Polygon((0 0,1 0,1 1,0 1,0 0))',
    pose(geometry 'Point(3 2)', pi() / 2), timestamptz '2001-01-02'))) AS un) t;
-- POLYGON((0 0,1 0,1 1,0 1,0 0)) | {[2001-01-01, 2001-01-01]}
-- POLYGON((3 2,3 3,2 3,2 2,3 2)) | {[2001-01-02, 2001-01-02]}
```

The pose is applied to the reference geometry in every row of the result: the value of a row is the reference geometry rotated by the orientation of the pose and then translated to its position, the geometry that `startValue` and `valueAtTimestamp` return at that pose. In the second example the pose rotates the square by 90° and moves it to (3, 2). A reference geometry that a rotation maps onto itself yields rows with equal geometries, one per pose.

- Return a temporal value translated by given offsets

```
translate(trgeometry,deltax float,deltay float,deltaz float=0) → trgeometry
```

A vertical offset takes a two-dimensional value out of its plane, so the result is three-dimensional.

The motion moves the pose of every instant and keeps the reference geometry. A motion that takes a two-dimensional value out of its plane takes the reference geometry with it, in the plane z = 0.

The motion is a planar map, so a geodetic value raises an error.

```
SELECT asText(translate(trgeometry 'Polygon((0 0,2 0,2 1,0 1,0 0));
  [Pose(Point(1 1),0.3)@2001-01-01, Pose(Point(3 2),1.1)@2001-01-02]', 1, 2), 6);
/* POLYGON((0 0,2 0,2 1,0 1,0 0));[Pose(POINT(2 3),0.3)@2001-01-01,
   Pose(POINT(4 4),1.1)@2001-01-02] */
```

- Return a temporal value rotated counter-clockwise about the origin or about a point

```
rotate(trgeometry,angle float) → trgeometry
rotate(trgeometry,angle float,x0 float,y0 float) → trgeometry
```

The motion moves the pose of every instant and keeps the reference geometry. A motion that takes a two-dimensional value out of its plane takes the reference geometry with it, in the plane z = 0.

The motion is a planar map, so a geodetic value raises an error.

```
SELECT asText(rotate(trgeometry 'Polygon((0 0,2 0,2 1,0 1,0 0));
  [Pose(Point(1 1),0.3)@2001-01-01, Pose(Point(3 2),1.1)@2001-01-02]', 0.7, 1, 1), 6);
/* POLYGON((0 0,2 0,2 1,0 1,0 0));[Pose(POINT(1 1),1)@2001-01-01,
   Pose(POINT(1.885467 3.053278),1.8)@2001-01-02] */
```

- Return a temporal value rotated counter-clockwise about the x axis

```
rotateX(trgeometry,angle float) → trgeometry
```

The rotation takes a two-dimensional value out of its plane, so the result is three-dimensional: each position gains the height the rotation gives it, and each orientation is the rotation composed with the angle about the vertical that the value states.

The motion moves the pose of every instant and keeps the reference geometry. A motion that takes a two-dimensional value out of its plane takes the reference geometry with it, in the plane z = 0.

The motion is a planar map, so a geodetic value raises an error.

```
SELECT asText(rotateX(trgeometry 'Polygon((0 0,2 0,2 1,0 1,0 0));
  [Pose(Point(1 1),0.3)@2001-01-01, Pose(Point(3 2),1.1)@2001-01-02]', 0.7), 6);
/* POLYGON Z ((0 0 0,2 0 0,2 1 0,0 1 0,0 0 0));
   [Pose(POINT Z (1 0.764842 0.644218),0.928825,0.339047,-0.051242,0.140378)@2001-01-01,
   Pose(POINT Z (3 1.529684 1.288435),0.800838,0.292329,-0.179228,0.490998)@2001-01-02] */
```

- Return a temporal value rotated counter-clockwise about the y axis

```
rotateY(trgeometry,angle float) → trgeometry
```

The rotation takes a two-dimensional value out of its plane, so the result is three-dimensional: each position gains the height the rotation gives it, and each orientation is the rotation composed with the angle about the vertical that the value states.

The motion moves the pose of every instant and keeps the reference geometry. A motion that takes a two-dimensional value out of its plane takes the reference geometry with it, in the plane z = 0.

The motion is a planar map, so a geodetic value raises an error.

- Return a temporal value rotated counter-clockwise about the z axis

```
rotateZ(trgeometry,angle float) → trgeometry
```

The motion moves the pose of every instant and keeps the reference geometry. A motion that takes a two-dimensional value out of its plane takes the reference geometry with it, in the plane z = 0.

The motion is a planar map, so a geodetic value raises an error.

- Return a temporal rigid geometry projected to the ground

```
force2D(trgeometry) → trgeometry
```

The reference geometry loses its Z and each pose is projected as force2D projects a temporal pose, so the functions in two dimensions answer about the ground: in the second example, two flat bodies at different heights touch. A reference geometry that is not a polygon in a plane gives an error, and a value in two dimensions stays as it is.

```
SELECT asText(force2D(trgeometry 'Polygon((-2 -1 0,2 -1 0,2 1 0,-2 1 0,-2 -1 0));
  [Pose(Point(0 0 0),1,0,0,0)@2001-01-01,
  Pose(Point(10 0 0),0.7071067811865476,0,0,0.7071067811865476)@2001-01-02]'), 6);
/* POLYGON((-2 -1,2 -1,2 1,-2 1,-2 -1));[Pose(POINT(0 0),0)@2001-01-01,
   Pose(POINT(10 0),1.570796)@2001-01-02] */
SELECT round(nearestApproachDistance(
  force2D(trgeometry 'Polygon((-2 -1 0,2 -1 0,2 1 0,-2 1 0,-2 -1 0));
    [Pose(Point(0 0 0),1,0,0,0)@2001-01-01, Pose(Point(10 0 0),1,0,0,0)@2001-01-02]'),
  force2D(trgeometry 'Polygon((-2 -1 0,2 -1 0,2 1 0,-2 1 0,-2 -1 0));
    [Pose(Point(5 0 0.8),1,0,0,0)@2001-01-01, Pose(Point(5 0 0.8),1,0,0,0)@2001-01-02]')) ↩
      ::numeric, 6);
-- 0.000000
```

14.9 Modifications

- Append a single instant to an existing trgeometry, growing the underlying pose path

```
appendInstant(trgeometry,trgeometry [, maxdist float [, maxt interval]]) → trgeometry
```

```
SELECT appendInstant( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));Pose(Point(5 0), 0.0)@2001-01-02');
```

- Append an entire sequence to an existing trgeometry

```
appendSequence(trgeometry,trgeometry) → trgeometry
```

```
SELECT asText(appendSequence(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(20 0), 0.0)@2001-01-03, Pose(Point(30 0), 0.0)@2001-01-04]'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));
-- {[Pose(POINT(0 0),0)@2001-01-01, Pose(POINT(10 0),0)@2001-01-02],
--   [Pose(POINT(20 0),0)@2001-01-03, Pose(POINT(30 0),0)@2001-01-04]}
```

- Combine two trgeometries with the same reference geometry into a single sequence-set

```
merge(trgeometry,trgeometry) → trgeometry
merge(trgeometry) → trgeometry
```

```
SELECT asText(merge( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(5 0), 0.0)@2001-01-02]',
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-04, Pose(Point(5 0), 0.0)@2001-01-05]'));
```

14.10 Restrictions

The `atGeometry`, `minusGeometry`, `atStbox`, and `minusStbox` functions restrict a trgeometry to (the complement of) a geometry or a spatiotemporal box. The semantic mirrors `tpose`: a temporal rigid geometry is "in" a region at time `t` iff its body, the reference geometry placed by the pose at `t`, meets that region. `atGeometry` and `minusGeometry` are not yet supported for a value in three dimensions, nor for a geometry with Z or with holes, and `atStbox` and `minusStbox` do not yet read the Z dimension of a box.

- Restrict to (the complement of) a pose or a set of poses

```
atValue(trgeometry,pose) → trgeometry
minusValue(trgeometry,pose) → trgeometry
atValues(trgeometry,poseset) → trgeometry
minusValues(trgeometry,poseset) → trgeometry
```

```
SELECT asText(atValue(trgeometry 'Polygon((1 1,2 2,3 1,1 1));
  [Pose(Point(1 1), 0.2)@2001-01-01, Pose(Point(1 1), 0.4)@2001-01-02,
  Pose(Point(1 1), 0.5)@2001-01-03]', pose 'Pose(Point(1 1), 0.5)'));
-- POLYGON((1 1,2 2,3 1,1 1));{[Pose(POINT(1 1),0.5)@2001-01-03]}
SELECT asText(minusValue(trgeometry 'Polygon((1 1,2 2,3 1,1 1));
  {Pose(Point(1 1), 0.3)@2001-01-01, Pose(Point(1 1), 0.5)@2001-01-02,
  Pose(Point(1 1), 0.5)@2001-01-03}', pose 'Pose(Point(1 1), 0.5)'));
-- POLYGON((1 1,2 2,3 1,1 1));{Pose(POINT(1 1),0.3)@2001-01-01}
```

The restricting value is a pose, the base value a temporal rigid geometry interpolates. The reference geometry is carried through unchanged, so the result describes the same body.

- Restrict a trgeometry to (the complement of) a geometry

```
atGeometry(trgeometry,geometry) → trgeometry
minusGeometry(trgeometry,geometry) → trgeometry
```

```
SELECT getTime(atGeometry( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(4 0), 0.0)@2001-01-05]',
  geometry 'Polygon((1 -1,3 -1,3 1,1 1,1 -1))'));
-- {[2001-01-01, 2001-01-04]}
```

```
SELECT getTime(minusGeometry( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(4 0), 0.0)@2001-01-05]',
  geometry 'Polygon((1 -1,3 -1,3 1,1 1,1 -1))'));
-- {(2001-01-04, 2001-01-05]}
```

Restriction against an empty geometry returns NULL for `atGeometry` and the original input for `minusGeometry`.

- Restrict a trgeometry to (the complement of) a spatiotemporal box

```
atStbox(trgeometry,stbox[,border_inc boolean=true]) → trgeometry
minusStbox(trgeometry,stbox[,border_inc boolean=true]) → trgeometry
```

```
SELECT getTime(atStbox( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(4 0), 0.0)@2001-01-05]',
  stbox 'STBOX X((1, -1), (3, 1))'));
-- {[2001-01-01, 2001-01-04]}
```

A box with only a temporal component restricts the time domain, as `atTime` does; if the box has only a spatial component the temporal domain is preserved.

14.11 Distance Operations

Operands of different dimensions read the dimensions both of them have, see Section 8.3.

The distance functions compute the distance between the body of a temporal rigid geometry and a geometry, a temporal point, or the body of another temporal rigid geometry. The body at an instant is the reference geometry moved and turned by the pose at that instant.

The values of a temporal distance are exact. When a body turns, the distance between two instants does not change linearly, so the temporal distance also has an instant at each minimum and maximum of the distance. Thus the nearest approach distance and the nearest approach instant are exact.

The distance functions need a reference geometry that is a convex polygon without holes. A polygon of the other argument must also be convex and without holes. Other values give an error. The distance of a temporal rigid geometry in three dimensions is not yet supported, and neither is the distance to a geometry collection or a curve. The temporal distance between two concave geometries is thus not yet supported either.

```
SELECT tDistance(trgeometry 'Polygon((0 0,4 0,4 4,2 1,0 4,0 0));
 Pose(Point(0 0),0)@2001-01-01', geometry 'Point(7 3)');
-- ERROR:  The distance of a temporal rigid geometry needs a reference geometry that is
-- a convex polygon without holes
```

- Return the temporal distance

```
{geometry,tgeompoint,trgeometry} <-> {geometry,tgeompoint,trgeometry} → tfloat
tDistance({geometry,tgeompoint,trgeometry},{geometry,tgeompoint,trgeometry}) → tfloat
```

In the first example, a square moves past a point. Its side stays at distance 2 from the point from 10:00 to 14:00. In the second example, the square turns a quarter turn next to the point. A corner of the square comes nearest to the point at 12:00, and the result has an instant there.

```
SELECT trgeometry 'Polygon((-1 -1,1 -1,1 1,-1 1,-1 -1));
  [Pose(Point(-6 0),0)@2001-01-01, Pose(Point(6 0),0)@2001-01-02]' <->
  geometry 'Point(0 3)';
-- [5.385164807134504@2001-01-01, 2@2001-01-01 10:00:00, 2@2001-01-01 14:00:00,
--  5.385164807134504@2001-01-02]
SELECT round(trgeometry 'Polygon((-1 -1,1 -1,1 1,-1 1,-1 -1));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),1.5707963267948966)@2001-01-02]' <->
  geometry 'Point(3 0)', 6);
```

```
-- [2@2001-01-01, 1.828427@2001-01-01 05:11:32.371809, 1.585786@2001-01-01 12:00:00,
--  1.828427@2001-01-01 18:48:27.628191, 2@2001-01-02]
SELECT trgeometry 'Polygon((-1 -1,1 -1,1 1,-1 1,-1 -1));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),0)@2001-01-02]' <->
  tgeompoint '[Point(3 -4)@2001-01-01, Point(3 4)@2001-01-02]';
-- [3.605551275463989@2001-01-01, 2@2001-01-01 09:00:00, 2@2001-01-01 15:00:00,
--  3.605551275463989@2001-01-02]
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),0)@2001-01-02]' <->
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(2 0),0)@2001-01-01, Pose(Point(10 0),0)@2001-01-02]';
-- [1@2001-01-01, 9@2001-01-02]
```

- Return the nearest approach distance

```
{geometry,stbox,tgeompoint,trgeometry} |=| {geometry,stbox,tgeompoint,trgeometry} → float
nearestApproachDistance({geometry,stbox,tgeompoint,trgeometry},
  {geometry,stbox,tgeompoint,trgeometry}) → float
```

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(10 0),0)@2001-01-02]' |=|
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(50 0),0)@2001-01-01, Pose(Point(50 0),0)@2001-01-02]';
-- 39
SELECT round(nearestApproachDistance(trgeometry 'Polygon((-1 -1,1 -1,1 1,-1 1,-1 -1));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),1.5707963267948966)@2001-01-02]',
  geometry 'Point(3 0)')::numeric, 6);
-- 1.585786
```

The `|=|` operator can be used for nearest-neighbour searches with GiST or SP-GiST indexes (see Section 14.20).

- Return the instant of the first argument at which the two arguments are nearest

```
nearestApproachInstant({geometry,tgeompoint,trgeometry},
  {geometry,tgeompoint,trgeometry}) → trgeometry
```

If the distance stays at its minimum during an interval, the result is the first instant of the interval. In the second example, the square is at distance 4 from the point from 09:36 to 12:00.

```
SELECT asText(nearestApproachInstant(trgeometry 'Polygon((-1 -1,1 -1,1 1,-1 1,-1 -1));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),1.5707963267948966)@2001-01-02]',
  geometry 'Point(3 0)'), 6);
-- POLYGON((-1 -1,1 -1,1 1,-1 1,-1 -1));Pose(POINT(0 0),0.785398)@2001-01-01 12:00:00
SELECT asText(nearestApproachInstant(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(10 0),0)@2001-01-02]',
  geometry 'Point(5 5)'));
-- POLYGON((0 0,1 0,1 1,0 1,0 0));Pose(POINT(4 0),0)@2001-01-01 09:36:00
```

- Return the line between the nearest points of the two arguments at their nearest approach instant

```
shortestLine({geometry,tgeompoint,trgeometry},{geometry,tgeompoint,trgeometry})
  → geometry
```

```
SELECT ST_AsText(shortestLine(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  Pose(Point(0 0),0)@2001-01-01', geometry 'Point(10 0)'));
-- LINESTRING(1 0,10 0)
```

14.12 Similarity

A temporal rigid geometry has no similarity distance of its own. Comparing two poses combines a difference of positions with a difference of orientations, which needs a length a pose does not carry, and the distances a rigid body admits, such as the Hausdorff distance of the placed shapes, the largest displacement of a point of the body or the area of their symmetric difference, order pairs of values differently. The trajectories of the reference points of two temporal rigid geometries, the origins of their local frames, are compared with the similarity functions of temporal points over their conversion to `tgeompoint`.

- Compare the trajectories of the reference points of two temporal rigid geometries

```
SELECT round(frechetDistance(
  tgeompoint(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
    [Pose(Point(0 0),0)@2001-01-01, Pose(Point(2 0),0)@2001-01-03]'),
  tgeompoint(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
    [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 2),0)@2001-01-03]')), 6);
-- 2.828427
```

14.13 Bounding Box Operations

Topological and position operators compare the spatiotemporal bounding boxes of the operands; both can be a `trgeometry`, an `stbox`, a `geometry`, or a `tstzspan`.

- Standard topological operators on bounding boxes

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]' &&
       stbox 'STBOX X((5, -1), (15, 1))';
-- t
```

- Spatial and temporal position operators

  The full set `<<`, `>>`, `&<`, `&>`, `<<|`, `&<|`, `|>>`, `|&>`, `<</`, `&</`, `/>>`, `/&>`, `<<#`, `&<#`, `#>>`, `#&>` all accept `trgeometry` on either side. The operators `<</`, `&</`, `/>>` and `/&>` compare the Z axis and require both arguments to have a Z dimension.

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2001-01-01,
  Pose(Point(3 0),0)@2001-01-02]' << stbox 'STBOX X((5,5),(6,6))';
-- true
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0),0)@2001-01-01,
  Pose(Point(3 0),0)@2001-01-02]' <<# tstzspan '[2001-01-03, 2001-01-04]';
-- true
SELECT trgeometry 'Polygon Z((0 0 0,1 0 0,1 1 0,0 1 0,0 0 0));
  [Pose(Point(0 0 0),1,0,0,0)@2001-01-01, Pose(Point(3 0 0),1,0,0,0)@2001-01-02]'
  <</ stbox 'STBOX Z((5,5,5),(6,6,6))';
-- true
SELECT trgeometry 'Polygon Z((0 0 0,1 0 0,1 1 0,0 1 0,0 0 0));
  [Pose(Point(0 0 0),1,0,0,0)@2001-01-01, Pose(Point(3 0 0),1,0,0,0)@2001-01-02]'
  /&> stbox 'STBOX Z((5,5,5),(6,6,6))';
-- false
```

14.14 Ever and Always Relationships

- Ever and always relationships

```
eContains({geometry,trgeometry},{geometry,trgeometry}) → boolean
aContains({geometry,trgeometry},{geometry,trgeometry}) → boolean
eCovers({geometry,trgeometry},{geometry,trgeometry}) → boolean
aCovers({geometry,trgeometry},{geometry,trgeometry}) → boolean
eDisjoint({geometry,trgeometry},{geometry,trgeometry}) → boolean
aDisjoint({geometry,trgeometry},{geometry,trgeometry}) → boolean
eDwithin({geometry,trgeometry},{geometry,trgeometry},float) → boolean
aDwithin({geometry,trgeometry},{geometry,trgeometry},float) → boolean
eIntersects({geometry,trgeometry},{geometry,trgeometry}) → boolean
aIntersects({geometry,trgeometry},{geometry,trgeometry}) → boolean
eTouches({geometry,trgeometry},{geometry,trgeometry}) → boolean
aTouches({geometry,trgeometry},{geometry,trgeometry}) → boolean
```

The *ever* relationships determine whether the relationship holds at some instant, and the *always* ones whether it holds at every instant. A temporal rigid geometry answers them at the body its reference geometry places at each instant, so the relationship read is the one of the placed body rather than of the reference geometry at the origin. With linear interpolation the body moves between its instants, and `eIntersects`, `aIntersects`, `eDisjoint` and `aDisjoint` read that motion through the temporal distance, so they need what the distance needs: a convex body in two dimensions, against a geometry whose polygons are convex and without holes. The other relationships do not yet read the motion between the instants. These relationships do not yet follow Section 8.3: an operand of another dimension gives an error. For a value in three dimensions, `eIntersects`, `eDisjoint`, `aDisjoint`, `eContains`, `eCovers` and `eTouches` do not yet read Z, so a geometry above the body can answer as if it met the body, and `aContains` and `aCovers` do not yet support a geometry with Z in every form.

```
SELECT eIntersects(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(3 3),0)@2001-01-03]',
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- true
SELECT aIntersects(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(3 3),0)@2001-01-03]',
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- false
```

`eDwithin` and `aDwithin` use the temporal distance of Section 14.11, with the same requirements on the geometries. In the example, a square turns next to a point. It is at distance 2 from the point at the two instants, but it comes within 1.7 of the point between them.

```
SELECT eDwithin(trgeometry 'Polygon((-1 -1,1 -1,1 1,-1 1,-1 -1));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),1.5707963267948966)@2001-01-02]',
  geometry 'Point(3 0)', 1.7);
-- true
SELECT aDwithin(trgeometry 'Polygon((-1 -1,1 -1,1 1,-1 1,-1 -1));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),1.5707963267948966)@2001-01-02]',
  geometry 'Point(3 0)', 1.7);
-- false
```

14.15 Spatiotemporal Relationships

- Spatiotemporal relationships

```
tContains({geometry,trgeometry},{geometry,trgeometry}) → tbool
tCovers({geometry,trgeometry},{geometry,trgeometry}) → tbool
tDisjoint({geometry,trgeometry},{geometry,trgeometry}) → tbool
tDwithin({geometry,trgeometry},{geometry,trgeometry},float) → tbool
tIntersects({geometry,trgeometry},{geometry,trgeometry}) → tbool
tTouches({geometry,trgeometry},{geometry,trgeometry}) → tbool
```

The *temporal* relationships compute the topological or distance relationship at each instant and result in a `tbool`. A rigid geometry covers an area, so every relationship is declared in both directions and between two temporal rigid geometries. A body that meets another only between two instants of a linear value thus answers false at both. For a value in three dimensions, `tIntersects` and `tDisjoint` do not yet answer rightly, since a point inside the body at an instant is reported as not meeting it, and `tContains`, `tCovers` and `tTouches` are not yet supported.

```
SELECT tIntersects(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(3 3),0)@2001-01-03]',
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- [t@2001-01-01, f@2001-01-03]
```

`tDwithin` uses the temporal distance of Section 14.11, with the same requirements on the geometries. Its result changes at the exact times at which the distance crosses the given distance. In the example, a square turns a quarter turn next to a point. It is within 1.7 of the point from 07:26:20 to 16:33:39.

```
SELECT tDwithin(trgeometry 'Polygon((-1 -1,1 -1,1 1,-1 1,-1 -1));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(0 0),1.5707963267948966)@2001-01-02]',
  geometry 'Point(3 0)', 1.7);
-- {[f@2001-01-01, t@2001-01-01 07:26:20.193911, t@2001-01-01 16:33:39.806089],
--  (f@2001-01-01 16:33:39.806089, f@2001-01-02]}
```

14.16 Comparisons

- Traditional comparisons

```
trgeometry {=, <>, <, >, <=, >=} trgeometry → boolean
```

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
    [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]' =
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
    [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]';
-- t
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
    [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0),0.0)@2001-01-02]' <>
  trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
    [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]';
-- f
```

- Ever and always comparisons

```
{geometry,trgeometry} {?=, ?<>, %=, %<>} {geometry,trgeometry} → boolean
```

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));[Pose(Point(0 0), 0.0)@2001-01-01,
  Pose(Point(5 0), 0.0)@2001-01-02]' ?= geometry 'Polygon((5 0,6 0,6 1,5 1,5 0))';
-- t
```

- Temporal comparisons

```
{geometry,trgeometry} {#=, #<>} {geometry,trgeometry} → tbool
```

```
SELECT trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0),
  0.0)@2001-01-02]' %= trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]';
-- t
```

14.17 Tiles

These functions return the spatiotemporal boxes of a multidimensional grid that the temporal rigid geometry intersects, computed on its centroid trajectory.

- Return the spatiotemporal boxes of a temporal rigid geometry split with respect to a spatial, temporal, or spatiotemporal grid

```
spaceBoxes(trgeometry,xsize float [, ysize float [, zsize float]],
  sorigin geometry='Point(0 0 0)',bitmatrix boolean=true,
  borderInc boolean=true) → stbox[]
timeBoxes(trgeometry,duration interval,torigin timestamptz='2000-01-03',
  bitmatrix boolean=true,borderInc boolean=true) → stbox[]
spaceTimeBoxes(trgeometry,xsize float [, ysize float [, zsize float]],duration interval,
  sorigin geometry='Point(0 0 0)',torigin timestamptz='2000-01-03',
  bitmatrix boolean=true,borderInc boolean=true) → stbox[]
```

```
SELECT array_length(spaceBoxes( trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0),0)@2001-01-01, Pose(Point(10 0),0)@2001-01-02]', 2.0), 1);
-- 6
```

14.18 Bounding Boxes

These functions decompose a temporal rigid geometry into the bounding boxes or time spans of its segments or of a requested number of bins.

- Return the array of spatiotemporal boxes or time spans of the segments of a temporal rigid geometry

```
stboxes(trgeometry) → stbox[]
spans(trgeometry) → tstzspan[]
```

```
SELECT stboxes(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- {"STBOX XT(((0,0),(11,1)),[2001-01-01, 2001-01-02])"}
SELECT spans(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- {"[2001-01-01, 2001-01-02]"}
```

- Return the spatiotemporal boxes or time spans of a temporal rigid geometry split into a given number of bins, or with a given number of segments per bin

```
splitNStboxes(trgeometry,integer) → stbox[]
splitEachNStboxes(trgeometry,integer) → stbox[]
splitNSpans(trgeometry,integer) → tstzspan[]
splitEachNSpans(trgeometry,integer) → tstzspan[]
```

```
SELECT splitNStboxes(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]', 2);
-- {"STBOX XT(((0,0),(11,1)),[2001-01-01, 2001-01-02])"}
SELECT splitNSpans(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]', 2);
-- {"[2001-01-01, 2001-01-02]"}
```

14.19 Aggregations

- Number of overlapping temporal rigid geometries at each instant, equivalent to `tCount` on the underlying tpose

```
tCount(trgeometry) → tint
```

```
SELECT tCount(trgeometry 'Polygon((0 0,1 0,1 1,0 1,0 0));
  [Pose(Point(0 0), 0.0)@2001-01-01, Pose(Point(10 0), 0.0)@2001-01-02]');
-- {[1@2001-01-01, 1@2001-01-02]}
```

- Aggregate bounding box of a column of temporal rigid geometries

```
extent(trgeometry) → stbox
```

```
SELECT extent(temp) FROM tbl_trgeometry;
```

14.20 Indexing

Three index types are available on `trgeometry` columns. Each uses the spatiotemporal bounding box of the materialised geometry and supports topological, position, and KNN distance queries.

- `GIST(temp)`, R-tree GiST.
- `SPGIST(temp trgeometry_quadtree_ops)`, quad-tree SP-GiST.
- `SPGIST(temp trgeometry_kdtree_ops)`, kd-tree SP-GiST.

```
CREATE INDEX tbl_trgeometry_gist_idx ON tbl_trgeometry USING GIST(temp);
-- Topological lookup
SELECT count(*) FROM tbl_trgeometry WHERE temp && stbox 'STBOX XT(((0, 0),
  (10, 10)),[2001-01-01, 2001-01-02])';
-- KNN (ordered scan)
SELECT mmsi FROM tbl_trgeometry ORDER BY temp <-> trgeometry 'Polygon((0 0,1 0,1 1,
  0 1,0 0));Pose(Point(100 100), 0.0)@2001-01-01' LIMIT 10;
```

The kd-tree variant typically wins on workloads dominated by KNN queries with a tight ordering bound; the quad-tree variant is more uniform. The R-tree GiST is the safe default for mixed read/write workloads.

Chapter 15

JSON Types in MobilityDB

PostgreSQL provides two types for storing JSON data: `json` and `jsonb`. The `json` type stores an exact copy of the input text, which must be reparsed each time it is processed. On the other hand, the `jsonb` type stores the JSON data in a decomposed binary format that makes it slightly slower to input due to added conversion overhead, but significantly faster to process, since no reparsing is needed. The `jsonb` type also supports indexing.

In MobilityDB, the `textset` and the `ttext` types are used for representing, respectively, set of JSON values and temporal JSON values. Furthermore, the `jsonb` type serves as base type for defining the `jsonbset` and the `tjsonb` types. Most functions and operators described in the previous chapters for set and temporal types are also applicable for the corresponding JSON types. In addition, there are specific functions defined for these types, which are derived from the corresponding functions of the `json` and the `jsonb` types.

In this chapter, we describe the MobilityDB JSON types and its associated operations. We refer to the PostgreSQL documentation for a detailed explanation of the `json` and `jsonb` types and their functionality. We aimed at enabling the same syntax of the original PostgreSQL functions for the corresponding MobilityDB types, as illustrated in this document.

Consider for example the -> operator, which extracts a JSON object field with a given key.

```
SELECT jsonb '{"unit": "km", "speed": 10}' -> text 'speed';
-- 10
```

Applying the same operator to a JSONB set and to a temporal JSONB value yields the following results

```
SELECT jsonbset '{"{\"unit\": \"km\", \"speed\": 10}",
  "{\"unit\": \"km\", \"speed\": 20}"}' -> text 'speed';
-- {10, 20}
SELECT tjsonb '[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02]' -> text 'speed';
-- {["10"@2001-01-01, "20"@2001-01-02]}
```

which are obtained by applying the PostgreSQL operator -> at every element of the JSONB set and to every instant of the temporal JSONB value.

15.1 JSONB Operations

PostgreSQL provides the following operations of JSONB values under its own names and operators. MobilityDB names them as their JSONB set and temporal JSONB counterparts, so that a query calling these names runs unchanged in every engine of the ecosystem (Section 18.8). The operators of PostgreSQL keep their symbols.

- Extract a JSON object field specified by a key

```
jsonbObjectField(jsonb,text) → jsonb
jsonbObjectFieldText(jsonb,text) → text
```

```
SELECT jsonbObjectField(jsonb '{"a": {"b": 1}}', 'a');
-- {"b": 1}
SELECT jsonbObjectFieldText(jsonb '{"a": "x"}', 'a');
-- x
```

- Extract a JSON value specified by a path

```
jsonbExtractPath(jsonb,path text[]) → jsonb
jsonbExtractPathText(jsonb,path text[]) → text
```

```
SELECT jsonbExtractPath(jsonb '{"a": {"b": [10, 20]}}', ARRAY['a', 'b', '1']);
-- 20
SELECT jsonbExtractPathText(jsonb '{"a": {"b": "z"}}', ARRAY['a', 'b']);
-- z
```

- Extract an element from a JSON array

```
jsonbArrayElement(jsonb,integer) → jsonb
jsonbArrayElementText(jsonb,integer) → text
```

Elements are numbered from zero, and a negative integer counts from the end of the array.

```
SELECT jsonbArrayElement(jsonb '[1, [2, 3]]', 1);
-- [2, 3]
SELECT jsonbArrayElement(jsonb '[1, 2, 3]', -1);
-- 3
SELECT jsonbArrayElementText(jsonb '["x", "y"]', 0);
-- x
```

- Return the number of elements of a JSON array

```
jsonbArrayLength(jsonb) → integer
```

```
SELECT jsonbArrayLength(jsonb '[1, 2, 3]');
-- 3
```

- Return the elements of a JSON array as rows

```
jsonbArrayElements(jsonb) → setof jsonb
jsonbArrayElementsText(jsonb) → setof text
```

```
SELECT * FROM jsonbArrayElements(jsonb '[1, "two", {"three": 3}]');
-- 1
-- "two"
-- {"three": 3}
SELECT * FROM jsonbArrayElementsText(jsonb '[1, "two"]');
-- 1
-- two
```

- Return the fields of a JSON object as rows of a key and a value

```
jsonbEach(jsonb) → setof (key text,value jsonb)
jsonbEachText(jsonb) → setof (key text,value text)
```

```
SELECT * FROM jsonbEach(jsonb '{"a": 1, "b": [2]}') ORDER BY key;
-- a | 1
-- b | [2]
SELECT * FROM jsonbEachText(jsonb '{"a": 1, "b": "x"}') ORDER BY key;
-- a | 1
-- b | x
```

- Return the keys of a JSON object as rows

```
jsonbObjectKeys(jsonb) → setof text
```

```
SELECT * FROM jsonbObjectKeys(jsonb '{"b": 1, "a": 2}') ORDER BY 1;
-- a
-- b
```

- Return the indented text of a JSON value

```
jsonbPretty(jsonb) → text
```

```
SELECT jsonbPretty(jsonb '{"a": 1}');
-- {
--     "a": 1
-- }
```

- Concatenate two JSON values

```
jsonbConcat(jsonb,jsonb) → jsonb
```

```
SELECT jsonbConcat(jsonb '{"a": 1}', jsonb '{"b": 2}');
-- {"a": 1, "b": 2}
```

- Delete a key, the keys of an array, an array element or the item at a path from a JSON value

```
jsonbDelete(jsonb,text) → jsonb
jsonbDeleteArray(jsonb,text[]) → jsonb
jsonbDeleteIndex(jsonb,integer) → jsonb
jsonbDeletePath(jsonb,path text[]) → jsonb
```

```
SELECT jsonbDelete(jsonb '{"a": 1, "b": 2}', 'a');
-- {"b": 2}
SELECT jsonbDeleteArray(jsonb '{"a": 1, "b": 2, "c": 3}', ARRAY['a', 'c']);
-- {"b": 2}
SELECT jsonbDeleteIndex(jsonb '[1, 2, 3]', -1);
-- [1, 2]
SELECT jsonbDeletePath(jsonb '{"a": {"b": 1, "c": 2}}', ARRAY['a', 'b']);
-- {"a": {"c": 2}}
```

- Does a text, or any or all the texts of an array, exist as a top-level key or array element of a JSON value?

```
jsonbExists(jsonb,text) → boolean
jsonbExistsAny(jsonb,text[]) → boolean
jsonbExistsAll(jsonb,text[]) → boolean
```

```
SELECT jsonbExists(jsonb '{"a": 1}', 'a');
-- true
SELECT jsonbExistsAny(jsonb '{"a": 1}', ARRAY['b', 'a']);
-- true
SELECT jsonbExistsAll(jsonb '{"a": 1}', ARRAY['b', 'a']);
-- false
```

- Does the first JSON value contain, or is it contained in, the second one?

```
jsonbContains(jsonb,jsonb) → boolean
jsonbContained(jsonb,jsonb) → boolean
```

```
SELECT jsonbContains(jsonb '{"a": 1, "b": 2}', jsonb '{"a": 1}');
-- true
SELECT jsonbContained(jsonb '{"a": 1}', jsonb '{"a": 1, "b": 2}');
-- true
```

- Return a JSON value with the item at a path set to a value

  ```
  jsonbSet(jsonb,path text[],val jsonb,create_missing boolean=true) → jsonb
  jsonbSetLax(jsonb,path text[],val jsonb,create_missing boolean=true,
    handle_null text='use_json_null') → jsonb
  ```

  The item is added when it does not exist and `create_missing` is true. Function `jsonbSetLax` treats a NULL value according to `handle_null`, which is one of `'raise_exception'`, `'use_json_null'`, `'delete_key'` or `'return_target'`.

  ```
  SELECT jsonbSet(jsonb '{"a": 1}', ARRAY['b'], jsonb '2');
  -- {"a": 1, "b": 2}
  SELECT jsonbSet(jsonb '{"a": 1}', ARRAY['b'], jsonb '2', false);
  -- {"a": 1}
  SELECT jsonbSetLax(jsonb '{"a": 1}', ARRAY['a'], NULL);
  -- {"a": null}
  SELECT jsonbSetLax(jsonb '{"a": 1}', ARRAY['a'], NULL, true, 'delete_key');
  -- {}
  ```

- Return a JSON value with a value inserted at a path

  ```
  jsonbInsert(jsonb,path text[],val jsonb,after boolean=false) → jsonb
  ```

  ```
  SELECT jsonbInsert(jsonb '[1, 3]', ARRAY['1'], jsonb '2');
  -- [1, 2, 3]
  SELECT jsonbInsert(jsonb '[1, 3]', ARRAY['1'], jsonb '2', true);
  -- [1, 3, 2]
  ```

- Return a JSON value without its object fields whose value is null

  ```
  jsonbStripNulls(jsonb,strip_in_arrays boolean=false) → jsonb
  ```

  The last argument states whether null array elements are also stripped.

  ```
  SELECT jsonbStripNulls(jsonb '{"a": null, "b": [1, null]}');
  -- {"b": [1, null]}
  SELECT jsonbStripNulls(jsonb '{"a": null, "b": [1, null]}', true);
  -- {"b": [1]}
  ```

- Does the JSON path return any item for a JSON value?

  ```
  jsonbPathExists(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → boolean
  jsonbPathExistsTz(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → boolean
  ```

  The last argument suppresses the following errors: missing object field or array element, unexpected JSON item type, datetime and numeric errors; the result of a suppressed error is NULL. The functions ending in `Tz` compare date and time values across time zones.

  ```
  SELECT jsonbPathExists(jsonb '{"a": [1, 2, 3]}', '$.a[*] ? (@ > 2)');
  -- true
  SELECT jsonbPathExists(jsonb '{"a": [1, 2, 3]}', '$.a[*] ? (@ > $x)', '{"x": 5}');
  -- false
  ```

- Return the result of a JSON path predicate check for a JSON value

```
jsonbPathMatch(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → boolean
jsonbPathMatchTz(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → boolean
```

```
SELECT jsonbPathMatch(jsonb '{"a": [1, 2, 3]}', 'exists($.a[*] ? (@ > 2))');
-- true
```

- Return the items a JSON path returns for a JSON value, as rows, as a JSON array, or the first one

```
jsonbPathQuery(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → setof jsonb
jsonbPathQueryTz(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → setof jsonb
jsonbPathQueryArray(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → jsonb
jsonbPathQueryArrayTz(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → jsonb
jsonbPathQueryFirst(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → jsonb
jsonbPathQueryFirstTz(jsonb,jsonpath,vars jsonb='{}',silent boolean=false) → jsonb
```

```
SELECT * FROM jsonbPathQuery(jsonb '{"a": [1, 2, 3]}', '$.a[*] ? (@ >= 2)');
-- 2
-- 3
SELECT jsonbPathQueryArray(jsonb '{"a": [1, 2, 3]}', '$.a[*] ? (@ >= 2)');
-- [2, 3]
SELECT jsonbPathQueryFirst(jsonb '{"a": [1, 2, 3]}', '$.a[*] ? (@ >= 2)');
-- 2
```

- Compare two JSON values, or return the hash value of a JSON value

```
jsonb {=, <>, <, <=, >=, >} jsonb → boolean
cmp(jsonb,jsonb) → integer
hash(jsonb) → integer
hashExtended(jsonb,seed bigint) → bigint
```

The operators are those of PostgreSQL, and the functions `eq`, `ne`, `lt`, `le`, `ge` and `gt` answer as them, together with `cmp`, which returns -1, 0, or 1. Functions `hash` and `hashExtended` answer as the functions `jsonb_hash` and `jsonb_hash_extended` of PostgreSQL.

```
SELECT eq(jsonb '{"a": 1}', jsonb '{"a": 1}');
-- true
SELECT cmp(jsonb '1', jsonb '2');
-- -1
```

15.2 JSONB Set Operations

When manipulating JSON collections of varying structure, it may be the case that an item is defined in some of the documents but not all. PostgreSQL provides the *lax mode* for this purpose, where the argument `null_value_treatment` determines the behavior in the case a function returns a NULL value. The argument can take one of the following values: `'raise_exception'`, `'use_json_null'`, `'delete_key'`, `'return_target'`, where `'use_json_null'` is the default value. In PostgreSQL the lax mode is supported only for *update* (not *query*) operations with the function `jsonb_set_lax`. In MobilityDB, we kept the same semantics for the corresponding function `jsonbsetSetLax`, but we enabled similar behavior for *all* JSON operations that may return a null value, except that we replaced the value `'return_target'` with `'return_null'`, since the former is not meaningful for set operations. For operators such as `->`, the default value `'use_json_null'` is used and cannot be changed, whereas for the corresponding functions `jsonbsetObjectField`, the last argument specifies the behavior in the case the function returns NULL. We illustrate this behavior for function `jsonbsetObjectField` below.

- Extract a JSON object field specified by a key

```
{textset,jsonbset} -> text → {textset,jsonbset}
jsonbset ->> text → textset
jsonbsetObjectField(jsonbset,text,null_handle text='use_json_null') → jsonbset
jsonbsetObjectFieldText(jsonbset,text,null_handle text='use_json_null') → textset
```

```
SELECT set(ARRAY[jsonb '{"unit": "km", "speed": 10}',
  '{"unit": "km", "speed": 20}']) -> text 'speed';
-- {10, 20}
SELECT set(ARRAY[jsonb '{"position":"Point(1 1)", "speed":10}',
  '{"position":"Point(2 2)", "speed":20}']) ->> text 'position';
-- {"Point(1 1)", "Point(2 2)"}
SELECT set(ARRAY[jsonb '{"unit": "km", "speed": 10}', '{"unit": "km"}',
  '{"unit": "km", "speed": 10}']) -> text 'speed';
-- {"null", 10}
```

We show below the behavior of the function `jsonbsetObjectField` for the possible values of the `null_handle` argument. All JSON functions that may return a NULL value behave similarly in MobilityDB.

```
SELECT jsonbsetObjectField(set(ARRAY[jsonb '{"unit": "km", "speed": 10}',
  '{"unit": "km"}', '{"unit": "km", "speed": 10}']), 'speed', 'raise_exception');
-- ERROR:  The lifted operation returned NULL
SELECT jsonbsetObjectField(set(ARRAY[jsonb '{"unit": "km", "speed": 10}',
  '{"unit": "km"}', '{"unit": "km", "speed": 10}']), 'speed', 'use_json_null');
-- {"null", 10}
SELECT jsonbsetObjectField(set(ARRAY[jsonb '{"unit": "km", "speed": 10}',
  '{"unit": "km"}', '{"unit": "km", "speed": 10}']), 'speed', 'delete_key');
-- {10}
SELECT jsonbsetObjectField(set(ARRAY[jsonb '{"unit": "km", "speed": 10}',
  '{"unit": "km"}', '{"unit": "km", "speed": 10}']), 'speed', 'return_null');
-- NULL
```

- Extract a JSON object field specified by a path

```
{textset,jsonbset} #> text[] → {textset,jsonbset}
jsonbsetExtractPath(jsonbset,path text[],null_handle text='use_json_null') → jsonbset
jsonbsetExtractPathText(jsonbset,path text[],null_handle text='use_json_null') → textset
```

```
SELECT set(ARRAY[jsonb '{"speed": {"unit": "km", "value": 10}}',
  '{"speed": {"unit": "km", "value": 20}}']) #> ARRAY[text 'speed', 'value'];
-- {10, 20}
SELECT set(ARRAY[jsonb '{"position":"Point(1 1)", "PoIs":["Grand Place", "La Bourse"]}',
  '{"position":"Point(2 2)", "PoIs":["Palais Royal", "Manneken Pis"]}'])
  #>> ARRAY[text 'PoIs', '1'];
-- {"La Bourse", "Manneken Pis"}
```

- Extract an element from a JSON array

```
jsonbsetArrayElement(jsonbset,integer,null_handle text='use_json_null') → jsonbset
jsonbsetArrayElementText(jsonbset,integer,null_handle text='use_json_null') → textset
```

```
SELECT jsonbsetArrayElement(set(ARRAY[jsonb '["Grand Place", "La Bourse"]',
  '["Palais Royal", "Manneken Pis"]']), 0);
-- {"\"Grand Place\"", "\"Palais Royal\""}
SELECT jsonbsetArrayElement(set(ARRAY[jsonb '["Grand Place", "La Bourse"]',
  '["Palais Royal", "Manneken Pis"]']), 1);
-- {"\"La Bourse\"", "\"Manneken Pis\""}
```

- Extract a alphanumeric value from a JSONB set given by a key

```
intset(jsonbset,text,null_handle text='raise_exception') → tint
floatset(jsonbset,text,null_handle text='raise_exception') → tfloat
textset(jsonbset,text,null_handle text='raise_exception') → textset
```

Note that the value `use_json_null` cannot be used for the above functions and thus the default value `'raise_exception'` is used.

```
SELECT intset(jsonbset '{"{\"speed\": 10, \"units\": \"km/h\"}",
  "{\"speed\": 20, \"units\": \"km/h\"}"}', 'speed');
-- {10, 20}
SELECT floatset(jsonbset '{"{\"speed\": 10, \"units\": \"km/h\"}",
  "{\"speed\": 20, \"units\": \"km/h\"}", "{\"speed\": 25}"}', 'speed');
-- {10, 20, 25}
SELECT textset(jsonbset '{"{\"road\": \"Bvd Gén. Jacques\", \"category\": \"primary\"}",
  "{\"road\": \"Bvd de la Cambre\", \"category\": \"residential\"}"}', 'category');
-- {"primary", "residential"}
```

- JSONB set concatenation

```
{jsonb,jsonbset} || {jsonbset,jsonb} → jsonbset
```

The value is concatenated with every element of the set, and the operand order is free.

```
SELECT set(ARRAY[jsonb '{"speed":10}', '{"speed":20}']) || '{"unit":"km"}'::jsonb;
-- {"{\"unit\": \"km\", \"speed\": 10}", "{\"unit\": \"km\", \"speed\": 20}"}
SELECT jsonb '{"unit":"km"}' || set(ARRAY[jsonb '{"speed":10}', '{"speed":20}']);
-- {"{\"unit\": \"km\", \"speed\": 10}", "{\"unit\": \"km\", \"speed\": 20}"}
```

- JSONB set deletion

```
jsonbset - {integer,text,text[]} → jsonbset
jsonbset #- text[] → jsonbset
```

```
SELECT set(ARRAY[jsonb '{"unit": "km", "speed": 10}',
  '{"unit": "km", "speed": 20}']) - text 'unit';
-- {"{\"speed\": 10}", "{\"speed\": 20}"}
SELECT set(ARRAY[jsonb '["Grand Place", "La Bourse"]',
  '["Palais Royal", "Manneken Pis"]']) - 1;
-- {"[\"Grand Place\"]", "[\"Palais Royal\"]"}
SELECT set(ARRAY[jsonb '{"location":"Point(1 1)", "PoIs": ["Grand Place", "La Bourse"]}',
  '{"location":"Point(2 2)", "PoIs": ["Palais Royal", "Manneken Pis"]}'])
  #- ARRAY[text 'PoIs', '1'];
/* {"{\"PoIs\": [\"Grand Place\"], \"location\": \"Point(1 1)\"}", "{\"PoIs\":
   [\"Palais Royal\"], \"location\": \"Point(2 2)\"}"} */
```

As shown below, the result of a delete operation may be an empty record or an empty array. To remove these values, the function `setMinus` can be used.

```
SELECT set(ARRAY[jsonb '{"unit": "km", "speed": 10, "light": true}',
  '{"unit": "km", "speed": 20}']) - ARRAY[text 'unit', 'speed'];
-- {"{}", "{\"light\": true}"}
SELECT set(ARRAY[jsonb '["Grand Place", "La Bourse"]', '["Palais Royal"]']) - 0;
-- {[], "[\"La Bourse\"]"}
```

```
SELECT setMinus(set(ARRAY[jsonb '{"unit": "km", "speed": 10, "light": true}',
  '{"unit": "km", "speed": 20}']) - ARRAY[text 'unit', 'speed'],
  set(ARRAY[jsonb '{}', '[]']));
-- {"{\"light\": true}"}
SELECT setMinus(set(ARRAY[jsonb '["Grand Place", "La Bourse"]', '["Palais Royal"]']) - 0,
  set(ARRAY[jsonb '{}', '[]']));
-- {"[\"La Bourse\"]"}
```

- JSONB set exists

```
jsonbset ? text → boolean[]
jsonbset {?|, ?&} text[] → boolean[]
```

As in PostgreSQL, the operators `?|` and `?&` test, respectively, whether *any* or *all* of the strings in the text array exist as top-level keys or array elements. The result is a Boolean for every value of the set, in the order of the values of the set.

```
SELECT jsonbset '{"{\"speed\": 10}", "{\"speed\": 20, \"units\": \"km/h\"}"}';
-- {"{\"speed\": 10}", "{\"speed\": 20, \"units\": \"km/h\"}"}
SELECT jsonbset '{"{\"speed\": 10}",
  "{\"speed\": 20, \"units\": \"km/h\"}"}' ? text 'units';
-- {f,t}
SELECT jsonbset '{"{\"speed\": 10}",
  "{\"speed\": 20, \"units\": \"km/h\"}"}' ?| ARRAY[text 'units', text 'lights'];
-- {f,t}
SELECT jsonbset '{"{\"speed\": 10}",
  "{\"speed\": 20, \"units\": \"km/h\"}"}' ?& ARRAY[text 'speed', text 'units'];
-- {f,t}
```

- JSONB set assignment

```
jsonbsetSet(jsonbset,path text[],val jsonb,create_missing boolean=true) → jsonbset
jsonbsetSetLax(jsonbset,path text[],val jsonb,create_missing boolean=true,
  handle_null text='use_json_null') → jsonbset
```

Return the `jsonbset` value with the item specified by path replaced by `jsonb`, or added if `create` is true and the item does not exist. All earlier steps in the path must exist, or the target is returned unchanged. As with the path oriented operators, negative integers that appear in the path count from the end of JSON arrays. If the last path step is an array index that is out of range, and create_if_missing is true, the new value is added at the beginning of the array if the index is negative, or at the end of the array if it is positive. Function `jsonbsetSetLax` behaves identically to `jsonbsetSet` if the given value is not NULL, Otherwise, it behaves according to the value of `handle_null`, which must be one of `'raise_exception'`, `'use_json_null'`, `'delete_key'`, or `'return_target'`. As stated above, we kept PostgreSQL behavior for this function enabling the value `'return_target'` whereas for all other *query* operations, this value has been replaced with `'return_null'`.

```
SELECT jsonbsetSet(set(ARRAY[jsonb '{"speed":10}', '{"speed":20}']), ARRAY['units'],
  '"km/h"'::jsonb);
-- {"{\"speed\": 10, \"units\": \"km/h\"}", "{\"speed\": 20, \"units\": \"km/h\"}"}
SELECT jsonbsetSet(set(ARRAY[jsonb '{"speed":10}', '{"speed":20}']), ARRAY['units'],
  '"km/h"'::jsonb, false);
-- {"{\"speed\": 10}", "{\"speed\": 20}"}
SELECT jsonbsetSet(set(ARRAY[jsonb '{"speed": 10, "units": "km/h"}',
  '{"speed": 20, "units": "km/h"}']), ARRAY['units'], '"mi/h"'::jsonb);
-- {"{\"speed\": 10, \"units\": \"mi/h\"}", "{\"speed\": 20, \"units\": \"mi/h\"}"}
SELECT jsonbsetSetLax(set(ARRAY[jsonb '{"speed": 10, "units": "km/h"}', '{"speed": 20}']),
  ARRAY['units'], NULL, true, 'delete_key');
-- {"{\"speed\": 10}", "{\"speed\": 20}"}
```

- Return the `jsonbset` value with `jsonb` inserted

```
jsonbsetInsert(jsonbset,path text[],val jsonb,after boolean=false) → jsonbset
```

If the item specified by the path is an array element, the new value will be inserted before that item if `after` is false, or after it otherwise. If the item specified by the path is an object field, the new value will be inserted only if the object does not already contain that key. All earlier steps in the path must exist, or the target is returned unchanged. As with the path oriented operators, negative integers that appear in the path count from the end of JSON arrays. If the last path step is an array index that is out of range, the new value is added at the beginning of the array if the index is negative, or at the end of the array if it is positive.

```
SELECT jsonbsetInsert(set(ARRAY[jsonb '{"speed":10}', '{"speed":20}']), ARRAY['units'],
  '"km/h"'::jsonb);
-- {"{\"speed\": 10}", "{\"speed\": 20}"}
SELECT jsonbsetInsert(set(ARRAY[jsonb '{"speed":10}', '{"speed":20}']), ARRAY['units'],
  '"km/h"'::jsonb, false);
-- {"{\"speed\": 10}", "{\"speed\": 20}"}
```

```
SELECT jsonbsetInsert(set(ARRAY[jsonb '{"speed": 10, "units": "km/h"}',
  '{"speed": 20, "units": "km/h"}']), ARRAY['units'], '"mi/h"'::jsonb);
-- {"{\"speed\": 10, \"units\": \"mi/h\"}", "{\"speed\": 20, \"units\": \"mi/h\"}"}
```

- Return a JSONB set without nulls

```
jsonbsetStripNulls(jsonbset,strip_in_arrays boolean=false) → jsonbset
```

The last argument states whether null array elements are also stripped. Bare null values are never stripped.

```
SELECT jsonbsetStripNulls(set(ARRAY[jsonb '{"speed": 10, "lights": null}',
  '{"speed": 20, "PoIs": ["Grand Place", "La Bourse", null]}']));
/* {"{\"speed\": 10}", "{\"PoIs\": [\"Grand Place\", \"La Bourse\", null], \"speed\":
   20}"} */
SELECT jsonbsetStripNulls(set(ARRAY[jsonb '{"speed": 10, "lights": null}',
  '{"speed": 20, "PoIs": ["Grand Place", "La Bourse", null]}']), true);
-- {"{\"speed\": 10}", "{\"PoIs\": [\"Grand Place\", \"La Bourse\"], \"speed\": 20}"}
SELECT jsonbsetStripNulls(set(ARRAY[jsonb
  '{"road": "Bvd Gén. Jacques", "category": "primary"}',
  '{"road": "Bvd de la Cambre", "category": "primary"}',
  '{"road": "rue de l''Abbaye", "category": null}']));
/* {"{\"road\": \"rue de l'Abbaye\"}", "{\"road\": \"Bvd Gén. Jacques\", \"category\":
   \"primary\"}", "{\"road\": \"Bvd de la Cambre\", \"category\": \"primary\"}"} */
```

As shown below, the function does not strip a bare null value. The function `setMinus` can be used for this purpose.

```
SELECT jsonbsetStripNulls(set(ARRAY[jsonb '{"speed": 10}', 'null',
  '{"speed": 20, "PoIs": ["Grand Place", "La Bourse"]}']));
/* {"null", "{\"speed\": 10}", "{\"PoIs\": [\"Grand Place\", \"La Bourse\"], \"speed\":
   20}"} */
SELECT setMinus(set(ARRAY[jsonb '{"speed": 10}', 'null',
  '{"speed": 20, "PoIs": ["Grand Place", "La Bourse"]}']), set(ARRAY[jsonb 'null']));
-- {"{\"speed\": 10}", "{\"PoIs\": [\"Grand Place\", \"La Bourse\"], \"speed\": 20}"}
```

- Does the JSON path return any item for the specified JSONB set?

```
jsonbset @? jsonpath → boolean[]
jsonbsetPathExists(jsonbset,vars jsonb='{}',silent boolean=false) → boolean[]
jsonbsetPathExistsTz(jsonbset,vars jsonb='{}',silent boolean=false) → boolean[]
```

The operator `@?` suppresses the following errors: missing object field or array element, unexpected JSON item type, datetime and numeric errors. The above functions can also suppress these types of errors by setting the last argument to true. This behavior might be helpful when searching JSON document collections of varying structure. The result is a Boolean for every value of the set, in the order of the values of the set.

```
SELECT jsonbset '{"{\"speed\": 10}", "{\"speed\": 20, \"units\": \"km/h\"}"}';
-- {"{\"speed\": 10}", "{\"speed\": 20, \"units\": \"km/h\"}"}
SELECT jsonbset '{"{\"speed\": 10}",
  "{\"speed\": 20, \"units\": \"km/h\"}"}' @? '$.units';
-- {f,t}
SELECT jsonbsetPathExists(jsonbset '{"{\"speed\": 10}",
  "{\"speed\": 20, \"units\": \"km/h\"}"}', '$.speed ? (@ > $min)', '{"min": 15}');
-- {f,t}
```

- Return the result of a JSON path predicate check for a JSONB set

```
jsonbset @@ jsonpath → boolean[]
jsonbsetPathMatch(jsonbset,vars jsonb='{}',silent boolean=false) → boolean[]
jsonbsetPathMatchTz(jsonbset,vars jsonb='{}',silent boolean=false) → boolean[]
```

The operator `@@` suppresses the following errors: missing object field or array element, unexpected JSON item type, datetime and numeric errors. The above functions can also suppress these types of errors by setting the last argument to true. This behavior might be helpful when searching JSON document collections of varying structure. The result is a Boolean for every value of the set, in the order of the values of the set.

```
SELECT jsonbset '{"{\"speed\": 10}", "{\"speed\": 20, \"units\": \"km/h\"}"}';
-- {"{\"speed\": 10}", "{\"speed\": 20, \"units\": \"km/h\"}"}
SELECT jsonbset '{"{\"speed\": 10}",
  "{\"speed\": 20, \"units\": \"km/h\"}"}' @@ '$.speed > 15';
-- {f,t}
SELECT jsonbsetPathMatch(jsonbset '{"{\"speed\": 10}",
  "{\"speed\": 20, \"units\": \"km/h\"}"}', '$.speed > $min', '{"min": 15}');
-- {f,t}
```

- Return all items returned by a JSON path from a JSONB set, as a JSON array

```
jsonbsetPathQueryArray(jsonbset,vars jsonb='{}',silent boolean=false) → jsonbset
jsonbsetPathQueryArrayTz(jsonbset,vars jsonb='{}',silent boolean=false) → jsonbset
```

The above function can suppress the following errors by setting the last argument to true: missing object field or array element, unexpected JSON item type, datetime and numeric errors. This behavior might be helpful when searching JSON document collections of varying structure.

```
SELECT jsonbsetPathQueryArray(set(ARRAY[jsonb '{"speed":10}',
  '{"speed": 20, "units": "km/h"}']), '$ ? (@.speed >= $min && @.speed <= $max)',
  '{"min":10, "max":30}');
-- {"[{\"speed\": 10}]", "[{\"speed\": 20, \"units\": \"km/h\"}]"}
SELECT jsonbsetPathQueryArrayTz(set(ARRAY[jsonb
  '{"cameraId":25, "inspections":["2001-01-03", "2001-01-06", "2001-01-09"]}',
  '{"cameraId":35, "inspections":["2001-01-04", "2001-01-07", "2001-01-10"]}']),
  '$.inspections ? (@.timestamp_tz() >= $min.timestamp_tz() &&
   @.timestamp_tz() <= $max.timestamp_tz())', '{"min":"2001-01-05", "max":"2001-01-09"}');
-- {"[\"2001-01-07\"]", "[\"2001-01-06\", \"2001-01-09\"]"}
```

- Return the fist item returned by a JSON path from a JSONB set

```
jsonbsetPathQueryFirst(jsonbset,vars jsonb='{}',silent boolean=false) → jsonbset
jsonbsetPathQueryFirstTz(jsonbset,vars jsonb='{}',silent boolean=false) → jsonbset
```

The above functions can suppress the following errors by setting the last argument to true: missing object field or array element, unexpected JSON item type, datetime and numeric errors. This behavior might be helpful when searching JSON document collections of varying structure.

```
SELECT jsonbsetPathQueryFirst(set(ARRAY[jsonb '{"speed":10}',
  '{"speed": 20, "units": "km/h"}']), '$.speed ? (@ >= $min && @ <= $max)',
  '{"min":10, "max":30}');
-- {10, 20}
SELECT jsonbsetPathQueryArrayTz(set(ARRAY[jsonb
  '{"cameraId":25, "inspections":["2001-01-03", "2001-01-06", "2001-01-09"]}',
  '{"cameraId":35, "inspections":["2001-01-04", "2001-01-07", "2001-01-10"]}']),
  '$.inspections ? (@.datetime() >= $min.datetime() && @.datetime() <= $max.datetime())',
  '{"min":"2001-01-05", "max":"2001-01-09"}');
-- {"[\"2001-01-07\"]", "[\"2001-01-06\", \"2001-01-09\"]"}
```

15.3 Temporal JSONB values

The temporal type `tjsonb` allows to represent the evolution in time of `jsonb` values. As all temporal types, it comes in three subtypes, namely, instant, sequence, and sequence set. Examples of `tjsonb` values in these subtypes are given next.

```
-- Instant
SELECT tjsonb '"{\"vehicleId\": 1, \"location\": \"Point(1 1)\"}"@2001-01-01';
-- "{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01
SELECT tjsonb '{{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02}';
/* {"{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02} */
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]';
/* ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02] */
SELECT tjsonb '{[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02],
  [{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03,
  {"vehicleId": 1, "location": "Point(3 3)"}@2001-01-04]}';
/* {["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02], ["{\"location\": \"Point(2 2)\",
   \"vehicleId\": 1}"@2001-01-03, "{\"location\": \"Point(3 3)\", \"vehicleId\":
   1}"@2001-01-04]} */
```

As can be seen above, for the `Instant` subtype we need to enclose the JSONB value between quotes and escape the inner quotes, otherwise the opening brace of the JSONB value would be interpreted as the beginning of a `Sequence` subtype with discrete interpolation.

The `tjsonb` type accepts type modifiers (or `typmod` in PostgreSQL terminology) to specify the subtype. The possible values for the subtype are `Instant`, `Sequence`, and `SequenceSet`. The argument is optional and if not specified for a column, values of any subtype are allowed.

```
SELECT tjsonb(Instant) '"{\"vehicleId\": 1, \"location\": \"Point(1 1)\"}"@2001-01-01';
-- "{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01
SELECT tjsonb(Sequence) '"{\"vehicleId\": 1, \"location\": \"Point(1 1)\"}"@2001-01-01';
-- ERROR: Temporal type (Instant) does not match column type (Sequence)
```

Temporal JSONB values of sequence or sequence set subtype are converted into a normal form so that equivalent values have identical representations. For this, consecutive instant values are merged when possible. Three consecutive instant values can be merged into two if their JSONB values are equal. Also, two consecutive sequences that are adjacent and have the same end and start JSONB value can be merged into a single one. Examples of transformation into a normal form are as follows.

```
SELECT asText(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]');
/* ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-03] */
SELECT asText(tjsonb '{[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03],
  ({"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-04]}');
/* {["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-03, "{\"location\": \"Point(2 2)\",
   \"vehicleId\": 1}"@2001-01-04]} */
```

15.4 Validity of temporal JSONB values

Temporal JSONB values must satisfy the constraints specified in Section 4.3 so that they are well defined. An error is raised whenever one of these constraints are not satisfied. Examples of incorrect values are as follows.

```
-- Null values are not allowed
SELECT tjsonb 'NULL@2001-01-01 08:05:00';
SELECT tjsonb 'Point(0 0)@NULL';
-- Base type is not a JSONB
SELECT tjsonb 'Point(0 0)@2001-01-01 08:05:00';
```

We give next the functions and operators for temporal JSONB. Most functions and operators for temporal types described in the previous chapters can be applied for temporal JSONB types. Therefore, in the signatures of the functions given previously, the notation `base` represents a `jsonb` value and the notation `ttype`, represents a `tjsonb` value. To avoid redundancy, we only present next some examples of these functions and operators for temporal JSONB values.

15.5 Input and Output

- Return the Well-Known Text (WKT) representation

  ```
  asText({tjsonb,tjsonb[],jsonb,jsonpath}) → {text,text[]}
  ```

  Over a JSONB value or a JSON path, the function returns the text PostgreSQL writes for the value, as `jsonb::text` and `jsonpath::text` do.

  ```
  SELECT asText(jsonb '{"b": {"c": [1, {"d": null}]}, "a": "x"}');
  -- {"a": "x", "b": {"c": [1, {"d": null}]}}
  SELECT asText(jsonpath 'strict $.a[0 to 2].b');
  -- strict $."a"[0 to 2]."b"
  SELECT asText(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]');
  /* ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
     \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02] */
  SELECT asText(ARRAY[tjsonb '"{\"vehicleId\":1, \"location\": \"Point(1 1)\"}"@2001-01-01',
    '"{\"vehicleId\": 1, \"location\": \"Point(2 2)\"}"@2001-01-02']);
  /* {"\"{\\\"location\\\": \\\"Point(1 1)\\\", \\\"vehicleId\\\":
     1}\"@2001-01-01","\"{\\\"location\\\": \\\"Point(2 2)\\\", \\\"vehicleId\\\":
     1}\"@2001-01-02"} */
  ```

  As can be seen above, for arrays of JSONB values, the elements must be enclosed between quotes and thus, the inner quotes must be escaped.

- Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation

  ```
  asBinary(tjsonb,endian text='') → bytea
  asHexWKB(tjsonb,endian text='') → text
  ```

  The result is encoded using either the little-endian (NDR) or the big-endian (XDR) encoding. If no encoding is specified, then the encoding of the machine is used.

  ```
  SELECT asBinary(tjsonb '"{\"vehicleId\": 1, \"location\": \"Point(1 2)\"}"@2001-01-01');
  /* \x0142000138000000000000000200002008000080090000000a000000090000106c6f636174696f6e76
     656869636c654964506f696e7428312032290020000000008001000040eba9c21c0000 */
  SELECT asHexWKB(tjsonb '"{\"vehicleId\": 1, \"location\": \"Point(1 2)\"}"@2001-01-01');
  /* 0142000138000000000000000200002008000080090000000A000000090000106C6F636174696F6E7665
     6869636C654964506F696E7428312032290020000000008001000040EBA9C21C0000 */
  ```

- Return the Moving Features JSON (MF-JSON) representation

  ```
  asMFJSON(text) → tjsonb
  ```

  ```
  SELECT asMFJSON(tjsonb '[{"Position": "Point(1 1)"}@2001-01-01,
    {"Position": "Point(2 2)"}@2001-01-02]');
  /* {"type":"MovingJsonb","values":[{"Position": "Point(1 1)"},{"Position": "Point(2
     2)"}],"datetimes":["2001-01-01T00:00:00","2001-01-02T00:00:00"],"lower_inc":true,"up
     per_inc":true,"interpolation":"Step"} */
  ```

- Input from the Well-Known Text (WKT) representation

```
tjsonbFromText(text) → tjsonb
jsonbFromText(text) → jsonb
jsonpathFromText(text) → jsonpath
```

A JSONB value and a JSON path are read from the text PostgreSQL reads for them, as `text::jsonb` and `text::jsonpath` do; the functions are the inverse of `asText`.

```
SELECT jsonbFromText('{"b": [1, 2], "a": null}');
-- {"a": null, "b": [1, 2]}
SELECT asText(jsonpathFromText('lax $.a.**{1 to last}'));
-- $."a".**{1 to last}
SELECT tjsonbFromText(text '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]');
/* ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02] */
```

- Input from the Well-Known Binary (WKB) or from the Hexadecimal Well-Known Binary (HexWKB) representation

```
tjsonbFromBinary(bytea) → tjsonb
tjsonbFromHexWKB(text) → tjsonb
```

```
SELECT tjsonbFromBinary(asBinary( tjsonb '[{"vehicleId": 1,
  "location": "Point(1 2)"}@2001-01-01]'));
-- ["{\"location\": \"Point(1 2)\", \"vehicleId\": 1}"@2001-01-01]
SELECT tjsonbFromHexWKB(asHexWKB( tjsonb '[{"vehicleId": 1,
  "location": "Point(1 2)"}@2001-01-01]'));
-- ["{\"location\": \"Point(1 2)\", \"vehicleId\": 1}"@2001-01-01]
```

- Input from the Moving Features JSON (MF-JSON) representation

```
tjsonbFromMFJSON(text) → tjsonb
```

```
SELECT tjsonbFromMFJSON('{"type": "MovingJsonb", "interpolation": "Step",
  "values": [{"Position": "Point(1 1)"}, {"Position": "Point(2 2)"}],
  "datetimes": ["2001-01-01T10:00:00", "2001-01-01T12:00:00"]}');
/* ["{\"Position\": \"Point(1 1)\"}"@2001-01-01 10:00:00, "{\"Position\": \"Point(2
   2)\"}"@2001-01-01 12:00:00] */
```

15.6 Constructors

- Constructor for temporal JSONB values having a constant value

```
tjsonb(jsonb,timestamptz) → tjsonbInst
tjsonb(jsonb,tstzset) → tjsonbDiscSeq
tjsonb(jsonb,tstzspan) → tjsonbContSeq
tjsonb(jsonb,tstzspanset) → tjsonbSeqSet
```

```
SELECT tjsonb('{"vehicleId": 1, "location": "Point(1 1)"}', timestamptz '2001-01-01');
-- "{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01
SELECT tjsonb('{"vehicleId": 1, "location": "Point(1 1)"}',
  tstzset '{2001-01-01, 2001-01-03, 2001-01-05}');
/* {"{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-03, "{\"location\": \"Point(1 1)\",
   \"vehicleId\": 1}"@2001-01-05} */
SELECT tjsonb('{"vehicleId": 1, "location": "Point(1 1)"}',
  tstzspan '[2001-01-01, 2001-01-02]');
```

```
/* ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-02] */
SELECT tjsonb('{"vehicleId": 1, "location": "Point(1 1)"}',
  tstzspanset '{[2001-01-01, 2001-01-02],[2001-01-03, 2001-01-04]}');
/* {["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-02], ["{\"location\": \"Point(1 1)\",
   \"vehicleId\": 1}"@2001-01-03, "{\"location\": \"Point(1 1)\", \"vehicleId\":
   1}"@2001-01-04]} */
```

- Constructor for temporal JSONB values of sequence subtype

```
tjsonbSeq(tjsonbInst[],lowerInc boolean=true,upperInc boolean=true) → tjsonbSeq
```

```
SELECT tjsonbSeq(ARRAY[tjsonb '"{\"vehicleId\": 1,
  \"location\": \"Point(1 1)\"}"@2001-01-01',
  '"{\"vehicleId\": 1, \"location\": \"Point(2 2)\"}"@2001-01-02',
  '"{\"vehicleId\": 1, \"location\": \"Point(1 1)\"}"@2001-01-03']);
/* ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02, "{\"location\": \"Point(1 1)\",
   \"vehicleId\": 1}"@2001-01-03] */
```

- Constructor for temporal JSONB values of sequence set subtype

```
tjsonbSeqSet(tjsonb[]) → tjsonbSeqSet
tjsonbSeqSetGaps(tjsonbInst[],maxt interval=NULL,maxdist float=NULL) → tjsonbSeqSet
```

```
SELECT tjsonbSeqSet(ARRAY[tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]',
  '[{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03,
  {"vehicleId": 1, "location": "Point(3 3)"}@2001-01-04]']);
/* {["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02], ["{\"location\": \"Point(2 2)\",
   \"vehicleId\": 1}"@2001-01-03, "{\"location\": \"Point(3 3)\", \"vehicleId\":
   1}"@2001-01-04]} */
SELECT tjsonbSeqSetGaps(ARRAY[tjsonb '"{\"vehicleId\": 1,
  \"location\": \"Point(1 1)\"}"@2001-01-01',
  '"{\"vehicleId\": 1, \"location\": \"Point(2 2)\"}"@2001-01-03',
  '"{\"vehicleId\": 1, \"location\": \"Point(3 3)\"}"@2001-01-05'], interval '1 day');
/* {["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01], ["{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-03], ["{\"location\": \"Point(3 3)\",
   \"vehicleId\": 1}"@2001-01-05]} */
```

15.7 Conversions

- Convert a temporal JSONB to a `tstzspan`

```
tjsonb::tstzspan
```

```
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]'::tstzspan;
-- [2001-01-01, 2001-01-02]
```

- Convert between a temporal JSONB and a text

```
{tjsonb,text}::{text,tjsonb}
```

```
SELECT (text '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]')::tjsonb;
/* ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02] */
SELECT pg_typeof(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]'::text);
-- text
```

- Convert between a temporal JSONB and a temporal text

```
{tjsonb,ttext}::{ttext,tjsonb}
```

```
SELECT (ttext '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]')::tjsonb;
/* ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-02] */
SELECT pg_typeof(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]'::ttext);
-- ttext
```

15.8 Accessors

- Return the values

```
getValues(tjsonb) → jsonbset
```

```
SELECT getValues(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02]');
-- {"{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"}
```

- Return the value at a timestamp

```
valueAtTimestamp(tjsonb,timestamptz) → jsonb
```

```
SELECT valueAtTimestamp(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]', '2001-01-02');
-- {"location": "Point(2 2)", "vehicleId": 1}
```

15.9 Transformations

- Transform a temporal JSONB to another subtype

```
tjsonbInst(tjsonb) → tjsonbInst
tjsonbSeq(tjsonb,interp text='step') → tjsonbSeq
tjsonbSeqSet(tjsonb) → tjsonbSeqSet
```

```
SELECT tjsonbSeq(tjsonb '"{\"vehicleId\": 1, \"location\": \"Point(1 1)\"}"@2001-01-01');
-- ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01]
SELECT tjsonbSeq(tjsonb '"{\"vehicleId\": 1, \"location\": \"Point(1 1)\"}"@2001-01-01',
  'discrete');
-- {"{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01}
SELECT tjsonbSeqSet(tjsonb '"{\"vehicleId\": 1,\"location\":\"Point(1 1)\"}"@2001-01-01');
-- {["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01]}
```

- Transform a temporal JSONB value to another interpolation

```
setInterp(tjsonb,interp text) → tjsonb
```

```
SELECT setInterp(tjsonb '"{\"vehicleId\": 1,\"location\": \"Point(1 1)\"}"@2001-01-01',
  'step');
-- ["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01]
SELECT setInterp(tjsonb '{[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01],
  [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02]}', 'discrete');
/* {"{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-02} */
```

15.10 Temporal JSON Operations

In this section, we give the temporal versions of the functions for the `json` and `jsonb` types. We refer to the PostgreSQL documentation for detailed explanations of these functions. We aimed at keeping as much as possible the same syntax for the non-temporal and the temporal versions of these functions as illustrated in this document.

As stated in Section 4.4, a common way to generalize the traditional operations to the temporal types is to apply the operation *at each instant*, which yields a temporal value as result. Consider for example the `->` operator that extracts a JSON object field with the given key.

```
SELECT jsonb '{"unit": "km", "speed": 10}' -> text 'speed';
-- 10
```

Applying the same operator to a temporal JSON or JSONB value yields the following result

```
SELECT ttext '{[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02]}' -> text 'speed';
-- {["10"@2001-01-01, "20"@2001-01-02]}
SELECT tjsonb '{[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02]}' -> text 'speed';
-- {["10"@2001-01-01, "20"@2001-01-02]}
```

which is obtained by applying the PostgreSQL operator `->` at every instant of the temporal JSON or JSONB value.

When manipulating JSON collections of varying structure, it may be the case that an item is defined in some of the documents but not all. PostgreSQL provides the lax mode for this purpose, where the argument `null_value_treatment` determines the behavior in the case a function returns a NULL value. The argument can take one of the following values: `’raise_exception’`, `’use_json_null’`, `’delete_key’`, `’return_target’`, where `’use_json_null’` is the default value. In PostgreSQL the lax mode is supported only for *update* (not *query*) operations with the function `jsonb_set_lax`. In MobilityDB, we kept the same semantics for the corresponding function `tjsonbSetLax`, but we enabled similar behavior for *all* temporal JSON operations that may return a null value, except that we replaced the value `’return_target’` with `’return_null’`, since the former is not meaningful for temporal operations. For operators such as `->`, the default value `’use_json_null’` is used and cannot be changed, whereas for the corresponding functions `tjsonObjectField` and `tjsonbObjectField`, the last argument specifies the behavior in the case the function returns NULL. We illustrate this behavior for function `tjsonObjectField` below.

- Extract a temporal JSON object field specified by a key

```
{ttext,tjsonb} -> text → {ttext,tjsonb}
tjsonb ->> text → ttext
tjsonObjectField(ttext,text,null_handle text='use_json_null') → ttext
tjsonbObjectField(tjsonb,text,null_handle text='use_json_null') → tjsonb
tjsonbObjectFieldText(tjsonb,text,null_handle text='use_json_null') → ttext
```

```
SELECT ttext '{[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02]}' -> text 'speed';
-- {["10"@2001-01-01, "20"@2001-01-02]}
SELECT tjsonb '[{"position":"Point(1 1)", "speed":10}@2001-01-01,
  {"position":"Point(2 2)", "speed":20}@2001-01-03]' ->> text 'position';
-- {["Point(1 1)"@2001-01-01, "Point(2 2)"@2001-01-03]}
SELECT ttext '{[{"unit": "km", "speed": 10}@2001-01-01, {"unit": "km"}@2001-01-02,
  {"unit": "km", "speed": 10}@2001-01-03]}' -> text 'speed';
-- {["10"@2001-01-01, "null"@2001-01-02, "10"@2001-01-03]}
```

We show below the behavior of the function `tjsonObjectField` for the possible values of the `null_handle` argument. All JSON functions that may return a NULL value behave similarly in MobilityDB.

```
SELECT tjsonObjectField(ttext '{[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03]}', 'speed',
  'raise_exception');
-- ERROR:  The lifted operation returned NULL
SELECT tjsonObjectField(ttext '{[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03]}', 'speed',
  'use_json_null');
-- {["10"@2001-01-01, "null"@2001-01-02, "10"@2001-01-03]}
SELECT tjsonObjectField(ttext '{[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03]}', 'speed',
  'delete_key');
-- {["10"@2001-01-01, "10"@2001-01-02), ["10"@2001-01-03]}
SELECT tjsonObjectField(ttext '{[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km"}@2001-01-02, {"unit": "km", "speed": 10}@2001-01-03]}', 'speed',
  'return_null');
-- NULL
```

- Extract a temporal JSON object field specified by a path

```
{ttext,tjsonb} #> text[] → {ttext,tjsonb}
tjsonExtractPath(ttext,path text[],null_handle text='use_json_null') → ttext
tjsonbExtractPath(tjsonb,path text[],null_handle text='use_json_null') → tjsonb
tjsonbExtractPathText(tjsonb,path text[],null_handle text='use_json_null') → ttext
```

```
SELECT ttext '[{"speed": {"unit": "km", "value": 10}}@2001-01-01,
  {"speed": {"unit": "km", "value": 20}}@2001-01-02]' #> ARRAY[text 'speed', 'value'];
-- {["10"@2001-01-01, "20"@2001-01-02]}
SELECT tjsonb '[{"position":"Point(1 1)", "PoIs":["Grand Place", "La Bourse"]}@2001-01-01,
  {"position":"Point(2 2)",
  "PoIs":["Palais Royal", "Manneken Pis"]}@2001-01-03]' #>> ARRAY[text 'PoIs', '1'];
-- {["La Bourse"@2001-01-01, "Manneken Pis"@2001-01-03]}
```

- Extract an element from a temporal JSON array

```
tjsonArrayElement(ttext,integer,null_handle text='use_json_null') → ttext
tjsonbArrayElement(tjsonb,integer,null_handle text='use_json_null') → tjsonb
tjsonbArrayElementText(tjsonb,integer,null_handle text='use_json_null') → ttext
```

```
SELECT tjsonArrayElement(ttext '[["Grand Place", "La Bourse"]@2001-01-01,
  ["Palais Royal", "Manneken Pis"]@2001-01-02]', 0);
-- {["\"Grand Place\""@2001-01-01, "\"Palais Royal\""@2001-01-02]}
SELECT tjsonbArrayElement(tjsonb '[["Grand Place", "La Bourse"]@2001-01-01,
  ["Palais Royal", "Manneken Pis"]@2001-01-02]', 1);
-- {["\"La Bourse\""@2001-01-01, "\"Manneken Pis\""@2001-01-02]}
```

- Extract a temporal alphanumeric value from a temporal JSONB value given by a key

```
tbool(tjsonb,text,null_handle text='raise_exception') → tbool
tint(tjsonb,text,null_handle text='raise_exception') → tint
tfloat(tjsonb,text,interp text='linear',null_handle text='raise_exception') → tfloat
ttext(tjsonb,text,null_handle text='raise_exception') → ttext
```

Note that the value `use_json_null` cannot be used for the above functions and thus the default value `'raise_exception'` is used.

```
SELECT tbool(tjsonb '[{"speed": 10, "lights": "on"}@2001-01-01,
  {"speed": 20, "lights": "on"}@2001-01-02]', 'lights');
-- {[t@2001-01-01, t@2001-01-02]}
SELECT tint(tjsonb '[{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02]', 'speed');
-- {[10@2001-01-01, 20@2001-01-02]}
SELECT tfloat(tjsonb '{[{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02],
  [{"speed": 20}@2001-01-03, {"speed": 20}@2001-01-04]}', 'speed', 'step');
-- Interp=Step;{[10@2001-01-01, 20@2001-01-02], [20@2001-01-03, 20@2001-01-04]}
SELECT ttext(tjsonb '[{"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-01,
  {"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-02,
  {"road": "Bvd de la Cambre", "category": "residential"}@2001-01-03]', 'category');
-- {["primary"@2001-01-01, "residential"@2001-01-03]}
```

- Extract a temporal geo value from a temporal JSONB value given by a key

```
tgeompoint(tjsonb,text,interp text='linear',null_handle text='raise_exception') → ↩
    tgeompoint
tgeogpoint(tjsonb,text,interp text='linear',null_handle text='raise_exception') → ↩
    tgeogpoint
tgeometry(tjsonb,text,null_handle text='raise_exception') → tgeometry
tgeography(tjsonb,text,null_handle text='raise_exception') → tgeography
```

The value of the key is a string in a text format of a geometry or a geography, such as WKT or EWKT, which is a point for the temporal points. As for the functions above, the value `use_json_null` cannot be used.

```
SELECT asText(tgeompoint(tjsonb '[{"vehicle": "bus", "pos": "Point(0 0)"}@2001-01-01,
  {"vehicle": "bus", "pos": "Point(2 2)"}@2001-01-03]', 'pos'));
-- {[POINT(0 0)@2001-01-01, POINT(2 2)@2001-01-03]}
SELECT asEWKT(tgeogpoint(tjsonb '[{"pos": "Point(4.35 50.85)"}@2001-01-01,
  {"pos": "Point(4.40 50.80)"}@2001-01-02]', 'pos'));
-- SRID=4326;{[POINT(4.35 50.85)@2001-01-01, POINT(4.4 50.8)@2001-01-02]}
SELECT asText(tgeometry(tjsonb '[{"area": "Polygon((0 0,1 0,1 1,0 0))"}@2001-01-01,
  {"area": "Point(2 2)"}@2001-01-02]', 'area'));
-- {[POLYGON((0 0,1 0,1 1,0 0))@2001-01-01, POINT(2 2)@2001-01-02]}
```

- Temporal JSONB concatenation

```
tjsonb || {jsonb,tjsonb} → tjsonb
```

```
SELECT tjsonb '{[{"speed":10}@2001-01-01,
  {"speed":20}@2001-01-02]}' || '{"unit":"km"}'::jsonb;
/* {["{\"unit\": \"km\", \"speed\": 10}"@2001-01-01, "{\"unit\": \"km\", \"speed\":
   20}"@2001-01-02]} */
SELECT tjsonb '{[{"speed":10}@2001-01-01, {"speed":20}@2001-01-03]}' ||
  tjsonb '{[{"Position":"Point(1 1)"}@2001-01-02, {"Position":"Point(2 2)"}@2001-01-04]}';
/* {["{\"speed\": 10, \"Position\": \"Point(1 1)\"}"@2001-01-02, "{\"speed\": 20,
   \"Position\": \"Point(1 1)\"}"@2001-01-03]} */
```

- Temporal JSONB deletion

```
tjsonb - {integer,text,text[]} → tjsonb
tjsonb #- text[] → tjsonb
```

```
SELECT tjsonb '{[{"unit": "km", "speed": 10}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02]}' - 'unit';
-- {["{\"speed\": 10}"@2001-01-01, "{\"speed\": 20}"@2001-01-02]}
SELECT tjsonb '[["Grand Place", "La Bourse"]@2001-01-01,
  ["Palais Royal", "Manneken Pis"]@2001-01-02]' - 1;
-- {["[\"Grand Place\"]"@2001-01-01, "[\"Palais Royal\"]"@2001-01-02]}
SELECT tjsonb
  '[{"location":"Point(1 1)", "PoIs": ["Grand Place", "La Bourse"]}@2001-01-01,
  {"location":"Point(2 2)", "PoIs": ["Palais Royal", "Manneken Pis"]}@2001-01-02]'
  #- ARRAY[text 'PoIs', '1'];
/* {["{\"PoIs\": [\"Grand Place\"], \"location\": \"Point(1 1)\"}"@2001-01-01,
   "{\"PoIs\": [\"Palais Royal\"], \"location\": \"Point(2 2)\"}"@2001-01-02]} */
```

As shown below, the result of a delete operation may be an empty record or an empty array. To remove these values, the function `minusValues` can be used.

```
SELECT tjsonb '{[{"unit": "km", "speed": 10, "light": true}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02]}' - ARRAY[text 'unit', 'speed'];
-- {["{\"light\": true}"@2001-01-01, "{}"@2001-01-02]}
SELECT tjsonb '[["Grand Place", "La Bourse"]@2001-01-01,
  ["Palais Royal"]@2001-01-02]' - 0;
-- {["[\"La Bourse\"]"@2001-01-01, "[]"@2001-01-02]}
```

```
SELECT minusValues(tjsonb '{[{"unit": "km", "speed": 10, "light": true}@2001-01-01,
  {"unit": "km", "speed": 20}@2001-01-02]}' - ARRAY[text 'unit', 'speed'],
  jsonbset '{"[]","{}"}');
-- {["{\"light\": true}"@2001-01-01, "{\"light\": true}"@2001-01-02)}
SELECT minusValues(tjsonb '[["Grand Place", "La Bourse"]@2001-01-01,
  ["Palais Royal"]@2001-01-02]' - 0, jsonbset '{"[]","{}"}');
-- {["[\"La Bourse\"]"@2001-01-01, "[\"La Bourse\"]"@2001-01-02)}
```

- Temporal JSONB exists

```
tjsonb ? jsonb → tbool
tjsonb {?|, ?&} jsonb[] → tbool
```

As in PostgreSQL, the operators `?|` and `?&` test, respectively, whether *any* or *all* of the strings in the text array exist as top-level keys or array elements.

```
SELECT tjsonb '"{\"geom\": \"Point(1 1)\"}"@2001-01-01' ? text 'geom';
-- t@2001-01-01
SELECT tjsonb '{{"geom": "Point(1 1)"}@2001-01-01, {"geom": "Point(2 2)"}@2001-01-02,
  {"geom": "Point(1 1)"}@2001-01-03}' ?| ARRAY[text 'geom'];
-- {t@2001-01-01, t@2001-01-02, t@2001-01-03}
SELECT tjsonb '[{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02]' ?& ARRAY['speed', 'units'];
-- [t@2001-01-01, t@2001-01-02]
```

- Temporal JSONB contains/contained

```
{tjsonb,jsonb} {@>, <@} {tjsonb,jsonb} → tbool
```

```
SELECT tjsonb '"{\"geom\": \"Point(1 1)\"}"@2001-01-01' @> jsonb '{"geom": "Point(1 1)"}';
-- t@2001-01-01
SELECT tjsonb '{{"geom": "Point(1 1)"}@2001-01-01, {"geom": "Point(2 2)"}@2001-01-02,
  {"geom": "Point(1 1)"}@2001-01-03}' @> jsonb '{"geom": "Point(1 1)"}';
-- {t@2001-01-01, f@2001-01-02, t@2001-01-03}
```

```
SELECT jsonb '{"geom": "Point(1 1)"}' <@ tjsonb '{[{"geom": "Point(1 1)"}@2001-01-01,
  {"geom": "Point(1 1)"}@2001-01-02, {"geom": "Point(1 1)"}@2001-01-03],
  [{"geom": "Point(2 2)"}@2001-01-04, {"geom": "Point(2 2)"}@2001-01-05]}';
-- {[t@2001-01-01, t@2001-01-03], [f@2001-01-04, f@2001-01-05]}
```

- Temporal JSONB set

```
tjsonbSet(tjsonb,path text[],val jsonb,create_missing boolean=true) → tjsonb
tjsonbSetLax(tjsonb,path text[],val jsonb,create_missing boolean=true,
  handle_null text='use_json_null') → tjsonb
```

Return the `tjsonb` value with the item specified by path replaced by `jsonb`, or added if `create` is true and the item does not exist. All earlier steps in the path must exist, or the target is returned unchanged. As with the path oriented operators, negative integers that appear in the path count from the end of JSON arrays. If the last path step is an array index that is out of range, and create_if_missing is true, the new value is added at the beginning of the array if the index is negative, or at the end of the array if it is positive. Function `tjsonbSetLax` behaves identically to `tjsonbSet` if the given value is not NULL, Otherwise, it behaves according to the value of `handle_null`, which must be one of `'raise_exception'`, `'use_json_null'`, `'delete_key'`, or `'return_target'`. As stated above, we kept PostgreSQL behavior for this function enabling the value `'return_target'` whereas for all other *query* operations, this value has been replaced with `'return_null'`.

```
SELECT tjsonbSet(tjsonb '[{"speed":10}@2001-01-01, {"speed":20}@2001-01-02]',
  ARRAY['units'], '"km/h"'::jsonb);
/* {["{\"speed\": 10, \"units\": \"km/h\"}"@2001-01-01, "{\"speed\": 20, \"units\":
   \"km/h\"}"@2001-01-02]} */
SELECT tjsonbSet(tjsonb '[{"speed":10}@2001-01-01, {"speed":20}@2001-01-02]',
  ARRAY['units'], '"km/h"'::jsonb, false);
-- {["{\"speed\": 10}"@2001-01-01, "{\"speed\": 20}"@2001-01-02]}
SELECT tjsonbSet(tjsonb '[{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02]', ARRAY['units'], '"mi/h"'::jsonb);
/* {["{\"speed\": 10, \"units\": \"mi/h\"}"@2001-01-01, "{\"speed\": 20, \"units\":
   \"mi/h\"}"@2001-01-02]} */
SELECT tjsonbSetLax(tjsonb '[{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20}@2001-01-02]', ARRAY['units'], NULL, true, 'delete_key');
-- {["{\"speed\": 10}"@2001-01-01, "{\"speed\": 20}"@2001-01-02]}
```

- Temporal JSONB insert

```
tjsonbInsert(tjsonb,path text[],val jsonb,after boolean=false) → tjsonb
```

Return the `tjsonb` value with `jsonb` inserted. If the item specified by the path is an array element, the new value will be inserted before that item if `after` is false, or after it otherwise. If the item specified by the path is an object field, the new value will be inserted only if the object does not already contain that key. All earlier steps in the path must exist, or the target is returned unchanged. As with the path oriented operators, negative integers that appear in the path count from the end of JSON arrays. If the last path step is an array index that is out of range, the new value is added at the beginning of the array if the index is negative, or at the end of the array if it is positive.

```
SELECT tjsonbInsert(tjsonb '[{"speed":10}@2001-01-01, {"speed":20}@2001-01-02]',
  ARRAY['units'], '"km/h"'::jsonb);
-- {["{\"speed\": 10}"@2001-01-01, "{\"speed\": 20}"@2001-01-02]}
SELECT tjsonbInsert(tjsonb '[{"speed":10}@2001-01-01, {"speed":20}@2001-01-02]',
  ARRAY['units'], '"km/h"'::jsonb, false);
-- {["{\"speed\": 10}"@2001-01-01, "{\"speed\": 20}"@2001-01-02]}
SELECT tjsonbInsert(tjsonb '[{"speed": 10, "units": "km/h"}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02]', ARRAY['units'], '"mi/h"'::jsonb);
/* {["{\"speed\": 10, \"units\": \"mi/h\"}"@2001-01-01, "{\"speed\": 20, \"units\":
   \"mi/h\"}"@2001-01-02]} */
```

- Return a temporal JSON value without nulls

```
tjsonStripNulls(ttext,strip_in_arrays boolean=false) → ttext
tjsonbStripNulls(tjsonb,strip_in_arrays boolean=false) → tjsonb
```

The last argument states whether null array elements are also stripped. Bare null values are never stripped.

```
SELECT tjsonStripNulls(ttext '[{"speed": 10, "lights": null}@2001-01-01,
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse", null]}@2001-01-02]');
/* ["{\"speed\":10}"@2001-01-01, "{\"speed\":20,\"PoIs\":[\"Grand Place\",\"La
   Bourse\",null]}"@2001-01-02] */
SELECT tjsonbStripNulls(tjsonb '[{"speed": 10, "lights": null}@2001-01-01,
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse", null]}@2001-01-02]', true);
/* {["{\"speed\": 10}"@2001-01-01, "{\"PoIs\": [\"Grand Place\", \"La Bourse\"],
   \"speed\": 20}"@2001-01-02]} */
SELECT tjsonbStripNulls(tjsonb '{{"road": "Bvd Gén. Jacques",
  "category": "primary"}@2001-01-01,
  {"road": "Bvd de la Cambre", "category": "primary"}@2001-01-02,
  {"road": "rue de l''Abbaye", "category": null}@2001-01-03}');
/* {"{\"road\": \"Bvd Gén. Jacques\", \"category\": \"primary\"}"@2001-01-01,
   "{\"road\": \"Bvd de la Cambre\", \"category\": \"primary\"}"@2001-01-02,
   "{\"road\": \"rue de l'Abbaye\"}"@2001-01-03} */
```

As shown below, the function does not strip bare null values. The function `minusValues` can be used for this purpose.

```
SELECT tjsonStripNulls(ttext '[{"speed": 10}@2001-01-01, null@2001-01-02,
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse"]}@2001-01-03]');
/* ["{\"speed\":10}"@2001-01-01, "null"@2001-01-02, "{\"speed\":20,\"PoIs\":[\"Grand
   Place\",\"La Bourse\"]}"@2001-01-03] */
SELECT minusValues(ttext '[{"speed": 10}@2001-01-01, null@2001-01-02,
  {"speed": 20, "PoIs": ["Grand Place", "La Bourse"]}@2001-01-03]',
  set(ARRAY[text 'null']));
/* {["{\"speed\": 10}"@2001-01-01, "{\"speed\": 10}"@2001-01-02), ["{\"speed\": 20,
   \"PoIs\": [\"Grand Place\", \"La Bourse\"]}"@2001-01-03]} */
```

- Does the JSON path return any item for the specified temporal JSONB value?

```
tjsonb @? jsonpath → tbool
tjsonbPathExists(tjsonb,vars jsonb='{}',silent boolean=false) → tbool
tjsonbPathExistsTz(tjsonb,vars jsonb='{}',silent boolean=false) → tbool
```

The operator `@?` suppresses the following errors: missing object field or array element, unexpected JSON item type, datetime and numeric errors. The above functions can also suppress these types of errors by setting the last argument to true. This behavior might be helpful when searching JSON document collections of varying structure.

```
SELECT tjsonb '[{"speed":10}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02]' @? '$.units ? (@ == "km/h")';
-- [f@2001-01-01, t@2001-01-02]
SELECT tjsonbPathExists(tjsonb '{{"road": "Bvd Gén. Jacques",
  "category": "primary"}@2001-01-01,
  {"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-02,
  {"road": "Bvd de la Cambre", "category": "residential"}@2001-01-03}',
  '$.category ? (@ == "residential")');
-- {f@2001-01-01, f@2001-01-02, t@2001-01-03}
SELECT tjsonbPathExistsTz(tjsonb '[{"speed":10, "cameraId": "25",
  "lastInspection": "2001-08-01 12:00:00"}@2001-01-01,
  {"speed":20, "cameraId": "35", "lastInspection": "2001-03-01 12:00:00"}@2001-01-02]',
  '$.lastInspection ? (@.datetime() > "2001-06-01".datetime() )');
-- [t@2001-01-01, f@2001-01-02]
```

- Return the result of a JSON path predicate check for a temporal JSONB value

```
tjsonb @@ jsonpath → tbool
tjsonbPathMatch(tjsonb,vars jsonb='{}',silent boolean=false) → tbool
tjsonbPathMatchTz(tjsonb,vars jsonb='{}',silent boolean=false) → tbool
```

The operator `@@` suppresses the following errors: missing object field or array element, unexpected JSON item type, datetime and numeric errors. The above functions can also suppress these types of errors by setting the last argument to true. This behavior might be helpful when searching JSON document collections of varying structure.

```
SELECT tjsonb '[{"speed":10}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02]' @@ '$.units == "km/h"';
-- [f@2001-01-01, t@2001-01-02]
SELECT tjsonbPathMatch(tjsonb '{{"road": "Bvd Gén. Jacques",
  "category": "primary"}@2001-01-01,
  {"road": "Bvd Gén. Jacques", "category": "primary"}@2001-01-02,
  {"road": "Bvd de la Cambre", "category": "residential"}@2001-01-03}',
  '$.category == "residential"');
-- {f@2001-01-01, f@2001-01-02, t@2001-01-03}
SELECT tjsonbPathMatchTz(tjsonb '[{"speed":10, "cameraId": "25",
  "lastInspection": "2001-08-01 12:00:00"}@2001-01-01,
  {"speed":20, "cameraId": "35", "lastInspection": "2001-03-01 12:00:00"}@2001-01-02]',
  '$.lastInspection.datetime() > "2001-06-01".datetime()');
-- [t@2001-01-01, f@2001-01-02]
```

- Return all items returned by a JSON path from a temporal JSONB value, as a JSON array

```
tjsonbPathQueryArray(tjsonb,vars jsonb='{}',silent boolean=false) → tjsonb
tjsonbPathQueryArrayTz(tjsonb,vars jsonb='{}',silent boolean=false) → tjsonb
```

The above function can suppress the following errors by setting the last argument to true: missing object field or array element, unexpected JSON item type, datetime and numeric errors. This behavior might be helpful when searching JSON document collections of varying structure.

```
SELECT tjsonbPathQueryArray(tjsonb '[{"speed":10}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02]',
  '$ ? (@.speed >= $min && @.speed <= $max)', '{"min":10, "max":30}');
-- {["[{\"speed\": 10}]"@2001-01-01, "[{\"speed\": 20, \"units\": \"km/h\"}]"@2001-01-02]}
SELECT tjsonbPathQueryArrayTz(tjsonb '[{"cameraId":25,
  "inspections":["2001-01-03", "2001-01-06", "2001-01-09"]}@2001-01-01,
  {"cameraId":35, "inspections":["2001-01-04", "2001-01-07", "2001-01-10"]}@2001-01-02]',
  '$.inspections ? (@.timestamp_tz() >= $min.timestamp_tz() &&
   @.timestamp_tz() <= $max.timestamp_tz())', '{"min":"2001-01-05", "max":"2001-01-09"}');
-- {["[\"2001-01-06\", \"2001-01-09\"]"@2001-01-01, "[\"2001-01-07\"]"@2001-01-02]}
```

- Return the fist item returned by a JSON path from a temporal JSONB value

```
tjsonbPathQueryFirst(tjsonb,vars jsonb='{}',silent boolean=false) → tjsonb
tjsonbPathQueryFirstTz(tjsonb,vars jsonb='{}',silent boolean=false) → tjsonb
```

The above functions can suppress the following errors by setting the last argument to true: missing object field or array element, unexpected JSON item type, datetime and numeric errors. This behavior might be helpful when searching JSON document collections of varying structure.

```
SELECT tjsonbPathQueryFirst(tjsonb '[{"speed":10}@2001-01-01,
  {"speed": 20, "units": "km/h"}@2001-01-02]',
  '$.speed ? (@ >= $min && @ <= $max)', '{"min":10, "max":30}');
-- {["10"@2001-01-01, "20"@2001-01-02]}
SELECT tjsonbPathQueryArrayTz(tjsonb
  '[{"cameraId":25, "inspections":["2001-01-03", "2001-01-06", "2001-01-09"]}@2001-01-01,
   {"cameraId":35, "inspections":["2001-01-04", "2001-01-07", "2001-01-10"]}@2001-01-02]',
  '$.inspections ? (@.datetime() >= $min.datetime() && @.datetime() <= $max.datetime())',
  '{"min":"2001-01-05", "max":"2001-01-09"}');
-- {["[\"2001-01-06\", \"2001-01-09\"]"@2001-01-01, "[\"2001-01-07\"]"@2001-01-02]}
```

15.11 Restrictions

- Restrict to (the complement of) a value or a set of values

```
atValue(tjsonb,base) → tjsonb
minusValue(tjsonb,base) → tjsonb
atValues(tjsonb,values) → tjsonb
minusValues(tjsonb,values) → tjsonb
```

```
SELECT atValue(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03]',
  jsonb '{"vehicleId": 1, "location": "Point(2 2)"}');
-- {["{\"location\": \"Point(2 2)\", \"vehicleId\": 1}"@2001-01-03]}
SELECT minusValue(tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03]',
  jsonb '{"vehicleId": 1, "location": "Point(2 2)"}');
/* {["{\"location\": \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-01, "{\"location\":
   \"Point(1 1)\", \"vehicleId\": 1}"@2001-01-03)} */
```

15.12 Bounding Box Operations

- Topological operators

```
{tjsonb,tstzspan} {&&, <@#, #@>, ~=, -|-} {tjsonb,tstzspan} → boolean
```

The temporal contains and contained operator are marked with a # to distinguish them from the corresponding JSONB operators.

```
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' &&
  tstzspan '[2001-01-01, 2001-03-01]';
-- true
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' #@>
  timestamptz '[2001-01-02]'::tstzspan;
-- true
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' -|-
  tjsonb '({"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02,
    {"vehicleId": 1, "location": "Point(3 3)"}@2001-01-03]';
-- true
```

- Position operators

```
{tjsonb,tstzspan} {<<#, &<#, #>>, #&>} {tjsonb,tstzspan} → boolean
```

```
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' <<# tstzspan '[2001-01-01,
  2001-03-01]';
-- false
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03]' #>>
  timestamptz '[2001-01-01]'::tstzspan;
-- true
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-03]' #&>
  tjsonb '[{"vehicleId": 1, "location": "Point(2 2)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(3 3)"}@2001-01-03]';
-- true
```

15.13 Comparisons

- Traditional comparisons

```
tjsonb {=, <>, <, >, <=, >=} tjsonb → boolean
```

```
SELECT tjsonb '{[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02),
  [{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
   {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]}' =
  tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]';
-- true
SELECT tjsonb '{[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]}' <>
  tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]';
-- false
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]' <
  tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]';
-- false
```

- Ever and always comparisons

```
{jsonb,tjsonb} {?=, %=} {jsonb,tjsonb} → boolean
```

```
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' ?= jsonb '{"vehicleId": 1,
  "location": "Point(1 1)"}';
-- true
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(2 2)"}@2001-01-02]' %= jsonb '{"vehicleId": 1,
  "location": "Point(1 1)"}';
-- false
```

- Temporal comparisons

```
{jsonb,tjsonb} {#=, #<>} {jsonb,tjsonb} → tbool
```

```
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03)' #= jsonb '{"vehicleId": 1,
  "location": "Point(1 1)"}';
-- [t@2001-01-01, t@2001-01-03)
SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03)' #<>
  tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
  {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03)';
-- [f@2001-01-01, f@2001-01-03)
```

15.14 Aggregations

The aggregate functions for temporal JSONB values are illustrated next.

- Temporal count

```
tCount(tjsonb) → {tintSeq,tintSeqSet}
```

```
WITH Temp(temp) AS (
  SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]' UNION
  SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-04]' )
SELECT tCount(Temp) FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 2@2001-01-03], (1@2001-01-03, 1@2001-01-04]}
```

- Window count

```
wCount(tjsonb) → {tintSeq,tintSeqSet}
```

```
WITH Temp(temp) AS (
  SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-01,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-03]' UNION
  SELECT tjsonb '[{"vehicleId": 1, "location": "Point(1 1)"}@2001-01-02,
    {"vehicleId": 1, "location": "Point(1 1)"}@2001-01-04]' )
SELECT wCount(Temp, interval '2 days') FROM Temp;
-- {[1@2001-01-01, 2@2001-01-02, 2@2001-01-05], (1@2001-01-05, 1@2001-01-06]}
```

15.15 Indexing

GiST and SP-GiST indexes can be created for table columns of temporal JSONB values. An example of index creation is follows:

```
CREATE INDEX Trips_Trip_SPGist_Idx ON Trips USING SPGist(Trip);
```

The GiST and SP-GiST indexes store the bounding box for the temporal JSONB values, which is an `stbox`, as all spatiotemporal types.

A GiST or SP-GiST index can accelerate queries involving the following operators:

- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, which only consider the spatial dimension in temporal JSONB values,
- `<<#`, `&<#`, `#&>`, `#>>`, which only consider the time dimension in temporal JSONB values,
- `&&`, `@>`, `<@`, `~=`, `-|-`, and `|=|` , which consider as many dimensions as they are shared by the indexed column and the query argument.

These operators work on bounding boxes, not the entire values.

Chapter 16

Temporal Tessellations

A *discrete global grid system* (DGGS) partitions the Earth into a hierarchy of cells, each identified by a 64-bit integer, so that a position becomes a cell identifier and a containment test becomes an integer comparison. MobilityDB supports three of them, and a temporal cell index type records the movement of an object as the sequence of cells it occupies over time. Each grid tessellates the Earth, and these three tessellate it globally: a cell identifier names a cell of one of them wherever it is read, which is what lets a temporal type carry cells and nothing else.

Grid	**Cell type**	**Set type**	**Temporal type**	**Resolutions**
Uber H3	`h3index`	`h3indexset`	`th3index`	0–15
CARTO QUADBIN	`quadbin`	`quadbinset`	`tquadbin`	0–26
Google S2	`s2cell`	`s2cellset`	`ts2cell`	0–30

Figure 16.1 shows one AIS vessel track leaving Frederikshavn on the three grids, at the resolution of each whose cells are closest in area — 0.57 km^2 for H3, 0.43 km^2 for QUADBIN and 0.32 km^2 for S2 at this latitude — with the cells the track passes through shaded and a marine protected area in green. A temporal cell index records that shaded sequence, so the track is one and the same and each grid states it as a different sequence of identifiers. Asking whether the vessel entered the protected area then becomes an integer test between two sets of identifiers rather than a test between two geometries. The lower row states the excerpt outlined in black above, one resolution finer in each grid, where an H3 cell has about a seventh of the area of its parent and a QUADBIN or S2 cell about a fourth, and the sequence follows the track closely enough to stand for it.

The three differ in the shape of a cell and in how the hierarchy branches. Uber's H3 tessellates the sphere into 122 base cells at resolution 0 and subdivides each into 7 children at every finer resolution, up to 15, so a cell is a hexagon on an icosahedron and its identifier encodes the base cell, the resolution and the hierarchical position. CARTO's QUADBIN is built on the slippy-map (Web-Mercator) tile scheme: a single world cell at resolution 0 subdivides into four equal children at every finer resolution, up to 26, and the identifier encodes a header tag, the resolution and the tile coordinates. Google's S2 projects the six faces of a circumscribed cube onto the sphere and subdivides each face into four children, up to level 30, so a cell is a spherical quadrilateral and the identifier encodes the face, the position along a Hilbert curve and the level.

Each shape buys something. A hexagon has six neighbours all at the same distance from its centre, so an H3 step is the same move in every direction, which is what a flow or a density surface wants. A QUADBIN square is a slippy-map tile, so a cell IS the tile a map server already serves and the identifier converts to the `(x,y,z)` triple and the quadkey string of that ecosystem. An S2 cell is a quadtree cell of a cube face projected onto the sphere, so the descendants of a cell occupy one contiguous interval of the Hilbert curve and an ancestor test is a pair of integer comparisons. The grids are documented by their authors at h3geo.org, CARTO and s2geometry.io, and the family they belong to at Discrete global grid.

They answer the same questions, so this chapter states each operation once and names the grid only where the answer depends on it. S2 calls a resolution a *level*; the two words denote the same quantity.

A cell index type shares its on-disk representation with `tbigint`: both store a temporal 64-bit integer. Not every integer is a cell, since each grid gives its cells the structure its specification defines, QUADBIN its header, resolution and unused bits, S2 its face and the one bit ending its position, and H3 its mode bits. The distinct SQL type exists so that the type-checker rejects a cell-specific function applied to an arbitrary `tbigint` trajectory, and the reverse, and an integer entering a cell type is checked

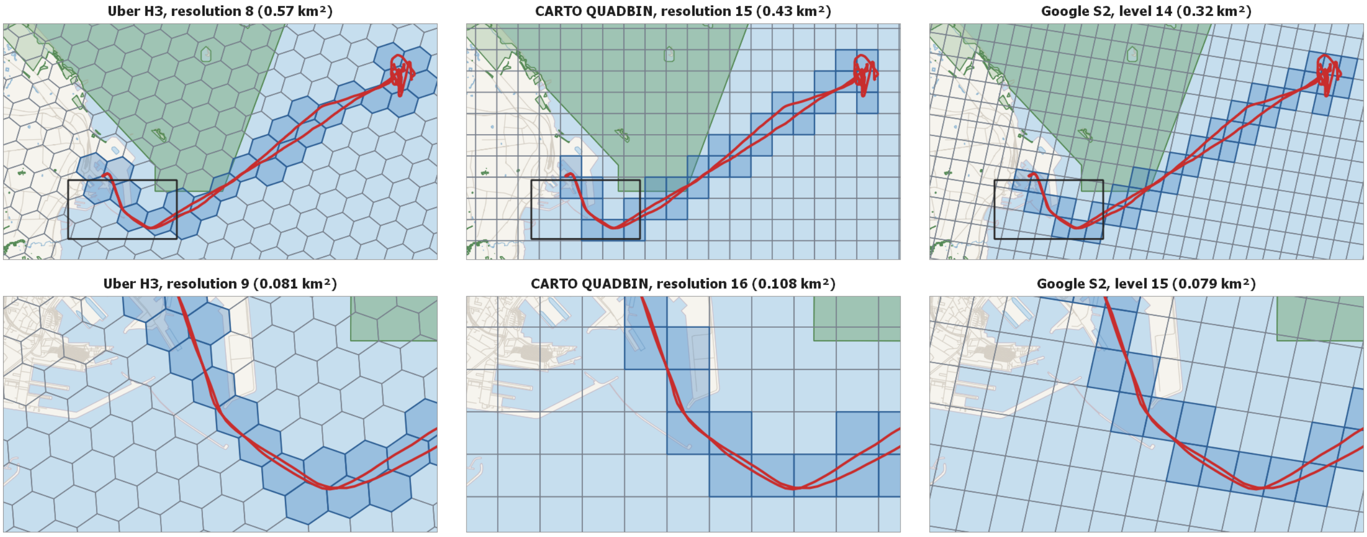
Uber H3, resolution 8 (0.57 km²)
CARTO QUADBIN, resolution 15 (0.43 km²)
Google S2, level 14 (0.32 km²)
Uber H3, resolution 9 (0.081 km²)
CARTO QUADBIN, resolution 16 (0.108 km²)
Google S2, level 15 (0.079 km²)

Figure 16.1: An AIS vessel track leaving Frederikshavn on the three grids, with the cells the track passes through shaded and a marine protected area in green. The upper row states the whole track at the resolutions whose cells are closest in area and outlines in black the excerpt the lower row states, one resolution finer in each grid. Vessel track and protected areas from the Danish AIS data; basemap © OpenStreetMap contributors.

as the text input of the type checks its string: an S2 value is a cell of its grid, a QUADBIN value a well-formed index of any mode, and an H3 value a cell, a directed edge or a vertex.

As with other temporal types, a cell index type has four subtypes: `Instant`, discrete sequence, continuous sequence with step interpolation, and `SequenceSet`. Cell identifiers are discrete, so linear interpolation between two of them has no meaning and step interpolation is always used.

A raster is a tessellation of its own: a grid of pixels laid over a footprint, whose cell is a pixel. It stands in this chapter beside the three global grids, and differs from them in what a cell carries. An H3, QUADBIN or S2 cell carries identity, so what a trajectory over one of them makes is a temporal cell index; a raster cell carries a value, so what a trajectory read against a raster makes is a `tfloat` of the values it passes over, and there is no temporal raster type. The sections below therefore state a raster wherever it answers the operation a cell answers, and state the reading of one along a trip on its own.

The PostGIS raster extension stores gridded coverage data (elevation, temperature, satellite imagery, . . .) in the `raster` type. Each raster has one or more bands; a band holds a 2D array of pixel values, a spatial extent, a pixel size, and an optional nodata sentinel value.

Raquet is a cloud-native raster format that stores raster tiles as rows in an Apache Parquet file. Each row holds the pixel bytes for one Web-Mercator tile together with its QUADBIN cell identifier, a 64-bit integer that encodes the tile's zoom level and Morton-interleaved x/y coordinates. No separate spatial metadata is required: the bounding box and pixel-to-coordinate mapping are fully determined by the QUADBIN value. The specification names that column `block` and keeps one more row, at `block` = 0, whose `metadata` column states in JSON the tile size, the type and nodata value of each band, and the compression of the band bytes.

The two differ in what a value carries. A `raquet` value holds its pixels and its QUADBIN cell in one value, so its extent and its pixel grid follow from the cell and it answers the Web-Mercator grid alone. A PostGIS `raster` states an arbitrary spatial reference system and an arbitrary affine georeferencing, and it is the one the processing functions of Section 16.24 take: clip, transform, rescale, reclass, summaryStats and dumpAsPolygons. MEOS computes both, so neither needs PostgreSQL to evaluate.

A multi-spectral coverage is read band by band on either path. The RaQuet specification gives each band of a tile a column of its own, `band_1`, `band_2` and so on, so such a coverage is a set of single-band tiles sharing one QUADBIN cell and rasterTileValue samples the band it is given, while on the PostGIS `raster` path the `band` argument names it. What a `raquet` value does not carry is the interleaved layout the specification offers for RGB imagery, which packs the bands of a tile into a single JPEG- or WebP-compressed column.

16.1 Notation

Most functions and operators for temporal types described in the previous chapters can be applied to temporal cell index types. Therefore, in the signatures of the functions, the notation `base` represents a `cell` and the notation `ttype` also represents a `tcell`. To avoid redundancy, we present next only the functions and operators specific to the cell index types.

- `cell` represents any cell index type, that is, `h3index`, `quadbin`, or `s2cell`, and in the position of a function name it is the constructor of that type,
- `cellset` represents any cell index set type, that is, `h3indexset`, `quadbinset`, or `s2cellset`,
- `tcell` represents any temporal cell index type, that is, `th3index`, `tquadbin`, or `ts2cell`,
- `geo` and `tgeo` represent the geometry type a grid answers in, planar for QUADBIN and geodetic for H3 and S2.
- `raster` is the PostGIS raster type and `raquet` a single Web-Mercator raster tile; neither is a cell type, so an operation that reads one names it.

16.2 Input and Output

A temporal cell value is entered using the standard temporal syntax with the underlying 64-bit cell identifier. The input parser accepts the canonical hexadecimal form each grid publishes — the form the H3 CLI and `h3_cell_to_string` emit, and the form the CARTO reference implementation reads — with an optional `0x` prefix, in either letter case, and at most 16 significant digits. Hexadecimal is idiomatic in all three ecosystems and is what the output function emits, so every example below uses it. The value must also denote something its grid indexes: for H3 a cell, a directed edge or a vertex, so that a well-formed hexadecimal literal such as `12345`, which is none of the three, is rejected rather than stored as a value with no location. For reference, the hexadecimal `480fffffffffffff` used in the examples is the QUADBIN resolution-0 world cell, equal to the decimal `5192650370358181887`.

The functions `asBinary` and `asHexWKB` serialize a `raster` or a `raquet` value, and rasterFromBinary, rasterFromHexWKB, raquetFromBinary and raquetFromHexWKB reconstruct it. A raster is written in the form PostGIS raster writes, so a raster column round-trips between PostgreSQL and any program reading that form. A tile carries its pixels and its QUADBIN georeferencing in one value, so it round-trips through a portable byte string with no spatial extension involved, and the hexadecimal form lets a tile column round-trip through a text-based `COPY`. Both output functions accept an optional endianness argument, `NDR` for little-endian or `XDR` for big-endian, defaulting to the endianness of the server.

- Input and output of a temporal cell in its text form

  ```
  tcell(text) → tcell
  text(tcell) → text
  ```

  The companion static type `cell` holds a single (non-temporal) cell identifier and `cellset` a finite set of them; they share the temporal type's hexadecimal / decimal textual representation, carry the standard equality and ordering operators through a btree and a hash operator class, and expose the validity predicates below. A type modifier constrains a column to one subtype, which is `Instant`, or `Sequence`, or `SequenceSet`.

  ```
  SELECT th3index '831c02fffffffff@2001-01-01';
  -- 831c02fffffffff@2001-01-01
  SELECT th3index '[831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02]';
  -- [831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02]
  SELECT th3index '{[831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02]}';
  -- {[831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02]}
  SELECT tquadbin '48a6227affffffff@2001-01-01';
  -- 48a6227affffffff@2001-01-01
  SELECT tquadbin '[48a6227affffffff@2001-01-01, 485623ffffffffff@2001-01-02]';
  -- [48a6227affffffff@2001-01-01, 485623ffffffffff@2001-01-02]
  SELECT tquadbin '{[48a6227affffffff@2001-01-01, 485623ffffffffff@2001-01-02]}';
  -- {[48a6227affffffff@2001-01-01, 485623ffffffffff@2001-01-02]}
  ```

```
SELECT ts2cell '47c3c@2001-01-01';
-- 47c3c@2001-01-01
SELECT ts2cell '[47c3c@2001-01-01, 47c4@2001-01-02]';
-- [47c3c@2001-01-01, 47c4@2001-01-02]
SELECT ts2cell '{[47c3c@2001-01-01, 47c4@2001-01-02]}';
-- {[47c3c@2001-01-01, 47c4@2001-01-02]}
```

- Serialize a temporal cell as text, as binary, or as hexadecimal well-known binary, with or without its SRID, or as MF-JSON

```
asText(tcell) → text
asEWKT(tcell) → text
asBinary(tcell,endian text='') → bytea
asEWKB(tcell,endian text='') → bytea
asHexWKB(tcell,endian text='') → text
asHexEWKB(tcell,endian text='') → text
asMFJSON(tcell) → text
```

The extended forms `asEWKT`, `asEWKB` and `asHexEWKB` state the SRID 4326 of a cell, which the other forms omit. The hexadecimal well-known binary form lets a temporal cell column round-trip through a text-based `COPY`. In MF-JSON each cell of the `values` array is its text form in a JSON string, the hexadecimal index of H3 and QUADBIN and the token of S2, since a JSON reader holding its numbers as double precision numbers loses the low bits of a 64-bit cell identifier, and the `bbox` member states the spatiotemporal box of the cells.

```
SELECT asEWKT(th3index '831c02fffffffff@2001-01-01');
-- SRID=4326;831c02fffffffff@2001-01-01
SELECT asHexEWKB(th3index '831c02fffffffff@2001-01-01');
-- 01460061E6100000FFFFFFF2FC031080060885EC91C0000
SELECT asHexWKB(th3index '831c02fffffffff@2001-01-01');
-- 01460021FFFFFFF2FC031080060885EC91C0000
SELECT asHexWKB(tquadbin '48a6227affffffff@2001-01-01');
-- 01490001FFFFFFFF7A22A6480060885EC91C0000
SELECT asHexWKB(ts2cell '47c3c@2001-01-01');
-- 015700210000000000C0C3470060885EC91C0000
SELECT asMFJSON(tquadbin '48a6227affffffff@2001-01-01');
/* {"type":"MovingQuadbin","crs":{"type":"Name","properties":{"name":"EPSG:4326"}},
   "values":["48a6227affffffff"],"datetimes":["2001-01-01T00:00:00+00:00"],
   "interpolation":"None"} */
```

- Reconstruct a temporal cell from its text, binary, hexadecimal, or MF-JSON representation

```
tcellFromText(text) → tcell
tcellFromEWKT(text) → tcell
tcellFromBinary(bytea) → tcell
tcellFromEWKB(bytea) → tcell
tcellFromHexWKB(text) → tcell
tcellFromHexEWKB(text) → tcell
tcellFromMFJSON(text) → tcell
```

An SRID stated by an extended input must be 4326, the SRID of a cell. The MF-JSON input reads a cell written as its text form in a JSON string, and a cell written as the JSON number of its 64-bit identifier.

```
SELECT ts2cellFromText('47c3c@2001-01-01');
-- 47c3c@2001-01-01
SELECT th3indexFromEWKT('SRID=4326;831c02fffffffff@2001-01-01');
-- 831c02fffffffff@2001-01-01
SELECT tquadbinFromEWKB(asEWKB(tquadbin '48a6227affffffff@2001-01-01'));
-- 48a6227affffffff@2001-01-01
SELECT tquadbinFromMFJSON('{"type":"MovingQuadbin","values":["48a6227affffffff"],
  "datetimes":["2001-01-01T00:00:00"],"interpolation":"None"}');
-- 48a6227affffffff@2001-01-01
SELECT tquadbinFromMFJSON('{"type":"MovingQuadbin","values":[5234909528541102079],
  "datetimes":["2001-01-01T00:00:00"],"interpolation":"None"}');
-- 48a6227affffffff@2001-01-01
SELECT th3indexFromHexWKB(asHexWKB(th3index '831c02ffffffffff@2001-01-01'));
-- 831c02ffffffffff@2001-01-01
SELECT tquadbinFromHexWKB(asHexWKB(tquadbin '48a6227affffffff@2001-01-01'));
-- 48a6227affffffff@2001-01-01
SELECT ts2cellFromHexWKB(asHexWKB(ts2cell '47c3c@2001-01-01'));
-- 47c3c@2001-01-01
```

- Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation of a cell

```
asBinary(cell,endian text='') → bytea
asHexWKB(cell,endian text='') → text
```

A cell is written as its 64-bit identifier, encoded using either the little-endian (NDR) or the big-endian (XDR) encoding, the encoding of the machine when none is specified. The SRID 4326 of a cell, which its grid fixes, is omitted.

```
SELECT asBinary(quadbin '48427ffffffffff');
-- \x0100ffffffffff7f4248
SELECT asHexWKB(quadbin '48427ffffffffff', 'XDR');
-- 000048427FFFFFFFFFFF
SELECT asHexWKB(s2cell '47c3c444c');
-- 0100000000C044C4C347
```

- Input of a cell from the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation

```
cellFromBinary(bytea) → cell
cellFromHexWKB(text) → cell
```

An SRID stated by an extended input must be 4326, the SRID of a cell, and the identifier must encode a cell of the grid.

```
SELECT quadbinFromHexWKB('0140E6100000FFFFFFFFFF7F4248');
-- 48427fffffffffff
SELECT quadbinFromHexWKB('0140110F0000FFFFFFFFFF7F4248');
-- ERROR:  Only SRID 4326 (WGS84) is accepted for the type quadbin, received 3857
```

- Return the text representation of a cell

```
asText(cell) → text
```

In the signature above, `cell` replaces `h3index`, `quadbin` or `s2cell`. The text is the hexadecimal index of H3 and QUADBIN and the token of S2, the form the output function emits; for `h3index`, the type of h3-pg, it is the text the output function of h3-pg writes.

```
SELECT asText(h3index '831c02ffffffffff');
-- 831c02ffffffffff
SELECT asText(quadbin '48a6227affffffff');
-- 48a6227affffffff
SELECT asText(s2cell '47c3c');
-- 47c3c
```

- Input from the text representation of a cell

```
cellFromText(text) → cell
```

There is one function per cell type, the name of the function has as prefix the name of the type, which is `h3index`, `quadbin` or `s2cell`. The function is the inverse of `asText` and reads the forms a literal of the type accepts, from a constant or from a `text` column alike.

```
SELECT h3indexFromText('0x831C02FFFFFFFFF');
-- 831c02fffffffff
SELECT quadbinFromText('48a6227affffffff');
-- 48a6227affffffff
SELECT quadbinFromText('0x48a6227affffffff');
-- 48a6227affffffff
SELECT s2cellFromText('47c3c');
-- 47c3c
```

- Return the Well-Known Binary (WKB) representation of a raster or a Raquet tile

```
asBinary({raster,raquet},endian text='') → bytea
```

```
SELECT length(asBinary(ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 4326)));
-- 61
SELECT get_byte(asBinary(ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 4326),
  'XDR'), 0);
-- 0
SELECT length(asBinary(raquet('\x01020304'::bytea, 2, 2,
  quadbin '4817ffffffffffff', 'uint8')));
-- 31
```

- Return the Hexadecimal Well-Known Binary (HexWKB) representation of a raster or a Raquet tile

```
asHexWKB({raster,raquet},endian text='') → text
```

```
SELECT length(asHexWKB(ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 4326)));
-- 122
SELECT length(asHexWKB(raquet('\x01020304'::bytea, 2, 2,
  quadbin '4817ffffffffffff', 'uint8')));
-- 62
```

- Return a raster or a Raquet tile from its Well-Known Binary (WKB) representation

```
rasterFromBinary(bytea) → raster
raquetFromBinary(bytea) → raquet
```

```
SELECT rasterFromBinary(asBinary(r, 'XDR')) = r
FROM (SELECT ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 4326) AS r) t;
-- true
SELECT raquetFromBinary(asBinary(raquet('\x01020304'::bytea, 2, 2,
  quadbin '4817ffffffffffff', 'uint8')))
  = raquet('\x01020304'::bytea, 2, 2, quadbin '4817ffffffffffff', 'uint8');
-- true
```

- Return a raster or a Raquet tile from its Hexadecimal Well-Known Binary (HexWKB) representation

```
rasterFromHexWKB(text) → raster
raquetFromHexWKB(text) → raquet
```

```
SELECT rasterFromHexWKB(asHexWKB(r, 'XDR')) = r
FROM (SELECT ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 4326) AS r) t;
-- true
SELECT raquetFromHexWKB(asHexWKB(raquet('\x01020304'::bytea, 2, 2,
  quadbin '4817ffffffffffff', 'uint8')))
  = raquet('\x01020304'::bytea, 2, 2, quadbin '4817ffffffffffff', 'uint8');
-- true
```

16.3 Constructors

A `raquet` tile is built either from pixel bytes already in hand or by decoding a raster file, and carries its QUADBIN georeferencing in the value itself. A PostGIS `raster` is built likewise from the bytes of the pixels of a band and its georeferencing, or as a grid with no band to which bands are added and whose pixels are then set.

- Construct a temporal cell holding one cell over a time value

```
tcell(cell,timestamptz) → tcellInst
tcell(cell,tstzset) → tcellDiscSeq
tcell(cell,tstzspan) → tcellSeq
tcell(cell,tstzspanset) → tcellSeqSet
```

```
SELECT th3index(h3index '871fa4418ffffff', timestamptz '2001-01-01');
-- 871fa4418ffffff@2001-01-01
SELECT th3index(h3index '871fa4418ffffff', tstzspan '[2001-01-01, 2001-01-03]');
-- [871fa4418ffffff@2001-01-01, 871fa4418ffffff@2001-01-03]
SELECT tquadbin(quadbin '48a6227affffffff', timestamptz '2001-01-01');
-- 48a6227affffffff@2001-01-01
SELECT tquadbin(quadbin '48a6227affffffff', tstzspan '[2001-01-01, 2001-01-03]');
-- [48a6227affffffff@2001-01-01, 48a6227affffffff@2001-01-03]
SELECT ts2cell(s2cell '47c3c', timestamptz '2001-01-01');
-- 47c3c@2001-01-01
SELECT ts2cell(s2cell '47c3c', tstzspan '[2001-01-01, 2001-01-03]');
-- [47c3c@2001-01-01, 47c3c@2001-01-03]
```

- Construct a temporal cell of a given subtype from its components

```
tcellInst(tcell) → tcellInst
tcellSeq(tcell[],interp text='step') → tcellSeq
tcellSeqSet(tcell[]) → tcellSeqSet
```

```
SELECT th3indexSeq(ARRAY[th3index '871fa4418ffffff@2001-01-01',
  th3index '871fa44a8ffffff@2001-01-02'], 'step');
-- [871fa4418ffffff@2001-01-01, 871fa44a8ffffff@2001-01-02]
SELECT tquadbinSeq(ARRAY[tquadbin '48a6227affffffff@2001-01-01',
  tquadbin '485623ffffffffff@2001-01-02'], 'step');
-- [48a6227affffffff@2001-01-01, 485623ffffffffff@2001-01-02]
SELECT ts2cellSeq(ARRAY[ts2cell '47c3c@2001-01-01', ts2cell '47c4@2001-01-02'], 'step');
-- [47c3c@2001-01-01, 47c4@2001-01-02]
```

- Construct a Raquet raster tile from its pixel bytes, dimensions, QUADBIN cell and pixel type

```
raquet(bytea,width integer,height integer,quadbin,pixtype text,
  nodata float) → raquet
```

  Packages a row-major single-band `pixels` array of `width` × `height` pixels of type `pixtype` (one of `uint8`, `int8`, `uint16`, `int16`, `uint32`, `int32`, `uint64`, `int64`, `float16`, `float32`, or `float64`), georeferenced by the `quadbin` cell, into a single self-describing `raquet` value. The `nodata` argument is optional; when omitted (`NULL`) the tile carries no nodata sentinel. An error is raised when the `pixels` array is smaller than `width` × `height` × the pixel size. Pixels of more than one byte are little-endian, which is the byte order the RaQuet specification gives them whatever the byte order of the server, so that a tile keeps its values when it is read on another machine. A `raquet` is a GDAL-free, variable-length value with a portable Hex-WKB text and binary representation, distinct from and coexisting with the PostGIS `raster` type used by rasterValue. A Raquet table states the dimensions, the pixel type and the nodata value of its tiles once, in the metadata row at `block` = 0, which the query below reads and leaves out of the tiles: 0 is no QUADBIN index, so the cast to `quadbin` refuses it. The constructor reads the pixel bytes as they stand, so the bands of a Raquet file whose metadata states a `gzip` compression are decompressed before they reach it.

```
-- Package the tiles of a Raquet table, whose band_1 column is uncompressed,
-- into raquet values keyed by their QUADBIN cell
CREATE TABLE elevation_tiles AS
SELECT r.block::quadbin AS quadbin, raquet(r.band_1,
    (m.meta->'tiling'->>'block_width')::integer,
    (m.meta->'tiling'->>'block_height')::integer, r.block::quadbin,
    m.meta->'bands'->0->>'type', (m.meta->'bands'->0->>'nodata')::float8) AS tile
FROM elevation_raquet r,
  (SELECT metadata::jsonb AS meta FROM elevation_raquet WHERE block = 0 LIMIT 1) m
WHERE r.block <> 0;
-- A 2 x 2 UINT8 tile with no nodata
SELECT raquet('\x01020304'::bytea, 2, 2, quadbin '4817ffffffffffff', 'uint8');
```

- Construct a raster of one band from the bytes of its pixels and its georeferencing

```
raster(pixels bytea,width integer,height integer,upperleftx float,upperlefty float,
  scalex float,scaley float,skewx float,skewy float,srid integer,pixeltype text,
  nodataval float=NULL) → raster
```

The twin of the raquet constructor for a PostGIS `raster`, the georeferencing of a raster standing for the QUADBIN cell of a tile. The `pixels` are those pixels answers: row by row, each in the bytes of its type, little-endian when wider than one byte, and one byte for the types `1BB`, `2BUI` and `4BUI`, which bound a pixel to less than a byte. The `pixeltype` is named as PostGIS names it, `8BUI`, `16BSI`, `32BF` and so on, or as bandPixelType answers it, the name `uint8` standing for `8BUI`. A raster band has no 16-bit float and no 64-bit integer, so `float16`, `int64` and `uint64` are refused, as are bytes of another number than the pixels take and a byte beyond the bound of a type bounding a pixel to less than a byte. The `nodataval` is optional; when omitted the band states no nodata value. A raster of one band is the raster built from its own pixels and georeferencing, `raster(pixels(r), width(r), height(r), upperLeftX(r), upperLeftY(r), scaleX(r), scaleY(r), skewX(r), skewY(r), SRID(r), bandPixelType(r), bandNoDataValue(r))`, every value it holds kept exactly. A band of `1BB`, `2BUI` or `4BUI` holds a byte a pixel, as one of `8BUI` does, and bandPixelType names it `uint8`, so it is rebuilt as `8BUI` with the same bytes, the bound its name states being the one thing it loses. Further bands are added with addBand.

```
SELECT dumpValues(raster('\x0100feff2c010080'::bytea, 2, 2, 0, 2, 1, -1, 0, 0,
  3857, '16BSI'), 1);
-- {{1,-2},{300,-32768}}
```

- Construct a raster with no band

```
makeEmptyRaster(width integer,height integer,upperleftx float,upperlefty float,
  scalex float,scaley float,skewx float,skewy float,srid integer=0) → raster
```

The raster PostGIS's `ST_MakeEmptyRaster` answers: a grid of `width` × `height` pixels, each dimension from 0 to 65535, georeferenced by its upper left corner, its scale and its skew in the spatial reference system `srid`.

```
SELECT width(r), height(r), numBands(r), scaleX(r)
FROM (SELECT makeEmptyRaster(3, 2, 10, 20, 2, -1, 0, 0, 3857) AS r) t;
-- 3 | 2 | 0 | 2
```

- Add a band to a raster

```
addBand(rast raster,index integer,pixeltype text,initialvalue float=0,
  nodataval float=NULL) → raster
addBand(torast raster,fromrast raster,fromband integer=1,
  torastindex integer=NULL) → raster
```

The raster PostGIS's `ST_AddBand` answers. The first form adds a band of type `pixeltype`, named as raster reads it, each of its pixels holding `initialvalue`, stating the nodata value `nodataval` or none when it is omitted. The second form adds a copy of band `fromband` of `fromrast`, a raster of the same width and height. The band takes the number `index` or `torastindex`, from 1 to one more than the number of bands of the raster, and is added after the last band when the number is `NULL`, written `NULL::integer` in the first form so that it is not read as a raster of the second. A number out of that range, a band `fromrast` lacks and a raster of another size are refused, where PostGIS adds the band elsewhere or answers the raster unchanged.

```
SELECT numBands(r), bandPixelType(r, 1), bandNoDataValue(r, 2)
FROM (SELECT addBand(addBand(makeEmptyRaster(2, 2, 0, 2, 1, -1, 0, 0, 3857),
  NULL::integer, '8BUI', 1), 1, '32BF', 0, -9999) AS r) t;
-- 2 | float32 | NULL
SELECT numBands(addBand(r, r))
FROM (SELECT addBand(makeEmptyRaster(2, 2, 0, 2, 1, -1, 0, 0, 3857),
  NULL::integer, '8BUI', 1) AS r) t;
-- 2
```

- Decode a raster file into a Raquet raster tile via GDAL

```
raquetRead(bytea,quadbin) → raquet
raquetRead(text,quadbin) → raquet
```

Reads the bytes of `rasterfile`, a raster in any format GDAL supports (GeoTIFF, PNG, . . .), through GDAL's `/vsimem/` virtual filesystem, and packs its first band, row-major, into a single self-describing `raquet` value georeferenced by the `quadbin` cell. The band data type must be one of `Byte`, `Int16`, `Int32`, `Float32`, or `Float64`; the band nodata value, when set, is carried into the tile. Because the file is supplied as bytes, no server-side file access is required. GDAL performs the decode, so the raster's format driver must be permitted by PostGIS's `postgis.gdal_enabled_drivers` setting (which disables all drivers by default); enable it with, for example, `SET postgis.gdal_enabled_drivers = 'ENABLE_ALL'`. This is the ingest counterpart of the raquet constructor, which builds a tile from already-decoded pixel bytes. When `quadbin` is omitted or `NULL`, the tile identifier is derived from the raster's georeferencing: the raster must carry an `EPSG:3857` (Web-Mercator) spatial reference and an axis-aligned geotransform describing a single QUADBIN tile. A raster with no spatial reference, one that is not in `EPSG:3857`, or a geotransform that is rotated or not aligned to the tile grid raises an error rather than being coerced. The form taking `text` reads the raster file at that path on the server instead, where PostGIS allows a raster band stored outside the database to be read, as rasterValue states.

```
-- Decode a GeoTIFF stored in a bytea column into a raquet tile
SELECT raquetRead(file_bytes, quadbin) AS tile FROM elevation_files;
-- Derive the QUADBIN cell from an EPSG:3857 GeoTIFF's georeferencing
SELECT raquetRead(file_bytes) AS tile FROM elevation_files;
```

16.4 Conversions

A `tcell` value shares the same on-disk representation as `tbigint`, and a cell the same representation as `bigint`. A cast between them is an assignment cast: it applies in an assignment, such as inserting a `tbigint` into a `tcell` column, and elsewhere a query states `::`, so cell-specific functions cannot be silently called on arbitrary integers.

A tile converts to the spatiotemporal box of its footprint, which is what gives a table of tiles a spatial index.

- Convert between a `tcell` and a `tbigint`

```
{bigint,quadbin}::{quadbin,bigint}
{bigint,s2cell}::{s2cell,bigint}
{tbigint,tcell}::{tcell,tbigint}
```

The cast out of a cell type is a binary coercion, since every cell is an integer. The cast into a cell type checks every value as the text input of the type does and raises an error otherwise, as the MF-JSON and WKB input of the type do: an S2 value must be a cell of its grid, a QUADBIN value a well-formed index of any mode, as the `quadbin` type holds, and an H3 value a cell, a directed edge or a vertex, since an H3 index names all three. The `h3index` type and its casts to and from `bigint` are those of the H3 extension. The cast round trips: casting a cell to an integer and back answers the cell it started from.

```
SELECT (tbigint '590464338553208831@2001-01-01')::th3index;
-- 831c02fffffffff@2001-01-01
SELECT (tbigint '5234909528541102079@2001-01-01')::tquadbin;
-- 48a6227affffffff@2001-01-01
SELECT (tbigint '5171187903383994368@2001-01-01')::ts2cell;
-- 47c3c@2001-01-01
SELECT 5234909528541102079::bigint::quadbin;
-- 48a6227affffffff
SELECT (tbigint '42@2001-01-01')::tquadbin;
-- ERROR:  The value 42 does not encode a valid quadbin index
```

- Cast a temporal cell to a temporal geodetic or planar point of cell centroids

  ```
  {th3index,tquadbin,ts2cell}::{tgeogpoint,tgeompoint}
  ```

  The conversions follow the rule of [?listitem].

  The cast of a QUADBIN cell answers a point of the Web-Mercator plane the grid is defined on, while the cast of an S2 cell answers a geodetic one; each grid declares the single cast its own definition calls for. H3 declares both, so it is the one grid where the caller chooses between the geodetic and the SRID-4326 planar representation.

  ```
  SELECT asText((th3index '831c02fffffffff@2001-01-01')::tgeogpoint, 6);
  -- POINT(-144.523991 49.716503)@2001-01-01
  SELECT asText((th3index '831c02fffffffff@2001-01-01')::tgeompoint, 6);
  -- POINT(-144.523991 49.716503)@2001-01-01
  SELECT asText((tquadbin '48a6227affffffff@2001-01-01')::tgeompoint, 6);
  -- POINT(4.394531 50.847573)@2001-01-01
  SELECT asText((ts2cell '47c3c@2001-01-01')::tgeogpoint, 6);
  -- POINT(4.222019 50.936164)@2001-01-01
  ```

- Convert a Raquet tile into a spatiotemporal box

  ```
  stbox(raquet) → stbox
  ```

  Returns the tile footprint: the longitude and latitude envelope of its QUADBIN cell, in SRID 4326 and without a time dimension. The footprint follows from the cell alone, so tiles differing in pixels, dimensions or pixel type share it. The conversion is also available as a cast.

  ```
  -- The footprint of a tile covering the north-eastern quadrant at zoom 1
  SELECT round(stbox(raquet('\x01020304'::bytea, 2, 2, quadbin '4817ffffffffffff',
    'uint8')), 6);
  -- SRID=4326;STBOX X((0,0),(180,85.051129))
  -- Keep the tiles whose footprint overlaps a region of interest
  SELECT * FROM elevation_tiles WHERE tile::stbox && stbox 'SRID=4326;
    STBOX X((0,0),(30,30))';
  ```

  The footprint carries the `stbox` operators, aggregates and operator classes, so a tile table is searched by spatial overlap at any zoom level rather than by an equality test on the QUADBIN cell at a single fixed one. Indexing the conversion gives the tile table a spatial index, and the extent of a set of tiles is the extent of their footprints.

  ```
  -- A spatial index over the tile footprints
  CREATE INDEX elevation_tiles_gist ON elevation_tiles USING gist (stbox(tile));
  CREATE INDEX elevation_tiles_spgist ON elevation_tiles USING spgist (stbox(tile));
  -- The extent covered by a set of tiles
  SELECT extent(stbox(tile)) FROM elevation_tiles;
  -- Tiles a trajectory passes through, at whatever zoom levels the table holds
  SELECT t.tripid, r.tile FROM trips t,
    elevation_tiles r WHERE stbox(r.tile) && stbox(t.trip);
  ```

- Convert a raster into a spatiotemporal box

```
stbox(raster) → stbox
```

Returns the extent of the raster in the reference system the raster itself states, without a time dimension. This is the one shape of a PostGIS `raster` the extension states, because PostGIS has no `stbox`: the width, height, georeferencing and band properties are read with PostGIS's own accessors, which are not restated here. The conversion is also available as a cast.

```
WITH rast AS (
  SELECT ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0, 4326) AS r
)
SELECT stbox(r) AS box, stbox(r) = r::stbox AS cast_agrees FROM rast;
-- SRID=4326;STBOX X((0,0),(3,3)) | t
-- The box states the reference system of the raster, not a fixed one
SELECT stbox(ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 3857));
-- SRID=3857;STBOX X((0,0),(2,2))
```

The extent bears the rotation of the geotransform, so a skewed raster answers the box containing its four rotated corners rather than the one its scale alone would give.

```
WITH plain AS (
  SELECT ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 4326) AS r
), skewed AS (
  SELECT ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 1.0, 0.0, 4326) AS r
)
SELECT stbox((SELECT r FROM plain)) AS plain_box,
  stbox((SELECT r FROM skewed)) AS skewed_box;
-- SRID=4326;STBOX X((0,0),(2,2)) | SRID=4326;STBOX X((0,0),(4,2))
```

The extent carries the `stbox` operators, aggregates and operator classes, so a raster table is searched by spatial overlap and indexed on the conversion.

```
SELECT stbox(ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0, 4326)) &&
  stbox 'SRID=4326;STBOX X((1,1),(2,2))';
-- t
```

16.5 Accessors

numBands, SRID and the corner, scale and skew accessors read the grid of a PostGIS `raster`; width, height and the band accessors read a raster and a `raquet` tile alike, and the other accessors read a `raquet` tile.

A `raquet` value is self-describing: the accessors below read back the georeferencing and layout it carries, so a tile packaged with the raquet constructor or decoded with raquetRead can be inspected without keeping the loose columns it was built from.

- Return the first, the last, or the nth distinct cell identifier of a temporal value

```
startValue(tcell) → cell
endValue(tcell) → cell
valueN(tcell,integer) → cell
```

`valueN` counts from one and returns `NULL` when `n` is out of range.

```
SELECT startValue(th3index '831c02fffffffff@2001-01-01');
-- 831c02fffffffff
SELECT endValue(th3index '{831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02}');
-- 831c00fffffffff
SELECT valueN(th3index '{831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02}', 1);
-- 831c02fffffffff
SELECT startValue(tquadbin '48a6227affffffff@2001-01-01');
-- 48a6227affffffff
SELECT endValue(tquadbin '{48a6227affffffff@2001-01-01, 485623ffffffffff@2001-01-02}');
```

```
-- 485623fffffffff
SELECT valueN(tquadbin '{48a6227affffffff@2001-01-01, 485623fffffffff@2001-01-02}', 1);
-- 48a6227affffffff
SELECT startValue(ts2cell '47c3c@2001-01-01');
-- 47c3c
SELECT endValue(ts2cell '{47c3c@2001-01-01, 47c4@2001-01-02}');
-- 47c4
SELECT valueN(ts2cell '{47c3c@2001-01-01, 47c4@2001-01-02}', 1);
-- 47c3c
```

- Return the set of distinct cell identifiers the trajectory takes

  ```
  getValues(tcell) → cellset
  ```

  ```
  SELECT getValues(th3index '831c02ffffffff@2001-01-01');
  -- {"831c02ffffffff"}
  SELECT getValues(th3index '{831c02ffffffff@2001-01-01, 831c00ffffffff@2001-01-02}');
  -- {"831c00ffffffff", "831c02ffffffff"}
  SELECT getValues(tquadbin '48a6227affffffff@2001-01-01');
  -- {"48a6227affffffff"}
  SELECT getValues(tquadbin '{48a6227affffffff@2001-01-01, 485623fffffffff@2001-01-02}');
  -- {"485623fffffffff", "48a6227affffffff"}
  SELECT getValues(ts2cell '47c3c@2001-01-01');
  -- {"47c3c"}
  SELECT getValues(ts2cell '{47c3c@2001-01-01, 47c4@2001-01-02}');
  -- {"47c3c", "47c4"}
  ```

- Return the cell identifier at a given timestamp, using step interpolation

  ```
  valueAtTimestamp(tcell,timestamptz) → cell
  ```

  ```
  SELECT valueAtTimestamp(th3index
    '[831c02ffffffff@2001-01-01, 831c00ffffffff@2001-01-05]', '2001-01-03');
  -- 831c02ffffffff
  SELECT valueAtTimestamp(tquadbin
    '[48a6227affffffff@2001-01-01, 485623fffffffff@2001-01-05]', '2001-01-03');
  -- 48a6227affffffff
  SELECT valueAtTimestamp(ts2cell
    '[47c3c@2001-01-01, 47c4@2001-01-05]', '2001-01-03');
  -- 47c3c
  ```

- Return the number of bands of a raster

  ```
  numBands(raster) → integer
  ```

  The band number accepted by rasterValue and by the operators built on it ranges from 1 to this value.

  ```
  WITH rast1 AS (
    SELECT ST_AddBand(ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0, 4326),
      '32BF'::text, 0.0::float8, NULL::float8 ) AS r ), rast2 AS (
    SELECT ST_AddBand(r, '32BF'::text, 0.0::float8, NULL::float8) AS r FROM rast1 )
  SELECT numBands((SELECT r FROM rast1)) AS num_bands_one,
    numBands((SELECT r FROM rast2)) AS num_bands_two;
  -- 1 | 2
  ```

- Return the spatial reference system identifier of a raster

  ```
  SRID(raster) → integer
  ```

```
SELECT SRID(ST_MakeEmptyRaster(3, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 3857));
-- 3857
```

- Return the X or the Y coordinate of the upper left corner of a raster

  ```
  upperLeftX(raster) → float
  upperLeftY(raster) → float
  ```

  The corner is the origin the geotransform states, which the skew rotates the grid about, so it is a corner of the extent only when both skews are zero.

  ```
  SELECT upperLeftX(r), upperLeftY(r)
  FROM (SELECT ST_MakeEmptyRaster(3, 2, 10.0, 20.0, 1.5, -2.0, 0.5, 0.25, 3857) AS r) t;
  -- 10 | 20
  ```

- Return the pixel width or the pixel height of a raster, that is, the X or the Y component of its scale

  ```
  scaleX(raster) → float
  scaleY(raster) → float
  ```

  The pixel height is negative for a grid whose rows run north to south, which is how a raster is ordinarily written.

  ```
  SELECT scaleX(r), scaleY(r)
  FROM (SELECT ST_MakeEmptyRaster(3, 2, 10.0, 20.0, 1.5, -2.0, 0.5, 0.25, 3857) AS r) t;
  -- 1.5 | -2
  ```

- Return the X or the Y component of the skew of a raster

  ```
  skewX(raster) → float
  skewY(raster) → float
  ```

  A raster whose two skews are zero is axis-aligned.

  ```
  SELECT skewX(r), skewY(r)
  FROM (SELECT ST_MakeEmptyRaster(3, 2, 10.0, 20.0, 1.5, -2.0, 0.5, 0.25, 3857) AS r) t;
  -- 0.5 | 0.25
  ```

- Return the QUADBIN cell of a Raquet tile

  ```
  quadbin(raquet) → quadbin
  ```

  ```
  SELECT quadbin(raquet('\x01020304'::bytea, 2, 2, quadbin '4817ffffffffffff', 'uint8'));
  -- 4817ffffffffffff
  ```

- Return the width in pixels of a raster or a Raquet tile

  ```
  width({raster,raquet}) → integer
  ```

  ```
  SELECT width(ST_MakeEmptyRaster(3, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 4326));
  -- 3
  SELECT width(raquet('\x01020304'::bytea, 2, 2, quadbin '4817ffffffffffff', 'uint8'));
  -- 2
  ```

- Return the height in pixels of a raster or a Raquet tile

  ```
  height({raster,raquet}) → integer
  ```

```
SELECT height(ST_MakeEmptyRaster(3, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 4326));
-- 2
SELECT height(raquet('\x01020304'::bytea, 2, 2, quadbin '4817ffffffffffff', 'uint8'));
-- 2
```

- Return the name of the pixel data type of a raster band or of a Raquet tile

```
bandPixelType(raster,band integer=1) → text
bandPixelType(raquet) → text
```

The returned name is the one the constructors accept, that is, one of `uint8`, `int8`, `uint16`, `int16`, `uint32`, `int32`, `uint64`, `int64`, `float16`, `float32`, or `float64`. The constructors read the name without regard to case, so the name PostGIS raster gives a pixel type carries into them as it stands; the name returned here is the lower-case spelling the RaQuet specification gives a tile's `type` field, for a PostGIS band as for a tile, so a band and a tile of the same type report the same name.

```
SELECT bandPixelType(ST_AddBand(ST_MakeEmptyRaster(1, 1, 0, 0, 1), '16BSI'));
-- int16
-- Read back the layout of a packaged tile
SELECT quadbin(tile), width(tile), height(tile), bandPixelType(tile)
FROM (SELECT raquet('\x01020304'::bytea, 2, 2, quadbin '4817ffffffffffff', 'uint8')
  AS tile) t;
-- 4817ffffffffffff | 2 | 2 | uint8
```

- Return whether a raster band or a Raquet tile states a nodata value

```
bandHasNoDataValue(raster,band integer=1) → boolean
bandHasNoDataValue(raquet) → boolean
```

A band that states none has no pixel to exclude, so every pixel it holds carries a value.

```
SELECT bandHasNoDataValue(ST_AddBand(ST_MakeEmptyRaster(1, 1, 0, 0, 1), '16BSI'::text,
  0::float8, -9999::float8));
-- true
SELECT bandHasNoDataValue(raquet('\x01020304'::bytea, 2, 2, quadbin '4817ffffffffffff',
  'uint8'));
-- false
```

- Return the nodata value of a raster band or of a Raquet tile

```
bandNoDataValue(raster,band integer=1) → float
bandNoDataValue(raquet) → float
```

A band that states no nodata value answers `NULL`.

```
SELECT bandNoDataValue(ST_AddBand(ST_MakeEmptyRaster(1, 1, 0, 0, 1), '16BSI'::text,
  0::float8, -9999::float8));
-- -9999
SELECT bandNoDataValue(raquet('\x01020304'::bytea, 2, 2, quadbin '4817ffffffffffff',
  'uint8', 255));
-- 255
-- A tile built without a nodata value states none
SELECT bandNoDataValue(raquet('\x01020304'::bytea, 2, 2, quadbin '4817ffffffffffff',
  'uint8'));
-- NULL
```

- Return the pixel bytes of a raster band or of a Raquet tile

```
pixels(rast raster,nband integer=1) → bytea
pixels(raquet) → bytea
```

The bytes are row-major and packed, `width` × `height` pixels of the size of bandPixelType each, little-endian when a pixel is wider than one byte, which is the layout the constructors accept. A tile and a raster band therefore round trip through their own accessors, and every value the band holds is kept exactly, where dumpValues answers its values as double precision numbers. A band of a type bounding a pixel to less than a byte takes a byte a pixel.

```
SELECT pixels(raquet('\x01020304'::bytea, 2, 2, quadbin '4817ffffffffffff', 'uint8'));
-- \x01020304
SELECT pixels(raster('\x0100feff2c010080'::bytea, 2, 2, 0, 2, 1, -1, 0, 0,
  3857, '16BSI'));
-- \x0100feff2c010080
```

16.6 Transformations

- Shift, scale, or shift and scale the time span of a temporal cell

```
shiftTime(tcell,interval) → tcell
scaleTime(tcell,interval) → tcell
shiftScaleTime(tcell,interval,interval) → tcell
```

```
SELECT shiftTime(th3index '871fa4418ffffff@2001-01-01', interval '1 day');
-- 871fa4418ffffff@2001-01-02
SELECT scaleTime(th3index '[871fa4418ffffff@2001-01-01,
  871fa44a8ffffff@2001-01-03]', interval '4 days');
-- [871fa4418ffffff@2001-01-01, 871fa44a8ffffff@2001-01-05]
SELECT shiftTime(tquadbin '48a6227affffffff@2001-01-01', interval '1 day');
-- 48a6227affffffff@2001-01-02
SELECT scaleTime(tquadbin '[48a6227affffffff@2001-01-01,
  485623ffffffffff@2001-01-03]', interval '4 days');
-- [48a6227affffffff@2001-01-01, 485623ffffffffff@2001-01-05]
SELECT shiftTime(ts2cell '47c3c@2001-01-01', interval '1 day');
-- 47c3c@2001-01-02
SELECT scaleTime(ts2cell '[47c3c@2001-01-01, 47c4@2001-01-03]', interval '4 days');
-- [47c3c@2001-01-01, 47c4@2001-01-05]
```

- Append a temporal instant or a temporal sequence

```
appendInstant(tcell,tcellInst) → tcell
appendSequence(tcell,tcellSeq) → tcell
```

```
SELECT asText(appendInstant(th3index '831c02fffffffff@2001-01-01',
  th3index '831c00fffffffff@2001-01-02'));
-- [831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-02]
SELECT asText(appendSequence(th3index '[831c02fffffffff@2001-01-01]',
  th3index '[831c00fffffffff@2001-01-02]'));
-- {[831c02fffffffff@2001-01-01], [831c00fffffffff@2001-01-02]}
SELECT asText(appendInstant(tquadbin '48a6227affffffff@2001-01-01',
  tquadbin '485623ffffffffff@2001-01-02'));
-- [48a6227affffffff@2001-01-01, 485623ffffffffff@2001-01-02]
SELECT asText(appendSequence(tquadbin '[48a6227affffffff@2001-01-01]',
  tquadbin '[485623ffffffffff@2001-01-02]'));
-- {[48a6227affffffff@2001-01-01], [485623ffffffffff@2001-01-02]}
SELECT asText(appendInstant(ts2cell '47c3c@2001-01-01',
  ts2cell '47c4@2001-01-02'));
-- [47c3c@2001-01-01, 47c4@2001-01-02]
SELECT asText(appendSequence(ts2cell '[47c3c@2001-01-01]',
  ts2cell '[47c4@2001-01-02]'));
-- {[47c3c@2001-01-01], [47c4@2001-01-02]}
```

- Merge the temporal cells

```
merge(tcell,tcell) → tcell
merge(tcell[]) → tcell
```

```
SELECT asText(merge(th3index '[831c02fffffffff@2001-01-01, 831c02fffffffff@2001-01-03]',
  th3index '[831c02fffffffff@2001-01-03, 831c00fffffffff@2001-01-05]'));
-- [831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-05]
SELECT asText(merge(ARRAY[th3index '831c02fffffffff@2001-01-01',
  th3index '831c00fffffffff@2001-01-03']));
-- {831c02fffffffff@2001-01-01, 831c00fffffffff@2001-01-03}
SELECT asText(merge(tquadbin '[48a6227affffffff@2001-01-01, 48a6227affffffff@2001-01-03]',
  tquadbin '[48a6227affffffff@2001-01-03, 485623ffffffffff@2001-01-05]'));
-- [48a6227affffffff@2001-01-01, 485623ffffffffff@2001-01-05]
SELECT asText(merge(ARRAY[tquadbin '48a6227affffffff@2001-01-01',
  tquadbin '485623ffffffffff@2001-01-03']));
-- {48a6227affffffff@2001-01-01, 485623ffffffffff@2001-01-03}
SELECT asText(merge(ts2cell '[47c3c@2001-01-01, 47c3c@2001-01-03]',
  ts2cell '[47c3c@2001-01-03, 47c4@2001-01-05]'));
-- [47c3c@2001-01-01, 47c4@2001-01-05]
SELECT asText(merge(ARRAY[ts2cell '47c3c@2001-01-01',
  ts2cell '47c4@2001-01-03']));
-- {47c3c@2001-01-01, 47c4@2001-01-03}
```

- Set the interpolation of a temporal cell

```
setInterp(tcell,interp text) → tcell
```

```
SELECT setInterp(th3index '{871fa4418ffffff@2001-01-01,
  871fa44a8ffffff@2001-01-02}', 'step');
-- {[871fa4418ffffff@2001-01-01], [871fa44a8ffffff@2001-01-02]}
SELECT setInterp(tquadbin '{48a6227affffffff@2001-01-01,
  485623ffffffffff@2001-01-02}', 'step');
-- {[48a6227affffffff@2001-01-01], [485623ffffffffff@2001-01-02]}
SELECT setInterp(ts2cell '{47c3c@2001-01-01,
  47c4@2001-01-02}', 'step');
-- {[47c3c@2001-01-01], [47c4@2001-01-02]}
```

16.7 Modifications

- Insert a temporal cell into another one, or update one with another

```
insert(tcell,tcell,connect boolean=true) → tcell
update(tcell,tcell,connect boolean=true) → tcell
```

```
SELECT insert(th3index '[871fa4418ffffff@2001-01-01,
  871fa4418ffffff@2001-01-02]', th3index '[871fa44a8ffffff@2001-01-03,
  871fa44a8ffffff@2001-01-04]');
-- [871fa4418ffffff@2001-01-01, 871fa44a8ffffff@2001-01-03, 871fa44a8ffffff@2001-01-04]
SELECT insert(tquadbin '[48a6227affffffff@2001-01-01,
  48a6227affffffff@2001-01-02]', tquadbin '[485623ffffffffff@2001-01-03,
  485623ffffffffff@2001-01-04]');
/* [48a6227affffffff@2001-01-01, 485623ffffffffff@2001-01-03,
   485623ffffffffff@2001-01-04] */
SELECT insert(ts2cell '[47c3c@2001-01-01,
  47c3c@2001-01-02]', ts2cell '[47c4@2001-01-03,
  47c4@2001-01-04]');
-- [47c3c@2001-01-01, 47c4@2001-01-03, 47c4@2001-01-04]
```

- Delete a time value from a temporal cell

```
deleteTime(tcell,times,connect boolean=true) → tcell
```

```
SELECT deleteTime(th3index '[871fa4418ffffff@2001-01-01,
  871fa4418ffffff@2001-01-04]', tstzspan '[2001-01-02, 2001-01-03]');
-- [871fa4418ffffff@2001-01-01, 871fa4418ffffff@2001-01-04]
SELECT deleteTime(tquadbin '[48a6227affffffff@2001-01-01,
  48a6227affffffff@2001-01-04]', tstzspan '[2001-01-02, 2001-01-03]');
-- [48a6227affffffff@2001-01-01, 48a6227affffffff@2001-01-04]
SELECT deleteTime(ts2cell '[47c3c@2001-01-01,
  47c3c@2001-01-04]', tstzspan '[2001-01-02, 2001-01-03]');
-- [47c3c@2001-01-01, 47c3c@2001-01-04]
```

- Set the values of a block of pixels of a raster band

```
setValues(rast raster,nband integer,x integer,y integer,newvalueset float[],
  noset boolean[]=NULL,keepnodata boolean=false) → raster
```

The raster PostGIS's `ST_SetValues` answers: band `nband` takes the values of `newvalueset`, an array of one row or of rows of the same length, over the block of pixels whose upper left pixel is at column `x` and row `y`, both starting at 1. A `NULL` value sets the pixel to the nodata value of the band, or to the least value of its type when the band states none. A pixel whose flag in `noset` is true is left as it is, the flags read over the block they share with the values, and so is a pixel holding the nodata value when `keepnodata` is true. The pixels of the block beyond the raster are left out, and a value beyond the pixel type is read into it as PostGIS reads it. A band the raster lacks, a pixel `x`, `y` outside it and an array of more than two dimensions are refused.

```
SELECT dumpValues(setValues(addBand(makeEmptyRaster(3, 2, 0, 2, 1, -1, 0, 0,
  3857), NULL::integer, '16BSI', 0, -1), 1, 2, 1, ARRAY[[1, 2], [NULL, 4]]), 1);
-- {{0,1,2},{0,NULL,4}}
```

16.8 Restrictions

- Restrict a temporal cell to (the complement of) a cell or a set of cells

```
atValue(tcell,cell) → tcell
minusValue(tcell,cell) → tcell
atValues(tcell,cellset) → tcell
minusValues(tcell,cellset) → tcell
```

```
SELECT atValue(th3index '{871fa4418ffffff@2001-01-01,
  871fa44a8ffffff@2001-01-02}', h3index '871fa4418ffffff');
-- {871fa4418ffffff@2001-01-01}
SELECT atValue(tquadbin '{48a6227affffffff@2001-01-01, 485623ffffffffff@2001-01-02}',
  quadbin '48a6227affffffff');
-- {48a6227affffffff@2001-01-01}
SELECT atValue(ts2cell '{47c3c@2001-01-01, 47c4@2001-01-02}',
  s2cell '47c3c');
-- {47c3c@2001-01-01}
```

- Restrict a temporal cell to (the complement of) a time value

```
atTime(tcell,times) → tcell
minusTime(tcell,times) → tcell
```

```
SELECT atTime(th3index '[871fa4418ffffff@2001-01-01,
  871fa4418ffffff@2001-01-04]', tstzspan '[2001-01-02, 2001-01-03]');
-- [871fa4418ffffff@2001-01-02, 871fa4418ffffff@2001-01-03]
SELECT atTime(tquadbin '[48a6227afffffff@2001-01-01, 48a6227afffffff@2001-01-04]',
  tstzspan '[2001-01-02, 2001-01-03]');
-- [48a6227afffffff@2001-01-02, 48a6227afffffff@2001-01-03]
SELECT atTime(ts2cell '[47c3c@2001-01-01, 47c3c@2001-01-04]',
  tstzspan '[2001-01-02, 2001-01-03]');
-- [47c3c@2001-01-02, 47c3c@2001-01-03]
```

16.9 Static Geometry Coverage

These functions convert an ordinary (non-temporal) geometry into the cell or cell set that covers it at a given resolution, and test whether a temporal cell trajectory ever enters that cell set. The cell-set output together with the ever-intersects predicate form the cross-platform spatial prefilter used by the portable BerlinMOD queries.

- Return the cell of a grid that contains a point geometry at the given resolution

```
latLngToCell(geometry,resolution integer) → h3index
geoToQuadbinCell(geometry,integer) → quadbin
geoToS2Cell(geography,integer) → s2cell
```

The geometry must be a non-empty `POINT`, its longitude and latitude are interpreted in SRID 4326, and the resolution lies between 0 and 15 for H3, 0 and 26 for QUADBIN, and 0 and 30 for S2; any other argument raises an error. The H3 form takes the name of the h3-pg function `h3_latlng_to_cell`; QUADBIN and S2 have no host extension, and their forms keep the names MobilityDB gives them.

```
SELECT latLngToCell(geometry 'SRID=4326;Point(4.35 50.85)', 7);
-- 871fa4418ffffff
SELECT geoToQuadbinCell(geometry 'SRID=4326;Point(4.35 50.85)', 10);
-- 48a6227afffffff
SELECT geoToS2Cell(geography 'SRID=4326;Point(4.35 50.85)', 10);
-- 47c3c3
```

- Return the set of cells of a grid covering a geometry at the given resolution

```
geoToH3IndexSet(geometry,integer) → h3indexset
geoToQuadbinSet(geometry,integer) → quadbinset
geoToS2CellSet(geography,integer) → s2cellset
```

The set holds the cells that hold a point of the geometry, each point assigned to its cell as `latLngToCell`, `geoToQuadbinCell` or `geoToS2Cell` assigns it. It is the set a temporal cell of a trajectory takes its values from, so a cell set test against it never misses a trajectory sharing a point with the geometry. A `POINT` yields a single cell, a `LINESTRING` the cells its segments pass through, a `POLYGON` those of its rings together with those it encloses, and `MULTI*` / `GEOMETRYCOLLECTION` values return the union of their components. A curve, a `TIN` or a `TRIANGLE`, alone or inside a collection, is refused, since a cover omitting its cells would drop a trajectory the prefilter must keep. H3 walks each segment from cell to cell across the edges of the cells, and for a polygon takes the cells holding no point of a ring whose centre the polygon holds, reading them outwards from the cells of the rings. A QUADBIN tile is bounded by two meridians and two parallels, so QUADBIN walks each segment from tile to tile and fills the rows of tiles between the boundary tiles of a polygon. An S2 cell is bounded by four arcs of great circles, the path a geodetic segment follows, so S2 walks each segment of a geography from cell to cell and reads the cube faces down the cell hierarchy for the cells a polygon encloses. This covering is the one a spatial prefilter needs, where the h3-pg function `h3_polygon_to_cells` answers the cells whose centre a polygon holds, which is why the name stays MobilityDB's.

```
SELECT geoToH3IndexSet(geometry 'SRID=4326;Point(4.35 50.85)', 7);
-- {"871fa4418ffffff"}
SELECT geoToQuadbinSet(geometry 'SRID=4326;Point(4.35 50.85)', 10);
```

```
-- {"48a6227affffffff"}
SELECT geoToS2CellSet(geography 'SRID=4326;Point(4.35 50.85)', 10);
-- {"47c3c3"}
```

- Return whether a temporal cell trajectory ever takes one of the cells in a cell set

```
h3indexset ?= th3index → boolean
quadbinset ?= tquadbin → boolean
s2cellset ?= ts2cell → boolean
```

Combined with the cell set constructor it answers whether a trip ever passes through a region without materialising the trajectory geometry, a conservative prefilter for the exact `eIntersects`. It rests on a kernel every grid shares.

```
SELECT geoToH3IndexSet(geometry 'SRID=4326;Point(4.35 50.85)', 7) ?=
  th3index '871fa4418ffffff@2001-01-01';
-- t
SELECT geoToQuadbinSet(geometry 'SRID=4326;Point(4.35 50.85)', 10) ?=
  tquadbin '48a6227affffffff@2001-01-01';
-- t
SELECT geoToS2CellSet(geography 'SRID=4326;Point(4.35 50.85)', 10) ?=
  ts2cell '47c3c3@2001-01-01';
-- t
```

16.10 Inspection

- Return the resolution of each cell in a trajectory

```
getResolution(cell) → integer
getResolution(tcell) → tint
```

Return the resolution of a cell, as an `integer` from the static form and as a `tint` of the resolution of each cell in a trajectory from the temporal one. QUADBIN calls it a zoom level and S2 a level, and the range belongs to the grid: 0 to 15 for H3, 0 to 26 for QUADBIN, 0 to 30 for S2.

```
SELECT getResolution(h3index '831c02fffffffff');
-- 3
SELECT getResolution(th3index '831c02fffffffff@2001-01-01');
-- 3@2001-01-01
SELECT getResolution(quadbin '48a6227affffffff');
-- 10
SELECT getResolution(tquadbin '48a6227affffffff@2001-01-01');
-- 10@2001-01-01
SELECT getResolution(s2cell '47c3c');
-- 7
SELECT getResolution(ts2cell '47c3c@2001-01-01');
-- 7@2001-01-01
```

- Return a temporal boolean stating at each instant whether the value is a valid cell

```
isValidCell(cell) → boolean
isValidCell(tcell) → tbool
```

A value is a valid cell when its bit pattern denotes something its grid indexes: for QUADBIN, tile coordinates within the bounds of the resolution the identifier states; for H3, a base cell and digit sequence the icosahedron carries; for S2, a face and a Hilbert position at a level between 0 and 30.

Only H3 can be asked the question of a value that answers `false`: its input function admits any bit pattern and leaves the verdict to this function, while the QUADBIN and the S2 input functions reject a pattern that indexes nothing, so `tquadbin '0'` and `ts2cell '0'` raise an error before the call is made.

```
SELECT isValidCell(h3index '831c02ffffffffff');
-- t
SELECT isValidCell(th3index '831c02ffffffffff@2001-01-01');
-- t@2001-01-01
SELECT isValidCell(th3index '0@2001-01-01');
-- f@2001-01-01
SELECT isValidCell(quadbin '48a6227affffffff');
-- t
SELECT isValidCell(tquadbin '48a6227affffffff@2001-01-01');
-- t@2001-01-01
SELECT isValidCell(s2cell '47c3c');
-- t
SELECT isValidCell(ts2cell '47c3c@2001-01-01');
-- t@2001-01-01
```

16.11 Hierarchy

- Return the temporal parent cell at the given resolution

```
cellToParent(cell,integer) → cell
cellToParent(tcell,integer) → tcell
cellToParent(tcell) → tcell
```

Return the ancestor of a cell at the given coarser resolution. The static form takes a single `cell`; the temporal form returns the ancestor of each cell in a trajectory. The no-argument form drops to the next-coarser resolution.

```
SELECT getResolution(cellToParent(h3index '8a2a1072b59ffff', 5));
-- 5
SELECT getResolution(cellToParent(th3index '8a2a1072b59ffff@2001-01-01', 5));
-- 5@2001-01-01
SELECT getResolution(cellToParent(quadbin '48a6227affffffff', 5));
-- 5
SELECT getResolution(cellToParent(tquadbin
  '48a6227affffffff@2001-01-01', 5));
-- 5@2001-01-01
SELECT getResolution(cellToParent(s2cell '47c3c', 5));
-- 5
SELECT getResolution(cellToParent(ts2cell '47c3c@2001-01-01', 5));
-- 5@2001-01-01
```

- Return the cells covering a cell, or the cells of a cell set, at the given resolution

```
cellToCover(h3index,integer) → h3indexset
cellToCover(quadbin,integer) → quadbinset
cellToCover(s2cell,integer) → s2cellset
coverCells(h3indexset,integer) → h3indexset
coverCells(quadbinset,integer) → quadbinset
coverCells(s2cellset,integer) → s2cellset
```

The cover holds the cells that hold a point of the cell, each point assigned to its cell as `latLngToCell` assigns it, which is what `geoToH3IndexSet` states for a geometry. A trajectory passing through the cell therefore takes a cell of the cover there, so reading the values of a temporal cell this way states a cover of its trajectory at a resolution its values are not stored at, without reading the trajectory again. The resolution is the cell's own or a coarser one, and a finer one raises an error. A QUADBIN tile and an S2 cell are each exactly the union of their four children, so their cover at a coarser resolution is their ancestor there (`cellToParent`). The parent of an H3 cell is not its cover: the children of a hexagon do not tile it, so an H3 cell can reach cells other than its parent.

```
SELECT cellToCover(h3index '8c1f005018733ff', 10);
-- {"8a1f00501877fff"}
SELECT coverCells(getValues(th3index '{8c1f005018733ff@2001-01-01,
  8c1f005018731ff@2001-01-02}'), 10);
-- {"8a1f00501877fff"}
SELECT cellToCover(quadbin '48a6227affffffff', 7);
-- {"4876227fffffffff"}
SELECT coverCells(getValues(tquadbin '{48a6227affffffff@2001-01-01,
  48a6227bffffffff@2001-01-02}'), 7);
-- {"4876227fffffffff"}
SELECT cellToCover(s2cell '47c3c', 5);
-- {"47c4"}
SELECT coverCells(cellToChildren(s2cell '47c3c', 9), 7);
-- {"47c3c"}
```

16.12 Latitude and Longitude Conversions

- Return the temporal cell at the given resolution that contains each point in a temporal geodetic or planar point

```
th3index(tpoint,integer,borderInc boolean=true) → th3index
tquadbin(tpoint,integer,borderInc boolean=true) → tquadbin
ts2cell(tpoint,integer,borderInc boolean=true) → ts2cell
```

SRID must be 4326 for the `tgeompoint` overloads. The constructor over a temporal point is stated for every grid, over a geodetic and a planar point.

A trajectory that states nothing between its instants, discrete or stepwise, yields the cells holding them and keeps its interpolation; one that moves between them yields every cell it crosses on the way, since the segment is traversed cell by cell rather than sampled. The segment is traversed along the path its point follows: the great circle of a `tgeogpoint`, so a trajectory crossing the antimeridian or passing near a pole yields the cells along its shortest route, and the straight line in longitude and latitude of a `tgeompoint`. A great circle bends toward the pole in longitude and latitude, so the arc between two positions in one QUADBIN tile may leave the tile and come back to it. An S2 cell edge is an arc of a great circle, which a straight line in longitude and latitude is not, so that line between two positions in one S2 cell may likewise leave the cell and come back to it. For `th3index` and `ts2cell`, each instant of the result marks the time the trajectory entered that cell, and the last cell holds to the end of the trajectory, so the result spans its period. Under an exclusive upper bound, a cell the trajectory reaches only at its end, holding it for no time, is not part of the result. Two crossings falling in the same microsecond, the smallest separation a timestamp states, leave the cell entered at that instant: a cell left again before it holds any time is not part of the result. A crossing at an end of a segment is stated at the instant of that end, as the space split of spaceSplit states the tile of a corner, so the cover of a trajectory cut at successive half-open periods merges into the cover of the whole trajectory. The distinct cells of the result, which `getValues` answers, are the tiles a trajectory covers, the join key of a Raquet table, where a tile the trip crosses and the key omits is a tile the join never reads.

The QUADBIN cover states the multidimensional tiling of the trajectory that spaceSplit states over tiles of the same grid. A tile holds its west and north boundaries, so each position belongs to the one tile holding it, and a path through the point where four tiles meet holds the tile east of the meridian and south of the parallel meeting there at the instant it passes through it. The instant of a crossing belongs to the tile holding the crossing position: the tile the trajectory enters when it moves east or south, and the tile it leaves when it moves west or north, where the result is a sequence set whose next sequence starts after that instant. A crossing is dated as the restriction of the trajectory to a tile dates it, truncating its parameter to the microsecond, and where several crossings fall in one microsecond the last tile reached holds it. The optional argument `borderInc` states, as for `spaceSplit`, whether the upper border of the extent of the trajectory is included: without it, a tile holding nothing of the trajectory but that border, on its east meridian or on its south parallel, is not part of the result. For `th3index` and `ts2cell`, whose cells have no axis to align with the extent, the grid assigns a position on a cell boundary to one of the cells meeting there, and the cell the trajectory enters at its last instant is the one cell it holds only on the upper border of its extent: `borderInc` states whether that cell is part of the result, and without it the cell held before holds to the end of the trajectory with an exclusive bound.

```
SELECT getResolution(th3index(tgeogpoint 'POINT(-73.96 40.78)@2001-01-01', 9));
```

```
-- 9@2001-01-01
SELECT getResolution(th3index(tgeompoint
  'SRID=4326;POINT(-73.96 40.78)@2001-01-01', 9));
-- 9@2001-01-01
SELECT getResolution(tquadbin(tgeompoint
  'SRID=4326;POINT(-73.96 40.78)@2001-01-01', 9));
-- 9@2001-01-01
SELECT getResolution(tquadbin(tgeogpoint 'POINT(-73.96 40.78)@2001-01-01', 9));
-- 9@2001-01-01
SELECT numInstants(tquadbin(tgeogpoint
  '[POINT(-170 65)@2001-01-01, POINT(-100 65)@2001-01-02]', 2));
-- 5
SELECT tquadbin(tgeompoint
  'SRID=4326;[Point(0 0)@2001-01-01, Point(45 33.3)@2001-01-03]', 2);
-- {[482cffffffffffff@2001-01-01], (4826ffffffffffff@2001-01-01,
--   4826ffffffffffff@2001-01-03]}
SELECT tquadbin(tgeompoint
  'SRID=4326;[Point(0 0)@2001-01-01, Point(45 33.3)@2001-01-03]', 2, false);
-- (4826ffffffffffff@2001-01-01, 4826ffffffffffff@2001-01-03]
SELECT getResolution(ts2cell(tgeogpoint 'POINT(-73.96 40.78)@2001-01-01', 9));
-- 9@2001-01-01
SELECT getResolution(ts2cell(tgeompoint
  'SRID=4326;POINT(-73.96 40.78)@2001-01-01', 9));
-- 9@2001-01-01
SELECT numInstants(ts2cell(tgeompoint
  'SRID=4326;[POINT(-15 44.5)@2001-01-01, POINT(15 44.5)@2001-01-02]', 0));
-- 4
```

- Split a temporal point at the QUADBIN cells it crosses {}

```
quadbinSplit(tpoint,integer,borderInc boolean=true) → {(cell,tpoint)}
```

The cover states which cell a trajectory holds and when. The split states the same cells and, beside each, the trajectory over the periods the cover states for that cell, so a value indexed by cell is read in one pass where the cover alone asks for a restriction per cell. The cells of a split are the cells of the cover, the fragment of a cell is the trajectory restricted to the periods the cover states for it, and the fragments merge into the trajectory. A trajectory reaching a cell at its last instant, or passing through the corner a cell holds, holds it there for an instant, and the split states that fragment over that instant. The optional argument `borderInc` is the one of tquadbin.

The segment is traversed along the path its point follows, so a geodetic trajectory is split at the cells its great circles cross and a planar one at the cells its straight line in longitude and latitude crosses. The period of a fragment is what `getTime` answers for it, so the split carries no period of its own.

```
SELECT (s).cell, asText((s).tpoint, 6) AS tpoint
FROM (SELECT quadbinSplit(tgeompoint
  'SRID=4326;[Point(-135 -10)@2001-01-01, Point(-45 10)@2001-01-03]', 2) AS s) t;
-- 4823ffffffffffff | {(POINT(-90 0)@2001-01-02, POINT(-45 10)@2001-01-03]}
-- 4828ffffffffffff | {[POINT(-135 -10)@2001-01-01, POINT(-90 0)@2001-01-02)}
-- 4829ffffffffffff | {[POINT(-90 0)@2001-01-02]}
SELECT count(*) FROM quadbinSplit(tgeogpoint
  '[Point(-170 65)@2001-01-01, Point(-100 65)@2001-01-02]', 2);
-- 2
SELECT count(*) FROM quadbinSplit(tgeompoint
  'SRID=4326;[Point(-170 65)@2001-01-01, Point(-100 65)@2001-01-02]', 2);
-- 1
```

- Split a temporal point at the H3 cells it crosses {}

```
h3Split(tpoint,integer,borderInc boolean=true) → {(cell,tpoint)}
```

The cover states which cell a trajectory holds and when. The split states the same cells and, beside each, the trajectory over the periods the cover states for that cell, so a value indexed by cell is read in one pass where the cover alone asks for a restriction per cell. The cells of a split are the cells of the cover, the fragment of a cell is the trajectory restricted to the periods the cover states for it, and the fragments merge into the trajectory. The period of a fragment is what `getTime` answers for it, so the split carries no period of its own. The optional argument `borderInc` is the one of the cover, so without it the cell the trajectory enters at its last instant has no fragment.

The segment is traversed along the path its point follows, so a geodetic trajectory is split at the cells its great circles cross and a planar one at the cells its straight line in longitude and latitude crosses. An H3 cell is a hexagon on the sphere, and a great circle is the shortest path between two positions on it, so a geodetic trajectory crosses no more of those cells than the straight line in longitude and latitude between the same positions does.

```
SELECT (s).cell, asText((s).tpoint, 6) AS tpoint
FROM (SELECT h3Split(tgeompoint
  'SRID=4326;[Point(-0.1 51.5)@2001-01-01, Point(0.1 51.5)@2001-01-02]', 4) AS s) t;
-- 84194adffffffff | {[POINT(-0.1 51.5)@2001-01-01, POINT(0.004068 51.5)@2001-01-01  ↩
   12:29:17.362145)}
-- 84194e7ffffffff | {[POINT(0.004068 51.5)@2001-01-01 12:29:17.362145, POINT(0.1 51.5) ↩
   @2001-01-02]}
SELECT count(*) FROM h3Split(tgeogpoint
  '[Point(-170 65)@2001-01-01, Point(-100 65)@2001-01-02]', 2);
-- 14
SELECT count(*) FROM h3Split(tgeompoint
  'SRID=4326;[Point(-170 65)@2001-01-01, Point(-100 65)@2001-01-02]', 2);
-- 16
```

- Split a temporal point at the S2 cells it crosses {}

```
s2Split(tpoint,integer,borderInc boolean=true) → {(cell,tpoint)}
```

The cover states which cell a trajectory holds and when. The split states the same cells and, beside each, the trajectory over the periods the cover states for that cell, so a value indexed by cell is read in one pass where the cover alone asks for a restriction per cell. The cells of a split are the cells of the cover, the fragment of a cell is the trajectory restricted to the periods the cover states for it, and the fragments merge into the trajectory. The period of a fragment is what `getTime` answers for it, so the split carries no period of its own. The optional argument `borderInc` is the one of the cover, so without it the cell the trajectory enters at its last instant has no fragment.

The segment is traversed along the path its point follows, so a geodetic trajectory is split at the cells its great circles cross and a planar one at the cells its straight line in longitude and latitude crosses. The two paths cross different cells, and the count of them is a property of the trajectory rather than of the grid: the arc from longitude -170 to -100 along latitude 65 crosses as many S2 cells as the straight line between the same positions does, while the cells themselves differ.

```
SELECT count(*) FROM s2Split(tgeogpoint
  '[Point(-170 65)@2001-01-01, Point(-100 65)@2001-01-02]', 6);
-- 29
SELECT count(*) FROM s2Split(tgeompoint
  'SRID=4326;[Point(-170 65)@2001-01-01, Point(-100 65)@2001-01-02]', 6);
-- 29
SELECT getValues(ts2cell(tgeogpoint
    '[Point(-170 65)@2001-01-01, Point(-100 65)@2001-01-02]', 6))
  = getValues(ts2cell(tgeompoint
    'SRID=4326;[Point(-170 65)@2001-01-01, Point(-100 65)@2001-01-02]', 6));
-- f
```

- Return the centroid of a cell, or of each cell in a trajectory

```
cellToPoint({h3index,quadbin}) → geometry
cellToPoint(s2cell) → geography
cellToPoint({th3index,ts2cell}) → tgeogpoint
cellToPoint(tquadbin) → tgeompoint
tgeompoint(th3index) → tgeompoint
```

The centroid is in SRID 4326, and its type follows the grid. An S2 cell answers a geodetic point, since its edges are arcs of great circles, and a QUADBIN tile a planar one, since the tile is a rectangle in longitude and latitude. An H3 cell answers a planar point and a temporal H3 cell a geodetic one, and `tgeompoint` answers the same centroid as a planar point, so H3 is the one grid publishing both forms for a trajectory. The operation is named `cellToPoint` for every grid, the `cell_to_point` slot of the descriptor being what fixes it.

```
SELECT ST_AsText(cellToPoint(h3index '871fa44a8ffffff'), 6);
-- POINT(4.297401 50.8043)
SELECT asText(cellToPoint(th3index '871fa44a8ffffff@2001-01-01'), 6);
-- POINT(4.297401 50.8043)@2001-01-01
SELECT asText(tgeompoint(th3index '871fa44a8ffffff@2001-01-01'), 6);
-- POINT(4.297401 50.8043)@2001-01-01
SELECT ST_AsText(cellToPoint(quadbin '48a6227afffffff'), 6);
-- POINT(4.394531 50.847573)
SELECT asText(cellToPoint(tquadbin '48a6227afffffff@2001-01-01'), 6);
-- POINT(4.394531 50.847573)@2001-01-01
SELECT ST_AsText(cellToPoint(s2cell '47c3c'), 6);
-- POINT(4.222019 50.936164)
SELECT asText(cellToPoint(ts2cell '47c3c@2001-01-01'), 6);
-- POINT(4.222019 50.936164)@2001-01-01
```

- Return the polygon boundary of a cell, or of each cell in a trajectory

```
cellToBoundary({h3index,quadbin}) → geometry
cellToBoundary(s2cell) → geography
cellToBoundary({th3index,ts2cell}) → tgeography
cellToBoundary(tquadbin) → tgeometry
```

The boundary is in SRID 4326, and its type follows the grid as the centroid of cellToPoint does: a geography for S2, a geometry for QUADBIN, and for H3 a geometry for a cell and a geography for a trajectory. The ring closes on its first vertex, so an H3 hexagon carries seven points and a QUADBIN square or an S2 spherical quadrilateral five.

```
SELECT ST_NPoints(cellToBoundary(h3index '871fa44a8ffffff'));
-- 7
SELECT ST_GeometryType(getValue(cellToBoundary(th3index
  '871fa44a8ffffff@2001-01-01'))::geometry);
-- ST_Polygon
SELECT ST_NPoints(getValue(cellToBoundary(th3index
  '871fa44a8ffffff@2001-01-01'))::geometry);
-- 7
SELECT ST_GeometryType(getValue(cellToBoundary(tquadbin
  '48a6227afffffff@2001-01-01'))::geometry);
-- ST_Polygon
SELECT ST_NPoints(getValue(cellToBoundary(tquadbin
  '48a6227afffffff@2001-01-01'))::geometry);
-- 5
SELECT ST_GeometryType(getValue(cellToBoundary(ts2cell '47c3c@2001-01-01'))::geometry);
-- ST_Polygon
SELECT ST_NPoints(getValue(cellToBoundary(ts2cell '47c3c@2001-01-01'))::geometry);
-- 5
```

16.13 Bounding Box Operations

The following operators compare the spatiotemporal *bounding box* of temporal cells rather than containment in the cell hierarchy. `tcell` is a spatial temporal type: its bounding box is an `stbox` that combines the spatial extent of the cell boundaries (their longitude/latitude envelope in SRID 4326), geodetic for an H3 or S2 cell and planar for a QUADBIN cell, with the trajectory’s `tstzspan` time period. The operators below compare that box along the spatial and/or temporal axes. See the indexing section for the opclasses that make these operators index-capable.

Note

The `@>` operator tests spatiotemporal bounding-box containment (spatial extent and time together), *not* containment in the cell hierarchy. To test whether one cell is the ancestor of another at a given resolution, use cellToParent.

- Return the spatiotemporal bounding box of a temporal cell, or that box expanded in the spatial dimension by a float value

```
stbox(tcell) → stbox
expandSpace(tcell,float) → stbox
```

The box is the envelope of the cell boundaries, geodetic for an H3 or S2 cell and planar for a QUADBIN cell, together with the trajectory's time period, so it is the key both index operator classes of the indexing section store.

```
SELECT round(stbox(th3index '871fa44a8ffffff@2001-01-01'), 6);
/* SRID=4326;GEODSTBOX XT(((4.280027,50.79295),(4.314775,50.815648)),
   [2001-01-01, 2001-01-01]) */
SELECT round(expandSpace(th3index '871fa44a8ffffff@2001-01-01', 1), 6);
/* SRID=4326;GEODSTBOX XT(((3.280027,49.79295),(5.314775,51.815648)),
   [2001-01-01, 2001-01-01]) */
SELECT round(stbox(tquadbin '48a6227afffffff@2001-01-01'), 6);
/* SRID=4326;STBOX XT(((4.21875,50.736455),(4.570313,50.958427)),
   [2001-01-01, 2001-01-01]) */
SELECT round(expandSpace(tquadbin '48a6227afffffff@2001-01-01', 1), 6);
/* SRID=4326;STBOX XT(((3.21875,49.736455),(5.570313,51.958427)),
   [2001-01-01, 2001-01-01]) */
SELECT round(stbox(ts2cell '47c3c@2001-01-01'), 6);
/* SRID=4326;GEODSTBOX XT(((3.780069,50.590023),(4.676786,51.283168)),
   [2001-01-01, 2001-01-01]) */
SELECT round(expandSpace(ts2cell '47c3c@2001-01-01', 1), 6);
/* SRID=4326;GEODSTBOX XT(((2.780069,49.590023),(5.676786,52.283168)),
   [2001-01-01, 2001-01-01]) */
```

- Topological operators

```
{tstzspan,stbox,tcell} {&&, <@, @>, ~=, -|-} {tstzspan,stbox,tcell} → boolean
```

Bounding-box overlap, contains, contained-by, same, and adjacent

```
SELECT th3index '871fa44a8ffffff@2001-01-01' && tstzspan '[2001-01-01, 2001-01-02]';
-- t
SELECT th3index '871fa44a8ffffff@2001-01-01' ~= th3index '871fa44a8ffffff@2001-01-01';
-- t
SELECT tquadbin '485623fffffffff@2001-01-01' && tstzspan '[2001-01-01, 2001-01-02]';
-- t
SELECT tquadbin '485623fffffffff@2001-01-01' ~= tquadbin '485623fffffffff@2001-01-01';
-- t
SELECT ts2cell '47c3c@2001-01-01' && tstzspan '[2001-01-01, 2001-01-02]';
-- t
SELECT ts2cell '47c3c@2001-01-01' ~= ts2cell '47c3c@2001-01-01';
-- t
```

- Position operators

```
{stbox,tcell} {<<, &<, >>, &>} {stbox,tcell} → boolean
{stbox,tcell} {<<|, &<|, |>>, |&>} {stbox,tcell} → boolean
{tstzspan,stbox,tcell} {<<#, &<#, #>>, #&>} {tstzspan,stbox,tcell} → boolean
```

Relative position along the spatial axes (strictly-left, overlaps-left, overlaps-right, strictly-right; strictly-below, overlaps-below, overlaps-above, strictly-above) and along the time axis (strictly-before, overlaps-before, strictly-after, overlaps-after)

```
SELECT th3index '871fa44a8ffffff@2001-01-01' <<
  th3index '871fa44aeffffff@2001-01-01';
-- f
SELECT tquadbin '485623fffffffff@2001-01-01' << tquadbin '48b6227abfffffff@2001-01-01';
-- f
SELECT ts2cell '47c3c@2001-01-01' << ts2cell '47c4@2001-01-01';
-- f
```

```
SELECT th3index '871fa44a8ffffff@2001-01-01' <<# th3index '871fa44aeffffff@2001-01-05';
-- t
SELECT th3index '871fa44a8ffffff@2001-01-01' #>>
  th3index '871fa44aeffffff@2001-01-05';
-- f
SELECT tquadbin '485623fffffffff@2001-01-01' <<# tquadbin '48b6227abfffffff@2001-01-05';
-- t
SELECT tquadbin '485623fffffffff@2001-01-01' #>> tquadbin '48b6227abfffffff@2001-01-05';
-- f
SELECT ts2cell '47c3c@2001-01-01' <<# ts2cell '47c4@2001-01-05';
-- t
SELECT ts2cell '47c3c@2001-01-01' #>> ts2cell '47c4@2001-01-05';
-- f
```

16.14 Metric Functions

Cell area is the one metric the three grids share, and `cellArea` answers it in square metres for each of them. The length of a directed edge and the great-circle distance between two temporal points are stated in the H3-specific section.

- Return the area of a cell, and the temporal area of each cell in a trajectory

```
cellArea(cell) → float
cellArea(tcell) → tfloat
```

A cell area shrinks toward the poles on the QUADBIN grid, as Web-Mercator squares cover progressively less ground at higher latitudes. An H3 or an S2 cell is defined on the sphere, so its area varies only with resolution and with the cell's position in the base solid.

The unit is the square metre for every grid, so `cellArea` answers the same quantity whichever grid holds the cell. The h3-pg function `h3_cell_area` answers square kilometres unless asked otherwise, so for an H3 cell `cellArea` answers a million times its default.

```
SELECT cellArea(th3index '871fa44a8ffffff@2001-01-01');
-- 4441914.270110932@2001-01-01
SELECT cellArea(tquadbin '48a6227affffffff@2001-01-01');
-- 609209287.1146417@2001-01-01
SELECT cellArea(ts2cell '47c3c@2001-01-01');
-- 4145164039.0857196@2001-01-01
```

16.15 Set-Returning Functions

The following static helpers return a `cellset`, the finite-collection companion type of `cell`. They operate on a static `cell` or on a `cellset` and are not lifted to `tcell`, the temporal lift being deferred pending a `tset<T>` primitive.

The QUADBIN cells a trajectory covers are the values of tquadbin over it, which are the join key a Raquet table is read by.

- All cells within `k` grid steps of the origin

```
gridDisk(h3index,integer) → h3indexset
gridDisk(quadbin,integer) → quadbinset
gridDisk(h3indexset,integer) → h3indexset
gridDisk(quadbinset,integer) → quadbinset
s2EdgeNeighbors(s2cell) → s2cellset
```

The disk includes the origin, which is the whole of it at `k` = 0. Its shape follows the grid: on the QUADBIN square grid the disk at step `k` is the `(2k+1)×(2k+1)` block of cells centred on the origin, clipped to the grid, which has no cell beyond its edges as in the CARTO reference: a column does not wrap at the antimeridian and a row does not cross a pole, so the disk of radius 1 of a cell on an edge holds 6 cells and that of a corner 4, while on the H3 hexagonal grid it is the ring of hexagons at that many steps and everything inside it. S2 states the four cells sharing an edge with the origin and takes no step count, so it answers the disk at `k` = 1 less the origin itself. A cell set gives the union of the disks of its cells, which at `k` = 1 widens a cover by the ring of cells around it, and a disk of radius 1 widened by one step is the disk of radius 2 around the same origin.

```
SELECT gridDisk(h3index '871fa44a8ffffff', 1);
/* {"871fa44a8ffffff", "871fa44a9ffffff", "871fa44aaffffff", "871fa44abffffff",
   "871fa44acffffff", "871fa44adffffff", "871fa44aeffffff"} */
SELECT gridDisk(quadbin '48a6227affffffff', 1);
/* {"48a6226dffffffff", "48a6226fffffffff", "48a62278ffffffff", "48a62279ffffffff",
   "48a6227affffffff", "48a6227bffffffff", "48a622c5ffffffff", "48a622d0ffffffff",
   "48a622d1ffffffff"} */
SELECT numValues(gridDisk(gridDisk(h3index '871fa44a8ffffff', 1), 1));
-- 19
SELECT numValues(gridDisk(gridDisk(quadbin '48a6227affffffff', 1), 1));
-- 25
SELECT s2EdgeNeighbors(s2cell '47c3c');
-- {"47c14", "47c24", "47c34", "47c44"}
```

- All children of a cell at the given target resolution

```
cellToChildren(h3index,integer) → h3indexset
cellToChildren(quadbin,integer) → quadbinset
cellToChildren(s2cell,integer) → s2cellset
```

The number of children is the grid's branching factor: four for a QUADBIN or an S2 quadtree cell, seven for an H3 cell. The helper is static, children being a per-instant set whose temporal lift is deferred pending a `tset<T>` primitive.

```
SELECT cellToChildren(h3index '831c02fffffffff', 4);
/* {"841c021ffffffff", "841c023ffffffff", "841c025ffffffff", "841c027ffffffff",
   "841c029ffffffff", "841c02bffffffff", "841c02dffffffff"} */
SELECT cellToChildren(quadbin '48a6227affffffff', 11);
-- {"48b6227a3fffffff", "48b6227a7fffffff", "48b6227abfffffff", "48b6227affffffff"}
SELECT cellToChildren(s2cell '47c3c', 8);
-- {"47c39", "47c3b", "47c3d", "47c3f"}
```

- Compacted representation of a cell set, finer cells being merged into their parent whenever the parent's full set of children is present

```
compactCells(cellset) → cellset
```

A QUADBIN tile and an S2 cell are each exactly the union of their four children, so the cells of a QUADBIN or S2 set may be of any resolutions: a cell covered by a coarser cell of the set is dropped, and every four children of one parent are replaced by that parent, from the finest resolution up, as the S2 library normalizes a cell union. An H3 cell covers its seven children only approximately, so an H3 set is compacted as the H3 library compacts it, from cells of a single resolution, and cells of two resolutions raise an error. A compacted set mixes resolutions, while `?=` compares cells, so a compacted cover is uncompacted to the resolution of a trajectory before the trajectory is read against it.

```
SELECT compactCells(cellToChildren(h3index '831c02fffffffff', 4));
-- {"831c02fffffffff"}
SELECT compactCells(cellToChildren(quadbin '48a6227affffffff', 12));
```

```
-- {"48a6227affffffff"}
SELECT compactCells(cellToChildren(s2cell '47c3c', 9));
-- {"47c3c"}
```

- Uncompacted representation at the given resolution, the inverse of `compactCells`

```
uncompactCells(cellset,integer) → cellset
```

A cell coarser than the resolution is replaced by its descendants at the resolution. A cell finer than the resolution cannot be stated at it and raises an error, as does a QUADBIN or S2 result of more than 4194304 cells.

```
SELECT numValues(uncompactCells(compactCells(
  cellToChildren(h3index '831c02fffffffff', 4)), 4));
-- 7
SELECT numValues(uncompactCells(quadbinset '{48a6227affffffff}', 12));
-- 16
SELECT numValues(uncompactCells(compactCells(cellToChildren(s2cell '47c3c', 8)), 9));
-- 16
```

16.16 Ever and Always Relationships

- Ever and always relationships

```
eContains({geometry,tquadbin},{geometry,tquadbin}) → boolean
aContains({geometry,tquadbin},{geometry,tquadbin}) → boolean
eCovers({geometry,tquadbin},{geometry,tquadbin}) → boolean
aCovers({geometry,tquadbin},{geometry,tquadbin}) → boolean
eTouches({geometry,tquadbin},{geometry,tquadbin}) → boolean
aTouches({geometry,tquadbin},{geometry,tquadbin}) → boolean
eDisjoint({geometry,tquadbin},{geometry,tquadbin}) → boolean
eDisjoint({geography,th3index},{geography,th3index}) → boolean
eDisjoint({geography,ts2cell},{geography,ts2cell}) → boolean
aDisjoint({geometry,tquadbin},{geometry,tquadbin}) → boolean
aDisjoint({geography,th3index},{geography,th3index}) → boolean
aDisjoint({geography,ts2cell},{geography,ts2cell}) → boolean
eIntersects({geometry,tquadbin},{geometry,tquadbin}) → boolean
eIntersects({geography,th3index},{geography,th3index}) → boolean
eIntersects({geography,ts2cell},{geography,ts2cell}) → boolean
aIntersects({geometry,tquadbin},{geometry,tquadbin}) → boolean
aIntersects({geography,th3index},{geography,th3index}) → boolean
aIntersects({geography,ts2cell},{geography,ts2cell}) → boolean
eDwithin({geometry,tquadbin},{geometry,tquadbin},float) → boolean
eDwithin({geography,th3index},{geography,th3index},float) → boolean
eDwithin({geography,ts2cell},{geography,ts2cell},float) → boolean
aDwithin({geometry,tquadbin},{geometry,tquadbin},float) → boolean
aDwithin(th3index,th3index,float) → boolean
aDwithin(ts2cell,ts2cell,float) → boolean
```

A relationship of a cell is the one of its boundary, as cellToBoundary returns it, and each grid is related in the one reading that states its cells exactly. A QUADBIN tile is bounded by meridians and parallels, which are straight lines in longitude and latitude, so a `tquadbin` is related through the temporal geometry of its boundary, to a `geometry` or another `tquadbin`, with the full set of relationships of the temporal geometry types. An H3 or S2 cell is bounded by great-circle arcs, so a `th3index` or `ts2cell` is related through the temporal geography of its boundary, to a `geography` or another cell of its grid, answered on the sphere and with a distance in metres; it has the relationships a temporal geography has, which state no containment and no touching. A `geometry` passed to an H3 or S2 relationship is read as the `geography` PostGIS converts it to. Each relationship has three quantified forms: the `e`-prefixed *ever* form returns `true` when the relationship holds at some instant, the `a`-prefixed *always* form when it holds at every instant, and the `t`-prefixed *temporal* form of the next section returns a `tbool` of the instant-by-instant result. `eContains` states whether one boundary contains the other, interiors only, and `eCovers` the same with the boundary included; `eIntersects` whether the boundaries share at least one point, `eDisjoint` whether they share none, `eTouches` whether they share a boundary point but no interior point, and `eDwithin` whether they lie within the distance passed as the third argument. The last example is a point H3 assigns to the resolution-0 cell and that lies 91 km outside the polygon joining the cell's vertices with straight lines in longitude and latitude.

```
SELECT eIntersects(geography 'Polygon((4.2 50.7,4.5 50.7,4.5 50.9,
  4.2 50.9,4.2 50.7))', th3index '871fa4418ffffff@2001-01-01');
-- t
SELECT eIntersects(geometry 'SRID=4326;Polygon((4.2 50.7,4.5 50.7,4.5 50.9,
  4.2 50.9,4.2 50.7))', tquadbin '48a6227affffffff@2001-01-01');
-- t
SELECT eIntersects(geography 'Polygon((4.2 50.7,4.5 50.7,4.5 50.9,
  4.2 50.9,4.2 50.7))', ts2cell '47c3c@2001-01-01');
-- t
SELECT eIntersects(th3index '8009fffffffffff@2001-01-01',
  geography 'Point(16.6328 71.9281)');
-- t
```

16.17 Spatiotemporal Relationships

- Spatiotemporal relationships

```
tContains({geometry,tquadbin},{geometry,tquadbin}) → tbool
tCovers({geometry,tquadbin},{geometry,tquadbin}) → tbool
tDisjoint({geometry,tquadbin},{geometry,tquadbin}) → tbool
tDisjoint(th3index,th3index) → tbool
tDisjoint(ts2cell,ts2cell) → tbool
tDwithin({geometry,tquadbin},{geometry,tquadbin},float) → tbool
tIntersects({geometry,tquadbin},{geometry,tquadbin}) → tbool
tIntersects(th3index,th3index) → tbool
tIntersects(ts2cell,ts2cell) → tbool
tTouches({geometry,tquadbin},{geometry,tquadbin}) → tbool
```

The temporal form answers a `tbool` that carries the result at each instant, where the ever and always forms of the previous section answer a single `boolean` over the whole time span. An H3 or S2 cell is related as the temporal geography of its boundary, which has the temporal disjoint and intersects relationships between two temporal values, so two cells of the grid are related on the sphere.

```
SELECT tIntersects(th3index '871fa4418ffffff@2001-01-01',
  th3index '871fa4418ffffff@2001-01-01');
-- t@2001-01-01
SELECT tIntersects(geometry 'SRID=4326;Polygon((4.2 50.7,4.5 50.7,4.5 50.9,
  4.2 50.9,4.2 50.7))', tquadbin '48a6227affffffff@2001-01-01');
-- t@2001-01-01
SELECT tIntersects(ts2cell '47c3c@2001-01-01', ts2cell '47c3c@2001-01-01');
-- t@2001-01-01
```

16.18 Comparisons

Tiles also support the full set of comparison operators. The ordering is total and is defined on the QUADBIN cell first, then the pixel type, the width, the height, and finally the pixel bytes; it is meant for indexing and deduplication rather than for a spatial interpretation. A default btree operator class makes `ORDER BY`, `DISTINCT` and merge joins work on `raquet` columns, and a default hash operator class enables hash joins and hash aggregation.

- Traditional comparisons

```
cell {=, <>, <, >, <=, >=} cell → boolean
tcell {=, <>, <, >, <=, >=} tcell → boolean
```

The functions backing these operators are `eq`, `ne`, `lt`, `le`, `ge` and `gt`, together with `cmp({cell,tcell},{cell,tcell}) → integer`, which returns -1, 0, or 1. The operators over `h3index` are those of the `h3` extension, which the functions answer as.

```
SELECT cmp(h3index '831c02fffffffff', h3index '8a2a1072b59ffff');
-- -1
SELECT th3index '831c02fffffffff@2001-01-01' = th3index '831c02fffffffff@2001-01-01';
-- t
SELECT tquadbin '48a6227afffffffff@2001-01-01' = tquadbin '48a6227afffffffff@2001-01-01';
-- t
SELECT ts2cell '47c3c@2001-01-01' = ts2cell '47c3c@2001-01-01';
-- t
```

- Ever and always comparisons

```
{cell,tcell} {?=, ?<>, %=, %<>} {cell,tcell} → boolean
```

The probe operand may be a bare `cell` identifier or another `tcell` trajectory, in either operand position. The `e`-prefixed forms hold when the relationship holds at at least one instant, the `a`-prefixed forms when it holds across the entire time span.

```
SELECT th3index '[831c02fffffffff@2001-01-01,
  880326b885fffff@2001-01-05]' ?= h3index '880326b885fffff';
-- t
SELECT tquadbin '[48a6227afffffffff@2001-01-01,
  485623fffffffffff@2001-01-05]' ?= quadbin '485623fffffffffff';
-- t
SELECT ts2cell '[47c3c@2001-01-01,
  47c4@2001-01-05]' ?= s2cell '47c4';
-- t
```

- Temporal comparisons

```
{cell,tcell} {#=, #<>} {cell,tcell} → tbool
```

Return a temporal boolean giving the per-instant equality, or inequality, between a temporal cell and a probe

```
SELECT th3index '831c02fffffffff@2001-01-01' #= h3index '831c02fffffffff';
-- t@2001-01-01
SELECT tquadbin '48a6227afffffffff@2001-01-01' #= quadbin '48a6227afffffffff';
-- t@2001-01-01
SELECT ts2cell '47c3c@2001-01-01' #= s2cell '47c3c';
-- t@2001-01-01
```

- Compare two Raquet tiles

```
raquet {=, <>, <, <=, >=, >} raquet → boolean
```

The functions backing these operators are `eq`, `ne`, `lt`, `le`, `ge` and `gt`, together with `cmp(raquet,raquet) → integer`, which returns -1, 0, or 1.

```
-- Two tiles with the same cell, layout and pixels are equal
SELECT raquet('\x0102'::bytea, 2, 1, quadbin '4817fffffffffff',
  'uint8') = raquet('\x0102'::bytea, 2, 1, quadbin '4817fffffffffff', 'uint8');
-- true
-- Deduplicate tiles, which the btree and hash operator classes support
SELECT DISTINCT tile FROM elevation_tiles;
```

- Hash values

```
hash({cell,cellset,tcell,raquet}) → integer
hashExtended({cell,cellset,tcell,raquet},seed bigint) → bigint
```

Function `hash` returns a 32-bit hash value and function `hashExtended` a 64-bit hash value computed with the given seed, equal values having equal hash values. Over `h3index` they answer as the functions `h3index_hash` and `h3index_hash_extended` of the `h3` extension, which provides the type.

```
SELECT hash(th3index '831c02ffffffffff@2001-01-01') =
  hash(th3index '{831c02ffffffffff@2001-01-01}');
-- t
SELECT hash(quadbinset '{48a6227afffffffff, 485623ffffffffff}') =
  hash(quadbinset '{485623ffffffffff, 48a6227afffffffff}');
-- t
SELECT hashExtended(s2cell '47c3c', 0) <> hashExtended(s2cell '47c3c', 1);
-- t
```

16.19 Aggregations

The aggregate functions for temporal cells are illustrated next.

- Temporal count

```
tCount(tcell) → {tintSeq,tintSeqSet}
```

```
SELECT tCount(temp) FROM (VALUES (th3index '831c02ffffffffff@2001-01-01')) AS t(temp);
-- {1@2001-01-01}
SELECT tCount(temp) FROM (VALUES (tquadbin '48a6227afffffffff@2001-01-01')) AS t(temp);
-- {1@2001-01-01}
SELECT tCount(temp) FROM (VALUES (ts2cell '47c3c@2001-01-01')) AS t(temp);
-- {1@2001-01-01}
```

- Window count

```
wCount(tcell) → {tintSeq,tintSeqSet}
```

```
SELECT wCount(temp, interval '1 day') FROM (VALUES
  (th3index '831c02ffffffffff@2001-01-01')) AS t(temp);
-- {[1@2001-01-01, 1@2001-01-02]}
SELECT wCount(temp, interval '1 day') FROM (VALUES
  (tquadbin '48a6227afffffffff@2001-01-01')) AS t(temp);
-- {[1@2001-01-01, 1@2001-01-02]}
SELECT wCount(temp, interval '1 day') FROM (VALUES
  (ts2cell '47c3c@2001-01-01')) AS t(temp);
-- {[1@2001-01-01, 1@2001-01-02]}
```

- Aggregate a set of temporal cells by merging them, or by assembling them into a sequence or a sequence set

```
merge(tcell) → tcell
appendInstant(tcell[,interp text[,maxt interval]]) → tcell
appendSequence(tcell) → tcell
```

DuckDB, Apache Spark and Apache Flink call them `mergeAgg`, `appendInstantAgg` and `appendSequenceAgg` (Section 18.10.1).

```
SELECT merge(temp) FROM (VALUES (th3index '831c02ffffffffff@2001-01-01')) AS t(temp);
-- {831c02ffffffffff@2001-01-01}
SELECT merge(temp) FROM (VALUES (tquadbin '48a6227affffffff@2001-01-01')) AS t(temp);
-- {48a6227affffffff@2001-01-01}
SELECT merge(temp) FROM (VALUES (ts2cell '47c3c@2001-01-01')) AS t(temp);
-- {47c3c@2001-01-01}
```

16.20 Indexing

GiST and SP-GiST indexes can be created for table columns of temporal cells. An example of index creation is as follows:

```
CREATE INDEX Trips_Cells_SPGist_Idx ON Trips USING SPGist(Cells);
```

The GiST and SP-GiST indexes store the bounding box for the temporal cells, which is an `stbox`, as all spatiotemporal types: the extent of the cell boundaries, geodetic for H3 and S2 cells and planar for QUADBIN cells, together with the time period.

Each operator class carries the name of the temporal type it indexes, so a `th3index` column takes `th3index_rtree_ops` for GiST and `th3index_quadtree_ops` or `th3index_kdtree_ops` for SP-GiST, and the `tquadbin` and `ts2cell` names likewise. The quadtree class is the SP-GiST default and the k-d tree class is named explicitly. A B-tree and a hash operator class are defined as well, ordering and hashing by the underlying temporal value.

A GiST or SP-GiST index can accelerate queries involving the following operators:

- `<<`, `&<`, `&>`, `>>`, `<<|`, `&<|`, `|&>`, `|>>`, which only consider the spatial dimension in temporal cells,
- `<<#`, `&<#`, `#&>`, `#>>`, which only consider the time dimension in temporal cells,
- `&&`, `@>`, `<@`, `~=`, `-|-`, which consider as many dimensions as they are shared by the indexed column and the query argument.

These operators work on bounding boxes, not the entire values.

16.21 H3-Specific Operations

An H3 cell is a hexagon on an icosahedron, so it has neighbours through six edges, vertices shared with them, and a base cell among the 122 the icosahedron carries. Twelve cells at each resolution are pentagons. None of that has a counterpart in a square or a spherical quadrilateral grid.

MobilityDB builds on the h3 extension (h3-pg) and reads its `h3index` type, so every h3-pg function stays available beside the ones below. A MobilityDB function over an H3 cell takes the name of its h3-pg counterpart without the `h3_` prefix, an operation the three grids share takes the name they share, and one signature answers each operation.

h3-pg	MobilityDB
`h3_latlng_to_cell`	`latLngToCell`
`h3_cell_to_geometry`, `h3_cell_to_latlng`	`cellToPoint`
`h3_cell_to_boundary`	`cellToBoundary`
`h3_cell_to_parent`, `h3_cell_to_children`	`cellToParent`, `cellToChildren`
`h3_get_resolution`, `h3_is_valid_cell`	`getResolution`, `isValidCell`
`h3_grid_disk`	`gridDisk`
`h3_cell_area`	`cellArea`

- Return the temporal base-cell number (0–121) of each cell in a trajectory

```
th3GetBaseCellNumber(th3index) → tint
```

```
SELECT th3GetBaseCellNumber(th3index '831c02fffffffff@2001-01-01');
-- 14@2001-01-01
```

- Return a temporal boolean stating whether each cell has Class-III orientation

  ```
  th3IsResClassIii(th3index) → tbool
  ```

  Class III alternates with resolution: odd resolutions are class III, even are class II.

  ```
  SELECT th3IsResClassIii(th3index '871fa44a8ffffff@2001-01-01');
  -- t@2001-01-01
  ```

- Return a temporal boolean stating whether each cell is a pentagon

  ```
  th3IsPentagon(th3index) → tbool
  ```

  12 cells per resolution are pentagons, the remaining 110 at each base cell descend as hexagons.

  ```
  SELECT th3IsPentagon(th3index '831c00fffffffff@2001-01-01');
  -- t@2001-01-01
  SELECT th3IsPentagon(th3index '831c02fffffffff@2001-01-01');
  -- f@2001-01-01
  ```

- Return the temporal center-child cell at the given resolution

  ```
  th3CellToCenterChild(th3index,integer) → th3index
  th3CellToCenterChild(th3index) → th3index
  ```

  The no-argument form drops to the next-finer resolution.

  ```
  SELECT th3CellToCenterChild(th3index '871fa44a8ffffff@2001-01-01', 8);
  -- 881fa44a81fffff@2001-01-01
  SELECT th3CellToCenterChild(th3index '871fa44a8ffffff@2001-01-01');
  -- 881fa44a81fffff@2001-01-01
  ```

- Return the ordinal position (0-based) of each cell among the children of its parent at the given resolution

  ```
  th3CellToChildPos(th3index,integer) → tbigint
  ```

  ```
  SELECT th3CellToChildPos(th3index '871fa44a8ffffff@2001-01-01', 6);
  -- 0@2001-01-01
  ```

- Return the temporal child cell at a given ordinal position of the given parent, at the given child resolution

  ```
  th3ChildPosToCell(tbigint,th3index,integer) → th3index
  ```

  ```
  SELECT th3ChildPosToCell(tbigint '0@2001-01-01', th3index '871fa44a8ffffff@2001-01-01',
    8);
  -- 881fa44a81fffff@2001-01-01
  ```

- The cells at *exactly* `k` grid steps from the origin

  ```
  h3GridRing(h3index,integer) → h3indexset
  ```

  Fails near pentagons.

  ```
  SELECT h3GridRing(h3index '871fa44a8ffffff', 1);
  /* {"871fa44a9ffffff", "871fa44aaffffff", "871fa44abffffff", "871fa44acffffff",
     "871fa44adffffff", "871fa44aeffffff"} */
  ```

- The inclusive path of cells between two cells of the same resolution

```
h3GridPathCells(h3index,h3index) → h3indexset
```

```
SELECT h3GridPathCells(h3index '871fa44a8ffffff', h3index '871fa44b1ffffff');
/* {"871fa4484ffffff", "871fa44a3ffffff", "871fa44a8ffffff", "871fa44aeffffff",
   "871fa44b1ffffff"} */
```

- All outgoing directed edges of a cell

```
h3OriginToDirectedEdges(h3index) → h3indexset
```

6 for a hexagon and 5 for a pentagon.

```
SELECT h3OriginToDirectedEdges(h3index '871fa44a8ffffff');
/* {"1171fa44a8ffffff", "1271fa44a8ffffff", "1371fa44a8ffffff", "1471fa44a8ffffff",
   "1571fa44a8ffffff", "1671fa44a8ffffff"} */
```

- All vertexes of a cell

```
h3CellToVertexes(h3index) → h3indexset
```

6 for a hexagon and 5 for a pentagon.

```
SELECT h3CellToVertexes(h3index '871fa44a8ffffff');
/* {"2071fa44a8ffffff", "2171fa44a8ffffff", "2271fa44a8ffffff", "2371fa44a8ffffff",
   "2471fa44a8ffffff", "2571fa44a8ffffff"} */
```

- The icosahedron face indexes (0..19) intersected by a cell

```
h3GetIcosahedronFaces(h3index) → intset
```

```
SELECT h3GetIcosahedronFaces(h3index '831c02fffffffff');
-- {6, 11}
```

- Return the temporal length of each directed edge in a trajectory, in metres

```
th3EdgeLength(th3index) → tfloat
```

```
SELECT th3EdgeLength(th3CellsToDirectedEdge(th3index '871fa44a8ffffff@2001-01-01',
    th3index '871fa44aeffffff@2001-01-01'));
-- 1.272084901305088@2001-01-01
```

- Return the temporal great-circle distance between two temporal geodetic points (not between cells)

```
greatCircleDistance(tgeogpoint,tgeogpoint) → tfloat
```

```
SELECT greatCircleDistance(tgeogpoint 'Point(0 0)@2001-01-01',
  tgeogpoint 'Point(1 0)@2001-01-01');
-- 111.19505197522943@2001-01-01
```

- Return a temporal boolean stating whether two temporal cells are grid neighbours at each instant

```
th3AreNeighborCells(th3index,th3index) → tbool
```

```
SELECT th3AreNeighborCells(th3index '880326b885fffff@2001-01-01',
  th3index '880326b88dfffff@2001-01-01');
-- t@2001-01-01
```

- Return a temporal directed-edge identifier from an origin/destination pair of temporal cells

```
th3CellsToDirectedEdge(th3index,th3index) → th3index
```

```
SELECT th3CellsToDirectedEdge(th3index '871fa44a8ffffff@2001-01-01',
    th3index '871fa44aeffffff@2001-01-01');
-- 1671fa44a8ffffff@2001-01-01
```

- Return a temporal boolean stating at each instant whether the value is a valid H3 directed edge

```
isValidDirectedEdge(tcell) → tbool
```

H3 directed edges are themselves 64-bit identifiers (encoded differently from cell identifiers, `isValidDirectedEdge` distinguishes them). The MobilityDB `th3index` type carries directed edges by convention: the same type, interpreted through edge-specific accessors.

```
SELECT isValidDirectedEdge(th3CellsToDirectedEdge(th3index '871fa44a8ffffff@2001-01-01',
    th3index '871fa44aeffffff@2001-01-01'));
-- t@2001-01-01
SELECT isValidDirectedEdge(th3index '871fa44a8ffffff@2001-01-01');
-- f@2001-01-01
```

- Return the temporal origin or destination cell of a temporal directed edge

```
th3GetDirectedEdgeOrigin(th3index) → th3index
th3GetDirectedEdgeDestination(th3index) → th3index
```

```
SELECT th3GetDirectedEdgeOrigin(th3CellsToDirectedEdge(th3index
  '871fa44a8ffffff@2001-01-01', th3index '871fa44aeffffff@2001-01-01'));
-- 871fa44a8ffffff@2001-01-01
SELECT th3GetDirectedEdgeDestination(th3CellsToDirectedEdge(th3index
  '871fa44a8ffffff@2001-01-01', th3index '871fa44aeffffff@2001-01-01'));
-- 871fa44aeffffff@2001-01-01
```

- Return the temporal polygon boundary of each directed edge in a trajectory, as a temporal geography

```
th3DirectedEdgeToBoundary(th3index) → tgeography
```

```
SELECT ST_NPoints(getValue(th3DirectedEdgeToBoundary(
  th3CellsToDirectedEdge(th3index '871fa44a8ffffff@2001-01-01',
    th3index '871fa44aeffffff@2001-01-01')))::geometry);
-- 3
```

- Return the temporal H3 vertex at the given vertex number

```
th3CellToVertex(th3index,integer) → th3index
```

0–5 for hexagons, 0–4 for pentagons.

```
SELECT th3CellToVertex(th3index '871fa44a8ffffff@2001-01-01', 0);
-- 2071fa44a8ffffff@2001-01-01
```

- Return the temporal geodetic coordinates of each vertex in a trajectory

```
th3VertexToPoint(th3index) → tgeogpoint
```

```
SELECT asText(th3VertexToPoint(th3CellToVertex(th3index '871fa44a8ffffff@2001-01-01',
  0)), 6);
-- POINT(4.280027 50.807655)@2001-01-01
```

- Return a temporal boolean stating at each instant whether the value is a valid H3 vertex

```
isValidVertex(tcell) → tbool
```

```
SELECT isValidVertex(th3CellToVertex(th3index '871fa44a8ffffff@2001-01-01', 0));
-- t@2001-01-01
```

- Return the temporal grid-hop distance (number of single-cell steps) between two temporal cells

```
th3GridDistance(th3index,th3index) → tbigint
tcell <-> tcell → tbigint
```

Equivalently, the `<->` operator returns the same distance.

```
SELECT th3index '880326b885fffff@2001-01-01' <-> th3index '880326b88dfffff@2001-01-01';
-- 1@2001-01-01
```

- Convert between a temporal cell and a temporal local (I, J) coordinate pair relative to an origin temporal cell

```
th3CellToLocalIj(th3index,th3index) → tgeompoint
th3LocalIjToCell(th3index,tgeompoint) → th3index
```

The local coordinate is exposed as a `tgeompoint` with the I axis in the `x` slot and J in `y`.

```
SELECT asText(th3CellToLocalIj(th3index '871fa44a8ffffff@2001-01-01',
  th3index '871fa44aeffffff@2001-01-01'));
-- POINT(497 320)@2001-01-01
SELECT th3LocalIjToCell(th3index '871fa44a8ffffff@2001-01-01',
  th3CellToLocalIj(th3index '871fa44a8ffffff@2001-01-01',
    th3index '871fa44aeffffff@2001-01-01'));
-- 871fa44aeffffff@2001-01-01
```

16.22 QUADBIN-Specific Operations

QUADBIN is built on the slippy-map (Web-Mercator) tile scheme, so a cell maps directly to a tile coordinate triple `(x,y,z)` and to the base-4 quadkey string tile servers use. These conversions are the QUADBIN-specific bridge to the tiling ecosystem, and have no counterpart in a hexagonal or a spherical-quadrilateral grid.

- Return whether a value is a structurally valid QUADBIN identifier

```
isValidIndex(quadbin) → boolean
```

Correct header tag and resolution field. The input of the `quadbin` type raises an error on any other value, so every value of the type is a valid index, while isValidCell answers whether the index is a data cell: the index 504b5fffffffffff carries mode 2.

```
SELECT isValidIndex(quadbin '480fffffffffffff');
-- t
SELECT isValidIndex(quadbin '504b5fffffffffff');
-- t
```

- Return the neighbouring cell at the same resolution in the given direction

```
quadbinCellSibling(quadbin,text) → quadbin
```

`up` / `down` / `left` / `right`. The answer is NULL for a sibling beyond an edge of the grid, which has no cell there: a column does not wrap at the antimeridian and a row does not cross a pole.

```
SELECT quadbinCellSibling(
  quadbinCellSibling(quadbin '48427ffffffffff', 'right'), 'left')
  = quadbin '48427ffffffffff';
-- t
```

- Emit the base-4 slippy-tile quadkey string of a cell

```
quadbinCellToQuadkey(quadbin) → text
tquadbinCellToQuadkey(tquadbin) → ttext
```

One digit per zoom level, most significant first, the resolution-0 world cell mapping to the empty string. The static form takes a single `quadbin`, the temporal form returning the quadkey of each cell in a trajectory as a `ttext`.

```
SELECT quadbinCellToQuadkey(quadbin '480fffffffffff');
-- (empty string)
SELECT quadbinCellToQuadkey(quadbin '48427ffffffffff');
-- 0213
SELECT quadbinCellToQuadkey(quadbin '48a6227affffffff');
-- 1202021322
```

- Return the temporal grid distance between two temporal QUADBIN cells

```
tquadbinGridDistance(tquadbin,tquadbin) → tbigint
tquadbin <-> tquadbin → tbigint
```

The grid distance of two cells of the same zoom is the smallest radius whose `gridDisk` around one holds the other, the larger of the differences of their tile columns and rows. The grid has no cell beyond its edges, so the cells at the two ends of a row are a whole row apart. Equivalently, the <-> operator returns the same distance.

```
SELECT asText(tquadbinGridDistance(
  tquadbin(quadbinTileToCell(1, 1, 3), tstzspan '[2001-01-01, 2001-01-03]'),
  tquadbin(quadbinTileToCell(4, 2, 3), tstzspan '[2001-01-02, 2001-01-04]')));
-- [3@2001-01-02, 3@2001-01-03]
SELECT tquadbin(quadbinTileToCell(0, 3, 3), timestamptz '2001-01-01') <->
  tquadbin(quadbinTileToCell(7, 3, 3), timestamptz '2001-01-01');
-- 7@2001-01-01
```

- Decompose a cell back into its tile coordinate triple, returned as an integer array holding x, then y, then z

```
quadbinCellToTile(quadbin) → integer[]
```

It is the inverse of the tile constructor

```
SELECT quadbinCellToTile(quadbin '480fffffffffff');
-- {0,0,0}
SELECT quadbinCellToTile(quadbin '48a6227affffffff');
-- {524,343,10}
```

- Build a cell from slippy-map tile coordinates: column `x`, row `y`, and zoom `z`

```
quadbinTileToCell(integer,integer,integer) → quadbin
```

```
SELECT quadbinTileToCell(0, 0, 0) = quadbin '480fffffffffff';
-- t
SELECT quadbinTileToCell(3, 5, 4) = quadbin '48427ffffffffff';
-- t
SELECT quadbinTileToCell(524, 343, 10) = quadbin '48a6227affffffff';
-- t
```

16.23 S2-Specific Operations

The operations below have no H3 or QUADBIN analogue. They are stated on a static `s2cell`, since a cell identifier answers them without reference to time; the shared operations the sections above state apply to a `ts2cell` through the `tcell` notation.

- Return the cube face in 0 to 5 an S2 cell sits on

```
s2GetFace(s2cell) → integer
```

S2 projects the sphere onto the six faces of a circumscribed cube and lays a Hilbert curve on each, so the face is the first three bits of the identifier and two cells on different faces share no ancestor.

```
SELECT s2GetFace(s2cell '47c3c3');
-- 2
SELECT s2GetFace(geoToS2Cell(geography 'SRID=4326;Point(-122.4 37.8)', 10));
-- 4
```

- Convert between an S2 cell and its token, the form the S2 libraries publish

```
s2CellToToken(s2cell) → text
s2TokenToCell(text) → s2cell
ts2CellToToken(ts2cell) → ttext
```

A token is the identifier in hexadecimal with its trailing zeros removed, so a coarse cell reads shorter than a fine one and the text sorts as the identifier does.

```
SELECT s2CellToToken(s2cell '47c3c3');
-- 47c3c3
SELECT s2TokenToCell('47c3c3');
-- 47c3c3
SELECT asText(ts2CellToToken(ts2cell '47c3c3@2001-01-01'));
-- "47c3c3"@2001-01-01
```

- Return the temporal grid distance between two temporal S2 cells

```
ts2GridDistance(ts2cell,ts2cell) → tbigint
ts2cell <-> ts2cell → tbigint
```

The grid distance of two cells of the same level is the number of steps between edge neighbours, those `s2EdgeNeighbors` returns, from one to the other, across the edges of the cube faces as within a face. Two cells on opposite faces are a path around the cube apart. Equivalently, the `<->` operator returns the same distance.

```
SELECT asText(ts2GridDistance(
  ts2cell(s2cell '47c3c', tstzspan '[2001-01-01, 2001-01-03]'),
  ts2cell(s2cell '47c34', tstzspan '[2001-01-02, 2001-01-04]')));
-- [1@2001-01-02, 1@2001-01-03]
SELECT ts2cell(geoToS2Cell(geography 'Point(0 0)', 30),
    timestamptz '2001-01-01') <->
  ts2cell(geoToS2Cell(geography 'Point(180 0)', 30),
    timestamptz '2001-01-01');
-- 2147483648@2001-01-01
```

- Return the level of the deepest cell containing two S2 cells

```
s2CommonAncestorLevel(s2cell,s2cell) → integer
```

Two cells on different cube faces are contained by no cell, and the function answers -1 for them.

```
SELECT s2CommonAncestorLevel(s2cell '47c3c3', s2cell '47c4');
-- 5
SELECT s2CommonAncestorLevel(s2cell '47c3c3', s2cell '808581');
-- -1
```

- Return true if the first S2 cell contains the second one

  ```
  s2CellContains(s2cell,s2cell) → boolean
  ```

  ```
  SELECT s2CellContains(s2cell '47c4', s2cell '47c3c3');
  -- t
  ```

- Return the first and the last leaf cell an S2 cell contains

  ```
  s2RangeMin(s2cell) → s2cell
  s2RangeMax(s2cell) → s2cell
  ```

  The descendants of an S2 cell occupy a contiguous interval of the Hilbert curve, so an ancestor test is the pair of comparisons `other >= s2RangeMin(cell)` and `other <= s2RangeMax(cell)`. That is what S2 brings that a hexagonal or a Web-Mercator grid does not, and it is a filter feeding the `stbox` index rather than an index key of its own.

  ```
  SELECT s2RangeMin(s2cell '47c3c3');
  -- 47c3c20000000001
  SELECT s2RangeMax(s2cell '47c3c3');
  -- 47c3c3ffffffffff
  ```

- Return the descendant of an S2 cell at the given position along the Hilbert curve

  ```
  s2CellToChild(s2cell,level integer,position integer) → s2cell
  ```

  The level is exactly one finer than the cell's own, so the position runs from 0 to 3; `cellToChildren`, which the hierarchy section states, answers every descendant at any finer level.

  ```
  SELECT s2CellToChild(s2cell '47c3c3', 11, 0);
  -- 47c3c24
  ```

- Return the length in metres of an edge of an S2 cell

  ```
  s2EdgeLength(s2cell,edge integer) → float
  ```

  The edge runs from the vertex of the given index, which is 0 to 3, to the next one counterclockwise, and the length is measured on the WGS84 sphere.

  ```
  SELECT round(s2EdgeLength(s2cell '47c3c3', 0)::numeric, 3);
  -- 9270.048
  ```

- Return the four cells sharing an edge with an S2 cell

  ```
  s2EdgeNeighbors(s2cell) → s2cellset
  ```

  An S2 cell is a quadrilateral, so it has four edge neighbours and no ring of a given radius; the H3 grid disk has no S2 analogue.

  ```
  SELECT s2EdgeNeighbors(s2cell '47c3c3');
  -- {"47c3c1", "47c3c5", "47c3dd", "47c3e9"}
  ```

A region covering, which answers the set of cells approximating an arbitrary geometry, is the one S2 operation this chapter states no signature for: the S2 libraries publish it and MobilityDB does not yet carry a kernel for it.

16.24 Raster-Specific Operations

The sections above state what a raster shares with the other tessellations. The operations below are the raster's own: a raster cell carries a *value* where an H3, QUADBIN or S2 cell carries only identity, so a trajectory read against a raster answers a `tfloat` rather than a temporal cell.

A raster is the same value in every engine, and in DuckDB and in Spark it also converts to and from the raster of the host, the one of DuckDB's raster extension and the one of Apache Sedona: Section 18.11 states what each conversion keeps and how a query writes it.

MobilityDB's raster sampling operator reads a raster band along a moving-point trajectory and returns a `tfloat`. A position outside the raster extent or on a nodata pixel carries no value. The sampling operators are double-valued whatever the pixel type of the band, so a pixel type belongs to the sampling surface when every value it can hold is exactly representable in a double precision number. That is what lets a band of any type be sampled into one `tfloat`, and a column holding tiles of several pixel types be read by a single query. A pixel type is also accepted under the name PostGIS raster gives it, so the band type an `ST_BandPixelType` call reports (`8BUI`, `16BSI`, `32BF`, `...`) passes into the tile constructors as it stands. PostGIS raster carries no 16-bit float and no 64-bit integer, so `float16`, `int64` and `uint64` take the spelling its grammar gives them, `16BF`, `64BSI` and `64BUI`. Its `1BB`, `2BUI` and `4BUI` types bound a pixel to less than a byte while PostGIS stores each of them a byte a pixel, so such a band is already the bytes of an unsigned 8-bit band and the constructors read it as `uint8`; the bound is the only thing the name adds, and a tile does not carry it. The name a tile reports is the one the RaQuet specification gives the type. The types `uint64` and `int64` hold values beyond that domain: a tile of either is stored and read back exactly, while sampling a pixel whose value no double precision number names raises an error rather than answering with a neighbouring value. This enables queries such as *what elevation profile did each vehicle follow?* or *which trips entered a temperature threshold band?*

A Raquet table keeps the QUADBIN cell of a tile in its `block` column as a 64-bit integer, beside the metadata row at `block` = 0, which holds no QUADBIN index. A query casts the cells of a trajectory to `bigint` to read that column, and a cell read from it to `quadbin`, the cast checking that it is a QUADBIN index. The typical query pattern joins a trajectory against a Raquet table in two steps:

```sql
-- Step 1: identify which tiles the trajectory overlaps
SELECT getValues(tquadbin(traj, 8));
-- Step 2: sample each matching tile, a 256 x 256 float32 band without nodata
SELECT rasterTileValueQuadbin(traj, band_1, 256, 256, block::quadbin, 'float32', 0,
  false) FROM elevation_raquet
WHERE block IN (SELECT unnest(getValues(tquadbin(traj, 8)))::bigint);
```

The restriction and predicate operators that close the section compose `rasterValue` with the standard MobilityDB temporal restriction and predicate functions. They accept an optional `band` argument (default 1) and call `rasterValue` exactly once per invocation.

A raster function takes the name of its PostGIS raster counterpart with the `ST_` prefix removed, and the names and defaults of its arguments. One signature answers each operation, so a PostGIS form that only fixes an argument of another is written through that other form. A name the SQL standard reserves takes the type as a prefix, so `ST_Value` is `rasterValue`, and the Well-Known Binary functions take the names every MobilityDB type takes.

PostGIS raster	**MobilityDB**
`ST_Value`	`rasterValue`
`ST_Clip`	`clip`
`ST_Transform`	`transform`
`ST_Rescale`	`rescale`
`ST_Reclass`	`reclass`
`ST_SummaryStats`	`summaryStats`
`ST_DumpAsPolygons`	`dumpAsPolygons`
`ST_NumBands`, `ST_Width`, `ST_Height`, `ST_SRID`	`numBands`, `width`, `height`, `S`
`ST_UpperLeftX`, `ST_UpperLeftY`, `ST_ScaleX`, `ST_ScaleY`, `ST_SkewX`, `ST_SkewY`	`upperLeftX`, `upperLeftY`, `s`
`ST_BandPixelType`, `ST_BandNoDataValue`	`bandPixelType`, `bandNoDat`
`ST_MakeEmptyRaster`, `ST_AddBand`, `ST_SetValues`	`makeEmptyRaster`, `addBand`
`ST_AsBinary`, `ST_AsHexWKB`	`asBinary`, `asHexWKB`

PostGIS raster	MobilityDB
`ST_RastFromWKB`, `ST_RastFromHexWKB`	`rasterFromBinary`, `raster`

`asBinary` and `asHexWKB` take a byte order where PostGIS takes `outasin`, and without one answer the bytes of `ST_AsBinary` and `ST_AsHexWKB`.

- Read a raster band along a trajectory

```
rasterValue(tgeompoint,rast raster,band integer=1,exclude_nodata_value boolean=true,
  resample text='nearest') → tfloat
rasterValue(tgeompoint,path text,band integer=1) → tfloat
```

Reads band `band` along `traj`, taking the value of the pixel each position falls in. A trajectory that states nothing between its instants is read at its instants and answers an instant-set `tfloat`. A trajectory that moves between them passes over the pixels between them, so each segment is traversed pixel by pixel, and it answers a step `tfloat` holding each value from the instant the trip reaches the pixel holding it, a pixel whose corner the trip only clips included. Under an exclusive upper bound, a pixel the trajectory reaches only at its end, holding it for no time, carries no value into the result, so the reads of a trajectory cut at successive half-open periods merge into the read of the whole trajectory. A position outside the raster or over a nodata pixel carries no value and ends the run, so a trip that leaves and re-enters the raster answers one sequence per visit, and `NULL` when it never meets a pixel carrying data. A band whose pixels cannot be read, such as a band stored outside the database in a file that cannot be opened, raises an error rather than reading as a band without values. The form taking `path` reads a raster file on the server, in any format GDAL supports, through GDAL, where PostGIS allows a raster band stored outside the database to be read: `postgis.enable_outdb_rasters` must be on and `postgis.gdal_enabled_drivers` must enable the driver of the file. It answers what the form taking a `raster` answers over the same raster.

`exclude_nodata_value` and `resample` are those of the PostGIS `ST_Value`: with the first false a position over a nodata pixel keeps the nodata value where it is otherwise dropped, and the second reads the pixel under the position with `nearest` or `bilinear`, named without regard to case. A bilinear value varies within a pixel. Along a segment of a trajectory that moves between its instants it is, over each half-pixel square the segment crosses, a polynomial of degree two in time, so a trajectory read with `bilinear` answers a linear `tfloat` holding the value at the instants of the trajectory, where it crosses the pixel edges and the lines through the pixel centres, and at the extremum each of those pieces reaches, linear between them, as the product of two temporal floats holds its extremum. Its minimum and maximum are those of the value, and it is the value itself where the trajectory moves along a grid axis, over which the value is linear; elsewhere, between the instants it holds, it is the chord of the value. The value jumps across a pixel edge where a pixel read there holds nodata, which takes the value of the pixel the position falls in, and a new sequence then starts at the instant the trajectory crosses the edge.

```
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1 degree pixels)
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, NULL::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT rasterValue(tgeompoint 'SRID=4326;[POINT(0.5 2.5)@2001-01-01,
  POINT(2.5 2.5)@2001-01-02, POINT(0.5 0.5)@2001-01-03]', r)::text FROM rast;
/* Interp=Step;[10@2001-01-01, 20@2001-01-01 06:00:00, 30@2001-01-01 18:00:00,
   50@2001-01-02 06:00:00, 70@2001-01-02 18:00:00, 70@2001-01-03] */
-- A bilinear read reaching its maximum between two pixel centres
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, NULL::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4], [20.0::float4, 10.0::float4]]) AS r )
SELECT round(rasterValue(tgeompoint 'SRID=4326;[POINT(0.5 1.5)@2001-01-01,
  POINT(1.5 1.1)@2001-01-03]', r, 1, true, 'bilinear'), 6)::text FROM rast;
/* [10@2001-01-01, 15@2001-01-02, 16.125@2001-01-02 18:00:00, 16@2001-01-03] */
```

Applied to a trajectory dataset with a terrain elevation raster:

```
-- Elevation profile per trip (requires a loaded elevation raster)
SELECT tripid, rasterValue(trip, elev) AS elevation FROM trips, elevation_raster;
```

```
-- Average elevation per trip
SELECT tripid, avgValue(rasterValue(trip, elev)) AS mean_elev FROM trips,
  elevation_raster;
-- Trips that crossed terrain above 200 m
SELECT tripid FROM trips, elevation_raster
WHERE atSpan(rasterValue(trip, elev), floatspan '[200, 9999]') IS NOT NULL;
```

A raster loaded as a table of tiles, as `raster2pgsql -t` loads it, is read along a trajectory by merging the reads of its tiles with `merge`. A position lies in a pixel of one tile, so the reads of the tiles hold disjoint times, and their merge answers the read of the raster the tiles cover, a trip crossing from one tile to the next included.

```
-- A 4x4 raster read along a trip whole, and as its four tiles of 2x2 pixels
WITH r AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(4, 4, 0.0, 4.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, NULL::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4, 40.0::float4],
    [50.0::float4, 60.0::float4, 70.0::float4, 80.0::float4],
    [90.0::float4, 100.0::float4, 110.0::float4, 120.0::float4],
    [130.0::float4, 140.0::float4, 150.0::float4, 160.0::float4]]) AS rast ),
t AS (
  SELECT tgeompoint 'SRID=4326;[POINT(0.5 3.5)@2001-01-01, POINT(3.5 0.5)@2001-01-02]'
    AS trip )
SELECT merge(rasterValue(trip, tile)) = (SELECT rasterValue(trip, rast) FROM t, r)
FROM t, r, ST_Tile(rast, 2, 2) AS tile;
-- true
```

- Return the number of pixels of a raster band holding each value

```
valueCount(rast raster,nband integer=1,exclude_nodata_value boolean=true,
  searchvalues float[]=NULL,roundto float=0) → setof (value float,count integer)
valueCount(rast raster,nband integer,searchvalues float[],roundto float=0)
  → setof (value float,count integer)
valueCount(rast raster,searchvalues float[],roundto float=0)
  → setof (value float,count integer)
valueCount(rast raster,nband integer,exclude_nodata_value boolean,searchvalue float,
  roundto float=0) → integer
valueCount(rast raster,nband integer,searchvalue float,roundto float=0) → integer
valueCount(rast raster,searchvalue float,roundto float=0) → integer
```

The histogram of PostGIS's `ST_ValueCount`, with its arguments and defaults: one row for each distinct value of the band, or one for each search value, which counts zero where no pixel holds it, and the number of pixels holding one value in the forms taking a single search value. A value is rounded to the nearest multiple of `roundto` before it is counted where `roundto` is positive. A band whose every pixel is nodata answers no row where nodata is excluded, and a band the raster does not have raises an error where PostGIS answers `NULL`. The forms of PostGIS reading a raster column of a table query the database, so they stay PostGIS's.

```
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, -9999.0::float8), 1, 1, 1,
    ARRAY[[1.24, 1.26, -9999], [2, 2, 2], [3, 1.24, 2]]::double precision[][]) AS r )
SELECT round(value::numeric, 2), count
FROM rast, valueCount(r, 1, true, NULL::float[], 0.1) ORDER BY 1;
-- 1.20 | 2
-- 1.30 | 1
-- 2.00 | 4
-- 3.00 | 1
```

- Return the share of the pixels of a raster band holding each value

```
valuePercent(rast raster,nband integer=1,exclude_nodata_value boolean=true,
  searchvalues float[]=NULL,roundto float=0) → setof (value float,percent float)
```

```
valuePercent(rast raster,nband integer,searchvalues float[],roundto float=0)
  → setof (value float,percent float)
valuePercent(rast raster,searchvalues float[],roundto float=0)
  → setof (value float,percent float)
valuePercent(rast raster,nband integer,exclude_nodata_value boolean,
  searchvalue float,roundto float=0) → float
valuePercent(rast raster,nband integer,searchvalue float,roundto float=0) → float
valuePercent(rast raster,searchvalue float,roundto float=0) → float
```

The share of PostGIS's `ST_ValuePercent`, between 0 and 1, of the pixels valueCount reads, with the same arguments and forms. Where no pixel is read, the share of a search value is zero divided by zero, `NaN`, as for PostGIS.

```
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, -9999.0::float8), 1, 1, 1,
    ARRAY[[1.24, 1.26, -9999], [2, 2, 2], [3, 1.24, 2]]::double precision[][]) AS r )
SELECT valuePercent(r, 2::float), valuePercent(r, 3::float) FROM rast;
-- 0.5 | 0.125
```

- Return true if rasters lie on the same grid

```
sameAlignment(rast1 raster,rast2 raster) → boolean
sameAlignment(raster) → boolean
sameAlignment(raster) → boolean
```

The test of PostGIS's `ST_SameAlignment`: two rasters are aligned when they have the same scale and skew and a pixel corner of one falls on a pixel corner of the other, so that their pixels can be combined one to one. The aggregate answers whether every raster of a column is aligned with the first, skipping a `NULL` raster as `setUnion` skips a `NULL` set, where PostGIS answers `NULL`; over no raster it answers `NULL`. Rasters of different SRIDs raise an error, where PostGIS answers false with a notice.

```
WITH r AS (SELECT
  ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0, 4326) AS base,
  ST_MakeEmptyRaster(3, 3, 1.0, 3.0, 1.0, -1.0, 0.0, 0.0, 4326) AS east1,
  ST_MakeEmptyRaster(3, 3, 0.5, 3.0, 1.0, -1.0, 0.0, 0.0, 4326) AS easthalf)
SELECT sameAlignment(base, east1), sameAlignment(base, easthalf),
  (SELECT sameAlignment(x) FROM (VALUES (base), (east1), (easthalf)) t(x))
FROM r;
-- t | f | f
```

- Return why two rasters do not lie on the same grid

```
notSameAlignmentReason(rast1 raster,rast2 raster) → text
```

The reason of PostGIS's `ST_NotSameAlignmentReason`, in its words: the first property in which the grids differ, or that the rasters are aligned.

```
WITH r AS (SELECT
  ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0, 4326) AS base,
  ST_MakeEmptyRaster(3, 3, 0.5, 3.0, 1.0, -1.0, 0.0, 0.0, 4326) AS easthalf,
  ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 2.0, -1.0, 0.0, 0.0, 4326) AS scalex2)
SELECT notSameAlignmentReason(base, easthalf), notSameAlignmentReason(base, scalex2)
FROM r;
-- The rasters (pixel corner coordinates) are not aligned |
--   The rasters have different scales on the X axis
```

- Return the union of the rasters of a column

```
rasterUnion(raster) → raster
rasterUnion(raster,nband integer) → raster
rasterUnion(raster,uniontype text) → raster
rasterUnion(raster,nband integer,uniontype text) → raster
```

The aggregate of PostGIS's `ST_Union` over rasters, named as `geoUnion` names the union of geometries, since `union` is a keyword of SQL. The result covers the union of the extents of the rasters, each pixel the pixel operation of the pixels holding a value there: `LAST`, the default, `FIRST`, `MIN`, `MAX`, `COUNT`, `SUM`, `MEAN` or `RANGE`, regardless of case. Without `nband` every band is united, a band a raster lacks read as nodata; with `nband` only that band, which every raster must hold. The pixel type and the nodata value are those of the band read first, and a count is an unsigned 32-bit band without a nodata value. A `NULL` raster is skipped, and over no raster the aggregate answers `NULL`. The aggregate runs in parallel, the unions of the workers combined, where PostGIS's has no combine function; `LAST` and `FIRST` depend on the order of the rasters, which an `ORDER BY` in the aggregate fixes. Rasters of different SRIDs or on different grids raise an error, and so does a pixel operation outside these, which PostGIS reads as `LAST`.

```
WITH tiles AS (
  SELECT 1 AS id, ST_AddBand(ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 3857) ↩
      ,
    '32BF'::text, 1.0::float8, -9999.0::float8) AS r
  UNION ALL
  SELECT 2, ST_AddBand(ST_MakeEmptyRaster(2, 2, 1.0, 1.0, 1.0, -1.0, 0.0, 0.0, 3857),
    '32BF'::text, 3.0::float8, -9999.0::float8) )
SELECT ST_DumpValues(rasterUnion(r, 'MEAN' ORDER BY id), 1) FROM tiles;
-- {{1,1,NULL},{1,2,3},{NULL,3,3}}
```

- Return a raster whose band holds the classes an expression maps its values to

```
reclass(raster,nband integer,reclassexpr text,pixeltype text,nodataval float=NULL)
  → raster
```

The expression follows the grammar of the PostGIS `ST_Reclass`: a comma-separated list of mappings, each a source range and a destination around a colon, a range being one value or two around a dash. A range written plainly includes its low end and excludes its high end, while `(` excludes the low end and `]` includes the high end, so `0-50` leaves 50 unmapped where `[50-100]` takes it. A pixel no range takes holds the nodata value of the resulting band, or zero when the band states none. That band has the pixel type `pixeltype`, named as PostGIS raster names it, such as `8BUI`, and states no nodata value when `nodataval` is left out. The raster keeps its grid and its other bands, so the classes can be read along a trajectory with rasterValue.

```
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1 degree pixels)
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, NULL::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT ST_DumpValues(reclass(r, 1, '0-50:1, 51-100:2', '8BUI', 0), 1) AS plain,
  ST_DumpValues(reclass(r, 1, '0-50:1, [50-100]:2', '8BUI', 0), 1) AS bracket
FROM rast;
-- {{1,1,1},{1,NULL,2},{2,2,2}} | {{1,1,1},{1,2,2},{2,2,2}}
-- Read the classes along a trajectory
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, NULL::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT rasterValue(tgeompoint 'SRID=4326;{POINT(0.5 2.5)@2001-01-01,
  POINT(1.5 1.5)@2001-01-02, POINT(2.5 0.5)@2001-01-03}',
  reclass(r, 1, '0-50:1, [50-100]:2', '8BUI', 0))::text FROM rast;
-- {1@2001-01-01, 2@2001-01-02, 2@2001-01-03}
```

- Addition, subtraction, multiplication, and division of rasters, pixel by pixel

```
{float,raster} {+, -, *, /} {float,raster} → raster
```

A raster takes the arithmetic operators of a temporal number, so a derived raster is written as the expression it is, such as `(nir - red) / (nir + red)` or `depth - 8.5`, where PostGIS writes it as the expression of `ST_MapAlgebra`. Every band of the result has the pixel type `64BF`, whatever the pixel types of the operands, so a pixel holds the arithmetic as computed: a negative difference of two unsigned bands, a product beyond their range, and a ratio between zero and one are kept rather than clamped or truncated, and reclass narrows the result where a smaller pixel type is needed. Two rasters are operated band by band over the pixels they share, as two temporal numbers are over the time they share: they are to have the same reference system, the same grid, and the same number of bands, and two rasters sharing no pixel have no result. A pixel that is nodata in either operand is nodata in the result, which takes the nodata value of the first operand stating one and states none when neither does, so a result of zero is a value. A division by zero raises an error, whether the divisor is a number or a pixel. The functions behind the operators are `rasterAdd`, `rasterSub`, `rasterMul`, and `rasterDiv`.

```
-- The normalized difference of two unsigned 16-bit bands
WITH bands AS (
  SELECT
    ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
      4326), '16BUI'::text, 0.0::float8, NULL::float8), 1, 1, 1,
      ARRAY[[3000, 500, 2000], [4000, 1000, 6000], [100, 0, 65535]]
        ::double precision[][]) AS nir,
    ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
      4326), '16BUI'::text, 0.0::float8, NULL::float8), 1, 1, 1,
      ARRAY[[1000, 1500, 2000], [1000, 3000, 2000], [300, 1, 0]]
        ::double precision[][]) AS red )
SELECT ST_BandPixelType((nir - red) / (nir + red), 1),
  ST_DumpValues((nir - red) / (nir + red), 1)
FROM bands;
-- 64BF | {{0.5,-0.5,0},{0.6,-0.5,0.5},{-0.5,-1,1}}
-- Read a derived raster along a trajectory: the depth under a vessel of draught 8.5
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, NULL::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS depth )
SELECT rasterValue(tgeompoint 'SRID=4326;[POINT(0.5 2.5)@2001-01-01,
  POINT(2.5 2.5)@2001-01-02, POINT(0.5 0.5)@2001-01-03]', depth - 8.5)::text
FROM rast;
-- Interp=Step;[1.5@2001-01-01 00:00:00+00, 11.5@2001-01-01 06:00:00+00,
--   21.5@2001-01-01 18:00:00+00, 41.5@2001-01-02 06:00:00+00,
--   61.5@2001-01-02 18:00:00+00, 61.5@2001-01-03 00:00:00+00]
```

- Return the slope of a raster band, pixel by pixel

```
slope(rast raster,nband integer=1,pixeltype text='32BF',units text='DEGREES',
  scale float=1.0,interpolate_nodata boolean=false) → raster
slope(rast raster,nband integer,customextent raster,pixeltype text='32BF',
  units text='DEGREES',scale float=1.0,interpolate_nodata boolean=false) → raster
```

The slope of PostGIS's `ST_Slope`, with its arguments and defaults: the gradients of each pixel are read from its eight neighbours, a neighbour that is nodata or lies outside the raster taking the value of the pixel, and a nodata pixel stays nodata. The units are `DEGREES`, `RADIANS` or `PERCENT`, read by their first three letters regardless of case, and units outside these raise an error; PostGIS answers radians in degrees. The scale is the ratio of the vertical units to the horizontal ones, such as 111120 for elevations in meters over a grid in degrees. Where `interpolate_nodata` is true, a nodata pixel first takes the mean of its neighbours weighted by the inverse of their distances, as PostGIS's `ST_InvDistWeight4ma` computes it. The result takes the grid of `customextent` where it is given.

```
-- A plane rising by 1 for each pixel to the east
WITH plane AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, NULL::float8), 1, 1, 1,
    ARRAY[[0, 1, 2], [0, 1, 2], [0, 1, 2]]::double precision[][]) AS r )
SELECT ST_Value(slope(r), 2, 2), ST_Value(slope(r, units => 'PERCENT'), 2, 2)
```

```
FROM plane;
-- 45 | 100
```

- Return the terrain ruggedness index of a raster band, pixel by pixel

```
tri(rast raster,nband integer=1,pixeltype text='32BF',interpolate_nodata boolean=false)
  → raster
tri(rast raster,nband integer,customextent raster,pixeltype text='32BF',
  interpolate_nodata boolean=false) → raster
```

The index of PostGIS's `ST_TRI`, with its arguments and defaults: the mean of the absolute differences between a pixel and its eight neighbours, read as slope reads them.

```
WITH dem AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(4, 4, 0.0, 4.0, 1.0, -1.0, 0.0, 0.0,
    3857), '32BF'::text, 0.0::float8, -9999.0::float8), 1, 1, 1,
    ARRAY[[10, 20, 30, 40], [15, 25, -9999, 45], [20, 30, 40, 50],
    [25, 35, 45, 55]]::double precision[][]) AS r )
SELECT ST_DumpValues(tri(r), 1) FROM dem;
-- {{3.75,3.75,5,1.875},{5,7.5,NULL,3.75},{5,8.125,8.125,3.125},{2.5,5.625,5.625,3.75}}
```

- Return the topographic position index of a raster band, pixel by pixel

```
tpi(rast raster,nband integer=1,pixeltype text='32BF',interpolate_nodata boolean=false)
  → raster
tpi(rast raster,nband integer,customextent raster,pixeltype text='32BF',
  interpolate_nodata boolean=false) → raster
```

The index of PostGIS's `ST_TPI`, with its arguments and defaults: the value of a pixel minus the mean of its eight neighbours, read as slope reads them.

```
WITH dem AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(4, 4, 0.0, 4.0, 1.0, -1.0, 0.0, 0.0,
    3857), '32BF'::text, 0.0::float8, -9999.0::float8), 1, 1, 1,
    ARRAY[[10, 20, 30, 40], [15, 25, -9999, 45], [20, 30, 40, 50],
    [25, 35, 45, 55]]::double precision[][]) AS r )
SELECT ST_DumpValues(tpi(r, interpolate_nodata => true), 1) FROM dem;
-- {{-3.75,-1.875,-1.875,1.25},{-3.75,0,0,3.75},{-3.75,0,0,3.75},{-1.25,1.875,1.875,3.75}}
```

- Return a statistic of the neighbourhood of each pixel of a raster band

```
mapAlgebra(rast raster,nband integer,stat text,pixeltype text=NULL,
  extenttype text='FIRST',customextent raster=NULL,distancex integer=0,
  distancey integer=0) → raster
```

The focal statistics of PostGIS's `ST_MapAlgebra` with the callbacks `ST_Min4ma`, `ST_Max4ma`, `ST_Sum4ma`, `ST_Mean4ma`, `ST_Range4ma` and `ST_StdDev4ma`, with its arguments and defaults, the callback named by its statistic: `MIN`, `MAX`, `SUM`, `MEAN`, `RANGE` or `STDDEV`, regardless of case, as `ST_Union` names its operations. The neighbourhood of a pixel extends `distancex` pixels to the west and to the east and `distancey` pixels to the north and to the south; its nodata pixels, and those outside the raster, are skipped. The sum of no pixel is 0, every other statistic of no pixel is nodata, and the standard deviation is the sample one. As for PostGIS, a neighbourhood whose distance is zero along one axis only is the pixel alone. A pixel type left NULL is the one of the band, and the result keeps the nodata value of the band, or its minimum value where it states none. Every extent type but `CUSTOM` is the extent of the raster, and `CUSTOM` takes the grid of `customextent`. A statistic or an extent type outside these raises an error, where PostGIS reads an unknown extent type as `INTERSECTION`.

```
WITH dem AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(4, 4, 0.0, 4.0, 1.0, -1.0, 0.0, 0.0,
    3857), '32BF'::text, 0.0::float8, -9999.0::float8), 1, 1, 1,
    ARRAY[[10, 20, 30, 40], [15, 25, -9999, 45], [20, 30, 40, 50],
    [25, 35, 45, 55]]::double precision[][]) AS r )
SELECT ST_DumpValues(mapAlgebra(r, 1, 'MEAN', '64BF', distancex => 1, distancey => 1), 1)
```

```
FROM dem;
-- {{17.5,20,32,38.333333333333336},{20,23.75,35,41},{25,29.375,40.625,47},
--  {27.5,32.5,42.5,47.5}}
```

- Return a raster keeping the pixels of another that a geometry covers

```
clip(raster,geom geometry,crop boolean=true,touched boolean=false) → raster
clip(raster,nband integer[],geom geometry,nodataval float[]=NULL,crop boolean=true,
  touched boolean=false) → raster
```

Every band is read against a mask the geometry is burnt into, as the PostGIS `ST_Clip` reads it: a pixel the geometry does not cover answers the nodata value of its band, and the smallest value its pixel type holds when the band declares none. With `crop` the result carries the extent the raster and the geometry share, and otherwise the extent of the raster. The geometry is in the reference system of the raster; one in another system raises an error. With `nband` the result keeps the bands named, every band when it is NULL, `nodataval` states the nodata value of the pixels the geometry does not cover, one for every band or one per band, and with `touched` a pixel the geometry touches is kept where otherwise only one whose centre it covers is.

```
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1 degree pixels)
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, -9999::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT ST_DumpValues(clip(r, geometry 'SRID=4326;POLYGON((0 1,2 1,2 3,0 3,0 1))'), 1)
  AS cropped,
  ST_DumpValues(clip(r, geometry 'SRID=4326;POLYGON((0 1,2 1,2 3,0 3,0 1))', false), 1)
  AS kept_extent
FROM rast;
-- {{10,20},{40,50}} | {{10,20,NULL},{40,50,NULL},{NULL,NULL,NULL}}
```

- Return a raster stated in another spatial reference system

```
transform(raster,srid integer,algorithm text='NearestNeighbour',maxerr float=0.125,
  scalex float=0,scaley float=0) → raster
transform(raster,alignto raster,algorithm text='NearestNeighbour',
  maxerr float=0.125) → raster
transformPipeline(raster,pipeline text,srid integer=0,is_forward boolean=true,
  algorithm text='NearestNeighbour',maxerr float=0.125,scalex float=0,
  scaley float=0) → raster
```

Every band is carried into the target system and resampled onto the grid the reprojection implies, through the same GDAL warp as the PostGIS `ST_Transform`, so the result states the same coverage read through another system. The algorithm is one of `NearestNeighbour`, `Bilinear`, `Cubic`, `CubicSpline`, `Lanczos`, `Max` and `Min`, named without regard to case, and `maxerr` is the error in input pixels the warp may commit, 0 for an exact calculation. `scalex` and `scaley` state the pixel size of the result, 0 letting the warp derive it, and `alignto` takes the reference system, the pixel size, the grid origin and the skew of another raster, so the result lies on its grid.

The `transformPipeline` function carries the raster through a coordinate operation, stated as a PROJ string, a WKT or an authority code such as `urn:ogc:def:coordinateOperation:EPSG::16031`, instead of the one GDAL derives from the two systems, and reads it as the geometry `transformPipeline` does: longitude first and in degrees. The result states the reference system given by `srid`, 0 by default. By default the operation is applied forward; setting `is_forward` to false applies its inverse. The remaining arguments are those of `transform`, so an operation GDAL would derive answers as `transform` to the same system.

```
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1 degree pixels)
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, -9999::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT SRID(transform(r, 3857)) AS srid, width(transform(r, 3857)) AS width,
  height(transform(r, 3857)) AS height
FROM rast;
-- 3857 | 3 | 3
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1 degree pixels)
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, -9999::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT SRID(transformPipeline(r, '+proj=utm +zone=31 +ellps=WGS84', 32631)) AS srid,
  width(transformPipeline(r, '+proj=utm +zone=31 +ellps=WGS84', 32631)) AS width,
  height(transformPipeline(r, '+proj=utm +zone=31 +ellps=WGS84', 32631)) AS height
FROM rast;
-- 32631 | 3 | 3
```

- Return a raster resampled to another pixel size

```
rescale(raster,scalex float,scaley float,algorithm text='NearestNeighbour',
  maxerr float=0.125) → raster
```

The result keeps the reference system and the upper left corner of the raster and states its coverage on a grid of the pixel size asked for, through the same GDAL warp as the PostGIS `ST_Rescale`, so a coarser scale reads fewer pixels over the same ground. The scales are in the units of the raster's reference system, and the algorithm and the error are those of transform.

```
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1 degree pixels)
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, -9999::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT width(rescale(r, 0.5, -0.5)) AS width,
  height(rescale(r, 0.5, -0.5)) AS height,
  scaleX(rescale(r, 0.5, -0.5)) AS scalex
FROM rast;
-- 6 | 6 | 0.5
```

- Return the statistics of the pixels of a raster band

```
summaryStats(raster,nband integer=1,exclude_nodata_value boolean=true) → summarystats
```

The band is read once and answers the PostGIS `summarystats` record that `ST_SummaryStats` answers: the number of pixels counted, and the sum, mean, standard deviation, minimum and maximum of their values. Every pixel is read, so the answer is exact rather than sampled; the pixels the band states as nodata are left out unless `exclude_nodata_value` is false, and a band with no pixel to count answers a count of zero and no statistic.

```
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1 degree pixels)
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, -9999::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT (summaryStats(r)).* FROM rast;
-- 9 | 450 | 50 | 25.81988897471611 | 10 | 90
```

- Return the statistics of the pixels of a band over the rasters of a column

```
summaryStatsAgg(raster,nband integer,exclude_nodata_value boolean) → summarystats
summaryStatsAgg(raster,exclude_nodata_value boolean) → summarystats
```

The aggregate of PostGIS's `ST_SummaryStatsAgg`, answering the `summarystats` record that summaryStats answers for one raster over every pixel of the band of the rasters, the band 1 where `nband` is left out. Every pixel is read, as for `summaryStats`, so the forms take no sample percentage and the standard deviation is the one of every pixel rather than of a sample. A `NULL` raster is skipped, over no raster the aggregate answers `NULL`, and rasters with no pixel to count answer a count of zero and no statistic. The aggregate runs in parallel, the statistics of the workers combined, where PostGIS's has no combine function. A band a raster does not have raises an error, where PostGIS skips the raster.

```
WITH tiles AS (
  SELECT ST_AddBand(ST_MakeEmptyRaster(2, 2, 0.0, 2.0, 1.0, -1.0, 0.0, 0.0, 3857),
    '32BF'::text, 1.0::float8, -9999.0::float8) AS r
  UNION ALL
  SELECT ST_AddBand(ST_MakeEmptyRaster(2, 2, 1.0, 1.0, 1.0, -1.0, 0.0, 0.0, 3857),
    '32BF'::text, 3.0::float8, -9999.0::float8) )
SELECT (summaryStatsAgg(r, 1, true)).* FROM tiles;
-- 8 | 16 | 2 | 1 | 1 | 3
```

- Return the number of pixels, the sum, the mean, the standard deviation, the minimum or the maximum of the statistics of a raster band

```
summarystatsCount(summarystats) → bigint
summarystatsSum(summarystats) → float
summarystatsMean(summarystats) → float
summarystatsStddev(summarystats) → float
summarystatsMin(summarystats) → float
summarystatsMax(summarystats) → float
```

Each function reads one field of the `summarystats` record that summaryStats and summaryStatsAgg return, the form in which every engine reads it (Chapter 18), beside the reading of a field by its name, `(summaryStats(r)).mean`, that PostgreSQL keeps. Over statistics that count no pixel the count is 0 and the other functions return `NULL`.

```
-- Build a 3x3 synthetic elevation raster (SRID 4326, 1 degree pixels)
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, -9999::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT summarystatsCount(summaryStats(r)), summarystatsMean(summaryStats(r)),
  summarystatsStddev(summaryStats(r)) FROM rast;
-- 9 | 50 | 25.81988897471611
```

- Return the polygons of a raster band, one for each group of pixels sharing a value {}

```
dumpAsPolygons(raster,band integer=1,exclude_nodata_value boolean=true) → {geomval}
```

A band states a value per pixel, and this states the same band as the regions those values cover, answering the PostGIS `geomval` records that `ST_DumpAsPolygons` answers. Each polygon carries the reference system of the raster, where `ST_DumpAsPolygons` states none, so it can be compared with the trajectories the coverage is read along. The pixels the band states as nodata are left out unless `exclude_nodata_value` is false, a band whose every pixel is nodata covers nothing and answers no row, and a band the raster does not have raises an error.

```
-- Classify a 3x3 synthetic elevation raster, then read each class as a region
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, -9999::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT val, ST_AsText(geom)
FROM rast, dumpAsPolygons(reclass(r, 1, '0-50:1, [50-100]:2', '8BUI', 0))
ORDER BY val;
-- 1 | POLYGON((0 3,0 1,1 1,1 2,3 2,3 3,0 3))
-- 2 | POLYGON((1 2,1 1,0 1,0 0,3 0,3 2,1 2))
```

- Return the pixels of a raster band as points {}

```
pixelAsPoints(rast raster,band integer=1,exclude_nodata_value boolean=true)
  → setof (geom geometry,val float,x integer,y integer)
```

The points of PostGIS's `ST_PixelAsPoints`: for each pixel, row by row, its upper-left corner in the reference system of the raster, its value, and its column and row, starting at 1. The pixels the band states as nodata are left out unless `exclude_nodata_value` is false, in which case they keep the value the band holds for them, and a band stating no nodata value leaves none out. A band the raster does not have raises an error, where PostGIS answers no row.

```
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(3, 2, 10.0, 20.0, 2.0, -1.0, 0.0, 0.0,
    3857), '32BF'::text, 0.0::float8, -9999.0::float8), 1, 1, 1,
    ARRAY[[1.5, -9999, 3], [4, 5, 6]]::double precision[][]) AS r )
SELECT ST_AsText(geom), val, x, y FROM rast, pixelAsPoints(r) LIMIT 2;
-- POINT(10 20) | 1.5 | 1 | 1
-- POINT(14 20) |   3 | 3 | 1
```

- Return the values of the bands of a raster as arrays

```
dumpValues(rast raster,nband integer[]=NULL,exclude_nodata_value boolean=true)
  → setof (nband integer,valarray float[])
dumpValues(rast raster,nband integer,exclude_nodata_value boolean=true) → float[]
```

The values of PostGIS's `ST_DumpValues`: an array per band, a row of the array per row of the raster, in the order of the bands given in `nband`, every band when it is `NULL`; the form with one band answers its array alone. A pixel the band states as nodata is `NULL` unless `exclude_nodata_value` is false, in which case it is the value the band holds for it. A band the raster does not have, or a `NULL` band number, raises an error, where PostGIS answers no row or skips it.

```
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand(ST_MakeEmptyRaster(3, 2, 10.0, 20.0, 2.0, -1.0, 0.0, 0.0,
    3857), '32BF'::text, 0.0::float8, -9999.0::float8), 1, 1, 1,
    ARRAY[[1.5, -9999, 3], [4, 5, 6]]::double precision[][]) AS r )
SELECT dumpValues(r, 1), dumpValues(r, 1, false) FROM rast;
-- {{1.5,NULL,3},{4,5,6}} | {{1.5,-9999,3},{4,5,6}}
```

- Sample a `raquet` raster tile along a trajectory

```
rasterTileValue(tgeompoint,rast raquet) → tfloat
```

Reads the tile along `traj` (SRID 4326) as rasterValue reads a raster, returning the pixel values as a `tfloat`. This is the typed equivalent of rasterTileValueQuadbin: the dimensions, QUADBIN cell, pixel type and nodata handling are taken from the `raquet` value instead of from separate arguments. Returns `NULL` when the trip never meets a pixel of the tile carrying data.

```
-- Sample pre-packaged raquet tiles along ship trajectories
SELECT t.tripid, rasterTileValue(t.trip, r.tile)
FROM trips t JOIN elevation_tiles r
  ON r.quadbin IN (SELECT unnest(getValues(tquadbin(t.trip, 8))));
```

- Sample an array of `raquet` raster tiles along a trajectory

```
rasterTileValue(tgeompoint,rast raquet[]) → tfloat
```

Reads every tile of `rast` along `traj` (SRID 4326) and returns the pixel values as a single `tfloat`. A trajectory that leaves one tile is covered by several of them, each answering the part of the trip over it. Tiles of one zoom level partition the plane, so they answer disjoint times. Tiles of different zoom levels overlap, and where two of them answer the same time the value of the tile of higher zoom is kept, that being the one carrying the finer resolution. Returns `NULL` when the trip never meets a pixel of a tile carrying data.

```
-- Sample a trajectory across every tile it crosses, in one value
SELECT t.tripid, rasterTileValue(t.trip, array_agg(r.tile)) FROM trips t
JOIN elevation_tiles r ON r.quadbin IN (SELECT unnest(getValues(tquadbin(t.trip, 8))))
GROUP BY t.tripid, t.trip;
```

- Sample a Raquet raster chip along a trajectory using a QUADBIN cell for georeferencing

```
rasterTileValueQuadbin(tgeompoint,pixels bytea,width integer,height integer,
  quadbin,pixtype text,nodata float,has_nodata boolean) → tfloat
```

Reads a row-major single-band pixel array along `traj` (SRID 4326), as rasterValue reads a raster. The tile georeferencing (bounding box, pixel-to-coordinate mapping) is derived from `quadbin` alone via the Web-Mercator slippy-tile transform. The `pixtype` argument must be one of `uint8`, `int8`, `uint16`, `int16`, `uint32`, `int32`, `uint64`, `int64`, `float16`, `float32`, or `float64`. When `has_nodata` is true, a pixel equal to `nodata` carries no value. Returns `NULL` when the trip never meets a pixel carrying data.

```
-- Sample the 256 x 256 float32 elevation tiles of a Raquet table, stating no
-- nodata value, along every ship trajectory
SELECT t.tripid, rasterTileValueQuadbin(t.trip, r.band_1, 256, 256,
  r.block::quadbin, 'float32', 0, false) FROM trips t
JOIN elevation_raquet r
  ON r.block IN (SELECT unnest(getValues(tquadbin(t.trip, 8)))::bigint);
-- Average sea-surface temperature per trajectory
SELECT t.tripid, avgValue(rasterTileValueQuadbin(t.trip,
  r.band_1, 256, 256, r.block::quadbin, 'int16', -9999, true)) AS mean_sst FROM trips t
JOIN sst_raquet r ON r.block IN (SELECT unnest(getValues(tquadbin(t.trip, 6)))::bigint);
```

- Return the part of a trajectory where the raster value it reads falls inside a float range

```
atRasterValue(tgeompoint,rast raster,vspan floatspan,band integer=1) → tgeompoint
atRasterValue(tgeompoint,path text,vspan floatspan,band integer=1) → tgeompoint
```

Reads `rasterValue` along `traj` and returns the sub-trajectory restricted to the times when the pixel value lies within `vspan`. Returns `NULL` when no time satisfies the condition. The form taking `path` reads a raster file on the server, as the corresponding form of rasterValue does.

```
WITH rast AS (
  SELECT ST_SetValues(ST_AddBand( ST_MakeEmptyRaster(3, 3, 0.0, 3.0, 1.0, -1.0, 0.0, 0.0,
    4326), '32BF'::text, 0.0::float8, NULL::float8), 1, 1, 1,
    ARRAY[[10.0::float4, 20.0::float4, 30.0::float4],
    [40.0::float4, 50.0::float4, 60.0::float4],
    [70.0::float4, 80.0::float4, 90.0::float4]]) AS r )
SELECT asText(atRasterValue(tgeompoint 'SRID=4326;
  [POINT(0.5 2.5)@2001-01-01, POINT(1.5 1.5)@2001-01-02, POINT(0.5 0.5)@2001-01-03]', r,
  floatspan '[40, 90]')) FROM rast;
/* {[POINT(1 2)@2001-01-01 12:00:00, POINT(1.5 1.5)@2001-01-02,
   POINT(0.5 0.5)@2001-01-03]} */
```

- Return the part of a trajectory where the raster value it reads falls outside a float range

```
minusRasterValue(tgeompoint,rast raster,vspan floatspan,band integer=1) → tgeompoint
minusRasterValue(tgeompoint,path text,vspan floatspan,band integer=1) → tgeompoint
```

The complement of `atRasterValue`: returns the sub-trajectory restricted to the times when the pixel value lies outside `vspan`. Returns `NULL` when the pixel value lies within `vspan` at every time. The form taking `path` reads a raster file on the server, as the corresponding form of rasterValue does.

```
-- Keep only the instant whose raster value (10) is below 40
SELECT asText(minusRasterValue(traj, elev_raster, floatspan '[40, 9999]')) FROM trips,
  elevation_raster;
```

- Return true if the trajectory ever reads a raster pixel value inside a float range

  ```
  eRasterValue(tgeompoint,rast raster,vspan floatspan,band integer=1) → boolean
  eRasterValue(tgeompoint,path text,vspan floatspan,band integer=1) → boolean
  ```

  Returns `true` when `traj` ever reads a pixel value within `vspan`, `false` otherwise. Returns `NULL` only when the input is `NULL`. The form taking `path` reads a raster file on the server, as the corresponding form of rasterValue does.

  ```
  -- Trips that ever crossed terrain above 200 m
  SELECT tripid FROM trips,
    elevation_raster WHERE eRasterValue(trip, elev_raster, floatspan '[200, 9999]');
  ```

- Return true if every pixel value the trajectory reads over the raster lies inside a float range

  ```
  aRasterValue(tgeompoint,rast raster,vspan floatspan,band integer=1) → boolean
  aRasterValue(tgeompoint,path text,vspan floatspan,band integer=1) → boolean
  ```

  Returns `true` when every pixel value `traj` reads over the raster lies within `vspan`, `false` when at least one does not. Returns `NULL` only when the input is `NULL`. The form taking `path` reads a raster file on the server, as the corresponding form of rasterValue does.

  ```
  -- Trips that stayed entirely below 100 m elevation
  SELECT tripid FROM trips,
    elevation_raster WHERE aRasterValue(trip, elev_raster, floatspan '[0, 100]');
  ```

To use the operators described here, MobilityDB must be built with `-DQUADBIN=ON` and `-DRASTER=ON`, since a Raquet tile is identified by a `quadbin` cell. On such a build the generated `mobilitydb.control` declares `requires = 'postgis, postgis_raster'`, so a single `CASCADE` creates the full stack:

```
CREATE EXTENSION mobilitydb CASCADE;
-- NOTICE:  installing required extension "postgis"
-- NOTICE:  installing required extension "postgis_raster"
```

PostGIS raster is part of the standard `postgresql-N-postgis-3` package on Debian/Ubuntu; no extra package is required. Pass `-DQUADBIN=ON` and `-DRASTER=ON` to CMake:

```
cmake -DQUADBIN=ON -DRASTER=ON ..
make -j$(nproc)
sudo make install
```

The CI builds, `.github/workflows/pgversion.yml` among them, pass `-DALL=ON`, which includes raster alongside the other optional families.

Chapter 17

Temporal Point Clouds

The pgPointCloud extension stores LIDAR point cloud data in PostgreSQL using two static types: `pcpoint`, a single multi-dimensional point whose dimensions are described by a *schema* stored in the `pointcloud_formats` catalog table; and `pcpatch`, a compressed batch of `pcpoint` values that share the same `pcid` (schema id). Each schema fixes the dimensions present (`X`, `Y`, `Z`, `Intensity`, . . .), their numeric type, scale, and offset.

MobilityDB lifts these types into the temporal world via `tpcpoint` (a moving LIDAR/GPS sensor) and `tpcpatch` (a time series of compressed point clusters). It also adds set types `pcpointset` / `pcpatchset` and the spatiotemporal bounding-box type `tpcbox`. A complete reference for the static `pcpoint` and `pcpatch` types is given in the pgPointCloud documentation; this chapter covers MobilityDB's additions on top of pgPointCloud.

To use the types described here, MobilityDB must be built with the `POINTCLOUD=ON` CMake option. On such a build the generated `mobilitydb.control` declares `requires = 'postgis, pointcloud'`, so a single `CASCADE` creates the full stack:

```
CREATE EXTENSION mobilitydb CASCADE;
-- NOTICE:  installing required extension "postgis"
-- NOTICE:  installing required extension "pointcloud"
```

Creating the extensions by hand in the wrong order, without `CASCADE`, raises a missing-type error.

Both `tpcpoint` and `tpcpatch` support the standard MobilityDB binary I/O through `asBinary` / `tpcpointFromBinary` / `tpcpatchFromBinary` and through PostgreSQL's `COPY ... (FORMAT BINARY)`. The encoding carries the `pcid` of each value beside the dimension payload and, whenever the encoding backend knows the schema XML for that `pcid`, the schema itself, so a value dumped from one cluster imports into another that has never seen the `pcid`.

17.1 Point Cloud Schemas

Every `pcpoint` and `pcpatch` value carries a `pcid` that resolves to a *schema*, which states the dimensions the value holds (X, Y, Z, . . .), the numeric type each is stored as, and its scale and offset. The schema is stated in SQL, in two tables: `pointcloud_schemas` holds what a schema states as a whole, and `pointcloud_dimensions` one row per dimension. A minimal 3D schema, every dimension stored as `int32_t` at unit scale:

```
INSERT INTO pointcloud_schemas (pcid, srid, compression, description)
VALUES (1, 4326, 'none', 'Minimal 3D schema');
INSERT INTO pointcloud_dimensions (pcid, dim_no, dim_name, interpretation, dim_scale,
  dim_offset, active, description)
VALUES (1, 1, 'X', 'int32_t', 1, 0, true, 'Easting'),
  (1, 2, 'Y', 'int32_t', 1, 0, true, 'Northing'),
  (1, 3, 'Z', 'int32_t', 1, 0, true, 'Elevation');
```

The size of each dimension and its offset within a point are absent because they follow from the interpretation, and the X, Y, Z and M dimensions are absent because they resolve from the names. The `dim_no` column states the order the dimensions are stored in, so it does not depend on the order the rows are inserted in, and `dim_name` is the name they resolve by. No column name needs quoting in any host these tables are materialised in. A schema is global to the database and usually registered once, at fixture-load time; the SRID is per schema and is inherited by every value carrying that `pcid`.

Registering a schema in the two tables also writes its XML document into the `pointcloud_formats` catalog of pgPointCloud, and a change to their rows rewrites or removes it, so the `pcpoint` and `pcpatch` types and the `PC_` functions of pgPointCloud read the schema as well. The document is written once the schema is one pgPointCloud accepts, that is, once it states an X and a Y dimension and a dimension at every position, for a `pcid` below 65536. For a `pcid` the two tables state, the document in `pointcloud_formats` is the one their rows write.

A database whose schemas are already registered in the `pointcloud_formats` catalog of pgPointCloud, as an XML document, keeps working: that catalog is read for a `pcid` that the two tables above do not state.

A change to the rows of these tables, or of `pointcloud_formats`, reaches every statement that starts after it: a statement builds and reads its values through the schemas their rows state when it starts, as pgPointCloud does.

Mixing schemas is rejected at construction time: a `tpcpoint` sequence whose instants carry different `pcid` values cannot be built, and the same holds for a `pcpointset` and a `pcpatchset`. The schema is what gives the dimensions their meaning, so a value mixing schemas would carry coordinates nothing can interpret.

Three helpers answer what a `pcid` names without a value of that schema in hand and without depending on the table layout. They resolve the schema as a value of it is read, from the two tables above and else from `pointcloud_formats`, so a `pcid` pgPointCloud states is answered as one the tables state. Each answers `NULL` for a `pcid` no schema states, and `pointCloudSchemaSRID` answers 0 for a schema stating no reference system.

- Return the spatial reference system every value of a schema is expressed in

```
pointCloudSchemaSRID(integer) → integer
```

```
SELECT pointCloudSchemaSRID(1);
-- 4326
```

- Return how a patch of a schema is stored

```
pointCloudSchemaCompression(integer) → text
```

```
SELECT pointCloudSchemaCompression(1);
-- none
```

- Return how many dimensions of a schema hold values

```
pointCloudSchemaNDims(integer) → integer
```

```
SELECT pointCloudSchemaNDims(1);
-- 3
```

17.2 Static Point Clouds

The static types come from pgPointCloud. MobilityDB exposes the schema-aware coordinate accessors needed to project pgPointCloud values into the rest of the temporal-types ecosystem.

17.2.1 Input and Output

The text of a `pcpoint` and of a `pcpatch` is the hex of its pgPointCloud Well-Known Binary (WKB), the text the type input and output functions of pgPointCloud read and write, and every engine reads and writes it with the functions below.

- Return the Hexadecimal Well-Known Binary (HexWKB) representation of a pcpoint or a pcpatch

  ```
  asHexWKB({pcpoint,pcpatch}) → text
  ```

  The result is the text the type output function of pgPointCloud writes, in the byte order of the machine.

  ```
  SELECT asHexWKB(pcpoint(1, 1.0, 2.0, 3.0));
  -- 010100000064000000C80000002C010000
  SELECT asHexWKB(pcpatch(pcpoint(1, 1.0, 1.0, 1.0), pcpoint(1, 2.0, 2.0, 2.0)));
  -- 01010000000000000002000000640000006400000064000000C8000000C8000000C8000000
  ```

- Input of a pcpoint or a pcpatch from its Hexadecimal Well-Known Binary (HexWKB) representation

  ```
  pcpointFromHexWKB(text) → pcpoint
  pcpatchFromHexWKB(text) → pcpatch
  ```

  The input is read as the type input function of pgPointCloud reads it, in either byte order, under the schema of its `pcid`.

  ```
  SELECT getZ(pcpointFromHexWKB('010100000064000000C80000002C010000'));
  -- 3
  SELECT numPoints(pcpatchFromHexWKB('01010000000000000002000000640000006400000064000000 ↩
      C8000000C8000000C8000000'));
  -- 2
  ```

17.2.2 Constructors

pgPointCloud's `pcpoint_in` only accepts hex-WKB, so the canonical builder is `PC_MakePoint(pcid, ARRAY[...]::float[])`. MobilityDB ships two thin SQL overloads on top of it for shorter test literals and ad-hoc queries; the underlying schema-dimension validation still happens inside `PC_MakePoint`.

- Build a 2D or 3D pcpoint directly from x/y(/z) coordinates

  ```
  pcpoint(pcid integer,x float,y float) → pcpoint
  pcpoint(pcid integer,x float,y float,z float) → pcpoint
  ```

  ```
  SELECT pcpoint(1, 10.0, 20.0, 30.0);
  -- Equivalent to pcpoint(1, 10.0, 20.0, 30.0)
  ```

- Build a pcpoint from the coordinates of every dimension of the schema

  ```
  pcpoint(pcid integer,coordinates float[]) → pcpoint
  ```

  The layout of the schema states every dimension, inactive ones included, which is how a schema of more dimensions than the overloads above reach is given its values. That count is the one `PC_MakePoint` takes and is NOT the count of active dimensions that `pointCloudSchemaNDims` reports

  ```
  SELECT pcpoint(1, ARRAY[10.0, 20.0, 30.0]::float[]);
  ```

- Build a pcpatch from the coordinates of its points, one point after another

  ```
  pcpatch(pcid integer,coordinates float[]) → pcpatch
  ```

  Every dimension of the layout of the schema is given for each point, inactive ones included. That count is the one `PC_MakePatch` takes and is NOT the count of active dimensions that `pointCloudSchemaNDims` reports; a count that divides evenly by the wrong one of the two builds a patch of the wrong points rather than reporting an error. A patch built this way states the schema itself, there being no point to read it from

  ```
  SELECT pcpatch(1, ARRAY[1.0, 1.0, 1.0, 2.0, 2.0, 2.0]::float[]);
  ```

- Build a pcpatch from a variadic list of pcpoints sharing the same pcid

```
pcpatch(VARIADIC points pcpoint[]) → pcpatch
```

```
SELECT pcpatch(pcpoint(1, 1.0, 1.0, 1.0), pcpoint(1, 2.0, 2.0, 2.0));
-- Equivalent to pcpatch(pcpoint(1, 1.0, 1.0, 1.0), pcpoint(1, 2.0, 2.0, 2.0))
```

17.2.3 Accessors

- Return the schema id of a pcpoint or pcpatch

```
pcid({pcpoint,pcpatch}) → integer
```

```
SELECT pcid(pcpoint(1, 10.0, 20.0, 30.0));
-- 1
```

- Return a coordinate of a pcpoint, NULL if the dimension is absent from the schema

```
getX(pcpoint) → float
getY(pcpoint) → float
getZ(pcpoint) → float
```

```
WITH pt AS (SELECT pcpoint(1, 10.0, 20.0, 30.0) p)
  SELECT getX(p), getY(p), getZ(p) FROM pt;
-- 10 | 20 | 30
```

- Return any named dimension of a pcpoint, NULL if the name is unknown for the schema

```
getDim(pcpoint,text) → float
```

```
SELECT getDim(pcpoint(1, 10.0, 20.0, 30.0), 'X');
-- 10
```

- Return the spatial reference identifier (inherited from the schema)

```
SRID({pcpoint,pcpatch}) → integer
```

```
SELECT SRID(pcpoint(1, 10.0, 20.0, 30.0));
-- 0
SELECT SRID(pcpatch(pcpoint(1, 10.0, 20.0, 30.0), pcpoint(1, 11.0, 21.0, 31.0)));
-- 0
```

- Return the number of points of a pcpatch

```
numPoints(pcpatch) → integer
```

```
SELECT numPoints(pcpatch(pcpoint(1, 1.0, 1.0, 1.0), pcpoint(1, 2.0, 2.0, 2.0)));
-- 2
```

- Return the n-th point of a pcpatch

```
pointN(pcpatch,integer) → pcpoint
```

  Points are numbered from one, a negative integer counts from the end of the patch, and an integer addressing no point returns NULL.

```
SELECT pointN(pcpatch(pcpoint(1, 1.0, 1.0, 1.0), pcpoint(1, 2.0, 2.0, 2.0)), 2)
  = pcpoint(1, 2.0, 2.0, 2.0);
-- true
SELECT pointN(pcpatch(pcpoint(1, 1.0, 1.0, 1.0), pcpoint(1, 2.0, 2.0, 2.0)), 3)
  IS NULL;
-- true
```

- Return the multipoint of the positions the points of a patch occupy

```
geometry(pcpatch) → geometry
geography(pcpatch) → geography
```

The schema the `pcid` names decides the SRID and whether the result carries a Z dimension

```
SELECT ST_AsText(geometry(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2))));
-- MULTIPOINT Z ((1 1 1),(2 2 2))
```

17.3 Point Cloud Sets

A `pcpointset` is an ordered set of distinct `pcpoint` values; a `pcpatchset` is the analogous set type for `pcpatch`. Both follow MobilityDB's general set semantics (see Chapter 2): values are deduplicated on construction, the binary representation is canonical, and all generic set-level operators (`=`, `<>`, `<`, `<=`, `>`, `>=`, `@>`, `<@`, `&&`, `+`, `-`, `*`) and accessors (`numValues`, `memSize`, `asText`, `asEWKT`, `asBinary`, `asEWKB`, `asHexWKB`, `asHexEWKB`, ...) are available (see the binary representations of a spatial set). The extended forms state the SRID the schema of the `pcid` declares in the `pointcloud_formats` catalog, so they coincide with the plain ones when that schema declares no SRID.

All values in a set must share the same `pcid`. Mixing schemas in a single set raises an error at construction time.

17.3.1 Constructors

- Construct a set from an array of values, all of which must share the same pcid

```
set(pcpoint[]) → pcpointset
set(pcpatch[]) → pcpatchset
```

```
SELECT numValues(set(ARRAY[pcpoint(1, 10.0, 20.0, 30.0),
  pcpoint(1, 10.0, 20.0, 30.0),  -- duplicate, deduped
  pcpoint(1, 11.0, 21.0, 31.0)]));
-- 2
```

17.4 Point Cloud Bounding Boxes

The `tpcbox` type is the spatiotemporal bounding box for the temporal point-cloud types. It mirrors the structure of `stbox` (see Chapter 3), same X/Y/Z/T axes, same flag bits for which dimensions are present, and adds a single field: the `pcid`. Two `tpcbox` values with different `pcid` values describe different schemas, and an operation between them raises an error instead of comparing their geometric or temporal extent.

Carrying the `pcid` through bounding-box arithmetic guarantees that bounding-box pruning never drops a row whose schema differs from the query's schema, and never returns a hit across mismatched schemas.

17.4.1 Input and Output

- Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation of a point cloud box

  ```
  asBinary(tpcbox,endian text='') → bytea
  asHexWKB(tpcbox,endian text='') → text
  ```

  The result is the representation of the `stbox` the box spans followed by its `pcid`, encoded using either the little-endian (NDR) or the big-endian (XDR) encoding, the encoding of the machine when none is specified. As for every spatial type, the plain representation does not write the SRID, which `asEWKB` and `asHexEWKB` write.

  ```
  SELECT asHexWKB(tpcboxX(0, 0, 10, 10, 1));
  -- 0101000000000000000000000000000024400000000000000000000000000000244001000000
  SELECT asHexWKB(tpcboxX(0, 0, 10, 10, 5), 'XDR');
  -- 0001000000000000000040240000000000000000000000000000000040240000000000000000000005
  ```

- Return the Extended Well-Known Binary (EWKB) or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation of a point cloud box

  ```
  asEWKB(tpcbox,endian text='') → bytea
  asHexEWKB(tpcbox,endian text='') → text
  ```

  The result is the representation `asBinary` and `asHexWKB` write with the SRID of the box, the one the schema of its `pcid` states, as every spatial type writes its SRID in its extended representation.

  ```
  SELECT asHexEWKB(tpcboxX(0, 0, 10, 10, 1));
  -- 0101000000000000000000000000000024400000000000000000000000000000244001000000
  SELECT asHexEWKB(tpcboxX(0, 0, 10, 10, 5), 'XDR');
  -- 0041000010E6000000000000000040240000000000000000000000000000000040240000000000000000000005
  ```

- Input of a point cloud box from the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation

  ```
  tpcboxFromBinary(bytea) → tpcbox
  tpcboxFromHexWKB(text) → tpcbox
  ```

  The SRID of the result is the one the schema of its `pcid` states, which the plain representation does not write.

  ```
  SELECT asText(tpcboxFromBinary(asBinary(tpcboxX(0, 0, 10, 10, 5))));
  -- SRID=4326;TPCBOX(X((0,0),(10,10)), 5)
  ```

- Input of a point cloud box from the Extended Well-Known Binary (EWKB) or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

  ```
  tpcboxFromEWKB(bytea) → tpcbox
  tpcboxFromHexEWKB(text) → tpcbox
  ```

  The SRID the input states must be the one the schema of its `pcid` states, when that schema states one.

  ```
  SELECT tpcboxFromHexEWKB(asHexEWKB(tpcboxX(0, 0, 10, 10, 1)));
  -- TPCBOX(X((0,0),(10,10)), 1)
  SELECT tpcboxFromHexEWKB(replace(asHexEWKB(tpcboxX(0, 0, 10, 10, 5)), 'E6100000', '110 ↩
    F0000'));
  -- ERROR:  Could not parse tpcbox value: The value states SRID 3857 and the schema of pcid ↩
      5 states SRID 4326
  ```

- Return the Well-Known Text (WKT) representation

  ```
  asText(tpcbox,maxdecimaldigits integer=15) → text
  ```

The `maxdecimaldigits` argument can be used to set the maximum number of decimal places in the output of floating point values (default 15). As for every spatial type, the text of a `tpcbox` does not write its SRID, which `asEWKT` writes.

```
SELECT asText(tpcbox 'TPCBOX(X((0.12,1.23),(2.35,3.46)), 1)');
-- TPCBOX(X((0.12,1.23),(2.35,3.46)), 1)
SELECT asText(tpcbox 'TPCBOX(X((0.12,1.23),(2.35,3.46)), 1)', 1);
-- TPCBOX(X((0.1,1.2),(2.4,3.5)), 1)
SELECT asText(tpcbox 'SRID=4326;TPCBOX(X((0,0),(10,10)), 5)');
-- TPCBOX(X((0,0),(10,10)), 5)
```

- Return the Extended Well-Known Text (EWKT) representation of a point cloud box

```
asEWKT(tpcbox,maxdecimaldigits integer=15) → text
```

The result is the text `asText` writes prefixed with the SRID of the box, as every spatial type writes its SRID in its extended representation.

```
SELECT asEWKT(tpcbox 'SRID=4326;TPCBOX(X((0,0),(10,10)), 5)');
-- SRID=4326;TPCBOX(X((0,0),(10,10)), 5)
```

- Input from the Well-Known Text (WKT) representation

```
tpcboxFromText(text) → tpcbox
```

The function is the inverse of `asText`: it reads the text a literal of the type is written in, from a constant or from a `text` column alike.

```
SELECT tpcboxFromText('TPCBOX(ZT(((0,0,0),(10,10,10)),[2024-01-01, 2024-01-02]), 1)');
-- TPCBOX(ZT(((0,0,0),(10,10,10)),[2024-01-01, 2024-01-02]), 1)
```

- Input of a point cloud box from the Extended Well-Known Text (EWKT) representation

```
tpcboxFromEWKT(text) → tpcbox
```

The function is the inverse of `asEWKT`, reading the SRID the text states.

```
SELECT tpcboxFromEWKT('SRID=4326;TPCBOX(X((0,0),(10,10)), 5)');
-- SRID=4326;TPCBOX(X((0,0),(10,10)), 5)
```

17.4.2 Constructors

- Construct a tpcbox from explicit X/Y/Z/T bounds and a pcid, whose schema states the SRID

```
tpcboxX(xmin float,ymin float,xmax float,ymax float,pcid integer=0) → tpcbox
tpcboxZ(xmin float,ymin float,zmin float,xmax float,ymax float,zmax float,
  pcid integer=0) → tpcbox
tpcboxT(period tstzspan,pcid integer=0) → tpcbox
tpcboxXT(xmin float,ymin float,xmax float,ymax float,period tstzspan,
  pcid integer=0) → tpcbox
tpcboxZT(xmin float,ymin float,zmin float,xmax float,ymax float,zmax float,
  period tstzspan,pcid integer=0) → tpcbox
```

The SRID is not a parameter: it is read from the schema the pcid names, and a pcid naming no registered schema is an error

```
SELECT tpcboxX(0, 0, 10, 10, 1);
-- TPCBOX(X((0,0),(10,10)), 1)
SELECT tpcboxZT(0, 0, 0, 10, 10, 10, tstzspan '[2024-01-01, 2024-01-02]', 1);
-- TPCBOX(ZT(((0,0,0),(10,10,10)),[2024-01-01, 2024-01-02]), 1)
```

17.4.3 Conversions

- Convert a pcpoint to a degenerate (zero-extent) tpcbox; SRID and pcid come from the schema

```
tpcbox(pcpoint) → tpcbox
```

```
SELECT tpcbox(pcpoint(1, 10.0, 20.0, 30.0));
-- TPCBOX(Z((10,20,30),(10,20,30)), 1)
```

- Convert a pcpatch to a tpcbox

```
tpcbox(pcpatch) → tpcbox
```

The X and Y extent of the result is the PCBOUNDS the patch embeds, and its Z extent, where the schema the pcid names holds a Z dimension, is the one the points of the patch state. The SRID is the one the schema the pcid names states.

```
SELECT tpcbox(pcpatch(pcpoint(1, 10.0, 20.0, 30.0)));
-- TPCBOX(Z((10,20,30),(10,20,30)), 1)
```

17.4.4 Accessors

- Return whether a tpcbox has the X/Y, Z, or T dimensions set

```
hasX(tpcbox) → boolean
hasZ(tpcbox) → boolean
hasT(tpcbox) → boolean
```

```
WITH b AS (SELECT tpcboxX(0, 0, 10, 10, 1) AS b)
  SELECT hasX(b), hasZ(b) FROM b;
-- t | f
```

- Return a coordinate bound, NULL if the corresponding dimension is absent

```
xmin(tpcbox) → float
xmax(tpcbox) → float
ymin(tpcbox) → float
ymax(tpcbox) → float
zmin(tpcbox) → float
zmax(tpcbox) → float
```

```
WITH b AS (SELECT tpcboxX(0, 0, 10, 10, 1) AS b)
  SELECT xmin(b), xmax(b) FROM b;
-- 0 | 10
```

- Return a temporal bound, NULL if the box has no T dimension

```
tmin(tpcbox) → timestamptz
tmax(tpcbox) → timestamptz
```

```
SELECT tmin(tpcboxT(tstzspan '[2024-01-01, 2024-01-02]', 1));
-- 2024-01-01
```

- Return the schema id and the SRID its schema states

```
pcid(tpcbox) → integer
SRID(tpcbox) → integer
```

```
WITH b AS (SELECT tpcboxX(0, 0, 10, 10, 1) AS b)
  SELECT pcid(b), SRID(b) FROM b;
-- 1 | 4326
```

17.4.5 Transformations

- Return a tpcbox with coordinates rounded to a given number of decimal digits

```
round(tpcbox,integer=0) → tpcbox
```

```
SELECT round(tpcboxX(0.123456, 1.234567, 2.345678, 3.456789, 1), 2);
-- TPCBOX(X((0.12,1.23),(2.35,3.46)), 1)
```

- Return a tpcbox stating a reference system its schema does not state

```
setSRID(tpcbox,integer) → tpcbox
```

The schema a pcid names is what holds the reference system, so where that schema states one the box is refused: a pcid of 0 names no schema, and a schema may state none. Does not reproject coordinates, for that, project the underlying tpcpoint first and take the bbox of the result

```
SELECT SRID(setSRID(tpcboxX(0, 0, 10, 10), 4326));
-- 4326
SELECT setSRID(tpcboxX(0, 0, 10, 10, 1), 3857);
-- ERROR: The SRID of a TPCBox is the one its schema states: pcid 1 states SRID 4326
```

17.4.6 Set Operations

- Union and intersection of two tpcboxes

```
tpcbox {+, *} tpcbox → tpcbox
```

Both operands must share the same `pcid`, and the intersection is NULL when the two boxes are disjoint.

```
SELECT tpcboxX(0, 0, 5, 5, 1) + tpcboxX(3, 3, 10, 10, 1);
-- TPCBOX(X((0,0),(10,10)), 1)
SELECT tpcboxX(0, 0, 5, 5, 1) * tpcboxX(3, 3, 10, 10, 1);
-- TPCBOX(X((3,3),(5,5)), 1)
```

Note: a generic box-vs-box `minus` is not provided, mirroring the convention for `stbox` and `tbox`: the difference of two boxes is not generally a box.

17.4.7 Topological Operations

All topological operators take two `tpcbox` values that state the same schema, the same reference system, and the same kind of coordinate, planar or spherical. A mismatch in either raises an error rather than answering false: the `pcid` is what gives a stored number its meaning as a coordinate, so two boxes naming different schemas have no common extent to compare.

They also take two boxes sharing an axis. A box holding coordinates and a box holding a period meet on no axis at all, so a topological predicate between them has nothing to compare and raises in its turn.

- Topological operators

```
tpcbox {&&, @>, <@, ~=, -|-} tpcbox → boolean
```

```
SELECT tpcboxX(0, 0, 10, 10, 1) @> tpcboxX(2, 2, 8, 8, 1);
-- t
SELECT tpcboxX(2, 2, 8, 8, 1) <@ tpcboxX(0, 0, 10, 10, 1);
-- t
SELECT tpcboxX(0, 0, 5, 5, 1) && tpcboxX(3, 3, 10, 10, 1);
-- t
```

```
SELECT tpcboxX(0, 0, 2, 2) ~= tpcboxX(0, 0, 2, 2);
-- t
SELECT tpcboxX(0, 0, 2, 2) -|- tpcboxX(5, 5, 7, 7);
-- f
SELECT tpcboxX(0, 0, 5, 5, 1) && tpcboxX(0, 0, 5, 5, 2);
-- ERROR:  Operation on TPCBox values with different schemas: 1 vs 2
SELECT tpcboxX(0, 0, 10, 10, 1) && tpcboxT(tstzspan '[2024-01-01, 2024-01-02]', 0);
-- ERROR:  The temporal values must have at least one common dimension
SELECT tpcbox 'SRID=4326;TPCBOX(X((1,1),(2,2)), 1)' &&
  tpcbox 'GEODTPCBOX(X((1,1),(2,2)), 1)';
-- ERROR:  Operation on mixed planar and geodetic coordinates
```

17.4.8 Position Operations

Sixteen axis-aligned position predicates, parallel to those on `stbox`. Each tests a strict-or-overlap relationship along one axis. The twelve that read coordinates require the same schema, the same reference system and the same kind of coordinate, and a mismatch in any of the three raises an error; the four that read the time axis require neither, a timestamp meaning the same thing whichever schema the coordinates are read in. Every predicate requires both boxes to carry the axis it reads, so the Z-axis ones need a Z dimension in both and the T-axis ones a time span in both.

- Position operators

```
tpcbox {<<, &<, >>, &>} tpcbox → boolean
tpcbox {<<|, &<|, |>>, |&>} tpcbox → boolean
tpcbox {<</, &</, />>, /&>} tpcbox → boolean
tpcbox {<<#, &<#, #>>, #&>} tpcbox → boolean
```

  The four groups test the X, the Y, the Z and the time axis, and each states, in order, strictly before, does not extend after, strictly after, and does not extend before along its axis.

```
SELECT tpcboxX(0, 0, 2, 2) << tpcboxX(5, 5, 7, 7);
-- t
SELECT tpcboxX(0, 0, 2, 2) <<| tpcboxX(5, 5, 7, 7);
-- t
-- Neither box carries a z dimension or a time span, so neither of the next two
-- has an axis to compare along. The last pair states two schemas, which the
-- time axis does not read.
SELECT tpcboxX(0, 0, 2, 2) <</ tpcboxX(5, 5, 7, 7);
-- ERROR:  The tpcbox must have Z dimension
SELECT tpcboxX(0, 0, 2, 2) <<# tpcboxX(5, 5, 7, 7);
-- ERROR:  The tpcbox must have T dimension
SELECT tpcboxXT(0, 0, 5, 5, tstzspan '[2024-01-01, 2024-01-15]', 1) <<#
  tpcboxXT(0, 0, 5, 5, tstzspan '[2025-01-01, 2025-01-15]', 2);
-- t
```

17.4.9 Comparisons

- Compare two tpcboxes; total order over the canonical encoding (pcid, srid, flags, period, then spatial bounds in XYZ-min/max order)

```
tpcbox {=, <>, <, <=, >, >=} tpcbox → boolean
```

```
SELECT tpcboxX(0, 0, 2, 2) = tpcboxX(0, 0, 2, 2);
-- t
SELECT tpcboxX(0, 0, 2, 2) <> tpcboxX(5, 5, 7, 7);
-- t
SELECT tpcboxX(0, 0, 2, 2) < tpcboxX(5, 5, 7, 7);
-- t
```

17.5 Temporal Point Cloud Points

A `tpcpoint` is the lifting of `pcpoint` through MobilityDB's temporal machinery. It supports the same three subtypes as the other temporal types, instant, sequence (with discrete or step interpolation), and sequence set. All instants in a single `tpcpoint` value must share the same `pcid`; the constructor enforces this.

The bounding box of a `tpcpoint` is a `tpcbox`, computed at construction time from the X/Y/Z dimensions of the underlying `pcpoint` values together with the timestamps of the instants.

We give next the functions and operators for temporal point cloud points. Most functions and operators for temporal types described in the previous chapters can be applied for temporal point cloud points. Therefore, in the signatures of the functions, the notation `base` represents a `pcpoint` and the notations `ttype` and `tpoint` also represent a `tpcpoint`. To avoid redundancy, we only present next some examples of these functions and operators for temporal point cloud points.

A column or domain may pin the schema by adding the `pcid` as a single-integer typmod, exactly like PostGIS' `geometry(Point, SRID)`. Mixing schemas in such a column is rejected at INSERT or cast time. The same construct applies to `tpcpatch`.

```
CREATE TABLE scans (id int, traj tpcpoint(1), full_scan tpcpatch(1));
-- Accepts pcid 1
INSERT INTO scans VALUES (1,
  tpcpoint(pcpoint(1, 1.0, 2.0, 3.0), '2024-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2024-01-01'::timestamptz));
-- Rejects any other pcid:
-- ERROR: Pcid of tpcpoint value (2) does not match column typmod pcid (1)
```

Omitting the typmod (`tpcpoint` with no parenthesised pcid) leaves the column schema-agnostic. The runtime check in the `extent` aggregate still rejects mid-query mixed-pcid input even on unconstrained columns.

17.5.1 Input and Output

- Return the text representation of a tpcpoint, or of an array of them, with or without its SRID, and cast a tpcpoint to text

  ```
  asText(tpcpoint) → text
  asText(tpcpoint[]) → text[]
  asEWKT(tpcpoint) → text
  asEWKT(tpcpoint[]) → text[]
  tpcpoint::text → text
  ```

  `asEWKT` prefixes the SRID the schema of the `pcid` declares, and writes the text form alone under a schema declaring none, as pcid 1 does below.

  ```
  -- The pcpoint payload renders as hex-encoded WKB.
  SELECT asText(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz));
  -- 0101000000640000006400000064000000@2001-01-01
  SELECT asEWKT(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz));
  -- 0101000000640000006400000064000000@2001-01-01
  ```

- Return the Moving Features JSON (MF-JSON) representation of a tpcpoint and read it back; the round trip is lossless

  ```
  asMFJSON(tpcpoint,options integer=0,flags integer=0,maxdecimaldigits integer=15) → text
  tpcpointFromMFJSON(text) → tpcpoint
  ```

  A `tpcpoint` is emitted as `{"type":"MovingPCPoint", ... "coordinates":[[X,Y,Z], ...], "values":[{"pcid":N,"pt":[...]}, ...], "datetimes":[...]}`. The coordinates are the X, Y and Z of each instant read from its `pcpoint` through the schema cache, and a schema stating neither X nor Y leaves the coordinate array of that instant empty. The values are pgPointCloud's text form of each point, every dimension of its schema in the order the schema states them and in the shortest decimal form that reads back as the same double, whatever `maxdecimaldigits` states. `tpcpointFromMFJSON` rebuilds each point from its values through the schema of its `pcid`, so a scaled integer dimension reads back as the integer it stores.

```
SELECT asMFJSON(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz));
/* {"type":"MovingPCPoint","coordinates":[[1,1,1]],"values":[{"pcid":1,"pt":[1,1,1]}],
   "datetimes":["2001-01-01T00:00:00"],"interpolation":"None"} */
SELECT tpcpointFromMFJSON(asMFJSON(t)) = t
FROM (SELECT tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz) AS t) s;
-- t
```

- Return the binary or hexadecimal well-known binary representation of a tpcpoint, with or without its SRID

```
asBinary(tpcpoint,endian text='') → bytea
asEWKB(tpcpoint,endian text='') → bytea
asHexWKB(tpcpoint,endian text='') → text
asHexEWKB(tpcpoint,endian text='') → text
```

```
SELECT tpcpointFromEWKB(asEWKB(t)) = t
FROM (SELECT tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz) AS t) s;
-- t
```

- Return a tpcpoint from its text, binary, or hexadecimal well-known binary representation, with or without its SRID

```
tpcpointFromText(text) → tpcpoint
tpcpointFromEWKT(text) → tpcpoint
tpcpointFromBinary(bytea) → tpcpoint
tpcpointFromEWKB(bytea) → tpcpoint
tpcpointFromHexWKB(text) → tpcpoint
tpcpointFromHexEWKB(text) → tpcpoint
```

  The `pcid` each point carries names its schema, so the text form reads back as the binary one does, and an SRID the input states must be the one that schema declares.

```
SELECT tpcpointFromText(asText(t)) = t
FROM (SELECT tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz) AS t) s;
-- t
SELECT tpcpointFromEWKT('SRID=4326;' ||
  asText(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz)));
-- ERROR:  The SRID of a pcpoint is the one its schema declares and cannot be set
```

`asText` takes no `maxdecimaldigits` argument, a `pcpoint` printing as the hex of its pgPointCloud WKB, the text the type output function of pgPointCloud writes, which has no decimal places to round. MF-JSON carries a `"bbox"` key when `options` is 1, a TPCBox JSON object adding a `"pcid"` field next to the standard stbox-shaped `"bbox"` array.

17.5.2 Constructors

- Constructor for temporal point cloud points having a constant value

```
tpcpoint(pcpoint,timestamptz) → tpcpointInst
tpcpoint(pcpoint,tstzset) → tpcpointDiscSeq
tpcpoint(pcpoint,tstzspan) → tpcpointContSeq
tpcpoint(pcpoint,tstzspanset) → tpcpointSeqSet
```

```
SELECT asText(tpcpoint(pcpoint(1, 1, 1, 1), timestamptz '2001-01-01'));
-- 0101000000640000006400000064000000@2001-01-01
SELECT asText(tpcpoint(pcpoint(1, 1, 1, 1), tstzspan '[2001-01-01, 2001-01-02]'));
-- [0101000000640000...64000000@2001-01-01, 0101000000640000...64000000@2001-01-02]
```

- Constructor for temporal point cloud points of sequence subtype

```
tpcpointSeq(tpcpointInst[],interp text='step',leftInc boolean=true,
  rightInc boolean=true) → tpcpointSeq
```

The interpolation is `'discrete'` or `'step'`; the dimensions a `pcpoint` carries do not interpolate linearly, so `'linear'` raises `The temporal type cannot have linear interpolation`.

```
SELECT tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1.0, 2.0, 3.0), '2024-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 4.0, 5.0, 6.0), '2024-01-02'::timestamptz)], 'step');
```

- Construct a temporal pcpoint of sequence-set subtype from an array of sequences

```
tpcpointSeqSet(tpcpoint[]) → tpcpointSeqSet
```

```
SELECT numSequences(tpcpointSeqSet(ARRAY[tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)])]));
-- 1
SELECT numInstants(tpcpointSeqSet(ARRAY[tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)])]));
-- 2
```

17.5.3 Conversions

- Convert a temporal point cloud point to a temporal geometry point

```
tpcpoint::{tgeompoint,tgeogpoint}
```

The conversions follow the rule of [?listitem].

Every non-spatial dimension is projected away and the timestamps are preserved. The result carries the SRID the schema states, and is three-dimensional when the schema states a Z dimension. The reverse conversion is not provided, since rebuilding a `pcpoint` needs a target schema that the temporal value alone does not carry.

```
SELECT ST_AsText(startValue(tpcpoint(pcpoint(1, 10.0, 20.0, 30.0),
  '2024-01-01'::timestamptz)::tgeompoint));
-- POINT Z (10 20 30)
```

- Convert a temporal point cloud point to its bounding box

```
tpcbox(tpcpoint) → tpcbox
tpcpoint::tpcbox
```

The box is the one the extent of the value answers: the spatial extent of its points, its time span, the SRID its schema states and its `pcid`.

```
SELECT tpcpoint(pcpoint(1, 10.0, 20.0, 30.0), '2024-01-01'::timestamptz)::tpcbox;
-- TPCBOX(ZT(((10,20,30),(10,20,30)),[2024-01-01, 2024-01-01]), 1)
```

17.5.4 Accessors

- Return the pgPointCloud schema id shared by all instants

```
pcid(tpcpoint) → integer
```

```
SELECT pcid(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz));
-- 1
```

- Return the SRID inherited from the schema

```
SRID({tpcpoint,tpcpatch}) → integer
```

```
-- The SRID is that of the schema registered for the pcid.
SELECT SRID(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz));
-- 0
SELECT SRID(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1)), '2001-01-01'::timestamptz));
-- 0
```

- Project a tpcpoint onto a spatial dimension, yielding a tfloat

```
getX(tpcpoint) → tfloat
getY(tpcpoint) → tfloat
getZ(tpcpoint) → tfloat
```

```
SELECT startValue(getX(tpcpoint(pcpoint(1, 10.0, 20.0, 30.0),
  '2024-01-01'::timestamptz)));
-- 10
```

- Project a tpcpoint onto any named schema dimension, yielding a tfloat

```
getDim(tpcpoint,text) → tfloat
```

```
SELECT getDim(tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)]), 'X');
-- Interp=Step;[1@2001-01-01, 2@2001-01-02]
```

17.5.5 Transformations

- Transform a tpcpoint to another subtype

```
tpcpointInst(tpcpoint) → tpcpoint
tpcpointSeq(tpcpoint,interp text='step') → tpcpoint
tpcpointSeqSet(tpcpoint) → tpcpoint
```

```
SELECT numInstants(tpcpointInst(tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz)));
-- 1
SELECT numInstants(tpcpointSeq(tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)])));
-- 2
```

- Transform a tpcpoint to another interpolation

```
setInterp(tpcpoint,interp text) → tpcpoint
```

```
-- A tpcpoint carries step interpolation.
SELECT interp(tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)]));
-- Step
SELECT interp(setInterp(tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)], 'discrete'), 'step'));
-- Step
```

17.5.6 Modifications

- Insert a tpcpoint into another one, or update another one with it

```
insert(tpcpoint,tpcpoint,connect boolean=true) → tpcpoint
update(tpcpoint,tpcpoint,connect boolean=true) → tpcpoint
```

```
SELECT numInstants(insert(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)]),
  tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)])));
-- 3
```

- Delete a time selector (timestamp, set, period, or spanset) from a tpcpoint

```
deleteTime(tpcpoint,times,connect boolean=true) → tpcpoint
```

```
-- Deleting the middle instant splits the sequence into {[2001-01-01, 2001-01-02),
-- (2001-01-02, 2001-01-03]}; each half stores its bound at the deleted timestamp, giving
-- four instants.
SELECT numInstants(deleteTime(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), timestamptz '2001-01-02'));
-- 4
```

- Append an instant or a sequence to a tpcpoint

```
appendInstant(tpcpoint,tpcpoint) → tpcpoint
appendSequence(tpcpoint,tpcpoint) → tpcpoint
```

```
SELECT numInstants(appendInstant(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)]),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)));
-- 3
```

- Merge the temporal values

```
merge(tpcpoint,tpcpoint) → tpcpoint
merge(tpcpoint[]) → tpcpoint
```

```
SELECT numInstants(merge(tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01':: ↩
    timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz)]),
  tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)])));
-- 3
SELECT numInstants(merge(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]));
-- 2
```

17.5.7 Restrictions

- Restrict a tpcpoint to (the complement of) a pcpoint value or a set of pcpoint values

```
atValue(tpcpoint,pcpoint) → tpcpoint
minusValue(tpcpoint,pcpoint) → tpcpoint
atValues(tpcpoint,pcpointset) → tpcpoint
minusValues(tpcpoint,pcpointset) → tpcpoint
```

```
-- With step interpolation the matching value is held on [2001-01-01, 2001-01-02); its
-- closing bound is stored as a second instant.
SELECT numInstants(atValue(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), pcpoint(1, 1, 1, 1)));
-- 2
SELECT numInstants(minusValue(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), pcpoint(1, 1, 1, 1)));
-- 2
```

- Restrict a tpcpoint to a time selector (timestamp, period, set, or spanset), or remove it

```
atTime(tpcpoint,times) → tpcpoint
minusTime(tpcpoint,times) → tpcpoint
```

```
SELECT numInstants(atTime(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), timestamptz '2001-01-02'));
-- 1
SELECT numInstants(atTime(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]),
  tstzspan '[2001-01-01, 2001-01-02]'));
-- 2
```

- Restrict a tpcpoint to the spatial / temporal extent of a tpcbox, or remove it

```
atTpcbox(tpcpoint,tpcbox,border_inc boolean=true) → tpcpoint
minusTpcbox(tpcpoint,tpcbox,border_inc boolean=true) → tpcpoint
```

Returns NULL when the tpcbox's pcid does not match the tpcpoint's. Internally projects to a tgeompoint via the schema cache, restricts the projection by the equivalent stbox, and lifts the result's time span back onto the original tpcpoint to preserve the full pcpoint values

```
SELECT numInstants(atTpcbox(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]),
  tpcboxZT(0, 0, 0, 2, 2, 2, tstzspan '[2001-01-01, 2001-01-03]', 1)));
-- 2
-- The box does not intersect the tpcpoint, so nothing is removed.
SELECT numInstants(minusTpcbox(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]),
  tpcboxZT(10, 10, 10, 12, 12, 12, tstzspan '[2001-01-01, 2001-01-03]', 1)));
-- 3
```

- Restrict a tpcpoint to the instants before or after a timestamp

```
beforeTimestamp(tpcpoint,timestamptz,strict boolean=true) → tpcpoint
afterTimestamp(tpcpoint,timestamptz,strict boolean=true) → tpcpoint
```

The strict flag excludes the instant at the timestamp itself

```
SELECT timeSpan(beforeTimestamp(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), timestamptz '2001-01-02'));
-- [2001-01-01, 2001-01-02)
SELECT timeSpan(afterTimestamp(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), timestamptz '2001-01-02'));
-- (2001-01-02, 2001-01-03]
```

17.5.8 Splitting Operations

- Return the distinct values of a tpcpoint with the span set on which each of them is taken

```
unnest(tpcpoint) → {(value,time)}
```

```
SELECT (un).time FROM (SELECT unnest(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2024-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2024-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 1, 1, 1), '2024-01-03'::timestamptz)])) AS un) t;
-- {[2024-01-01, 2024-01-02), [2024-01-03, 2024-01-03]}
-- {[2024-01-02, 2024-01-03)}
```

- Split a tpcpoint into fragments, one per time bin

```
timeSplit(tpcpoint,duration interval,
  torigin timestamptz='2000-01-03',borderInc boolean=true) → {(time,tpcpoint)}
```

The optional argument `borderInc` states whether the upper border of the extent is included and thus, the fragment starting at that border and holding only it is generated.

```
SELECT (ts).time, numInstants((ts).temp) FROM (SELECT timeSplit(tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2024-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2024-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2024-01-03'::timestamptz)]),
  interval '1 day', '2024-01-01'::timestamptz) AS ts) t;
-- 2024-01-01 | 2
-- 2024-01-02 | 2
-- 2024-01-03 | 1
```

- Return the array of time spans of a tpcpoint's segments

```
spans(tpcpoint) → tstzspan[]
```

```
SELECT spans(tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1), '2024-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2024-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2024-01-03'::timestamptz)]));
-- {"[2024-01-01, 2024-01-02]","[2024-01-02, 2024-01-03]"}
```

- Return the time spans of a tpcpoint split into a given number of bins, or with a given number of segments per bin

```
splitNSpans(tpcpoint,integer) → tstzspan[]
splitEachNSpans(tpcpoint,integer) → tstzspan[]
```

```
SELECT splitNSpans(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2024-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2024-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2024-01-03'::timestamptz)]), 2);
-- {"[2024-01-01, 2024-01-02]","[2024-01-02, 2024-01-03]"}
```

17.5.9 Bounding Box Operations

The bbox operators evaluate against the value's `tpcbox`; see Section 17.4.7 and Section 17.4.8 for the per-operator geometry. Each operator below is wired against `tpcbox`, `tstzspan`, and `tpcpoint` itself.

- Topological operators

```
{tstzspan,tpcbox,tpcpoint} {&&, @>, <@, ~=, -|-} {tstzspan,tpcbox,tpcpoint} → boolean
```

```
SELECT tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]) &&
  tstzspan '[2001-01-02, 2001-01-04]';
-- true
SELECT tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]) @>
  tpcboxZ(1, 1, 1, 2, 2, 2, 1);
-- true
```

- Position operators

```
{tpcbox,tpcpoint} {<<, >>, <<|, |>>, <</, />>} {tpcbox,tpcpoint} → boolean
{tstzspan,tpcbox,tpcpoint} {<<#, #>>} {tstzspan,tpcbox,tpcpoint} → boolean
```

```
SELECT tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]) <<
  tpcboxZ(10, 10, 10, 12, 12, 12, 1);
-- true
SELECT tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]) <<#
  tstzspan '[2001-01-05, 2001-01-06]';
-- true
```

17.5.10 Distance Operations

- Return the smallest distance ever

```
nearestApproachDistance(tpcpoint,{tpcbox,tpcpoint}) → float
```

Two values whose schemas differ are not comparable, so a pcid mismatch raises `Operation on pcpoint values with different schemas`, naming both, rather than answering.

```
SELECT round(nearestApproachDistance( tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 10, 10, 10), '2001-01-01'::timestamptz))::numeric, 4);
-- 15.5885
-- A tpcbox without a time span measures the spatial approach only.
SELECT round(nearestApproachDistance(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]),
  tpcboxZ(10, 10, 10, 12, 12, 12, 1))::numeric, 4);
-- 12.1244
SELECT round((tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz) |=|
  tpcpoint(pcpoint(1, 10, 10, 10), '2001-01-01'::timestamptz))::numeric, 4);
-- 15.5885
```

- Return the instant of the tpcpoint at which the two arguments are at the nearest distance

```
nearestApproachInstant({geometry,tpcpoint},{geometry,tpcpoint}) → tpcpoint
```

```
SELECT asText(nearestApproachInstant(tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 4, 4, 4), '2001-01-02'::timestamptz)]), geometry 'Point Z(5 5 5)'));
-- 0101000000900100009001000090010000@2001-01-02
```

- Return the line connecting the nearest approach point between the two arguments

```
shortestLine({geometry,tpcpoint},{geometry,tpcpoint}) → geometry
```

```
SELECT ST_AsText(shortestLine(tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 4, 4, 4), '2001-01-02'::timestamptz)]), geometry 'Point Z(5 5 5)'));
-- LINESTRING Z (4 4 4,5 5 5)
```

- Return the temporal distance

```
{geometry,tpcpoint} <-> {geometry,tpcpoint} → tfloat
```

```
SELECT round(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 4, 4, 4), '2001-01-02'::timestamptz)]) <->
  geometry 'Point Z(5 5 5)', 4);
-- [6.9282@2001-01-01, 1.7321@2001-01-02]
```

17.5.11 Ever and Always Relationships

A `tpcpoint` is projected to a `tgeompoint` through the schema cache and the relationship is answered there, so a tpcpoint declares the relationships its cast target declares.

- Ever and always relationships

```
eContains(geometry,tpcpoint) → boolean
aContains(geometry,tpcpoint) → boolean
eCovers(geometry,tpcpoint) → boolean
aCovers(geometry,tpcpoint) → boolean
eDisjoint({geometry,tpcpoint},{geometry,tpcpoint}) → boolean
aDisjoint({geometry,tpcpoint},{geometry,tpcpoint}) → boolean
eDwithin({geometry,tpcpoint},{geometry,tpcpoint},dist float) → boolean
aDwithin({geometry,tpcpoint},{geometry,tpcpoint},dist float) → boolean
eIntersects({geometry,tpcpoint},{geometry,tpcpoint}) → boolean
aIntersects({geometry,tpcpoint},{geometry,tpcpoint}) → boolean
eTouches(geometry,tpcpoint) → boolean
aTouches(geometry,tpcpoint) → boolean
eTouches(tpcpoint,geometry) → boolean
aTouches(tpcpoint,geometry) → boolean
```

A moving point neither contains nor covers a geometry and two moving points do not touch, so `eContains`, `eCovers` and their always twins take the geometry first only, and `eTouches` and `aTouches` read either order but not two points. They are planar relationships, so a point whose schema states a Z dimension meets the refusal `The tgeompoint cannot have Z dimension`.

```
SELECT eContains(geometry 'Polygon((0 0,0 4,4 4,4 0,0 0))',
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz));
-- ERROR:  The tgeompoint cannot have Z dimension
SELECT eIntersects(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), geometry 'POINT Z (1 1 1)');
-- t
SELECT aIntersects(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), geometry 'POINT Z (1 1 1)');
-- f
SELECT eIntersects(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]),
  tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]));
-- t
SELECT eDisjoint(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), geometry 'POINT Z (1 1 1)');
-- t
SELECT aDisjoint(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), geometry 'POINT Z (1 1 1)');
-- f
SELECT eDwithin(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), geometry 'POINT Z (5 5 5)',
  4);
-- t
SELECT aDwithin(tpcpointSeq(ARRAY[ tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), geometry 'POINT Z (5 5 5)',
  4);
-- f
```

17.5.12 Spatiotemporal Relationships

The spatiotemporal relationships compute the topological or distance relationship at each instant and result in a `tbool`. They read the same `tgeompoint` projection the ever/always relationships read, and are wired in the same three argument orders. A tpcpoint carries the Z its schema declares, if any: `tContains`, `tCovers` and `tTouches` are planar relationships, so a schema whose dimensions do not include Z answers them and one carrying Z is refused. That projection carries the interpolation of its source, so a tpcpoint sequence is answered along its trajectory rather than at its instants alone.

- Temporal spatial relationships

```
tDisjoint({geometry,tpcpoint},{geometry,tpcpoint}) → tbool
tDwithin({geometry,tpcpoint},{geometry,tpcpoint},float) → tbool
tIntersects({geometry,tpcpoint},{geometry,tpcpoint}) → tbool
tContains(geometry,tpcpoint) → tbool
tCovers(geometry,tpcpoint) → tbool
tTouches(geometry,tpcpoint) → tbool
tTouches(tpcpoint,geometry) → tbool
```

A temporal point cloud point declares the relationships its cast target declares, so `tContains` and `tCovers` take the geometry first only and `tTouches` has no direction between two point cloud points. The three are planar relationships: a schema whose dimensions do not include Z answers them, and one carrying Z meets the refusal `The tgeompoint cannot have Z dimension`.

```
SELECT tIntersects(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- t@2001-01-01
SELECT tDisjoint(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- f@2001-01-01
SELECT tDisjoint(tpcpoint(pcpoint(1, 9, 9, 9), '2001-01-02'::timestamptz),
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- t@2001-01-02
SELECT tDisjoint(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 9, 9, 9), '2001-01-01'::timestamptz));
-- t@2001-01-01
SELECT tDwithin(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  geometry 'Point(1 2)', 2.0);
-- t@2001-01-01
SELECT tDwithin(tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  geometry 'Point(1 5)', 2.0);
-- f@2001-01-01
```

17.5.13 Comparisons

- Traditional comparisons

```
tpcpoint {=, <>, <, <=, >, >=} tpcpoint → boolean
```

The order is lexicographic over the canonical encodings of the two values, not geometric.

```
SELECT tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz) = tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz);
-- t
SELECT tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz) <> tpcpoint(pcpoint(1, 2,
  2, 2), '2001-01-02'::timestamptz);
-- t
SELECT tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz) < tpcpoint(pcpoint(1, 2,
  2, 2), '2001-01-02'::timestamptz);
-- t
```

- Ever and always comparisons

```
{pcpoint,tpcpoint} {?=, %=, ?<>, %<>} {pcpoint,tpcpoint} → boolean
```

```
SELECT pcpoint(1, 1, 1, 1) ?= tpcpointSeq(ARRAY[
  tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]);
-- t
SELECT tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]) %= pcpoint(1, 1, 1, 1);
-- f
```

- Temporal comparisons

```
{pcpoint,tpcpoint} {#=, #<>} {pcpoint,tpcpoint} → tbool
```

```
SELECT tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]) #= pcpoint(1, 1, 1, 1);
-- [t@2001-01-01, f@2001-01-02, f@2001-01-03]
```

17.5.14 Simplification

- Return a tpcpoint simplified ensuring that consecutive points are at least a certain distance or time interval apart, or simplified using the Douglas-Peucker algorithm

```
minDistSimplify(tpcpoint,mindist float) → tpcpoint
minTimeDeltaSimplify(tpcpoint,mint interval) → tpcpoint
maxDistSimplify(tpcpoint,maxdist float,syncdist boolean=true) → tpcpoint
douglasPeuckerSimplify(tpcpoint,maxdist float,syncdist boolean=true) → tpcpoint
```

Simplification applies only to temporal sequences or sequence sets with linear interpolation, and a tpcpoint has step or discrete interpolation, so each function returns a copy of the given value.

```
SELECT minDistSimplify(tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1),
  '2001-01-01'::timestamptz), tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]), 2) =
  tpcpointSeq(ARRAY[tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz),
  tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz),
  tpcpoint(pcpoint(1, 3, 3, 3), '2001-01-03'::timestamptz)]);
-- t
```

17.6 Temporal Point Cloud Patches

A `tpcpatch` is the lifting of `pcpatch`. It mirrors `tpcpoint` in every regard, same subtypes, same accessors, same casts and operators, except that each instant carries an entire compressed batch of points instead of a single point. The bounding box is again a `tpcbox`, whose X and Y extent per instant is the patch's embedded `PCBOUNDS` and whose Z extent, where the schema holds a Z dimension, is the one the points of the patch state.

Operations on a `tpcpatch` come at two granularities, because the points of a patch sit in a compressed payload that the bounding box layer cannot inspect. At *patch level* each instant's patch is kept or dropped as a whole, read from its extent and the instant's timestamp alone: this is the granularity of atTpcbox / minusTpcbox, of the bounding box operators and of the `extent` aggregate. The X and Y extent is the 4-double `PCBOUNDS` header (xmin / xmax / ymin / ymax), read without decompressing the payload; `PCBOUNDS` being 2D, the Z extent, which a `tpcbox` with Z tests, is the one the points of the patch state.

At *point level* every point of every instant is walked in C. points emits one row per instant timestamp and point, while atTpcboxFine / minusTpcboxFine, atGeometry / minusGeometry and eIntersects(tpcpatch,geometry) apply their predicate to the actual coordinates and rebuild each surviving instant from the points that passed, dropping the instants that keep none. Prune at patch level first, with `atTpcbox` or a GiST / SP-GiST index scan, and refine the survivors when point-level fidelity is needed.

17.6.1 Input and Output

- Return the text representation of a tpcpatch, or of an array of them, with or without its SRID, and cast a tpcpatch to text

```
asText(tpcpatch) → text
asText(tpcpatch[]) → text[]
asEWKT(tpcpatch) → text
asEWKT(tpcpatch[]) → text[]
tpcpatch::text → text
```

As for a `tpcpoint`, `asEWKT` prefixes the SRID the schema of the `pcid` declares, and writes the text form alone under a schema declaring none.

```
-- The pcpatch payload renders as hex-encoded WKB.
SELECT left(asText(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz)), 24);
-- EC0100000100000000000000
SELECT left(asEWKT(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz)), 24);
-- EC0100000100000000000000
```

- Return the binary or hexadecimal well-known binary representation of a tpcpatch, with or without its SRID

```
asBinary(tpcpatch,endian text='') → bytea
asEWKB(tpcpatch,endian text='') → bytea
asHexWKB(tpcpatch,endian text='') → text
asHexEWKB(tpcpatch,endian text='') → text
```

```
SELECT tpcpatchFromHexEWKB(asHexEWKB(t)) = t
FROM (SELECT tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)),
  '2001-01-01'::timestamptz) AS t) s;
-- t
```

- Return a tpcpatch from its text, binary, or hexadecimal well-known binary representation, with or without its SRID

```
tpcpatchFromText(text) → tpcpatch
tpcpatchFromEWKT(text) → tpcpatch
tpcpatchFromBinary(bytea) → tpcpatch
tpcpatchFromEWKB(bytea) → tpcpatch
tpcpatchFromHexWKB(text) → tpcpatch
tpcpatchFromHexEWKB(text) → tpcpatch
```

The `pcid` each patch carries names its schema, so the text form reads back as the binary one does, and an SRID the input states must be the one that schema declares.

```
SELECT tpcpatchFromText(asText(t)) = t
FROM (SELECT tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)),
  '2001-01-01'::timestamptz) AS t) s;
-- t
SELECT tpcpatchFromEWKT('SRID=4326;' || asText(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz)));
-- ERROR:  The SRID of a pcpatch is the one its schema declares and cannot be set
```

- Return the Moving Features JSON (MF-JSON) representation of a tpcpatch and read it back; the round trip is lossless

```
asMFJSON(tpcpatch,options integer=0,flags integer=0,maxdecimaldigits integer=15) → text
tpcpatchFromMFJSON(text) → tpcpatch
```

A `tpcpatch` is emitted under the type tag `"MovingPCPatch"`, each instant stating the object `{"pcid":N, "npoints":N, "bounds":[xmin,xmax,ymin,ymax], "pts":[[...], ...]}`. The points are pgPointCloud's text form of the patch, every dimension of each point in the order its schema states them and in the shortest decimal form that reads back as the same double, whatever `maxdecimaldigits` states. `tpcpatchFromMFJSON` rebuilds each patch from its points in the compression its schema states, and refuses a point that does not state every dimension of the schema.

```
SELECT asMFJSON(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz));
/* {"type":"MovingPCPatch","values":[{"pcid":1,"npoints":2,"bounds":[1,2,1,2],
   "pts":[[1,1,1],[2,2,2]]}],"datetimes":["2001-01-01T00:00:00"],"interpolation":"None"}  ↩
       */
SELECT tpcpatchFromMFJSON(asMFJSON(t)) = t
FROM (SELECT tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)),
  '2001-01-01'::timestamptz) AS t) s;
-- t
```

As for a `tpcpoint`, `asText` takes no `maxdecimaldigits` argument.

17.6.2 Constructors

- Constructor for temporal point cloud patches having a constant value

```
tpcpatch(pcpatch,timestamptz) → tpcpatchInst
tpcpatch(pcpatch,tstzset) → tpcpatchDiscSeq
tpcpatch(pcpatch,tstzspan) → tpcpatchContSeq
tpcpatch(pcpatch,tstzspanset) → tpcpatchSeqSet
```

```
SELECT tempSubtype(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz));
-- Instant
SELECT tempSubtype(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), tstzspan '[2001-01-01, 2001-01-02]'));
-- Sequence
```

- Construct a temporal pcpatch of sequence subtype

```
tpcpatchSeq(tpcpatch[]) → tpcpatch
tpcpatchSeq(tpcpatch[],text) → tpcpatch
```

```
SELECT numInstants(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)])), interp(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]));
-- 2 | Step
```

- Construct a temporal pcpatch of sequence-set subtype

```
tpcpatchSeqSet(tpcpatch[]) → tpcpatch
```

```
SELECT numSequences(tpcpatchSeqSet(ARRAY[tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3),
  pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)])]));
-- 1
```

17.6.3 Conversions

- Convert a temporal patch into the temporal geometry of the multipoints its patches occupy

```
tgeometry(tpcpatch) → tgeometry
tgeography(tpcpatch) → tgeography
```

Each instant's patch becomes the multipoint of the positions its points occupy, read through the schema its `pcid` names

```
SELECT asText(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz)::tgeometry);
-- MULTIPOINT Z ((1 1 1),(2 2 2))@2001-01-01
```

- Convert a temporal point cloud patch to its bounding box

```
tpcbox(tpcpatch) → tpcbox
tpcpatch::tpcbox
```

The box is the one the extent of the value answers: the spatial extent of its points, its time span, the SRID its schema states and its `pcid`.

```
SELECT tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz)::tpcbox;
-- TPCBOX(ZT(((1,1,1),(2,2,2)),[2001-01-01, 2001-01-01]), 1)
```

17.6.4 Accessors

- Return the number of points in the patch carried by the first or last instant

```
startNumPoints(tpcpatch) → integer
endNumPoints(tpcpatch) → integer
```

```
SELECT startNumPoints(tpcpatch(pcpatch(pcpoint(1, 10.0, 20.0, 30.0)),
  '2024-02-01'::timestamptz));
-- 1
```

- Return the total number of points across every instant's patch

```
numPoints(tpcpatch) → bigint
```

  Read directly from each patch's header, no decompression.

```
SELECT numPoints(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1.0, 1.0, 1.0),
  pcpoint(1, 2.0, 2.0, 2.0)), '2024-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3.0, 3.0, 3.0)), '2024-01-02'::timestamptz)]));
-- 3
```

- Set-returning function: emits one row per (instant timestamp, point) by walking each instant's patch and decomposing it through pgPointCloud's in-memory C API

```
points(tpcpatch) → setof (t timestamptz,point pcpoint)
```

  Useful for joining a tpcpatch column against per-point predicates that the bbox-level operators can't express. Cost is O(total points), one per-instant decompression plus one row emission per point.

```
SELECT t,
  point FROM points(tpcpatch(pcpatch(pcpoint(1, 1.0, 1.0, 1.0),
  pcpoint(1, 2.0, 2.0, 2.0)), '2024-01-01'::timestamptz));
-- ('2024-01-01',
-- '01010000006400000064000000640000000'::pcpoint)
-- ('2024-01-01',
-- '0101000000C8000000C8000000C8000000'::pcpoint)
```

17.6.5 Transformations

- Transform a tpcpatch to another subtype

```
tpcpatchInst(tpcpatch) → tpcpatch
tpcpatchSeq(tpcpatch,interp text='step') → tpcpatch
tpcpatchSeqSet(tpcpatch) → tpcpatch
```

```
SELECT tempSubtype(tpcpatchSeq(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz)));
-- Sequence
```

- Transform a tpcpatch to another interpolation

```
setInterp(tpcpatch,interp text) → tpcpatch
```

```
SELECT interp(setInterp(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)],
  'discrete'), 'step'));
-- Step
```

17.6.6 Modifications

- Insert a tpcpatch into another one, or update another one with it

```
insert(tpcpatch,tpcpatch,connect boolean=true) → tpcpatch
update(tpcpatch,tpcpatch,connect boolean=true) → tpcpatch
```

```
SELECT numInstants(insert(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]), tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5)), '2001-01-03'::timestamptz)])));
-- 3
```

- Delete a time selector (timestamp, set, period, or spanset) from a tpcpatch

```
deleteTime(tpcpatch,times,connect boolean=true) → tpcpatch
```

```
-- Deleting the middle instant splits the sequence into {[2001-01-01, 2001-01-02),
-- (2001-01-02, 2001-01-03]}; each half stores its bound at the deleted timestamp, giving
-- four instants.
SELECT numInstants(deleteTime(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5)), '2001-01-03'::timestamptz)]),
  timestamptz '2001-01-02'));
-- 4
```

- Append an instant or a sequence to a tpcpatch

```
appendInstant(tpcpatch,tpcpatch) → tpcpatch
appendSequence(tpcpatch,tpcpatch) → tpcpatch
```

```
SELECT numInstants(appendInstant(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3),
  pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)]),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5)), '2001-01-03'::timestamptz)));
-- 3
```

- Merge the temporal values

```
merge(tpcpatch,tpcpatch) → tpcpatch
merge(tpcpatch[]) → tpcpatch
```

```
SELECT numInstants(merge(tpcpatchSeq(ARRAY[tpcpatch(pcpatch(pcpoint(1, 1, 1, 1)), ' ↩
    2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 2, 2, 2)), '2001-01-02'::timestamptz)]),
  tpcpatchSeq(ARRAY[tpcpatch(pcpatch(pcpoint(1, 2, 2, 2)), '2001-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3)), '2001-01-03'::timestamptz)])));
-- 3
SELECT numInstants(merge(ARRAY[tpcpatch(pcpatch(pcpoint(1, 1, 1, 1)), '2001-01-01':: ↩
    timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3)), '2001-01-03'::timestamptz)]));
-- 2
```

17.6.7 Restrictions

- Restrict a tpcpatch to (the complement of) a pcpatch value or a set of pcpatch values

```
atValue(tpcpatch,pcpatch) → tpcpatch
minusValue(tpcpatch,pcpatch) → tpcpatch
atValues(tpcpatch,pcpatchset) → tpcpatch
minusValues(tpcpatch,pcpatchset) → tpcpatch
```

```
-- The matching value is held up to the next instant; the closing bound is stored as a
-- second instant.
SELECT numInstants(atValue(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]), pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2))));
-- 2
SELECT numInstants(minusValue(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]), pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2))));
-- 1
```

- Restrict a tpcpatch to the instants whose patch extent overlaps the tpcbox, or remove them

```
atTpcbox(tpcpatch,tpcbox,border_inc boolean=true) → tpcpatch
minusTpcbox(tpcpatch,tpcbox,border_inc boolean=true) → tpcpatch
```

Granularity is patch-level: each surviving instant keeps its pcpatch payload verbatim. The X and Y extent of a patch is pgPointCloud's 2D `PCBOUNDS`; when the box has Z, the Z extent of the patch is the one its points state, and a patch of a schema without Z fails the box. Returns NULL when the tpcbox's pcid does not match.

```
SELECT numInstants(atTpcbox(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3),
  pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)]),
  tpcboxXT(0, 0, 2, 2, tstzspan '[2001-01-01, 2001-01-03]', 1)));
-- 1
-- The dropped patch stays present on the open interval before its instant,
-- so the complement keeps two instants.
SELECT numInstants(minusTpcbox(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3),
  pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)]),
  tpcboxXT(0, 0, 2, 2, tstzspan '[2001-01-01, 2001-01-03]', 1)));
-- 2
```

- Restrict a tpcpatch to the points of each instant that the tpcbox holds, or remove them

```
atTpcboxFine(tpcpatch,tpcbox,border_inc boolean=true) → tpcpatch
minusTpcboxFine(tpcpatch,tpcbox,border_inc boolean=true) → tpcpatch
```

Each surviving instant carries a freshly built pcpatch holding only the points inside, or outside for the minus form, the tpcbox in two dimensions, and in three when the box has a Z dimension. Instants whose patch filters to zero points are dropped. Slower than the coarse variant because every patch is decompressed and rebuilt; use when you need point-level fidelity.

```
SELECT numInstants(atTpcboxFine(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3),
  pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)]),
  tpcboxXT(0, 0, 1, 1, tstzspan '[2001-01-01, 2001-01-03]', 1)));
-- 1
```

- Restrict a tpcpatch to the points whose XY projection intersects (or does not intersect, for minus) a 2D geometry

```
atGeometry(tpcpatch,geometry) → tpcpatch
minusGeometry(tpcpatch,geometry) → tpcpatch
```

Z is ignored. SRID compatibility between the patch schema and the geometry must be ensured by the caller.

```
-- The polygon boundary touches the point (3 3) of the patch at 2001-01-02, so that patch
-- survives both the at and the minus side.
SELECT numInstants(atGeometry(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]), geometry 'POLYGON((0 0, 0 3, 3 3, 3 0, 0 0))'));
-- 2
SELECT numInstants(minusGeometry(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]), geometry 'POLYGON((0 0, 0 3, 3 3, 3 0, 0 0))'));
-- 1
```

- Restrict a tpcpatch to the instants before or after a timestamp

```
beforeTimestamp(tpcpatch,timestamptz,strict boolean=true) → tpcpatch
afterTimestamp(tpcpatch,timestamptz,strict boolean=true) → tpcpatch
```

The strict flag excludes the instant at the timestamp itself

```
SELECT timeSpan(beforeTimestamp(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5)), '2001-01-03'::timestamptz)]),
  timestamptz '2001-01-02'));
-- [2001-01-01, 2001-01-02)
SELECT timeSpan(afterTimestamp(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5)), '2001-01-03'::timestamptz)]),
  timestamptz '2001-01-02'));
-- (2001-01-02, 2001-01-03]
```

17.6.8 Splitting Operations

- Return the distinct values of a tpcpatch with the span set on which each of them is taken

```
unnest(tpcpatch) → {(value,time)}
```

```
SELECT (un).time FROM (SELECT unnest(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2024-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5), pcpoint(1, 6, 6, 6)), '2024-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2024-01-03'::timestamptz)])) AS un) t;
-- {[2024-01-02, 2024-01-03)}
-- {[2024-01-01, 2024-01-02), [2024-01-03, 2024-01-03]}
```

- Split a tpcpatch into fragments, one per time bin

```
timeSplit(tpcpatch,duration interval,
  torigin timestamptz='2000-01-03',borderInc boolean=true) → {(time,tpcpatch)}
```

The optional argument `borderInc` states whether the upper border of the extent is included and thus, the fragment starting at that border and holding only it is generated.

```
SELECT (ts).time, numInstants((ts).temp) FROM (SELECT timeSplit(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2024-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5), pcpoint(1, 6, 6, 6)), '2024-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)),
  '2024-01-03'::timestamptz)]),
  interval '1 day', '2024-01-01'::timestamptz) AS ts) t;
-- 2024-01-01 | 2
-- 2024-01-02 | 2
-- 2024-01-03 | 1
```

- Return the array of time spans of a tpcpatch's segments

```
spans(tpcpatch) → tstzspan[]
```

```
SELECT spans(tpcpatchSeq(ARRAY[tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2024-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5), pcpoint(1, 6, 6, 6)), '2024-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2024-01-03'::timestamptz)]));
-- {"[2024-01-01, 2024-01-02]","[2024-01-02, 2024-01-03]"}
```

- Return the time spans of a tpcpatch split into a given number of bins, or with a given number of segments per bin

```
splitNSpans(tpcpatch,integer) → tstzspan[]
splitEachNSpans(tpcpatch,integer) → tstzspan[]
```

```
SELECT splitNSpans(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2024-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 5, 5, 5), pcpoint(1, 6, 6, 6)), '2024-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2024-01-03'::timestamptz)]), 2);
-- {"[2024-01-01, 2024-01-02]","[2024-01-02, 2024-01-03]"}
```

17.6.9 Spatial Operations

- Return the swath of a temporal patch, the region of the plane its patches sweep

```
swath(tpcpatch,maxt interval=NULL) → geometry
```

The footprint of an instant is the convex hull of the positions of the points of its patch, read through the schema its `pcid` names and projected on the plane. The footprints of two consecutive instants are joined by the convex hull of both, and the swath is the union of these strips, together with the footprint of every instant joined to neither of its neighbours. Consecutive instants are joined unless they are more than `maxt` apart, so that a pause in the acquisition splits the swath; every gap is joined when `maxt` is not given. A patch is the set of points one acquisition captures, such as a ping of a multibeam echosounder, and the swath is then the strip of the seabed the pings of a survey line sound. Whereas the traversed area of the conversion to `tgeometry` is the soundings themselves, whose area is zero, the swath covers the seabed between them. The union is computed exactly.

```
SELECT ST_AsText(swath(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 0, 0, 0), pcpoint(1, 1, 0, 0), pcpoint(1, 2, 0, 0)),
    '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 0, 1, 0), pcpoint(1, 1, 1, 0), pcpoint(1, 2, 1, 0)),
    '2001-01-02'::timestamptz)])));
-- POLYGON((0 0,0 1,2 1,2 0,0 0))
SELECT ST_NumGeometries(swath(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 0, 0, 0), pcpoint(1, 2, 0, 0)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 0, 1, 0), pcpoint(1, 2, 1, 0)), '2001-01-02'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 0, 5, 0), pcpoint(1, 2, 5, 0)), '2001-01-05'::timestamptz)]) ↩
    ,
  interval '1 day'));
-- 2
```

17.6.10 Bounding Box Operations

The same bbox operator surface as Section 17.5.9 applies to `tpcpatch`. The predicate evaluates against the value's `tpcbox`, whose X and Y extent per instant is the patch's embedded `PCBOUNDS` and whose Z extent is the one the points of the patch state.

- Topological operators

```
{tstzspan,tpcbox,tpcpatch} {&&, @>, <@, ~=, -|-} {tstzspan,tpcbox,tpcpatch} → boolean
```

```
SELECT tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)])
  && tstzspan '[2001-01-01, 2001-01-03]';
-- true
```

- Position operators

```
{tpcbox,tpcpatch} {<<, >>, <<|, |>>, <</, />>} {tpcbox,tpcpatch} → boolean
{tstzspan,tpcbox,tpcpatch} {<<#, #>>} {tstzspan,tpcbox,tpcpatch} → boolean
```

```
SELECT tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)])
  <<# tstzspan '[2001-01-05, 2001-01-06]';
-- true
```

17.6.11 Distance Operations

- Return the smallest distance ever

```
nearestApproachDistance(tpcpatch,{tpcbox,tpcpatch}) → float
```

```
SELECT round(nearestApproachDistance(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 8, 8, 8), pcpoint(1, 9, 9, 9)),
  '2001-01-01'::timestamptz))::numeric, 4);
-- 8.4853
```

17.6.12 Ever and Always Relationships

- Return true iff at least one point in any of the tpcpatch's instants intersects the geometry (XY only)

```
eIntersects(tpcpatch,geometry) → boolean
```

A patch is a set of points, so it neither contains nor covers a geometry and two patches do not touch: `eContains`, `eCovers` and their always twins take the geometry first only, `eTouches` and `aTouches` read either order but not two patches, and `eDisjoint`, `eIntersects`, `eDwithin` and their always twins read the three argument orders. `eIntersects(tpcpatch,geometry)` is the one the type answers natively, walking the points of each instant and stopping at the first hit; the rest convert the patch to the temporal geometry of its multipoints.

```
SELECT eIntersects(tpcpatchSeq(ARRAY[ tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
  pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3),
  pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz)]), geometry 'POINT(1 1)');
-- t
```

17.6.13 Spatiotemporal Relationships

The *temporal* relationships compute the topological or distance relationship at each instant and result in a `tbool`. A patch reaches the geometry engine through the `tgeometry` conversion above, so one point of a patch inside the argument is enough for the patch to intersect it. A tpcpatch carries the Z its schema declares, and `tContains`, `tCovers` and `tTouches` are planar relationships that refuse a Z dimension, so the type answers the three that a Z-carrying value can.

- Temporal spatial relationships

```
tDisjoint({geometry,tpcpatch},{geometry,tpcpatch}) → tbool
tDwithin({geometry,tpcpatch},{geometry,tpcpatch},float) → tbool
tIntersects({geometry,tpcpatch},{geometry,tpcpatch}) → tbool
tContains(geometry,tpcpatch) → tbool
tCovers(geometry,tpcpatch) → tbool
tTouches(geometry,tpcpatch) → tbool
tTouches(tpcpatch,geometry) → tbool
```

  A patch reaches the geometry engine as the multipoint of the positions its points occupy, and as a set of points it declares the matrix of a temporal point: `tContains` and `tCovers` take the geometry first only, `tTouches` reads either order but not two patches, and the others read the three argument orders. The planar ones answer for a schema whose dimensions do not include Z, and a schema carrying Z meets the refusal `The tgeometry cannot have Z dimension`.

```
SELECT tIntersects(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 9, 9, 9)),
  '2001-01-01'::timestamptz), geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- t@2001-01-01
SELECT tDisjoint(tpcpatch(pcpatch(pcpoint(1, 9, 9, 9)), '2001-01-02'::timestamptz),
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- t@2001-01-02
SELECT tDwithin(tpcpatch(pcpatch(pcpoint(1, 1, 1, 1)), '2001-01-01'::timestamptz),
  geometry 'Point(1 5)', 2.0);
-- f@2001-01-01
SELECT tIntersects(tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 9, 9, 9)), '2001-01-02'::timestamptz)]),
  geometry 'Polygon((0 0,0 2,2 2,2 0,0 0))');
-- [t@2001-01-01, f@2001-01-02]
```

17.6.14 Comparisons

- Ever and always comparisons

```
{pcpatch,tpcpatch} {?=, %=, ?<>, %<>} {pcpatch,tpcpatch} → boolean
```

```
SELECT pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)) ?= tpcpatchSeq(ARRAY[
  tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz),
  tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]);
-- t
SELECT tpcpatchSeq(ARRAY[tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)),
  '2001-01-01'::timestamptz), tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]) %= pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2));
-- f
```

- Temporal comparisons

```
{pcpatch,tpcpatch} {#=, #<>} {pcpatch,tpcpatch} → tbool
```

```
SELECT tpcpatchSeq(ARRAY[tpcpatch(pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2)),
  '2001-01-01'::timestamptz), tpcpatch(pcpatch(pcpoint(1, 3, 3, 3), pcpoint(1, 4, 4, 4)),
  '2001-01-02'::timestamptz)]) #= pcpatch(pcpoint(1, 1, 1, 1), pcpoint(1, 2, 2, 2));
-- [t@2001-01-01, f@2001-01-02]
```

17.7 Indexing

B-tree (`tpcpoint_btree_ops`, `tpcpatch_btree_ops`, `tpcbox_btree_ops`) and hash (`tpcpoint_hash_ops`, `tpcpatch_hash_ops`) operator classes support equality and ordering predicates. The R-tree GiST and quadtree / kd-tree SP-GiST operator classes accelerate the bbox-based predicates (`&&`, `@>`, `<@`, `~=`, `-|-`) on all three of `tpcbox`, `tpcpoint`, and `tpcpatch`:

```
-- GiST (R-tree)
CREATE INDEX ON my_table USING gist(my_tpcpoint_column);
CREATE INDEX ON my_table USING gist(my_tpcpatch_column);
-- SP-GiST quadtree (default) or kd-tree
CREATE INDEX ON my_table USING spgist(my_tpcpoint_column);
CREATE INDEX ON my_table USING spgist(my_tpcpoint_column tpcpoint_kdtree_ops);
```

SP-GiST opclasses store an `stbox` internally (the `pcid` filter is recovered by the operator's recheck), so a query against an index on a column whose values mix multiple `pcid`s will return a slightly larger set of candidate rows for recheck, a non-issue when each table holds values of a single schema, which is the common case.

17.8 Aggregations

- Bounding-box aggregate over a column of tpcpoint, tpcpatch, or tpcbox values

  ```
  extent({tpcpoint,tpcpatch,tpcbox}) → tpcbox
  ```

  Returns the smallest tpcbox containing every input. Aggregating across distinct pcids raises an error, the bbox would be uninterpretable since dimensions are pcid-relative.

  ```
  SELECT extent(v) FROM (VALUES (tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz)),
    (tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz))) t(v);
  -- TPCBOX(ZT(((1,1,1),(2,2,2)),[2001-01-01, 2001-01-02]), 1)
  ```

- Temporal count and window count of overlapping rows at each timestamp

  ```
  tCount({tpcpoint,tpcpatch}) → tint
  wCount({tpcpoint,tpcpatch},interval) → tint
  ```

  Result is a tint. All input rows must share the same temporal subtype (instant / sequence / sequence set).

  ```
  SELECT tCount(v) FROM (VALUES (tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz)),
    (tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz))) t(v);
  -- {1@2001-01-01, 1@2001-01-02}
  SELECT wCount(v, interval '1 day') FROM (VALUES
    (tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz)),
    (tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz))) t(v);
  -- {[1@2001-01-01, 2@2001-01-02], (1@2001-01-02, 1@2001-01-03]}
  ```

- Per-instant pcpatch point count, summed across rows

  ```
  tnpoints(tpcpatch) → tint
  ```

  Distinct from `tCount(tpcpatch)`, which counts the number of patches (always 1 per instant). `tnpoints` reads as "how many points were in the cloud at time t", the natural primitive for ingest-QC and density-over-time dashboards.

  ```
  SELECT tnpoints(v) FROM (VALUES (tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
   pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz)),
    (tpcpatch(pcpatch(pcpoint(1, 3, 3, 3),
   pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz))) t(v);
  -- {2@2001-01-01, 2@2001-01-02}
  ```

- Per-instant point density (`numPoints / xy-bbox-area`) summed across rows

  ```
  tdensity(tpcpatch) → tfloat
  ```

  Each pcpatch carries the inline `PCBOUNDS` (xmin / xmax / ymin / ymax) so the bbox-area term is read directly, no recomputation. A patch whose XY bounds have no area, a single point or points on a line, has no density: the division by zero raises an error. A query over patches that may have no area reads those whose box has an area, `area(stbox(tpcbox(patch))) > 0`.

  ```
  -- Each patch holds 2 points over a 1x1 xy-bbox.
  SELECT tdensity(v) FROM (VALUES (tpcpatch(pcpatch(pcpoint(1, 1, 1, 1),
    pcpoint(1, 2, 2, 2)), '2001-01-01'::timestamptz)),
    (tpcpatch(pcpatch(pcpoint(1, 3, 3, 3),
    pcpoint(1, 4, 4, 4)), '2001-01-02'::timestamptz))) t(v);
  -- {2@2001-01-01, 2@2001-01-02}
  ```

- Combine multiple temporal values into a single one with all input instants merged in temporal order

  ```
  merge(tpcpoint) → tpcpoint
  merge(tpcpatch) → tpcpatch
  ```

  All inputs must share the same pcid and subtype.

  ```
  SELECT numInstants(merge(v)) FROM (VALUES (tpcpoint(pcpoint(1, 1, 1, 1),
    '2001-01-01'::timestamptz)),
    (tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz))) t(v);
  -- 2
  ```

- Streaming construction aggregates: `appendInstant` assembles instants into a sequence; `appendSequence` assembles sequences into a sequence set

  ```
  appendInstant({tpcpoint,tpcpatch}[,interp text[,maxt interval]]) → {tpcpoint,tpcpatch}
  appendSequence(tpcpoint) → tpcpoint
  appendSequence(tpcpatch) → tpcpatch
  ```

  Useful for trajectory loaders that consume rows in time order.

  ```
  SELECT numInstants(appendInstant(v ORDER BY startTimestamp(v))) FROM (VALUES
    (tpcpoint(pcpoint(1, 1, 1, 1), '2001-01-01'::timestamptz)),
    (tpcpoint(pcpoint(1, 2, 2, 2), '2001-01-02'::timestamptz))) t(v);
  -- 2
  ```

Chapter 18

Writing Portable Queries

MobilityDB is used from several SQL engines: PostgreSQL through MobilityDB itself, DuckDB through MobilityDuck, Apache Spark through MobilitySpark and Apache Flink through MobilityFlink. The types and the functions are the same in all of them, but the operators are not: PostgreSQL lets MobilityDB define operators such as `&&` or `<<#`, while the other engines accept only some of them or none. DuckDB, for instance, reserves `#` for positional column references, so it accepts no operator containing it.

For this reason every operator is also a function, which runs the same code and returns exactly the same result, and a query written with functions only runs unchanged on every engine. The bounding box overlap `t1.trip && t2.trip`, for example, is written `stboxOverlaps(t1.trip, t2.trip)`, and the time position test `p1.period <<# p2.period` is written `spanBefore(p1.period, p2.period)`. The sections below give the function of each operator, grouped as the chapters of each type group them.

The name of a function says what it operates on. An arithmetic or set operation takes the prefix of the family of its argument: `set`, `span` or `spanset`, and `t` for a temporal value, as in `spanUnion` or `tAdd`. A position test takes the prefix of what it compares: `set`, `span` or `spanset`, or, for a box or a temporal value, the type of its bounding box, `tbox`, `stbox` or `tpcbox`; a temporal Boolean, text or JSONB value is compared through the time span that bounds it, and so takes `span`. These names are also free in every engine: `left` and `right` are functions of Spark and Flink, so the position tests are named `spanLeft` or `stboxLeft`, and an ever or always comparison spells out its operation, as in `eLessThan`, because `eLt` would be read as the function `elt` of Spark and Flink, which ignore the case of a name.

Some operations have no operator and are written the same way everywhere: the spatial relationships (`eIntersects`, `aIntersects`, `eTouches`, `eDwithin`, ...), the restrictions (`atTime`, `atGeometry`, `atValue`, ...) and the ordering comparison `cmp(a, b)`, which returns `-1`, `0` or `1`.

Spark and Flink ask for a few more changes besides the functions, which Section 18.10 describes.

18.1 Set Operations

The union, intersection and difference of two sets, spans or span sets, and the concatenation of two sets or of two temporal values.

Category	Operator	Portable function
Union (set / span / span set)	+	setUnion / spanUnion / spansetUnion (a, b)
Intersection (set / span / span set)	*	setIntersection / spanIntersection / spansetIntersection (a, b)
Difference (set / span / span set)	-	setMinus / spanMinus / spansetMinus (a, b)
Concatenation (set / temporal)	||	setConcat / tConcat (a, b)

18.2 Bounding Box Operations

These operations compare the bounding boxes of their arguments: the span of a set or of a span set, or the box of a temporal value, `tbox`, `stbox` or `tpcbox`.

18.2.1 Topological Operations

Whether two bounding boxes overlap, contain one another, are adjacent or are the same. The last rows test the time span of a temporal JSONB value and the route identifiers of network points.

Category	Operator	Portable function
Bounding box	&&	setOverlaps / spanOverlaps / spansetOverlaps / tboxOverlaps / stboxOverlaps / tpcboxOverlaps (a, b)
	@>	setContains / spanContains / spansetContains / tboxContains / stboxContains / tpcboxContains (a, b)
	<@	setContained / spanContained / spansetContained / tboxContained / stboxContained / tpcboxContained (a, b)
	-|-	spanAdjacent / spansetAdjacent / tboxAdjacent / stboxAdjacent / tpcboxAdjacent (a, b)
	~=	spanSame / tboxSame / stboxSame / tpcboxSame (a, b)
Time span of a temporal JSONB value	#@>	tjsonbContains(a, b)
	<@#	tjsonbContained(a, b)
Route identifier (network point)	@=	sameRid(a, b)
	@?	containsRid(a, b)
	?@	containedRid(a, b)
	@@	overlapsRid(a, b)

18.2.2 Position Operations

Whether a bounding box is strictly before, after, left of, right of, below, above, in front of or behind another, and, in the forms starting with `over`, whether it does not extend beyond it on that side.

Category	Operator	Portable function
Time position	<<#	setBefore / spanBefore / spansetBefore / tboxBefore / stboxBefore / tpcboxBefore (a, b)
	#>>	setAfter / spanAfter / spansetAfter / tboxAfter / stboxAfter / tpcboxAfter (a, b)
	&<#	setOverbefore / spanOverbefore / spansetOverbefore / tboxOverbefore / stboxOverbefore / tpcboxOverbefore (a, b)
	#&>	setOverafter / spanOverafter / spansetOverafter / tboxOverafter / stboxOverafter / tpcboxOverafter (a, b)
Space, X axis	<<	setLeft / spanLeft / spansetLeft / tboxLeft / stboxLeft / tpcboxLeft (a, b)
	>>	setRight / spanRight / spansetRight / tboxRight / stboxRight / tpcboxRight (a, b)
	&<	setOverleft / spanOverleft / spansetOverleft / tboxOverleft / stboxOverleft / tpcboxOverleft (a, b)
	&>	setOverright / spanOverright / spansetOverright / tboxOverright / stboxOverright / tpcboxOverright (a, b)
Space, Y axis	<<|	stboxBelow / tpcboxBelow (a, b)
	|>>	stboxAbove / tpcboxAbove (a, b)
	&<|	stboxOverbelow / tpcboxOverbelow (a, b)
	|&>	stboxOverabove / tpcboxOverabove (a, b)
Space, Z axis	<</	stboxFront / tpcboxFront (a, b)
	/>>	stboxBack / tpcboxBack (a, b)
	&</	stboxOverfront / tpcboxOverfront (a, b)
	/&>	stboxOverback / tpcboxOverback (a, b)

18.3 Distance Operations

Operands of different dimensions read the dimensions both of them have, see Section 8.3.

`tDistance` returns the distance between two values at each instant, as a temporal float, and `nearestApproachDistance` the smallest of these distances. `distance` and its set, span and span set forms return the distance between two values, and `th3GridDistance` the grid distance between two temporal H3 cells at each instant, as a temporal big integer.

`tquadbinGridDistance` and `ts2GridDistance` return the grid distance between two temporal QUADBIN or S2 cells in the same way.

Category	**Operator**	**Portable function**
Distance	<->	tDistance(a, b)
	|=|	nearestApproachDistance(a, b)
Distance (base / set / span / span set)	<->	distance / setDistance / spanDistance / spansetDistance (a, b)
Grid distance (cell index)	<->	th3GridDistance(a, b)
	<->	tquadbinGridDistance(a, b)
	<->	ts2GridDistance(a, b)

18.4 Comparisons

18.4.1 Traditional Comparisons

The comparisons of two values of the same type. With `cmp` they order values, as an `ORDER BY` does.

Operator	**Portable function**
=	eq(a, b)
<>	ne(a, b)
<	lt(a, b)
<=	le(a, b)
>	gt(a, b)
>=	ge(a, b)

18.4.2 Ever and Always Comparisons

Whether a temporal value is ever, or always, equal to, different from, less than or greater than a value or another temporal value.

Category	**Operator**	**Portable function**
Ever comparison	?=	eEqual(a, b)
	?<>	eNotEqual(a, b)
	?<	eLessThan(a, b)
	?<=	eLessEqual(a, b)
	?>	eGreaterThan(a, b)
	?>=	eGreaterEqual(a, b)
Always comparison	%=	aEqual(a, b)
	%<>	aNotEqual(a, b)
	%<	aLessThan(a, b)
	%<=	aLessEqual(a, b)
	%>	aGreaterThan(a, b)
	%>=	aGreaterEqual(a, b)

18.4.3 Temporal Comparisons

The comparison of two values at each instant, returning a temporal Boolean.

Operator	Portable function
#=	tEqual(a, b)
#<>	tNotEqual(a, b)
#<	tLessThan(a, b)
#<=	tLessEqual(a, b)
#>	tGreaterThan(a, b)
#>=	tGreaterEqual(a, b)

18.5 Boolean Operations

The conjunction, disjunction and negation of temporal Booleans at each instant.

Operator	Portable function
&	tAnd(a, b)
|	tOr(a, b)
~	tNot(a)

18.6 Mathematical Operations

The arithmetic of temporal numbers at each instant, and of rasters pixel by pixel.

Operator	Portable function
+	tAdd / rasterAdd (a, b)
-	tSub / rasterSub (a, b)
*	tMul / rasterMul (a, b)
/	tDiv / rasterDiv (a, b)

18.7 Temporal JSON Operations

Access to the fields and elements of JSONB sets and of temporal JSONB values, the existence of keys, the JSON path queries and the deletion of a path. Each function takes the prefix of the type of its first argument, `jsonbset` or `tjsonb`.

Category	Operator	Portable function
JSON field / element access	->	jsonbsetObjectField / tjsonbObjectField / jsonbsetArrayElement / tjsonbArrayElement (a, b)
	->>	jsonbsetObjectFieldText / tjsonbObjectFieldText / jsonbsetArrayElementText / tjsonbArrayElementText (a, b)
JSON key existence	?	jsonbsetExists / tjsonbExists (a, b)
	?&	jsonbsetExistsAll / tjsonbExistsAll (a, b)
	?|	jsonbsetExistsAny / tjsonbExistsAny (a, b)
JSON path	@?	jsonbsetPathExists / tjsonbPathExists (a, b)
	@@	jsonbsetPathMatch / tjsonbPathMatch (a, b)
	#-	jsonbsetDeletePath / tjsonbDeletePath (a, b)

18.8 Base Values Provided by the Host

Some base types of MobilityDB come from PostgreSQL itself or from an extension MobilityDB requires: `jsonb` and `jsonpath` from PostgreSQL, `h3index` from the `h3` extension, and `pcpoint` and `pcpatch` from the `pointcloud` extension. In PostgreSQL their operators and functions keep the names their provider gives them, and every engine, PostgreSQL included, also calls them by the names of the following table.

Their values are compared with the functions of Section 18.4.1 and with `cmp`, and hashed with `hash` and `hashExtended`, which Spark and Flink call `jsonbHash`, `jsonbHashExtended`, `h3indexHash` and `h3indexHashExtended`, since these engines name the hash function of every type after the type. A value is read from its text with `jsonbFromText`, `jsonpathFromText` and `h3indexFromText`, and written as text with `asText`; in PostgreSQL a literal and a cast from or to `text` do the same. A `pcpoint` or a `pcpatch` is written as the hexadecimal form of its Well-Known Binary, read with `pcpointFromHexWKB` and `pcpatchFromHexWKB` and written with `asHexWKB`.

PostgreSQL	Portable function
jsonb ’. . . ’, text::jsonb	jsonbFromText(a)
jsonb::text	asText(a)
jsonpath ’. . . ’, text::jsonpath	jsonpathFromText(a)
jsonpath::text	asText(a)
h3index ’. . . ’, text::h3index	h3indexFromText(a)
h3index::text	asText(a)
pcpoint ’. . . ’, text::pcpoint	pcpointFromHexWKB(a)
pcpoint::text	asHexWKB(a)
pcpatch ’. . . ’, text::pcpatch	pcpatchFromHexWKB(a)
pcpatch::text	asHexWKB(a)
jsonb -> text	jsonbObjectField(a, b)
jsonb -> integer	jsonbArrayElement(a, b)
jsonb ->> text	jsonbObjectFieldText(a, b)
jsonb ->> integer	jsonbArrayElementText(a, b)
jsonb #> text[], jsonb_extract_path	jsonbExtractPath(a, b)
jsonb #>> text[], jsonb_extract_path_text	jsonbExtractPathText(a, b)
jsonb ? text	jsonbExists(a, b)
jsonb ?| text[]	jsonbExistsAny(a, b)
jsonb ?& text[]	jsonbExistsAll(a, b)
jsonb @> jsonb	jsonbContains(a, b)
jsonb <@ jsonb	jsonbContained(a, b)
jsonb || jsonb	jsonbConcat(a, b)
jsonb - text	jsonbDelete(a, b)
jsonb - text[]	jsonbDeleteArray(a, b)
jsonb - integer	jsonbDeleteIndex(a, b)
jsonb #- text[]	jsonbDeletePath(a, b)
jsonb @? jsonpath, jsonb_path_exists	jsonbPathExists(a, b)
jsonb_path_exists_tz	jsonbPathExistsTz(a, b)
jsonb @@ jsonpath, jsonb_path_match	jsonbPathMatch(a, b)
jsonb_path_match_tz	jsonbPathMatchTz(a, b)
jsonb_path_query	jsonbPathQuery(a, b)
jsonb_path_query_tz	jsonbPathQueryTz(a, b)
jsonb_path_query_array	jsonbPathQueryArray(a, b)
jsonb_path_query_array_tz	jsonbPathQueryArrayTz(a, b)
jsonb_path_query_first	jsonbPathQueryFirst(a, b)
jsonb_path_query_first_tz	jsonbPathQueryFirstTz(a, b)
jsonb_array_length	jsonbArrayLength(a)
jsonb_array_elements	jsonbArrayElements(a)
jsonb_array_elements_text	jsonbArrayElementsText(a)
jsonb_each	jsonbEach(a)
jsonb_each_text	jsonbEachText(a)
jsonb_object_keys	jsonbObjectKeys(a)

PostgreSQL	Portable function
jsonb_insert	jsonbInsert(a, b, c)
jsonb_set	jsonbSet(a, b, c)
jsonb_set_lax	jsonbSetLax(a, b, c)
jsonb_pretty	jsonbPretty(a)
jsonb_strip_nulls	jsonbStripNulls(a)
PC_NumPoints(pcpatch)	numPoints(a)
PC_PointN(pcpatch, integer)	pointN(a, b)

18.9 Aggregations

Every aggregate of MobilityDB, such as `tCount`, `tSum` or `extent`, is called by the same name in every engine, with one exception. Eleven aggregates share their name with a function: `tMin`, `tMax`, `merge`, `appendInstant`, `appendSequence`, `sameAlignment`, `tAnd`, `tOr`, `setUnion`, `spanUnion` and `spansetUnion`. PostgreSQL lets an aggregate and a function share a name, and MobilityDB calls these aggregates by it. DuckDB, Apache Spark and Apache Flink hold one function per name, and call them by the name with `Agg`: `tMinAgg`, `mergeAgg`, `setUnionAgg`, and so on (Section 18.10.1).

An aggregate computes its result in up to three steps, each a function named after the aggregate and the step. The transition function `<aggregate>Transition` adds one value to a running state, the combine function `<aggregate>Combine` joins two states computed in parallel, and the final function `<aggregate>Final` turns the state into the result. The combine and final steps are optional: `extent` has no final function, since its state is already its result, and the append aggregates have no combine function, since they add their values in time order. A query never calls these functions; each engine builds the aggregate from them.

The transition function takes the state and a value, and has one version per type of value, as the aggregate has: `tMinTransition` takes a `tint`, a `tbigint`, a `tfloat` or a `ttext`. The combine and final functions take only states, and a state knows the type of the values it holds, so each is one function per aggregate: `tMinCombine` combines the states of the temporal minimum of every type of value. In PostgreSQL, where a final function declares the type it returns, the final function is declared once per type of value with `FINALFUNC_EXTRA`, as `tMinFinal(internal, tint)` returning a `tint`, all its versions running the same code. In Spark and Flink each aggregate is one object, which chooses its transition function from the type of the value.

The following table lists every aggregate and its functions; a dash (—) marks a step the aggregate does not have.

Category	Transition	Combine	Final
Count	tCountTransition	tCountCombine	tCountFinal
	wCountTransition	wCountCombine	wCountFinal
Sum	tSumTransition	tSumCombine	tSumFinal
	wSumTransition	wSumCombine	wSumFinal
Average	tAvgTransition	tAvgCombine	tAvgFinal
	wAvgTransition	wAvgCombine	wAvgFinal
Minimum / maximum	tMinTransition	tMinCombine	tMinFinal
	tMaxTransition	tMaxCombine	tMaxFinal
	wMinTransition	wMinCombine	wMinFinal
	wMaxTransition	wMaxCombine	wMaxFinal
Boolean	tAndTransition	tAndCombine	tAndFinal
	tOrTransition	tOrCombine	tOrFinal
Centroid	tCentroidTransition	tCentroidCombine	tCentroidFinal
Point cloud	tnpointsTransition	tnpointsCombine	tnpointsFinal
	tdensityTransition	tdensityCombine	tdensityFinal
Bounding box	extentTransition	extentCombine	—
Union	setUnionTransition	setUnionCombine	setUnionFinal
	spanUnionTransition	spanUnionCombine	spanUnionFinal
	spansetUnionTransition	spansetUnionCombine	spansetUnionFinal
Merge	mergeTransition	mergeCombine	mergeFinal
Append	appendInstantTransition	—	appendInstantFinal

Category	Transition	Combine	Final
	appendSequenceTransition	—	appendSequenceFinal
Raster	rasterUnionTransition	rasterUnionCombine	rasterUnionFinal
	sameAlignmentTransition	sameAlignmentCombine	sameAlignmentFinal
	summaryStatsAggTransition	summaryStatsAggCombine	summaryStatsAggFinal

18.10 Running a Query on Apache Spark and Apache Flink

Written with functions only, a query no longer depends on operators. Spark and Flink differ from PostgreSQL in three more ways, and each asks for a change. They already define some of the names MobilityDB uses: both have `round`, `lower` and `length`, and Flink refuses `set` and `unnest` without quotes; such a name takes a prefix naming the class or base type of its argument (Section 18.10.1). They never call the input or output function of a type for you, so a literal, a cast and the text of a value become function calls the query writes (Section 18.10.2). And each engine calls a function returning rows in its own way (Section 18.10.3). The following table sums up these changes; a function it does not list keeps its name.

What the query holds	MobilityDB	Spark and Flink
An operator: its named function, as the sections above give it	trip && box	stboxOverlaps(trip, box)
A name the engine owns, over a value whose base type has the operation: the base type in front	round(speed(trip), 2)	floatRound(speed(trip), 2)
A name the engine owns, over another set, span, span set or temporal value: the class in front	lower(timeSpan(trip))	spanLower(timeSpan(trip))
A name the engine owns, over a box or a base type: the type in front	round(box, 2)	stboxRound(box, 2)
A typed literal: the text constructor of its type	tint '1@2001-01-01'	tintFromText('1@2001-01-01')
The text of a value, which PostgreSQL writes through the output function of its type: asText	SELECT speed(trip)	SELECT asText(speed(trip))
A cast: the function its entry gives beside it, as tbool::tint is tint(tbool)	flag::tint	tint(flag)

18.10.1 Names the Engines Own

A name the engine already defines takes the prefix of the class or base type of its argument, and functions that belong together take the same prefix: `spanLowerInc` goes with `spanLower`, `geoCumulativeLength` with `geoLength`. Spark and Flink use the same names, so a query written for one runs on the other, and `round(1.23456, 2)` keeps meaning the engine's own rounding of a number. The names that change are the following.

MobilityDB name	Class of the argument	Spark and Flink name
lower, upper, initcap	textset, ttext	textLower, textUpper, textInitcap
lower, upper, lowerInc, upperInc	span, span set	spanLower, spanUpper, spanLowerInc, spanUpperInc, spansetLower, spansetUpper, spansetLowerInc, spansetUpperInc
lowerInc, upperInc	temporal	temporalLowerInc, temporalUpperInc
round	float and its set, span, span set and temporal types	floatRound
	geometry and geography, their sets and temporal types, trgeometry	geoRound
	cbuffer, cbufferset, tcbuffer; npoint, npointset, tnpoint; nsegment; pose, poseset, tpose; posechain, posechainset, tposechain; tbox; stbox; tpcbox	cbufferRound, npointRound, nsegmentRound, poseRound, posechainRound, tboxRound, stboxRound, tpcboxRound

MobilityDB name	**Class of the argument**	**Spark and Flink name**
abs	tint, tbigint, tfloat	intAbs, bigintAbs, floatAbs
ceil, floor, degrees, radians, cos, sin, tan, exp, ln, log10	float and its set, span, span set and temporal types	floatCeil, floatFloor, floatDegrees, floatRadians, floatCos, floatSin, floatTan, floatExp, floatLn, floatLog10
transform	the sets and temporal types of geometry and geography, trgeometry	geoTransform
transformPipeline	geometry and geography, their sets and temporal types, trgeometry	geoTransformPipeline
	cbuffer, cbufferset, tcbuffer; pose, poseset, tpose; posechain, posechainset, tposechain; stbox; raster	cbufferTransform, poseTransform, posechainTransform, stboxTransform, rasterTransform, and the . . . TransformPipeline of each
translate, affine, rotate, rotateX, rotateY, rotateZ, scale, transscale	geometry, tgeompoint, tgeometry	geoTranslate, geoAffine, geoRotate, geoRotateX, geoRotateY, geoRotateZ, geoScale, geoTransscale
translate, rotate, rotateX, rotateY, rotateZ	tcbuffer; tpose; tposechain	cbufferTranslate, cbufferRotate, cbufferRotateZ; poseTranslate, poseRotate, poseRotateX, poseRotateY, poseRotateZ; the same for posechain (posechainTranslate, . . .)
	trgeometry	geoTranslate, geoRotate, geoRotateX, geoRotateY, geoRotateZ
width, height, bandPixelType, bandHasNoDataValue, bandNoDataValue, pixels, asHexWKB, stbox	raster; raquet	rasterWidth, rasterHeight, rasterBandPixelType, rasterBandHasNoDataValue, rasterBandNoDataValue, rasterPixels, rasterAsHexWKB, rasterStbox; raquetWidth, raquetHeight, raquetBandPixelType, raquetBandHasNoDataValue, raquetBandNoDataValue, raquetPixels, raquetAsHexWKB, raquetStbox
length, cumulativeLength	geometry, geography, tgeompoint, tgeogpoint, trgeometry	geoLength, geoCumulativeLength
	tnpoint	npointLength, npointCumulativeLength
contains, covers, disjoint, intersects, touches, dwithin, relate	geometry, geography	geoContains, geoCovers, geoDisjoint, geoIntersects, geoTouches, geoDwithin, geoRelate
reverse	geometry	geoReverse
collect	geometry[]	geoCollect
transform	geometry, geography	geoTransform
hash, hashExtended	set, span, span set, box, temporal, base type	setHash, spanHash, spansetHash, tboxHash, stboxHash, temporalHash, cbufferHash, . . . , and their . . . HashExtended
insert, update, merge	temporal	temporalInsert, temporalUpdate, temporalMerge
tMin, tMax, merge, appendInstant, appendSequence, tAnd, tOr (the aggregates)	temporal	tMinAgg, tMaxAgg, mergeAgg, appendInstantAgg, appendSequenceAgg, tAndAgg, tOrAgg
setUnion, spanUnion, spansetUnion (the aggregates)	base value, set, span, span set	setUnionAgg, spanUnionAgg, spansetUnionAgg
sameAlignment (the aggregate)	raster	sameAlignmentAgg
wAvg (the aggregate)	tint, tbigint, tfloat	wAvgAgg
jsonbSet, jsonbSetLax	jsonb	pgJsonbSet, pgJsonbSetLax
unnest	set, temporal	setUnnest, temporalUnnest
set (the constructor)	any	setMake

The functions `jsonbSet` and `jsonbSetLax` take the prefix `pg` of PostgreSQL, which provides them, since Spark and Flink, ignoring the case of a name, would read `jsonbSet` as `jsonbset`, the conversion of a textset into a jsonbset.

The names a raster and a raquet share take the prefix of their type too, even where the engine defines no such name: both travel as WKB, so an engine holding one function per name cannot tell a raster given to `width` from a raquet.

18.10.2 Reading and Writing Values

PostgreSQL reads and writes a value through the four functions its type declares, and calls them for you: a typed literal, a cast from text and a text `COPY` go through the input function, the text a client displays through the output function, and a binary `COPY` or a driver in binary mode through the receive and send functions. Spark and Flink call none of them. A value travels between them as its Well-Known Binary (WKB) bytes, and each conversion is a function the query calls, each with its inverse, as the following table shows.

What PostgreSQL uses	**Direction**	**Spark and Flink**
the input function: a typed literal, a cast from text	text to value	ttypeFromText(text), settypeFromText(text), boxFromText(text), . . . , one per type, prefixed by its name; with an SRID also tspatialFromEWKT(text)
the output function: the text a client prints	value to text	asText(value); with an SRID also asEWKT(value)
the receive function: binary COPY, a binary driver	binary to value	ttypeFromBinary(bytea), ttypeFromHexWKB(text), and likewise for every other type; with an SRID also tspatialFromEWKB(bytea), tspatialFromHexEWKB(text)
the send function	value to binary	asBinary(value), asHexWKB(value); with an SRID also asEWKB(value), asHexEWKB(value)
none	MF-JSON	ttypeFromMFJSON(text), asMFJSON(ttype)

Every type has the functions of the table that apply to it: the `E` forms when it carries an SRID its plain text and binary forms omit, and MF-JSON when it is temporal. This includes the types PostgreSQL reads and writes only through its four functions: the set, span and span set types of integers, big integers, floats, dates and timestamps, textset and jsonbset, tbox and stbox, tbool, tint, tbigint, tfloat and ttext, and quadbin, s2cell, nsegment and tpcbox.

The text of a raquet is its hexadecimal WKB, so it is read with `raquetFromHexWKB` and written with `asHexWKB`, as a PostGIS raster is with `ST_RastFromHexWKB` and `ST_AsHexWKB`. A box, stbox or tpcbox, writes its SRID in both its text and its WKB, and so has no `E` form. Every WKB output takes the byte order `endian`, and every text output of a value with floats or coordinates the number of decimal digits `maxdecimaldigits`. A column of WKB bytes, such as a TemporalParquet column, is read in every engine with the binary constructor of its type, as in `tgeompointFromBinary(col)`.

18.10.3 Functions Returning Rows

A function returning several rows, such as `timeSplit` or `unnest`, is called in PostgreSQL as a table in the `FROM` clause. In Spark and Flink the same function, `unnest` under the names `setUnnest` and `temporalUnnest`, returns an array with one element per row, an array of records when it returns several columns. Flink turns the array into rows with `CROSS JOIN UNNEST`, and Spark with `LATERAL VIEW explode`, or `inline` for an array of records.

18.10.4 Example

The query

```
SELECT vehicle, round(length(trip)::numeric, 1), lower(timeSpan(trip))
FROM Trips
WHERE trip && stbox 'SRID=3857;STBOX X((0,0),(1000,1000))';
```

is written, in Spark and Flink,

```
SELECT vehicle, round(geoLength(trip), 1), spanLower(timeSpan(trip))
FROM Trips
WHERE stboxOverlaps(trip, stboxFromText('SRID=3857;STBOX X((0,0),(1000,1000))'));
```

Here `round` applies to a number, so it is the engine's own function and keeps its name.

18.11 The Rasters of the Engines

A `raster` of MobilityDB is the same type, with the same functions, in every engine (Section 16.24). Two engines also have a raster of their own: DuckDB through its raster extension, and Apache Spark through Apache Sedona; Apache Flink has none. Each host names everything it adds with a prefix, `RT_` in DuckDB and `RS_` in Sedona, and MobilityDB names nothing with either, so the name of a function tells whose raster it works on: `RT_Read` and `RS_FromGeoTiff` are the host's, `summaryStats` and `rasterValue` MobilityDB's. A query computes with MobilityDB's raster and converts a value at the boundary only where it needs the host: to read a raster file with the host's reader, or to hand a raster to one of the host's functions. The conversion keeps every pixel value and the georeferencing, so a raster converted to the host's raster and back is the raster it was.

What the query does	DuckDB with its raster extension	Spark with Apache Sedona
Make the host's raster available	LOAD raster	SedonaContext.create(spark), beside the registration of MobilitySpark
Read a raster file	RT_Read(file), then raster(databand_1, tile_x, tile_y, metadata) for each tile	RS_FromGeoTiff(content), then the conversion of Section 18.11.2
Give a raster to the host's functions	CAST(rast AS RT_DATACUBE) for its pixels, rasterMetadata(rast) for its georeferencing	RS_AddBandFromArray over RS_MakeEmptyRaster, from its accessors
Give the statistics of a band to the host	CAST(stats AS RT_STATS), and back CAST(stats AS summarystats)	none

The two hosts hold different pixel types. Each pixel type of MobilityDB converts to the type of each host on its row, and a raster of a type a host has none for has no raster in that host.

Pixel type of the raster	Datacube of DuckDB	Pixel type of Sedona
1BB, 2BUI, 4BUI	UINT8	none
8BUI	UINT8	B
8BSI	INT8	none
16BUI	UINT16	US
16BSI	INT16	S
32BUI	UINT32	none
32BSI	INT32	I
32BF	FLOAT	F
64BF	DOUBLE	D

18.11.1 DuckDB and Its Raster Extension

DuckDB's raster extension, a community extension, reads a raster file with `RT_Read` as a table with one row per tile: the pixels of each band in a `databand_N` column of type `RT_DATACUBE`, the georeferencing of the whole file in a `metadata` JSON column, and the position of the tile in `tile_x` and `tile_y`. It has no raster value of its own, only these columns. A MobilityDB raster is built from the four columns of a tile, and gives back a datacube and a metadata column, the two columns the extension's own functions read a tile from. The metadata `rasterMetadata` writes is that of a file holding the raster as its one tile, so the raster is built back from it at the tile (0, 0).

The type `RT_DATACUBE` exists only once the extension is loaded, so a query naming it runs after `LOAD raster`; before it, DuckDB answers that no type `RT_DATACUBE` exists. The conversions are explicit casts and functions: a raster is never converted to a datacube by itself when it is passed to an `RT_` function, since a datacube leaves the georeferencing behind.

From	To	SQL
a tile of RT_Read, or any datacube with its metadata	raster	raster(databand_1, tile_x, tile_y, metadata)
raster	RT_DATACUBE	CAST(rast AS RT_DATACUBE)
raster	the metadata of a tile	rasterMetadata(rast)
summarystats	RT_STATS	CAST(stats AS RT_STATS)
RT_STATS	summarystats	CAST(stats AS summarystats)

A datacube holds one pixel type for all its bands, and each pixel type of MobilityDB becomes the datacube type that GDAL gives it, as a PostGIS raster written through GDAL does. The 1-, 2- and 4-bit types have no smaller type in a datacube: their values are kept exactly in 8 bits, and a raster built back from the datacube has the type `8BUI`.

A conversion either keeps every value or raises an error; it never answers an approximation. A datacube holds one pixel type and one nodata value, so a raster whose bands differ in either cannot be cast to `RT_DATACUBE`. A datacube of type `UINT64` or `INT64` has no pixel type of MobilityDB to become. A datacube or a metadata column that is not well formed, truncated, carrying an unknown type or format, or declaring more pixels than it holds, is refused rather than read. The coordinate reference system of the metadata is read as `EPSG:`*n* and gives the SRID of the raster; a raster combined with a value of another SRID raises the error any mix of SRIDs raises. A raster cast to a datacube writes its pixels uncompressed; a datacube is read whichever compression the extension wrote it with, `RAW`, `SNAPPY`, `GZIP`, `ZSTD` or `LZ4`.

```
LOAD raster;
-- The tiles of 2 x 2 pixels of a 4 x 4 GeoTIFF of elevations, as MobilityDB rasters
CREATE TABLE elevation AS
SELECT tile_x, tile_y, raster(databand_1, tile_x, tile_y, metadata) AS rast
FROM RT_Read('elevation.tif', blocksize_x := 2, blocksize_y := 2);
-- MobilityDB's raster functions over them
SELECT count(*), summarystatsMean(summaryStatsAgg(rast, true)) FROM elevation;
-- 4 | 57.06666666666667
-- A MobilityDB raster handed to a function of the extension
SELECT tile_x, tile_y, RT_CubeStats(CAST(rast AS RT_DATACUBE), 0).mean
FROM elevation ORDER BY tile_y, tile_x;
-- 0 | 0 | 53.5
-- 1 | 0 | 68.5
-- 0 | 1 | 46.0
-- 1 | 1 | 61.333333333333336
-- A raster converted to the extension's columns and back is the raster it was
SELECT bool_and(raster(CAST(rast AS RT_DATACUBE), 0, 0, rasterMetadata(rast)) = rast)
FROM elevation;
-- true
```

18.11.2 Spark and Apache Sedona

Apache Sedona registers its raster type and its `RS_` functions in a Spark session through `SedonaContext.create(spark)`, beside the functions of MobilitySpark; none of the names of either is a name of the other. A Sedona raster is a value, read from a file with `RS_FromGeoTiff` and written with `RS_AsGeoTiff`, and it converts to and from a MobilityDB raster through the functions both already have: the georeferencing passes through the accessors of each, and the values of a band as an array.

A Sedona raster is built from a MobilityDB raster `r` by `RS_MakeEmptyRaster` with the georeferencing of `r` and `RS_AddBandFromArray` with the values of a band, its pixel type and its nodata value. A MobilityDB raster is built from a Sedona raster `s` by `raster(pixels, width, height, upperleftx, upperlefty, scalex, scaley, skewx, skewy, srid, pixeltype[, nodataval])` over the `RS_` accessors of `s`, its pixels being the bytes of the values of `RS_BandAsArray(s, 1)` in their order; each value becomes its bytes as the pixel of a one-pixel band, `rasterPixels(addBand(makeEmptyRaster(1, 1, 0.0, 0.0, 1.0, -1.0, 0.0, 0.0, 0), 1, pixeltype, v), 1)`. Both directions read one band; a raster of several bands converts band by band.

The six pixel types Sedona holds convert both ways, with or without a nodata value, without changing a value, and a raster converted to Sedona and back is the raster it was, byte for byte; the others have no Sedona raster.

```
-- A Sedona raster from the georeferencing and the band of a MobilityDB raster r
```

```
SELECT RS_AddBandFromArray(
    RS_MakeEmptyRaster(1, CASE rasterBandPixelType(r, 1) WHEN 'uint8' THEN 'B'
        WHEN 'int16' THEN 'S' WHEN 'uint16' THEN 'US' WHEN 'int32' THEN 'I'
        WHEN 'float32' THEN 'F' WHEN 'float64' THEN 'D' END,
      rasterWidth(r), rasterHeight(r), upperLeftX(r), upperLeftY(r),
      scaleX(r), scaleY(r), skewX(r), skewY(r), SRID(r)),
    flatten(dumpValues(r, 1, false)), 1, rasterBandNoDataValue(r, 1)) AS s
FROM t;
```

Appendix A

MobilityDB Reference

A.1 MobilityDB Types

MobilityDB defines four *template types* that act as type constructors over *base types*. These template types are `set`, `span`, `spanset`, and `temporal`. Table A.1 shows the types defined over these template types. Furthermore, MobilityDB defines two bounding box types, namely, `tbox` and `stbox`.

Base Types	**Template Types**			
	`set`	**`span`**	**`spanset`**	**`temporal`**
`bool`				`tbool`
`text`	`textset`			`ttext`
`jsonb`	`jsonbset`			`tjsonb`
`integer`	`intset`	`intspan`	`intspanset`	`tint`
`bigint`	`bigintset`	`bigintspan`	`bigintspanset`	`tbigint`
`float`	`floatset`	`floatspan`	`floatspanset`	`tfloat`
`date`	`dateset`	`datespan`	`datespanset`	
`timestamptz`	`tstzset`	`tstzspan`	`tstzspanset`	
`geometry`	`geomset`			`tgeompoint`, `tgeometry`
`geography`	`geogset`			`tgeogpoint`, `tgeography`
`pose`	`poseset`			`tpose`, `trgeometry`
`posechain`	`posechainset`			`tposechain`
`npoint`	`npointset`			`tnpoint`
`cbuffer`	`cbufferset`			`tcbuffer`
`h3index`	`h3indexset`			`th3index`
`quadbin`	`quadbinset`			`tquadbin`
`s2cell`	`s2cellset`			`ts2cell`
`pcpoint`	`pcpointset`			`tpcpoint`
`pcpatch`	`pcpatchset`			`tpcpatch`

Table A.1: MobilityDB current instantiations of template types

A.2 Set and Span Types

A.2.1 Input and Output

- asText: Return the Well-Known Text (WKT) representation
- asEWKT: Return the Extended Well-Known Text (EWKT) representation of a spatial set

- asBinary, asHexWKB: Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation
- asEWKB, asHexEWKB: Return the Extended Well-Known Binary (EWKB) or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation of a spatial set
- settypeFromText, spantypeFromText, spansettypeFromText: Input from the Well-Known Text (WKT) representation
- settypeFromBinary, spantypeFromBinary, spansettypeFromBinary: Input from the Well-Known Binary (WKB) representation
- settypeFromHexWKB, spantypeFromHexWKB, spansettypeFromHexWKB: Input from the Hexadecimal Well-Known Binary (HexWKB) representation
- spatialsettypeFromText, spatialsettypeFromEWKT: Input from the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation of a spatial set
- spatialsettypeFromEWKB, spatialsettypeFromHexEWKB: Input from the Extended Well-Known Binary (EWKB) or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation of a spatial set

A.2.2 Constructors

- set: Constructor for set types
- span: Constructor for span types
- spanset: Constructor for span set types

A.2.3 Conversions

- ::, set, span, spanset: Convert a base value to a set, span, or span set value
- ::, spanset: Convert a set value to a span set value
- ::, geometry, geography: Convert a geo set into the collection of its values
- ::, geomset, geogset: Convert between a geometry and a geography
- ::, spanset: Convert a span value to a span set value
- span: Convert a set or a span set into a span, ignoring the potential time gaps
- ::, span, range: Convert a span value to and from a PostgreSQL range value
- ::, spanset, multirange: Convert a span set value to and from a PostgreSQL multirange value

A.2.4 Accessors

- memSize: Return the memory size in bytes
- lower, upper: Return the lower or upper bound
- lowerInc, upperInc: Is the lower or upper bound inclusive?
- width: Return the width of the span as a float
- duration: Return the duration
- numValues, getValues: Return the (number of) values
- startValue, endValue, valueN: Return the start, end, or n-th value
- unnest: Transform a set into a set of rows, one per value of the set, in the order of the set {}

- numSpans, spans: Return the (number of) spans
- startSpan, endSpan, spanN: Return the start, end, or n-th span
- numDates, dates: Return the (number of) different dates
- startDate, endDate, dateN: Return the start, end, or n-th date
- numTimestamps, timestamps: Return the (number of) different timestamps
- startTimestamp, endTimestamp, timestampN: Return the start, end, or n-th timestamp

A.2.5 Transformations

- expand: Expand or shrink the bounds by a value or an interval
- shift: Shift by a value or interval
- scale: Scale by a value or interval
- shiftScale: Shift and scale by the values or intervals
- floor, ceil: Round down or up to the nearest integer
- round: Round to a number of decimal places
- degrees, radians: Convert to degrees or radians
- lower, upper, initcap: Transform to lowercase, uppercase, or initcap
- ||: Text concatenation
- tprecision: Set the temporal precision of the time value to the interval with respect to the origin

A.2.6 Spatial Reference System

- SRID, setSRID: Return or set the spatial reference identifier
- transform, transformPipeline: Transform to a spatial reference identifier

A.2.7 Set Operations

- +, -, *: Union, difference, or intersection of sets or spans

A.2.8 Bounding Box Operations

A.2.8.1 Topological Operations

- &&: Do the values overlap (have values in common)?
- @>: Does the first value contain the second one?
- <@: Is the first value contained by the second one?
- -|-: Is the first value adjacent to the second one?

A.2.8.2 Position Operations

- <<, <<#: Is the first value strictly left of the second one?
- >>, #>>: Is the first value strictly to the right of the second one?
- &<, &<#: Is the first value not to the right of the second one?
- &>, #&>: Is the first value not to the left of the second one?

A.2.8.3 Splitting Operations

- splitNSpans: Return an array of N spans obtained by merging the elements of a set or the spans of a spanset
- splitEachNSpans: Return an array of spans obtained by merging N consecutive elements of a set or N consecutive spans of a spanset

A.2.9 Distance Operations

- <->: Return the smallest distance ever

A.2.10 Comparisons

- =, <>, <, >, <=, >=, cmp: Traditional comparisons
- hash, hashExtended: Hash values

A.2.11 Aggregations

- extent: Bounding span
- setUnion, spanUnion, spansetUnion: Aggregate union
- tCount: Temporal count

A.3 Box Types

A.3.1 Input and Output

- asText: Return the Well-Known Text (WKT) representation
- asEWKT: Return the Extended Well-Known Text (EWKT) representation of a spatiotemporal box
- asBinary, asHexWKB: Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation
- asEWKB, asHexEWKB: Return the Extended Well-Known Binary (EWKB) or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation of a spatiotemporal box
- boxFromText: Input from the Well-Known Text (WKT) representation
- stboxFromEWKT: Input of a spatiotemporal box from the Extended Well-Known Text (EWKT) representation
- boxFromBinary, boxFromHexWKB: Input from the Well-Known Binary (WKB) or from the Hexadecimal Well-Known Binary (HexWKB) representation
- stboxFromEWKB, stboxFromHexEWKB: Input of a spatiotemporal box from the Extended Well-Known Binary (EWKB) or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

A.3.2 Constructors

- tbox: Constructor for `tbox`
- stboxX, stboxZ, stboxT, stboxXT, stboxZT, geodstboxZ, geodstboxZT, stbox: Constructor for `stbox`

A.3.3 Conversions

- ::, intspan, floatspan, tstzspan: Convert a `tbox` to another type
- ::, tbox: Convert another type to a `tbox`
- ::, box2d, box3d, geometry, geography, tstzspan: Convert an `stbox` to a another type
- ::, stbox: Convert another type to an `stbox`

A.3.4 Accessors

- hasX, hasZ, hasT: Has X/Z/T dimension?
- isGeodetic: Is geodetic?
- xMin, yMin, zMin, tMin: Return the minimum X/Y/Z/T value
- xMax, yMax, zMax, tMax: Return the maximum X/Y/Z/T value
- xMinInc, tMinInc: Is the minimum X/T value inclusive?
- xMaxInc, tMaxInc: Is the maximum X/T value inclusive?
- area, perimeter, volume: Area, volume, perimeter

A.3.5 Transformations

- shiftValue, scaleValue, shiftScaleValue: Shift and/or scale the span of a bounding box by one or two values
- shiftTime, scaleTime, shiftScaleTime: Shift and/or scale the span or the period of the bounding box to a value or interval
- getSpace: Return the spatial dimension of the bounding box, removing the temporal dimension if any
- expandValue, expandSpace, expandTime: Expand the numeric, spatial, or temporal dimension of a bounding box by a value or an interval
- round: Round the value or the coordinates of the bounding box to a number of decimal places

A.3.6 Spatial Reference System

- SRID, setSRID: Return or set the spatial reference identifier
- transform, transformPipeline: Transform to a spatial reference identifier

A.3.7 Splitting Operations

- quadSplit: Split the bounding box in quadrants or octants {}

A.3.8 Set Operations

- +, *: Union and intersection of two bounding boxes

A.3.9 Bounding Box Operations

A.3.9.1 Topological Operations

- &&: Do the bounding boxes overlap?
- @>: Does the first bounding box contain the second one?
- <@: Is the first bounding box contained in the second one?
- ~=: Are the bounding boxes equal in their common dimensions?
- -|-: Are the bounding boxes adjacent?

A.3.9.2 Position Operations

- <<, <<|, <</, <<#: Are the X/Y/Z/T values of the first bounding box strictly less than those of the second one?
- >>, |>>, />>, #>>: Are the X/Y/Z/T values of the first bounding box strictly greater than those of the second one?
- &<, &<|, &</, &<#: Are the X/Y/Z/T values of the first bounding box not greater than those of the second one?
- &>, |&>, /&>, #&>: Are the X/Y/Z/T values of the first bounding box not less than those of the second one?

A.3.10 Comparisons

- =, <>, <, >, <=, >=, cmp: Traditional comparisons
- hash, hashExtended: Hash values

A.3.11 Aggregations

- extent: Bounding box extent

A.4 Temporal Types

A.4.1 Input and Output

- asText: Return the Well-Known Text (WKT) representation
- asMFJSON: Return the Moving Features JSON (MF-JSON) representation
- asBinary, asHexWKB: Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation
- ttypeFromText: Input from the Well-Known Text (WKT) representation
- ttypeFromMFJSON: Input from the Moving Features JSON (MF-JSON) representation
- ttypeFromBinary, ttypeFromHexWKB: Input from the Well-Known Binary (WKB) or from the Hexadecimal Well-Known Binary (HexWKB) representation

A.4.2 Constructors

- ttype: Constructors for temporal types having a constant value
- ttypeSeq: Constructors for temporal types of sequence subtype
- ttypeSeqSet, ttypeSeqSetGaps: Constructors for temporal types of sequence set subtype

A.4.3 Conversions

- ::: Convert a temporal value to a timestamptz span

A.4.4 Accessors

- memSize: Return the memory size in bytes
- tempSubtype: Return the temporal type
- tempBasetype: Return the base type
- interp: Return the interpolation
- getValue, getTimestamp: Return the value or the timestamp of an instant
- getValues, getTime: Return the values or the time on which a temporal value is defined
- timeSpan: Return the bounding period on which a temporal value is defined
- startValue, endValue, valueN: Return the start, end, or n-th value
- valueAtTimestamp: Return the value at a timestamp
- duration: Return the time interval
- lowerInc, upperInc: Is the start/end instant inclusive?
- numInstants, instants: Return the (number of) different instants
- startInstant, endInstant, instantN: Return the start, end, or n-th instant
- numTimestamps, timestamps: Return the (number of) different timestamps
- startTimestamp, endTimestamp, timestampN: Return the start, end, or n-th timestamp
- numSequences, sequences: Return the (number of) sequences
- startSequence, endSequence, sequenceN: Return the start, end, or n-th sequence
- segments: Return the segments

A.4.5 Transformations

- ttypeInst, ttypeSeq, ttypeSeqSet: Transform a temporal value to another subtype
- setInterp: Transform a temporal value to another interpolation
- segmentMinDuration, segmentMaxDuration: Return a temporal boolean stating whether each segment of a continuous temporal sequence (set) has, respectively, at least or at most a given duration
- shiftTime, scaleTime, shiftScaleTime: Shift and/or scale the time span of a temporal value by one or two intervals
- unnest: Transform a nonlinear temporal value into a set of rows, each one is a pair composed of a base value and a period set during which the temporal value has the base value {}

A.4.6 Modifications

- insert: Insert a temporal value into another one
- update: Update a temporal value with another one
- deleteTime: Delete the instants of a temporal value that intersect a time value
- appendInstant: Append a temporal instant to a temporal value
- appendSequence: Append a temporal sequence to a temporal value
- merge: Merge the temporal values

A.4.7 Restrictions

- atValue, minusValue, atValues, minusValues, atSpan, minusSpan, atSpanset, minusSpanset: Restrict to (the complement of) a value or a set of values, or, for a temporal number, a span or a span set of values
- atTime, minusTime: Restrict to (the complement of) a time value
- beforeTimestamp, afterTimestamp: Keep the instants of a temporal value that are before/after or equal a timestamp

A.4.8 Comparisons

A.4.8.1 Traditional Comparisons

- =, <>, <, >, <=, >=, cmp: Traditional comparisons
- hash, hashExtended: Hash values

A.4.8.2 Ever and Always Comparisons

- ?=, ?<>, ?<, ?>, ?<=, ?>=, %=, %<>, %<, %>, %<=, %>=: Ever and always comparisons

A.4.8.3 Temporal Comparisons

- #=, #<>, #<, #>, #<=, #>=: Temporal comparisons

A.4.9 Miscellaneous

- mobilitydbVersion: Version of the MobilityDB extension
- mobilitydbFullVersion: Versions of the MobilityDB extension and its dependencies

A.5 Temporal Alphanumeric Types

A.5.1 Conversions

- ::, valueSpan(tnumber), timeSpan(tnumber), tbox(tnumber), valueSpan, timeSpan, tbox: Convert a temporal number to a span or to a temporal bounding box
- ::, tint(tbool), tint: Convert between a temporal Boolean and a temporal integer
- tnumber, ::, tnumber(tnumber): Convert between temporal integers, temporal big integers, and temporal floats

A.5.2 Accessors

- valueSpan: Return the span of values, ignoring any gaps
- valueSet: Return the values of a temporal number as a set
- minValue, maxValue, avgValue: Return the minimum, maximum, or average value
- minInstant, maxInstant: Return the instant with the minimum or maximum value

A.5.3 Transformations

- shiftValue, scaleValue, shiftScaleValue: Shift and/or the value span of a temporal number by one or two numbers
- stops: Extract from a temporal float with linear interpolation the subsequences where the values stay within a span of a given width for at least a given duration

A.5.4 Restrictions

- atMin, atMax, minusMin, minusMax: Restrict to (the complement of) the minimum or maximum value
- atTbox, minusTbox: Restrict to (the complement of) a `tbox`

A.5.5 Boolean Operations

- &, |: Temporal and, temporal or
- ~: Temporal not
- whenTrue: Return the time when a temporal Boolean takes the value true

A.5.6 Mathematical Operations

- +, -, *, /: Temporal addition, subtraction, multiplication, and division
- abs: Return the absolute value of the temporal number
- floor, ceil: Round up or down to the neareast integer
- round: Round to a number of decimal places
- degrees, radians: Convert to degrees or radians
- deltaValue: Return the value difference between consecutive instants of the temporal number
- trend: Return the trend of a temporal float with linear interpolation, which states whether its value is increasing, constant, or decreasing, represented, respectively, by 1, 0, and -1
- derivative: Return the derivative over time of a temporal float in units per second
- integral: Return the area under the curve
- twAvg: Return the time-weighted average
- ln, log10: Return the natural logarithm and the base 10 logarithm of a temporal float
- exp: Return the exponential (e raised to the given power) of a temporal float
- sin, cos, tan: Return the sine, cosine, or tangent of a temporal float (argument in radians)

A.5.7 Text Operations

- ||: Text concatenation
- lower, upper, initcap: Transform in lowercase, uppercase, or initcap

A.6 Temporal Geometry Types

A.6.1 Functions on Geometries

- orientedEnvelope: Return the rectangle of minimum area enclosing a geometry
- convexHull: Return the smallest convex geometry enclosing a geometry
- isSimple: Return true if a geometry has no point at which it crosses or touches itself
- buffer: Return the geometry holding every point within a distance of a geometry
- relate: Return the DE-9IM intersection matrix of two geometries or whether it matches a pattern
- contains, covers, disjoint, intersects, touches, geoEquals: Return true if two geometries satisfy a spatial relationship
- dwithin: Return true if two geometries are within a distance
- area, perimeter, centroid: Return the area, the perimeter or the centroid of a geometry or a geography
- distance, shortestLine, maxDistance: Return the distance, the line joining the closest points, or the maximum distance of two geometries or geographies
- boundary, reverse, numGeometries, geometryN, numPoints: Return the boundary, the reversal or the parts of a geometry
- lineInterpolatePoint, lineSubstring, lineLocatePoint: Return the point at a fraction of a line, the part between two fractions, or the fraction closest to a point
- collect, makeLine: Return the collection of an array of geometries or the line joining them
- asText, asEWKT, asEWKB, asGeoJSON: Return the Well-Known Text, Extended Well-Known Text, Extended Well-Known Binary or GeoJSON representation of a geometry or a geography
- transform: Transform a geometry or a geography to a spatial reference identifier
- segmentize: Densify a geometry or a geography so that no segment is longer than a distance
- clusterKMeans, clusterDBSCAN, clusterIntersecting, clusterWithin: Return the clusters of an array of geometries
- intersection: Return the region two geometries share
- geoUnion: Return the region two geometries cover together
- difference: Return the region of the first geometry that the second does not cover
- symDifference: Return the region exactly one of two geometries covers

A.6.2 Input and Output

- asText, asEWKT: Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation 🧊🌐
- asMFJSON: Return the Moving Features JSON representation 🧊🌐
- asBinary, asEWKB, asHexEWKB: Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation 🧊🌐
- tspatialFromText, tspatialFromEWKT: Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation 🧊🌐
- tspatialFromMFJSON: Input from the Moving Features JSON representation 🧊🌐
- tspatialFromBinary, tspatialFromEWKB, tspatialFromHexWKB, tspatialFromHexEWKB, geometryFromHexEWKB, geographyFromHexEWKB: Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), from the Hexadecimal Well-Known Binary (HexWKB), or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation 🧊🌐

A.6.3 Conversions

- stbox, ::, stbox(tspatial): Convert a spatiotemporal value to a spatiotemporal box
- ::: Convert between a temporal geometry and a temporal geography
- ::: Convert between a temporal point and a PostGIS trajectory

A.6.4 Accessors

- trajectory, traversedArea: Return the trajectory or the traversed area 🧊🌐
- centroid: Return the centroid as a temporal point 🌐
- getX, getY, getZ: Return the X/Y/Z coordinate values as a temporal float 🧊🌐
- isSimple: Return true if the temporal point does not spatially self-intersect 🧊
- length: Return the length traversed by the temporal point 🧊🌐
- cumulativeLength: Return the cumulative length traversed by the temporal point 🧊🌐
- speed: Return the speed of the temporal point in units per second 🧊🌐
- twCentroid: Return the time-weighted centroid 🧊
- direction: Return the direction, that is, the azimuth between the start and end locations 🧊🌐
- azimuth: Return the temporal azimuth 🧊🌐
- angularDifference: Return the temporal angular difference 🌐
- bearing: Return the temporal bearing 🧊🌐

A.6.5 Transformations

- round: Round the coordinate values to a number of decimal places ⬡🌐
- makeSimple: Return an array of fragments of the temporal point which are simple ⬡
- geoMeasure: Construct a geometry/geography with M measure from a temporal point and a temporal float ⬡🌐
- force2D, force3D: Return a temporal geometry with no Z or with a Z, as the PostGIS functions `ST_Force2D` and `ST_Force3D` do for a geometry ⬡🌐
- affine: Return the 3D affine transform of a temporal geometry to translate, rotate, and/or scale it in a single step ⬡
- rotate: Return the temporal point rotated counter-clockwise about the origin point
- rotateX: Return a temporal geometry rotated counter-clockwise about the x axis ⬡
- rotateY: Return a temporal geometry rotated counter-clockwise about the y axis ⬡
- rotateZ: Return a temporal geometry rotated counter-clockwise about the z axis ⬡
- scale: Return a temporal point scaled by given factors ⬡
- transscale: Return a temporal geometry translated by given offsets and then scaled by given factors ⬡
- translate: Return a temporal geometry translated by given offsets ⬡
- asMVTGeom: Transform a temporal geometric point into the coordinate space of a Mapbox Vector Tile ⬡
- stops: Extract from a temporal point with linear interpolation the subsequences where the point stays within an area with a specified maximum size for at least the given duration ⬡🌐

A.6.6 Restrictions

- atGeometry, minusGeometry: Restrict to (the complement of) a geometry ⬡
- atStbox, minusStbox: Restrict to (the complement of) an `stbox` ⬡
- atElevation, minusElevation: Restrict to (the complement of) an elevation span ⬡

A.6.7 Spatial Reference System

- SRID, setSRID: Return or set the spatial reference identifier ⬡🌐
- transform, transformPipeline: Transform to a spatial reference identifier ⬡🌐

A.6.8 Bounding Box Operations

- expandSpace: Return the spatiotemporal bounding box expanded in the spatial dimension by a float value ⬡🌐

A.6.9 Distance Operations

- |=|: Return the smallest distance ever 🧊 🌐
- nearestApproachInstant: Return the instant of the first temporal point at which the two arguments are at the nearest distance 🧊 🌐
- minDistance: Return the minimum spatial distance between two sets of temporal geometries, ignoring time
- eDwithinPairs, aDwithinPairs, tDwithinPairs, eIntersectsPairs, aIntersectsPairs, tIntersectsPairs, eDisjointPairs, aDisjointPairs, tDisjointPairs, eTouchesPairs, aTouchesPairs, tTouchesPairs: Return the qualifying index pairs of two arrays of temporal geometries, generalising the scalar spatial relationships to a set-set spatial join
- shortestLine: Return the line connecting the nearest approach point 🧊 🌐
- <->: Return the temporal distance 🧊 🌐

A.6.10 Ever and Always Relationships

- eContains, aContains: Ever or always contains
- eCovers, aCovers: Ever or always covers
- eDisjoint, aDisjoint: Is ever or always disjoint 🧊 🌐
- eDwithin, aDwithin: Is ever or always at distance within 🧊 🌐
- eIntersects, aIntersects: Ever or always intersects 🧊 🌐
- eTouches, aTouches: Ever or always touches

A.6.11 Spatiotemporal Relationships

- tContains: Temporal contains
- tDisjoint: Temporal disjoint 🧊 🌐
- tDwithin: Temporal distance within 🧊 🌐
- tIntersects: Temporal intersects 🧊 🌐
- tTouches: Temporal touches

A.7 Operations for Temporal Types: Analytics Operations

A.7.1 Simplification

- minDistSimplify, minTimeDeltaSimplify: Return a temporal float or a temporal geometry/geography simplified ensuring that consecutive values are at least a certain distance or time interval apart 🧊 🌐
- maxDistSimplify, douglasPeuckerSimplify: Return a temporal float or a temporal point simplified using the Douglas-Peucker algorithm 🧊

A.7.2 Reduction

- tsample: Sample a temporal value with respect to an interval
- tprecision: Reduce the temporal precision of a temporal value with respect to an interval computing the time-weighted average/centroid in each time bin

A.7.3 Similarity

- hausdorffDistance, frechetDistance, dynTimeWarpDistance, averageHausdorffDistance, lcssDistance: Return the discrete Hausdorff distance, the discrete Fréchet distance, the Dynamic Time Warp (DTW) distance, the average Hausdorff distance, or the Longest Common SubSequence (LCSS) distance between two temporal values
- frechetDistancePath, dynTimeWarpPath: Return the correspondence pairs between two temporal values with respect to the discrete Fréchet distance or the Dynamic Time Warp Distance {}

A.7.4 Extended Kalman Filter

- extendedKalmanFilter: Return a smoothed temporal float or temporal point obtained by filtering a noisy trajectory with an Extended Kalman Filter

A.7.5 Reachability Filter

- reachabilityFilter: Return a temporal float or temporal point with the instants the reachability filter keeps
- reachabilityFilterAgg: Return a temporal float or temporal point with the instants the reachability filter keeps among the instants of a group

A.7.6 Splitting Operations

- splitNSpans: Return an array of N time spans obtained by merging the instants or segments of a temporal value
- splitEachNSpans: Return an array of time spans obtained by merging N consecutive instants or segments of a temporal value
- splitNTboxes: Return an array of N temporal boxes obtained by merging the instants or segments of a temporal number
- splitEachNTboxes: Return an array of temporal boxes obtained by merging N consecutive instants or segments of a temporal number
- splitNStboxes: Return either an array of N spatial boxes obtained by merging the segments of a (multi)line or an array of N spatiotemporal boxes obtained by merging the instants or segments of a temporal point
- splitEachNStboxes: Return either an array of spatial boxes obtained by merging N consecutive segments of a (multi)line or an array of spatiotemporal boxes obtained by merging N consecutive instants or segments of a temporal point

A.7.7 Multidimensional Tiling

A.7.7.1 Bin Operations

- bins: Return an array of bins that cover a value or time span with bins of the same size or duration
- getBin: Return the bin that contains a number or a timestamp

A.7.7.2 Tile Operations

- valueTiles, timeTiles, valueTimeTiles: Return the set of tiles that covers a temporal box with tiles of the same size and/or duration {}
- spaceTiles, timeTiles, spaceTimeTiles: Return the set of tiles that covers a spatiotemporal box with tiles of the same size and/or duration ⬡ {}
- getValueTile, getTboxTimeTile, getValueTimeTile: Return the temporal tile that covers a value and/or a timestamp
- getSpaceTile, getStboxTimeTile, getSpaceTimeTile: Return the spatiotemporal tile that covers a point and/or a timestamp ⬡

A.7.7.3 Bounding Box Operations

- valueBins, timeBins: Return an array of value or time spans obtained from the instants or segments of a temporal number with respect to a value or time grid
- valueBoxes, timeBoxes, valueTimeBoxes: Return an array of temporal boxes obtained from the instants or segments of a temporal number with respect to a value and/or time grid
- spaceBoxes, timeBoxes, spaceTimeBoxes: Return an array of spatiotemporal boxes obtained from the instants or segments of a temporal point with respect to a space and/or time grid ⬡

A.7.7.4 Splitting Operations

- valueSplit, timeSplit, valueTimeSplit: Split a temporal number with respect to value and/or time bins {}
- spaceSplit, timeSplit, spaceTimeSplit: Split a temporal point with respect to a spatial grid ⬡ {}

A.7.8 Aggregation

- tCount: Temporal count
- extent: Bounding box extent
- tAnd, tOr: Temporal and, temporal or
- tMin, tMax, tSum, tAvg: Temporal minimum, maximum, sum, and average
- wCount: Window count
- wMin, wMax, wSum, wAvg: Window minimum, maximum, sum, and average
- tCentroid: Temporal centroid

A.8 Temporal Poses and Pose Chains

A.8.1 Static Poses and Pose Chains

A.8.1.1 Input and Output

- asText, asEWKT: Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation
- asBinary, asEWKB, asHexWKB, asHexEWKB: Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), the Hexadecimal Well-Known Binary (HexWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

- poseFromText, poseFromEWKT: Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation
- poseFromBinary, poseFromEWKB, poseFromHexWKB, poseFromHexEWKB: Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), from the Hexadecimal Well-Known Binary (HexWKB), or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

A.8.1.2 Constructors

- geodpose: Constructor for geodetic poses from a geography point
- pose: Constructor for poses

A.8.1.3 Conversions

- ::, stbox: Convert a pose and, optionally, a timestamp or a period, into a spatiotemporal box
- ::: Convert a pose, planar or geodetic, into a geography point
- ::: Convert a pose, planar or geodetic, into a geometry point

A.8.1.4 Accessors

- point: Return the point, which for a pose chain is the point of its innermost frame
- quaternion: Return the orientation quaternion of a pose

A.8.1.5 Transformations

- round: Round the point and the orientation of the pose to the number of decimal places

A.8.1.6 Spatial Reference System

- SRID, setSRID: Return or set the spatial reference identifier, which for a pose chain is the identifier of its outer frame
- transform, transformPipeline: Transform to a spatial reference identifier

A.8.1.7 Distance Operations

- <->: Return the distance
- |=|: Return the nearest approach distance

A.8.1.8 Comparisons

- =, <>, <, >, <=, >=: Traditional comparisons

A.8.2 Pose Chain Composition

- posechain: Construct a pose chain from an array of poses ordered from the outermost frame inwards
- posechain: Convert a pose into a pose chain of a single link
- pose: Return the pose of a frame the chain defines, read in the outer frame of the chain
- appendPose: Return a pose chain with a pose appended as its innermost link
- numPoses: Return the number of links of a pose chain
- startPose, endPose, poseN, poses: Return the links of a pose chain
- numPoses: Return the number of links every value of a temporal pose chain holds

A.8.3 Temporal Poses and Pose Chains

A.8.3.1 Input and Output

- asText, asEWKT: Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation
- asMFJSON: Return the Moving Features JSON (MF-JSON) representation
- asBinary, asEWKB, asHexWKB, asHexEWKB: Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), the Hexadecimal Well-Known Binary (HexWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation
- tposeFromText, tposeFromEWKT: Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation
- tposeFromMFJSON, tposechainFromMFJSON: Input from the Moving Features JSON (MF-JSON) representation
- tposeFromBinary, tposeFromEWKB, tposeFromHexWKB, tposeFromHexEWKB: Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), from the Hexadecimal Well-Known Binary (HexWKB), or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

A.8.3.2 Constructors

- tpose: Constructor for temporal poses having a constant value
- tposeSeq: Constructor for temporal poses of sequence subtype
- tposeSeqSet, tposeSeqSetGaps: Constructor for temporal poses of sequence set subtype
- tpose: Construct a 2D temporal pose from a temporal geometry point and a temporal float

A.8.3.3 Conversions

- ::: Convert a temporal pose, planar or geodetic, into a temporal geometry point
- ::: Convert a temporal pose, planar or geodetic, into a temporal geography point

A.8.3.4 Accessors

- getValues: Return the values
- points: Return the points
- trajectory: Return the trajectory
- valueAtTimestamp: Return the value at a timestamp
- speed: Return the speed of a temporal pose as a temporal float, the magnitude of the position-component velocity (distance travelled per unit time)
- angularSpeed: Return the angular speed of a temporal pose as a step-interpolated temporal float (radians per unit time)
- getValue, getValues, timeSpan: Return the value of a temporal instant, the set of values, or the time span

A.8.3.5 Transformations

- tposeInst, tposeSeq, tposeSeqSet: Transform to another subtype
- setInterp: Transform to another interpolation
- round: Round the points and the orientations of the temporal pose to the number of decimal places
- translate: Return a temporal value translated by given offsets ⬡
- rotate: Return a temporal value rotated counter-clockwise about the origin or about a point ⬡
- rotateX: Return a temporal value rotated counter-clockwise about the x axis ⬡
- rotateY: Return a temporal value rotated counter-clockwise about the y axis ⬡
- rotateZ: Return a temporal value rotated counter-clockwise about the z axis ⬡
- force2D: Return a temporal pose projected to the ground ⬡

A.8.3.6 Modifications

- appendInstant, appendSequence: Append a temporal instant or a temporal sequence
- merge: Merge the temporal poses
- insert, update, deleteTime: Insert or update the second temporal pose chain into the first one, or delete a time value from it

A.8.3.7 Restrictions

- atValue, minusValue, atValues, minusValues: Restrict to (the complement of) a value or a set of values
- atGeometry, minusGeometry: Restrict to (the complement of) a geometry
- atElevation, minusElevation: Restrict to (the complement of) an elevation span
- atTime, atValue: Restrict a temporal pose chain to a time or to a value

A.8.3.8 Spatial Reference System

- SRID, setSRID: Return or set the spatial reference identifier, which for a temporal pose chain is the identifier of its outer frame
- transform, transformPipeline: Transform to a spatial reference identifier

A.8.3.9 Bounding Box Operations

- &&, <@, @>, ~=, -|-: Topological operators
- :::: Position operators
- expandSpace, stboxes: Return the spatiotemporal bounding box expanded in the spatial dimension, or the array of boxes covering the temporal pose chain

A.8.3.10 Distance Operations

- |=|: Return the smallest distance ever
- nearestApproachInstant: Return the instant of the first temporal pose at which the two arguments are at the nearest distance
- shortestLine: Return the line connecting the nearest approach point
- <->: Return the temporal distance

A.8.3.11 Ever and Always Relationships

- eContains, aContains, eCovers, aCovers, eDisjoint, aDisjoint, eDwithin, aDwithin, eIntersects, aIntersects, eTouches, aTouches: Ever and always relationships

A.8.3.12 Spatiotemporal Relationships

- tContains, tCovers, tDisjoint, tDwithin, tIntersects, tTouches: Spatiotemporal relationships

A.8.3.13 Comparisons

- :::: Traditional comparisons
- ?=, %=, ?<>, %<>: Ever and always comparisons
- :::: Temporal comparisons
- eEqual: Compare two temporal pose chains, or a temporal pose chain and a pose chain

A.8.3.14 Aggregations

- tCount: Temporal count
- wCount: Window count
- extent: Bounding box extent
- appendInstant: Append the aggregated instants into a sequence
- appendSequence: Append the aggregated sequences into a sequence set

A.8.3.15 Spatial Operations

- centroid: Return the temporal geometry point given by the points of a temporal pose
- convexHull: Return the convex hull of the points of a temporal pose

A.8.3.16 Simplification

- minDistSimplify, minTimeDeltaSimplify: Return a temporal pose simplified ensuring that consecutive points are at least a certain distance or time interval apart
- maxDistSimplify, douglasPeuckerSimplify: Return a temporal pose simplified using the Douglas-Peucker algorithm

A.8.4 OGC GeoPose v1.0 Support

A.8.4.1 JSON Input and Output

- asGeoPose, poseFromGeoPose: Convert to or from the OGC GeoPose v1.0 JSON encoding (Basic-Quaternion, Basic-YPR or Advanced conformance class)

A.8.4.2 Quaternion Renormalization

- poseNormalize: Return a pose whose orientation quaternion has been transformed to a unit norm
- poseInverse: Return the inverse of a pose

A.8.4.3 Euler-Angle Accessors

- yaw, pitch, roll: Return the yaw, pitch, or roll angle of a (temporal) pose, in radians, under the ZYX intrinsic Tait-Bryan convention required by the OGC GeoPose Basic-YPR conformance class
- ypr: Return the orientation of a pose as a yaw / pitch / roll triple, in radians
- yaw, pitch, roll: Lift the Euler angle accessors over a temporal pose, returning a temporal float in radians

A.8.4.4 Frame Metadata Registry

- geoPoseFrames: Return every frame the standard and the encoders name
- geoPoseFrameSRID: Return the SRID of a frame MEOS registers, or `NULL` if the frame is parametric (LTP, BODY)
- geoPoseFrameName: Return the human-readable name of a frame MEOS registers
- geoPoseFrameIsGeographic: Return `true` for a lat/lon/h frame MEOS registers, `false` for a Cartesian or projected one

A.8.4.5 Body and World Rigid Transform

- applyPose: Applies the rigid-body transform encoded by a pose (the OGC GeoPose *body to world* mapping) to a body-frame geometry, producing the corresponding world-frame geometry

A.8.4.6 Temporal JSON I/O

- asGeoPose, tposeFromGeoPose: Convert a temporal pose to or from its OGC GeoPose v1.0 encoding
- asGeoPoseStream: Write a temporal pose as an OGC GeoPose Stream
- asGeoPoseStreamHeader, asGeoPoseStreamElement: Write the OGC GeoPose StreamHeader of a temporal pose, or the StreamElement of one of its instants

A.8.4.7 Composite Chain and Graph Documents

- asGeoPose, tposechainFromGeoPose: Write or read an OGC GeoPose Composite Chain document
- asGeoPose: Write an OGC GeoPose Composite Graph document

A.9 Temporal Network Points

A.9.1 Static Network Types

A.9.1.1 Input and Output

- asText, asEWKT: Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation
- asBinary, asEWKB, asHexWKB, asHexEWKB: Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), the Hexadecimal Well-Known Binary (HexWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation
- npointFromText, npointFromEWKT, nsegmentFromText, nsegmentFromEWKT: Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation
- npointFromBinary, npointFromEWKB, npointFromHexWKB, npointFromHexEWKB: Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), from the Hexadecimal Well-Known Binary (HexWKB), or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation
- asBinary, asEWKB, asHexWKB, asHexEWKB: Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), the Hexadecimal Well-Known Binary (HexWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation of a network segment
- nsegmentFromBinary, nsegmentFromEWKB, nsegmentFromHexWKB, nsegmentFromHexEWKB: Input of a network segment from the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), the Hexadecimal Well-Known Binary (HexWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

A.9.1.2 Constructors

- npoint, nsegment: Constructors for network points and network segments

A.9.1.3 Conversions

- ::: Convert between a network point or segment and a geometry
- stbox: Construct a spatiotemporal box from a network point and, optionally, a timestamp or a period

A.9.1.4 Accessors

- route: Return the route identifier
- getPosition: Return the position
- startPosition, endPosition: Return the start/end position

A.9.1.5 Transformations

- round: Round the position(s) of the network point or the network segment to the number of decimal places

A.9.1.6 Spatial Operations

- SRID: Return the spatial reference identifier

A.9.1.7 Comparisons

- =, <>, <, >, <=, >=: Traditional comparisons

A.9.2 Temporal Network Points

A.9.2.1 Constructors

- tnpoint, tnpointSeq, tnpointSeqSet: Constructor for temporal network points having a constant value
- tnpointSeq: Constructor for temporal network points of sequence subtype
- tnpointSeqSet, tnpointSeqSetGaps: Constructor for temporal network points of sequence set subtype

A.9.2.2 Conversions

- ::: Convert between a temporal network point and a temporal geometry point

A.9.2.3 MF-JSON Input and Output

- asMFJSON, tnpointFromMFJSON: Output a temporal network point as MF-JSON and read it back; the round trip is lossless

A.9.2.4 Accessors

- getValues: Return the values
- routes: Return the road identifiers
- valueAtTimestamp: Return the value at a timestamp
- length: Return the length traversed by the temporal network point
- cumulativeLength: Return the cumulative length traversed by the temporal network point
- speed: Return the speed of the temporal network point in units per second

A.9.2.5 Transformations

- tnpointInst, tnpointSeq, tnpointSeqSet: Transform a temporal network point to another subtype
- setInterp: Transform a temporal network point to another interpolation
- round: Round the fraction of the temporal network point to the number of decimal places

A.9.2.6 Modifications

- appendInstant, appendSequence: Append a temporal instant or a temporal sequence
- merge: Merge the temporal network points

A.9.2.7 Restrictions

- atValue, minusValue, atValues, minusValues: Restrict to (the complement of) a value or a set of values
- atGeometry, minusGeometry: Restrict to (the complement of) a geometry

A.9.2.8 Spatial Operations

- twCentroid: Return the time-weighted centroid

A.9.2.9 Bounding Box Operations

- ?@, @?, @@, @=: Route operators, which compare the route identifiers a temporal network point visits: contained in, contains, overlaps, and equal
- &&, <@, @>, ~=, -|-: Topological operators
- :::: Position operators

A.9.2.10 Distance Operations

- |=|: Return the smallest distance ever
- nearestApproachInstant: Return the instant of the first temporal network point at which the two arguments are at the nearest distance
- shortestLine: Return the line connecting the nearest approach point between the two arguments
- <->: Return the temporal distance

A.9.2.11 Ever and Always Relationships

- eContains, aContains, eCovers, aCovers, eDisjoint, aDisjoint, eDwithin, aDwithin, eIntersects, aIntersects, eTouches, aTouches: Ever and always relationships

A.9.2.12 Spatiotemporal Relationships

- tContains, tCovers, tDisjoint, tDwithin, tIntersects, tTouches: Spatiotemporal relationships

A.9.2.13 Comparisons

- =, <>, <, >, <=, >=: Traditional comparisons
- ?=, ?<>, %=, %<>: Ever and always comparisons
- #=, #<>: Temporal comparisons

A.9.2.14 Aggregations

- tCount: Temporal count
- wCount: Window count
- tCentroid: Temporal centroid

A.9.2.15 Aggregation

- extent: Return the spatiotemporal bounding box covering every materialised position in a set of network points

A.10 Temporal Circular Buffers

A.10.1 Static Circular Buffers

A.10.1.1 Input and Output

- asText, asEWKT: Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation
- asBinary, asEWKB, asHexWKB, asHexEWKB: Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), the Hexadecimal Well-Known Binary (HexWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation
- cbufferFromText, cbufferFromEWKT: Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation
- cbufferFromBinary, cbufferFromEWKB, cbufferFromHexWKB, cbufferFromHexEWKB: Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), from the Hexadecimal Well-Known Binary (HexWKB), or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

A.10.1.2 Constructors

- cbuffer: Constructor for circular buffers

A.10.1.3 Conversions

- ::: Convert between a circular buffer and a geometry
- stbox: Convert a circular buffer and, optionally, a timestamp or a period, to a spatiotemporal box

A.10.1.4 Accessors

- point: Return the point
- radius: Return the radius

A.10.1.5 Transformations

- round: Round the point and the radius of the circular buffer to the number of decimal places

A.10.1.6 Spatial Operations

- SRID, setSRID: Return or set the spatial reference identifier
- transform, transformPipeline: Transform to a spatial reference identifier

A.10.1.7 Distance Operations

- <->: Return the distance
- |=|: Return the nearest approach distance

A.10.1.8 Comparisons

- =, <>, <, >, <=, >=: Traditional comparisons

A.10.2 Temporal Circular Buffers

A.10.2.1 Input and Output

- asText, asEWKT: Return the Well-Known Text (WKT) or the Extended Well-Known Text (EWKT) representation
- asMFJSON: Return the Moving Features JSON (MF-JSON) representation
- asBinary, asEWKB, asHexWKB, asHexEWKB: Return the Well-Known Binary (WKB), the Extended Well-Known Binary (EWKB), the Hexadecimal Well-Known Binary (HexWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation
- tcbufferFromText, tcbufferFromEWKT: Input from the Well-Known Text (WKT) or from the Extended Well-Known Text (EWKT) representation
- tcbufferFromMFJSON: Input from the Moving Features JSON (MF-JSON) representation
- tcbufferFromBinary, tcbufferFromEWKB, tcbufferFromHexWKB, tcbufferFromHexEWKB: Input from the Well-Known Binary (WKB), from the Extended Well-Known Binary (EWKB), from the Hexadecimal Well-Known Binary (HexWKB), or from the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

A.10.2.2 Constructors

- tcbuffer: Constructor for temporal circular buffers having a constant value
- tcbufferSeq: Constructor for temporal circular buffers of sequence subtype
- tcbufferSeqSet, tcbufferSeqSetGaps: Constructor for temporal circular buffers of sequence set subtype
- tcbuffer: Construct a temporal circular buffer from a temporal geometry point and a temporal float

A.10.2.3 Conversions

- ::: Convert a temporal circular buffer to a temporal geometry point
- ::: Convert a temporal circular buffer to a temporal float

A.10.2.4 Accessors

- getValues: Return the values
- points, radius: Return the points or the radii
- valueAtTimestamp: Return the value at a timestamp

A.10.2.5 Transformations

- tcbufferInst, tcbufferSeq, tcbufferSeqSet: Transform a temporal circular buffer to another subtype
- setInterp: Transform a temporal circular buffer to another interpolation
- round: Round the points and the radii of the temporal circular buffer to the number of decimal places
- stops: Extract the subsequences where the temporal circular buffer stays within an area with a specified maximum size for at least the given duration
- translate: Return a temporal value translated by given offsets
- rotate: Return a temporal value rotated counter-clockwise about the origin or about a point
- rotateZ: Return a temporal value rotated counter-clockwise about the z axis

A.10.2.6 Modifications

- appendInstant, appendSequence: Append a temporal instant or a temporal sequence
- merge: Merge the temporal circular buffers

A.10.2.7 Restrictions

- atValue, minusValue, atValues, minusValues: Restrict to (the complement of) a value or a set of values
- atGeometry, minusGeometry: Restrict to (the complement of) a geometry
- atStbox, minusStbox: Restrict a temporal circular buffer to (the complement of) a spatiotemporal box

A.10.2.8 Spatial Operations

- SRID, setSRID: Return or set the spatial reference identifier
- transform, transformPipeline: Transform to a spatial reference identifier
- traversedArea: Return the area traversed by the temporal circular buffer
- centroid: Return the temporal geometric point given by the centers of a temporal circular buffer
- convexHull: Return the convex hull of the area traversed by the temporal circular buffer

A.10.2.9 Bounding Box Operations

- &&, <@, @>, ~=, -|-: Topological operators
- :::: Position operators
- expandSpace: Return the spatiotemporal bounding box expanded in the spatial dimension by a float value

A.10.2.10 Distance Operations

- |=|: Return the smallest distance ever
- nearestApproachInstant: Return the instant of the first temporal circular buffer at which the two arguments are at the nearest distance
- shortestLine: Return the line connecting the nearest approach point between the two arguments
- <->: Return the temporal distance
- minDistance: Return the minimum spatial distance between two sets of temporal circular buffers, ignoring time

A.10.2.11 Ever and Always Relationships

- eContains, aContains, eCovers, aCovers, eDisjoint, aDisjoint, eDwithin, aDwithin, eIntersects, aIntersects, eTouches, aTouches: Ever and always relationships

A.10.2.12 Spatiotemporal Relationships

- tContains, tCovers, tDisjoint, tDwithin, tIntersects, tTouches: Spatiotemporal relationships

A.10.2.13 Comparisons

- =, <>, <, >, <=, >=: Traditional comparisons
- ?=, ?<>, %=, %<>: Ever and always comparisons
- #=, #<>: Temporal comparisons

A.10.2.14 Aggregations

- extent: Bounding box extent
- tCount: Temporal count
- wCount: Window count
- tCentroid: Aggregate the centres of a set of temporal circular buffers into their temporal centroid, by casting to `tgeompoint`

A.10.2.15 Simplification

- minDistSimplify, minTimeDeltaSimplify: Return a temporal circular buffer simplified ensuring that consecutive centers are at least a certain distance or time interval apart
- maxDistSimplify, douglasPeuckerSimplify: Return a temporal circular buffer simplified using the Douglas-Peucker algorithm

A.11 Temporal Rigid Geometries

A.11.1 Input and Output

- asText: Return the Well-Known Text (WKT) representation
- asEWKT: Return the Extended Well-Known Text (EWKT) representation, prefixed with the SRID
- asMFJSON: Return the Moving Features JSON representation
- asEWKB, asHexWKB, asHexEWKB: Return the Extended Well-Known Binary (EWKB), the Hexadecimal Well-Known Binary (HexWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation
- trgeometryFromText, trgeometryFromMFJSON, trgeometryFromEWKB, trgeometryFromEWKT, trgeometryFromHexWKB, trgeometryFromHexEWKB: Input from the Well-Known Text (WKT), the Extended Well-Known Text (EWKT), the Moving Features JSON (MF-JSON), the Extended Well-Known Binary (EWKB), or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation

A.11.2 Constructors

- trgeometry: Construct a temporal rigid geometry from a reference geometry and a temporal pose

A.11.3 Conversions

- tpose: Return the underlying temporal pose
- tgeompoint: Return the centroid (antenna) trajectory as a temporal point

A.11.4 Accessors

- startValue, endValue, valueAtTimestamp: Return the materialised geometry at the start, end, or a chosen timestamp
- numInstants, numSequences, numTimestamps: Return the number of distinct instants, sequences, or timestamps
- instants, sequences, segments: Return the array of constituent instants, sequences, or inter-instant segments
- points: Return the set of distinct centroid (antenna) points seen along the trgeometry
- yaw, pitch, roll: Return the yaw, pitch, or roll angle as a temporal float, in radians, under the ZYX intrinsic Tait-Bryan convention required by the OGC GeoPose Basic-YPR conformance class
- geometry: Return the static reference geometry

A.11.5 Traversed Area

- traversedArea: Return the area traversed by the temporal rigid geometry

A.11.6 Spatial Functions

- centroid: Return the centroid of the moving body as a temporal point
- convexHull: Return the convex hull of the area traversed by the moving body
- bodyPointTrajectory: Return the world-frame trajectory of a body-frame point carried by the moving rigid geometry

A.11.7 Motion Metrics

- length, cumulativeLength: Return the total length traversed by the centroid, or its cumulative length over time
- speed: Return the speed of the centroid as a temporal float
- angularSpeed: Return the angular speed as a temporal float
- twCentroid: Return the time-weighted centroid of the centroid trajectory as a geometry

A.11.8 Transformations

- round: Round all numeric components to a given number of decimal places
- setInterp: Convert between linear and step interpolations
- shiftTime, scaleTime, shiftScaleTime: Shift, scale, or both shift and scale the time domain of a trgeometry
- unnest: Transform a nonlinear trgeometry into a set of rows, one per distinct pose, each one is a pair composed of the reference geometry with the pose applied and the period set during which the trgeometry takes that pose {}
- translate: Return a temporal value translated by given offsets ⬡
- rotate: Return a temporal value rotated counter-clockwise about the origin or about a point ⬡

- rotateX: Return a temporal value rotated counter-clockwise about the x axis
- rotateY: Return a temporal value rotated counter-clockwise about the y axis
- rotateZ: Return a temporal value rotated counter-clockwise about the z axis
- force2D: Return a temporal rigid geometry projected to the ground

A.11.9 Modifications

- appendInstant: Append a single instant to an existing trgeometry, growing the underlying pose path
- appendSequence: Append an entire sequence to an existing trgeometry
- merge: Combine two trgeometries with the same reference geometry into a single sequence-set

A.11.10 Restrictions

- atValue, minusValue, atValues, minusValues: Restrict to (the complement of) a pose or a set of poses
- atGeometry, minusGeometry: Restrict a trgeometry to (the complement of) a geometry
- atStbox, minusStbox: Restrict a trgeometry to (the complement of) a spatiotemporal box

A.11.11 Distance Operations

- <->: Return the temporal distance
- |=|: Return the nearest approach distance
- nearestApproachInstant: Return the instant of the first argument at which the two arguments are nearest
- shortestLine: Return the line between the nearest points of the two arguments at their nearest approach instant

A.11.12 Bounding Box Operations

- &&, @>, <@, ~=, -|-: Standard topological operators on bounding boxes
- <<, >>, &<, &>, <</, />>, <<#, #>>: Spatial and temporal position operators

A.11.13 Ever and Always Relationships

- eContains, aContains, eCovers, aCovers, eDisjoint, aDisjoint, eDwithin, aDwithin, eIntersects, aIntersects, eTouches, aTouches: Ever and always relationships

A.11.14 Spatiotemporal Relationships

- tContains, tCovers, tDisjoint, tDwithin, tIntersects, tTouches: Spatiotemporal relationships

A.11.15 Comparisons

- =, <>, <, >, <=, >=: Traditional comparisons
- ?=, %=, ?<>, %<>: Ever and always comparisons
- %=, %<>: Temporal comparisons

A.11.16 Tiles

- spaceBoxes, timeBoxes, spaceTimeBoxes: Return the spatiotemporal boxes of a temporal rigid geometry split with respect to a spatial, temporal, or spatiotemporal grid

A.11.17 Bounding Boxes

- stboxes, spans: Return the array of spatiotemporal boxes or time spans of the segments of a temporal rigid geometry
- splitNStboxes, splitEachNStboxes, splitNSpans, splitEachNSpans: Return the spatiotemporal boxes or time spans of a temporal rigid geometry split into a given number of bins, or with a given number of segments per bin

A.11.18 Aggregations

- tCount: Number of overlapping temporal rigid geometries at each instant, equivalent to `tCount` on the underlying tpose
- extent: Aggregate bounding box of a column of temporal rigid geometries

A.12 Temporal JSON

A.12.1 JSONB Operations

- jsonbObjectField, jsonbObjectFieldText: Extract a JSON object field specified by a key
- jsonbExtractPath, jsonbExtractPathText: Extract a JSON value specified by a path
- jsonbArrayElement, jsonbArrayElementText: Extract an element from a JSON array
- jsonbArrayLength: Return the number of elements of a JSON array
- jsonbArrayElements, jsonbArrayElementsText: Return the elements of a JSON array as rows
- jsonbEach, jsonbEachText: Return the fields of a JSON object as rows of a key and a value
- jsonbObjectKeys: Return the keys of a JSON object as rows
- jsonbPretty: Return the indented text of a JSON value
- jsonbConcat: Concatenate two JSON values
- jsonbDelete, jsonbDeleteArray, jsonbDeleteIndex, jsonbDeletePath: Delete a key, the keys of an array, an array element or the item at a path from a JSON value
- jsonbExists, jsonbExistsAny, jsonbExistsAll: Does a text, or any or all the texts of an array, exist as a top-level key or array element of a JSON value?
- jsonbContains, jsonbContained: Does the first JSON value contain, or is it contained in, the second one?
- jsonbSet, jsonbSetLax: Return a JSON value with the item at a path set to a value
- jsonbInsert: Return a JSON value with a value inserted at a path
- jsonbStripNulls: Return a JSON value without its object fields whose value is null
- jsonbPathExists, jsonbPathExistsTz: Does the JSON path return any item for a JSON value?
- jsonbPathMatch, jsonbPathMatchTz: Return the result of a JSON path predicate check for a JSON value
- jsonbPathQuery, jsonbPathQueryTz, jsonbPathQueryArray, jsonbPathQueryArrayTz, jsonbPathQueryFirst, jsonbPathQueryFirstTz: Return the items a JSON path returns for a JSON value, as rows, as a JSON array, or the first one
- eq, cmp, hash, hashExtended: Compare two JSON values, or return the hash value of a JSON value

A.12.2 JSONB Set Operations

- ->, ->>, jsonbsetObjectField, jsonbsetObjectFieldText: Extract a JSON object field specified by a key
- ->, ->>, jsonbsetExtractPath, jsonbsetExtractPathText: Extract a JSON object field specified by a path
- jsonbsetArrayElement, jsonbsetArrayElementText: Extract an element from a JSON array
- intset, floatset, textset: Extract a alphanumeric value from a JSONB set given by a key
- ||: JSONB set concatenation
- -: JSONB set deletion
- ?: JSONB set exists
- jsonbsetSet, jsonbsetSetLax: JSONB set assignment
- jsonbsetInsert: Return the `jsonbset` value with `jsonb` inserted
- jsonbsetStripNulls: Return a JSONB set without nulls
- @?, jsonbsetPathExists, jsonbsetPathExistsTz: Does the JSON path return any item for the specified JSONB set?
- @@, jsonbsetPathMatch, jsonbsetPathMatchTz: Return the result of a JSON path predicate check for a JSONB set
- jsonbsetPathQueryArray, jsonbsetPathQueryArrayTz: Return all items returned by a JSON path from a JSONB set, as a JSON array
- jsonbsetPathQueryFirst, jsonbsetPathQueryFirstTz: Return the fist item returned by a JSON path from a JSONB set

A.12.3 Input and Output

- asText: Return the Well-Known Text (WKT) representation
- asBinary, asHexWKB: Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation
- asMFJSON: Return the Moving Features JSON (MF-JSON) representation
- tjsonbFromText, jsonbFromText, jsonpathFromText: Input from the Well-Known Text (WKT) representation
- tjsonbFromBinary, tjsonbFromHexWKB: Input from the Well-Known Binary (WKB) or from the Hexadecimal Well-Known Binary (HexWKB) representation
- tjsonbFromMFJSON: Input from the Moving Features JSON (MF-JSON) representation

A.12.4 Constructors

- tjsonb: Constructor for temporal JSONB values having a constant value
- tjsonbSeq: Constructor for temporal JSONB values of sequence subtype
- tjsonbSeqSet, tjsonbSeqSetGaps: Constructor for temporal JSONB values of sequence set subtype

A.12.5 Conversions

- ::: Convert a temporal JSONB to a `tstzspan`
- ::: Convert between a temporal JSONB and a text
- ::: Convert between a temporal JSONB and a temporal text

A.12.6 Accessors

- getValues: Return the values
- valueAtTimestamp: Return the value at a timestamp

A.12.7 Transformations

- tjsonbInst, tjsonbSeq, tjsonbSeqSet: Transform a temporal JSONB to another subtype
- setInterp: Transform a temporal JSONB value to another interpolation

A.12.8 Temporal JSON Operations

- ->, ->>, tjsonObjectField, tjsonbObjectField, tjsonbObjectFieldText: Extract a temporal JSON object field specified by a key
- ->, ->>, tjsonExtractPath, tjsonbExtractPath, tjsonbExtractPathText: Extract a temporal JSON object field specified by a path
- tjsonArrayElement, tjsonbArrayElement, tjsonbArrayElementText: Extract an element from a temporal JSON array
- tbool, tint, tfloat, ttext: Extract a temporal alphanumeric value from a temporal JSONB value given by a key
- tgeompoint, tgeogpoint, tgeometry, tgeography: Extract a temporal geo value from a temporal JSONB value given by a key
- ||: Temporal JSONB concatenation
- -: Temporal JSONB deletion
- ?: Temporal JSONB exists
- @>, <@: Temporal JSONB contains/contained
- tjsonbSet, tjsonbSetLax: Temporal JSONB set
- tjsonbInsert: Temporal JSONB insert
- tjsonStripNulls, tjsonbStripNulls: Return a temporal JSON value without nulls
- @?, tjsonbPathExists, tjsonbPathExistsTz: Does the JSON path return any item for the specified temporal JSONB value?
- @@, tjsonbPathMatch, tjsonbPathMatchTz: Return the result of a JSON path predicate check for a temporal JSONB value
- tjsonbPathQueryArray, tjsonbPathQueryArrayTz: Return all items returned by a JSON path from a temporal JSONB value, as a JSON array
- tjsonbPathQueryFirst, tjsonbPathQueryFirstTz: Return the fist item returned by a JSON path from a temporal JSONB value

A.12.9 Restrictions

- atValue, minusValue, atValues, minusValues: Restrict to (the complement of) a value or a set of values

A.12.10 Bounding Box Operations

- &&, <@, @>, ~=, -|-: Topological operators
- :::: Position operators

A.12.11 Comparisons

- =, <>, <, >, <=, >=: Traditional comparisons
- ?=, %=: Ever and always comparisons
- :::: Temporal comparisons

A.12.12 Aggregations

- tCount: Temporal count
- wCount: Window count

A.13 Temporal Tessellations

A.13.1 Input and Output

- tcell: Input and output of a temporal cell in its text form
- asText, asEWKT, asBinary, asEWKB, asHexWKB, asHexEWKB, asMFJSON: Serialize a temporal cell as text, as binary, or as hexadecimal well-known binary, with or without its SRID, or as MF-JSON
- tcellFromText, tcellFromEWKT, tcellFromBinary, tcellFromEWKB, tcellFromHexWKB, tcellFromHexEWKB, tcellFromMFJSON: Reconstruct a temporal cell from its text, binary, hexadecimal, or MF-JSON representation
- asBinary, asHexWKB: Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation of a cell
- h3indexFromBinary, h3indexFromHexWKB, quadbinFromBinary, quadbinFromHexWKB, s2cellFromBinary, s2cellFromHexWKB: Input of a cell from the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation
- asText: Return the text representation of a cell
- h3indexFromText, quadbinFromText, s2cellFromText: Input from the text representation of a cell
- asBinary: Return the Well-Known Binary (WKB) representation of a raster or a Raquet tile
- asHexWKB: Return the Hexadecimal Well-Known Binary (HexWKB) representation of a raster or a Raquet tile
- rasterFromBinary, raquetFromBinary: Return a raster or a Raquet tile from its Well-Known Binary (WKB) representation
- rasterFromHexWKB, raquetFromHexWKB: Return a raster or a Raquet tile from its Hexadecimal Well-Known Binary (HexWKB) representation

A.13.2 Constructors

- tcell: Construct a temporal cell holding one cell over a time value
- tcellInst, tcellSeq, tcellSeqSet: Construct a temporal cell of a given subtype from its components
- raquet: Construct a Raquet raster tile from its pixel bytes, dimensions, QUADBIN cell and pixel type
- raster: Construct a raster of one band from the bytes of its pixels and its georeferencing
- makeEmptyRaster: Construct a raster with no band
- addBand: Add a band to a raster
- raquetRead: Decode a raster file into a Raquet raster tile via GDAL

A.13.3 Conversions

- ::: Convert between a `tcell` and a `tbigint`
- ::: Cast a temporal cell to a temporal geodetic or planar point of cell centroids
- stbox: Convert a Raquet tile into a spatiotemporal box
- stbox: Convert a raster into a spatiotemporal box

A.13.4 Accessors

- startValue, endValue, valueN: Return the first, the last, or the nth distinct cell identifier of a temporal value
- getValues: Return the set of distinct cell identifiers the trajectory takes
- valueAtTimestamp: Return the cell identifier at a given timestamp, using step interpolation
- numBands: Return the number of bands of a raster
- SRID: Return the spatial reference system identifier of a raster
- upperLeftX, upperLeftY: Return the X or the Y coordinate of the upper left corner of a raster
- scaleX, scaleY: Return the pixel width or the pixel height of a raster, that is, the X or the Y component of its scale
- skewX, skewY: Return the X or the Y component of the skew of a raster
- quadbin: Return the QUADBIN cell of a Raquet tile
- width: Return the width in pixels of a raster or a Raquet tile
- height: Return the height in pixels of a raster or a Raquet tile
- bandPixelType: Return the name of the pixel data type of a raster band or of a Raquet tile
- bandHasNoDataValue: Return whether a raster band or a Raquet tile states a nodata value
- bandNoDataValue: Return the nodata value of a raster band or of a Raquet tile
- pixels: Return the pixel bytes of a raster band or of a Raquet tile

A.13.5 Transformations

- shiftTime, scaleTime, shiftScaleTime: Shift, scale, or shift and scale the time span of a temporal cell
- appendInstant, appendSequence: Append a temporal instant or a temporal sequence
- merge: Merge the temporal cells
- setInterp: Set the interpolation of a temporal cell

A.13.6 Modifications

- insert, update: Insert a temporal cell into another one, or update one with another
- deleteTime: Delete a time value from a temporal cell
- setValues: Set the values of a block of pixels of a raster band

A.13.7 Restrictions

- atValue, minusValue, atValues, minusValues: Restrict a temporal cell to (the complement of) a cell or a set of cells
- atTime, minusTime: Restrict a temporal cell to (the complement of) a time value

A.13.8 Static Geometry Coverage

- latLngToCell, geoToQuadbinCell, geoToS2Cell: Return the cell of a grid that contains a point geometry at the given resolution
- cellset, geoToH3IndexSet, geoToQuadbinSet, geoToS2CellSet: Return the set of cells of a grid covering a geometry at the given resolution
- eEqual: Return whether a temporal cell trajectory ever takes one of the cells in a cell set

A.13.9 Inspection

- getResolution: Return the resolution of each cell in a trajectory
- isValidCell: Return a temporal boolean stating at each instant whether the value is a valid cell

A.13.10 Hierarchy

- cellToParent: Return the temporal parent cell at the given resolution
- cellToCover, coverCells: Return the cells covering a cell, or the cells of a cell set, at the given resolution

A.13.11 Latitude and Longitude Conversions

- tcell: Return the temporal cell at the given resolution that contains each point in a temporal geodetic or planar point
- quadbinSplit: Split a temporal point at the QUADBIN cells it crosses `{}`
- h3Split: Split a temporal point at the H3 cells it crosses `{}`
- s2Split: Split a temporal point at the S2 cells it crosses `{}`
- cellToPoint, tgeompoint: Return the centroid of a cell, or of each cell in a trajectory
- cellToBoundary: Return the polygon boundary of a cell, or of each cell in a trajectory

A.13.12 Bounding Box Operations

- stbox, expandSpace: Return the spatiotemporal bounding box of a temporal cell, or that box expanded in the spatial dimension by a float value
- &&, @>, <@, ~=, -|-: Topological operators
- <<, &<, &>, >>, <<|, &<|, |&>, |>>, <<#, &<#, #>>, #&>: Position operators

A.13.13 Metric Functions

- cellArea: Return the area of a cell, and the temporal area of each cell in a trajectory

A.13.14 Set-Returning Functions

- gridDisk: All cells within `k` grid steps of the origin
- cellToChildren: All children of a cell at the given target resolution
- compactCells: Compacted representation of a cell set, finer cells being merged into their parent whenever the parent's full set of children is present
- uncompactCells: Uncompacted representation at the given resolution, the inverse of `compactCells`

A.13.15 Ever and Always Relationships

- eContains, aContains, eCovers, aCovers, eDisjoint, aDisjoint, eDwithin, aDwithin, eIntersects, aIntersects, eTouches, aTouches: Ever and always relationships

A.13.16 Spatiotemporal Relationships

- tContains, tCovers, tDisjoint, tDwithin, tIntersects, tTouches: Spatiotemporal relationships

A.13.17 Comparisons

- =, <>, <, >, <=, >=, cmp: Traditional comparisons
- ?=, ?<>, %=, %<>: Ever and always comparisons
- #=, #<>: Temporal comparisons
- =, <>, <, <=, >=, >, cmp: Compare two Raquet tiles
- hash, hashExtended: Hash values

A.13.18 Aggregations

- tCount: Temporal count
- wCount: Window count
- merge, appendInstant, appendSequence: Aggregate a set of temporal cells by merging them, or by assembling them into a sequence or a sequence set

A.13.19 H3-Specific Operations

- th3GetBaseCellNumber: Return the temporal base-cell number (0–121) of each cell in a trajectory
- th3IsResClassIii: Return a temporal boolean stating whether each cell has Class-III orientation
- th3IsPentagon: Return a temporal boolean stating whether each cell is a pentagon
- th3CellToCenterChild: Return the temporal center-child cell at the given resolution
- th3CellToChildPos: Return the ordinal position (0-based) of each cell among the children of its parent at the given resolution
- th3ChildPosToCell: Return the temporal child cell at a given ordinal position of the given parent, at the given child resolution
- h3GridRing: The cells at *exactly* `k` grid steps from the origin
- h3GridPathCells: The inclusive path of cells between two cells of the same resolution

- h3OriginToDirectedEdges: All outgoing directed edges of a cell
- h3CellToVertexes: All vertexes of a cell
- h3GetIcosahedronFaces: The icosahedron face indexes (0..19) intersected by a cell
- th3EdgeLength: Return the temporal length of each directed edge in a trajectory, in metres
- greatCircleDistance: Return the temporal great-circle distance between two temporal geodetic points (not between cells)
- th3AreNeighborCells: Return a temporal boolean stating whether two temporal cells are grid neighbours at each instant
- th3CellsToDirectedEdge: Return a temporal directed-edge identifier from an origin/destination pair of temporal cells
- isValidDirectedEdge: Return a temporal boolean stating at each instant whether the value is a valid H3 directed edge
- th3GetDirectedEdgeOrigin, th3GetDirectedEdgeDestination: Return the temporal origin or destination cell of a temporal directed edge
- th3DirectedEdgeToBoundary: Return the temporal polygon boundary of each directed edge in a trajectory, as a temporal geography
- th3CellToVertex: Return the temporal H3 vertex at the given vertex number
- th3VertexToPoint: Return the temporal geodetic coordinates of each vertex in a trajectory
- isValidVertex: Return a temporal boolean stating at each instant whether the value is a valid H3 vertex
- <->: Return the temporal grid-hop distance (number of single-cell steps) between two temporal cells
- th3CellToLocalIj, th3LocalIjToCell: Convert between a temporal cell and a temporal local (I, J) coordinate pair relative to an origin temporal cell

A.13.20 QUADBIN-Specific Operations

- isValidIndex: Return whether a value is a structurally valid QUADBIN identifier
- quadbinCellSibling: Return the neighbouring cell at the same resolution in the given direction
- quadbinCellToQuadkey, tquadbinCellToQuadkey: Emit the base-4 slippy-tile quadkey string of a cell
- tquadbinGridDistance, <->: Return the temporal grid distance between two temporal QUADBIN cells
- quadbinCellToTile: Decompose a cell back into its tile coordinate triple, returned as an integer array holding `x`, then `y`, then `z`
- quadbinTileToCell: Build a cell from slippy-map tile coordinates: column `x`, row `y`, and zoom `z`

A.13.21 S2-Specific Operations

- s2GetFace: Return the cube face in 0 to 5 an S2 cell sits on
- s2CellToToken, s2TokenToCell, ts2CellToToken: Convert between an S2 cell and its token, the form the S2 libraries publish
- ts2GridDistance, <->: Return the temporal grid distance between two temporal S2 cells
- s2CommonAncestorLevel: Return the level of the deepest cell containing two S2 cells
- s2CellContains: Return true if the first S2 cell contains the second one
- s2RangeMin, s2RangeMax: Return the first and the last leaf cell an S2 cell contains
- s2CellToChild: Return the descendant of an S2 cell at the given position along the Hilbert curve
- s2EdgeLength: Return the length in metres of an edge of an S2 cell
- s2EdgeNeighbors: Return the four cells sharing an edge with an S2 cell

A.13.22 Raster-Specific Operations

- rasterValue: Read a raster band along a trajectory
- valueCount: Return the number of pixels of a raster band holding each value
- valuePercent: Return the share of the pixels of a raster band holding each value
- sameAlignment: Return true if rasters lie on the same grid
- notSameAlignmentReason: Return why two rasters do not lie on the same grid
- rasterUnion: Return the union of the rasters of a column
- reclass: Return a raster whose band holds the classes an expression maps its values to
- +, -, *, /: Addition, subtraction, multiplication, and division of rasters, pixel by pixel
- slope: Return the slope of a raster band, pixel by pixel
- tri: Return the terrain ruggedness index of a raster band, pixel by pixel
- tpi: Return the topographic position index of a raster band, pixel by pixel
- mapAlgebra: Return a statistic of the neighbourhood of each pixel of a raster band
- clip: Return a raster keeping the pixels of another that a geometry covers
- transform, transformPipeline: Return a raster stated in another spatial reference system
- rescale: Return a raster resampled to another pixel size
- summaryStats: Return the statistics of the pixels of a raster band
- summaryStatsAgg: Return the statistics of the pixels of a band over the rasters of a column
- summarystatsCount, summarystatsSum, summarystatsMean, summarystatsStddev, summarystatsMin, summarystatsMax: Return the number of pixels, the sum, the mean, the standard deviation, the minimum or the maximum of the statistics of a raster band
- dumpAsPolygons: Return the polygons of a raster band, one for each group of pixels sharing a value `{}`
- pixelAsPoints: Return the pixels of a raster band as points `{}`
- dumpValues: Return the values of the bands of a raster as arrays
- rasterTileValue: Sample a `raquet` raster tile along a trajectory
- rasterTileValue: Sample an array of `raquet` raster tiles along a trajectory
- rasterTileValueQuadbin: Sample a Raquet raster chip along a trajectory using a QUADBIN cell for georeferencing
- atRasterValue: Return the part of a trajectory where the raster value it reads falls inside a float range
- minusRasterValue: Return the part of a trajectory where the raster value it reads falls outside a float range
- eRasterValue: Return true if the trajectory ever reads a raster pixel value inside a float range
- aRasterValue: Return true if every pixel value the trajectory reads over the raster lies inside a float range

A.14 Temporal Point Clouds

A.14.1 Point Cloud Schemas

- pointCloudSchemaSRID: Return the spatial reference system every value of a schema is expressed in
- pointCloudSchemaCompression: Return how a patch of a schema is stored
- pointCloudSchemaNDims: Return how many dimensions of a schema hold values

A.14.2 Static Point Clouds

A.14.2.1 Input and Output

- asHexWKB: Return the Hexadecimal Well-Known Binary (HexWKB) representation of a pcpoint or a pcpatch
- pcpointFromHexWKB, pcpatchFromHexWKB: Input of a pcpoint or a pcpatch from its Hexadecimal Well-Known Binary (HexWKB) representation

A.14.2.2 Constructors

- pcpoint: Build a 2D or 3D pcpoint directly from x/y(/z) coordinates
- pcpoint: Build a pcpoint from the coordinates of every dimension of the schema
- pcpatch: Build a pcpatch from the coordinates of its points, one point after another
- pcpatch: Build a pcpatch from a variadic list of pcpoints sharing the same pcid

A.14.2.3 Accessors

- pcid: Return the schema id of a pcpoint or pcpatch
- getX, getY, getZ: Return a coordinate of a pcpoint, NULL if the dimension is absent from the schema
- getDim: Return any named dimension of a pcpoint, NULL if the name is unknown for the schema
- SRID: Return the spatial reference identifier (inherited from the schema)
- numPoints: Return the number of points of a pcpatch
- pointN: Return the n-th point of a pcpatch
- geometry: Return the multipoint of the positions the points of a patch occupy

A.14.3 Point Cloud Sets

A.14.3.1 Constructors

- set: Construct a set from an array of values, all of which must share the same pcid

A.14.4 Point Cloud Bounding Boxes

A.14.4.1 Input and Output

- asBinary, asHexWKB: Return the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation of a point cloud box
- asEWKB, asHexEWKB: Return the Extended Well-Known Binary (EWKB) or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation of a point cloud box
- tpcboxFromBinary, tpcboxFromHexWKB: Input of a point cloud box from the Well-Known Binary (WKB) or the Hexadecimal Well-Known Binary (HexWKB) representation
- tpcboxFromEWKB, tpcboxFromHexEWKB: Input of a point cloud box from the Extended Well-Known Binary (EWKB) or the Hexadecimal Extended Well-Known Binary (HexEWKB) representation
- asText: Return the Well-Known Text (WKT) representation
- asEWKT: Return the Extended Well-Known Text (EWKT) representation of a point cloud box
- tpcboxFromText: Input from the Well-Known Text (WKT) representation
- tpcboxFromEWKT: Input of a point cloud box from the Extended Well-Known Text (EWKT) representation

A.14.4.2 Constructors

- tpcbox, tpcbox_z, tpcbox_t, tpcbox_xt, tpcbox_zt: Construct a tpcbox from explicit X/Y/Z/T bounds and a pcid, whose schema states the SRID

A.14.4.3 Conversions

- tpcbox: Convert a pcpoint to a degenerate (zero-extent) tpcbox; SRID and pcid come from the schema
- tpcbox: Convert a pcpatch to a tpcbox

A.14.4.4 Accessors

- hasX, hasZ, hasT: Return whether a tpcbox has the X/Y, Z, or T dimensions set
- xmin, xmax, ymin, ymax, zmin, zmax: Return a coordinate bound, NULL if the corresponding dimension is absent
- tmin, tmax: Return a temporal bound, NULL if the box has no T dimension
- pcid, SRID: Return the schema id and the SRID its schema states

A.14.4.5 Transformations

- round: Return a tpcbox with coordinates rounded to a given number of decimal digits
- setSRID: Return a tpcbox stating a reference system its schema does not state

A.14.4.6 Set Operations

- +, *: Union and intersection of two tpcboxes

A.14.4.7 Topological Operations

- &&, @>, <@, ~=, -|-: Topological operators

A.14.4.8 Position Operations

- <<, &<, >>, &>, <<|, &<|, |>>, |&>, <</, &</, />>, /&>, <<#, &<#, #>>, #&>: Position operators

A.14.4.9 Comparisons

- =, <>, <, <=, >=, >: Compare two tpcboxes; total order over the canonical encoding (pcid, srid, flags, period, then spatial bounds in XYZ-min/max order)

A.14.5 Temporal Point Cloud Points

A.14.5.1 Input and Output

- asText, asEWKT: Return the text representation of a tpcpoint, or of an array of them, with or without its SRID, and cast a tpcpoint to text
- asMFJSON, tpcpointFromMFJSON: Return the Moving Features JSON (MF-JSON) representation of a tpcpoint and read it back; the round trip is lossless
- asBinary, asEWKB, asHexWKB, asHexEWKB: Return the binary or hexadecimal well-known binary representation of a tpcpoint, with or without its SRID
- tpcpointFromText, tpcpointFromEWKT, tpcpointFromBinary, tpcpointFromEWKB, tpcpointFromHexWKB, tpcpointFromHexEWKB: Return a tpcpoint from its text, binary, or hexadecimal well-known binary representation, with or without its SRID

A.14.5.2 Constructors

- tpcpoint: Constructor for temporal point cloud points having a constant value
- tpcpointSeq: Constructor for temporal point cloud points of sequence subtype
- tpcpointSeqSet: Construct a temporal pcpoint of sequence-set subtype from an array of sequences

A.14.5.3 Conversions

- ::: Convert a temporal point cloud point to a temporal geometry point
- tpcbox, ::: Convert a temporal point cloud point to its bounding box

A.14.5.4 Accessors

- pcid: Return the pgPointCloud schema id shared by all instants
- SRID: Return the SRID inherited from the schema
- getX, getY, getZ: Project a tpcpoint onto a spatial dimension, yielding a tfloat
- getDim: Project a tpcpoint onto any named schema dimension, yielding a tfloat

A.14.5.5 Transformations

- tpcpointInst, tpcpointSeq, tpcpointSeqSet: Transform a tpcpoint to another subtype
- setInterp: Transform a tpcpoint to another interpolation

A.14.5.6 Modifications

- insert, update: Insert a tpcpoint into another one, or update another one with it
- deleteTime: Delete a time selector (timestamp, set, period, or spanset) from a tpcpoint
- appendInstant, appendSequence: Append an instant or a sequence to a tpcpoint
- merge: Merge the temporal values

A.14.5.7 Restrictions

- atValue, minusValue, atValues, minusValues: Restrict a tpcpoint to (the complement of) a pcpoint value or a set of pcpoint values
- atTime, minusTime: Restrict a tpcpoint to a time selector (timestamp, period, set, or spanset), or remove it
- atTpcbox, minusTpcbox: Restrict a tpcpoint to the spatial / temporal extent of a tpcbox, or remove it
- beforeTimestamp, afterTimestamp: Restrict a tpcpoint to the instants before or after a timestamp

A.14.5.8 Splitting Operations

- unnest: Return the distinct values of a tpcpoint with the span set on which each of them is taken
- timeSplit: Split a tpcpoint into fragments, one per time bin
- spans: Return the array of time spans of a tpcpoint's segments
- splitNSpans, splitEachNSpans: Return the time spans of a tpcpoint split into a given number of bins, or with a given number of segments per bin

A.14.5.9 Bounding Box Operations

- &&, @>, <@, ~=, -|-: Topological operators
- <<, >>, <<|, |>>, <</, />>, <<#, #>>: Position operators

A.14.5.10 Distance Operations

- |=|: Return the smallest distance ever
- nearestApproachInstant: Return the instant of the tpcpoint at which the two arguments are at the nearest distance
- shortestLine: Return the line connecting the nearest approach point between the two arguments
- <->: Return the temporal distance

A.14.5.11 Ever and Always Relationships

- eContains, aContains, eCovers, aCovers, eDisjoint, aDisjoint, eDwithin, aDwithin, eIntersects, aIntersects, eTouches, aTouches: Ever and always relationships

A.14.5.12 Spatiotemporal Relationships

- tContains, tCovers, tDisjoint, tDwithin, tIntersects, tTouches: Temporal spatial relationships

A.14.5.13 Comparisons

- =, <>, <, <=, >=, >: Traditional comparisons
- ?=, %=: Ever and always comparisons
- #=: Temporal comparisons

A.14.5.14 Simplification

- minDistSimplify, minTimeDeltaSimplify, maxDistSimplify, douglasPeuckerSimplify: Return a tpcpoint simplified ensuring that consecutive points are at least a certain distance or time interval apart, or simplified using the Douglas-Peucker algorithm

A.14.6 Temporal Point Cloud Patches

A.14.6.1 Input and Output

- asText, asEWKT: Return the text representation of a tpcpatch, or of an array of them, with or without its SRID, and cast a tpcpatch to text
- asBinary, asEWKB, asHexWKB, asHexEWKB: Return the binary or hexadecimal well-known binary representation of a tpcpatch, with or without its SRID
- tpcpatchFromText, tpcpatchFromEWKT, tpcpatchFromBinary, tpcpatchFromEWKB, tpcpatchFromHexWKB, tpcpatchFromHexEWKB: Return a tpcpatch from its text, binary, or hexadecimal well-known binary representation, with or without its SRID
- asMFJSON, tpcpatchFromMFJSON: Return the Moving Features JSON (MF-JSON) representation of a tpcpatch and read it back; the round trip is lossless

A.14.6.2 Constructors

- tpcpatch: Constructor for temporal point cloud patches having a constant value
- tpcpatchSeq: Construct a temporal pcpatch of sequence subtype
- tpcpatchSeqSet: Construct a temporal pcpatch of sequence-set subtype

A.14.6.3 Conversions

- tgeometry: Convert a temporal patch into the temporal geometry of the multipoints its patches occupy
- tpcbox, ::: Convert a temporal point cloud patch to its bounding box

A.14.6.4 Accessors

- startNumPoints, endNumPoints: Return the number of points in the patch carried by the first or last instant
- numPoints: Return the total number of points across every instant's patch
- points: Set-returning function: emits one row per (instant timestamp, point) by walking each instant's patch and decomposing it through pgPointCloud's in-memory C API

A.14.6.5 Transformations

- tpcpatchInst, tpcpatchSeq, tpcpatchSeqSet: Transform a tpcpatch to another subtype
- setInterp: Transform a tpcpatch to another interpolation

A.14.6.6 Modifications

- insert, update: Insert a tpcpatch into another one, or update another one with it
- deleteTime: Delete a time selector (timestamp, set, period, or spanset) from a tpcpatch
- appendInstant, appendSequence: Append an instant or a sequence to a tpcpatch
- merge: Merge the temporal values

A.14.6.7 Restrictions

- atValue, minusValue, atValues, minusValues: Restrict a tpcpatch to (the complement of) a pcpatch value or a set of pcpatch values
- atTpcbox, minusTpcbox: Restrict a tpcpatch to the instants whose patch extent overlaps the tpcbox, or remove them
- atTpcboxFine, minusTpcboxFine: Restrict a tpcpatch to the points of each instant that the tpcbox holds, or remove them
- atGeometry, minusGeometry: Restrict a tpcpatch to the points whose XY projection intersects (or does not intersect, for minus) a 2D geometry
- beforeTimestamp, afterTimestamp: Restrict a tpcpatch to the instants before or after a timestamp

A.14.6.8 Splitting Operations

- unnest: Return the distinct values of a tpcpatch with the span set on which each of them is taken
- timeSplit: Split a tpcpatch into fragments, one per time bin
- spans: Return the array of time spans of a tpcpatch's segments
- splitNSpans, splitEachNSpans: Return the time spans of a tpcpatch split into a given number of bins, or with a given number of segments per bin

A.14.6.9 Spatial Operations

- swath: Return the swath of a temporal patch, the region of the plane its patches sweep

A.14.6.10 Bounding Box Operations

- &&, @>, <@, ~=, -|-: Topological operators
- <<, >>, <<|, |>>, <</, />>, <<#, #>>: Position operators

A.14.6.11 Distance Operations

- |=|: Return the smallest distance ever

A.14.6.12 Ever and Always Relationships

- eIntersects, aIntersects, eContains, aContains, eCovers, aCovers, eDisjoint, aDisjoint, eTouches, aTouches, eDwithin, aDwithin: Return true iff at least one point in any of the tpcpatch's instants intersects the geometry (XY only)

A.14.6.13 Spatiotemporal Relationships

- tContains, tCovers, tDisjoint, tDwithin, tIntersects, tTouches: Temporal spatial relationships

A.14.6.14 Comparisons

- ?=, %=: Ever and always comparisons
- #=: Temporal comparisons

A.14.7 Aggregations

- extent: Bounding-box aggregate over a column of tpcpoint, tpcpatch, or tpcbox values
- tCount, wCount: Temporal count and window count of overlapping rows at each timestamp
- tnpoints: Per-instant pcpatch point count, summed across rows
- tdensity: Per-instant point density (`numPoints / xy-bbox-area`) summed across rows
- merge: Combine multiple temporal values into a single one with all input instants merged in temporal order
- appendInstant, appendSequence: Streaming construction aggregates: `appendInstant` assembles instants into a sequence; `appendSequence` assembles sequences into a sequence set

A.14.8 Indexes

- GiST, SP-GiST, B-tree, and hash opclasses for tpcpoint, tpcpatch, and tpcbox

Appendix B

Synthetic Data Generator

In many circumstances, it is necessary to have a test dataset to evaluate alternative implementation approaches or to perform benchmarks. It is often required that such a data set have particular requirements in size or in the intrinsic characteristics of its data. Even if a real-world dataset could be available, it may be not ideal for such experiments for multiple reasons. Therefore, a synthetic data generator that could be customized to produce data according to the given requirements is often the best solution. Obviously, experiments with synthetic data should be complemented with experiments with real-world data to have a thorough understanding of the problem at hand.

MobilityDB provides a simple synthetic data generator that can be used for such purposes. In particular, such a data generator was used for generating the database used for the regression tests in MobilityDB. The data generator is programmed in PL/pgSQL so it can be easily customized. It is located in the directory `datagen` in the repository. In this appendix, we briefly introduce the basic functionality of the generator. We first list the functions generating random values for the various PostgreSQL, PostGIS, and MobilityDB data types, and then give examples how to use these functions for generating tables of such values. The parameters of the functions are not specified, refer to the source files where detailed explanations about the various parameters can be found.

B.1 Generator for PostgreSQL Types

- `random_bool`: Generate a random boolean
- `random_int`: Generate a random integer
- `random_int_array`: Generate a random array of integers
- `random_int4range`: Generate a random integer range
- `random_float`: Generate a random float
- `random_float_array`: Generate a random array of floats
- `random_text`: Generate a random text
- `random_timestamptz`: Generate a random timestamp with time zone
- `random_timestamptz_array`: Generate a random array of timestamps with time zone
- `random_minutes`: Generate a random interval of minutes
- `random_tstzrange`: Generate a random timestamp with time zone range
- `random_tstzrange_array`: Generate a random array of timestamp with time zone ranges

B.2 Generator for PostGIS Types

- `random_geom_point`: Generate a random 2D geometric point
- `random_geom_point3D`: Generate a random 3D geometric point
- `random_geog_point`: Generate a random 2D geographic point
- `random_geog_point3D`: Generate a random 3D geographic point
- `random_geom_point_array`: Generate a random array of 2D geometric points
- `random_geom_point3D_array`: Generate a random array of 3D geometric points
- `random_geog_point_array`: Generate a random array of 2D geographic points
- `random_geog_point3D_array`: Generate a random array of 3D geographic points
- `random_geom_linestring`: Generate a random geometric 2D linestring
- `random_geom_linestring3D`: Generate a random geometric 3D linestring
- `random_geog_linestring`: Generate a random geographic 2D linestring
- `random_geog_linestring3D`: Generate a random geographic 3D linestring
- `random_geom_polygon`: Generate a random geometric 2D polygon without holes
- `random_geom_polygon3D`: Generate a random geometric 3D polygon without holes
- `random_geog_polygon`: Generate a random geographic 2D polygon without holes
- `random_geog_polygon3D`: Generate a random geographic 3D polygon without holes
- `random_geom_multipoint`: Generate a random geometric 2D multipoint
- `random_geom_multipoint3D`: Generate a random geometric 3D multipoint
- `random_geog_multipoint`: Generate a random geographic 2D multipoint
- `random_geog_multipoint3D`: Generate a random geographic 3D multipoint
- `random_geom_multilinestring`: Generate a random geometric 2D multilinestring
- `random_geom_multilinestring3D`: Generate a random geometric 3D multilinestring
- `random_geog_multilinestring`: Generate a random geographic 2D multilinestring
- `random_geog_multilinestring3D`: Generate a random geographic 3D multilinestring
- `random_geom_multipolygon`: Generate a random geometric 2D multipolygon without holes
- `random_geom_multipolygon3D`: Generate a random geometric 3D multipolygon without holes
- `random_geog_multipolygon`: Generate a random geographic 2D multipolygon without holes
- `random_geog_multipolygon3D`: Generate a random geographic 3D multipolygon without holes

B.3 Generator for MobilityDB Span, Time, and Box Types

- `random_intspan`: Generate a random integer span
- `random_floatspan`: Generate a random float span
- `random_tstzspan`: Generate a random `tstzspan`
- `random_tstzspan_array`: Generate a random array of `tstzspan` values
- `random_tstzset`: Generate a random `tstzset`
- `random_tstzspanset`: Generate a random `tstzspanset`
- `random_tbox`: Generate a random `tbox`
- `random_stbox`: Generate a random 2D `stbox`
- `random_stbox3D`: Generate a random 3D `stbox`
- `random_geodstbox`: Generate a random 2D geodetic `stbox`
- `random_geodstbox3D`: Generate a random 3D geodetic `stbox`

B.4 Generator for MobilityDB Temporal Types

- `random_tbool_inst`: Generate a random temporal Boolean of instant subtype
- `random_tint_inst`: Generate a random temporal integer of instant subtype
- `random_tfloat_inst`: Generate a random temporal float of instant subtype
- `random_ttext_inst`: Generate a random temporal text of instant subtype
- `random_tgeompoint_inst`: Generate a random temporal geometric 2D point of instant subtype
- `random_tgeompoint3D_inst`: Generate a random temporal geometric 3D point of instant subtype
- `random_tgeogpoint_inst`: Generate a random temporal geographic 2D point of instant subtype
- `random_tgeogpoint3D_inst`: Generate a random temporal geographic 3D point of instant subtype
- `random_tbool_discseq`: Generate a random temporal Boolean of sequence subtype and discrete interpolation
- `random_tint_discseq`: Generate a random temporal integer of sequence subtype and discrete interpolation
- `random_tfloat_discseq`: Generate a random temporal float of sequence subtype and discrete interpolation
- `random_ttext_discseq`: Generate a random temporal text of sequence subtype and discrete interpolation
- `random_tgeompoint_discseq`: Generate a random temporal geometric 2D point of sequence subtype and discrete interpolation
- `random_tgeompoint3D_discseq`: Generate a random temporal geometric 3D point of sequence subtype and discrete interpolation
- `random_tgeogpoint_discseq`: Generate a random temporal geographic 2D point of sequence subtype and discrete interpolation
- `random_tgeogpoint3D_discseq`: Generate a random temporal geographic 3D point of sequence subtype and discrete interpolation
- `random_tbool_seq`: Generate a random temporal Boolean of sequence subtype

- `random_tint_seq`: Generate a random temporal integer of sequence subtype
- `random_tfloat_seq`: Generate a random temporal float of sequence subtype
- `random_ttext_seq`: Generate a random temporal text of sequence subtype
- `random_tgeompoint_seq`: Generate a random temporal geometric 2D point of sequence subtype
- `random_tgeompoint3D_seq`: Generate a random temporal geometric 3D point of sequence subtype
- `random_tgeogpoint_seq`: Generate a random temporal geographic 2D point of sequence subtype
- `random_tgeogpoint3D_seq`: Generate a random temporal geographic 3D point of sequence subtype
- `random_tbool_seqset`: Generate a random temporal Boolean of sequence set subtype
- `random_tint_seqset`: Generate a random temporal integer of sequence set subtype
- `random_tfloat_seqset`: Generate a random temporal float of sequence set subtype
- `random_ttext_seqset`: Generate a random temporal text of sequence set subtype
- `random_tgeompoint_seqset`: Generate a random temporal geometric 2D point of sequence set subtype
- `random_tgeompoint3D_seqset`: Generate a random temporal geometric 3D point of sequence set subtype
- `random_tgeogpoint_seqset`: Generate a random temporal geographic 2D point of sequence set subtype
- `random_tgeogpoint3D_seqset`: Generate a random temporal geographic 3D point of sequence set subtype

B.5 Generation of Tables with Random Values

The files `create_test_tables_temporal.sql` and `create_test_tables_tpoint.sql` provide usage examples for the functions generating random values listed above. For example, the first file defines the following function.

```
CREATE OR REPLACE FUNCTION create_test_tables_temporal(size integer DEFAULT 100)
RETURNS text AS $$ DECLARE perc integer;
BEGIN perc := size * 0.01;
IF perc < 1 THEN perc := 1; END IF;
-- ... Table generation ...
RETURN 'The End';
END;
$$ LANGUAGE 'plpgsql';
```

The function has a `size` parameter that defines the number of rows in the tables. If not provided, it creates by default tables of 100 rows. The function defines a variable `perc` that computes the 1% of the size of the tables. This parameter is used, for example, for generating tables having 1% of null values. We illustrate next some of the commands generating tables.

The creation of a table `tbl_float` containing random `float` values in the range [0,100] with 1% of null values is given next.

```
CREATE TABLE tbl_float AS
/* Add perc NULL values */
SELECT k, NULL AS f FROM generate_series(1, perc) AS k UNION
SELECT k, random_float(0, 100) FROM generate_series(perc+1, size) AS k;
```

The creation of a table `tbl_tbox` containing random `tbox` values where the bounds for values are in the range [0,100] and the bounds for timestamps are in the range [2001-01-01, 2001-12-31] is given next.

```
CREATE TABLE tbl_tbox AS
/* Add perc NULL values */
SELECT k, NULL AS b FROM generate_series(1, perc) AS k UNION
SELECT k, random_tbox(0, 100, '2001-01-01', '2001-12-31', 10, 10)
FROM generate_series(perc+1, size) AS k;
```

The creation of a table `tbl_floatspan` containing random `floatspan` values where the bounds for values are in the range [0,100] and the maximum difference between the lower and the upper bounds is 10 is given next.

```
CREATE TABLE tbl_floatspan AS
/* Add perc NULL values */
SELECT k, NULL AS f FROM generate_series(1, perc) AS k UNION
SELECT k, random_floatspan(0, 100, 10) FROM generate_series(perc+1, size) AS k;
```

The creation of a table `tbl_tstzset` containing random `tstzset` values having between 5 and 10 timestamps where the timestamps are in the range [2001-01-01, 2001-12-31] and the maximum interval between consecutive timestamps is 10 minutes is given next.

```
CREATE TABLE tbl_tstzset AS
/* Add perc NULL values */
SELECT k, NULL AS ts FROM generate_series(1, perc) AS k UNION
SELECT k, random_tstzset('2001-01-01', '2001-12-31', 10, 5, 10)
FROM generate_series(perc+1, size) AS k;
```

The creation of a table `tbl_tstzspan` containing random `tstzspan` values where the timestamps are in the range [2001-01-01, 2001-12-31] and the maximum difference between the lower and the upper bounds is 10 minutes is given next.

```
CREATE TABLE tbl_tstzspan AS
/* Add perc NULL values */
SELECT k, NULL AS p FROM generate_series(1, perc) AS k UNION
SELECT k,
  random_tstzspan('2001-01-01', '2001-12-31', 10) FROM generate_series(perc+1, size) AS k;
```

The creation of a table `tbl_geom_point` containing random `geometry` 2D point values, where the x and y coordinates are in the range [0, 100] and in SRID 3812 is given next.

```
CREATE TABLE tbl_geom_point AS
SELECT 1 AS k, geometry 'SRID=3812;point empty' AS g UNION
SELECT k, random_geom_point(0, 100, 0, 100, 3812) FROM generate_series(2, size) k;
```

Notice that the table contains an empty point value. If the SRID is not given it is set by default to 0.

The creation of a table `tbl_geog_point3D` containing random `geography` 3D point values, where the x, y, and z coordinates are, respectively, in the ranges [-10, 32], [35, 72], and [0, 1000] and in SRID 7844 is given next.

```
CREATE TABLE tbl_geog_point3D AS
SELECT 1 AS k, geography 'SRID=7844;pointZ empty' AS g UNION
SELECT k,
  random_geog_point3D(-10, 32, 35, 72, 0, 1000, 7844) FROM generate_series(2, size) k;
```

Notice that latitude and longitude values are chosen to approximately cover continental Europe. If the SRID is not given it is set by default to 4326.

The creation of a table `tbl_geom_linestring` containing random `geometry` 2D linestring values having between 5 and 10 vertices, where the x and y coordinates are in the range [0, 100] and in SRID 3812 and the maximum difference between consecutive coordinate values is 10 units in the underlying SRID is given next.

```
CREATE TABLE tbl_geom_linestring AS
SELECT 1 AS k, geometry 'linestring empty' AS g UNION
SELECT k,
  random_geom_linestring(0, 100, 0, 100, 10, 5, 10, 3812) FROM generate_series(2, size) k;
```

The creation of a table `tbl_geom_linestring` containing random `geometry` 2D linestring values having between 5 and 10 vertices, where the x and y coordinates are in the range [0, 100] and the maximum difference between consecutive coordinate values is 10 units in the underlying SRID is given next.

```
CREATE TABLE tbl_geom_linestring AS
SELECT 1 AS k, geometry 'linestring empty' AS g UNION
SELECT k,
  random_geom_linestring(0, 100, 0, 100, 10, 5, 10) FROM generate_series(2, size) k;
```

The creation of a table `tbl_geom_polygon3D` containing random `geometry` 3D polygon values without holes, having between 5 and 10 vertices, where the x, y, and z coordinates are in the range [0, 100] and the maximum difference between consecutive coordinate values is 10 units in the underlying SRID is given next.

```
CREATE TABLE tbl_geom_polygon3D AS
SELECT 1 AS k, geometry 'polygon Z empty' AS g UNION
SELECT k, random_geom_polygon3D(0, 100, 0, 100, 0, 100, 10, 5, 10)
FROM generate_series(2, size) k;
```

The creation of a table `tbl_geom_multipoint` containing random `geometry` 2D multipoint values having between 5 and 10 points, where the x and y coordinates are in the range [0, 100] and the maximum difference between consecutive coordinate values is 10 units in the underlying SRID is given next.

```
CREATE TABLE tbl_geom_multipoint AS
SELECT 1 AS k, geometry 'multipoint empty' AS g UNION
SELECT k,
  random_geom_multipoint(0, 100, 0, 100, 10, 5, 10) FROM generate_series(2, size) k;
```

The creation of a table `tbl_geog_multilinestring` containing random `geography` 2D multilinestring values having between 5 and 10 linestrings, each one having between 5 and 10 vertices, where the x and y coordinates are, respectively, in the ranges [-10, 32] and [35, 72], and the maximum difference between consecutive coordinate values is 10 is given next.

```
CREATE TABLE tbl_geog_multilinestring AS
SELECT 1 AS k, geography 'multilinestring empty' AS g UNION
SELECT k, random_geog_multilinestring(-10, 32, 35, 72, 10, 5, 10, 5, 10)
FROM generate_series(2, size) k;
```

The creation of a table `tbl_geometry3D` containing random `geometry` 3D values of various types is given next. This function requires that the tables for the various geometry types have been created previously.

```
CREATE TABLE tbl_geometry3D (k serial PRIMARY KEY, g geometry);
INSERT INTO tbl_geometry3D(g)
(SELECT g FROM tbl_geom_point3D ORDER BY k LIMIT (size * 0.1)) UNION ALL
(SELECT g FROM tbl_geom_linestring3D ORDER BY k LIMIT (size * 0.1)) UNION ALL
(SELECT g FROM tbl_geom_polygon3D ORDER BY k LIMIT (size * 0.2)) UNION ALL
(SELECT g FROM tbl_geom_multipoint3D ORDER BY k LIMIT (size * 0.2)) UNION ALL
(SELECT g FROM tbl_geom_multilinestring3D ORDER BY k LIMIT (size * 0.2)) UNION ALL
(SELECT g FROM tbl_geom_multipolygon3D ORDER BY k LIMIT (size * 0.2));
```

The creation of a table `tbl_tbool_inst` containing random `tbool` values of instant subtype where the timestamps are in the range [2001-01-01, 2001-12-31] is given next.

```
CREATE TABLE tbl_tbool_inst AS
/* Add perc NULL values */
SELECT k, NULL AS inst FROM generate_series(1, perc) AS k UNION
SELECT k,
  random_tbool_inst('2001-01-01', '2001-12-31') FROM generate_series(perc+1, size) k;
/* Add perc duplicates */
UPDATE tbl_tbool_inst t1
SET inst = (SELECT inst FROM tbl_tbool_inst t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc rows with the same timestamp */
UPDATE tbl_tbool_inst t1
SET inst = (SELECT tboolInst(random_bool(), getTimestamp(inst))
  FROM tbl_tbool_inst t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
```

As can be seen above, the table has a percentage of null values, of duplicates, and of rows with the same timestamp.

The creation of a table `tbl_tint_discseq` containing random `tint` values of sequence subtype and discrete interpolation having between 5 and 10 timestamps where the integer values are in the range [0, 100], the timestamps are in the range [2001-01-01, 2001-12-31], the maximum difference between two consecutive values is 10, and the maximum interval between two consecutive instants is 10 minutes is given next.

```
CREATE TABLE tbl_tint_discseq AS
/* Add perc NULL values */
SELECT k, NULL AS ti FROM generate_series(1, perc) AS k UNION
SELECT k, random_tint_discseq(0, 100, '2001-01-01', '2001-12-31', 10, 10, 5, 10) AS ti
FROM generate_series(perc+1, size) k;
/* Add perc duplicates */
UPDATE tbl_tint_discseq t1
SET ti = (SELECT ti FROM tbl_tint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc rows with the same timestamp */
UPDATE tbl_tint_discseq t1
SET ti = (SELECT ti + random_int(1, 2) FROM tbl_tint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc rows that meet */
UPDATE tbl_tint_discseq t1
SET ti = (SELECT shift(ti, endTimestamp(ti)-startTimestamp(ti))
  FROM tbl_tint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE t1.k in (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);
/* Add perc rows that overlap */
UPDATE tbl_tint_discseq t1
SET ti = (SELECT shift(ti, date_trunc('minute',(endTimestamp(ti)-startTimestamp(ti))/2))
  FROM tbl_tint_discseq t2 WHERE t2.k = t1.k+2)
WHERE t1.k in (SELECT i FROM generate_series(1 + 8*perc, 9*perc) i);
```

As can be seen above, the table has a percentage of null values, of duplicates, of rows with the same timestamp, of rows that meet, and of rows that overlap.

The creation of a table `tbl_tfloat_seq` containing random `tfloat` values of sequence subtype having between 5 and 10 timestamps where the `float` values are in the range [0, 100], the timestamps are in the range [2001-01-01, 2001-12-31], the maximum difference between two consecutive values is 10, and the maximum interval between two consecutive instants is 10 minutes is given next.

```
CREATE TABLE tbl_tfloat_seq AS
/* Add perc NULL values */
SELECT k, NULL AS seq FROM generate_series(1, perc) AS k UNION
SELECT k, random_tfloat_seq(0, 100, '2001-01-01', '2001-12-31', 10, 10, 5, 10) AS seq
FROM generate_series(perc+1, size) k;
/* Add perc duplicates */
UPDATE tbl_tfloat_seq t1
SET seq = (SELECT seq FROM tbl_tfloat_seq t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc tuples with the same timestamp */
UPDATE tbl_tfloat_seq t1
SET seq = (SELECT seq + random_int(1, 2) FROM tbl_tfloat_seq t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc tuples that meet */
UPDATE tbl_tfloat_seq t1
SET seq = (SELECT shift(seq, timeSpan(seq)) FROM tbl_tfloat_seq t2 WHERE t2.k = t1.k+perc)
WHERE t1.k in (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);
/* Add perc tuples that overlap */
UPDATE tbl_tfloat_seq t1
SET seq = (SELECT shift(seq, date_trunc('minute',timeSpan(seq)/2))
  FROM tbl_tfloat_seq t2 WHERE t2.k = t1.k+perc)
WHERE t1.k in (SELECT i FROM generate_series(1 + 8*perc, 9*perc) i);
```

The creation of a table `tbl_ttext_seqset` containing random `ttext` values of sequence set subtype having between 5 and 10 sequences, each one having between 5 and 10 timestamps, where the text values have at most 10 characters, the timestamps are in the range [2001-01-01, 2001-12-31], and the maximum interval between two consecutive instants is 10 minutes is given next.

```
CREATE TABLE tbl_ttext_seqset AS
/* Add perc NULL values */
SELECT k, NULL AS ts FROM generate_series(1, perc) AS k UNION
SELECT k, random_ttext_seqset('2001-01-01', '2001-12-31', 10, 10, 5, 10, 5, 10) AS ts
FROM generate_series(perc+1, size) AS k;
/* Add perc duplicates */
UPDATE tbl_ttext_seqset t1
SET ts = (SELECT ts FROM tbl_ttext_seqset t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc tuples with the same timestamp */
UPDATE tbl_ttext_seqset t1
SET ts = (SELECT ts || text 'A' FROM tbl_ttext_seqset t2 WHERE t2.k = t1.k+perc)
WHERE k in (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc tuples that meet */
UPDATE tbl_ttext_seqset t1
SET ts = (SELECT shift(ts, timeSpan(ts)) FROM tbl_ttext_seqset t2 WHERE t2.k = t1.k+perc)
WHERE t1.k in (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);
/* Add perc tuples that overlap */
UPDATE tbl_ttext_seqset t1
SET ts = (SELECT shift(ts, date_trunc('minute', timeSpan(ts)/2))
  FROM tbl_ttext_seqset t2 WHERE t2.k = t1.k+perc)
WHERE t1.k in (SELECT i FROM generate_series(1 + 8*perc, 9*perc) i);
```

The creation of a table `tbl_tgeompoint_discseq` containing random `tgeompoint` 2D values of sequence subtype and discrete interpolation having between 5 and 10 instants, where the x and y coordinates are in the range [0, 100] and in SRID 3812, the timestamps are in the range [2001-01-01, 2001-12-31], the maximum difference between successive coordinates is at most 10 units in the underlying SRID, and the maximum interval between two consecutive instants is 10 minutes is given next.

```
CREATE TABLE tbl_tgeompoint_discseq AS
SELECT k, random_tgeompoint_discseq(0, 100, 0, 100, '2001-01-01', '2001-12-31',
  10, 10, 5, 10, 3812) AS ti FROM generate_series(1, size) k;
/* Add perc duplicates */
UPDATE tbl_tgeompoint_discseq t1
SET ti = (SELECT ti FROM tbl_tgeompoint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1, perc) i);
/* Add perc tuples with the same timestamp */
UPDATE tbl_tgeompoint_discseq t1
SET ti = (SELECT round(ti,6) FROM tbl_tgeompoint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc tuples that meet */
UPDATE tbl_tgeompoint_discseq t1
SET ti = (SELECT shift(ti, endTimestamp(ti)-startTimestamp(ti))
  FROM tbl_tgeompoint_discseq t2 WHERE t2.k = t1.k+perc)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc tuples that overlap */
UPDATE tbl_tgeompoint_discseq t1
SET ti = (SELECT shift(ti, date_trunc('minute',(endTimestamp(ti)-startTimestamp(ti))/2))
  FROM tbl_tgeompoint_discseq t2 WHERE t2.k = t1.k+2)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);
```

Finally, the creation of a table `tbl_tgeompoint3D_seqset` containing random `tgeompoint` 3D values of sequence set subtype having between 5 and 10 sequences, each one having between 5 and 10 timestamps, where the x, y, and z coordinates are in the range [0, 100] and in SRID 3812, the timestamps are in the range [2001-01-01, 2001-12-31], the maximum difference between successive coordinates is at most 10 units in the underlying SRID, and the maximum interval between two consecutive instants is 10 minutes is given next.

```
DROP TABLE IF EXISTS tbl_tgeompoint3D_seqset;
CREATE TABLE tbl_tgeompoint3D_seqset AS
SELECT k, random_tgeompoint3D_seqset(0, 100, 0, 100, 0, 100, '2001-01-01', '2001-12-31',
  10, 10, 5, 10, 5, 10, 3812) AS ts FROM generate_series(1, size) AS k;
/* Add perc duplicates */
UPDATE tbl_tgeompoint3D_seqset t1
SET ts = (SELECT ts FROM tbl_tgeompoint3D_seqset t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1, perc) i);
/* Add perc tuples with the same timestamp */
UPDATE tbl_tgeompoint3D_seqset t1
SET ts = (SELECT round(ts,3) FROM tbl_tgeompoint3D_seqset t2 WHERE t2.k = t1.k+perc)
WHERE k IN (SELECT i FROM generate_series(1 + 2*perc, 3*perc) i);
/* Add perc tuples that meet */
UPDATE tbl_tgeompoint3D_seqset t1
SET ts = (SELECT shift(ts,
  timeSpan(ts)) FROM tbl_tgeompoint3D_seqset t2 WHERE t2.k = t1.k+perc)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 4*perc, 5*perc) i);
/* Add perc tuples that overlap */
UPDATE tbl_tgeompoint3D_seqset t1
SET ts = (SELECT shift(ts, date_trunc('minute', timeSpan(ts)/2))
  FROM tbl_tgeompoint3D_seqset t2 WHERE t2.k = t1.k+perc)
WHERE t1.k IN (SELECT i FROM generate_series(1 + 6*perc, 7*perc) i);
```

B.6 Generator for Temporal Network Point Types

- `random_fraction`: Generate a random fraction in the range [0,1]
- `random_npoint`: Generate a random network point
- `random_nsegment`: Generate a random network segment
- `random_tnpoint_inst`: Generate a random temporal network point of instant subtype
- `random_tnpoint_discseq`: Generate a random temporal network point of sequence subtype and discrete interpolation
- `random_tnpoint_seq`: Generate a random temporal network point of sequence subtype and linear or step interpolation
- `random_tnpoint_seqset`: Generate a random temporal network point of sequence set subtype

The file `/datagen/npoint/create_test_tables_tnpoint.sql` provide usage examples for the functions generating random values listed above.

B.7 Generator for Temporal JSONB Types

- `random_jsonb`: Generate a random JSONB document
- `random_jsonb_array`: Generate a random array of JSONB documents
- `random_jsonb_set`: Generate a random set of JSONB documents
- `random_tjsonb_inst`: Generate a random temporal JSONB of instant subtype
- `random_tjsonb_discseq`: Generate a random temporal JSONB of sequence subtype and discrete interpolation
- `random_tjsonb_seq`: Generate a random temporal JSONB of sequence subtype and step interpolation
- `random_tjsonb_seqset`: Generate a random temporal JSONB of sequence set subtype

The file `/datagen/json/create_test_tables_tjsonb.sql` provides usage examples for the functions generating random values listed above.